CIS 6270
Discrete Generative Models
Lecture Notes

Pranam Chatterjee

Fall 2026



Welcome

Hi everyone! Welcome to CIS 6270, where we will learn the core math and formulations of generative modeling. Right now, an image of Gaussian noise and a string of mask tokens may seem like unpromising first drafts to get to final outputs. However, by the end of this book, I hope both will feel like sensible places to start. We will learn how to turn simple random samples into images, language, and molecular sequences, then guide generation toward properties we care about.

We will work through the mathematics together, from the change in one sample to the evolution of a whole distribution. The derivations, numerical examples, and code are here so that you can follow each step and try it yourself. In the end, I hope you'll share in the excitement of getting a useful and practical model out of math you thought you'd never need! It's my job to get you there, so let's get started.

Contents

Welcome **3**

1 Introduction and Mathematical Foundations **11**
1.1 Generative Models . . . 11
1.2 State Spaces . . . 13
1.3 Discrete Random States . . . 14
1.4 Densities for Continuous States . . . 15
1.5 Expectation . . . 16
1.6 Variance and Standard Deviation . . . 20
1.7 Covariance, Correlation and the Covariance Matrix . . . 22
1.8 Gaussian Noise and the Noising Equation . . . 25
1.9 Joint, Marginal and Conditional Distributions . . . 26
1.10 Bayes' Rule and the Law of Total Probability . . . 28
1.11 Independence and the Chain Rule . . . 30
1.12 Probability Vectors and the Simplex . . . 32
1.13 Logits and the Softmax . . . 35
1.14 Vectors, Matrices and Linear Maps . . . 37
1.15 Probability Vectors Under a Transition Matrix . . . 41
1.16 Eigenvalues and Eigenvectors . . . 42
1.17 Stationary Distributions . . . 46
1.18 Linear Dynamics in Continuous Time . . . 47
1.19 Maps . . . 50
1.20 Random Variables Under a Map . . . 54
1.21 Time-Dependent Maps . . . 58
1.22 Derivatives, Partial Derivatives and the Jacobian . . . 60
1.23 Jacobian Determinants and Local Volume . . . 64
1.24 Change of Variables . . . 66
1.25 Composed Maps and the Log-Density Form . . . 69
1.26 Local Invertibility and Inverse Branches . . . 71
1.27 Scalar Fields and the Gradient . . . 72
1.28 Vector Fields . . . 74
1.29 Divergence . . . 75
1.30 The Continuity Equation . . . 76
1.31 Score Functions . . . 78
1.32 The Laplacian and Diffusion . . . 79

1.33 Information, Entropy and Cross-Entropy . . . 81
1.34 Kullback-Leibler Divergence . . . 83
1.35 Maximum Likelihood as KL Minimization . . . 85
1.36 Optimization Without Constraints . . . 86
1.37 Equality Constraints and Lagrange Multipliers . . . 87
1.38 Inequality Constraints and the KKT Conditions . . . 90
1.39 Generative Process Construction . . . 92

2 Flow Matching **95**
2.1 Time-indexed maps . . . 95
2.2 Trajectories and the velocity field . . . 96
2.3 Velocity fields in several dimensions . . . 99
2.4 The flow map . . . 100
2.5 Euler's method . . . 101
2.6 A random starting point . . . 103
2.7 Probability flux and the continuity equation . . . 104
2.8 Density along a moving trajectory . . . 109
2.9 Continuous normalizing flows . . . 111
2.10 Learning the velocity field . . . 112
2.11 Coupling the prior and the data . . . 113
2.12 The conditional path and its velocity . . . 113
2.13 The marginal velocity . . . 115
2.14 The flow-matching objective . . . 117
2.15 The conditional flow-matching equivalence . . . 119
2.16 The continuity equation for one conditional path . . . 120
2.17 A complete Gaussian translation . . . 122
2.18 Other interpolants . . . 122
2.19 Training and sampling . . . 125
2.20 Flow matching on MNIST . . . 126
2.21 Path choice and stochastic sampling . . . 133

3 Score Matching and Diffusion Models **135**
3.1 Stochastic generative paths . . . 135
3.2 Drift and Brownian increments . . . 136
3.3 Brownian increment scaling . . . 136
3.4 The Euler-Maruyama step . . . 138
3.5 Noise accumulation . . . 139
3.6 A restoring drift . . . 139
3.7 Changes in smooth measurements . . . 141
3.8 Quadratic variation . . . 141
3.9 Itô's lemma . . . 142
3.10 The Fokker-Planck equation . . . 144
3.11 Variance-preserving diffusion . . . 146
3.12 The forward corruption in code . . . 150
3.13 Forward corruption and the marginal density . . . 153
3.14 The score of a density . . . 153
3.15 Score matching . . . 155
3.16 Denoising score matching . . . 158

3.17 Noise prediction and the clean estimate . . . 163
3.18 The corruption, the density, and the score . . . 165
3.19 Training a denoiser . . . 166
3.20 Noise-prediction network . . . 170
3.21 Latent diffusion on protein language model embeddings . . . 172
3.22 Langevin dynamics across noise levels . . . 174
3.23 The reverse-time SDE . . . 175
3.24 DDPM discretization . . . 177
3.25 The variational objective . . . 181
3.26 Drawing the reverse step . . . 182
3.27 The shrinking-step limit . . . 183
3.28 Implementing DDPM . . . 183
3.29 AMP-Diffusion end to end . . . 190
3.30 Continuous limit of the DDPM step . . . 195
3.31 Stochastic and deterministic density evolution . . . 196
3.32 The probability-flow ODE . . . 197
3.33 Probability flow for Brownian marginals . . . 198
3.34 Probability flow for the variance-preserving path . . . 200
3.35 Deterministic backward-step example . . . 202
3.36 The density along a deterministic trajectory . . . 204
3.37 Conditioning the score . . . 205
3.38 The guidance scale and the tilted target . . . 206
3.39 Reward guidance . . . 208
3.40 Guidance inside the generative dynamics . . . 208
3.41 Guided backward-step example . . . 210
3.42 Guiding a flow-matching model . . . 210
3.43 Classifier-free guidance . . . 212
3.44 Multi-objective guidance . . . 213
3.45 Constraints and scheduled guidance . . . 215
3.46 Guided generation in latent space . . . 216
3.47 Discrete state spaces . . . 221

4 Discrete Diffusion **223**
4.1 Finite-vocabulary diffusion . . . 223
4.2 Vocabulary and sequence-space size . . . 224
4.3 Two ways to corrupt a sequence . . . 224
4.4 Continuous time for a discrete state . . . 225
4.5 Masked diffusion on one token . . . 225
4.6 Reversing one step of the corruption . . . 228
4.7 The denoiser and the learned reverse step . . . 229
4.8 Reverse-step example . . . 230
4.9 Masked DNA generation . . . 231
4.10 Comparing two predictions with KL divergence . . . 232
4.11 Pathwise variational bound . . . 233
4.12 Constraints on the denoiser . . . 235
4.13 The reverse divergence at one position . . . 236
4.14 The masked diffusion loss . . . 237
4.15 Continuous-time transitions . . . 239

4.16 Rate-matrix constraints . . . 241
4.17 Waiting times and destinations . . . 242
4.18 The master equation . . . 245
4.19 The matrix exponential . . . 248
4.20 Rate matrices with a reference distribution . . . 250
4.21 Eigenvalues and the stationary distribution . . . 255
4.22 Detailed balance . . . 256
4.23 Reverse jump processes . . . 258
4.24 The posterior average of conditional reverse rates . . . 261
4.25 Conditioning a jump process: Doob's h-transform . . . 265
4.26 A loss for the reverse rates . . . 269
4.27 Sampling with a denoiser . . . 273
4.28 Discrete diffusion requirements . . . 275
4.29 Discrete diffusion variants . . . 276
4.30 Masked diffusion with $\pi = \delta_m$ . . . 276
4.31 D3PM in discrete time . . . 281
4.32 Uniform replacement rates . . . 284
4.33 Predicting the ratios directly . . . 286
4.34 Blocks between diffusion and autoregression . . . 287
4.35 Per-token schedules . . . 289
4.36 Simplex diffusion . . . 290
4.37 DNA diffusion implementation . . . 293
4.38 Reveal-order planning . . . 295
4.39 Planner-weighted training . . . 296
4.40 Guiding generation by search . . . 298
4.41 Distilling search results . . . 299
4.42 Variable-length diffusion . . . 300
4.43 Model comparison . . . 301
4.44 Discrete diffusion construction . . . 302

5 Discrete Flow Matching **305**
5.1 Choosing the probability path . . . 305
5.2 Probability velocities . . . 306
5.3 The master equation in velocity form . . . 307
5.4 Position-wise sequence dynamics . . . 309
5.5 Source-data probability paths . . . 311
5.6 The velocity of the mixture path . . . 314
5.7 Posterior-averaged velocities . . . 316
5.8 The Euler sampling step . . . 319
5.9 Training the denoiser with cross-entropy . . . 321
5.10 Two sources and two samplers . . . 324
5.11 Discrete flow-matching implementation . . . 326
5.12 Discrete Flow Matching and the choice of schedule . . . 328
5.13 Zero-flux velocities . . . 329
5.14 The corrector . . . 331
5.15 Joint structure and sequence generation . . . 334
5.16 Kinetic-optimal paths . . . 336
5.17 Edit Flows with variable length . . . 342

5.18 Discrete flow variants . . . 346
5.19 Continuous token relaxations . . . 347
5.20 Dirichlet Flow Matching . . . 350
5.21 Fisher-Flow: the simplex as a sphere . . . 357
5.22 Gumbel-Softmax Flow Matching . . . 364
5.23 Three geometries . . . 369
5.24 The target distribution . . . 370
5.25 Guidance as a tilt of the rates . . . 370
5.26 Single-position jumps and the reachable set . . . 372
5.27 Single-base guidance example . . . 372
5.28 The guidance strength . . . 374
5.29 A first-order expansion of the log ratio . . . 375
5.30 Multi-objective guidance . . . 376
5.31 MOG-DFM: rank, direction and a cone . . . 376
5.32 pCoMole: a reward in place of a likelihood . . . 379
5.33 The guided sampler . . . 381
5.34 The guidance term . . . 382
5.35 Sampling step counts . . . 382
5.36 The pairing of source and data . . . 382
5.37 Rectification . . . 383
5.38 Rectification for a set of tokens . . . 383
5.39 Total correlation on two bases . . . 384
5.40 ReDi . . . 385
5.41 AReUReDi . . . 386
5.42 State spaces, paths, and velocities . . . 389
5.43 Summary . . . 390

6 Flow Maps **393**
6.1 Sampling cost . . . 393
6.2 Endpoint prediction . . . 394
6.3 The flow map . . . 395
6.4 The three axioms . . . 395
6.5 Time-independent velocity . . . 397
6.6 The inverse map . . . 398
6.7 The Jacobian of the map . . . 398
6.8 Liouville's formula . . . 399
6.9 Gaussian flow example . . . 400
6.10 Transporting a density along the flow . . . 401
6.11 The generator of the flow . . . 402
6.12 Flow-map computation and verification . . . 404
6.13 The average velocity . . . 405
6.14 The start-time and end-time domain . . . 406
6.15 Differentiating the cocycle . . . 407
6.16 Three residuals and the diagonal anchor . . . 408
6.17 The zero-velocity solution . . . 410
6.18 MeanFlow . . . 411
6.19 Identity-based losses . . . 413
6.20 Shortcut and consistency models . . . 414

6.21 Endpoint error under each residual . . . 417
6.22 Flow Map Matching and self-distillation . . . 420
6.23 The three residuals in code . . . 421
6.24 Deterministic maps and atoms . . . 423
6.25 Sequence-distribution states . . . 424
6.26 The two-time denoiser . . . 425
6.27 Flow Map Language Models . . . 430
6.28 Categorical Flow Maps . . . 434
6.29 Discrete Flow Maps . . . 438
6.30 Discrete flow-map comparison . . . 441
6.31 Flow maps of a stochastic differential equation . . . 442
6.32 Expanding flow maps . . . 448
6.33 Guidance and the posterior over endpoints . . . 451
6.34 Meta flow maps . . . 456
6.35 Map inputs and training identities . . . 458
6.36 Summary . . . 459

7 Optimal Transport and Schrödinger Bridges 461
7.1 Choosing the pairing . . . 461
7.2 Monge's formulation . . . 463
7.3 Kantorovich's relaxation . . . 465
7.4 Prices and the dual problem . . . 467
7.5 Distance-based costs . . . 470
7.6 Gaussian transport maps . . . 472
7.7 Dynamic optimal transport . . . 474
7.8 Optimal transport inside flow matching . . . 478
7.9 Entropic transport . . . 479
7.10 Sinkhorn's algorithm . . . 482
7.11 Endpoint couplings and reference paths . . . 486
7.12 The reference process and the bridge problem . . . 487
7.13 The chain rule for relative entropy . . . 489
7.14 The static problem and the two potentials . . . 491
7.15 Markov bridge construction . . . 494
7.16 A diffusion reference . . . 497
7.17 The bridge as a stochastic control problem . . . 500
7.18 The Brownian bridge . . . 503
7.19 Learning the bridge from samples . . . 505
7.20 Bridge-learning methods . . . 512
7.21 Finite-state bridges . . . 513
7.22 Rates reweighted by an h-transform . . . 515
7.23 Two-state backward propagation from $t = 1/2$ . . . 516
7.24 Discrete Diffusion Schrödinger Bridge Matching . . . 519
7.25 Categorical Schrödinger Bridge Matching . . . 524
7.26 Branched Schrödinger Bridge Matching . . . 527
7.27 Entangled Schrödinger Bridge Matching . . . 531
7.28 Given quantities and learned quantities . . . 535
7.29 Generative model summary . . . 535

Chapter 1

Introduction and Mathematical Foundations

1.1 Generative Models

A generative model learns to transform a simple distribution into the data distribution. We can sample directly from the source distribution p_0, such as Gaussian noise or a sequence of mask tokens. For the data distribution p_{data}, we have only a dataset of examples, such as natural images, protein sequences, or promoters that drive expression in one cell type and not another. We train the model to transform samples from the source into samples from this data distribution.

To generate a sample, we first draw from the source distribution

$$x_0 \sim p_0,$$

and we move it along a learned path of distributions p_t indexed by a time t, until the final state is distributed as the data:

$$x_T \sim p_{\text{data}}.$$

"We draw the initial state x_0 from the source distribution p_0 and evolve it through the intermediate distributions p_t, from time $t = 0$ to the terminal time T, so that the final state x_T follows the data distribution p_{data}."

The intermediate distributions p_t describe the partially generated states between the source p_0 and the endpoint, where we want $p_T \approx p_{\text{data}}$ at $t = T$. Each intermediate state is a little closer to the data. We use $t = 0$ for the source end and $t = T$ for the data end. Setting $T = 1$ places t in the unit interval. The diffusion convention reverses this direction: corruption proceeds from clean data to noise.

A *continuous* state is a point in Euclidean space,

$$x \in \mathbb{R}^d,$$

where d is the number of coordinates. An RGB pixel is three real numbers. A molecular conformation is three coordinates per atom. We can add and scale points in $\mathbb{R}^d$, or move them by an arbitrarily small amount. We describe distributions over these points with densities.

A *discrete* state is a sequence of symbols,

$$x \in \mathcal{V}^L,$$

where $\mathcal{V}$ is a finite vocabulary and L is the sequence length. A DNA sequence of length L is a point in $\{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}^L$. A protein of length L is a point in a space built from twenty amino acids. There is no halfway point between `A` and `C` in this space. The state moves through combinations of symbols, and a step changes one symbol into another symbol.

1.1.1 The Discrete Setting

The change in mathematics comes from four properties of symbolic data.

- Much of the data we care about is a sequence of symbols. Genomes, transcripts, protein sequences, molecular graphs written as strings, and natural language are all finite alphabets arranged in order.
- Outputs have to satisfy discrete constraints. A coding sequence must have a length divisible by three; a molecule must satisfy valences; a peptide must be made of amino acids that exist. A continuous model can produce a point that rounds to something invalid, and the constraint is lost at the rounding step.
- Small edits produce large semantic changes. One base substitution inside a codon can change the amino acid, truncate the protein, or do nothing at all. The three outcomes are neighboring points in sequence space.
- The stochastic processes themselves have to be rebuilt. A Gaussian step moves a point in $\mathbb{R}^d$ a short distance in a random direction. On $\mathcal{V}^L$ there is no short distance. A step either replaces the symbol or leaves it unchanged. We describe these replacements with a jump process. Diffusions have no counterpart on this space.

We work in discrete space with continuous time. The state belongs to $\mathcal{V}^L$ and changes by discrete jumps. Because jump times follow a continuous clock, the distribution over states changes smoothly with t:

$$p_0(x) \longrightarrow p_t(x) \longrightarrow p_T(x) \approx p_{\text{data}}(x).$$

"The distribution over states changes from the source distribution $p_0(x)$, through the intermediate distribution $p_t(x)$ at time t, to the terminal distribution $p_T(x)$, which approximates the data distribution $p_{\text{data}}(x)$ at time T."

To specify this process, we choose a state space, jump frequencies, and jump destinations. We then train a network to predict the quantities needed to move p_0 to p_{data} through these jumps.

1.2 State Spaces

Before assigning probabilities, we specify the state space: the set of values that the random variable can take.

Suppose the data are DNA sequences of length L. Each position holds one token from the vocabulary

$$\mathcal{V} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}.$$

The full state space is the set of all length-L strings over that vocabulary:

$$\mathcal{X} = \mathcal{V}^L.$$

"The state space $\mathcal{X}$ contains every sequence of length L whose tokens belong to the vocabulary $\mathcal{V}$."

A state $x \in \mathcal{X}$ is one complete sequence, written out position by position, with no part of it left unspecified. The sequence `ACCT` is a single state in $\mathcal{V}^4$. Partial sequences, probability distributions over sequences, and sequences of a different length are all outside $\mathcal{X}$.

Each of the L positions is filled independently from $K = |\mathcal{V}|$ tokens, so the size of the state space is

$$|\mathcal{X}| = K^L.$$

For DNA, $K = 4$ and

$$|\mathcal{X}| = 4^L.$$

For a protein with twenty possible amino acids at every position, $K = 20$ and

$$|\mathcal{X}| = 20^L.$$

A DNA sequence of length 100 has $4^{100} \approx 1.61 \times 10^{60}$ possible values, and a protein of length 100 has $20^{100} \approx 1.27 \times 10^{130}$. The number of atoms in the observable universe is estimated near 10^{80}. We cannot enumerate a state space of this size. We therefore need ways to normalize distributions, calculate marginals, and draw samples without listing every possible state.

We write X for a random state, p_X for its distribution, and x for a realized value:

$$X \sim p_X, \qquad x \in \mathcal{X}.$$

"The random state X follows the distribution p_X, and the lowercase state x denotes one realized value in the state space $\mathcal{X}$."

For example, X may be a random DNA sequence with the realization $x = \texttt{ACCT}$. We keep the capital letter for the random state and the lowercase letter for a particular value to distinguish a random variable from its realization.

1.3 Discrete Random States

For a discrete random state, the distribution assigns a probability mass to every element of the state space:

$$p_X(x) = \mathbb{P}(X = x).$$

"The probability mass $p_X(x)$ is the probability that the random state X equals the particular state x."

A valid probability distribution satisfies two requirements. The first is nonnegativity,

$$p_X(x) \geq 0 \qquad \text{for every } x \in \mathcal{X},$$

and the second is normalization,

$$\sum_{x \in \mathcal{X}} p_X(x) = 1.$$

"The sum of the probability masses $p_X(x)$ over all states x in the state space $\mathcal{X}$ equals one, so the total probability mass is one."

An array defines a distribution over $\mathcal{X}$ exactly when its entries are nonnegative and sum to one.

1.3.1 A Distribution Over Length-Two Sequences

Let's take sequences of length two over the two-letter vocabulary $\mathcal{V} = \{\texttt{A}, \texttt{C}\}$, so that $L = 2$ and the state space has four elements:

$$\mathcal{X} = \{\texttt{AA}, \texttt{AC}, \texttt{CA}, \texttt{CC}\}.$$

Suppose the distribution is

$$p_X(\texttt{AA}) = 0.4, \qquad p_X(\texttt{AC}) = 0.1, \qquad p_X(\texttt{CA}) = 0.2, \qquad p_X(\texttt{CC}) = 0.3.$$

Every probability is nonnegative. Checking normalization,

$$\sum_{x \in \mathcal{X}} p_X(x) = 0.4 + 0.1 + 0.2 + 0.3 = 1.0,$$

so this is a valid probability distribution over $\mathcal{X}$.

An *event* is a subset of the state space. Consider the event that the first token is `A`,

$$\mathcal{A} = \{x \in \mathcal{X} : x_1 = \texttt{A}\} = \{\texttt{AA}, \texttt{AC}\},$$

where x_1 denotes the token at the first position of x. The probability of an event is the sum of the probabilities of the states inside it:

$$\mathbb{P}(X \in \mathcal{A}) = \sum_{x \in \mathcal{A}} p_X(x).$$

"The probability that the random state X belongs to the event $\mathcal{A}$ is the sum of the probability masses $p_X(x)$ over all states x in that event."

For this example,

$$\mathbb{P}(X \in \mathcal{A}) = p_X(\texttt{AA}) + p_X(\texttt{AC}) = 0.4 + 0.1 = 0.5,$$

so the first token is `A` with probability 0.5. We calculate any discrete event probability by summing masses over a subset of states. In practice, that subset is often too large to enumerate.

We can place the same distribution inside the full DNA state space without changing any of the arithmetic. If we take $\mathcal{V} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$ and $L = 2$, the state space has $4^2 = 16$ elements, twelve of which have zero mass under the distribution above. The event $\{x_1 = \texttt{A}\}$ then contains four states, two of them with mass, and the sum is $0.4 + 0.1 + 0 + 0 = 0.5$ as before (Figure 1.1).

At $L = 3$, the DNA state space contains $4^3 = 64$ states. At $L = 10$, it contains $4^{10} = 1{,}048{,}576$. As the space grows, we work with functions evaluated at particular states and samples drawn from the distribution, without storing a table of every probability.

1.4 Densities for Continuous States

For a continuous state, an individual point generally has probability zero. We calculate the probability of a region by integrating its density:

$$\mathbb{P}(a \le X \le b) = \int_a^b p_X(x)\, dx.$$

"The probability that the continuous random variable X lies between the endpoints a and b equals the integral of its density $p_X(x)$ over that interval, which is the area under the density curve."

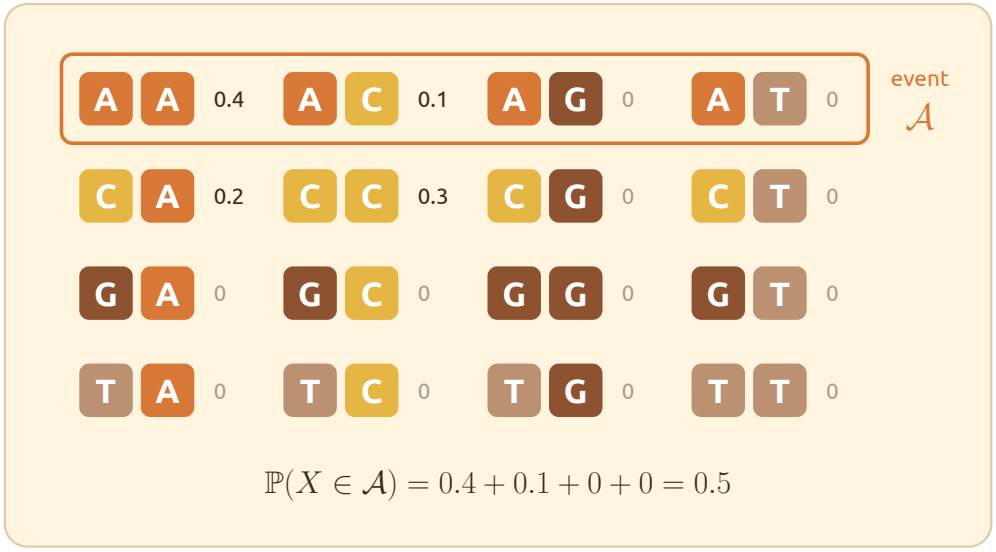


Figure 1.1: The sixteen states of $\mathcal{V}^2$ for $\mathcal{V} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$, each drawn as a pair of token tiles with its probability beside it. The distribution of Section 1.3.1 places mass 0.4, 0.1, 0.2 and 0.3 on `AA`, `AC`, `CA` and `CC`, and mass zero on the twelve states containing `G` or `T`. The outlined row is the event $\mathcal{A} = \{x : x_1 = \texttt{A}\}$, whose probability is the sum of the four masses it contains, $0.4 + 0.1 + 0 + 0 = 0.5$.

A valid density satisfies the same two requirements as a mass function, with the sum replaced by an integral:

$$p_X(x) \geq 0 \qquad \text{and} \qquad \int_{-\infty}^{\infty} p_X(x)\, dx = 1.$$

We calculate probability from the area under a density over a region. The density at a point has units of probability per unit length and can exceed 1 over a sufficiently narrow region. The uniform density on the interval $[0, 0.1]$ has the value $p_X(x) = 1/0.1 = 10$ on that interval, and the total area is still $10 \times 0.1 = 1$. The probability of landing in $[0, 0.02]$ is $10 \times 0.02 = 0.2$, within the allowed probability range. A Gaussian with standard deviation 0.1 has peak height $1/(\sqrt{2\pi} \times 0.1) \approx 3.989$ for the same reason.

A probability distribution, on either kind of state space, describes how one unit of probability mass is allocated. Discrete distributions allocate that mass directly to individual states. Continuous distributions allocate it to regions through a density and an integral. Biological sequence models operate on enormous discrete state spaces of the kind counted in Section 1.2, and continuous generative models operate in $\mathbb{R}^d$, where we calculate probabilities by integrating densities. The same formulas hold with $\sum$ for discrete states and $\int$ for continuous states.

We can also use a distribution to average a property of its states.

1.5 Expectation

We often need the average value of a property under a distribution. For generated sequences, we might calculate the average nucleotide count, length, binding score, or net charge. During training,

we calculate the expected loss.

Let X be a random state and let h be a numerical property of that state. For a DNA sequence, $h(x)$ might count the number of adenines; for a protein, it might evaluate net charge or predicted stability. The function is written

$$h : \mathcal{X} \to \mathbb{R},$$

so each possible state receives a real number. Expectation averages those numbers according to how likely the states are. For a discrete random state,

$$\mathbb{E}_{X \sim p_X}[h(X)] = \sum_{x \in \mathcal{X}} p_X(x)\, h(x).$$

"The expected property value $\mathbb{E}_{X \sim p_X}[h(X)]$ is the sum, over states x in the state space $\mathcal{X}$, of each probability $p_X(x)$ times the property value $h(x)$."

- $h(x)$ is the numerical property evaluated at the state x, in whatever units that property has.
- $p_X(x)$ is the probability mass at x, so it is dimensionless and the weights sum to one.
- $\mathbb{E}_{X \sim p_X}[\,\cdot\,]$ has the distribution in its subscript, because the same property has different averages under different distributions.

Expectation is a probability-weighted average, and states with higher probability count for more in that average.

1.5.1 The Expected Adenine Count

Let's continue with the distribution of Section 1.3.1 and define $h(x)$ as the number of `A` tokens in the sequence x:

$$h(\texttt{AA}) = 2, \qquad h(\texttt{AC}) = 1, \qquad h(\texttt{CA}) = 1, \qquad h(\texttt{CC}) = 0.$$

The expected number of `A` tokens is

$$\mathbb{E}[h(X)] = 0.4(2) + 0.1(1) + 0.2(1) + 0.3(0) = 0.8 + 0.1 + 0.2 + 0 = 1.1.$$

A single sequence cannot contain 1.1 adenines. The expectation is a property of the distribution, and no individual state has to realize it. For 1000 generated sequences, we expect about $1000 \times 1.1 = 1100$ adenines in total across the two positions.

When the random state is itself numerical, we recover the mean by choosing the identity property,

$$h(x) = x,$$

so that

$$\mathbb{E}[X] = \sum_x x\, p_X(x).$$

"The mean $\mathbb{E}[X]$ is the sum, over possible values x of the random variable X, of each value multiplied by its probability $p_X(x)$."

The mean is denoted $\mu_X = \mathbb{E}[X]$. For a continuous random variable the summation becomes integration,

$$\mathbb{E}[h(X)] = \int h(x)\, p_X(x)\, dx,$$

and the discrete and continuous definitions perform the same weighted averaging. We use a sum for discrete states and an integral for continuous ones.

1.5.2 Linearity

Expectation distributes across sums and constant coefficients:

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y],$$

for any constants $a, b \in \mathbb{R}$ and random variables X and Y with finite means. To derive this identity, use their joint distribution $p(x, y)$ in the definition:

$$\mathbb{E}[aX + bY] = \sum_x \sum_y p(x, y)\,(ax + by).$$

We can distribute $p(x, y)$ across the two terms:

$$\mathbb{E}[aX + bY] = \sum_x \sum_y \left[a\, x\, p(x, y) + b\, y\, p(x, y)\right].$$

Split the double sum into two double sums and pull the constants out:

$$\mathbb{E}[aX + bY] = a \sum_x \sum_y x\, p(x, y) + b \sum_x \sum_y y\, p(x, y).$$

The sum of $p(x, y)$ over y equals the marginal $p(x)$, and the sum over x equals $p(y)$, so we recognize both expectations:

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y].$$

Linearity requires no independence assumption. It holds even when the quantities being added are statistically dependent, because we did not factorize $p(x, y)$. When a loss is an expectation, we can use linearity to separate its terms or move a constant outside the expectation.

1.5.3 Monte Carlo Estimation

Summing over the complete data distribution is almost never possible, for the reason given in Section 1.2. Instead we hold a finite set of samples,

$$x^{(1)}, x^{(2)}, \ldots, x^{(N)} \overset{\text{iid}}{\sim} p_X,$$

and we estimate the expectation by the sample average:

$$\mathbb{E}[h(X)] \approx \frac{1}{N} \sum_{n=1}^{N} h\left(x^{(n)}\right).$$

"We approximate the expected property value $\mathbb{E}[h(X)]$ under the distribution p_X by averaging the values $h(x^{(n)})$ over N independent samples $x^{(n)}$, with equal weight $1/N$ for each sample."

In Monte Carlo estimation, we average over the available samples with equal weight per sample. The exact expectation weights each state by its probability; in the sample average, those probabilities determine how often each state appears.

If we draw $N = 10$ sequences from the distribution of Section 1.3.1, we might get `AA` five times, `AC` once, `CA` once and `CC` three times. The estimate is

$$\frac{1}{10}\left[5(2) + 1(1) + 1(1) + 3(0)\right] = \frac{12}{10} = 1.2,$$

against the exact value 1.1. The error of the estimator shrinks like $1/\sqrt{N}$. Here the property has variance $\mathrm{Var}(h(X)) = \mathbb{E}[h^2] - \mathbb{E}[h]^2 = 1.9 - 1.21 = 0.69$. The standard deviation of the ten-sample average is therefore $\sqrt{0.69/10} \approx 0.263$. The observed gap of 0.1 is well inside this standard deviation. If we raise N to 1000, that standard deviation falls to ≈ 0.0263, so a tenfold reduction in error requires a hundredfold increase in N.

1.5.4 Generative Training Objectives

A typical generative-model loss averages a per-example error:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}}[\ell_\theta(X)],$$

"The training loss $\mathcal{L}(\theta)$, for network parameters θ, averages the per-example loss $\ell_\theta(X)$ over examples X drawn from the data distribution p_{data}."

- θ collects the network parameters, so every quantity that depends on θ is a model prediction.

- $\ell_\theta(x)$ is the per-example loss at the single data point x, a real number.
- p_{data} is the data distribution, available only through samples.

We replace the exact expectation by a batch average,

$$\mathcal{L}(\theta) \approx \frac{1}{B}\sum_{b=1}^{B} \ell_\theta\left(x^{(b)}\right),$$

over a batch of B examples drawn from the dataset. The batch loss is a Monte Carlo estimate of the expectation under p_{data}. Its gradient estimates the gradient of that expectation. We choose ℓ_θ so that minimizing the loss estimates the quantity needed for sampling.

An expectation locates the center of a distribution. To describe its spread, we also need to measure how widely the random values fluctuate around that center.

1.6 Variance and Standard Deviation

Many distributions share the same center and place their probability mass quite differently around it. Consider two random variables, each taking two values with probability 1/2:

$$X \in \{-1, +1\}, \qquad Y \in \{-10, +10\}.$$

Both have mean zero,

$$\mathbb{E}[X] = \tfrac{1}{2}(-1) + \tfrac{1}{2}(+1) = 0, \qquad \mathbb{E}[Y] = \tfrac{1}{2}(-10) + \tfrac{1}{2}(+10) = 0,$$

although their values differ greatly in magnitude. The deviation of a realized value x from the mean is $x - \mu_X$. Averaging this deviation cannot measure spread because the positive and negative deviations cancel:

$$\mathbb{E}[X - \mu_X] = \mathbb{E}[X] - \mu_X = 0,$$

for every distribution with a finite mean. We avoid this cancellation by squaring each deviation before averaging. This average is the variance:

$$\operatorname{Var}(X) = \mathbb{E}\left[(X - \mu_X)^2\right].$$

"The variance $\operatorname{Var}(X)$ averages the squared deviation of the random variable X from its mean μ_X, measuring the size of the fluctuations without cancellation between positive and negative deviations."

Squaring gives larger deviations more weight. Because every squared deviation is nonnegative, the variance is also nonnegative. For a discrete random variable,

$$\mathrm{Var}(X) = \sum_x p_X(x)\,(x - \mu_X)^2,$$

and for a continuous one,

$$\mathrm{Var}(X) = \int (x - \mu_X)^2\, p_X(x)\, dx.$$

For the two variables above, $\mathrm{Var}(X) = \frac{1}{2}(1) + \frac{1}{2}(1) = 1$ and $\mathrm{Var}(Y) = \frac{1}{2}(100) + \frac{1}{2}(100) = 100$, which separates them where the mean did not.

1.6.1 Variance From the First Two Moments

There is a useful equivalent expression for the variance. Let's start from the definition,

$$\mathrm{Var}(X) = \mathbb{E}\Big[(X - \mu_X)^2\Big].$$

Expand the square:

$$(X - \mu_X)^2 = X^2 - 2\mu_X X + \mu_X^2.$$

Substituting the expanded square into the expectation, we obtain:

$$\mathrm{Var}(X) = \mathbb{E}\Big[X^2 - 2\mu_X X + \mu_X^2\Big].$$

Since μ_X is constant, we can apply linearity of expectation:

$$\mathrm{Var}(X) = \mathbb{E}\Big[X^2\Big] - 2\mu_X \mathbb{E}[X] + \mu_X^2.$$

Since $\mathbb{E}[X] = \mu_X$, the middle term is $-2\mu_X^2$, and we obtain

$$\mathrm{Var}(X) = \mathbb{E}\Big[X^2\Big] - 2\mu_X^2 + \mu_X^2 = \mathbb{E}\Big[X^2\Big] - \mu_X^2.$$

Therefore

$$\boxed{\mathrm{Var}(X) = \mathbb{E}\Big[X^2\Big] - (\mathbb{E}[X])^2}$$

"The variance $\mathrm{Var}(X)$ equals the second moment $\mathbb{E}[X^2]$ minus the square of the mean $\mathbb{E}[X]$, so we can compute it by accumulating both the observed values x and their squares x^2 in one pass."

The variance depends on the distribution only through the first moment $\mathbb{E}[X]$ and the second moment $\mathbb{E}[X^2]$. Standard deviation returns the quantity to the original units:

$$\sigma_X = \sqrt{\mathrm{Var}(X)}.$$

1.6.2 Variance Example

Let X take the values 0, 1 and 2 with probabilities

$$p_X(0) = 0.2, \qquad p_X(1) = 0.5, \qquad p_X(2) = 0.3.$$

The first moment is

$$\mathbb{E}[X] = 0.2(0) + 0.5(1) + 0.3(2) = 0 + 0.5 + 0.6 = 1.1,$$

and the second moment is

$$\mathbb{E}\Big[X^2\Big] = 0.2(0) + 0.5(1) + 0.3(4) = 0 + 0.5 + 1.2 = 1.7.$$

The variance follows from the boxed identity:

$$\mathrm{Var}(X) = 1.7 - (1.1)^2 = 1.7 - 1.21 = 0.49,$$

and the standard deviation is

$$\sigma_X = \sqrt{0.49} = 0.7.$$

If we compute the variance straight from the definition, we get the same value. The three squared deviations are $(0-1.1)^2 = 1.21$, $(1-1.1)^2 = 0.01$ and $(2-1.1)^2 = 0.81$, and their probability-weighted sum is $0.2(1.21) + 0.5(0.01) + 0.3(0.81) = 0.242 + 0.005 + 0.243 = 0.49$. Computing deviations requires μ_X first, so this calculation takes two passes over the data. The moment form is therefore used inside training loops.

1.7 Covariance, Correlation and the Covariance Matrix

Variance measures the fluctuation of one variable. Covariance measures how two variables fluctuate together:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]\,.$$

"The covariance $\mathrm{Cov}(X, Y)$ averages the product of the deviation of the random variable X from its mean μ_X and the deviation of the random variable Y from its mean μ_Y."

Positive covariance means that large values of X tend to occur with large values of Y. Negative covariance means that large values of X tend to occur with small values of Y. Covariance near zero indicates little linear co-fluctuation. Variance is covariance with itself,

$$\mathrm{Var}(X) = \mathrm{Cov}(X, X),$$

which follows by setting $Y = X$ in the definition.

1.7.1 Hydrophobicity and Binding Score

Suppose three peptides occur with equal probability $1/3$. Let X represent hydrophobicity on an arbitrary index and Y a membrane-binding score:

$$(x, y) \in \{(1, 2),\ (2, 3),\ (3, 7)\}.$$

The means are

$$\mu_X = \tfrac{1}{3}(1 + 2 + 3) = 2, \qquad \mu_Y = \tfrac{1}{3}(2 + 3 + 7) = 4.$$

The deviations are $(-1, -2)$, $(0, -1)$ and $(+1, +3)$, and their products are 2, 0 and 3. The covariance is

$$\mathrm{Cov}(X, Y) = \tfrac{1}{3}\left[(-1)(-2) + (0)(-1) + (1)(3)\right] = \tfrac{1}{3}\left[2 + 0 + 3\right] = \tfrac{5}{3} \approx 1.667.$$

The covariance is positive because hydrophobicity and binding score increase together across the three peptides. The two variances are

$$\mathrm{Var}(X) = \tfrac{1}{3}\left[1 + 0 + 1\right] = \tfrac{2}{3} \approx 0.6667, \qquad \mathrm{Var}(Y) = \tfrac{1}{3}\left[4 + 1 + 9\right] = \tfrac{14}{3} \approx 4.667,$$

so $\sigma_X \approx 0.8165$ and $\sigma_Y \approx 2.160$.

Covariance depends on the units of both variables, so rescaling the binding assay rescales the covariance. To calculate the unitless correlation, divide by both standard deviations:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X\, \sigma_Y}.$$

"The correlation ρ_{XY} equals the covariance $\mathrm{Cov}(X, Y)$ divided by the product of the standard deviations σ_X and σ_Y, which removes the units of both random variables."

For the three peptides,

$$\rho_{XY} = \frac{1.667}{0.8165 \times 2.160} = \frac{1.667}{1.764} \approx 0.9449.$$

Correlation satisfies

$$-1 \leq \rho_{XY} \leq 1,$$

with $\rho = 0$ meaning no linear correlation, $\rho = +1$ a perfect positive linear relation, and $\rho = -1$ a perfect negative one. Zero correlation rules out a linear relation, but does not generally imply independence. Independent variables have zero covariance whenever their moments exist. Correlation also does not establish causation.

1.7.2 The Covariance Matrix

Generative models operate on vectors. Let

$$X = (X_1, X_2, \ldots, X_d)^\top \in \mathbb{R}^d, \qquad \mu = \mathbb{E}[X] \in \mathbb{R}^d,$$

where μ is the vector of coordinate means, $\mu_i = \mathbb{E}[X_i]$. The covariance matrix is

$$\Sigma = \mathbb{E}\Big[(X - \mu)(X - \mu)^\top\Big] \in \mathbb{R}^{d \times d}.$$

"The covariance matrix Σ averages the outer product of the centered random vector $X - \mu$ with its transpose, where μ is the mean vector, to record the covariance between every pair of coordinates."

The entry in row i and column j is

$$\Sigma_{ij} = \mathrm{Cov}(X_i, X_j).$$

For two variables,

$$\Sigma = \begin{pmatrix} \mathrm{Var}(X_1) & \mathrm{Cov}(X_1, X_2) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Var}(X_2) \end{pmatrix}.$$

The diagonal entries hold variances, and the off-diagonal entries hold pairwise covariances. The peptide pair of Section 1.7.1 has

$$\Sigma = \begin{pmatrix} 2/3 & 5/3 \\ 5/3 & 14/3 \end{pmatrix} \approx \begin{pmatrix} 0.6667 & 1.667 \\ 1.667 & 4.667 \end{pmatrix}.$$

Covariance is symmetric,

$$\mathrm{Cov}(X, Y) = \mathrm{Cov}(Y, X),$$

because the product of the two deviations does not depend on their order, and therefore

$$\Sigma = \Sigma^\top.$$

A covariance matrix is symmetric, so all its eigenvalues are real.

1.8 Gaussian Noise and the Noising Equation

A common source distribution in generative models is Gaussian noise:

$$Z \sim \mathcal{N}(0, I).$$

"The noise vector Z follows a multivariate Gaussian with mean vector 0 and covariance matrix I, where I is the identity matrix."

The identity covariance means

$$\mathrm{Var}(Z_i) = 1 \quad \text{for every coordinate } i, \qquad \mathrm{Cov}(Z_i, Z_j) = 0 \quad \text{for } i \neq j.$$

Each coordinate varies on a unit scale, and no pair of coordinates tends to move together. We can use this Gaussian as the source distribution p_0.

To corrupt data, we scale the clean sample and add Gaussian noise. Here we use the diffusion convention for time: $t = 0$ corresponds to clean data, and corruption increases with t. This reverses the source-to-data direction. A standard noising equation is

$$X_t = \alpha_t X_0 + \sigma_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

"The noisy sample X_t at time t equals the clean sample X_0 multiplied by the signal coefficient α_t, plus independent standard Gaussian noise ε multiplied by the noise coefficient σ_t."

- X_0 is the clean data sample, a vector in $\mathbb{R}^d$.
- ε is a standard Gaussian vector in $\mathbb{R}^d$, drawn independently of X_0.
- $\alpha_t \geq 0$ is the signal coefficient at time t, which decreases as corruption proceeds.
- $\sigma_t \geq 0$ is the noise coefficient at time t, which increases as corruption proceeds.

If X_0 and ε are independent, the covariance of the sum is the sum of the covariances, each scaled by the square of its coefficient:

$$\mathrm{Cov}(X_t) = \alpha_t^2 \ \mathrm{Cov}(X_0) + \sigma_t^2 I.$$

"The noisy-sample covariance $\mathrm{Cov}(X_t)$ equals the squared signal coefficient α_t^2 times the clean-data covariance $\mathrm{Cov}(X_0)$, plus the squared noise coefficient σ_t^2 times the identity matrix I."

The signal covariance shrinks as α_t decreases. The isotropic noise covariance grows with σ_t^2, adding the same variance in every direction. Consider $\alpha_t = 0.6$ and $\sigma_t = 0.8$, a pair satisfying $\alpha_t^2 + \sigma_t^2 = 0.36 + 0.64 = 1$. For data with $\mathrm{Cov}(X_0) = I$,

$$\mathrm{Cov}(X_t) = 0.36\, I + 0.64\, I = I,$$

so the total variance is preserved and only the proportion due to the data changes. For anisotropic data with $\mathrm{Cov}(X_0) = \mathrm{diag}(4, 1)$,

$$\mathrm{Cov}(X_t) = 0.36 \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix} + 0.64 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 2.08 & 0 \\ 0 & 1.00 \end{pmatrix},$$

so the wide direction has contracted from variance 4 toward the noise scale, and the narrow direction is already at that scale. If we push α_t to zero and σ_t to one, the data covariance vanishes, leaving $\mathcal{N}(0, I)$. We control the amount of corruption through the schedule (α_t, σ_t).

Generative models rarely involve a single random variable. A biological sequence is associated with structure, binding, expression, toxicity and experimental measurements, so we need a distribution over several variables at once.

1.9 Joint, Marginal and Conditional Distributions

Let X represent the presence of a sequence motif and Y the binding activity of the same sequence. The joint distribution is

$$p(x, y) = \mathbb{P}(X = x,\ Y = y).$$

"The joint probability $p(x, y)$ is the probability that the random variables X and Y simultaneously take the values x and y."

A valid joint distribution satisfies the same two requirements as before,

$$p(x, y) \geq 0 \qquad \text{and} \qquad \sum_x \sum_y p(x, y) = 1,$$

so every joint probability is nonnegative and the sum over all pairs equals one.

1.9.1 A Two-by-Two Joint Distribution

Let $X = 1$ mean that the motif is present and $X = 0$ that it is absent, and let $Y = 1$ mean high binding and $Y = 0$ low binding. Table 1.1 lists the four joint probabilities.

	$Y = 1$ (high binding)	$Y = 0$ (low binding)
$X = 1$ (motif present)	0.36	0.24
$X = 0$ (motif absent)	0.08	0.32

Table 1.1: The joint distribution $p(x, y)$ over motif presence X and binding activity Y used throughout Sections 1.9 to 1.11. Each cell is the probability of the pair of outcomes labeling its row and column, and the four cells sum to one.

The entry 0.36 means that a sequence has the motif and high binding with probability 0.36. The four joint probabilities sum to one:

$$0.36 + 0.24 + 0.08 + 0.32 = 1.00.$$

1.9.2 Marginals

The marginal distribution describes one variable on its own, after summing over all possible values of the other:

$$p(x) = \sum_{y} p(x, y).$$

"The marginal probability $p(x)$ is the sum of the joint probabilities $p(x, y)$ over every possible value y of the other random variable Y."

For motif presence we sum over the two binding outcomes:

$$p(X = 1) = 0.36 + 0.24 = 0.60, \qquad p(X = 0) = 0.08 + 0.32 = 0.40.$$

For high binding we sum over the two motif outcomes:

$$p(Y = 1) = 0.36 + 0.08 = 0.44, \qquad p(Y = 0) = 0.24 + 0.32 = 0.56.$$

Both marginals are valid distributions, since $0.60 + 0.40 = 1$ and $0.44 + 0.56 = 1$. For continuous variables the summation becomes integration,

$$p(x) = \int p(x, y)\, dy,$$

and the interpretation is unchanged.

1.9.3 Conditionals

Suppose we know that the motif is present. To calculate the conditional distribution, we keep the outcomes consistent with that observation and renormalize their probabilities:

$$p(y \mid x) = \frac{p(x, y)}{p(x)}, \qquad p(x) > 0.$$

"The conditional probability $p(y \mid x)$ of the value y given the observed value x equals the joint probability $p(x, y)$ divided by the marginal probability $p(x)$, which normalizes the outcomes consistent with x to sum to one."

For sequences containing the motif,

$$p(Y = 1 \mid X = 1) = \frac{0.36}{0.60} = 0.6, \qquad p(Y = 0 \mid X = 1) = \frac{0.24}{0.60} = 0.4,$$

and for sequences without the motif,

$$p(Y = 1 \mid X = 0) = \frac{0.08}{0.40} = 0.2, \qquad p(Y = 0 \mid X = 0) = \frac{0.32}{0.40} = 0.8.$$

The overall probability of high binding is 0.44. After observing the motif it becomes 0.6, and after observing that the motif is absent it becomes 0.2. The denominator in each case is the mass of the surviving outcomes, which is why the two conditional distributions each sum to one even though the four joint entries do not sum to one within a row.

At sampling time, an autoregressive model conditions on the tokens already written. A diffusion model conditions on the current noisy state. A guided sampler also conditions on a desired property of the sample. In each case, we restrict the possible outcomes and renormalize their probabilities. The state space is large enough that computing this normalizing denominator is itself difficult.

1.10 Bayes' Rule and the Law of Total Probability

Rearranging the definition of conditional probability gives the joint probability:

$$p(x, y) = p(x)\, p(y \mid x).$$

"The joint probability $p(x, y)$ equals the marginal probability $p(x)$ of the first value x, multiplied by the conditional probability $p(y \mid x)$ of the second value y given x."

The same joint probability factors in the opposite order,

$$p(x, y) = p(y)\, p(x \mid y),$$

and setting the two factorizations equal,

$$p(x)\,p(y \mid x) = p(y)\,p(x \mid y),$$

leads to Bayes' rule:

$$\boxed{p(x \mid y) = \frac{p(y \mid x)\,p(x)}{p(y)}}$$

"The posterior probability $p(x \mid y)$ of the state x after observing y equals the likelihood $p(y \mid x)$ of that observation under x, times the prior probability $p(x)$, divided by the evidence $p(y)$."

- $p(x)$ is the prior, the distribution over x before y is observed.
- $p(y \mid x)$ is the likelihood, the probability of the observation under each value of x.
- $p(y)$ is the evidence, a normalizing constant that does not depend on x.
- $p(x \mid y)$ is the posterior, the distribution over x after y is observed.

Let's apply it to the motif question. We want the probability that a sequence contains the motif after observing high binding:

$$p(X = 1 \mid Y = 1) = \frac{p(Y = 1 \mid X = 1)\,p(X = 1)}{p(Y = 1)} = \frac{0.6 \times 0.60}{0.44} = \frac{0.36}{0.44} \approx 0.818.$$

Before observing binding, the motif probability is 0.60. After observing high binding, it rises to approximately 0.818. We assign more probability to motif presence because high binding is more likely when the motif is present. The size of this change depends on the ratio of the two likelihoods, 0.6 and 0.2.

We obtain the denominator by marginalizing over every possible cause:

$$p(y) = \sum_{x} p(y \mid x)\,p(x).$$

"The evidence $p(y)$, or probability of the observation y, sums the likelihood $p(y \mid x)$ times the prior probability $p(x)$ over every possible state x."

For high binding,

$$p(Y = 1) = 0.6 \times 0.60 + 0.2 \times 0.40 = 0.36 + 0.08 = 0.44,$$

which agrees with the marginal computed by summing the column of Table 1.1. This is the law of total probability. We use it to compute the normalizing constant of a posterior. For a sequence, the sum runs over a state space of size K^L, which can make exact normalization infeasible.

1.11 Independence and the Chain Rule

Two variables X and Y are independent when observing one does not change the distribution of the other:

$$p(y \mid x) = p(y) \qquad \text{for every } x \text{ with } p(x) > 0.$$

Equivalently, by the product rule, the joint factors into the two marginals:

$$p(x, y) = p(x)\, p(y).$$

"The random variables X and Y are independent when their joint probability $p(x, y)$ equals the product of the marginal probabilities $p(x)$ and $p(y)$ for every pair of values x and y."

In the numerical example of Table 1.1,

$$p(X = 1)\, p(Y = 1) = 0.60 \times 0.44 = 0.264,$$

and the joint entry is

$$p(X = 1, Y = 1) = 0.36.$$

The two numbers differ, so motif presence and high binding are dependent. A model trained on these data must represent the association measured by the gap between 0.36 and 0.264. Factorizing the two variables would discard this association.

1.11.1 Factorizing a Sequence

Repeated application of the product rule produces the probability of a complete sequence. For a sequence $x = (x_1, x_2, \ldots, x_L)$,

$$p(x_1, x_2, \ldots, x_L) = \prod_{i=1}^{L} p(x_i \mid x_1, \ldots, x_{i-1})\,.$$

"The joint probability $p(x_1, \ldots, x_L)$ of a sequence with L tokens equals the product over positions i of the probability of token x_i conditioned on all preceding tokens $x_1, \ldots, x_{i-1}$."

At $i = 1$, the first factor is the marginal $p(x_1)$, with no preceding tokens to condition on. This factorization holds for every joint distribution over sequences without assuming independence

between positions. At each step, we apply $p(a, b) = p(a)\, p(b \mid a)$, taking a as the prefix and b as the next token.

We can apply this identity to autoregressive models and diffusion models.

In an autoregressive model, we parameterize each conditional directly. A network with parameters θ produces $p_\theta(x_i \mid x_1, \ldots, x_{i-1})$ for every position. We draw a sample one token at a time, conditioning each token on those already generated. Large language models use this construction [Brown et al., 2020], as do protein language models that generate sequences residue by residue. We can compute the exact likelihood in one pass. Generating a sample requires L sequential network evaluations in a fixed factorization order.

In diffusion models we apply the same product rule along a different axis. Let X_0 be a clean sample and X_t its corrupted version at a noise level indexed by t. The forward corruption distribution is

$$p(x_t \mid x_0),$$

which we choose and therefore know in closed form. The reverse denoising distribution is

$$p(x_0 \mid x_t),$$

which we do not know. Bayes' rule connects the two:

$$p(x_0 \mid x_t) = \frac{p(x_t \mid x_0)\, p(x_0)}{p(x_t)}.$$

"The reverse conditional probability $p(x_0 \mid x_t)$ of the clean sample x_0 given the noisy sample x_t equals the forward corruption probability $p(x_t \mid x_0)$ times the clean-data prior $p(x_0)$, divided by the noisy marginal probability $p(x_t)$."

The noisy marginal in the denominator is the law of total probability applied to the corruption:

$$p(x_t) = \sum_{x_0} p(x_t \mid x_0)\, p(x_0),$$

with an integral replacing the sum for continuous states. The noisy marginal $p(x_t)$ averages the corruption distribution over all clean samples. During training, we approximate it by drawing x_0 from the dataset and corrupting it to obtain x_t.

We first draw clean data x_0 and corrupt it into x_t using the known forward distribution $p(x_t \mid x_0)$. From the resulting x_t, we train the network to predict x_0 or the added noise. Training across many examples approximates the reverse conditional $p(x_0 \mid x_t)$. We then sample by applying the learned reverse process from a fully corrupted state to a clean one. We use two decompositions for these constructions: the chain-rule factorization for autoregressive models and the Bayes factorization for diffusion models.

Both constructions require calculations with distributions over the state space. For a whole sequence, that space is enormous. We have written the distribution as a function p_X evaluated one state at a time. At a single position, however, the space contains only the $K = |\mathcal{V}|$ vocabulary elements, so these calculations involve just K numbers. A sampler repeats them at every step and position. By storing these probabilities in a vector, we can add, scale, multiply, and differentiate them using vector and matrix notation. We also need to specify which vectors represent valid distributions.

1.12 Probability Vectors and the Simplex

We can store a distribution over K states as a column vector of K probabilities.

Let X take one of the K values $x_1, x_2, \ldots, x_K$. Its distribution is fully specified by $\mathbb{P}(X = x_1), \ldots, \mathbb{P}(X = x_K)$. Let's stack these probabilities into a column:

$$p \;=\; \begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_K \end{pmatrix} \;\in\; \mathbb{R}^K, \qquad p_i \;=\; \mathbb{P}(X = x_i).$$

"The probability vector p contains one entry for each of the K states, with entry p_i equal to the probability that the random state X takes the value x_i."

Writing a distribution this way turns probabilistic statements into arithmetic on a vector in $\mathbb{R}^K$. A vector of K numbers is a distribution when two conditions hold:

$$p_i \;\geq\; 0 \quad \text{for every } i, \qquad \sum_{i=1}^{K} p_i \;=\; 1.$$

"Each entry p_i of the probability vector p is nonnegative, and the entries sum to one across all K possible states of the random variable X."

We call the set of vectors satisfying both conditions the probability simplex.

Definition 1.1 (Probability simplex)**.** The K-state probability simplex is

$$\Delta^{K-1} \;=\; \Big\{ p \in \mathbb{R}^K \;:\; p_i \geq 0 \text{ for all } i, \;\; \textstyle\sum_{i=1}^{K} p_i = 1 \Big\}.$$

Its elements are exactly the probability distributions over K states.

"The probability simplex Δ^{K-1} contains the vectors p with K nonnegative entries p_i that sum to one, so each point in the simplex represents a distribution over K states."

The superscript $K - 1$ records the dimension of the simplex. The vector p has K entries, and the normalization constraint $\sum_i p_i = 1$ removes one degree of freedom. Solving for the last coordinate makes this dependence explicit:

$$p_K \;=\; 1 \;-\; \sum_{i=1}^{K-1} p_i.$$

"The last probability p_K equals one minus the sum of the first $K-1$ probabilities p_i, so choosing those entries fixes the remaining probability mass."

We may choose $K-1$ of the probabilities freely, subject to keeping the remainder nonnegative. The last probability equals the remaining mass. The simplex therefore has dimension $K-1$.

1.12.1 The shape of the simplex

We can start with small K. For two states,

$$\Delta^1 \;=\; \{\,(p_1,p_2)\;:\;p_1+p_2=1,\;p_1\geq 0,\;p_2\geq 0\,\},$$

which is the line segment running from $(1,0)$ to $(0,1)$ in the plane. For three states,

$$\Delta^2 \;=\; \{\,(p_1,p_2,p_3)\;:\;p_1+p_2+p_3=1,\;p_i\geq 0\,\},$$

a filled triangle in $\mathbb{R}^3$. For four states Δ^3 is a filled tetrahedron, and for a general K the set is the $(K-1)$-dimensional analogue of a triangle.

The corners of these sets are the one-hot vectors. In $\mathbb{R}^3$ they are

$$\mathbf{e}_1=\begin{pmatrix}1\\0\\0\end{pmatrix},\qquad \mathbf{e}_2=\begin{pmatrix}0\\1\\0\end{pmatrix},\qquad \mathbf{e}_3=\begin{pmatrix}0\\0\\1\end{pmatrix},$$

and $\mathbf{e}_i$ is the distribution certain to produce state i. A point near a corner puts most of its mass on one state; the point at the center of the triangle, $(1/3,1/3,1/3)$, spreads the mass evenly. A token of observed data is at a corner and a model's prediction is in the interior, so the distance from the nearest corner measures how spread out the prediction is.

1.12.2 A three-state example

We can take the DNA alphabet restricted to the three bases A, C and G, with

$$p \;=\; \begin{pmatrix}p_{\mathrm{A}}\\p_{\mathrm{C}}\\p_{\mathrm{G}}\end{pmatrix} \;=\; \begin{pmatrix}0.5\\0.3\\0.2\end{pmatrix}.$$

This vector is a distribution: every entry is nonnegative, and $0.5+0.3+0.2=1$, so $p\in\Delta^2$. Within the triangle, it lies closest to the A corner, consistent with A having the highest probability, followed by C and then G.

When we store the probabilities and property values in vectors, we can calculate their probability-weighted average with a dot product. Let h be a numerical property with one value per state, stored as

$$h \;=\; \begin{pmatrix}h_1\\h_2\\\vdots\\h_K\end{pmatrix}\in\mathbb{R}^K,\qquad h_i=h(x_i).$$

Then

$$\mathbb{E}[h(X)] \;=\; \sum_{i=1}^{K} p_i\,h_i \;=\; p^{\top}h.$$

"The expected property value $\mathbb{E}[h(X)]$ equals the dot product $p^\top h$ of the probability vector p and the property vector h, which sums each probability p_i times its corresponding property value h_i."

For the three bases, let h score a base by the number of hydrogen-bond contacts it makes in some assay, with the values $h = (1, 0, 4)^\top$. The average score is

$$\mathbb{E}[h(X)] \;=\; p^\top h \;=\; (0.5)(1) + (0.3)(0) + (0.2)(4) \;=\; 0.5 + 0 + 0.8 \;=\; 1.3.$$

The number 1.3 is the average over many draws from this distribution. Each individual draw produces a score of 1, 0, or 4, so none has the score 1.3.

1.12.3 Mixtures and convexity

To interpolate between distributions in a generative model, we need intermediate vectors that also satisfy the probability constraints. Let p and q be two probability vectors over the same K states and let $\lambda \in [0, 1]$ be a mixture weight. Define

$$r \;=\; \lambda\, p \;+\; (1 - \lambda)\, q.$$

"Each entry of the mixture distribution r equals the corresponding probability in p multiplied by the mixture weight λ, plus the corresponding probability in q multiplied by the remaining weight $1 - \lambda$."

Every entry of r is a nonnegative combination of nonnegative numbers, so $r_i \geq 0$. The entries add to

$$\sum_{i=1}^{K} r_i \;=\; \lambda \sum_{i=1}^{K} p_i \;+\; (1 - \lambda) \sum_{i=1}^{K} q_i \;=\; \lambda \cdot 1 \;+\; (1 - \lambda) \cdot 1 \;=\; 1,$$

so $r \in \Delta^{K-1}$. The two-state case makes the arithmetic visible: with $p = (1, 0)$, $q = (0, 1)$ and $\lambda = 0.3$,

$$r \;=\; 0.3 \begin{pmatrix} 1 \\ 0 \end{pmatrix} \;+\; 0.7 \begin{pmatrix} 0 \\ 1 \end{pmatrix} \;=\; \begin{pmatrix} 0.3 \\ 0.7 \end{pmatrix}, \qquad 0.3 + 0.7 = 1.$$

Mixing the two corners of Δ^1 produces an interior point of Δ^1. The same holds in three states: mixing $p = (0.5, 0.3, 0.2)$ with $q = (0.1, 0.1, 0.8)$ at $\lambda = 0.5$ produces

$$r \;=\; 0.5 \begin{pmatrix} 0.5 \\ 0.3 \\ 0.2 \end{pmatrix} \;+\; 0.5 \begin{pmatrix} 0.1 \\ 0.1 \\ 0.8 \end{pmatrix} \;=\; \begin{pmatrix} 0.30 \\ 0.20 \\ 0.50 \end{pmatrix}, \qquad 0.30 + 0.20 + 0.50 = 1.$$

This holds for every λ between zero and one, so the whole segment joining p to q lies in the simplex. This is the defining property of a convex set, so Δ^{K-1} is convex. Two points of the three-state simplex and the segment between them stay inside the triangle (Figure 1.2).

Convexity allows us to interpolate between a source distribution and a data distribution while preserving a valid distribution at every intermediate time.

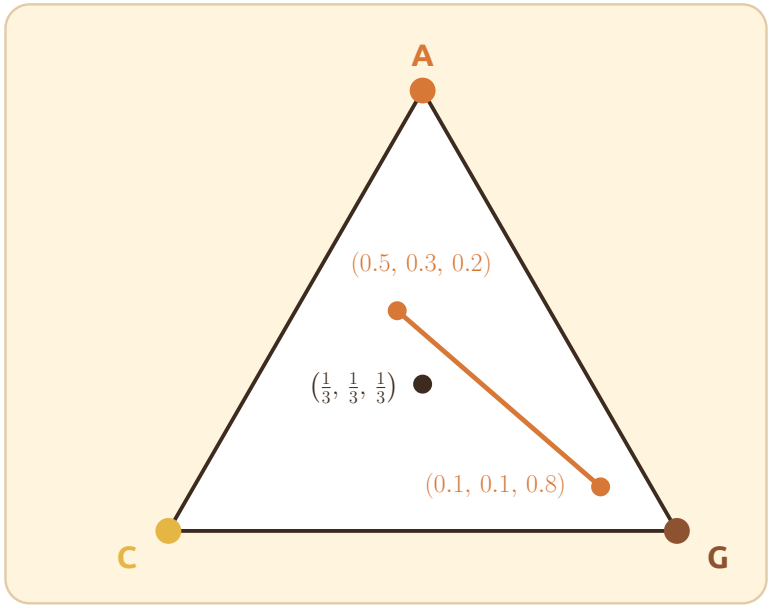


Figure 1.2: The three-state probability simplex Δ^2 over the bases A, C and G, drawn as a triangle. Each corner is a one-hot distribution placing all mass on the base that labels it, and the dark point at the center is the uniform distribution $(1/3, 1/3, 1/3)$. The two orange points are $p = (0.5, 0.3, 0.2)$ and $q = (0.1, 0.1, 0.8)$, and the orange segment between them collects all mixtures $\lambda p + (1 - \lambda)q$ for λ between 0 and 1. The segment never leaves the triangle, which is convexity.

1.13 Logits and the Softmax

A neural network produces unrestricted real numbers called *logits*. These outputs need not be nonnegative or sum to one. To convert a vector in $\mathbb{R}^K$ into a probability vector in Δ^{K-1}, we apply the softmax function.

Let $z = (z_1, \ldots, z_K) \in \mathbb{R}^K$ be the logit vector. The softmax produces

$$p_i \;=\; \mathrm{softmax}(z)_i \;=\; \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \ldots, K.$$

"The softmax probability p_i for state i equals the exponential of its logit z_i, divided by the sum of the exponentials of all K logits z_j, so larger logits produce larger probabilities."

- $z_i \in \mathbb{R}$ is the i-th logit, a network output with no sign or scale restriction.
- $e^{z_i} > 0$ is its exponential, which is positive for every real z_i.
- $\sum_{j=1}^{K} e^{z_j}$ is the normalizing sum, often called the partition function.

Both simplex conditions hold by construction. Positivity holds because an exponential is positive and the denominator is a sum of positive numbers, so each $p_i > 0$. Normalization holds because the denominator is the same for every entry:

$$\sum_{i=1}^{K} p_i \;=\; \sum_{i=1}^{K} \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \;=\; \frac{\sum_{i=1}^{K} e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \;=\; 1.$$

The softmax output lies in the interior of Δ^{K-1}, never on its boundary, because no state receives probability zero.

1.13.1 A worked softmax

We now take the three logits $z = (2, 1, 0)^\top$. Exponentiating each entry:

$$e^2 = 7.389, \qquad e^1 = 2.718, \qquad e^0 = 1.000.$$

Adding them:

$$\sum_{j=1}^{3} e^{z_j} \;=\; 7.389 + 2.718 + 1.000 \;=\; 11.107.$$

Next, we divide each exponential by the total:

$$p \;=\; \frac{1}{11.107}\begin{pmatrix} 7.389 \\ 2.718 \\ 1.000 \end{pmatrix} \;=\; \begin{pmatrix} 0.665 \\ 0.245 \\ 0.090 \end{pmatrix}, \qquad 0.665 + 0.245 + 0.090 \;=\; 1.$$

The probabilities preserve the ordering of the logits. Differences between logits become probability ratios, and the resulting vector lies inside the simplex (Figure 1.3).

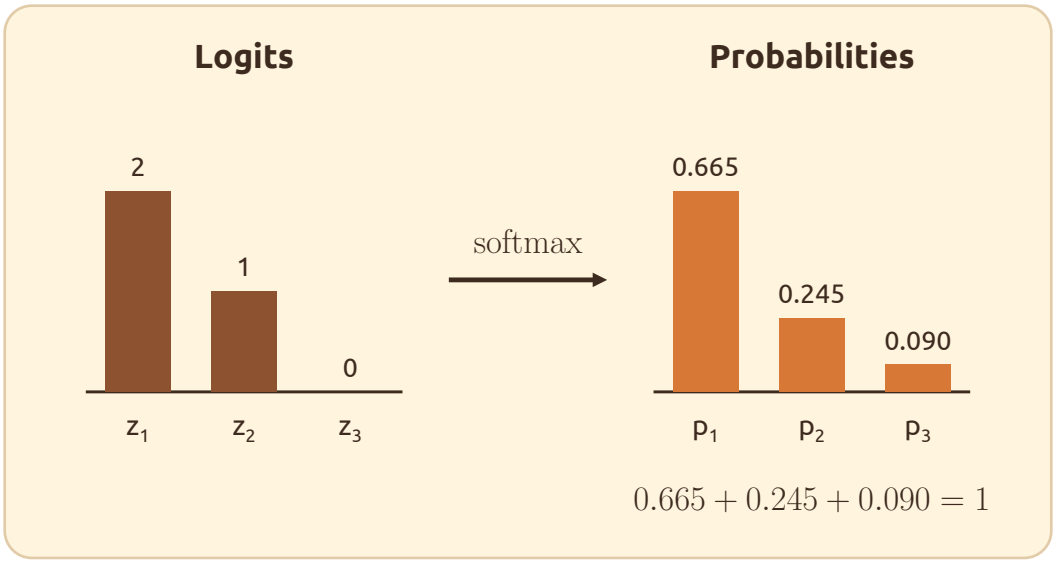


Figure 1.3: Three logits $z = (2, 1, 0)$ on the left, drawn as bars above a zero line, and the probabilities $\mathrm{softmax}(z) = (0.665, 0.245, 0.090)$ they produce on the right. The logit bars carry no constraint; the probability bars are positive and add to one, which is the sum written below them. The two groups use different vertical scales.

Adding the same constant c to every logit leaves the probabilities unchanged:

$$\frac{e^{z_i + c}}{\sum_j e^{z_j + c}} \;=\; \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} \;=\; \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The common factor e^c cancels. Only the differences between logits matter, and a shift of the whole vector moves nothing on the simplex.

1.13.2 The size of the simplex

A single DNA token has four possible values, so its distribution is a point of Δ^3, which is four numbers. A complete length-L DNA sequence has 4^L possible values, so a full distribution over sequences is a point of

$$\Delta^{4^L-1}.$$

For $L = 10$ that is $4^{10} = 1{,}048{,}576$ numbers, which a computer can hold. For $L = 100$ it is $4^{100} \approx 1.607 \times 10^{60}$ numbers, which no computer will ever hold. Storing one probability per sequence stops being possible almost immediately.

A neural model represents this sequence distribution implicitly through conditional or denoising distributions. Using the surrounding context, it computes a distribution for one token at a time. Together, these smaller distributions represent the point in Δ^{4^L-1} that we cannot store explicitly. We arrange the computation through an autoregressive or denoising factorization, with a softmax producing each K-entry probability vector.

We also use vectors to store positions, embeddings, and pixel values. Matrix transformations allow us to update these states and probability vectors.

1.14 Vectors, Matrices and Linear Maps

We can store an image, a protein embedding, molecular coordinates, or probabilities in a vector and transform that vector with a matrix. First, let's recall addition and scaling. We add two vectors $x, y \in \mathbb{R}^d$ coordinate by coordinate. Multiplying by a scalar $\alpha \in \mathbb{R}$ scales every coordinate:

$$x + y \;=\; \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_d + y_d \end{pmatrix}, \qquad \alpha x \;=\; \begin{pmatrix} \alpha x_1 \\ \vdots \\ \alpha x_d \end{pmatrix}.$$

Combining these operations produces a linear combination $\alpha x + \beta y$, where α and β are real scalars. The mixture of distributions in Section 1.12.3 is one example.

1.14.1 Dot product and norm

The dot product of two vectors in $\mathbb{R}^d$ is

$$x^\top y \;=\; \sum_{i=1}^{d} x_i\, y_i.$$

"The dot product $x^\top y$ sums the products $x_i y_i$ of matching coordinates across the two d-dimensional vectors x and y, producing a scalar measure of their alignment."

The dot product measures alignment. A positive value means the two vectors point in broadly similar directions, a negative value means they point in broadly opposing directions, and a value of zero means they are orthogonal. The Euclidean norm measures length, and it is the dot product of a vector with itself under a square root:

$$\|x\| \;=\; \sqrt{x^\top x} \;=\; \sqrt{\sum_{i=1}^{d} x_i^2}, \qquad \|x - y\| \;=\; \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}.$$

The second expression is the Euclidean distance between x and y. We use it in flow-matching regression, Gaussian densities, and reconstruction losses.

Let's take

$$x = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}, \qquad y = \begin{pmatrix} 3 \\ 0 \\ 4 \end{pmatrix}, \qquad w = \begin{pmatrix} 2 \\ -1 \\ 0 \end{pmatrix}.$$

Their dot products and lengths are

$$x^\top y = (1)(3) + (2)(0) + (2)(4) = 3 + 0 + 8 = 11, \qquad x^\top w = 2 - 2 + 0 = 0,$$

$$\|x\| = \sqrt{1 + 4 + 4} = 3, \qquad \|y\| = \sqrt{9 + 0 + 16} = 5.$$

A positive value of 11 means that x and y point the same general way, and the zero means that x and w are orthogonal. The distance between x and y uses the difference $x - y = (-2, 2, -2)$:

$$\|x - y\| = \sqrt{4 + 4 + 4} = \sqrt{12} = 2\sqrt{3} \approx 3.464.$$

1.14.2 Matrices as maps

A matrix with m rows and d columns is

$$A = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1d} \\ A_{21} & A_{22} & \cdots & A_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{md} \end{pmatrix},$$

with A_{ij} the entry in row i and column j. Applying A to a vector $x \in \mathbb{R}^d$ produces a vector $y = Ax \in \mathbb{R}^m$, so A is a map from d-dimensional space to m-dimensional space. Coordinate by coordinate,

$$y_i = \sum_{j=1}^{d} A_{ij}\, x_j, \qquad i = 1, \ldots, m.$$

"The output coordinate y_i sums each input coordinate x_j multiplied by its weight A_{ij}, where A_{ij} is the entry in row i and column j of the transformation matrix A."

For

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}, \qquad x = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

the two output coordinates are $y_1 = (2)(1) + (1)(2) = 4$ and $y_2 = (0)(1) + (3)(2) = 6$, so $Ax = (4, 6)^\top$.

1.14.3 Matrix columns and basis vectors

The standard basis vectors of $\mathbb{R}^2$ are $\mathbf{e}_1 = (1, 0)^\top$ and $\mathbf{e}_2 = (0, 1)^\top$, and applying A to them picks out single columns:

$$A\mathbf{e}_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \qquad A\mathbf{e}_2 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}.$$

These are the first and second columns of A. Writing an arbitrary vector in the basis as $x = x_1\mathbf{e}_1 + x_2\mathbf{e}_2$ and applying A term by term,

$$Ax \;=\; x_1\,(A\mathbf{e}_1) \;+\; x_2\,(A\mathbf{e}_2),$$

so the action of A on every vector is determined once we know where it sends the basis vectors. With $x = (1, 2)$,

$$Ax \;=\; 1\begin{pmatrix}2\\0\end{pmatrix} + 2\begin{pmatrix}1\\3\end{pmatrix} \;=\; \begin{pmatrix}2\\0\end{pmatrix} + \begin{pmatrix}2\\6\end{pmatrix} \;=\; \begin{pmatrix}4\\6\end{pmatrix},$$

which agrees with the row-by-row computation above. The quickest way to see what a transformation does to a shape is often to read its matrix column by column.

1.14.4 Linearity and affine maps

A transformation T is linear when it commutes with linear combinations:

$$T(\alpha x + \beta y) \;=\; \alpha\,T(x) \;+\; \beta\,T(y) \qquad \text{for all } x, y \in \mathbb{R}^d \text{ and all } \alpha, \beta \in \mathbb{R}.$$

"Applying the linear map T to the combination $\alpha x+\beta y$ gives the same result as applying T separately to the input vectors x and y, then combining their outputs with the scalar weights α and β."

Every matrix transformation satisfies this, because $A(\alpha x+\beta y) = \alpha Ax+\beta Ay$ follows from distributing the sum in $y_i = \sum_j A_{ij}x_j$. Setting $\alpha = \beta = 0$ gives $T(0) = 0$, so every linear map leaves the origin fixed.

We can check this on the matrix above with $x = \mathbf{e}_1$, $y = \mathbf{e}_2$, $\alpha = 2$ and $\beta = 1$. The combination is $\alpha x + \beta y = (2, 1)$, and the left side is

$$A\begin{pmatrix}2\\1\end{pmatrix} \;=\; \begin{pmatrix}(2)(2) + (1)(1)\\(0)(2) + (3)(1)\end{pmatrix} \;=\; \begin{pmatrix}5\\3\end{pmatrix}.$$

The right side is

$$2\,(A\mathbf{e}_1) + 1\,(A\mathbf{e}_2) \;=\; 2\begin{pmatrix}2\\0\end{pmatrix} + \begin{pmatrix}1\\3\end{pmatrix} \;=\; \begin{pmatrix}4\\0\end{pmatrix} + \begin{pmatrix}1\\3\end{pmatrix} \;=\; \begin{pmatrix}5\\3\end{pmatrix}.$$

Both sides agree, so linearity holds on this pair.

Adding a constant translation to a linear map produces an affine map,

$$y \;=\; Ax + b, \qquad b \in \mathbb{R}^m,$$

in which A rotates, stretches and shears while b shifts the whole picture. An affine map fails to be linear whenever $b \neq 0$, since it moves the origin to b. A neural-network layer has the form $Ax + b$ before its nonlinear activation. The Gaussian noising equation has this form too: A scales the clean data, and b is the added noise. With $b = (1, -2)$ and the $x = (1, 2)$ above,

$$Ax + b \;=\; \begin{pmatrix}4\\6\end{pmatrix} + \begin{pmatrix}1\\-2\end{pmatrix} \;=\; \begin{pmatrix}5\\4\end{pmatrix}.$$

1.14.5 Composition, the identity and the inverse

Richer transformations arise by applying several maps in sequence. If $y = Ax$ and $z = By$, then substituting the first into the second yields

$$z \;=\; B(Ax) \;=\; (BA)\,x,$$

so applying A and then B is the single transformation BA. Matrix multiplication is composition of maps, and the matrix applied first stands on the right.

Order matters, because $AB \neq BA$ in general. We can see this with the coordinate stretch and the shear

$$A \;=\; \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, \qquad B \;=\; \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Multiplying in the two orders,

$$BA \;=\; \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}, \qquad AB \;=\; \begin{pmatrix} 2 & 2 \\ 0 & 1 \end{pmatrix},$$

which differ in the top-right entry. Let's apply both products to $u = (1, 1)$. Stretching first and shearing second produces $B(Au) = B(2, 1) = (3, 1)$. Shearing first and stretching second produces $A(Bu) = A(2, 1) = (4, 1)$. The same two operations in the opposite order land on different points.

The identity matrix I has ones on the diagonal and zeros elsewhere, and $Ix = x$ for every x. A square matrix A is invertible when a matrix A^{-1} exists with

$$AA^{-1} \;=\; A^{-1}A \;=\; I,$$

and A^{-1} undoes whatever A did. Invertibility means that every output has exactly one input, which is the property a generative map needs if we want to run it backwards from a sample to its source. For

$$M \;=\; \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \qquad M^{-1} \;=\; \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix},$$

the product MM^{-1} works out to I, and $\det M = (2)(1) - (1)(1) = 1$ confirms that M is invertible. Starting from $x = (1, 2)$,

$$Mx \;=\; \begin{pmatrix} 2+2 \\ 1+2 \end{pmatrix} \;=\; \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \qquad M^{-1}\begin{pmatrix} 4 \\ 3 \end{pmatrix} \;=\; \begin{pmatrix} 4-3 \\ -4+6 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

recovering the original vector exactly.

1.14.6 Means and covariances under a linear map

Applying a map to a random vector changes its distribution. For an affine map, we can calculate the transformed mean and covariance directly. Let X be a random vector in $\mathbb{R}^d$ with mean $\mu = \mathbb{E}[X]$ and covariance $\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top]$, and set $Y = AX + b$. The mean follows from linearity of expectation:

$$\mathbb{E}[Y] \;=\; A\mu + b.$$

"The mean $\mathbb{E}[Y]$ of the transformed random vector Y equals the matrix A applied to the input mean vector μ, plus the translation vector b."

For the covariance, the translation cancels in the deviation, since $Y - \mathbb{E}[Y] = (AX + b) - (A\mu + b) = A(X - \mu)$. Substituting into the definition,

$$\mathrm{Cov}(Y) \;=\; \mathbb{E}\big[A(X-\mu)(X-\mu)^\top A^\top\big] \;=\; A\,\mathbb{E}\big[(X-\mu)(X-\mu)^\top\big]\,A^\top \;=\; A\Sigma A^\top.$$

"The output covariance $\mathrm{Cov}(Y)$ equals the transformation matrix A times the input covariance matrix Σ times the transpose $A^\top$, with no change in covariance from the translation vector b."

A translation moves the center and does not change the spread; the matrix changes the directions and magnitudes of the variation. Now take $\mu = (1, 2)$, $\Sigma = I$, and the affine map with

$$A \;=\; \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, \qquad b \;=\; \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The new mean is

$$A\mu + b \;=\; \begin{pmatrix} 2 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \;=\; \begin{pmatrix} 2 \\ 3 \end{pmatrix},$$

and the new covariance is

$$A\Sigma A^\top \;=\; \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \;=\; \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}.$$

The first coordinate now has variance four, so its standard deviation doubled from 1 to 2. The second coordinate retains variance one. We can use this rule in a noise schedule, with a time-dependent A shrinking the data and a time-dependent covariance accumulating the noise.

1.15 Probability Vectors Under a Transition Matrix

We can update a distribution over a finite set by multiplying its probability vector by a matrix. To preserve a valid distribution, we construct the matrix from conditional probabilities.

Let $p_t \in \Delta^{K-1}$ be the distribution over K states at step t, and let P be the $K \times K$ matrix with entries

$$P_{ij} \;=\; \mathbb{P}(X_{t+1} = j \mid X_t = i), \qquad P_{ij} \geq 0, \qquad \sum_{j=1}^{K} P_{ij} = 1.$$

Row i of P is the conditional distribution of the next state given that the current state is i, so each row is itself a point of the simplex. Such a matrix is called row-stochastic. The distribution one step later follows from the law of total probability, summing over the state we came from:

$$p_{t+1}(j) \;=\; \sum_{i=1}^{K} P_{ij}\, p_t(i), \qquad \text{that is} \qquad p_{t+1} \;=\; P^\top p_t.$$

"The next-step probability $p_{t+1}(j)$ of state j sums the current probability $p_t(i)$ of each starting state i times its transition probability P_{ij} into j; equivalently, we multiply the current probability vector p_t by the transpose $P^\top$ of the transition matrix."

The transpose appears because we write probability vectors as columns while the rows of P are indexed by the starting state. Each new probability sums the mass that moves in from every previous state.

The update keeps us inside the simplex. Nonnegativity is immediate, since $p_{t+1}(j)$ is a sum of products of nonnegative numbers. Normalization follows from the row sums of P:

$$\sum_{j=1}^{K} p_{t+1}(j) \;=\; \sum_{j=1}^{K}\sum_{i=1}^{K} P_{ij}p_t(i) \;=\; \sum_{i=1}^{K} p_t(i)\sum_{j=1}^{K} P_{ij} \;=\; \sum_{i=1}^{K} p_t(i) \;=\; 1.$$

We exchanged the two sums and used $\sum_j P_{ij} = 1$.

1.15.1 A two-state example

Consider two states and the transition matrix

$$P \;=\; \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}.$$

Under the first row, a sample in state one remains in state one with probability 0.8 and moves to state two with probability 0.2. Under the second row, a sample in state two moves to state one with probability 0.3 and remains in state two with probability 0.7. Both rows sum to one. Let the current probability vector be the even split $p_t = (0.5, 0.5)^\top$. Then

$$p_{t+1} \;=\; P^\top p_t \;=\; \begin{pmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{pmatrix}\begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \;=\; \begin{pmatrix} (0.8)(0.5)+(0.3)(0.5) \\ (0.2)(0.5)+(0.7)(0.5) \end{pmatrix} \;=\; \begin{pmatrix} 0.55 \\ 0.45 \end{pmatrix}.$$

The next-step probability of state one is 0.55 and of state two is 0.45. Both entries are nonnegative and sum to one, in agreement with the general argument above. State one gained mass because its self-transition probability 0.8 exceeds the 0.7 of state two, and because mass moves in from state two at rate 0.3.

Iterating this update produces a Markov chain. To construct a discrete diffusion model, we choose P to corrupt a sequence. Taking the step size to zero yields a continuous-time jump process.

Iterating the update n times produces $p_n = (P^\top)^n p_0$. The same pattern holds when we repeatedly apply any matrix A to a state: $x_1 = Ax_0$, $x_2 = A^2x_0$ and $x_n = A^nx_0$. Computing a large matrix power directly can be expensive. To calculate the limit more easily, we look for directions in which the matrix acts by scalar multiplication. Along each such direction, we only need to raise a number to a power.

1.16 Eigenvalues and Eigenvectors

Applying a matrix usually changes both the length and direction of a vector. Eigenvectors identify directions along which we can replace this matrix multiplication with scalar scaling. A nonzero vector v is an *eigenvector* of a square matrix A, with *eigenvalue* λ, when

$$Av \;=\; \lambda\, v, \qquad v \neq 0.$$

"Applying the matrix A to the nonzero eigenvector v scales that vector by its eigenvalue λ, so the output Av equals λv."

The eigenvalue determines the scaling along that direction. A value $\lambda > 1$ produces growth, a value $0 < \lambda < 1$ produces contraction, $\lambda = 1$ means the vector is unchanged, and $\lambda = 0$ collapses it to the zero vector. A negative λ reverses the direction while scaling it by $|\lambda|$, and a complex λ describes rotation combined with scaling.

Consider

$$A \;=\; \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad v_1 \;=\; \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Applying A,

$$Av_1 \;=\; \begin{pmatrix} (2)(1) + (1)(1) \\ (1)(1) + (2)(1) \end{pmatrix} \;=\; \begin{pmatrix} 3 \\ 3 \end{pmatrix} \;=\; 3 \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

so v_1 is an eigenvector with eigenvalue $\lambda_1 = 3$. Now take

$$v_2 \;=\; \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad Av_2 \;=\; \begin{pmatrix} 2 - 1 \\ 1 - 2 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ -1 \end{pmatrix} \;=\; 1 \cdot v_2,$$

so v_2 is an eigenvector with eigenvalue $\lambda_2 = 1$. The direction v_1 grows by a factor of three at every update, and the direction v_2 does not change. Geometrically v_1 points along the diagonal where both coordinates increase together, and v_2 points along the other diagonal where one coordinate increases as the other decreases.

1.16.1 Derivation: the characteristic equation

To find the eigenvalues without guessing we start from the defining equation and turn it into a condition on λ alone. Moving everything in $Av = \lambda v$ to one side, we get

$$Av - \lambda v \;=\; 0.$$

To factor out the vector, we need two matrices of the same shape. We therefore multiply the scalar λ by the identity matrix:

$$Av - \lambda I v \;=\; 0, \qquad \text{that is} \qquad (A - \lambda I)\, v \;=\; 0.$$

This is a homogeneous linear system. The zero vector always solves it. An eigenvector must be nonzero, so we need $A - \lambda I$ to have a nontrivial null space. This occurs exactly when $A - \lambda I$ is singular. We can therefore find the eigenvalues by setting its determinant to zero:

$$\det(A - \lambda I) \;=\; 0.$$

"An eigenvalue λ of the matrix A makes the determinant of $A - \lambda I$ zero, where I is the identity matrix, allowing the equation $(A - \lambda I)v = 0$ to have a nonzero eigenvector v."

This is the characteristic equation. For a $d \times d$ matrix, the determinant is a polynomial of degree d in λ. It therefore has d roots, counting multiplicity, which are the eigenvalues.

Let's apply it to the running example. Subtracting λ along the diagonal,

$$A - \lambda I \;=\; \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix},$$

and setting its determinant to zero,

$$\det(A - \lambda I) \;=\; (2-\lambda)^2 - (1)(1) \;=\; \lambda^2 - 4\lambda + 3 \;=\; 0.$$

Factoring, $(\lambda - 3)(\lambda - 1) = 0$, so $\lambda_1 = 3$ and $\lambda_2 = 1$, matching the two eigenvectors found by inspection.

For each eigenvalue, we can find an eigenvector in the null space of $A - \lambda I$. For $\lambda = 3$,

$$A - 3I \;=\; \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix},$$

and the equation $(A - 3I)v = 0$ requires $-v_1 + v_2 = 0$, that is $v_1 = v_2$. One valid eigenvector is $(1, 1)$. For $\lambda = 1$,

$$A - I \;=\; \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix},$$

and $(A - I)v = 0$ requires $v_1 + v_2 = 0$, that is $v_1 = -v_2$. One valid eigenvector is $(1, -1)$. Any nonzero scalar multiple of an eigenvector has the same eigenvalue, since $A(cv) = cAv = c\lambda v = \lambda(cv)$. An eigenvector therefore specifies a direction without fixing a particular length.

1.16.2 Powers along an eigendirection

Along an eigenvector, repeated multiplication collapses to a single scalar power. If $Av = \lambda v$, then

$$A^2 v \;=\; A(\lambda v) \;=\; \lambda(Av) \;=\; \lambda^2 v,$$

and continuing in the same way,

$$A^n v \;=\; \lambda^n v.$$

"Applying the matrix A to its eigenvector v for n successive updates multiplies v by the power λ^n of its eigenvalue λ."

For the running example, $A^n v_1 = 3^n v_1$ while $A^n v_2 = 1^n v_2 = v_2$. The v_1 component eventually dominates, because 3^n grows without bound while 1^n stays at one.

We handle a general vector by decomposing it into eigendirections first. Let's take the initial vector $x_0 = (3, 1)^\top$. Writing $x_0 = c_1 v_1 + c_2 v_2$ means solving $c_1 + c_2 = 3$ and $c_1 - c_2 = 1$, so $c_1 = 2$ and $c_2 = 1$:

$$x_0 \;=\; 2\begin{pmatrix} 1 \\ 1 \end{pmatrix} + 1\begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Applying A n times and using $A^n v_i = \lambda_i^n v_i$ on each piece,

$$x_n \;=\; A^n x_0 \;=\; c_1 \lambda_1^n v_1 + c_2 \lambda_2^n v_2 \;=\; 2 \cdot 3^n \begin{pmatrix} 1 \\ 1 \end{pmatrix} + 1 \cdot 1^n \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Each eigenvector component evolves on its own, governed by its own eigenvalue. For $n = 2$,

$$x_2 \;=\; 2 \cdot 9 \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} \;=\; \begin{pmatrix} 18 \\ 18 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} \;=\; \begin{pmatrix} 19 \\ 17 \end{pmatrix}.$$

Squaring the matrix directly confirms it, since $A^2 = \left(\begin{smallmatrix} 5 & 4 \\ 4 & 5 \end{smallmatrix}\right)$ and $A^2 x_0 = (15+4,\, 12+5) = (19, 17)$. By $n = 10$ the eigen-route still costs one multiplication, $3^{10} = 59049$, and returns $x_{10} = (118099,\, 118097)$.

1.16.3 Diagonalization

If we collect the eigenvectors into the columns of a matrix, the decomposition above becomes a matrix identity. We can put the eigenvectors in V and the eigenvalues on the diagonal of Λ:

$$V \;=\; \begin{pmatrix} | & | \\ v_1 & v_2 \\ | & | \end{pmatrix} \;=\; \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad \Lambda \;=\; \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} \;=\; \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

When A has d linearly independent eigenvectors, V is invertible and

$$A \;=\; V \Lambda V^{-1}.$$

"The matrix A equals the eigenvector matrix V multiplied by the diagonal eigenvalue matrix Λ and then by the inverse eigenvector matrix V^{-1}."

We apply the three factors from right to left. First, V^{-1} converts ordinary coordinates into eigenvector coordinates. Then Λ scales each eigenvector coordinate independently. Finally, V converts back to the original coordinates. For the running example $V^{-1} = \frac{1}{2}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, and multiplying out $V\Lambda V^{-1}$ returns $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, which is A.

To compute a power, substitute the factorization n times. Each adjacent pair $V^{-1}V$ equals the identity:

$$A^n \;=\; (V\Lambda V^{-1})(V\Lambda V^{-1}) \cdots (V\Lambda V^{-1}) \;=\; V\Lambda^n V^{-1}, \qquad \Lambda^n \;=\; \begin{pmatrix} \lambda_1^n & 0 \\ 0 & \lambda_2^n \end{pmatrix}.$$

"The matrix power A^n equals the eigenvector matrix V times the diagonal matrix Λ^n times V^{-1}, where Λ^n contains each eigenvalue λ_i raised to the update count n."

Evaluating $V\,\mathrm{diag}(27, 1)V^{-1}$ at $n = 3$ produces $\left(\begin{smallmatrix} 14 & 13 \\ 13 & 14 \end{smallmatrix}\right)$, which matches A^3 computed directly.

1.16.4 The dominant eigenvalue

The eigenvalue with the largest magnitude controls the long-run behavior of repeated multiplication, because its power outgrows all the others. Suppose a two-dimensional system has $\lambda_1 = 2$ and $\lambda_2 = 0.5$ and starts at $x_0 = v_1 + v_2$. After ten steps,

$$x_{10} \;=\; 2^{10} v_1 + (0.5)^{10} v_2 \;=\; 1024\, v_1 + 0.000977\, v_2.$$

The second component has nearly disappeared, and the ratio of the two coefficients is about 1.05×10^6. The state has become aligned with the first eigenvector, and the same happens for any starting vector whose first coefficient c_1 is nonzero. Under repeated transitions, the state aligns with the dominant direction as the influence of the other initial components decreases. The same eigenvalue analysis describes convergence in Markov chains.

1.17 Stationary Distributions

For a transition matrix the eigenvalue of interest is $\lambda = 1$, because a distribution left unchanged by the update is the equilibrium the chain relaxes to. A distribution p^* is *stationary* for the transition matrix P when applying one step returns it unchanged:

$$P^\top p^* \;=\; p^*.$$

"The stationary probability vector p^* remains unchanged when multiplied by the transpose $P^\top$ of the transition matrix, so one more transition leaves the distribution equal to p^*."

Written as $(P^\top - I)p^* = 0$, this is the eigenvector equation of Section 1.16 with eigenvalue one. A stationary distribution is an eigenvector of $P^\top$ for $\lambda = 1$, normalized to sum to one and to have nonnegative entries.

Now take the two-state matrix of Section 1.15.1, with $P = \left(\begin{smallmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{smallmatrix}\right)$. Its stationary distribution satisfies

$$\begin{pmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{pmatrix} \begin{pmatrix} p_1^* \\ p_2^* \end{pmatrix} \;=\; \begin{pmatrix} p_1^* \\ p_2^* \end{pmatrix}.$$

The first equation reads $0.8p_1^* + 0.3p_2^* = p_1^*$, and if we subtract $0.8p_1^*$ from both sides we get

$$0.3\,p_2^* \;=\; 0.2\,p_1^*, \qquad \text{that is} \qquad p_1^* \;=\; 1.5\,p_2^*.$$

The second equation repeats the same condition. To fix the scale, we use normalization, $p_1^* + p_2^* = 1$. Substituting $p_1^* = 1.5p_2^*$ yields $2.5p_2^* = 1$. Therefore $p_2^* = 0.4$ and $p_1^* = 0.6$:

$$p^* \;=\; \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix}.$$

Checking the result by applying one more step,

$$P^\top p^* \;=\; \begin{pmatrix} (0.8)(0.6) + (0.3)(0.4) \\ (0.2)(0.6) + (0.7)(0.4) \end{pmatrix} \;=\; \begin{pmatrix} 0.48 + 0.12 \\ 0.12 + 0.28 \end{pmatrix} \;=\; \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix},$$

so the distribution is unchanged under further transitions.

We can calculate the convergence rate from the second eigenvalue. For a 2×2 matrix, the eigenvalues sum to the trace and multiply to the determinant. Here $\operatorname{tr} P = 1.5$ and $\det P = (0.8)(0.7) - (0.2)(0.3) = 0.56 - 0.06 = 0.5$. The characteristic equation is therefore $\lambda^2 - 1.5\lambda + 0.5 = 0$, with roots $\lambda_1 = 1$ and $\lambda_2 = 0.5$. We can decompose the current distribution as the stationary one plus a deviation, $p_t = p^* + d_t$. The deviation's entries sum to zero, so it is a multiple of the second eigenvector. Each update scales it by the second eigenvalue. After n updates, $d_n = (0.5)^n d_0$. Starting from $p_0 = (0.5, 0.5)$, the deviation is $d_0 = (-0.1, 0.1)$, and the chain halves it at every step:

$$p_1 = \begin{pmatrix} 0.55 \\ 0.45 \end{pmatrix}, \quad p_2 = \begin{pmatrix} 0.575 \\ 0.425 \end{pmatrix}, \quad p_3 = \begin{pmatrix} 0.5875 \\ 0.4125 \end{pmatrix}, \quad p_{10} = \begin{pmatrix} 0.59990 \\ 0.40010 \end{pmatrix},$$

closing in on $p^* = (0.6, 0.4)$ at a geometric rate set by $|\lambda_2| = 0.5$. The step from p_0 to p_1 is the same arithmetic computed in Section 1.15.1, now read as the first of an infinite sequence with a known limit.

For a discrete diffusion model, we choose a forward noising chain with a stationary distribution that is easy to sample, such as a point mass on the mask token or a uniform distribution over the vocabulary. The magnitude of the second eigenvalue determines how quickly corruption removes the influence of the original sequence.

1.18 Linear Dynamics in Continuous Time

Powers A^n describe updates at integer steps. In continuous time, a token can jump at a random instant, and a particle can drift along a smooth path. We specify an instantaneous rate and accumulate its effect over an interval. For linear dynamics, the continuous-time analogue of repeated matrix multiplication is

$$\frac{dx_t}{dt} \;=\; A\,x_t, \qquad x_t \in \mathbb{R}^d, \qquad A \in \mathbb{R}^{d\times d}.$$

"The instantaneous rate of change dx_t/dt of the state x_t at time t equals the fixed matrix A multiplied by the current state x_t, specifying the velocity at that state."

A scalar x_t obeying $dx_t/dt = ax_t$ has the solution $x_t = e^{ta}x_0$, because the exponential reproduces itself under differentiation, $\frac{d}{dt}e^{ta} = ae^{ta}$. The matrix version has the same form,

$$\boxed{x_t \;=\; e^{tA}x_0}$$

"The state x_t at time t equals the matrix exponential e^{tA} applied to the initial state x_0, where the exponential accumulates the instantaneous linear dynamics defined by A over the interval from zero to t."

1.18.1 The power series

The matrix exponential is defined by the same series as the scalar exponential, with the scalar replaced by a matrix:

$$e^{tA} \;=\; I \;+\; tA \;+\; \frac{t^2A^2}{2!} \;+\; \frac{t^3A^3}{3!} \;+\; \cdots \;=\; \sum_{k=0}^{\infty} \frac{t^kA^k}{k!}.$$

"The matrix exponential e^{tA} sums, over orders k, the matrix power A^k multiplied by elapsed time t to the power k and divided by k factorial, with the identity matrix I as the order-zero term."

The terms are the result of applying the transformation once, twice, three times and so on, with the factorials making the higher-order contributions progressively smaller. Each term holds the effect of increasingly many infinitesimal applications of A, and the full series accumulates those infinitesimal updates over the interval $[0, t]$.

If we differentiate the series term by term, we can verify the solution. Each term $t^kA^k/k!$ has derivative $kt^{k-1}A^k/k! = t^{k-1}A^k/(k-1)!$, and the $k = 0$ term is constant, so

$$\frac{d}{dt}e^{tA} \;=\; \sum_{k=1}^{\infty} \frac{t^{k-1}A^k}{(k-1)!} \;=\; A\sum_{m=0}^{\infty} \frac{t^mA^m}{m!} \;=\; A\,e^{tA},$$

where we shifted the index by $m = k - 1$ and factored out one copy of A. Therefore, if $x_t = e^{tA}x_0$, then

$$\frac{dx_t}{dt} \;=\; Ae^{tA}x_0 \;=\; Ax_t,$$

so the differential equation holds. At time zero the series collapses to its first term, $e^{0\cdot A} = I$, so $x_0 = Ix_0 = x_0$ and the initial condition holds as well. The matrix exponential satisfies both.

1.18.2 Matrix exponentials using eigenvalues

We rarely evaluate e^{tA} by summing its infinite series. For a diagonalizable matrix, we can use its eigenvalues. Write $A = V\Lambda V^{-1}$. The cancellation in Section 1.16.3 gives $A^k = V\Lambda^k V^{-1}$. Substitute this into the series and factor out V and V^{-1}:

$$e^{tA} \;=\; V\left(\sum_{k=0}^{\infty}\frac{t^k\Lambda^k}{k!}\right)V^{-1} \;=\; V\,e^{t\Lambda}\,V^{-1}, \qquad e^{t\Lambda} \;=\; \begin{pmatrix} e^{t\lambda_1} & & \\ & \ddots & \\ & & e^{t\lambda_d}\end{pmatrix}.$$

"The matrix exponential e^{tA} equals $Ve^{t\Lambda}V^{-1}$, where V^{-1} converts to eigenvector coordinates, the diagonal factor $e^{t\Lambda}$ scales each direction by $e^{t\lambda_i}$ for its eigenvalue λ_i, and V converts back."

The exponential of a diagonal matrix is the diagonal matrix of exponentials, because the powers of a diagonal matrix are diagonal and the series acts on each entry separately. An initial state decomposed as $x_0 = \sum_i c_i v_i$ therefore evolves as

$$x_t \;=\; \sum_{i=1}^{d} c_i\, e^{t\lambda_i}\, v_i,$$

with each eigenvector direction evolving independently and continuously. This is the continuous-time version of $x_n = \sum_i c_i\lambda_i^n v_i$, with the discrete power λ_i^n replaced by the continuous factor $e^{t\lambda_i}$.

1.18.3 Matrix exponential example

We return to $A = \left(\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right)$ with $\lambda_1 = 3$, $\lambda_2 = 1$, $v_1 = (1,1)$ and $v_2 = (1,-1)$. Let's start by summing the series at $t = 0.1$. The powers needed are $A^2 = \left(\begin{smallmatrix}5 & 4\\ 4 & 5\end{smallmatrix}\right)$ and $A^3 = \left(\begin{smallmatrix}14 & 13\\ 13 & 14\end{smallmatrix}\right)$, and the first four terms are

$$I = \begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}, \quad tA = \begin{pmatrix}0.2 & 0.1\\ 0.1 & 0.2\end{pmatrix}, \quad \frac{t^2A^2}{2} = \begin{pmatrix}0.0250 & 0.0200\\ 0.0200 & 0.0250\end{pmatrix}, \quad \frac{t^3A^3}{6} = \begin{pmatrix}0.00233 & 0.00217\\ 0.00217 & 0.00233\end{pmatrix}.$$

Adding them,

$$I + tA + \frac{t^2A^2}{2} + \frac{t^3A^3}{6} \;=\; \begin{pmatrix}1.22733 & 0.12217\\ 0.12217 & 1.22733\end{pmatrix}.$$

Now take the eigenvalue route. With $e^{0.3} = 1.34986$ and $e^{0.1} = 1.10517$,

$$e^{tA} \;=\; V\begin{pmatrix}e^{0.3} & 0\\ 0 & e^{0.1}\end{pmatrix}V^{-1} \;=\; \begin{pmatrix}\frac{1}{2}(e^{0.3}+e^{0.1}) & \frac{1}{2}(e^{0.3}-e^{0.1})\\ \frac{1}{2}(e^{0.3}-e^{0.1}) & \frac{1}{2}(e^{0.3}+e^{0.1})\end{pmatrix} \;=\; \begin{pmatrix}1.22751 & 0.12234\\ 0.12234 & 1.22751\end{pmatrix}.$$

The four-term truncation is correct to within 1.8×10^{-4} in every entry. The first discarded term, $t^4A^4/24$, has entries 1.71×10^{-4} and 1.67×10^{-4}. These values account for the size of the error. The factorials shrink the tail quickly for a small t, and the eigenvalue formula is exact for every t.

To evolve a state over a longer interval we use the eigenvector decomposition directly. Starting from $x_0 = (3, 1) = 2v_1 + v_2$ as in Section 1.16.2, the solution at time t is

$$x_t \;=\; 2e^{3t}\begin{pmatrix}1\\1\end{pmatrix} + e^{t}\begin{pmatrix}1\\-1\end{pmatrix},$$

and at $t = 1$, with $e^3 = 20.0855$ and $e^1 = 2.71828$,

$$x_1 \;=\; \begin{pmatrix}2e^3 + e\\2e^3 - e\end{pmatrix} \;=\; \begin{pmatrix}40.171 + 2.718\\40.171 - 2.718\end{pmatrix} \;=\; \begin{pmatrix}42.889\\37.453\end{pmatrix}.$$

The v_1 component has grown by $e^3 \approx 20.1$ while the v_2 component has grown only by $e \approx 2.72$, so the state has turned toward the diagonal direction, the continuous-time counterpart of the dominant-eigenvalue argument of Section 1.16.4.

1.18.4 Stability

Each eigenvector component of $dx_t/dt = Ax_t$ evolves as $c_i e^{t\lambda_i}$. Its growth or decay depends on the real part of the eigenvalue. A negative real eigenvalue produces exponential decay, a zero eigenvalue means the component stays constant, a positive real eigenvalue produces exponential growth, and a complex eigenvalue produces oscillation whose envelope grows or decays according to its real part. For $\lambda = -2$ the component evolves as $c\,e^{-2t}$, which falls to $0.1353\,c$ at $t = 1$ and to $0.002479\,c$ at $t = 3$, approaching zero as time increases. A system whose eigenvalues all have negative real part returns to the origin from any starting point, which is the continuous-time meaning of stability.

1.18.5 Covariance eigenvalues

We can also use eigenvalues to describe the shape of a Gaussian. A covariance matrix Σ is symmetric and positive semidefinite. It therefore has an orthonormal eigenvector basis. For this basis, $V^{-1} = V^\top$, so

$$\Sigma \;=\; V\Lambda V^\top.$$

"The covariance matrix Σ equals the orthonormal eigenvector matrix V times the diagonal variance matrix Λ times the transpose $V^\top$, with each variance measured along its corresponding eigenvector."

The eigenvectors specify the principal directions of variation. Their eigenvalues are the variances along those directions and are therefore nonnegative. For a two-dimensional Gaussian one eigenvector points along the long axis of the density ellipse and the other along the short axis. Here we read the running matrix $\Sigma = \left(\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right)$ as a covariance. Its eigenvalues are 3 and 1, and its unit eigenvectors are

$$\frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix} \qquad \text{and} \qquad \frac{1}{\sqrt{2}}\begin{pmatrix}1\\-1\end{pmatrix}.$$

A Gaussian with this covariance has standard deviation $\sqrt{3} \approx 1.732$ along the diagonal direction and $\sqrt{1} = 1$ along the anti-diagonal direction. Its contours are therefore ellipses elongated along $(1, 1)$, with an axis ratio of about 1.73. The positive off-diagonal entry $\Sigma_{12} = 1$ tilts them.

1.18.6 Linear Dynamics and Distributions

In a Markov chain, we compute repeated updates with powers of the transition matrix. Stationary distributions are eigenvectors with eigenvalue one. Continuous-time Markov chains have transition matrices of the form e^{tQ}, where Q is a rate matrix. For linear differential equations on continuous states, we use e^{tA}. Eigenvectors and eigenvalues also describe the directions and variances of a Gaussian, allowing us to control growth and decay through a noise schedule. The eigenvalues of a local linearization determine the stability of a learned dynamical system.

To evolve a distribution over a finite set, we store it as a point of Δ^{K-1}, multiply by $P^\top$ for one discrete step or by e^{tQ} for a continuous interval, and use the eigenvalues to calculate its long-run behavior. A continuous generative model also moves samples in $\mathbb{R}^d$ from noise toward data. A linear map fixes the origin and stretches each eigendirection uniformly, so it cannot transform a spherical Gaussian into a curved, multimodal data distribution. For this task, we need a nonlinear map and a rule for calculating how it changes the density.

1.19 Maps

A nonlinear map moves individual points. Applying it to a random variable also changes that variable's probability distribution. To describe a generative transformation, we need both the pointwise map and its effect on the distribution.

A map from a source space $\mathcal{X}$ to a target space $\mathcal{Y}$ is a rule that assigns one output to each input,

$$\phi : \mathcal{X} \to \mathcal{Y}, \qquad y = \phi(x).$$

"The map ϕ assigns an output state y in the target space $\mathcal{Y}$ to each input state x in the source space $\mathcal{X}$, according to $y = \phi(x)$."

The input and output spaces may differ, and the map need not be invertible. For a map from one real number to another, let

$$\phi(x) = 2x + 3.$$

Then $\phi(1) = 2(1) + 3 = 5$ and $\phi(-2) = 2(-2) + 3 = -1$. The map scales each input by two and shifts it by three. The scaling and shift act uniformly on every input, so this map is affine. To find its inverse, we reverse these two operations.

A map can also transform vectors, in which case the source and target dimensions are stated explicitly,

$$\phi : \mathbb{R}^d \to \mathbb{R}^m.$$

Component-wise, the output is a stack of m scalar functions of the same d inputs,

$$\phi(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \vdots \\ \phi_m(x) \end{bmatrix}.$$

"The map ϕ takes an input vector x with d coordinates and produces an output with m components, where each scalar function ϕ_i, from ϕ_1 to ϕ_m, depends on the input coordinates."

For a worked case, let

$$\phi(x_1, x_2) = \begin{bmatrix} x_1 + x_2 \\ x_1 x_2 \end{bmatrix}.$$

At $x = (2, 3)$ the two components evaluate to $2 + 3 = 5$ and $2 \cdot 3 = 6$, so

$$\phi(2, 3) = \begin{bmatrix} 2 + 3 \\ 2(3) \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

This map is nonlinear, because its second component contains the product $x_1 x_2$. A linear map, in the sense established earlier in this chapter, has each output built from sums of the inputs with fixed coefficients, and $x_1 x_2$ is not of that form. Applying a linear map to a Gaussian prior produces another Gaussian. To represent the non-Gaussian data distributions we care about, we therefore need a nonlinear generative map.

1.19.1 Images and Inverse Images

A map acts on single points, and through them on whole sets. For a set A contained in the input space, its image collects every output the map produces from inputs in A,

$$\phi(A) = \{\, \phi(x) \mid x \in A \,\}.$$

"The image $\phi(A)$ of the input set A contains every output $\phi(x)$ obtained by applying the map ϕ to an input x in A."

For a set B in the output space, we define its inverse image as the set of inputs that map into B:

$$\phi^{-1}(B) = \{\, x \in \mathcal{X} \mid \phi(x) \in B \,\}.$$

"The inverse image $\phi^{-1}(B)$ of the output region B contains every input x in the source space $\mathcal{X}$ whose mapped value $\phi(x)$ lies in B."

Here ϕ^{-1} denotes an inverse image, which exists even when the map has no inverse function. This set contains every input that reaches the chosen output region and may have zero, one, or many

elements. We can therefore calculate which inputs reach an output region even when the map is not invertible.

We compute inverse images by solving an inequality. Let

$$\phi(x) = 2x + 1$$

and choose the output interval $B = [1.4, 2.0]$. We then solve

$$1.4 \leq 2x + 1 \leq 2.0.$$

Subtracting one from all three parts gives $0.4 \leq 2x \leq 1.0$, and dividing by two gives $0.2 \leq x \leq 0.5$. Therefore

$$\phi^{-1}(B) = [0.2, 0.5].$$

The input interval has width $0.5 - 0.2 = 0.3$ and its image has width $2.0 - 1.4 = 0.6$. The map doubled the width.

1.19.2 Inverse Maps

A map has an inverse function when every output corresponds to exactly one input. The inverse ϕ^{-1} is then a map in its own right, defined by the two identities

$$\phi^{-1}(\phi(x)) = x, \qquad \phi(\phi^{-1}(y)) = y.$$

"Applying the inverse map ϕ^{-1} after the forward map ϕ recovers the input x, and applying ϕ after ϕ^{-1} recovers the output y."

An invertible map must be one-to-one and onto:

- one-to-one means distinct inputs have distinct outputs, so no two inputs collide;
- onto means every target value is reached by at least one input, so no output is left unused;
- a map satisfying both properties is bijective.

For the affine map $\phi(x) = 2x + 3$ above, we write $y = 2x + 3$ and solve for x. Subtracting three gives $y - 3 = 2x$ and dividing by two gives $x = (y - 3)/2$, so

$$\phi^{-1}(y) = \frac{y - 3}{2}.$$

"The inverse map $\phi^{-1}(y)$ recovers the input corresponding to output y by subtracting the translation three and dividing by the scale factor two."

With numbers, $\phi(4) = 2(4) + 3 = 11$ and $\phi^{-1}(11) = (11 - 3)/2 = 4$, which returns the input we started from.

To see how invertibility can fail, consider

$$\phi(x) = x^2.$$

Both 2 and -2 map to 4, since $\phi(2) = \phi(-2) = 4$. The output 4 does not identify a unique input, so the map is not one-to-one on the full real line and has no inverse function there. If we restrict the domain to nonnegative numbers, the map is one-to-one again and the inverse becomes

$$\phi^{-1}(y) = \sqrt{y}.$$

Invertibility depends on both the formula and the domain. If we restrict the allowed inputs, a many-to-one map can become one-to-one.

The affine map in d dimensions inverts by the same two steps. Consider

$$y = Ax + b.$$

If A is invertible, we subtract the translation to obtain $y - b = Ax$ and then multiply by A^{-1}:

$$x = A^{-1}(y - b).$$

"We recover the input vector x from the output vector y by subtracting the translation vector b and multiplying the result by the inverse transformation matrix A^{-1}."

Let's put numbers in and take

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

The forward map scales the first coordinate by two and the second by three, then shifts:

$$y = Ax + b = \begin{bmatrix} 2(2) \\ 3(1) \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Running the inverse on y recovers x. The inverse of a diagonal matrix inverts each diagonal entry, so $A^{-1} = \operatorname{diag}(1/2, 1/3)$, and

$$x = A^{-1}(y - b) = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \left(\begin{bmatrix} 5 \\ 2 \end{bmatrix} - \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right) = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} 4 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

The determinant of A is $2 \cdot 3 = 6$. This map expands area sixfold and reduces the corresponding density by a factor of six.

1.19.3 Composition

We composed linear maps by matrix multiplication in Section 1.14.5. We can also compose nonlinear maps, as we do when applying the successive layers of a deep network. Suppose ϕ maps x to y and ψ maps y to z,

$$y = \phi(x), \qquad z = \psi(y).$$

The composed map applies them in order:

$$z = (\psi \circ \phi)(x) = \psi(\phi(x)).$$

"The composition $\psi \circ \phi$ applies the first map ϕ to the input x, producing the intermediate value y, and then applies the second map ψ to y, producing the output z."

The symbol nearest the input acts first, and the written order runs opposite to the order of execution. Let

$$\phi(x) = 2x, \qquad \psi(y) = y + 3.$$

Then

$$(\psi \circ \phi)(x) = \psi(2x) = 2x + 3, \qquad (\psi \circ \phi)(4) = 2(4) + 3 = 11.$$

The order matters. Reversing it scales the shifted value, so the shift is scaled too:

$$(\phi \circ \psi)(x) = 2(x + 3) = 2x + 6, \qquad (\phi \circ \psi)(4) = 2(4) + 6 = 14.$$

The composed maps differ by three because doubling after translation also doubles the shift. Composition describes how several transformations move one point. When the input is random, these transformations also change its probability distribution. We train a generative model to control this output distribution.

1.20 Random Variables Under a Map

Every map so far has acted on a single point. A generative model starts from a random point, so we now apply a map to a random variable and ask what distribution comes out. Let X be a random variable with distribution p_X, and define

$$Y = \phi(X).$$

"The output random variable Y equals the map ϕ applied to the input random variable X, so the randomness of X determines the distribution of Y."

For samples, we apply the map to the i-th input to obtain the i-th output:

$$y^{(i)} = \phi(x^{(i)}).$$

Applying ϕ to every sample produces an output population whose distribution is called the pushforward distribution.

Definition 1.2 (Pushforward distribution)**.** Let $X \sim p_X$ take values in $\mathcal{X}$, and let $\phi : \mathcal{X} \to \mathcal{Y}$. The distribution of $Y = \phi(X)$ is the pushforward of p_X through ϕ, written $p_Y = \phi_{\#} p_X$, and defined on output regions by

$$\mathbb{P}(Y \in B) = \mathbb{P}\Big(X \in \phi^{-1}(B)\Big) \qquad \text{for every measurable } B \subseteq \mathcal{Y}.$$

"The probability that the output random variable Y lies in the region B equals the probability that the input random variable X lies in the inverse image $\phi^{-1}(B)$, the set of inputs mapped into B."

To find the probability of an output region, we trace the region backward through the map and measure the probability of all inputs that land there. Here ϕ does not have to be invertible, because the inverse image of Section 1.19.1 is defined for every map. The notation $\phi_{\#}$ denotes this operation on a distribution.

1.20.1 The Discrete Case

The simplest pushforward calculation occurs in a finite discrete state space, where probability masses reaching the same output are added. Suppose X has three possible states

$$\mathcal{X} = \{a, b, c\}$$

with

$$p_X(a) = 0.2, \qquad p_X(b) = 0.5, \qquad p_X(c) = 0.3,$$

and define the map

$$\phi(a) = u, \qquad \phi(b) = v, \qquad \phi(c) = v.$$

The state a is the only input that reaches u, so its mass transfers untouched,

$$p_Y(u) = p_X(a) = 0.2.$$

Two inputs reach v, so the inverse image $\phi^{-1}(\{v\}) = \{b, c\}$ holds two states and their masses add,

$$p_Y(v) = p_X(b) + p_X(c) = 0.5 + 0.3 = 0.8.$$

The output masses sum to $0.2 + 0.8 = 1$. Every input maps to an output, so the total probability remains one. The states b and c reach the same output v, so their probability masses combine (Figure 1.4).

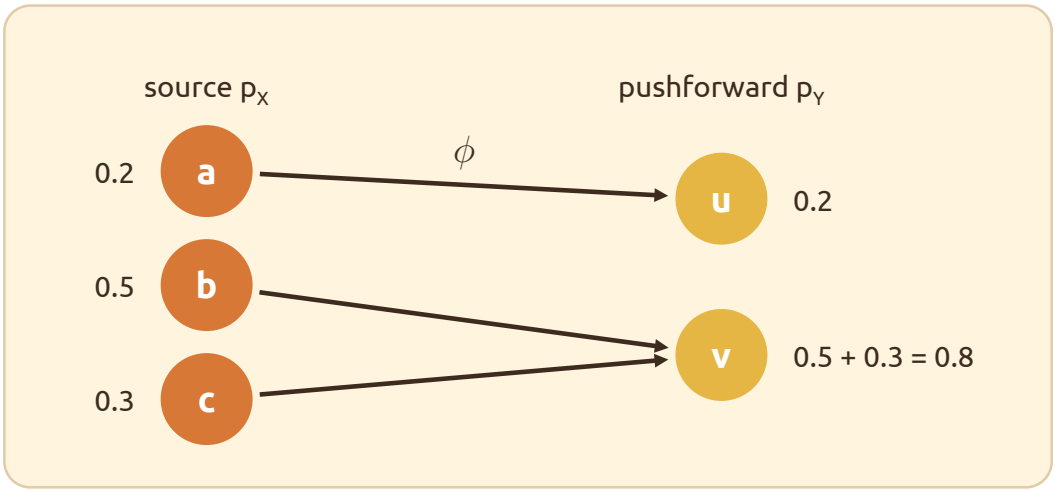


Figure 1.4: A discrete pushforward. The three source states a, b, c on the left carry masses 0.2, 0.5 and 0.3 under p_X. The map ϕ sends a to u and sends both b and c to v. The pushforward $p_Y = \phi_{\#} p_X$ on the right assigns 0.2 to u and $0.5 + 0.3 = 0.8$ to v, because the two inputs that share an output share its mass.

Merging is the discrete signature of a map that is not one-to-one. A biological map of this kind appears whenever we coarsen an alphabet. Consider a single DNA position with $p_X(\mathrm{A}) = 0.30$, $p_X(\mathrm{C}) = 0.20$, $p_X(\mathrm{G}) = 0.25$, and $p_X(\mathrm{T}) = 0.25$. Let ϕ map each base to its chemical class. It maps A and G to the purine class R, and C and T to the pyrimidine class Y. The pushforward over the two-letter alphabet $\{\mathrm{R}, \mathrm{Y}\}$ is

$$p_Y(\mathrm{R}) = 0.30 + 0.25 = 0.55, \qquad p_Y(\mathrm{Y}) = 0.20 + 0.25 = 0.45.$$

The map reduces four states to two by adding the masses within each class. We can no longer distinguish A from G using p_Y alone. Merging states loses information that we cannot recover from the output distribution.

1.20.2 The Continuous Case

The same principle applies in continuous spaces, where probability mass is spread across intervals and the geometry of the map changes the density inside them. Let

$$X \sim \text{Uniform}(0,1), \qquad Y = 2X + 1.$$

The input interval from 0 to 1 maps to the output interval from 1 to 3. Consider the output region $B = [1.4, 2.0]$. We computed its inverse image in Section 1.19.1:

$$\phi^{-1}(B) = [0.2, 0.5].$$

Because X is uniform on $[0, 1]$, its density is 1 there and the probability of an interval is its width,

$$\mathbb{P}(X \in [0.2, 0.5]) = 0.5 - 0.2 = 0.3.$$

By Definition 1.2, the same number is the probability of the output region,

$$\mathbb{P}(Y \in [1.4, 2.0]) = 0.3.$$

The input interval has width 0.3 and its output image has width 0.6, while the probability inside stays at 0.3. The same mass is spread over twice as much room, so the output density has to be half as large, $p_Y = 0.5$ against $p_X = 1$ (Figure 1.5). If we check the product in each case, we get $1 \times 0.3 = 0.3$ on the input side and $0.5 \times 0.6 = 0.3$ on the output side.

Width scaling and density scaling are reciprocal. When a map stretches different regions by different amounts, this relationship applies locally at each point.

1.20.3 Composition of Pushforwards

For a sequence of maps, we apply pushforwards in the same order as the point transformations. If

$$Y = \phi(X), \qquad Z = \psi(Y),$$

then

$$p_Z = \psi_{\#}(\phi_{\#} p_X)\,,$$

which is written equivalently as

$$(\psi \circ \phi)_{\#} p_X = \psi_{\#}(\phi_{\#} p_X)\,.$$

"Pushing the input distribution p_X through the composition $\psi \circ \phi$ gives the same distribution as first pushing p_X through the map ϕ and then pushing the resulting distribution through the map ψ."

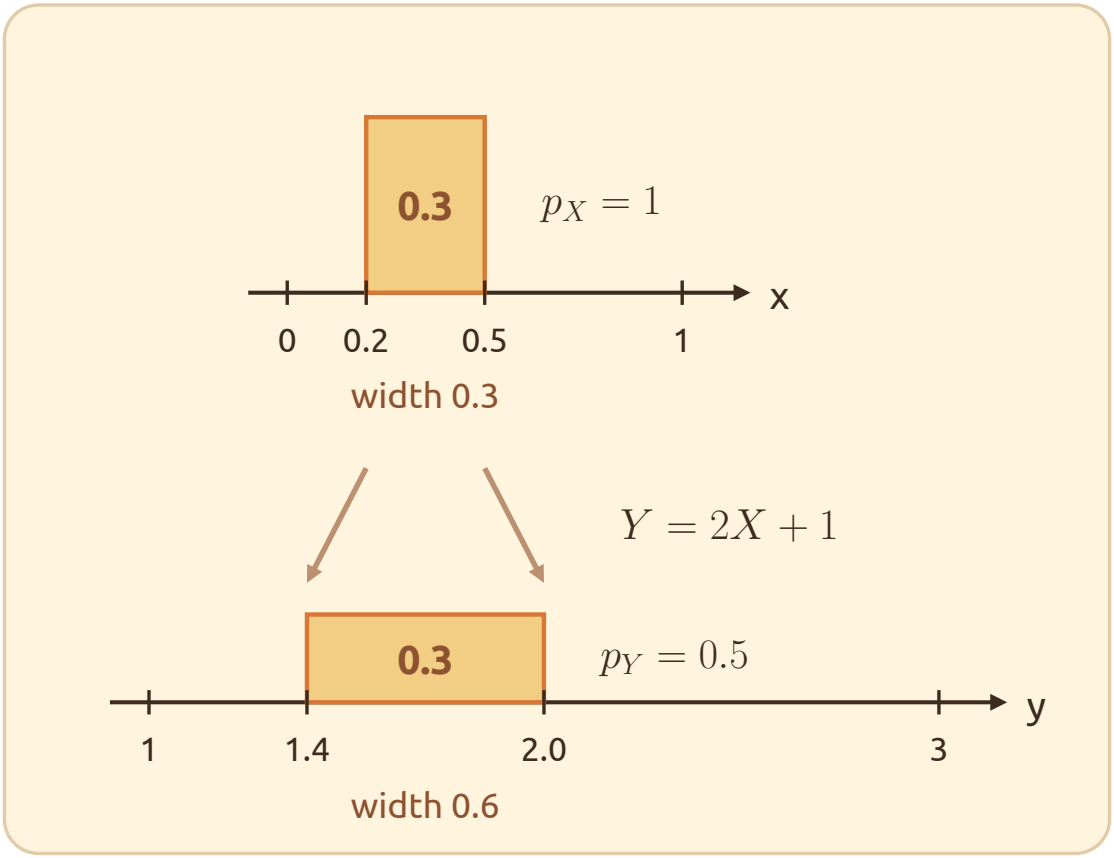


Figure 1.5: Probability through a map that doubles width. On the top axis, X is uniform on $[0, 1]$ with density $p_X = 1$, and the shaded interval $[0.2, 0.5]$ of width 0.3 carries probability 0.3. Under $Y = 2X + 1$ that interval becomes $[1.4, 2.0]$ on the bottom axis, of width 0.6. The probability inside is still 0.3, so the output density drops to $p_Y = 0.5$ and the two shaded rectangles have equal area.

The identity follows from Definition 1.2 and the set identity $(\psi \circ \phi)^{-1}(B) = \phi^{-1}(\psi^{-1}(B))$. An input lands in B after both maps exactly when it lands in $\psi^{-1}(B)$ after the first one, so the two probabilities are the probabilities of the same set of inputs.

We can therefore compose simple maps into a more complex generative transformation and track its distribution exactly through the sequence. For a deep network, we push the input distribution through the first layer, then through each remaining layer in order.

1.21 Time-Dependent Maps

Composition describes a finite sequence of transformations. Continuous generative models extend the idea to a family of maps indexed by time, with one map for each instant between the prior and the data. A continuous generative process uses

$$x_t = \phi_t(x_0).$$

"The state x_t at time t equals the time-dependent map ϕ_t applied to the initial state x_0, so varying

t while fixing x_0 traces one sample trajectory."

The symbols are:

- $x_0 \in \mathbb{R}^d$ is the initial state, a sample from the prior at time $t = 0$;
- $\phi_t : \mathbb{R}^d \to \mathbb{R}^d$ is the map at time t, with ϕ_0 the identity;
- x_t is the state the initial point has reached at time t, so if we hold x_0 fixed and let t run, we trace one trajectory through space.

If we push the initial distribution through the map at time t, we get the distribution at time t,

$$p_t = (\phi_t)_{\#} p_0.$$

"The distribution p_t at time t is the pushforward of the initial distribution p_0 through the time-dependent map ϕ_t, giving the distribution of all transformed samples at that time."

As t changes, individual samples trace trajectories. Their distribution changes with them, relating sample motion to probability evolution.

This is the mathematical form of a deterministic generative process. Let

$$X_0 \sim p_0$$

be a sample from a simple prior, to which we then apply a parameterized map,

$$X_1 = \phi_\theta(X_0).$$

The generated distribution is the pushforward of the prior through that map,

$$p_1 = (\phi_\theta)_{\#} p_0.$$

"The generated distribution p_1 is the pushforward of the initial distribution p_0 through the map ϕ_θ with parameters θ, which transforms initial samples X_0 into generated samples X_1."

We train the network parameters θ so that p_1 approaches the data distribution p_{data}. To sample, we draw X_0 from the prior and evaluate ϕ_θ in a forward pass. We choose the map family ϕ_θ and a loss that measures the mismatch between p_1 and p_{data}.

If the map is invertible, a generated or observed sample can also be mapped backward,

$$X_0 = \phi_\theta^{-1}(X_1).$$

"We recover the initial sample X_0 by applying the inverse map ϕ_θ^{-1}, with network parameters θ, to the final sample X_1."

Running the process backward from data to prior allows us to evaluate the prior density at the recovered point. To compute the exact density of p_1, we also need to account for how the map stretches or compresses the region around that point.

A nonlinear map can stretch, compress, rotate or shear different regions by different amounts. We therefore need a local description of the transformation. The derivative describes its effect near each input point.

1.22 Derivatives, Partial Derivatives and the Jacobian

For a scalar function f, the derivative at x is the limit of a ratio of changes,

$$f'(x) = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon) - f(x)}{\varepsilon}.$$

"The derivative $f'(x)$ is the limiting ratio of the output change $f(x+\varepsilon) - f(x)$ to the input displacement ε, as that displacement approaches zero around the input point x."

The derivative is the sensitivity of the output to a small change in the input, in output units per input unit. We can use it to approximate the function near x with a straight line:

$$f(x + \Delta x) \approx f(x) + f'(x)\,\Delta x.$$

"For a small displacement Δx from the input x, we approximate the new output $f(x+\Delta x)$ by adding the local slope $f'(x)$ times Δx to the original output $f(x)$."

We can take $f(x) = x^2$ and compute the derivative from the definition:

$$\begin{aligned} f'(x) &= \lim_{\varepsilon \to 0} \frac{(x+\varepsilon)^2 - x^2}{\varepsilon} \\ &= \lim_{\varepsilon \to 0} \frac{x^2 + 2x\varepsilon + \varepsilon^2 - x^2}{\varepsilon} \\ &= \lim_{\varepsilon \to 0} \frac{2x\varepsilon + \varepsilon^2}{\varepsilon} \\ &= \lim_{\varepsilon \to 0} (2x + \varepsilon) \\ &= 2x. \end{aligned}$$

The squared term ε^2 is smaller than ε by a factor of ε, so it is still there after the division and vanishes in the limit. At $x = 2$ the slope is $f'(2) = 2(2) = 4$. Using the linear approximation with a displacement of 0.01, we get

$$f(2.01) \approx f(2) + f'(2)(0.01) = 4 + 4(0.01) = 4.04.$$

The exact value is $2.01^2 = 4.0401$, so the approximation is off by 10^{-4}, which is the discarded ε^2 term with $\varepsilon = 0.01$. The local linear approximation is accurate for a small displacement, and its error shrinks quadratically as the displacement shrinks. Doubling the displacement to 0.02 would quadruple the error to 4×10^{-4}.

1.22.1 Partial Derivatives

A scalar derivative applies to one input and one output. A multivariable map has several inputs, so we measure sensitivity with respect to each input coordinate separately. Let

$$f : \mathbb{R}^d \to \mathbb{R}.$$

To calculate the partial derivative with respect to input coordinate x_j, we vary the input only along the j-th axis:

$$\frac{\partial f}{\partial x_j}(x) = \lim_{\varepsilon \to 0} \frac{f(x + \varepsilon e_j) - f(x)}{\varepsilon},$$

where $e_j \in \mathbb{R}^d$ is the j-th basis vector, with a one in position j and zeros elsewhere.

"The partial derivative of the scalar function f with respect to coordinate x_j is the limiting ratio of its output change to a displacement ε along the basis vector e_j, which changes only coordinate j, as ε approaches zero."

In practice we differentiate with respect to one coordinate while holding the others fixed. Let

$$f(x_1, x_2) = x_1^2 + 3x_1x_2.$$

If we differentiate with respect to x_1 while holding x_2 fixed, then $3x_2$ is a constant coefficient on x_1,

$$\frac{\partial f}{\partial x_1} = 2x_1 + 3x_2.$$

Differentiating with respect to x_2 while holding x_1 fixed, we are left with only the mixed term, since x_1^2 is constant for that derivative,

$$\frac{\partial f}{\partial x_2} = 3x_1.$$

At the point $(1, 2)$,

$$\frac{\partial f}{\partial x_1} = 2(1) + 3(2) = 8, \qquad \frac{\partial f}{\partial x_2} = 3(1) = 3.$$

Near this point, changing x_1 by a small amount has a larger effect on f than applying the same change to x_2, by a factor of $8/3 \approx 2.67$. The two numbers are the two components of the local slope, one per coordinate direction.

1.22.2 The Jacobian Matrix

A vector-valued map has several outputs as well as several inputs, so we need one partial derivative for every output-input pair. Let

$$\phi : \mathbb{R}^d \to \mathbb{R}^m$$

have output components $\phi_1, \ldots, \phi_m$. For every output ϕ_i and every input x_j we compute $\partial \phi_i / \partial x_j(x)$, and arrange the results in a matrix,

$$J_\phi(x) = \begin{bmatrix} \dfrac{\partial \phi_1}{\partial x_1} & \dfrac{\partial \phi_1}{\partial x_2} & \cdots & \dfrac{\partial \phi_1}{\partial x_d} \\ \dfrac{\partial \phi_2}{\partial x_1} & \dfrac{\partial \phi_2}{\partial x_2} & \cdots & \dfrac{\partial \phi_2}{\partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial \phi_m}{\partial x_1} & \dfrac{\partial \phi_m}{\partial x_2} & \cdots & \dfrac{\partial \phi_m}{\partial x_d} \end{bmatrix}.$$

"The Jacobian $J_\phi(x)$ contains m rows for the output components ϕ_i and d columns for the input coordinates x_j, with entry (i, j) equal to the partial derivative of ϕ_i with respect to x_j at the input point x."

Row i holds the sensitivities of one output to all inputs, and column j holds the sensitivities of all outputs to one input. The Jacobian is the multivariable replacement for the ordinary derivative, and it reduces to the scalar $f'(x)$ when $d = m = 1$.

For a worked case, consider

$$\phi(x_1, x_2) = \begin{bmatrix} x_1^2 + x_2 \\ \sin(x_1) + 3x_2 \end{bmatrix}.$$

The first output is $\phi_1(x_1, x_2) = x_1^2 + x_2$, with partial derivatives

$$\frac{\partial \phi_1}{\partial x_1} = 2x_1, \qquad \frac{\partial \phi_1}{\partial x_2} = 1.$$

The second output is $\phi_2(x_1, x_2) = \sin(x_1) + 3x_2$, with partial derivatives

$$\frac{\partial \phi_2}{\partial x_1} = \cos(x_1), \qquad \frac{\partial \phi_2}{\partial x_2} = 3.$$

Collecting the four entries into the matrix,

$$J_\phi(x_1, x_2) = \begin{bmatrix} 2x_1 & 1 \\ \cos(x_1) & 3 \end{bmatrix}.$$

Two entries depend on x_1, and two are constant. For a nonlinear map, the Jacobian generally varies with position. At the point $(1, 0)$, with $\cos(1) = 0.5403$ to four figures,

$$J_\phi(1, 0) = \begin{bmatrix} 2 & 1 \\ \cos(1) & 3 \end{bmatrix} \approx \begin{bmatrix} 2 & 1 \\ 0.540 & 3 \end{bmatrix}.$$

1.22.3 Local Linearization

We can use the Jacobian to approximate the nonlinear map near a point. For a small input displacement Δx,

$$\phi(x + \Delta x) \approx \phi(x) + J_\phi(x)\, \Delta x.$$

"For a small input displacement Δx around the point x, we approximate the new output $\phi(x + \Delta x)$ by adding the Jacobian $J_\phi(x)$ times Δx to the original output $\phi(x)$."

This is the vector form of the scalar approximation $f(x+\Delta x) \approx f(x)+f'(x)\Delta x$, with a matrix-vector product replacing scalar multiplication. Although the map may curve over a large region, we can approximate its local behavior using the linear transformation $J_\phi(x)$. This approximation allows us to calculate local changes in volume.

Using the Jacobian computed at $(1, 0)$, we choose the displacement

$$\Delta x = \begin{bmatrix} 0.01 \\ -0.02 \end{bmatrix}.$$

The predicted change in the output is

$$J_\phi(1, 0)\, \Delta x \approx \begin{bmatrix} 2 & 1 \\ 0.540 & 3 \end{bmatrix} \begin{bmatrix} 0.01 \\ -0.02 \end{bmatrix} = \begin{bmatrix} 2(0.01) + 1(-0.02) \\ 0.540(0.01) + 3(-0.02) \end{bmatrix} = \begin{bmatrix} 0 \\ -0.0546 \end{bmatrix}.$$

The first output changes by $0.02 - 0.02 = 0$, because the move in x_1 and the move in x_2 cancel in that component. The second output decreases by approximately 0.0546, almost all of it from the $3(-0.02)$ term. If we evaluate the exact map, we can confirm the prediction: $\phi(1, 0) = (1, 0.8415)$ and $\phi(1.01, -0.02) = (1.0001, 0.7868)$, an exact change of $(0.0001, -0.05464)$ against the predicted $(0, -0.0546)$.

1.23 Jacobian Determinants and Local Volume

The Jacobian describes local directional changes, with one output per row. When the input and output dimensions are equal, we use its determinant to calculate the local change in volume. For a two-dimensional matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

the determinant is

$$\det(A) = ad - bc.$$

"The determinant $\det(A)$ of the two-by-two matrix A equals the product of its diagonal entries a and d minus the product of its off-diagonal entries b and c."

The absolute determinant is the factor by which area scales. We can go back to the matrix from Section 1.19.2,

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \qquad \det(A) = 2(3) - 0(0) = 6.$$

A unit square transformed by A becomes a rectangle with width 2, height 3 and area $2 \times 3 = 6$. Tracking the four corners, $(0, 0)$ stays at $(0, 0)$, $(1, 0)$ moves to $(2, 0)$, $(1, 1)$ moves to $(2, 3)$ and $(0, 1)$ moves to $(0, 3)$. Therefore

$$\text{output area} = |\det(A)| \times \text{input area},$$

and in d dimensions the absolute determinant is the factor for d-dimensional volume. The sign of the determinant separates three cases:

- a positive determinant preserves orientation;
- a negative determinant reverses orientation, reflecting the region through a line or a hyperplane;
- a zero determinant collapses at least one dimension.

For

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \det(A) = 1(0) - 0(0) = 0,$$

every point (x_1, x_2) lands at $(x_1, 0)$. The whole plane collapses onto a line, so the image of the unit square has area zero. Every output point on that line corresponds to an entire vertical line of

inputs. We therefore cannot recover a unique input, and no inverse exists. A zero determinant is the two-dimensional version of the many-to-one failure met in Section 1.19.2.

A nonlinear map has a different Jacobian at each point, so its Jacobian determinant is a function of position, equal to the local volume scaling at each point separately. For a differentiable map

$$\phi : \mathbb{R}^d \to \mathbb{R}^d,$$

a small volume near x transforms approximately according to

$$dV_y \approx |\det J_\phi(x)|\, dV_x.$$

"The output volume dV_y near the mapped point $y = \phi(x)$ is approximately the input volume dV_x near x multiplied by the absolute Jacobian determinant $|\det J_\phi(x)|$, the local volume expansion factor."

Over a small enough region, we approximate the map by $J_\phi(x)$. This linear map scales volume by $|\det J_\phi(x)|$. As the region shrinks, the approximation error vanishes faster than the volume itself.

For a worked nonlinear case, let

$$\phi(x_1, x_2) = \begin{bmatrix} x_1^2 \\ x_2 \end{bmatrix}, \qquad x_1 > 0.$$

The restriction $x_1 > 0$ makes the map one-to-one, for the reason given in Section 1.19.2. Its Jacobian is

$$J_\phi(x_1, x_2) = \begin{bmatrix} 2x_1 & 0 \\ 0 & 1 \end{bmatrix},$$

with determinant

$$\det J_\phi(x_1, x_2) = (2x_1)(1) - 0(0) = 2x_1.$$

At $x_1 = 2$,

$$\det J_\phi = 2(2) = 4,$$

so a sufficiently small region near $x_1 = 2$ expands by approximately a factor of four. At $x_1 = 0.25$ the same formula returns $\det J_\phi = 0.5$, and a small region there contracts to half its area. One map stretches in one place and compresses in another, so we have to evaluate the volume factor point by point.

1.24 Change of Variables

To calculate the density after a transformation, we preserve probability while accounting for the change in local volume. Let

$$Y = \phi(X),$$

where ϕ is invertible. A small interval of width dx near x has probability approximately

$$p_X(x)\,dx,$$

and its image near y is a small interval of width dy with the same probability,

$$p_X(x)\,dx = p_Y(y)\,dy.$$

"The input density $p_X(x)$ times the small interval width dx equals the output density $p_Y(y)$ times the mapped interval width dy, preserving the probability mass between corresponding points x and y."

1.24.1 Derivation in One Dimension

The derivative relates the two interval widths. Over a small interval the map stretches length by the absolute slope,

$$dy = |\phi'(x)|\,dx,$$

where the absolute value appears because a decreasing map reverses the interval while keeping its length positive. Substituting this into the conservation statement gives

$$p_X(x)\,dx = p_Y(y)\,|\phi'(x)|\,dx.$$

If we cancel dx, which is a common positive factor on both sides, we get

$$p_Y(y) = \frac{p_X(x)}{|\phi'(x)|}.$$

To express the density as a function of y alone, we must eliminate the input x from the right-hand side. Since ϕ is invertible, we can substitute $x = \phi^{-1}(y)$. The inverse derivative is the reciprocal of the forward derivative. Using both identities, we obtain

$$p_Y(y) = p_X\Big(\phi^{-1}(y)\Big)\left|\frac{d\phi^{-1}(y)}{dy}\right|.$$

"The output density $p_Y(y)$ equals the input density p_X at the inverse-mapped point $\phi^{-1}(y)$, multiplied by the absolute derivative of the inverse map ϕ^{-1} with respect to the output coordinate y, which corrects for the change in interval length."

A transformation stretches or compresses small intervals by the factor $|\phi'(x)|$, so the density must change inversely to preserve the same probability mass. Expansion spreads the same probability across more space, so the density decreases. Compression concentrates the same probability into less space, so the density increases.

Let's compute a case where we can check every number. We take

$$X \sim \text{Uniform}(0, 1), \qquad Y = 2X + 1,$$

the map already used in Section 1.20.2. The inverse map is

$$x = \phi^{-1}(y) = \frac{y - 1}{2},$$

and its derivative is

$$\frac{d\phi^{-1}(y)}{dy} = \frac{1}{2}.$$

The input density is $p_X(x) = 1$ for $0 \le x \le 1$. Therefore

$$p_Y(y) = 1 \cdot \left|\frac{1}{2}\right| = \frac{1}{2}, \qquad 1 \le y \le 3.$$

The interval doubled in width from 1 to 2, so the density decreased from 1 to 1/2. Checking normalization,

$$\int_1^3 \frac{1}{2}\, dy = \frac{1}{2}(3 - 1) = 1,$$

so the output is a valid density. The same halving appeared in Figure 1.5 as an area argument on one interval, and here we derive it for every y at once.

1.24.2 Change of Variables in d Dimensions

The same conservation argument extends to several dimensions by replacing the scalar derivative with the Jacobian determinant of Section 1.23. A small input box of volume dV_x becomes an output region of volume $|\det J_\phi(x)|\, dV_x$. The probability inside remains unchanged. We obtain the output density by dividing this probability by the output volume.

Theorem 1.3 (Change of variables)**.** *Let X be a continuous random variable on $\mathbb{R}^d$ with density p_X, let $\phi : \mathbb{R}^d \to \mathbb{R}^d$ be differentiable and invertible with differentiable inverse, and set $Y = \phi(X)$. Then the density of Y is*

$$p_Y(y) = p_X\left(\phi^{-1}(y)\right)\left|\det J_{\phi^{-1}}(y)\right|,$$

and equivalently, written at the input point x with $y = \phi(x)$,

$$p_Y(\phi(x)) = \frac{p_X(x)}{|\det J_\phi(x)|}.$$

"The output density $p_Y(y)$ equals the input density p_X at the corresponding input $\phi^{-1}(y)$, multiplied by the absolute determinant of the inverse Jacobian $J_{\phi^{-1}}(y)$, which corrects for the change in d-dimensional volume."

The two forms are the same statement read from the two ends. We use the first when we hold an output y and want its density, which is the situation at evaluation time. The second applies when we hold an input x and want the density at its image, which is the situation during training. The second form also shows the direction of the correction, since $p_Y(\phi(x))$ decreases when volume expands and increases when volume contracts.

For the worked case, let X be uniform on the unit square,

$$\mathcal{X} = [0,1] \times [0,1], \qquad p_X(x) = 1,$$

to which we then apply the linear map

$$Y = AX, \qquad A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}.$$

A linear map has a constant Jacobian, equal to the matrix itself,

$$J_\phi(x) = A, \qquad \det(A) = 2(3) - 0(0) = 6.$$

Applying Theorem 1.3 in its forward form,

$$p_Y(y) = \frac{p_X(x)}{|\det J_\phi(x)|} = \frac{1}{6}$$

inside the transformed rectangle, which is $[0,2] \times [0,3]$ by the corner calculation of Section 1.23. Since $|\det(A)| = 6$, the transformation expands area by a factor of six, so the density decreases by a factor of six to $p_Y(y) = 1/6$. Checking normalization on a uniform density, the total probability is the density multiplied by the area,

$$\text{density} \times \text{area} = \frac{1}{6}(6) = 1.$$

Both the one-dimensional and the two-dimensional examples use a map whose stretching is the same everywhere, so one determinant was enough for the whole calculation. For a map with a position-dependent Jacobian, such as $\phi(x_1, x_2) = (x_1^2, x_2)$ with determinant $2x_1$, the density correction varies from point to point. A uniform density on the unit square becomes the output density $p_Y(y) = 1/(2\sqrt{y_1})$ on the image, large near $y_1 = 0$ where the map compresses and equal to $1/2$ at $y_1 = 1$ where it expands by a factor of two.

1.25 Composed Maps and the Log-Density Form

A deep network composes many maps. To calculate its density using change of variables, we first calculate the Jacobian of the composition with the multivariable chain rule.

Proposition 1.4 (Jacobian chain rule)**.** *For a composition $z = \psi(\phi(x))$ of differentiable maps,*

$$J_{\psi\circ\phi}(x) = J_\psi(\phi(x))\, J_\phi(x).$$

"The composition Jacobian $J_{\psi\circ\phi}(x)$ equals the second-map Jacobian J_ψ evaluated at the intermediate output $\phi(x)$, multiplied by the first-map Jacobian J_ϕ evaluated at the initial input x."

The order on the right matches the order of execution read from the input outward. We first apply the local stretching from ϕ at x. We then apply the local stretching from ψ at the intermediate point $\phi(x)$. Both act linearly on small displacements, so we compose them by multiplying their matrices.

If we take determinants, the matrix product becomes a product of numbers, since the determinant of a product is the product of the determinants,

$$\det J_{\psi\circ\phi}(x) = \det J_\psi(\phi(x))\ \det J_\phi(x).$$

Taking logarithms of the absolute values, we turn that product into a sum,

$$\log|\det J_{\psi\circ\phi}(x)| = \log|\det J_\psi(\phi(x))| + \log|\det J_\phi(x)|.$$

"The log absolute Jacobian determinant of the composition $\psi \circ \phi$ equals the log absolute determinant of J_ϕ at the input x, plus the log absolute determinant of J_ψ at the intermediate point $\phi(x)$."

Successive volume factors multiply, so their logarithms add. For a network of invertible layers, we can therefore calculate one log-determinant per layer and sum the results.

Let's verify the arithmetic on two maps already in hand. We take $\phi(x_1, x_2) = (x_1^2, x_2)$ with $x_1 > 0$, followed by the linear map $\psi(y) = Ay$ with $A = \mathrm{diag}(2, 3)$. At the point $x_1 = 2$ the two determinants are

$$\det J_\phi = 2x_1 = 4, \qquad \det J_\psi = \det(A) = 6,$$

so the composition scales volume by $4 \times 6 = 24$, and in logarithms

$$\log 4 + \log 6 = 1.3863 + 1.7918 = 3.1781 = \log 24.$$

If we compose the two maps by hand, we can confirm the factor without the chain rule. The composed map is $(\psi \circ \phi)(x_1, x_2) = (2x_1^2,\, 3x_2)$. Differentiating gives the Jacobian $\mathrm{diag}(4x_1, 3)$. At $x_1 = 2$, this is $\mathrm{diag}(8, 3)$, with determinant $8 \times 3 = 24$.

The change of variables, written in logarithms, now has the form used by every flow-based model. Starting from Theorem 1.3 in its forward form and taking logs of both sides,

$$\boxed{\log p_Y(y) = \log p_X(x) - \log|\det J_\phi(x)|} \qquad \text{with } y = \phi(x).$$

"The output log density $\log p_Y(y)$ equals the input log density $\log p_X(x)$ minus the log absolute Jacobian determinant $\log|\det J_\phi(x)|$, which accounts for the volume change from the input x to its output y."

We use this equation to evaluate densities in normalizing flows, which are generative models built from invertible layers:

- the base density $p_X(x)$ is a simple formula, usually a Gaussian, so it is cheap to evaluate;
- the invertible map produces a complicated sample y from that simple base sample;
- the Jacobian determinant corrects the density for local volume changes, so the output is a valid density and the correction is exact;
- the log-determinants add across composed transformations, so a deep stack amounts to a sum of per-layer terms.

Connecting this back to the deterministic generative process of Section 1.21, we write the generator as an invertible parameterized map,

$$X_1 = \phi_\theta(X_0), \qquad X_0 \sim p_0.$$

The generated density then satisfies

$$\log p_1(x_1) = \log p_0(x_0) - \log|\det J_{\phi_\theta}(x_0)|, \qquad x_0 = \phi_\theta^{-1}(x_1).$$

"The generated log density $\log p_1(x_1)$ equals the base log density $\log p_0(x_0)$ minus the log absolute determinant of the generator Jacobian $J_{\phi_\theta}(x_0)$, evaluated at the initial sample x_0 recovered from the final sample x_1."

We can compute each term by recovering x_0 with the inverse map, evaluating the prior density in closed form, and summing log-determinants over layers. This yields the exact log-likelihood

of a data point under the model. We can then train by gradient ascent on that likelihood. The architecture must preserve invertibility, and each layer must have a tractable determinant.

1.26 Local Invertibility and Inverse Branches

To check local invertibility, we examine the Jacobian determinant. If

$$\det J_\phi(x) \neq 0,$$

then ϕ is invertible on some neighborhood of x, under the conditions of the inverse function theorem. On that neighborhood the inverse is differentiable, and its Jacobian is the matrix inverse of the forward Jacobian,

$$J_{\phi^{-1}}(y) = J_\phi(x)^{-1}, \qquad y = \phi(x).$$

"The inverse-map Jacobian $J_{\phi^{-1}}(y)$ equals the matrix inverse of the forward-map Jacobian $J_\phi(x)$ at corresponding points $y = \phi(x)$, provided the forward Jacobian has a nonzero determinant."

If we take determinants of this identity, the two correction factors in Theorem 1.3 become reciprocal, $|\det J_{\phi^{-1}}(y)| = 1/|\det J_\phi(x)|$, which is why the two forms of the theorem agree. A nonzero determinant guarantees local invertibility. Global invertibility depends on additional properties across the complete domain, and the map $\phi(x) = x^2$ on the full real line is the standing counterexample, with a derivative that is nonzero everywhere except the origin and no global inverse anywhere.

When a map is many-to-one, several inputs produce the same output, and each of them adds probability to that output. In one dimension the density becomes a sum over branches,

$$p_Y(y) = \sum_{x\,:\,\phi(x)=y} \frac{p_X(x)}{|\phi'(x)|}.$$

"For a many-to-one map ϕ, the output density $p_Y(y)$ sums the input density $p_X(x)$ divided by the absolute derivative $|\phi'(x)|$ over every input x satisfying $\phi(x) = y$."

Each branch contributes probability to the same output. We correct each contribution using the derivative at its own input point, because local stretching can differ between branches.

We return to the squaring map once more, now applied to a random input. Let

$$X \sim \text{Uniform}(-1, 1), \qquad Y = X^2,$$

so that $p_X(x) = 1/2$ on $[-1, 1]$. For an output $0 < y < 1$ the equation $x^2 = y$ has the two solutions $x = +\sqrt{y}$ and $x = -\sqrt{y}$, both inside the support. At either one the derivative is $\phi'(x) = 2x$, so $|\phi'(x)| = 2\sqrt{y}$. The two contributions are equal, and

$$p_Y(y) = \frac{1/2}{2\sqrt{y}} + \frac{1/2}{2\sqrt{y}} = \frac{1}{2\sqrt{y}}, \qquad 0 < y < 1.$$

At $y = 0.25$, the two branches are at $x = \pm 0.5$. Their derivative magnitude is $2(0.5) = 1$, so $p_Y(0.25) = 1/(2 \times 0.5) = 1$. At $y = 0.04$, the branches are at $x = \pm 0.2$. The derivative magnitude is now 0.4, so $p_Y(0.04) = 1/(2 \times 0.2) = 2.5$. The density grows without bound as y approaches zero, because the squaring map compresses a wide band of inputs around the origin into a narrow band of outputs there. Normalization still holds,

$$\int_0^1 \frac{1}{2\sqrt{y}}\, dy = [\sqrt{y}]_0^1 = 1.$$

The branch sum is the continuous counterpart of the merging seen in Section 1.20.1. There, the two states b and c reached the same output v, so we added their probability masses directly. Here the two branches $\pm\sqrt{y}$ reach the same output y. We divide each density contribution by the local stretching factor $|\phi'(x)|$ at its branch before adding them. Discrete masses add directly; continuous densities require this volume correction at each preimage. A related calculation averages reverse jump rates over the clean states that could have produced the current state.

Summing over two branches is manageable. A map that merges many inputs, or merges a different number of them in different regions, turns the sum into a quantity we can neither enumerate nor differentiate, making the likelihood intractable. By choosing invertible maps, we keep these density calculations tractable for training.

To generate a sample through a finite composition, we draw an initial point from the prior and apply each map in turn. The pushforward describes the output distribution. We calculate its density using the Jacobian determinant, adding log-determinants across layers to obtain the exact likelihood. The architecture fixes the number of updates from x_0 to x_1 to x_2, with no intermediate states defined between layers.

A continuous-time flow defines motion at every instant. Its volume correction accumulates infinitesimal changes along the trajectory. To specify such a flow, we assign a direction at every point and time, independently of any numerical step size.

1.27 Scalar Fields and the Gradient

A scalar field assigns one number to each point in space. Its gradient describes the direction in which that number increases most rapidly.

A scalar field is a function

$$f : \mathbb{R}^d \to \mathbb{R}$$

that assigns one real number to every point $x \in \mathbb{R}^d$. Examples include a probability density $p(x)$, a log density $\log p(x)$, a peptide binding score, an energy, and a training loss. To describe local changes in a scalar field, we calculate a partial derivative for every input coordinate. Collecting these d derivatives into a vector gives the gradient.

1.27.1 The gradient

The gradient of a scalar field f at a point x is the d-dimensional vector of its partial derivatives,

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}(x),\ \frac{\partial f}{\partial x_2}(x),\ \ldots,\ \frac{\partial f}{\partial x_d}(x)\right]^\top.$$

"The gradient $\nabla f(x)$ is a vector of d partial derivatives of the scalar field f, with component j measuring how f changes with input coordinate x_j at the point x."

- f is a scalar field on $\mathbb{R}^d$, differentiable at x.
- $x = (x_1, \ldots, x_d)$ is a point of $\mathbb{R}^d$, the location at which the derivatives are taken.
- $\nabla f(x)$ is a vector in $\mathbb{R}^d$, so it is in the same space as x and can be added to a point to move it.

The direction of this vector is the direction of steepest local increase in f, and its magnitude $\|\nabla f(x)\|$ is the rate at which f increases along that direction. A zero gradient marks a point where f has no first-order change in any direction.

Let's compute one. We start with the scalar field

$$f(x_1, x_2) = x_1^2 + 2x_2^2.$$

The partial derivatives are

$$\frac{\partial f}{\partial x_1} = 2x_1, \qquad \frac{\partial f}{\partial x_2} = 4x_2,$$

so the gradient as a function of position is

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}.$$

At the point $(x_1, x_2) = (1, 2)$,

$$\nabla f(1, 2) = \begin{bmatrix} 2 \\ 8 \end{bmatrix}.$$

The second component is four times the first, so at this point f increases four times as fast per unit of movement along x_2 as it does along x_1. The steepest direction is the unit vector $(2, 8)/\sqrt{68} \approx (0.243, 0.970)$, which is close to the x_2 axis.

1.27.2 The directional derivative

From the gradient we know the direction of fastest increase. To find the rate of change along a direction we choose ourselves, we take a unit vector $u \in \mathbb{R}^d$ with $\|u\| = 1$ and define the directional derivative

$$D_u f(x) = \nabla f(x) \cdot u.$$

"The directional derivative $D_u f(x)$ equals the gradient $\nabla f(x)$ dotted with the unit direction vector u, measuring the rate at which the scalar field f changes as we move from the point x along u."

The gradient points along the direction of fastest increase, and the dot product with u is the part of that increase realized along u. Continuing the example with $\nabla f(1,2) = (2,8)^\top$, we choose $u = (1,0)^\top$, the unit step in the positive x_1 direction. Then

$$D_u f(1,2) = \begin{bmatrix} 2 & 8 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2.$$

If we move a small distance h to the right from $(1,2)$, then f increases by about $2h$. Taking the same step along $u = (0,1)^\top$ would increase f by about $8h$, and taking it along $u = (-1,0)^\top$ would decrease f by about $2h$.

A gradient comes from a scalar field. For continuous generative dynamics, we can assign a vector directly at every point without specifying an underlying scalar function.

1.28 Vector Fields

A vector field is a map

$$v : \mathbb{R}^d \to \mathbb{R}^d$$

that assigns a vector $v(x) \in \mathbb{R}^d$ to every point $x \in \mathbb{R}^d$. At each point, $v(x)$ has both a direction and a speed, so a vector field is a complete specification of how a sample at any location should move. Picture a small arrow planted at every point of space; a sample follows the arrows, and the entire population of samples drifts through the field together.

Generative processes change their behavior as generation proceeds, so we allow the arrows to change with time. A time-dependent vector field is written

$$v_t : \mathbb{R}^d \to \mathbb{R}^d, \qquad t \in [0,1],$$

with $v_t(x)$ the vector assigned to the point x at time t. A trajectory x_t moving through this field satisfies the ordinary differential equation

$$\frac{dx_t}{dt} = v_t(x_t).$$

"The instantaneous velocity dx_t/dt of the state x_t at time t equals the time-dependent velocity field v_t evaluated at that current state, $v_t(x_t)$."

- $x_t \in \mathbb{R}^d$ is the position of one sample at time t, a realized value, written in lowercase to keep it distinct from a random state.
- $v_t(x) \in \mathbb{R}^d$ is the velocity assigned to position x at time t, in units of distance per unit time.
- t runs from 0 at the source to 1 at the data end, following the convention of this book.

The right-hand side depends on x_t, so the equation is autonomous in the sense that the field alone, together with a starting point x_0, determines the whole trajectory. This is the continuous-time replacement for a finite chain of maps, and no number of steps appears anywhere in the specification.

Consider the field

$$v(x_1, x_2) = \begin{bmatrix} -x_1 \\ -2x_2 \end{bmatrix},$$

which is time independent. At the point $x = (1, 1)^\top$ the velocity is

$$v(1, 1) = \begin{bmatrix} -1 \\ -2 \end{bmatrix}.$$

Both velocity components are negative at this point with positive coordinates, so the sample moves toward the origin. Its second coordinate contracts twice as fast as its first. If we solve the two decoupled equations $\dot{x}_1 = -x_1$ and $\dot{x}_2 = -2x_2$ starting from $(1, 1)$, we get $x_t = (e^{-t}, e^{-2t})$, a curve that approaches the origin and flattens onto the x_1 axis as it goes.

The arrows change from point to point, and the Jacobian of the field collects those rates of change. For this example,

$$J_v(x) = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix},$$

constant across all of $\mathbb{R}^2$. The Jacobian contains the local directional sensitivities of the field. Its trace measures whether a small region expands or contracts under the motion.

1.29 Divergence

We call this trace the divergence. For a vector field v with components $v(x) = (v_1(x), \ldots, v_d(x))$, the divergence at x is

$$\nabla \cdot v(x) = \sum_{i=1}^{d} \frac{\partial v_i}{\partial x_i}(x) = \operatorname{tr} J_v(x).$$

"The divergence $\nabla \cdot v(x)$ sums, over coordinates i, the derivative of each velocity component v_i with respect to its corresponding coordinate x_i, giving the trace of the velocity Jacobian $J_v(x)$."

The sum includes only the diagonal entries of the Jacobian. A change in v_1 along x_2 shears a small region without changing its area, and that shear stays out of the sum. The three signs have direct geometric readings.

- Positive divergence at x: a small region around x expands as the flow moves it.
- Negative divergence at x: a small region around x contracts.
- Zero divergence at x: a small region around x keeps its volume, although it may rotate or shear.

For the contracting field of Section 1.28, the divergence is $-1 + (-2) = -3$ everywhere, which is why trajectories crowd together as they approach the origin.

Consider instead the field

$$v(x_1, x_2) = \begin{bmatrix} 2x_1 \\ -x_2 \end{bmatrix}.$$

The two relevant partial derivatives are

$$\frac{\partial v_1}{\partial x_1} = 2, \qquad \frac{\partial v_2}{\partial x_2} = -1,$$

so

$$\nabla \cdot v = 2 - 1 = 1.$$

Horizontal stretching at rate 2 outweighs vertical compression at rate 1, and a small region expands at a net rate of 1 per unit time. Over an elapsed time of 0.5, its area is multiplied by $e^{0.5} \approx 1.649$.

Now take the rotational field

$$v(x_1, x_2) = \begin{bmatrix} -x_2 \\ x_1 \end{bmatrix}.$$

Here

$$\nabla \cdot v = \frac{\partial(-x_2)}{\partial x_1} + \frac{\partial x_1}{\partial x_2} = 0 + 0 = 0,$$

because each component varies only along the coordinate it is not differentiated against. Trajectories of this field are circles around the origin, and area is preserved exactly, matching the zero divergence. The two fields are drawn side by side as arrow grids in Figure 1.6.

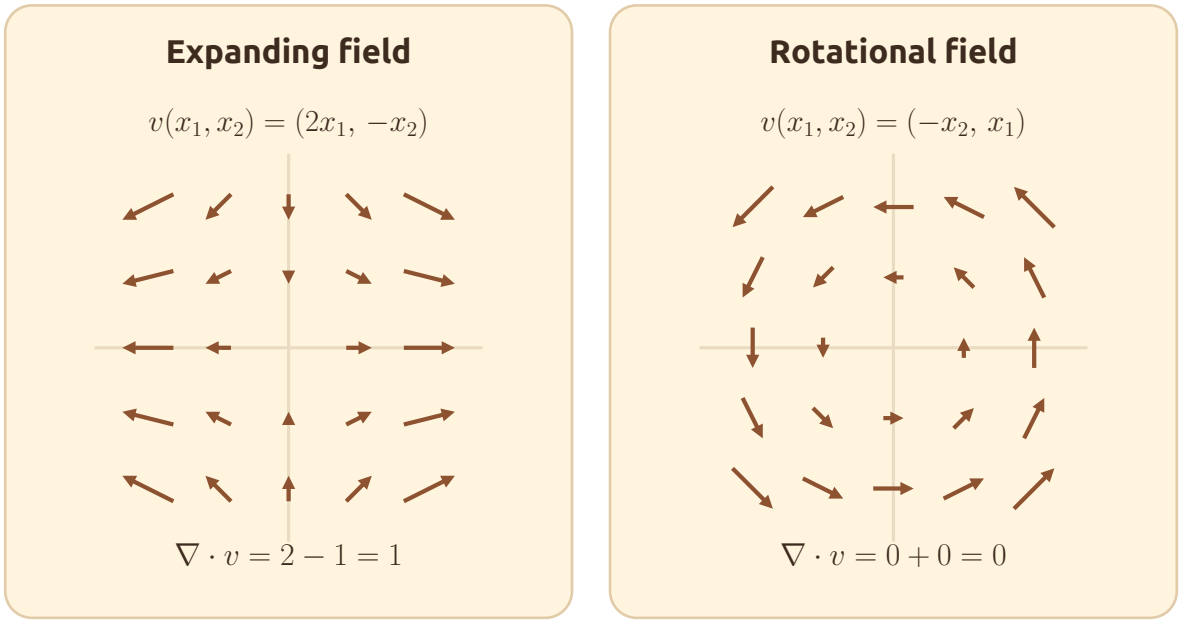


Figure 1.6: Two vector fields on $\mathbb{R}^2$, drawn as arrows on a five-by-five grid, with arrow length proportional to speed. Left: the expanding field $v(x_1, x_2) = (2x_1, -x_2)$, whose divergence $2 - 1 = 1$ is positive, so a small region grows as the flow carries it. Right: the rotational field $v(x_1, x_2) = (-x_2, x_1)$, whose divergence $0 + 0 = 0$ leaves every small region at its original area while turning it about the origin. The faint lines are the coordinate axes.

With the vector field we know how each individual point moves, and with the divergence we know whether nearby trajectories spread or crowd. Next we need the link between that motion and the probability density that moves with it.

1.30 The Continuity Equation

Let $p_t(x)$ be a probability density on $\mathbb{R}^d$ at time t and let $v_t(x)$ be the velocity field that moves the samples. Probability is conserved, since no mass is created or destroyed anywhere and all of it only

moves. If we write that conservation law locally, we get the continuity equation,

$$\boxed{\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x) v_t(x)) = 0}.$$

"The rate of change $\partial p_t(x)/\partial t$ of the density at the fixed location x, plus the divergence of the probability flux $p_t(x)v_t(x)$, equals zero, where the flux is the density $p_t(x)$ multiplied by the velocity $v_t(x)$."

- $p_t(x) \geq 0$ is the density at location x and time t, integrating to one over $\mathbb{R}^d$ at every t.
- $v_t(x) \in \mathbb{R}^d$ is the velocity field, the same one that drives $dx_t/dt = v_t(x_t)$.
- $j_t(x) = p_t(x)v_t(x)$ is the probability flux, a vector field in units of probability per unit area per unit time.

The divergence of the flux at x measures the net outflow of probability from a small region around x. The density decreases when outflow exceeds inflow and increases when inflow exceeds outflow. Setting the sum to zero balances this flow with the local density change at every point and time.

1.30.1 Derivation: the log-density form

The continuity equation describes density changes at a fixed location. To calculate the change along a moving sample, we first expand the probability flux using the product rule:

$$\nabla \cdot (p_t v_t) = v_t^\top \nabla p_t + p_t \, \nabla \cdot v_t,$$

where ∇p_t is the spatial gradient of the density, a vector, and the first term is its dot product with the velocity. Substituting this into the continuity equation and moving the second term to the right,

$$\frac{\partial p_t}{\partial t} + v_t^\top \nabla p_t = -p_t \, \nabla \cdot v_t.$$

Along a moving trajectory, the density changes both with time and with position. For a trajectory satisfying $dx_t/dt = v_t(x_t)$, the chain-rule derivative of $p_t(x_t)$ is therefore the left side. We can write it as

$$\frac{d}{dt} p_t(x_t) = -p_t(x_t) \, \nabla \cdot v_t(x_t).$$

Dividing both sides by $p_t(x_t)$, which is positive wherever a sample can be found, we get

$$\boxed{\frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v_t(x_t)}.$$

"The rate of change of the log density $\log p_t(x_t)$ along the moving state x_t equals the negative divergence of the velocity field v_t at that state, so expansion decreases the density along the trajectory and contraction increases it."

This is the continuous-time counterpart of the log Jacobian-determinant correction derived earlier in this chapter for a single invertible map. A finite map multiplies the density by $|\det J_\phi|^{-1}$; a flow integrates $-\nabla \cdot v_t$ along the trajectory and exponentiates the result. Both record the same fact, that volume change and density change are reciprocal.

1.30.2 A worked flow

Let's take $d = 1$, the velocity field $v(x) = x$, and a source density uniform on $[0, 1]$, so $p_0(x) = 1$ on that interval. The trajectory through x_0 solves $\dot{x}_t = x_t$, so $x_t = x_0 e^t$. At time t, the support has therefore stretched to $[0, e^t]$. The divergence of v is $dv/dx = 1$ everywhere. Substituting it into the log-density equation gives $\frac{d}{dt} \log p_t(x_t) = -1$. Integrating from the initial density gives $p_t(x_t) = e^{-t}$ along every trajectory.

At $t = \log 2 \approx 0.6931$, the support is $[0, 2]$ and the density is $e^{-\log 2} = 0.5$ on it. The total mass is $2 \times 0.5 = 1$, as conservation requires. At $t = 1$ the support is $[0, e] \approx [0, 2.718]$ and the density is $e^{-1} \approx 0.3679$, with mass $2.718 \times 0.3679 = 1.000$. The uniform density has been stretched and thinned by exactly matching factors.

1.31 Score Functions

The score of a density p_t is the gradient of its logarithm with respect to the state:

$$s_t(x) = \nabla_x \log p_t(x).$$

"The score $s_t(x)$ is the gradient of the log density $\log p_t(x)$ with respect to the state x, pointing toward the fastest local increase in log density at time t."

The score is a vector field with the same shape as a velocity field, so we can use it to construct a drift. Because the logarithm is monotone, the score points toward higher density, and its magnitude grows where the density falls off sharply. At a mode of p_t the score is zero.

For the standard Gaussian in one dimension,

$$p(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2},$$

the log density is

$$\log p(x) = -\frac{x^2}{2} - \frac{1}{2} \log(2\pi),$$

and when we differentiate in x, the constant term drops out entirely:

$$\nabla_x \log p(x) = -x.$$

The score of the standard Gaussian at x equals $-x$, so it always points from x back toward the high-density center at zero, and its magnitude is the distance to that center. At $x = 3$ the score is -3, a strong pull to the left. At $x = -2$ the score is 2, a weaker pull to the right. At $x = 0$ the score vanishes.

The same calculation for a general Gaussian $\mathcal{N}(\mu, \sigma^2)$, with mean μ and variance σ^2, produces

$$\nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2},$$

so the pull is toward μ and is scaled down by the variance. With $\mu = 1$ and $\sigma^2 = 4$, the score at $x = 5$ is $-(5 - 1)/4 = -1$. The point is four units from the mean, in a Gaussian wide enough that the restoring pull is modest.

We can estimate the score from noisy samples and use it to reverse a corruption process. This is possible because the score depends only on the density's shape: differentiating removes the normalizing constant, as it removed the $\frac{1}{2}\log(2\pi)$ term above.

1.32 The Laplacian and Diffusion

A gradient describes first-order change. A flow built from a velocity field alone moves probability without spreading it. Random motion spreads probability from concentrated regions into less dense neighboring regions. The amount of spreading at a point depends on the local curvature of the density. We measure this curvature with the Laplacian, a second-order differential operator.

For a scalar field f, the Laplacian is the divergence of the gradient, written ∇^2 in this book because Δ already names the probability simplex,

$$\nabla^2 f(x) = \nabla \cdot \nabla f(x) = \sum_{i=1}^{d} \frac{\partial^2 f}{\partial x_i^2}(x).$$

"The Laplacian $\nabla^2 f(x)$ is the divergence of the gradient of the scalar field f, equal to the sum of its second partial derivatives with respect to all input coordinates x_i."

The value of $\nabla^2 f$ at a point relates f there to its values in the immediate surroundings. A positive Laplacian occurs where the point lies below the average of its neighbors, a negative Laplacian where it lies above them, and a zero Laplacian where the curvatures in different directions cancel.

Consider $f(x_1, x_2) = x_1^2 + 3x_2^2$. The second derivatives are

$$\frac{\partial^2 f}{\partial x_1^2} = 2, \qquad \frac{\partial^2 f}{\partial x_2^2} = 6, \qquad \text{so} \qquad \nabla^2 f = 2 + 6 = 8$$

everywhere, both terms being constant. Every point of this bowl lies below its surroundings. Now take the saddle $g(x_1, x_2) = x_1^2 - x_2^2$, whose second derivatives are 2 and -2, so that

$$\nabla^2 g = 2 - 2 = 0.$$

The positive curvature along x_1 cancels the negative curvature along x_2, and every point of the saddle is exactly at the average of its neighbors even though the surface is far from flat.

1.32.1 The heat equation

The simplest equation describing diffusion sets the rate of change of a density proportional to its Laplacian,

$$\frac{\partial p_t(x)}{\partial t} = D\, \nabla^2 p_t(x),$$

with $D > 0$ the diffusion coefficient, in units of squared distance per unit time.

"The time derivative $\partial p_t(x)/\partial t$ of the density equals the diffusion coefficient D times the density Laplacian $\nabla^2 p_t(x)$, so density decreases at sharp peaks and increases in nearby lower-density regions."

At a sharp peak the Laplacian is negative, so the density there decreases. In a nearby valley the Laplacian is positive, so the density there increases. Repeating this everywhere smooths the distribution as time passes, and in the limit it flattens completely.

A Gaussian that widens at the right rate solves this equation exactly. We can take $d = 1$, $D = 1/2$, and $p_t = \mathcal{N}(0, t)$, so that the variance equals the elapsed time. At the center,

$$p_t(0) = \frac{1}{\sqrt{2\pi t}}, \qquad \frac{\partial p_t}{\partial t}(0) = -\frac{1}{2}\frac{1}{\sqrt{2\pi}} t^{-3/2}.$$

At $t = 1$ these are $p_1(0) = 0.39894$ and $\partial_t p_1(0) = -0.19947$. At the center of a one-dimensional Gaussian with variance σ^2, the Laplacian is $-p(0)/\sigma^2$. At $t = 1$, this equals -0.39894. Multiplying by the diffusion coefficient gives

$$D\,\nabla^2 p_1(0) = 0.5 \times (-0.39894) = -0.19947,$$

matching the time derivative to every digit carried. Mass drains out of the center at a rate fixed by the curvature there.

1.32.2 Drift and diffusion together

A stochastic process can contain directed motion and random spreading at the same time. Writing both in one line,

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t,$$

where $X_t \in \mathbb{R}^d$ is the random state, $b_t(x) \in \mathbb{R}^d$ is the drift, $\sigma_t > 0$ is the noise magnitude, and W_t is a standard Brownian motion in $\mathbb{R}^d$.

"The infinitesimal state change dX_t equals the drift $b_t(X_t)$ at the current random state X_t times the time increment dt, plus the Brownian increment dW_t multiplied by the noise magnitude σ_t."

The density of such a process obeys an equation that combines the two mechanisms derived separately above,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (b_t(x) p_t(x)) + \frac{1}{2}\sigma_t^2\,\nabla^2 p_t(x).$$

"The time derivative of the density $p_t(x)$ equals the negative divergence of the drift flux $b_t(x)p_t(x)$, plus one half the squared noise magnitude σ_t^2 times the density Laplacian $\nabla^2 p_t(x)$, combining directed motion and random spreading."

This is the Fokker-Planck equation for isotropic diffusion. Its first term is the continuity equation with b_t in place of v_t, moving density along the drift. Its second term is the heat equation with $D = \sigma_t^2/2$, smoothing the density. Setting $\sigma_t = 0$ recovers the continuity equation exactly, so a deterministic flow is the special case of a stochastic process with no noise.

The gradient describes how a scalar score or a density changes across space. The score is the gradient of the log density, a vector field pointing uphill in probability. A velocity field sets the speed and direction of every sample, and its divergence controls both local expansion and the change in log density along a trajectory. The Laplacian drives diffusive smoothing, and the Fokker-Planck equation combines directed transport with random spreading in a single statement about the density.

We can now describe how probability mass moves. To train a model, we also need a numerical measure of the mismatch between distributions. We can minimize information-theoretic measures during training.

1.33 Information, Entropy and Cross-Entropy

To compare probability distributions, we measure uncertainty within one distribution, the cost of scoring its samples with another distribution, and the discrepancy between the two. We derive these quantities from the information in a single observation.

1.33.1 Information content

The information associated with observing a state x under a distribution p is

$$I(x) = -\log p(x),$$

measured in nats because the logarithm is natural throughout this book.

"The information $I(x)$ in the observed state x equals the negative logarithm of its probability $p(x)$, so observing a less probable state produces more information."

The negative sign makes $I(x)$ nonnegative, since $p(x) \leq 1$. A certain event with $p(x) = 1$ has zero information, and the quantity grows without bound as $p(x)$ approaches zero.

For an outcome of probability 0.5 and for one of probability 0.01,

$$I(x) = -\log 0.5 = 0.693 \text{ nats}, \qquad I(x) = -\log 0.01 = 4.605 \text{ nats}.$$

The rarer event is more surprising by a factor of about 6.6 in information, even though its probability is fifty times smaller. The logarithm compresses the scale, which makes information additive over independent observations.

1.33.2 Entropy

Information content describes one observed outcome. If we average it over all outcomes, weighted by the same distribution that generated them, we get the uncertainty of the distribution as a whole. For a distribution p on a finite state space $\mathcal{X}$, the entropy is

$$H(p) = \mathbb{E}_{X \sim p}\big[-\log p(X)\big] = -\sum_{x \in \mathcal{X}} p(x) \log p(x).$$

"The entropy $H(p)$ averages the information $-\log p(x)$ over states x in the state space $\mathcal{X}$, weighting each state by its probability $p(x)$."

Entropy is high when probability is spread across many states and low when it is concentrated on a few. A deterministic distribution, with all mass on one state, has entropy zero, since the single term is $1 \times \log 1 = 0$. A uniform distribution over K states has entropy

$$H(p) = -\sum_{i=1}^{K} \frac{1}{K} \log \frac{1}{K} = \log K,$$

the largest possible entropy on K states.

We take the two-state distribution $p = (0.9, 0.1)$. Its entropy is

$$H(p) = -\big(0.9 \log 0.9 + 0.1 \log 0.1\big) = -\big(0.9 \times (-0.10536) + 0.1 \times (-2.30259)\big),$$

which evaluates to

$$H(p) = 0.09482 + 0.23026 = 0.325 \text{ nats}.$$

Now take the balanced distribution $p = (0.5, 0.5)$:

$$H(p) = -\big(0.5 \log 0.5 + 0.5 \log 0.5\big) = \log 2 = 0.693 \text{ nats}.$$

The balanced distribution has more than twice the uncertainty of the skewed one, because neither outcome is strongly preferred and a sample from it is correspondingly harder to anticipate.

For a continuous density p on $\mathbb{R}^d$, the sum becomes an integral and the quantity is called differential entropy,

$$h(p) = -\int_{\mathbb{R}^d} p(x) \log p(x)\, dx.$$

Differential entropy can be negative, and it changes when the coordinates are changed. Both properties follow from the fact that a density has units, so rescaling x rescales p and shifts h by the log of the Jacobian determinant. Discrete entropy is nonnegative and does not depend on a choice of coordinates [Shannon, 1948].

1.33.3 Cross-entropy

Entropy uses the same distribution to draw and score samples. During model training, we draw samples from the data distribution and score them with the model distribution. Suppose samples follow p while a model assigns probabilities according to q. Their cross-entropy is

$$H(p, q) = \mathbb{E}_{X \sim p}\big[-\log q(X)\big] = -\sum_{x \in \mathcal{X}} p(x) \log q(x).$$

"The cross-entropy $H(p, q)$ averages the negative log probability $-\log q(x)$ assigned by the scoring distribution q over outcomes x drawn from the sampling distribution p."

In this notation, the first distribution determines the averaging weights, and the second determines the probabilities used to score each outcome. Setting $q = p$ recovers the entropy, $H(p, p) = H(p)$.

Let

$$p = (0.8, 0.2), \qquad q = (0.6, 0.4)$$

be two distributions on the same pair of states. The cross-entropy is

$$H(p, q) = -\big(0.8 \log 0.6 + 0.2 \log 0.4\big) = 0.8 \times 0.51083 + 0.2 \times 0.91629,$$

so

$$H(p, q) = 0.40866 + 0.18326 = 0.592 \text{ nats}.$$

Scoring samples from p with the probabilities of q costs about 0.592 nats on average. The entropy of p itself is

$$H(p) = -\big(0.8 \log 0.8 + 0.2 \log 0.2\big) = 0.17851 + 0.32189 = 0.500 \text{ nats},$$

so scoring the same samples with the correct distribution costs 0.500 nats. Using q adds $0.592 - 0.500 = 0.092$ nats to that cost. We measure this excess over the entropy of p with the KL divergence between p and q.

1.34 Kullback-Leibler Divergence

The Kullback-Leibler divergence from p to q is the expected log ratio of the two probabilities, taken under p [Kullback and Leibler, 1951]:

$$D_{\mathrm{KL}}(p \,\|\, q) = \mathbb{E}_{X \sim p}\left[\log \frac{p(X)}{q(X)}\right] = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}.$$

"The KL divergence $D_{\mathrm{KL}}(p \,\|\, q)$ averages the log ratio of the reference probability $p(x)$ to the comparison probability $q(x)$ over states x drawn from p, measuring the discrepancy between the distributions."

- p is the reference distribution, the one the samples are drawn from and whose values weight the sum.
- q is the comparison distribution, the one whose probabilities are being judged.
- The sum runs over states where $p(x) > 0$; a state with $p(x) = 0$ contributes nothing, with the convention $0 \log 0 = 0$.

If $q(x) = 0$ at a state where $p(x) > 0$, the ratio is infinite and so is the divergence. The comparison distribution assigns zero probability to an event that occurs under the reference distribution. Agreement at other states cannot make the divergence finite.

Continue with $p = (0.8, 0.2)$ and $q = (0.6, 0.4)$. Then

$$D_{\mathrm{KL}}(p \,\|\, q) = 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} = 0.8 \times 0.28768 + 0.2 \times (-0.69315),$$

which evaluates to

$$D_{\mathrm{KL}}(p \,\|\, q) = 0.23015 - 0.13863 = 0.0915 \text{ nats}.$$

The second term is negative, because q assigns more probability than p does to the second state, and the first term outweighs it. A value of zero would mean that q matches p at every state. The number 0.0915 agrees with the 0.092 nats of excess cost found in Section 1.33, up to rounding. Figure 1.7 compares all three quantities for this pair of distributions.

1.34.1 Cross-entropy decomposition

We can expand the log ratio inside the KL definition into a difference of logarithms:

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_{x} p(x)\big(\log p(x) - \log q(x)\big) = \sum_{x} p(x) \log p(x) - \sum_{x} p(x) \log q(x).$$

The first sum is $-H(p)$ and the second is $-H(p, q)$, so

$$D_{\mathrm{KL}}(p \,\|\, q) = -H(p) + H(p, q).$$

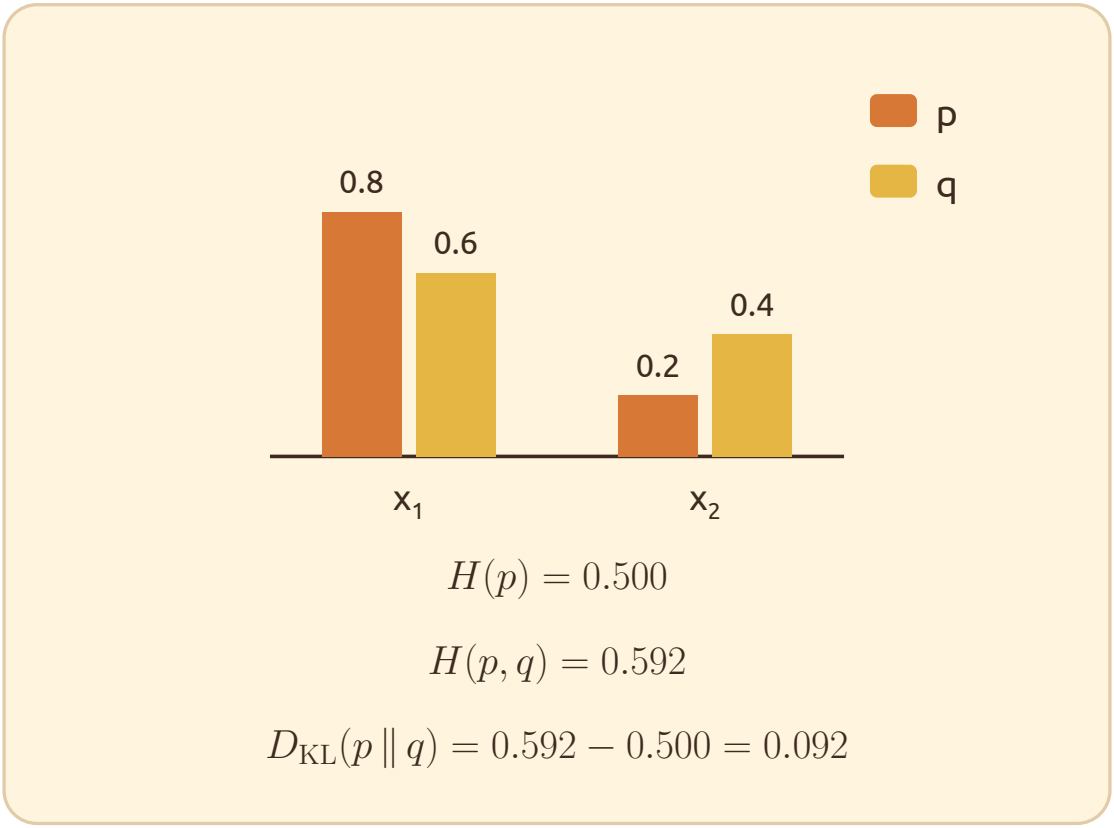


Figure 1.7: Two distributions over the same two states x_1 and x_2, drawn as paired bars: the reference $p = (0.8, 0.2)$ in the darker color and the comparison $q = (0.6, 0.4)$ in the lighter one, with each bar labeled by its probability. Below the bars are the entropy $H(p) = 0.500$ nats, the cross-entropy $H(p, q) = 0.592$ nats, and their difference, the KL divergence $D_{\mathrm{KL}}(p \,\|\, q) = 0.092$ nats.

Rearranging separates cross-entropy into entropy and KL divergence:

$$\boxed{H(p, q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)}.$$

"The cross-entropy $H(p, q)$ equals the entropy $H(p)$ of the sampling distribution p, plus the KL divergence $D_{\mathrm{KL}}(p \,\|\, q)$ between p and the scoring distribution q."

With a fixed data distribution, $H(p)$ is constant. We can therefore reduce the cost only by reducing the KL divergence through changes to q. For the running pair, $0.500 + 0.0915 = 0.592$, which is the cross-entropy computed directly.

1.34.2 Nonnegativity

KL divergence is nonnegative, so scoring samples from p with any distribution has an expected cost at least as large as the entropy of p. We can prove this with a logarithm inequality.

Proposition 1.5 (Nonnegativity of KL divergence)**.** *For any two distributions p and q on a common finite state space, $D_{\mathrm{KL}}(p \,\|\, q) \geq 0$, with equality exactly when $p(x) = q(x)$ at every state with $p(x) > 0$.*

Proof. The elementary inequality $\log t \le t - 1$ holds for all $t > 0$, with equality only at $t = 1$. Apply it to $t = q(x)/p(x)$ at every state where $p(x) > 0$:

$$-D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log \frac{q(x)}{p(x)} \le \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right).$$

The right-hand side collapses, since the factors of $p(x)$ cancel in the first term,

$$\sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) = \sum_x q(x) - \sum_x p(x) \le 1 - 1 = 0.$$

Therefore $-D_{\mathrm{KL}}(p \,\|\, q) \le 0$, proving nonnegativity. Equality in $\log t \le t - 1$ requires $t = 1$ at every state with positive probability under p, so the two distributions agree there. □

Nonnegativity allows the KL divergence to serve as a discrepancy. It is not a distance, because it fails the symmetry axiom, and the direction in which it is taken changes what it penalizes.

1.34.3 Asymmetry

In general

$$D_{\mathrm{KL}}(p \,\|\, q) \ne D_{\mathrm{KL}}(q \,\|\, p)\,,$$

because the expectation is taken under the first argument, so the first argument fixes which regions of the state space receive weight. Reversing the running example,

$$D_{\mathrm{KL}}(q \,\|\, p) = 0.6 \log \frac{0.6}{0.8} + 0.4 \log \frac{0.4}{0.2} = 0.6 \times (-0.28768) + 0.4 \times 0.69315,$$

so

$$D_{\mathrm{KL}}(q \,\|\, p) = -0.17261 + 0.27726 = 0.1046 \text{ nats},$$

against $D_{\mathrm{KL}}(p \,\|\, q) = 0.0915$ nats for the same pair. The reversed direction is about 14 percent larger here, and for distributions with near-zero entries the two directions can differ without bound. Choosing a direction is a modeling decision, and the most common choice for training places the real data in the first slot and the model in the second.

1.35 Maximum Likelihood as KL Minimization

Let p_{data} be the data distribution and p_θ the model distribution with parameters θ. The population negative log-likelihood is

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\mathrm{data}}} [\, -\log p_\theta(X)].$$

"The population loss $\mathcal{L}(\theta)$ averages the negative log probability $-\log p_\theta(X)$ assigned by the model with parameters θ over examples X drawn from the data distribution p_{data}."

The right-hand side is a cross-entropy with the data as the sampling distribution and the model as the scoring distribution,

$$\mathcal{L}(\theta) = H(p_{\mathrm{data}}, p_\theta),$$

and the identity of Section 1.34 splits it into a constant and a divergence:

$$\mathcal{L}(\theta) = H(p_{\mathrm{data}}) + D_{\mathrm{KL}}(p_{\mathrm{data}} \,\|\, p_\theta)\,.$$

The entropy $H(p_{\text{data}})$ does not depend on θ, so minimizing the negative log-likelihood over θ is the same problem as minimizing

$$D_{\text{KL}}(p_{\text{data}} \,\|\, p_\theta)\,.$$

Maximizing the expected log-likelihood therefore minimizes the KL divergence from the data distribution to the model.

With a finite dataset of N examples $x^{(1)}, \ldots, x^{(N)}$ drawn from p_{data}, the expectation becomes an average and the trainable objective is

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(x^{(i)}).$$

"The empirical negative log-likelihood $\mathcal{L}(\theta)$ averages the negative log probabilities $-\log p_\theta(x^{(i)})$ assigned by the model with parameters θ across the N training examples $x^{(i)}$."

Consider a dataset of ten single DNA bases with counts $n(\text{A}) = 4$, $n(\text{C}) = 3$, $n(\text{G}) = 2$ and $n(\text{T}) = 1$, and a model that is one categorical distribution over the four bases. Grouping the sum by state, the empirical loss is

$$\mathcal{L}(\theta) = -\frac{1}{10}\Big(4\log p_\theta(\text{A}) + 3\log p_\theta(\text{C}) + 2\log p_\theta(\text{G}) + 1\log p_\theta(\text{T})\Big).$$

If we evaluate it at the empirical frequencies $\hat{p} = (0.4, 0.3, 0.2, 0.1)$, we have four terms,

$$\begin{aligned} 0.4 \times 0.91629 &= 0.36652, & 0.3 \times 1.20397 &= 0.36119, \\ 0.2 \times 1.60944 &= 0.32189, & 0.1 \times 2.30259 &= 0.23026, \end{aligned}$$

which sum to $\mathcal{L} = 1.280$ nats. A uniform model $q = (0.25, 0.25, 0.25, 0.25)$ scores the same data at $\log 4 = 1.386$ nats. The gap of $1.386 - 1.280 = 0.1064$ nats is exactly $D_{\text{KL}}(\hat{p} \,\|\, q)$, the divergence from the empirical distribution to the uniform model, which is the decomposition of Section 1.34 evaluated numerically. The empirical distribution $\hat{p}$ minimizes this loss over all categorical models.

We use these quantities in several generative objectives. Autoregressive models minimize a token-level cross-entropy. Normalizing flows maximize an exact likelihood after the Jacobian-determinant correction of the change of variables. Variational objectives contain KL divergences between an approximate posterior and a target. Diffusion derivations contain KL terms between forward and reverse transitions, and discrete diffusion objectives collapse to a weighted cross-entropy. Guidance modifies sampling through property-dependent log probabilities. Entropy is the uncertainty inside a distribution, cross-entropy is the cost of predicting with the wrong one, and KL divergence isolates the mismatch between them.

How to minimize those losses is a separate matter, particularly when the quantity being optimized is itself a probability distribution and must stay one.

1.36 Optimization Without Constraints

Training a generative model means choosing parameters that minimize a loss. Some quantities require constraints: probabilities must be nonnegative and sum to one. We may also require

generated samples to have specified biological properties or a dynamical system to satisfy stability or conservation conditions. We use constrained optimization for these requirements and unconstrained optimization when parameters can vary freely.

An unconstrained problem has the form

$$\min_{\theta} \ \mathcal{L}(\theta),$$

with $\theta \in \mathbb{R}^n$ the parameter vector and $\mathcal{L}$ a differentiable loss. At a differentiable local minimum θ^*, no direction of first-order descent exists, which is the stationarity condition

$$\nabla \mathcal{L}(\theta^*) = 0.$$

The gradient points toward increasing loss, so a descent step moves against it:

$$\theta_{k+1} = \theta_k - \eta \, \nabla \mathcal{L}(\theta_k).$$

"The next parameter vector θ_{k+1} equals the current vector θ_k minus the learning rate η times the loss gradient $\nabla \mathcal{L}(\theta_k)$, moving against the direction of increasing loss at optimization step k."

Here k indexes optimization steps, distinct from the time t of the generative process, and $\eta > 0$ is the learning rate.

Consider the one-dimensional loss $\mathcal{L}(\theta) = (\theta - 3)^2$, whose derivative is $\mathcal{L}'(\theta) = 2(\theta - 3)$. Starting from $\theta_0 = 0$ with $\eta = 0.1$, the update is

$$\theta_{k+1} = \theta_k - 0.1 \times 2(\theta_k - 3) = 0.8\,\theta_k + 0.6.$$

Three steps produce

$$\theta_1 = 0.600, \qquad \theta_2 = 1.080, \qquad \theta_3 = 1.464.$$

Subtracting 3 from both sides of the update rule gives $\theta_{k+1} - 3 = 0.8(\theta_k - 3)$. The error therefore contracts by the same factor at every step. After k steps, $\theta_k - 3 = 0.8^k(\theta_0 - 3)$. The iterates thus approach the minimizer $\theta^* = 3$ geometrically. At $k = 3$ the closed form $\theta_k = 3 - 3(0.8)^k$ returns $3 - 3(0.512) = 1.464$, matching the iteration.

Many problems also require an equality to hold at every point of the search, which the plain stationarity condition cannot express.

1.37 Equality Constraints and Lagrange Multipliers

An equality-constrained problem has the form

$$\min_{x} \ f(x) \quad \text{subject to} \quad g(x) = c,$$

with $f : \mathbb{R}^n \to \mathbb{R}$ the objective, $g : \mathbb{R}^n \to \mathbb{R}$ the constraint function, and $c \in \mathbb{R}$ the required value. We can introduce a scalar λ, the Lagrange multiplier, and define the Lagrangian

$$\mathcal{L}(x, \lambda) = f(x) + \lambda(g(x) - c).$$

"The Lagrangian $\mathcal{L}(x, \lambda)$ adds the objective value $f(x)$ to the multiplier λ times the constraint error $g(x) - c$, where $g(x)$ is the constrained quantity and c is its required value."

Setting both partial derivatives to zero recovers the problem together with its solution condition:

$$\nabla_x \mathcal{L} = \nabla f(x) + \lambda \nabla g(x) = 0, \qquad \frac{\partial \mathcal{L}}{\partial \lambda} = g(x) - c = 0.$$

The second condition is the constraint itself. The first rearranges to

$$\nabla f(x) = -\lambda \nabla g(x),$$

so at a constrained optimum the two gradients are parallel. A movement along the constraint surface is orthogonal to ∇g. If ∇f had a component along that surface, we could move against it to lower the objective while preserving the constraint. At the constrained optimum, this component must therefore vanish.

1.37.1 A numerical example

Let's minimize the squared distance to the origin along a line,

$$f(x_1, x_2) = x_1^2 + x_2^2 \quad \text{subject to} \quad x_1 + x_2 = 1.$$

The Lagrangian is

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 1).$$

Differentiating in each variable,

$$\frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 + \lambda = 0, \qquad \frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 + \lambda = 0, \qquad \frac{\partial \mathcal{L}}{\partial \lambda} = x_1 + x_2 - 1 = 0.$$

The first two equations both solve to $-\lambda/2$, so $x_1 = x_2$. Using the constraint, $2x_1 = 1$, and therefore

$$x_1 = x_2 = \frac{1}{2}, \qquad \lambda = -1.$$

The minimum value is $f = 0.25 + 0.25 = 0.5$, against $f = 1$ at either endpoint $(1, 0)$ or $(0, 1)$ of the constraint line. The multiplier $\lambda = -1$ is the rate at which the optimal value would change if the required total $c = 1$ were raised.

1.37.2 Normalization as a constraint

Probability normalization is an equality constraint of exactly this type. Consider a probability vector p over K states, constrained by

$$\sum_{i=1}^{K} p_i = 1.$$

We can look for the distribution of greatest entropy under that single requirement, which is the problem

$$\max_{p} \; -\sum_{i=1}^{K} p_i \log p_i \quad \text{subject to} \quad \sum_{i=1}^{K} p_i = 1.$$

The Lagrangian, written for the entropy as the objective, is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right).$$

Differentiating with respect to a single p_i, and using $d(p \log p)/dp = \log p + 1$,

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0,$$

so that

$$\log p_i = \lambda - 1, \qquad p_i = e^{\lambda - 1}.$$

The right-hand side has no index i, so every p_i takes the same value. Normalization then requires

$$p_i = \frac{1}{K} \quad \text{for every } i,$$

and the maximum entropy is $H = \log K$. The uniform distribution has the largest entropy available on K states, which is the fact asserted in Section 1.33.

Let's calculate the categorical maximum-likelihood estimate. Minimizing the empirical negative log-likelihood $-\frac{1}{N}\sum_x n(x) \log p(x)$ over a categorical model, subject to $\sum_x p(x) = 1$, has the Lagrangian

$$\mathcal{L}(p, \lambda) = -\frac{1}{N} \sum_x n(x) \log p(x) + \lambda \left(\sum_x p(x) - 1 \right),$$

whose stationarity condition $-n(x)/(Np(x)) + \lambda = 0$ yields $p(x) = n(x)/(N\lambda)$. Summing over states gives $\lambda = 1$, so the minimizer is the empirical frequency $p(x) = n(x)/N$. For the counts of Section 1.35 that is $\hat{p} = (0.4, 0.3, 0.2, 0.1)$, and the minimum loss is its entropy, 1.280 nats.

1.37.3 Maximum entropy under property constraints

In biological design, we often require a sequence distribution to satisfy property targets in addition to normalization. Let $h_j(x)$ be a function that measures property j of state x, for $j = 1, \ldots, m$, and let c_j be the desired expected value of that property,

$$\sum_x p(x) h_j(x) = c_j.$$

"The expected property value sums the probability $p(x)$ times the value $h_j(x)$ of property j over all states x, and must equal the target c_j."

The problem is to find the highest-entropy distribution consistent with all m property targets and with normalization. Its Lagrangian has one multiplier per constraint,

$$\mathcal{L}(p, \lambda, \lambda_0) = -\sum_x p(x) \log p(x) + \sum_{j=1}^{m} \lambda_j \left(\sum_x p(x) h_j(x) - c_j \right) + \lambda_0 \left(\sum_x p(x) - 1 \right).$$

Differentiating with respect to $p(x)$ at a single state,

$$-\log p(x) - 1 + \sum_{j=1}^{m} \lambda_j h_j(x) + \lambda_0 = 0,$$

and if we solve for $p(x)$ this leads to an exponential family.

Theorem 1.6 (Maximum-entropy distribution under expectation constraints)**.** *Among all distributions on $\mathcal{X}$ satisfying $\sum_x p(x)h_j(x) = c_j$ for $j = 1, \dots, m$, the one of greatest entropy is*

$$p^*(x) = \frac{1}{Z(\lambda)} \exp\left(\sum_{j=1}^{m} \lambda_j h_j(x)\right), \qquad Z(\lambda) = \sum_{x \in \mathcal{X}} \exp\left(\sum_{j=1}^{m} \lambda_j h_j(x)\right),$$

where the multipliers $\lambda = (\lambda_1, \dots, \lambda_m)$ are fixed by requiring the constraints to hold [Jaynes, 1957].

"The maximum-entropy probability $p^*(x)$ is proportional to the exponential of the sum of the m property values $h_j(x)$, each weighted by its multiplier λ_j, and the partition function $Z(\lambda)$ normalizes these weights over all states x."

Let's work one case through with numbers. We take a single DNA position, so $\mathcal{X} = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$, with one property, the indicator of a strong base,

$$h(x) = \begin{cases} 1 & x \in \{\mathrm{G}, \mathrm{C}\} \\ 0 & x \in \{\mathrm{A}, \mathrm{T}\} \end{cases},$$

and the target GC content $c = 0.6$. In the maximum-entropy form the two strong bases have $p(\mathrm{G}) = p(\mathrm{C}) = e^{\lambda}/Z$ and the two weak bases have $p(\mathrm{A}) = p(\mathrm{T}) = 1/Z$, with $Z = 2e^{\lambda} + 2$. The constraint reads

$$\frac{2e^{\lambda}}{2e^{\lambda} + 2} = 0.6,$$

so $2e^{\lambda} = 1.2e^{\lambda} + 1.2$, hence $0.8e^{\lambda} = 1.2$ and $e^{\lambda} = 1.5$, that is $\lambda = \log 1.5 = 0.4055$. The partition function is $Z = 2(1.5) + 2 = 5$, and the distribution is

$$p^* = \big(p(\mathrm{A}), p(\mathrm{C}), p(\mathrm{G}), p(\mathrm{T})\big) = (0.2,\ 0.3,\ 0.3,\ 0.2).$$

Its GC content is $0.3 + 0.3 = 0.6$, as required. Its entropy is

$$H(p^*) = -2(0.3 \log 0.3) - 2(0.2 \log 0.2) = 0.72238 + 0.64378 = 1.366 \text{ nats},$$

against $\log 4 = 1.386$ nats for the unconstrained uniform distribution. Enforcing a GC content of 0.6 reduces entropy by 0.0201 nats while moving 10 percentage points of mass. A positive λ tilts the distribution toward states of high property value, and setting $\lambda = 0$ returns the uniform distribution.

An equality constraint sets an exact target value. For design requirements expressed as bounds, we use inequality constraints.

1.38 Inequality Constraints and the KKT Conditions

An inequality-constrained problem has the form

$$\min_x \ f(x) \quad \text{subject to} \quad g(x) \le 0,$$

with the convention that the constraint is written as a quantity that must stay at or below zero. The Lagrangian keeps the same shape,

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x),$$

and the solution conditions acquire two extra requirements.

Definition 1.7 (Karush-Kuhn-Tucker conditions)**.** A point x^* with multiplier λ satisfies the KKT conditions for the problem above when

$$\nabla f(x^*) + \lambda \nabla g(x^*) = 0, \qquad g(x^*) \le 0, \qquad \lambda \ge 0, \qquad \lambda\, g(x^*) = 0.$$

"Complementary slackness requires the multiplier λ times the constraint value $g(x^*)$ at the candidate solution x^* to equal zero: an inactive constraint with $g(x^*) < 0$ has $\lambda = 0$, while an active constraint with $g(x^*) = 0$ may have a positive multiplier."

The sign requirement $\lambda \ge 0$ distinguishes this case from the equality case, where λ could have either sign. An inequality restricts the solution from one side, which determines the allowed sign of the multiplier.

We can take the one-dimensional problem

$$\min_x \; f(x) = (x-3)^2 \quad \text{subject to} \quad x \le 1,$$

which in standard form has $g(x) = x - 1 \le 0$. The unconstrained minimizer $x = 3$ violates the constraint, so the solution is at the boundary, $x^* = 1$. The Lagrangian is

$$\mathcal{L}(x, \lambda) = (x-3)^2 + \lambda(x-1),$$

and stationarity requires $2(x-3) + \lambda = 0$. Evaluating that at $x = 1$,

$$2(1-3) + \lambda = 0, \qquad \text{so} \qquad \lambda = 4.$$

All four KKT conditions hold: stationarity by construction, $g(1) = 0$, $\lambda = 4 \ge 0$, and $\lambda g(1) = 0$. The objective value rises from 0 at the unconstrained optimum to $f(1) = 4$ under the bound, and the positive multiplier measures how strongly the active constraint moves the solution. Relaxing the bound to $x \le 1 + \epsilon$ would lower the optimal value by about 4ϵ for small ϵ.

When we design a peptide, we may require predicted toxicity below a threshold, stability above a threshold, or sequence identity to a known natural sequence below a chosen maximum.

1.38.1 Optimization over the simplex

For a probability vector, the constraint set is the simplex:

$$p_i \ge 0 \;\text{ for every } i, \qquad \sum_{i=1}^{K} p_i = 1,$$

which is K inequality constraints together with one equality constraint. We have two routes for keeping a model inside this set.

We can optimize directly with Lagrange multipliers, as in the maximum-entropy derivation. Alternatively, we can ensure that the constraints hold by parameterizing the distribution with real-valued logits $z \in \mathbb{R}^K$ and setting

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}},$$

the softmax of Section 1.13. Every p_i produced this way is strictly positive because the exponential never vanishes. Dividing by the denominator normalizes the sum to one. We can then apply unconstrained gradient descent to z, with valid probabilities at every step. This is why discrete models predict logits.

1.39 Generative Process Construction

We can construct a generative process by specifying an initial distribution, dynamics, distribution evolution, and a training loss.

Let's start with a simple distribution that is easy to sample,

$$x_0 \sim p_0.$$

A Gaussian in $\mathbb{R}^d$ or a uniform distribution over $\mathcal{V}^L$ will do, with the only requirements being that a draw is cheap and that the density is known in closed form.

Next, we construct a process that moves those samples through time. In the deterministic case the process is the ordinary differential equation driven by a velocity field,

$$\frac{dx_t}{dt} = v_t(x_t),$$

and in the stochastic case it has a Brownian term as well,

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t.$$

We then track the distribution that moves with the samples. For the deterministic process the density obeys the continuity equation,

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t v_t) = 0,$$

and for the stochastic process it obeys the Fokker-Planck equation, which adds the Laplacian term $\frac{1}{2}\sigma_t^2 \nabla^2 p_t$ to the right-hand side. Either way, the density at time 1 is determined by p_0 and by the field that transports it, with no separate modeling choice.

Last, we learn the unknown field by minimizing an expected loss,

$$\mathcal{L}(\theta) = \mathbb{E}\big[\ell(\text{target},\ \text{model prediction}_\theta)\big],$$

with the expectation taken over a time t, a sample path, and the data. The loss measures the mismatch between what the process requires at a given state and time, and what the network predicts there. For a vector prediction, we solve an unconstrained problem in θ. For a distribution prediction, we must keep the output in the simplex. The softmax parameterization of Section 1.38 ensures this, allowing us to optimize unconstrained logits.

A distribution assigns probability mass to states. We use expectation to calculate average losses and measured properties, variance to measure fluctuations, and covariance to measure how quantities vary together. Conditioning and Bayes' rule update probabilities after an observation and connect forward corruption to reverse denoising.

The simplex contains valid discrete distributions. Matrices and their eigenvalues determine growth, decay, and stationary behavior, while the matrix exponential accumulates linear changes in continuous time. For a general map, we move individual samples and calculate their distribution with the pushforward. The Jacobian determinant corrects the density for local volume change.

Gradients specify directions of increasing scalar values, including the score direction of increasing log density. A velocity field determines sample motion. Its divergence describes local volume change and the corresponding change in log density, and the Laplacian describes diffusive smoothing. To train these models, we minimize losses based on entropy, cross-entropy, and KL divergence, applying constraints when the parameters or distributions require them.

For continuous flow matching, we train a network to predict the velocity field v_t using a regression objective on pairs of samples. The continuity equation describes how the density evolves under that field.

To train that velocity field, we need intermediate states and target velocities that we can calculate from the data. In Chapter 2, we construct them by connecting pairs of noise and data samples. We then average the velocities of these individual paths conditional on the current state and time to obtain a field that transports the full distribution. We can learn this field from sampled pairs without evaluating that average explicitly.

Chapter 2

Flow Matching

2.1 Time-indexed maps

We can describe a generative transformation through a map, its Jacobian, and the resulting distribution. A map $\phi : \mathbb{R}^d \to \mathbb{R}^d$ transforms one sample into another. The Jacobian $J_\phi(x)$ contains the partial derivatives $\partial\phi_i/\partial x_j$ evaluated at x. The absolute determinant measures how much a small volume around x expands or contracts. When $X_0 \sim p_0$, the transformed sample $\phi(X_0)$ has distribution $\phi_\# p_0$, called the pushforward. To calculate its density, we divide the source density by the absolute Jacobian determinant using the change-of-variables formula.

A single map takes a noise sample directly to a data sample. For a continuous generative model, we describe the intermediate states as well, so we can train from local rules and sample in small steps. We need a rule for the sample's motion at each instant and an equation for the probability density as the samples move.

We can replace the single map by a family of maps indexed by a continuous time variable,

$$\phi_t : \mathbb{R}^d \to \mathbb{R}^d, \qquad 0 \le t \le 1.$$

"For each time t between zero and one, the flow map ϕ_t takes an initial point in d-dimensional space to its position after evolving for time t."

At the initial time, we use the identity map,

$$\phi_0(x) = x,$$

and at the end of generation we want the map to push the prior onto the data,

$$(\phi_1)_\# p_0 \approx p_{\text{data}}.$$

"The pushforward $(\phi_1)_\# p_0$, the distribution obtained by applying the final map ϕ_1 to samples from the prior p_0, approximates the data distribution p_{data}."

The time subscript in the flow map ϕ_t specifies how long the point has evolved. For each initial point, ϕ_t gives its position at time t.

Specifying every map ϕ_t separately would require an uncountable family of d-dimensional maps, too many to store in a network. We can construct the family by learning a local rule for motion at each instant.

2.2 Trajectories and the velocity field

Let's fix an initial point $x_0 \in \mathbb{R}^d$ and write x_t for its position at time t. As time advances, these positions form a continuous path through the state space. In molecular generation, x_t might contain atomic positions, continuous molecular descriptors, or a learned latent vector. Each time t corresponds to an intermediate molecular state.

To extract a local rule from such a path, we observe the state at time t and again after a short interval h. The displacement over that interval is $x_{t+h} - x_t$, and dividing by the elapsed time produces the average velocity during the interval,

$$\frac{x_{t+h} - x_t}{h}.$$

If we shrink the interval, the average velocity becomes the instantaneous one,

$$\frac{dx_t}{dt} = \lim_{h \to 0} \frac{x_{t+h} - x_t}{h}.$$

"The instantaneous velocity dx_t/dt of the state x_t is the limit of its displacement $x_{t+h} - x_t$ divided by the elapsed time h, as that interval shrinks to zero."

The derivative describes motion at one instant along a particular trajectory. To generate samples from any starting point, we need to specify a velocity at every position and time.

2.2.1 The velocity field

A velocity field is a function

$$v : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d,$$

which maps a state x and a time t to a velocity vector in the same d-dimensional space. We abbreviate the time argument as a subscript,

$$v_t(x) = v(x, t),$$

so that $v_t(x)$ is the velocity at position x and time t. A sample at x_t moves with this velocity, which connects the field to the trajectory,

$$\boxed{\frac{dx_t}{dt} = v_t(x_t)}$$

"The rate of change dx_t/dt of the state x_t equals the velocity $v_t(x_t)$ evaluated at the sample's current position and time."

- $x_t \in \mathbb{R}^d$ is the state of one sample at time t, a realized value, with X_t reserved for the random state.
- $v_t(x) \in \mathbb{R}^d$ is the velocity assigned to position x at time t, in units of state per unit time.

- $t \in [0, 1]$ runs from the prior at $t = 0$ to the data at $t = 1$.

In this ordinary differential equation, the velocity depends on the sample's current position and time. We do not need its initial position or intended endpoint to evaluate that velocity. Following the velocities through time reconstructs the trajectory. Repeating this from every starting point determines the family ϕ_t.

2.2.2 Constant velocity

The simplest field assigns the same velocity everywhere. In one dimension, we take

$$\frac{dx_t}{dt} = 2, \qquad x_0 = 1.$$

Over an interval of length Δt the displacement is $\Delta x \approx 2\Delta t$, and with $\Delta t = 0.25$ that is $\Delta x = 2(0.25) = 0.5$. The state moves half a unit during every quarter unit of time,

$$x_0 = 1, \quad x_{0.25} = 1.5, \quad x_{0.50} = 2.0, \quad x_{0.75} = 2.5, \quad x_1 = 3.0.$$

Every point receives the same velocity, so a whole collection of samples translates rigidly. The spacing between any two of them at time 1 equals their spacing at time 0, and no region expands, contracts or bends.

2.2.3 The integral form

We added small displacements in the constant-velocity example. To express their accumulation exactly, let's integrate the ODE. Write it with an intermediate time variable s, reserving t for the final time,

$$\frac{dx_s}{ds} = v_s(x_s),$$

and integrate both sides from time zero to time t,

$$\int_0^t \frac{dx_s}{ds}\, ds = \int_0^t v_s(x_s)\, ds.$$

By the fundamental theorem of calculus the left side is a difference of endpoint values,

$$\int_0^t \frac{dx_s}{ds}\, ds = x_t - x_0,$$

so that $x_t - x_0 = \int_0^t v_s(x_s)\, ds$. Adding x_0 to both sides, we get

$$x_t = x_0 + \int_0^t v_s(x_s)\, ds.$$

"The state x_t at time t equals the initial state x_0 plus the velocity $v_s(x_s)$ accumulated along the trajectory over intermediate times s from zero to t."

We evaluate the integrand at the solution x_s, so the integral expression still depends on the unknown trajectory. For the constant field, the integrand equals 2 at every s, and we can evaluate the integral directly. We obtain $x_t = 1 + 2t$ and $x_1 = 3$, matching the step-by-step calculation.

2.2.4 State-dependent velocity

A constant velocity field translates all samples equally. Allowing velocity to depend on position lets samples in different locations move at different speeds. In one dimension, consider

$$\frac{dx_t}{dt} = a\, x_t,$$

with a a real constant. The instantaneous rate of change of the state equals a times the current state, so a sample far from the origin moves quickly and a sample near the origin barely moves at all.

Let's solve this exactly. We can divide both sides by x_t and multiply by dt,

$$\frac{dx_t}{x_t} = a\, dt,$$

then integrate from the initial state to the state at time t,

$$\int_{x_0}^{x_t} \frac{dx}{x} = \int_0^t a\, ds.$$

The left side is a difference of logarithms and the right side is at,

$$\log |x_t| - \log |x_0| = at.$$

Combining the logarithms and exponentiating both sides,

$$\log \left|\frac{x_t}{x_0}\right| = at, \qquad \left|\frac{x_t}{x_0}\right| = e^{at}.$$

A trajectory that started positive can never reach zero in finite time, and neither can one that started negative, so the sign of x_0 is preserved and we can drop the absolute values,

$$x_t = x_0 e^{at}.$$

"The state x_t at time t equals the initial state x_0 multiplied by the exponential factor e^{at}, where the constant a sets the rate of expansion or contraction."

For positive a, trajectories expand exponentially away from the origin. For negative a, they contract exponentially toward it. Integrating the local velocity ax therefore scales every position by e^{at}.

Expansion. Let's choose $a = \log 2$. Then $e^{at} = e^{t \log 2} = 2^t$, and the solution is $x_t = 2^t x_0$, so the state doubles during one unit of time. Starting from $x_0 = 2$,

$$x_{0.5} = 2^{0.5}(2) = 2\sqrt{2} \approx 2.828, \qquad x_1 = 2^1(2) = 4.$$

Following three initial points through the same field,

$$x_0^{(1)} = -1, \quad x_0^{(2)} = 0, \quad x_0^{(3)} = 1 \qquad \longmapsto \qquad x_1^{(1)} = -2, \quad x_1^{(2)} = 0, \quad x_1^{(3)} = 2.$$

The distance between the outer points grows from 2 to 4, and the point at the origin stays where it is, since the velocity there is $(\log 2)(0) = 0$.

Contraction. With $a = -\log 2$, the solution becomes $x_t = 2^{-t}x_0$, so after one unit of time $x_1 = x_0/2$. The same three initial points $-1, 0, 1$ become

$$-\tfrac{1}{2}, \qquad 0, \qquad \tfrac{1}{2},$$

and the distance between the outer points shrinks from 2 to 1. The collection contracts toward zero.

We can translate, expand, or contract the same three points by changing the velocity field (Figure 2.1). Their collective motion follows directly from the velocity at each point.

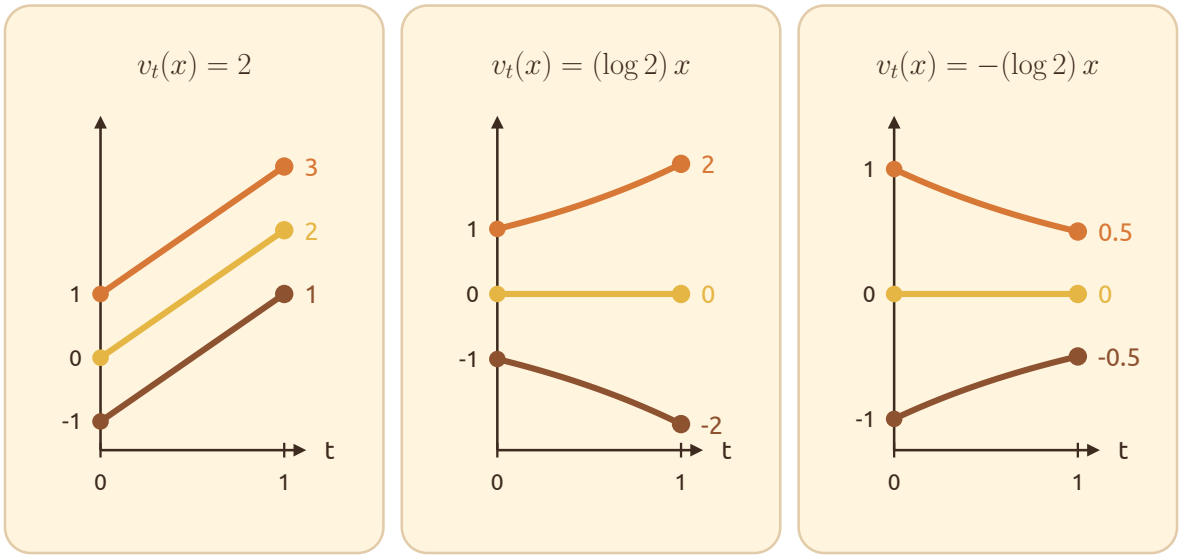


Figure 2.1: Trajectories of the three one-dimensional velocity fields of Section 2.2, with time on the horizontal axis and the state on the vertical axis. Each panel follows the same three starting points -1, 0 and 1 from $t = 0$ to $t = 1$, and the number beside each curve is its endpoint. Left: the constant field $v_t(x) = 2$ translates all three by 2. Middle: $v_t(x) = (\log 2)x$ doubles every coordinate, so the outer spacing grows from 2 to 4. Right: $v_t(x) = -(\log 2)x$ halves every coordinate, so the outer spacing shrinks from 2 to 1. In the two state-dependent panels the trajectory starting at the origin stays there, because the velocity vanishes at 0.

2.3 Velocity fields in several dimensions

Continuous generative models work in high-dimensional spaces, where a state has many coordinates and the motion of one coordinate can depend on all the others. Let the state have d coordinates,

$$x_t = \begin{bmatrix} x_{t,1} \\ x_{t,2} \\ \vdots \\ x_{t,d} \end{bmatrix} \in \mathbb{R}^d, \qquad v_t(x_t) = \begin{bmatrix} v_{t,1}(x_t) \\ v_{t,2}(x_t) \\ \vdots \\ v_{t,d}(x_t) \end{bmatrix}.$$

The vector ODE $dx_t/dt = v_t(x_t)$ is then d coupled scalar equations,

$$\frac{dx_{t,i}}{dt} = v_{t,i}(x_t), \qquad i = 1, \ldots, d.$$

"The rate of change of coordinate $x_{t,i}$ equals the corresponding velocity component $v_{t,i}(x_t)$, evaluated at the full state x_t, so each coordinate's motion can depend on all d coordinates."

In one dimension a trajectory can only move left or right. Once the velocity of one coordinate depends on the value of another, the direction of motion rotates as the state moves.

2.3.1 Rotation

We now take the two-dimensional field

$$v(x_1, x_2) = \begin{bmatrix} -x_2 \\ x_1 \end{bmatrix},$$

whose corresponding ODE is the pair

$$\frac{dx_{t,1}}{dt} = -x_{t,2}, \qquad \frac{dx_{t,2}}{dt} = x_{t,1}.$$

Our starting point is $x_0 = (1, 0)^\top$. The solution is

$$x_t = \begin{bmatrix} \cos t \\ \sin t \end{bmatrix},$$

which satisfies both equations, since $\frac{d}{dt}\cos t = -\sin t$ and $\frac{d}{dt}\sin t = \cos t$. Reading off three times,

$$x_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad x_{\pi/2} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad x_\pi = \begin{bmatrix} -1 \\ 0 \end{bmatrix},$$

so the state moves counterclockwise around the unit circle. The radius never changes,

$$\|x_t\|^2 = \cos^2 t + \sin^2 t = 1.$$

The same conclusion follows without solving the ODE at all. Differentiating the squared norm along any trajectory and substituting the ODE,

$$\frac{d}{dt}\|x_t\|^2 = 2\, x_t \cdot \frac{dx_t}{dt} = 2\, x_t \cdot v_t(x_t) = 2\,(x_{t,1}(-x_{t,2}) + x_{t,2}(x_{t,1})) = 0,$$

because the velocity is perpendicular to the position vector everywhere. The direction changes continuously while the radius stays fixed, so the trajectory follows a closed circle.

2.4 The flow map

Translation, expansion, contraction and rotation all came from a velocity field and an initial point. We can connect them back to the family ϕ_t of Section 2.1 with one definition. Fix an initial state x_0, solve the ODE until time t, and call the resulting state

$$\phi_t(x_0) = x_t.$$

The resulting state depends on the starting point and elapsed time. At time zero, $\phi_0(x_0) = x_0$. The velocity field describes the infinitesimal motion at one state and one time. The flow map describes the accumulated transformation after following that motion for a finite time.

Since $x_t = \phi_t(x_0)$, differentiation with respect to time at fixed x_0 produces

$$\frac{dx_t}{dt} = \frac{\partial}{\partial t}\phi_t(x_0),$$

and by the ODE that same derivative equals $v_t(x_t)$. Substituting $x_t = \phi_t(x_0)$,

$$\frac{\partial}{\partial t}\phi_t(x_0) = v_t(\phi_t(x_0)).$$

"The time derivative of the flow map $\phi_t(x_0)$ at a fixed starting point x_0 equals the velocity v_t evaluated at the point's current position $\phi_t(x_0)$."

Once we fix the velocity field, we can construct each map by solving the ODE from every starting point.

2.4.1 Existence, uniqueness and noncrossing trajectories

This construction assumes that a single initial state determines a single well-defined trajectory. A common sufficient regularity condition is that the velocity field be Lipschitz continuous in the state: there is a constant L with

$$\|v_t(x) - v_t(y)\| \le L\|x - y\|$$

for all states $x, y \in \mathbb{R}^d$ and all times $t \in [0, 1]$.

"The distance between the velocities $v_t(x)$ and $v_t(y)$ is at most the Lipschitz constant L times the distance between states x and y, bounding how quickly velocity can change across nearby positions."

Proposition 2.1 (Existence, uniqueness and noncrossing trajectories)**.** *Let $v_t(x)$ satisfy the Lipschitz condition above and be continuous in t. Then every initial state x_0 admits exactly one solution of $dx_t/dt = v_t(x_t)$, and two trajectories that occupy the same state at the same time coincide for all time.*

Uniqueness also constrains the trajectories. Suppose two trajectories reach the same state at the same time. From that shared state and time onward, both satisfy the same ODE with the same condition, so by uniqueness they must be the same trajectory. Distinct trajectories therefore cannot cross at the same time under a well-behaved deterministic flow. If we integrate backward from a state at time t, we recover the unique earlier state it came from, so each ϕ_t is invertible, and the finite change-of-variables formula of Chapter 1 applies to it.

For the simple fields above, we could solve the ODE analytically. With a neural velocity field, we need a numerical ODE solver to calculate the sample's trajectory.

2.5 Euler's method

To construct the simplest solver, we approximate the derivative with a difference over a small step Δt:

$$\frac{x_{t+\Delta t} - x_t}{\Delta t} \approx \frac{dx_t}{dt},$$

then substitute the ODE on the right,

$$\frac{x_{t+\Delta t} - x_t}{\Delta t} \approx v_t(x_t).$$

If we multiply by Δt and add x_t, we get an update rule,

$$x_{t+\Delta t} \approx x_t + \Delta t\, v_t(x_t).$$

On a discrete grid of times $t_k = k\Delta t$, with $\hat{x}_k$ written for the approximation to the exact state x_{t_k}, Euler's method is

$$\hat{x}_{k+1} = \hat{x}_k + \Delta t\, v_{t_k}(\hat{x}_k).$$

"The next approximate state $\hat{x}_{k+1}$ equals the current approximate state $\hat{x}_k$ plus the time step Δt times the velocity $v_{t_k}(\hat{x}_k)$, evaluated at the current grid time t_k and state."

Within each Euler step, we hold the velocity fixed at its starting value. This approximation is exact only when the field does not vary over the step. Before advancing again, we recompute the velocity at the updated state so that we do not carry the previous velocity into the next step.

2.5.1 Euler error at $\Delta t = 0.25$

We take the one-dimensional problem

$$\frac{dx_t}{dt} = x_t, \qquad x_0 = 1,$$

whose exact solution is $x_t = e^t$ by the exponential formula of Section 2.2 with $a = 1$. Let's run four Euler steps with $\Delta t = 0.25$, starting from $\hat{x}_0 = x_0 = 1$ and using $v_t(x) = x$ at each step:

$$\begin{aligned}
\hat{x}_1 &= \hat{x}_0 + 0.25\,\hat{x}_0 = 1 + 0.25(1) = 1.25,\\
\hat{x}_2 &= \hat{x}_1 + 0.25\,\hat{x}_1 = 1.25 + 0.25(1.25) = 1.5625,\\
\hat{x}_3 &= \hat{x}_2 + 0.25\,\hat{x}_2 = 1.5625 + 0.25(1.5625) = 1.953125,\\
\hat{x}_4 &= \hat{x}_3 + 0.25\,\hat{x}_3 = 1.953125 + 0.25(1.953125) = 2.44140625.
\end{aligned}$$

The exact value at time 1 is $x_1 = e \approx 2.7183$, so four steps of size 0.25 land at 2.4414 and fall short by

$$2.7183 - 2.4414 = 0.2769,$$

a relative error of $0.2769/2.7183 \approx 0.1019$, about ten percent. On this problem the velocity grows with the state, and each Euler step uses the velocity at the beginning of the step, which is the smallest velocity the exact solution attains anywhere in that step. Every step therefore undershoots, and the undershoots compound.

If we shrink the step, the estimate improves. One step of size $\Delta t = 1$ lands at $1 + 1(1) = 2.00$, with error 0.7183. Two steps of size 0.5 land at 1.5 and then 2.25, with error 0.4683. Four steps of size 0.25 land at 2.4414, with error 0.2769. Each halving of the step cuts the error by a factor of 1.53 and then 1.69, approaching the factor of 2 that a first-order method attains in the limit of small steps (Figure 2.2).

2.5.2 Step size and sampling cost

When a neural network represents v_t, every velocity evaluation is a forward pass through that network. Generating one sample with n Euler steps therefore costs n network evaluations, and generating a batch costs n evaluations on the whole batch. We control the step count, and it trades sample quality against compute directly. Halving Δt roughly halves the discretization error and exactly doubles the cost. Learning the finite-time map directly can reduce this cost by predicting the displacement across an entire interval in one step.

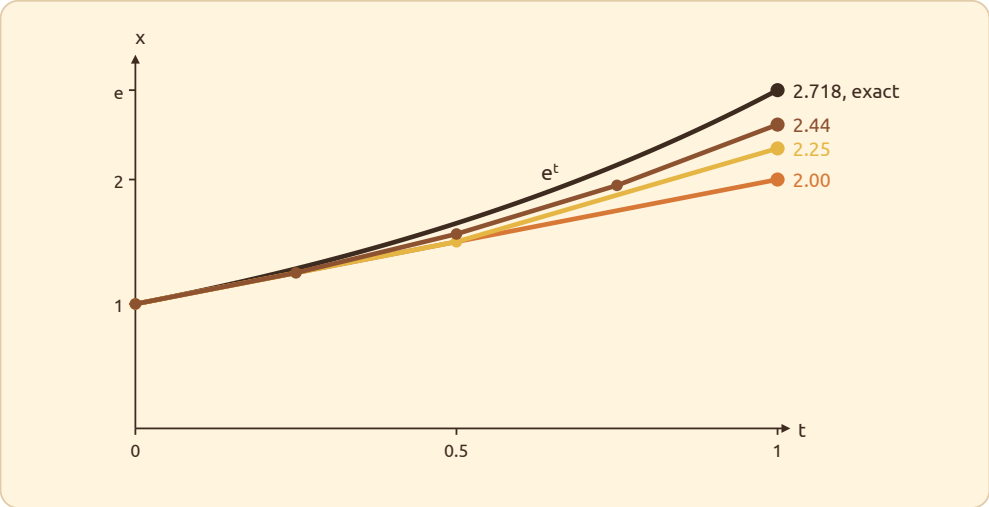


Figure 2.2: Euler's method on $dx_t/dt = x_t$ with $x_0 = 1$. The dark curve is the exact solution e^t, which reaches 2.718 at $t = 1$. The three piecewise-linear paths are Euler with one step of size 1, two steps of size 0.5 and four steps of size 0.25, landing at 2.00, 2.25 and 2.44. Markers sit at the grid times where each path recomputes the velocity. All three paths fall below the exact curve because each step uses the velocity at the start of the step, which on this problem is the smallest velocity in the step.

2.6 A random starting point

To generate a population of samples, we draw the initial state from a probability distribution,

$$X_0 \sim p_0,$$

and let every sampled value follow the same velocity field,

$$\frac{dX_t}{dt} = v_t(X_t).$$

Once we have drawn X_0, the trajectory is deterministic, and nothing else is random. Different draws follow different deterministic trajectories, so X_t remains a random variable, with all of its randomness inherited from the draw at time zero.

When we solve the ODE from each draw, we are applying the flow map,

$$X_t = \phi_t(X_0),$$

and applying ϕ_t to the entire initial population produces the distribution at time t,

$$p_t = (\phi_t)_{\#} p_0.$$

"The distribution p_t at time t is the pushforward of the initial distribution p_0 through the flow map ϕ_t, meaning that $\phi_t(X_0)$ has distribution p_t when X_0 is drawn from p_0."

We can draw X_0 from the prior p_0, transport it through the learned velocity field v_θ, where θ are the network parameters, and read off a final random state,

$$X_0 \sim p_0 \quad \longrightarrow \quad \frac{dX_t}{dt} = v_\theta(X_t, t) \quad \longrightarrow \quad X_1 \sim p_1.$$

We choose θ so that $p_1 \approx p_{\text{data}}$. The velocity field determines the ODE trajectories, and the flow map specifies each trajectory's position at time t. Applying that map to the prior determines the transported distribution.

2.6.1 Gaussian expansion

Let's calculate how a Gaussian distribution changes under expansion. We take Gaussian noise as the prior,

$$X_0 \sim \mathcal{N}(0, 1),$$

where the second parameter is the variance, and take the expansion field of Section 2.2,

$$v_t(x) = (\log 2)\, x, \qquad \frac{dX_t}{dt} = (\log 2) X_t.$$

Its solution is $X_t = 2^t X_0$. Scaling a centered Gaussian by a constant c multiplies its variance by c^2, so

$$X_{0.5} = \sqrt{2}\, X_0 \sim \mathcal{N}(0, 2), \qquad X_1 = 2X_0 \sim \mathcal{N}(0, 4).$$

The mean stays at zero as the standard deviation grows from 1 to 2 and the variance from 1 to 4. The samples spread over a wider region under the deterministic field. Because total probability remains one, the density near the center falls as the occupied region grows. Its local rate of change depends on both the velocity field and the density $p_t(x)$.

2.7 Probability flux and the continuity equation

Let A be a fixed region of the state space. The probability that the random state lies inside it is

$$\mathbb{P}(X_t \in A) = \int_A p_t(x)\, dx,$$

where $p_t(x)$ is probability per unit volume near x, so integrating it over A accumulates the total probability contained there. We can picture A as a box drawn once and then held still while samples move through it under the velocity field. The probability inside the box changes only when samples cross its boundary, so we need a measure of those crossings.

2.7.1 Probability flux

At a position x, the density $p_t(x)$ measures how much probability is locally available to move, and the velocity $v_t(x)$ is the direction and the speed at which it moves. Their product is the probability flux,

$$j_t(x) = p_t(x)\, v_t(x).$$

"The probability flux $j_t(x)$ at position x and time t equals the local density $p_t(x)$ times the velocity $v_t(x)$, measuring probability transported per unit time and unit area."

A region with high density and high speed has a large flux. A region with zero density has no flux however large the velocity, since there is no probability there to move. A region with zero velocity has no flux however large the density, since the probability there stays put. In one dimension the sign of $j_t(x)$ records the direction of motion. Positive flux transports probability toward increasing x, and negative flux transports it toward decreasing x.

2.7.2 Conservation on an interval

We can work in one dimension and fix the interval $A = [a, b]$. The probability mass inside it is

$$M_t([a, b]) = \int_a^b p_t(x)\, dx.$$

Probability can flow in through the left endpoint and out through the right endpoint, and the two signed fluxes are

$$j_t(a) = p_t(a)v_t(a), \qquad j_t(b) = p_t(b)v_t(b).$$

A positive $j_t(a)$ moves probability rightward across a and into the interval. A positive $j_t(b)$ moves probability rightward across b and out of it. The rate of change of the enclosed mass is the difference,

$$\frac{d}{dt}\int_a^b p_t(x)\, dx = p_t(a)v_t(a) - p_t(b)v_t(b).$$

"The rate of change of probability inside the interval $[a, b]$ equals the signed inflow $p_t(a)v_t(a)$ at its left endpoint a minus the signed outflow $p_t(b)v_t(b)$ at its right endpoint b, with each flux equal to density times velocity."

The enclosed mass increases when inflow exceeds outflow and decreases when outflow exceeds inflow. Equal inflow and outflow preserve the mass (Figure 2.3).

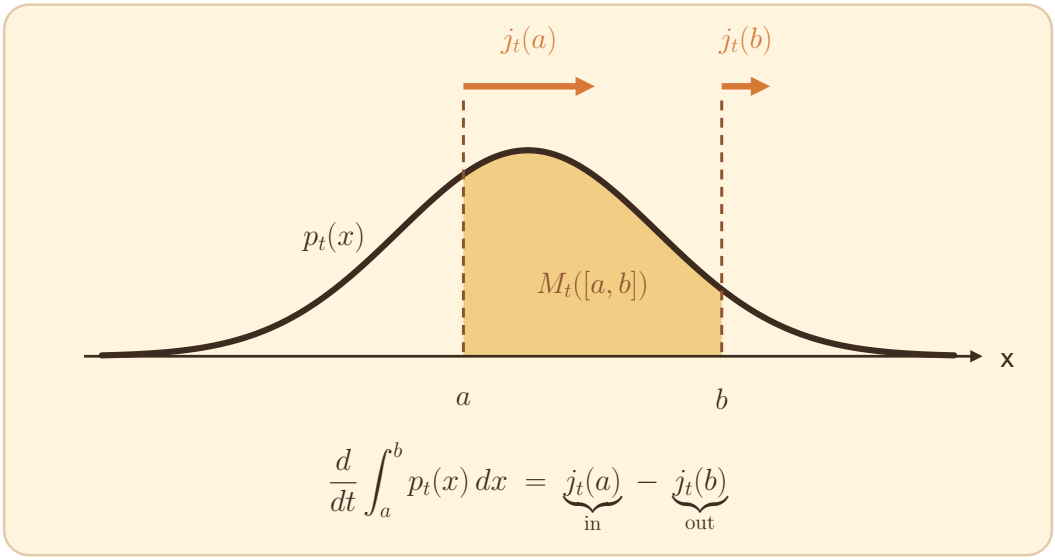


Figure 2.3: Probability flux at the two endpoints of a fixed interval. The dark curve is the density $p_t(x)$ over the state x, and the shaded area is the enclosed mass $M_t([a, b])$. The two arrows are the signed fluxes $j_t(a)$ and $j_t(b)$, both pointing toward increasing x here, so the flux at a carries probability into the interval and the flux at b carries probability out of it. The equation below the axis is the balance between them.

2.7.3 A worked flux balance

We return to the spreading Gaussian of Section 2.6, with $X_0 \sim \mathcal{N}(0, 1)$ and $v_t(x) = (\log 2)x$, and take the interval $[-1, 1]$ at time $t = 0$. The standard normal density at $x = \pm 1$ is

$$p_0(\pm 1) = \frac{1}{\sqrt{2\pi}} e^{-1/2} \approx 0.2420,$$

and $\log 2 \approx 0.6931$, so the two fluxes are

$$j_0(-1) = (0.2420)(0.6931)(-1) \approx -0.1677, \qquad j_0(1) = (0.2420)(0.6931)(1) \approx 0.1677.$$

The field points away from the origin on both sides, so probability flows out through both endpoints. The balance is

$$\frac{d}{dt} M_t([-1,1])\Big|_{t=0} = j_0(-1) - j_0(1) \approx -0.1677 - 0.1677 = -0.3354.$$

We can check this against the distribution directly. Since $X_t \sim \mathcal{N}(0, 4^t)$ has standard deviation 2^t,

$$M_t([-1,1]) = \mathbb{P}(|X_t| \le 1) = \operatorname{erf}\left(\frac{2^{-t}}{\sqrt{2}}\right),$$

which equals 0.6827 at $t = 0$ and falls at a rate of -0.3354 there. This rate agrees with the flux calculation to four figures. As the Gaussian spreads, the interval loses about a third of a unit of probability per unit time.

2.7.4 Local probability conservation

We can derive a local equation from the interval balance by expressing both sides as integrals. Write the flux as $j_t(x) = p_t(x)v_t(x)$. By the fundamental theorem of calculus the endpoint difference is an integral of a derivative,

$$p_t(b)v_t(b) - p_t(a)v_t(a) = \int_a^b \frac{\partial}{\partial x}\big[p_t(x)v_t(x)\big]\, dx,$$

and reversing the signs puts it in the orientation of the balance,

$$p_t(a)v_t(a) - p_t(b)v_t(b) = -\int_a^b \frac{\partial}{\partial x}\big[p_t(x)v_t(x)\big]\, dx.$$

On the left side, we move the time derivative inside the integral,

$$\frac{d}{dt}\int_a^b p_t(x)\, dx = \int_a^b \frac{\partial p_t(x)}{\partial t}\, dx.$$

Substituting both expressions into the balance,

$$\int_a^b \frac{\partial p_t(x)}{\partial t}\, dx = -\int_a^b \frac{\partial}{\partial x}\big[p_t(x)v_t(x)\big]\, dx,$$

and moving everything to the left side collects the two integrands,

$$\int_a^b \left[\frac{\partial p_t(x)}{\partial t} + \frac{\partial}{\partial x}\big(p_t(x)v_t(x)\big)\right] dx = 0.$$

This holds for every interval $[a, b]$, including arbitrarily short ones around any chosen point, so a continuous integrand must vanish everywhere,

$$\frac{\partial p_t(x)}{\partial t} + \frac{\partial}{\partial x}\big[p_t(x)v_t(x)\big] = 0.$$

Density falls where more probability flows away than flows in, and rises where more flows in than out.

2.7.5 The continuity equation in d dimensions

In d dimensions probability flows across an entire boundary surface. Let A be a fixed region with boundary ∂A, and at each boundary point let $n(x)$ be the outward-pointing unit normal vector. The component of velocity pointing out of the region is $v_t(x) \cdot n(x)$, and multiplying by the density produces the outward probability flux per unit boundary area, $p_t(x)v_t(x) \cdot n(x)$. If we integrate over the boundary, we get the total outward flux, and conservation of probability requires

$$\frac{d}{dt}\int_A p_t(x)\,dx = -\int_{\partial A} p_t(x)v_t(x) \cdot n(x)\,dS.$$

"The rate of change of probability mass inside the region A equals the negative integral of the outward flux $p_t(x)v_t(x) \cdot n(x)$ over its boundary ∂A, where $n(x)$ is the outward unit normal and dS is the boundary area element."

The minus sign is the d-dimensional version of the endpoint arithmetic. Outward flux is positive where the velocity points out of A, and it drains probability from A. An inward-pointing velocity has a negative dot product with the outward normal, so its outward flux is negative and probability accumulates in A.

To obtain a local equation at each point, we apply the divergence theorem and replace the boundary integral by an integral over the interior,

$$\int_{\partial A} p_t(x)v_t(x) \cdot n(x)\,dS = \int_A \nabla \cdot \left[p_t(x)v_t(x)\right] dx.$$

We also move the time derivative inside the integral on the left,

$$\frac{d}{dt}\int_A p_t(x)\,dx = \int_A \frac{\partial p_t(x)}{\partial t}\,dx,$$

then substitute both expressions into the conservation equation and collect terms,

$$\int_A \left[\frac{\partial p_t(x)}{\partial t} + \nabla \cdot \left(p_t(x)v_t(x)\right)\right] dx = 0.$$

The region A was arbitrary, so the integrand vanishes at every point, which is the continuity equation:

$$\boxed{\frac{\partial p_t(x)}{\partial t} + \nabla \cdot \left[p_t(x)v_t(x)\right] = 0}$$

"The time derivative of the density $p_t(x)$ at a fixed position x, plus the divergence of the probability flux $p_t(x)v_t(x)$, equals zero, so net outward flux lowers the local density."

- $p_t(x) \geq 0$ is the density at position x and time t, with $\int_{\mathbb{R}^d} p_t(x)\,dx = 1$.
- $\partial p_t(x)/\partial t$ is the rate of change of the density at a position held fixed, with no particle moving.
- $\nabla \cdot j_t(x) = \sum_{i=1}^d \partial j_{t,i}(x)/\partial x_i$ is the divergence of the flux, the net outward flow from an infinitesimal neighborhood of x, per unit volume.

Positive flux divergence means probability flows out of the neighborhood of x faster than it flows in, so the density there falls. Negative flux divergence means probability accumulates there, so the density rises. An equivalent arrangement, $\partial_t p_t(x) = -\nabla \cdot [p_t(x)v_t(x)]$, puts the time derivative of the density alone on the left, which is the form we use when we know the velocity field and want the density evolution.

2.7.6 Expanding the divergence

We can use the product rule to separate transport from local volume change. For a scalar field p_t and a vector field v_t,

$$\nabla \cdot [p_t(x)v_t(x)] = v_t(x) \cdot \nabla p_t(x) + p_t(x)\, \nabla \cdot v_t(x).$$

Substituting this identity into the continuity equation,

$$\frac{\partial p_t(x)}{\partial t} + v_t(x) \cdot \nabla p_t(x) + p_t(x)\, \nabla \cdot v_t(x) = 0.$$

"The rate of change of the density $p_t(x)$ at the fixed position x, plus the velocity $v_t(x)$ dotted with the spatial density gradient $\nabla p_t(x)$, plus the density multiplied by the velocity divergence $\nabla \cdot v_t(x)$, equals zero."

The transport term $v_t(x) \cdot \nabla p_t(x)$ describes changes caused by a spatially varying density moving past the fixed point x. The volume-change term $p_t(x)\, \nabla \cdot v_t(x)$ describes how neighboring trajectories separate or converge. Separation spreads the same probability over more volume, giving positive divergence and a lower density. Convergence concentrates that probability in less volume, giving negative divergence and a higher density.

2.7.7 Continuity equation for a Gaussian

Both sides of the continuity equation are computable for the expansion example, so let's evaluate them. With $v_t(x) = (\log 2)x$ and $X_t \sim \mathcal{N}(0, 4^t)$, the density is

$$p_t(x) = \frac{1}{2^t\sqrt{2\pi}} \exp\left(-\frac{x^2}{2 \cdot 4^t}\right).$$

We then take the point $x = 2$ at time $t = 0$, where $p_0(2) = e^{-2}/\sqrt{2\pi} \approx 0.05399$ and $\partial_x p_0(2) = -2p_0(2) \approx -0.10798$. Differentiating the density in time at fixed position,

$$\left.\frac{\partial p_t(2)}{\partial t}\right|_{t=0} = p_0(2)(\log 2)(2^2 - 1) \approx (0.05399)(0.6931)(3) \approx 0.11227.$$

For the other side we use the expanded form, with $\nabla \cdot v_t = \log 2$ in one dimension,

$$\begin{aligned} -\left[v_0(2)\, \partial_x p_0(2) + p_0(2)\, \nabla \cdot v_0\right] &= -\left[(1.3863)(-0.10798) + (0.05399)(0.6931)\right] \\ &= -(-0.14970 + 0.03742) \approx 0.11227. \end{aligned}$$

Both calculations give the same rate. At $x = 2$ and $t = 0$, inward probability flow exceeds the density loss from positive divergence. The local density therefore rises as the Gaussian spreads. Both effects are present in the continuity equation, and here they have opposite signs.

2.7.8 Conservation of total probability

A valid density stays normalized at every time, and that normalization follows from the continuity equation with no extra assumption. We can integrate the equation over the entire state space,

$$\frac{d}{dt}\int_{\mathbb{R}^d} p_t(x)\,dx = -\int_{\mathbb{R}^d} \nabla\cdot[p_t(x)v_t(x)]\,dx.$$

Next, we apply the divergence theorem on a ball of growing radius. On the right, we obtain a surface integral of the flux over the sphere. If the flux vanishes quickly enough at infinity, the boundary term tends to zero. This holds for any density with finite mass and a velocity field of at most polynomial growth, so

$$\frac{d}{dt}\int_{\mathbb{R}^d} p_t(x)\,dx = 0.$$

Since $\int_{\mathbb{R}^d} p_0(x)\,dx = 1$ at the start,

$$\int_{\mathbb{R}^d} p_t(x)\,dx = 1 \quad \text{for every } t.$$

Transport under a velocity field redistributes probability through the state space and preserves the total amount of it.

2.8 Density along a moving trajectory

The continuity equation describes density changes at a fixed position. For training and likelihood evaluation, we need the density $p_t(x_t)$ along the moving state x_t, which solves the ODE. Because both the density function and its input depend on time, we apply the multivariable chain rule:

$$\frac{d}{dt}p_t(x_t) = \frac{\partial p_t(x_t)}{\partial t} + \frac{dx_t}{dt}\cdot\nabla p_t(x_t).$$

Substituting the ODE $dx_t/dt = v_t(x_t)$,

$$\frac{d}{dt}p_t(x_t) = \frac{\partial p_t(x_t)}{\partial t} + v_t(x_t)\cdot\nabla p_t(x_t).$$

The first two terms of the expanded continuity equation appear on the right. Their sum equals $-p_t(x_t)\nabla\cdot v_t(x_t)$. Therefore

$$\frac{d}{dt}p_t(x_t) = -p_t(x_t)\,\nabla\cdot v_t(x_t).$$

"The rate of change of the density $p_t(x_t)$ along the moving state x_t equals the negative of that density times the velocity divergence $\nabla\cdot v_t(x_t)$, which measures local expansion or contraction."

Along the moving sample, the transport terms cancel. The density changes only through local expansion or contraction of volume. Positive divergence lowers the density along the trajectory, negative divergence raises it, and zero divergence preserves it exactly.

2.8.1 The instantaneous change-of-variables equation

Dividing by the density turns this into an equation for the log density, which is the quantity we want for a likelihood. For positive density,

$$\frac{d}{dt}\log p_t(x_t) = \frac{1}{p_t(x_t)}\frac{d}{dt}p_t(x_t),$$

and substituting the trajectory-density equation cancels the density entirely,

$$\boxed{\frac{d}{dt}\log p_t(x_t) = -\nabla \cdot v_t(x_t)}$$

"The rate of change of the log density $\log p_t(x_t)$ along the trajectory x_t equals the negative velocity divergence $\nabla \cdot v_t(x_t)$ at the current state."

This is the instantaneous change-of-variables equation. The divergence alone determines the rate, with no reference to the density value. A divergence of 0.4 at the current state means the log density is falling at an instantaneous rate of 0.4 per unit time; a divergence of -0.4 means it is rising at that rate. Integrating from time 0 to time t along the trajectory,

$$\log p_t(x_t) = \log p_0(x_0) - \int_0^t \nabla \cdot v_s(x_s)\, ds.$$

"The log density $\log p_t(x_t)$ at time t equals the initial log density $\log p_0(x_0)$ minus the velocity divergence $\nabla \cdot v_s(x_s)$ accumulated along the trajectory over times s from zero to t."

2.8.2 Agreement with the finite change-of-variables formula

In Chapter 1 we established the finite change-of-variables equation for an invertible map. For the flow map it reads

$$p_t(\phi_t(x_0)) = \frac{p_0(x_0)}{|\det J_{\phi_t}(x_0)|},$$

where $J_{\phi_t}(x_0)$ is the Jacobian of ϕ_t at x_0. Taking logarithms,

$$\log p_t(x_t) = \log p_0(x_0) - \log|\det J_{\phi_t}(x_0)|\,.$$

The determinant and the divergence are connected by the identity

$$\frac{d}{dt}\log|\det J_{\phi_t}(x_0)| = \nabla \cdot v_t(x_t).$$

The determinant records the accumulated volume change over the finite interval from 0 to t. The divergence records the instantaneous rate of that change at the current state. Differentiating the log form of the finite formula in time,

$$\frac{d}{dt}\log p_t(x_t) = -\frac{d}{dt}\log|\det J_{\phi_t}(x_0)|\,,$$

and substituting the identity recovers

$$\frac{d}{dt}\log p_t(x_t) = -\nabla \cdot v_t(x_t).$$

We obtain the same log-density equation from the continuity equation and from the Jacobian determinant, confirming that the two calculations agree.

2.8.3 Log-density example

Let's go back to the one-dimensional field $v(x) = (\log 2)x$. In one dimension the divergence is the derivative of the velocity,

$$\frac{dv(x)}{dx} = \log 2, \qquad \nabla \cdot v(x) = \log 2,$$

a constant, independent of position. After one unit of time the flow map is $x_1 = 2x_0$, which doubles all distances, so its Jacobian determinant is

$$|\det J_{\phi_1}(x_0)| = 2,$$

and by the finite change-of-variables formula,

$$p_1(x_1) = \frac{p_0(x_0)}{2}.$$

Through the continuous route, the divergence is $\log 2$ at every point of the trajectory, so the accumulated integral is $\int_0^1 \log 2\, dt = \log 2$ and

$$\log p_1(x_1) = \log p_0(x_0) - \log 2.$$

Exponentiating recovers $p_1(x_1) = p_0(x_0)/2$, the same answer.

With the standard Gaussian prior $X_0 \sim \mathcal{N}(0, 1)$, the density at the origin is

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989.$$

The origin is a fixed point of this flow, so $x_1 = 2(0) = 0$, and since $X_1 = 2X_0$ is $\mathcal{N}(0, 4)$ with standard deviation 2, its density at the origin is

$$p_1(0) = \frac{1}{2\sqrt{2\pi}} \approx 0.1995.$$

The standard deviation and local lengths double, while the density at corresponding points halves. The probability inside corresponding intervals is unchanged.

2.9 Continuous normalizing flows

In a continuous normalizing flow we represent the velocity field with a neural network $v_\theta(x, t)$ and integrate the state and log-density equations together [Chen et al., 2018]. The state equation tracks the sample and the log-density equation tracks the probability assigned to that sample,

$$\begin{cases} \dfrac{dX_t}{dt} = v_\theta(X_t, t), \\ \dfrac{d}{dt} \log p_t(X_t) = -\nabla \cdot v_\theta(X_t, t). \end{cases}$$

"The random state X_t changes at the learned velocity $v_\theta(X_t, t)$, parameterized by the network weights θ, while its log density $\log p_t(X_t)$ changes at the negative divergence of that velocity field along the trajectory."

For generation, we draw X_0 from the known prior and integrate the first equation forward to $t = 1$, discarding the second if we want only samples. For likelihood evaluation, we start from a data point X_1, integrate backward to its prior state X_0, and accumulate the divergence along the way. At the terminal time,

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla \cdot v_\theta(x_t, t)\, dt.$$

"The model log density $\log p_1(x_1)$ of a data point x_1 equals the prior log density $\log p_0(x_0)$ at its initial state x_0, minus the divergence of the learned velocity $v_\theta(x_t, t)$ integrated along the trajectory from time zero to one."

At each solver step, we evaluate the network velocity, advance along the trajectory, and accumulate the divergence to calculate the density change. We can compute every term on the right. The Gaussian prior p_0 has a closed-form log density, and we evaluate the divergence integral with the same solver used for the trajectory.

To train a continuous normalizing flow by maximum likelihood, we maximize this expression over θ. Each gradient update requires a full ODE solve and a divergence evaluation at every solver step. Computing the divergence exactly requires d derivative evaluations because it is the trace of a $d \times d$ Jacobian. Training cost therefore grows with both the solver step count and the dimension, making direct likelihood training impractical at the scale of images or sequences.

2.10 Learning the velocity field

Sample motion follows the ODE,

$$\frac{dx_t}{dt} = v_t(x_t).$$

Probability flux combines the density with the velocity,

$$j_t(x) = p_t(x) v_t(x).$$

Conservation of probability turns into the continuity equation,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot [p_t(x) v_t(x)].$$

Along an ODE trajectory, the divergence determines the log-density change,

$$\frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v_t(x_t),$$

and over a finite interval the flow map and its Jacobian record the same volume change, $p_t(\phi_t(x_0)) = p_0(x_0)/|\det J_{\phi_t}(x_0)|$.

We can now follow any velocity field v_t and track how the density moves with it. Integrating the field moves a sample along its trajectory. Substituting the same field into the continuity equation produces the time derivative of the density at every point at once. The divergence along a trajectory accumulates the log density that moves with the sample, in agreement with the finite change-of-variables formula of Chapter 1.

We still need to choose a velocity field v_t whose endpoint distribution p_1 matches the data distribution. Likelihood training requires us to integrate the ODE, accumulate the divergence, evaluate the likelihood, and adjust θ at every update. We can avoid this repeated ODE solve by choosing paths between prior and data samples, computing their velocities, and learning those velocities by regression. This training procedure is flow matching [Lipman et al., 2023].

2.11 Coupling the prior and the data

We begin each path with a prior sample $X_0 \sim p_0$, usually from a standard Gaussian in $\mathbb{R}^d$ with mean vector zero and identity covariance matrix I. For the other endpoint, we draw a data sample:

$$X_0 \sim p_0 = \mathcal{N}(0, I), \qquad X_1 \sim p_1 = p_{\text{data}}.$$

For continuous molecular generation X_1 holds atomic coordinates, continuous molecular descriptors, or a learned continuous representation of a molecule. The prior sample X_0 has the same dimension as X_1 and starts as Gaussian noise.

The marginals specify the distributions of noise and data samples. To specify how we pair the samples, we choose a joint distribution. A *coupling* is a joint distribution over the endpoint pair whose two marginals are the prior and the data,

$$(X_0, X_1) \sim \pi, \qquad X_0 \sim p_0, \qquad X_1 \sim p_1.$$

"We draw the prior endpoint X_0 and data endpoint X_1 jointly from the coupling π on $\mathbb{R}^d \times \mathbb{R}^d$, whose first marginal is the prior p_0 and second marginal is the data distribution p_1."

The simplest coupling draws the two endpoints independently,

$$\pi(x_0, x_1) = p_0(x_0)\, p_1(x_1).$$

"The independent coupling density $\pi(x_0, x_1)$ equals the prior density $p_0(x_0)$ times the data density $p_1(x_1)$, so we sample the two endpoints independently."

Any joint law with the correct marginals is a coupling. Different couplings connect the same two populations through different paths. Pairing nearby endpoints shortens those paths and lowers the total squared travel distance while preserving p_0 and p_1. Straighter paths can reduce the number of sampling steps [Liu et al., 2023]. Optimal transport determines a coupling that minimizes squared travel distance. We use the independent coupling here.

For each endpoint pair, we need a continuous path between the points and a velocity along that path to use as the network's training target.

2.12 The conditional path and its velocity

The simplest path between two fixed points is the straight segment traversed at constant speed. For an endpoint pair (X_0, X_1) we define the interpolation

$$X_t = (1 - t)X_0 + tX_1, \qquad 0 \le t \le 1.$$

"The intermediate state X_t equals the prior endpoint X_0 weighted by $1 - t$ plus the data endpoint X_1 weighted by t, shifting weight from the prior to the data as time advances from zero to one."

Let's evaluate the interpolation at the start, midpoint, and end of the time interval,

$$X_{t=0} = (1-0)X_0 + 0 \cdot X_1 = X_0, \qquad X_{t=1/2} = \tfrac{1}{2}X_0 + \tfrac{1}{2}X_1, \qquad X_{t=1} = X_1.$$

For one fixed endpoint pair the interpolation is a deterministic curve. Randomness in the training set comes from the fresh pair $(X_0, X_1) \sim \pi$ that we draw every time.

We can calculate the particle's velocity by differentiating the interpolation with respect to t. Hold both endpoints fixed, so each term is a scalar multiple of a constant vector,

$$\frac{dX_t}{dt} = \frac{d}{dt}\big[(1-t)X_0 + tX_1\big] = \underbrace{-X_0}_{\frac{d}{dt}[(1-t)X_0]} + \underbrace{X_1}_{\frac{d}{dt}[tX_1]} = X_1 - X_0.$$

We call this the *conditional velocity* of the path, written U_t,

$$U_t = X_1 - X_0.$$

"The conditional velocity U_t along the straight path equals the data endpoint X_1 minus the prior endpoint X_0, a displacement vector that stays constant throughout the unit time interval."

Written as a function of the state with the endpoints given,

$$u_t(x_t \mid x_0, x_1) = x_1 - x_0,$$

where $u_t(\cdot \mid x_0, x_1)$ is the velocity field of the path conditioned on the pair (x_0, x_1), in units of displacement per unit time. The conditional velocity points from the prior endpoint toward the data endpoint. Its magnitude stays constant because the particle covers equal distances in equal time intervals along the straight segment. We call this velocity *conditional* because we need both endpoints to calculate it.

2.12.1 Endpoint-pair example

Let's take the one-dimensional endpoints $x_0 = -2$ and $x_1 = 3$. The interpolation path is

$$x_t = (1-t)(-2) + 3t = -2 + 5t,$$

a line with intercept -2 and slope 5. Evaluating it on a grid of times,

$$\begin{aligned} x_0 &= -2, \\ x_{0.25} &= -2 + 5(0.25) = -0.75, \\ x_{0.50} &= -2 + 5(0.50) = 0.50, \\ x_{0.75} &= -2 + 5(0.75) = 1.75, \\ x_1 &= -2 + 5(1) = 3. \end{aligned}$$

The conditional velocity is the slope of that line,

$$u_t = x_1 - x_0 = 3 - (-2) = 5,$$

the same at every time. The particle covers 1.25 units in each quarter of the interval, and five units in total.

A network prediction of 5 at every point would reproduce this path exactly. During training, we sample many paths that can overlap at a given time. We therefore need a single velocity at each state and time where paths meet.

2.13 The marginal velocity

Each endpoint pair determines a path. At each time, the population of paths has a probability density. For a fixed pair, the interpolation is deterministic. Its conditional law therefore concentrates on a single point,

$$p_t(x \mid x_0, x_1) = \delta\big(x - [(1-t)x_0 + tx_1]\big),$$

where δ is the Dirac delta in $\mathbb{R}^d$. Averaging over the coupling produces the density that the whole population occupies at time t,

$$p_t(x) = \int p_t(x \mid x_0, x_1)\, \pi(x_0, x_1)\, dx_0\, dx_1.$$

"The marginal density $p_t(x)$ at state x and time t averages the conditional density $p_t(x \mid x_0, x_1)$ over endpoint pairs (x_0, x_1), weighted by their coupling density $\pi(x_0, x_1)$."

At the endpoints, we recover the prior and data distributions:

$$p_{t=0} = p_0, \qquad p_{t=1} = p_1,$$

because at $t = 0$ every conditional path starts at its own x_0, whose distribution is p_0, and at $t = 1$ every path ends at its own x_1, whose distribution is p_1. The coupling and interpolation therefore determine the complete density path p_t before training begins (Figure 2.4).

To predict the conditional velocity $U_t = X_1 - X_0$ for a specific path, the network would need both endpoints. At sampling time, however, it receives only the current state x and time t. Several conditional paths can pass through the same state and time with different velocities U_t. We need a single deterministic velocity for that input, so we average the conditional velocities of the paths passing through it:

$$\boxed{v_t(x) = \mathbb{E}\big[U_t \mid X_t = x\big].}$$

"The marginal velocity $v_t(x)$ averages the conditional velocities U_t over endpoint pairs whose interpolated state X_t equals x at time t."

For the linear interpolation the conditional velocity is the endpoint displacement, so

$$v_t(x) = \mathbb{E}\big[X_1 - X_0 \mid X_t = x\big],$$

the expected displacement from prior endpoint to data endpoint among all paths whose intermediate state at time t equals x. Sampling after training follows this marginal field. The generated trajectories need not reproduce the individual straight training paths; their distribution at each time must reproduce p_t.

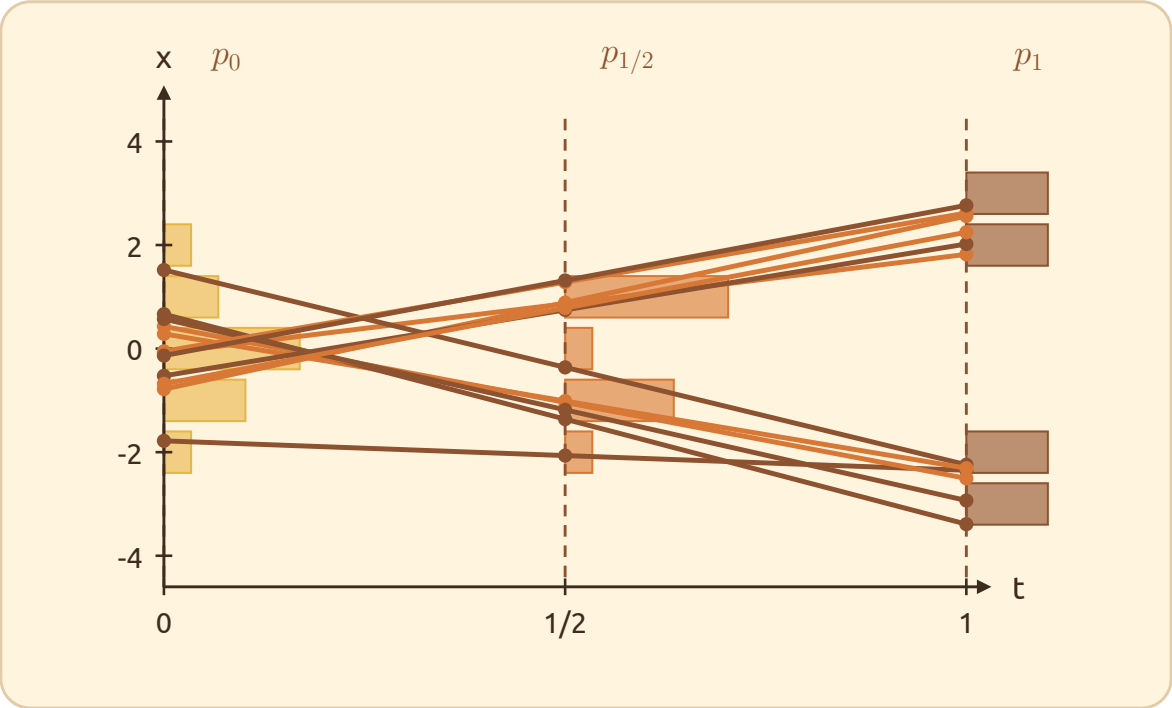


Figure 2.4: Straight-line conditional paths from several prior samples to several data samples. Each line follows one endpoint pair (x_0, x_1) drawn from the independent coupling, and time runs left to right from $t = 0$ to $t = 1$. The histograms collect the positions of all paths at $t = 0$, $t = 1/2$ and $t = 1$; they are the marginal densities p_0, $p_{1/2}$ and p_1. Individual paths cross, and the histogram at a given time records only where the paths are, not which pair each one belongs to.

2.13.1 Averaging at a crossing

Consider two equally likely endpoint pairs in one dimension. The first runs from $x_0^{(1)} = 0$ to $x_1^{(1)} = 4$, so its conditional velocity and its midpoint are

$$U_t^{(1)} = 4 - 0 = 4, \qquad x_{0.5}^{(1)} = \tfrac{1}{2}(0) + \tfrac{1}{2}(4) = 2.$$

The second runs from $x_0^{(2)} = 3$ to $x_1^{(2)} = 1$, so

$$U_t^{(2)} = 1 - 3 = -2, \qquad x_{0.5}^{(2)} = \tfrac{1}{2}(3) + \tfrac{1}{2}(1) = 2.$$

The two paths meet at $x = 2$ at time $t = 0.5$, one moving right at speed 4 and the other moving left at speed 2 (Figure 2.5). Since the two pairs are equally likely, the conditional expectation at that state and time is

$$v_{0.5}(2) = \tfrac{1}{2}(4) + \tfrac{1}{2}(-2) = 1.$$

A deterministic velocity field can have only one value at $x = 2$ and $t = 0.5$, and the value $+1$ reproduces the correct net movement of probability through that location. Over a short interval h, half the mass moves right by $4h$. The other half moves left by $2h$. Averaging these displacements gives h per unit mass, matching a velocity of $+1$.

Computing this average directly would require knowing which endpoint pairs reach a given state. We can recover it by minimizing the squared prediction error over sampled pairs.

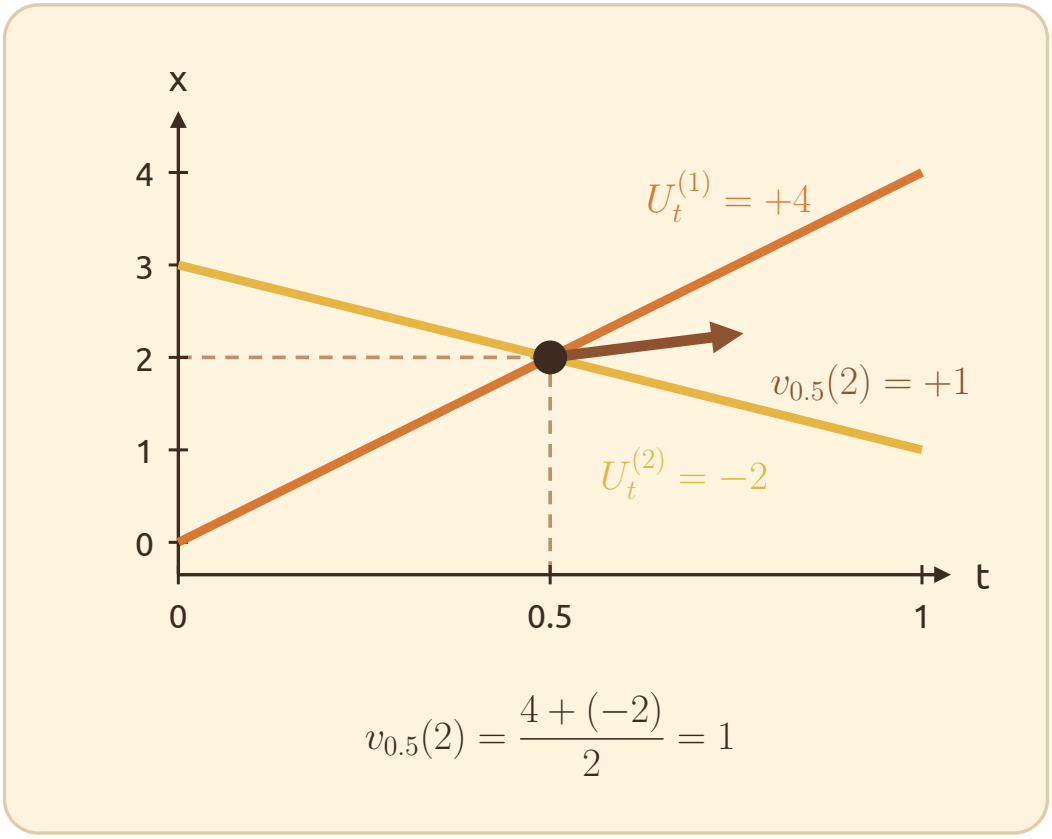


Figure 2.5: Two conditional paths that cross. The first runs from $x_0 = 0$ to $x_1 = 4$ with constant velocity $+4$; the second runs from $x_0 = 3$ to $x_1 = 1$ with constant velocity -2. Both occupy $x = 2$ at $t = 0.5$. The arrow at the crossing is the conditional average $v_{0.5}(2) = \frac{1}{2}(4) + \frac{1}{2}(-2) = 1$, which is the single velocity a deterministic field assigns there and the value least-squares regression converges to.

2.14 The flow-matching objective

We now use the network $v_\theta(x, t)$ of Section 2.9, which takes a state $x \in \mathbb{R}^d$ and a time $t \in [0, 1]$ and returns a velocity in $\mathbb{R}^d$. The ideal objective is the expected squared distance between its predicted velocity and the marginal velocity along the path,

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\Big[\big\|v_\theta(X_t, t) - v_t(X_t)\big\|^2\Big],$$

the expectation taken over $t \sim \mathrm{Uniform}(0, 1)$ and $X_t \sim p_t$.

"The flow-matching loss $\mathcal{L}_{\mathrm{FM}}(\theta)$ averages the squared distance between the predicted velocity $v_\theta(X_t, t)$ and the marginal velocity $v_t(X_t)$, over uniformly sampled times t and states X_t drawn from the corresponding density p_t."

We cannot evaluate this objective directly because the target $v_t(X_t)$ is itself a conditional expectation over every endpoint pair compatible with X_t, and that expectation has no closed form for a real data distribution. For a sampled pair, we can compute the conditional target U_t directly. Substituting this target produces an objective that requires one forward pass and one subtraction,

$$\boxed{\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}\Big[\big\|v_\theta(X_t, t) - U_t\big\|^2\Big].}$$

"The conditional flow-matching loss $\mathcal{L}_{\mathrm{CFM}}(\theta)$ averages the squared distance between the predicted velocity $v_\theta(X_t, t)$ and the path velocity U_t, over prior and data endpoint pairs (X_0, X_1) drawn from the coupling π and uniformly sampled times t, which together determine the intermediate state X_t."

During training we sample an endpoint pair and a time, construct the intermediate state, and compute the target,

$$(X_0, X_1) \sim \pi, \qquad t \sim \mathrm{Uniform}(0, 1), \qquad X_t = (1-t)X_0 + tX_1, \qquad U_t = X_1 - X_0.$$

For the linear interpolation the objective is therefore

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}\Big[\big\|v_\theta(X_t, t) - (X_1 - X_0)\big\|^2\Big].$$

For each training example we need one prior sample, one data sample, one time, one interpolation and one subtraction. We never solve an ODE to build a training example, since the intermediate point comes directly from the endpoints.

2.14.1 Least squares and the conditional mean

Let's fix a state x and a time t, and let $a \in \mathbb{R}^d$ be a candidate velocity at that input. The contribution of this input to the objective is the conditional mean-squared error

$$\mathbb{E}\big[\|a - U_t\|^2 \,\big|\, X_t = x\big],$$

where the expectation runs over the endpoint pairs that place the path at x at time t. We write the conditional mean of the target as

$$\mu_t(x) = \mathbb{E}\big[U_t \mid X_t = x\big].$$

We can add and subtract $\mu_t(x)$ inside the difference,

$$a - U_t = (a - \mu_t(x)) + (\mu_t(x) - U_t),$$

then square the norm and take the conditional expectation of both sides,

$$\mathbb{E}\big[\|a - U_t\|^2 \,\big|\, X_t = x\big] = \|a - \mu_t(x)\|^2 + \mathbb{E}\big[\|U_t - \mu_t(x)\|^2 \,\big|\, X_t = x\big].$$

The cross term drops out because the conditional mean of the centered target is zero,

$$\mathbb{E}\big[U_t - \mu_t(x) \mid X_t = x\big] = 0,$$

and $a - \mu_t(x)$ is a constant inside this conditional expectation. The second term on the right is the conditional variance of the target, which does not depend on a. Only the first term does, so the minimum over a is at

$$a^\star = \mu_t(x) = \mathbb{E}\big[U_t \mid X_t = x\big] = v_t(x).$$

"The optimal velocity prediction $a^\star$ at state x and time t equals the conditional mean $\mu_t(x)$ of the sampled velocities U_t given $X_t = x$, which is the marginal velocity $v_t(x)$."

Regression against sampled conditional velocities therefore recovers the marginal velocity field, provided the network has enough capacity to realize the optimum at every input and the optimization reaches the population minimum.

2.14.2 Regression at a crossing

At $x = 2$ and $t = 0.5$ in the crossing example, the two target velocities are 4 and -2, each with probability one half. For a scalar prediction a the average squared error there is

$$\ell(a) = \tfrac{1}{2}(a-4)^2 + \tfrac{1}{2}(a+2)^2.$$

Differentiating and setting the derivative to zero,

$$\ell'(a) = (a-4) + (a+2) = 2a - 2 = 0, \qquad \text{so} \qquad a^\star = 1,$$

which agrees with the conditional average $(4 + (-2))/2 = 1$. Minimizing the squared error therefore recovers the same velocity that we calculated from the net motion at the crossing.

2.15 The conditional flow-matching equivalence

We found the minimizer at one input. Let's apply the same decomposition under the full expectation to compare the conditional and marginal objectives.

Theorem 2.2 (Conditional flow-matching equivalence)**.** *Let $(X_0, X_1) \sim \pi$, let $t \sim \mathrm{Uniform}(0,1)$, let X_t be the interpolation of the pair at time t with conditional velocity U_t, and let $v_t(x) = \mathbb{E}[U_t \mid X_t = x]$ be the marginal velocity. Assume $\mathbb{E}\|U_t\|^2 < \infty$ and $\mathbb{E}\|v_\theta(X_t,t)\|^2 < \infty$ for every θ under consideration. Then*

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathcal{L}_{\mathrm{FM}}(\theta) + \mathbb{E}\Big[\big\|U_t - v_t(X_t)\big\|^2\Big],$$

where the last term does not depend on θ. Consequently

$$\boxed{\nabla_\theta \mathcal{L}_{\mathrm{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\mathrm{FM}}(\theta)}$$

and the two objectives have the same minimizers.

We reuse the decomposition of Section 2.14.1, now with the network output in place of the constant a. Recall the definition of the marginal velocity as a conditional expectation,

$$v_t(X_t) = \mathbb{E}\big[U_t \mid X_t, t\big].$$

Again we add and subtract $v_t(X_t)$, this time inside the residual of the conditional loss,

$$v_\theta(X_t,t) - U_t = \big[v_\theta(X_t,t) - v_t(X_t)\big] + \big[v_t(X_t) - U_t\big].$$

Next, we expand the squared norm and take expectations. The cross term is

$$2\,\mathbb{E}\Big[\big\langle v_\theta(X_t,t) - v_t(X_t),\; v_t(X_t) - U_t\big\rangle\Big],$$

and conditioning on (X_t, t) makes the first factor a constant while the second has conditional mean zero, so the cross term vanishes. The two remaining terms are

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \underbrace{\mathbb{E}\Big[\big\|v_\theta(X_t,t) - v_t(X_t)\big\|^2\Big]}_{\mathcal{L}_{\mathrm{FM}}(\theta)} + \mathbb{E}\Big[\big\|U_t - v_t(X_t)\big\|^2\Big].$$

The second term is the expected conditional variance of the regression target, built entirely from the interpolation and the coupling. No network parameter appears in it, so it differentiates to zero and the two gradients coincide.

The two losses differ by a fixed amount because conditional paths passing through the same state can have different velocities. Their conditional variance contributes to the loss for every network prediction. The spread is a property of the path construction alone, so a parameter update computed from $\mathcal{L}_{\mathrm{CFM}}$ equals the update computed from $\mathcal{L}_{\mathrm{FM}}$. Training against the conditional velocities, each computed by one subtraction, therefore gives the same parameter gradient as training against the marginal field.

2.15.1 The constant loss gap

In the crossing example the two equally likely targets at $x = 2$, $t = 0.5$ are $+4$ and -2, with mean 1. Let $a = v_\theta(2, 0.5)$ be the network prediction there. The conditional loss at that input is

$$\tfrac{1}{2}(a-4)^2 + \tfrac{1}{2}(a+2)^2 = \tfrac{1}{2}(a^2 - 8a + 16) + \tfrac{1}{2}(a^2 + 4a + 4) = a^2 - 2a + 10,$$

and completing the square,

$$\underbrace{\tfrac{1}{2}(a-4)^2 + \tfrac{1}{2}(a+2)^2}_{\text{conditional loss}} = \underbrace{(a-1)^2}_{\text{marginal loss}} + 9.$$

The marginal loss is the squared error against the mean target 1. The constant 9 is the variance of the targets at that input,

$$\tfrac{1}{2}(4-1)^2 + \tfrac{1}{2}(-2-1)^2 = \tfrac{1}{2}(9) + \tfrac{1}{2}(9) = 9.$$

Both loss curves are parabolas in a with leading coefficient one, derivative $2a - 2$, and a minimum at $a = 1$. The conditional loss exceeds the marginal loss by 9 at every prediction. At $a = 1$, the losses are 9 and 0. At either $a = 0$ or $a = 2$, they are 10 and 1. Reducing the conditional loss to 9 therefore reduces the marginal loss to zero. The remaining loss measures the variation in conditional velocities where paths meet. It persists even when the network fits the marginal field exactly.

2.16 The continuity equation for one conditional path

Theorem 2.2 establishes the regression target. To show that this field transports p_0 to p_{data}, we apply the continuity equation to each conditional path and average over the endpoint pairs.

For a fixed endpoint pair (x_0, x_1), the conditional density $p_t(x \mid x_0, x_1)$ is transported by the conditional velocity $u_t(x \mid x_0, x_1)$, so it satisfies its own continuity equation,

$$\frac{\partial}{\partial t} p_t(x \mid x_0, x_1) = -\nabla \cdot \big[p_t(x \mid x_0, x_1)\, u_t(x \mid x_0, x_1)\big].$$

Averaging both sides over the coupling, and exchanging the average with the time derivative and with the divergence, we obtain

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot \mathbb{E}_{X_0, X_1}\Big[p_t(x \mid X_0, X_1)\, u_t(x \mid X_0, X_1)\Big].$$

"The time derivative of the marginal density $p_t(x)$ equals the negative divergence of the average conditional flux, formed by multiplying the conditional density $p_t(x \mid X_0, X_1)$ by its velocity $u_t(x \mid X_0, X_1)$ and averaging over endpoint pairs (X_0, X_1)."

The averaged flux on the right is the marginal flux. We then apply Bayes' rule to the joint density of the state and the endpoints,

$$p_t(x \mid x_0, x_1)\, \pi(x_0, x_1) = p_t(x)\, \pi(x_0, x_1 \mid X_t = x),$$

and substitute it into the integral that defines the averaged flux,

$$\begin{aligned}
\mathbb{E}_{X_0, X_1}\Big[p_t(x \mid X_0, X_1)\, u_t(x \mid X_0, X_1)\Big] &= \int p_t(x \mid x_0, x_1)\, u_t(x \mid x_0, x_1)\, \pi(x_0, x_1)\, dx_0\, dx_1 \\
&= p_t(x) \int u_t(x \mid x_0, x_1)\, \pi(x_0, x_1 \mid X_t = x)\, dx_0\, dx_1 \\
&= p_t(x)\, \mathbb{E}\big[U_t \mid X_t = x\big] \\
&= p_t(x)\, v_t(x).
\end{aligned}$$

The marginal density factors out, and the integral that remains is the conditional expectation that defines the marginal velocity. Substituting back,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot \big[p_t(x)\, v_t(x)\big].$$

"The time derivative of the marginal density $p_t(x)$ equals the negative divergence of its flux $p_t(x)v_t(x)$, so the averaged velocity $v_t(x)$ transports the prescribed density path."

We specify the density path through a coupling and an interpolation, then recover its transporting velocity by least-squares regression. This procedure requires no partial differential equation solve.

Corollary 2.3 (Exact transport)**.** *Suppose the learned velocity matches the marginal velocity,*

$$v_\theta(x, t) = v_t(x)$$

for every x in the support of p_t and every $t \in [0, 1]$, and suppose the ODE $dX_t/dt = v_\theta(X_t, t)$ with $X_0 \sim p_0$ has a unique solution. Let p_t^θ be the density of its solution at time t. Then

$$p_t^\theta = p_t \quad \textit{for all } t \in [0, 1],$$

and in particular

$$\boxed{p_1^\theta = p_{\text{data}}.}$$

Both densities start at p_0 and both satisfy the same continuity equation with the same velocity field, so under the uniqueness assumptions they agree at every time. At the terminal time the prescribed marginal is the data distribution, so the samples produced by integrating the learned field are data samples. Least squares against conditional velocities recovers v_t. Because (p_t, v_t) satisfies the continuity equation, the learned ODE has terminal density p_{data}.

2.17 A complete Gaussian translation

Let's take a one-dimensional prior with mean 0 and variance 1, and build the data endpoint by shifting every prior sample four units to the right,

$$X_0 \sim \mathcal{N}(0,1), \qquad X_1 = X_0 + 4.$$

This is a deterministic coupling. The pair (X_0, X_1) is supported on the line $x_1 = x_0 + 4$, its first marginal is $\mathcal{N}(0,1)$, and its second marginal is $\mathcal{N}(4,1)$, which is the data distribution for this example.

The interpolation is

$$X_t = (1-t)X_0 + t(X_0 + 4) = X_0 + 4t,$$

a rigid translation of the prior sample. Since X_0 is standard normal and $4t$ is a constant at each fixed time, the marginal at time t is $X_t \sim \mathcal{N}(4t, 1)$, with mean $4t$ and variance 1. On the usual grid of times,

$$X_{0.25} \sim \mathcal{N}(1,1), \quad X_{0.50} \sim \mathcal{N}(2,1), \quad X_{0.75} \sim \mathcal{N}(3,1), \quad X_1 \sim \mathcal{N}(4,1).$$

The cloud slides to the right at constant speed and never changes width.

The conditional velocity is the same for every pair in the coupling,

$$U_t = X_1 - X_0 = (X_0 + 4) - X_0 = 4,$$

and a conditional expectation of a constant is that constant, so the marginal velocity is $v_t(x) = \mathbb{E}[U_t \mid X_t = x] = 4$ at every state and time. This leads to the sampling ODE and its solution,

$$\frac{dX_t}{dt} = 4, \qquad X_t = X_0 + 4t,$$

which reproduces the original interpolation, its trajectories, and its marginal densities.

We can check the continuity equation directly here. Let φ be the standard normal density, so that $p_t(x) = \varphi(x - 4t)$. Differentiating in time by the chain rule, and differentiating the flux $p_t(x)v_t(x) = 4\varphi(x-4t)$ in space,

$$\frac{\partial p_t(x)}{\partial t} = -4\,\varphi'(x-4t), \qquad \frac{\partial}{\partial x}\big[4\varphi(x-4t)\big] = 4\,\varphi'(x-4t).$$

The two derivatives have equal magnitudes and opposite signs, satisfying the continuity equation. The constant velocity translates the Gaussian along the line without changing its shape.

2.18 Other interpolants

Theorem 2.2 and Corollary 2.3 apply to other conditional paths with the correct endpoints and a velocity we can calculate by differentiation. We can construct such paths by replacing the weights $1-t$ and t by arbitrary time-dependent coefficients,

$$X_t = \alpha_t X_0 + \beta_t X_1.$$

"The intermediate state X_t equals the prior endpoint X_0 weighted by the scalar schedule α_t, plus the data endpoint X_1 weighted by the scalar schedule β_t, with both weights depending on time t."

To preserve the two ends the schedules must satisfy $\alpha_0 = 1$, $\beta_0 = 0$, $\alpha_1 = 0$ and $\beta_1 = 1$. Differentiating the path with the endpoints held fixed,

$$U_t = \dot{\alpha}_t X_0 + \dot{\beta}_t X_1,$$

where $\dot{\alpha}_t$ and $\dot{\beta}_t$ are the time derivatives of the two schedules. This is the target for the same squared-error regression. The linear interpolation is the case $\alpha_t = 1 - t$, $\beta_t = t$, with $\dot{\alpha}_t = -1$ and $\dot{\beta}_t = 1$, which returns $U_t = X_1 - X_0$. This construction is a stochastic interpolant between two distributions [Albergo and Vanden-Eijnden, 2023]. In diffusion models, analogous schedules specify the signal and noise coefficients.

2.18.1 Changing the speed

A schedule can change how quickly a sample moves away from the prior and toward the data, without changing the geometry of the path. We now take

$$X_t = (1 - t^2)X_0 + t^2 X_1.$$

The endpoints are correct, since $t^2 = 0$ at $t = 0$ and $t^2 = 1$ at $t = 1$. The state moves along the same straight segment as before, starting slowly and accelerating toward the data. Halfway through the time interval the weight on the data endpoint is $\beta_{0.5} = 0.25$, so the sample has covered a quarter of the distance.

Differentiating with the endpoints fixed,

$$U_t = \frac{dX_t}{dt} = 2t\,(X_1 - X_0).$$

"The conditional velocity U_t under the quadratic schedule equals twice the time t times the displacement from the prior endpoint X_0 to the data endpoint X_1, increasing the speed from zero to $2\|X_1 - X_0\|$ over the unit interval."

2.18.2 Changing the geometry

A schedule can also bend the path. Here we take

$$X_t = \cos\left(\frac{\pi t}{2}\right) X_0 + \sin\left(\frac{\pi t}{2}\right) X_1.$$

At $t = 0$ the coefficients are $(1, 0)$ and at $t = 1$ they are $(0, 1)$, so the endpoints are again correct. For endpoints that are not collinear with the origin the path curves between them, because the coefficient pair traces a quarter circle. For this schedule, if X_0 and X_1 are independent, centered, and each has identity covariance, then

$$\mathrm{Cov}(X_t) = \cos^2\left(\frac{\pi t}{2}\right) I + \sin^2\left(\frac{\pi t}{2}\right) I = I,$$

so the interpolation preserves identity covariance at every time.

Differentiating,

$$U_t = \frac{\pi}{2}\left[-\sin\left(\frac{\pi t}{2}\right) X_0 + \cos\left(\frac{\pi t}{2}\right) X_1\right].$$

"The conditional velocity U_t under the trigonometric schedule equals $\pi/2$ times the sum of the prior endpoint X_0 weighted by $-\sin(\pi t/2)$ plus the data endpoint X_1 weighted by $\cos(\pi t/2)$, so the direction changes with time t."

For each interpolant, we construct the intermediate state and differentiate in time with the endpoints held fixed. We then train the network to predict that derivative by regression.

2.18.3 Comparing interpolants

Let's put the three side by side at one state and one time, with the two-dimensional endpoints $X_0 = (2, 0)$ and $X_1 = (0, 2)$, evaluated at one quarter of the interval, $t = 0.25$. Under linear interpolation,

$$X_{0.25} = 0.75\,(2, 0) + 0.25\,(0, 2) = (1.50,\ 0.50), \qquad U_{0.25} = (-2,\ 2).$$

The quadratic schedule has $t^2 = 0.0625$, so

$$X_{0.25} = 0.9375\,(2, 0) + 0.0625\,(0, 2) = (1.875,\ 0.125), \qquad U_{0.25} = 2(0.25)(-2, 2) = (-1,\ 1).$$

The trigonometric schedule has $\cos(\pi/8) = 0.9239$ and $\sin(\pi/8) = 0.3827$, so

$$X_{0.25} = (1.848,\ 0.765), \qquad U_{0.25} = \frac{\pi}{2}\big(-0.3827 \cdot (2, 0) + 0.9239 \cdot (0, 2)\big) = (-1.202,\ 2.902).$$

We can compare the three positions and velocities in Table 2.1.

Table 2.1: The three interpolants at $X_0 = (2, 0)$, $X_1 = (0, 2)$ and $t = 0.25$. The linear path has covered 25% of the straight segment; the quadratic schedule has covered 6.25%, because progress follows t^2; the trigonometric schedule sits off the segment entirely and its velocity points in a different direction.

Interpolant	Position X_t	Velocity U_t
Linear	(1.50, 0.50)	(−2, 2)
Quadratic	(1.875, 0.125)	(−1, 1)
Trigonometric	(1.848, 0.765)	(−1.202, 2.902)

If we draw the three paths together, we can separate the effect of the schedule from the effect of the geometry (Figure 2.6). Changing the interpolant changes the intermediate distributions and the regression targets, which changes how hard the velocity field is to fit and how many integration steps a sampler needs at test time.

2.18.4 Interpolants with prescribed endpoint velocities

With a schedule of the form $\alpha_t X_0 + \beta_t X_1$, we set endpoint positions, and the schedule determines their velocities. We may also have endpoint velocities to match. For example, cell states measured at successive timepoints include a direction of change at each observation. Ignoring these directions creates abrupt changes in slope at the observations. A cubic Hermite interpolant uses four inputs: the two endpoint positions and a slope at each. On one interval rescaled to $t \in [0, 1]$,

$$X_t = (2t^3 - 3t^2 + 1)X_0 + (-2t^3 + 3t^2)X_1 + (t^3 - 2t^2 + t)M_0 + (t^3 - t^2)M_1,$$

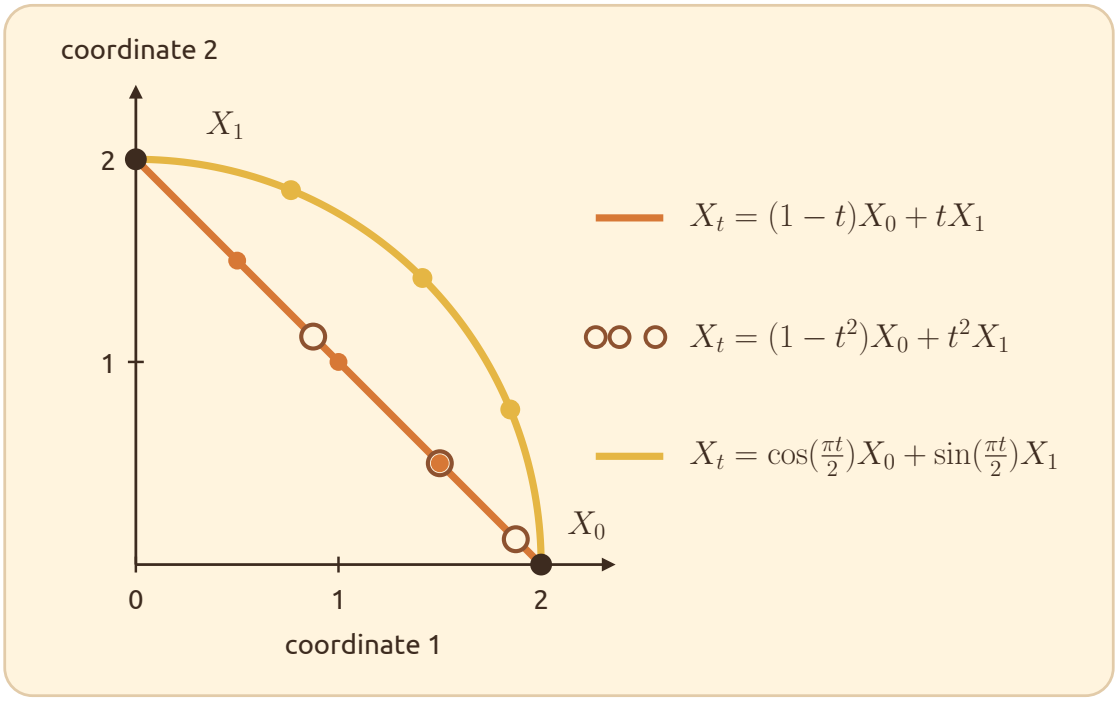


Figure 2.6: The same endpoints $X_0 = (2, 0)$ and $X_1 = (0, 2)$ joined by three interpolants, with markers at $t = 0$, 0.25, 0.5, 0.75, 1. The linear path is a straight segment with evenly spaced markers. The t^2 schedule follows the same segment with markers that spread apart as the sample accelerates. The cos/sin schedule leaves the segment and traces a quarter circle, so its velocity changes direction along the way.

where M_0 and M_1 are the endpoint slopes, each multiplied by the duration of the original interval so that the formula is correct after rescaling.

"The intermediate state X_t is a cubic combination of the endpoints X_0 and X_1 and their slope terms M_0 and M_1, where each slope is scaled by the original interval duration so the path follows the measured endpoint directions after rescaling time to $[0, 1]$."

Substituting $t = 0$ and $t = 1$ into the four weights returns X_0 and X_1, so the endpoints are preserved. Differentiating,

$$U_t = (6t^2 - 6t)(X_0 - X_1) + (3t^2 - 4t + 1)M_0 + (3t^2 - 2t)M_1,$$

and the two ends of this expression are $U_0 = M_0$ and $U_1 = M_1$. The slope terms are exactly the velocities at the two ends, which is why choosing them monotonically limits overshoot between observations. To train with this interpolant, we construct X_t, compute U_t using this formula, and minimize the squared prediction error.

2.19 Training and sampling

Let's combine the interpolation and loss in one training iteration. First, draw an endpoint pair $(X_0, X_1) \sim \pi$ and a time $t \sim \text{Uniform}(0, 1)$. Use them to construct the intermediate state $X_t =$

$(1-t)X_0 + tX_1$ and calculate the target velocity $U_t = X_1 - X_0$. We then take a gradient step on

$$\|v_\theta(X_t, t) - U_t\|^2.$$

The network receives the current state X_t and the current time t. It does not receive the endpoint pair that produced them, because at sampling time that pair does not exist. Across many examples the squared-error regression converges toward the marginal velocity $v_t(x)$, by Section 2.14.1. We can sample any time directly and construct X_t in closed form, so building a training example never requires simulating the ODE.

After training we discard the paired data endpoint, begin with a fresh prior sample $X_0 \sim p_0$, and solve

$$\frac{dX_t}{dt} = v_\theta(X_t, t)$$

from $t = 0$ to $t = 1$ with a numerical integrator, usually Euler's method on a uniform grid. The terminal state is

$$X_1 = X_0 + \int_0^1 v_\theta(X_s, s)\, ds,$$

and if the learned field matches the target marginal velocity then $X_1 \sim p_{\text{data}}$ by Corollary 2.3. In training we evaluate the network once per example, on a state built from known endpoints. During generation, we evaluate the network once per integration step at the state obtained from the previous step. The cost per sample is therefore the number of steps times the cost of one forward pass.

2.19.1 Conditioning the flow

We can condition generation by supplying extra inputs to the velocity network. Let c be a conditioning variable attached to each data sample, and let the network be $v_\theta(x, t, c)$. During training we draw the pair (X_1, c) together from the labelled data, pair X_1 with a prior sample as before, and minimize

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\|v_\theta(X_t, t, c) - U_t\|^2\Big].$$

The same derivations apply with c included, since c is fixed within a training example and the conditional expectation that defines the marginal velocity becomes an expectation given (X_t, t, c). For molecular generation c can encode a target protein sequence, a desired molecular class, or a measured biological context; for images it can be a class label or an embedded text prompt. Classifier and classifier-free guidance control how strongly generation follows the condition.

2.20 Flow matching on MNIST

Let's train an unconditional flow model on MNIST in PyTorch [Paszke et al., 2019], with a small U-Net [Ronneberger et al., 2015] as the velocity parametrization. The state is a 28×28 grayscale image, so $d = 784$ and the network predicts one signed velocity per pixel. The script walked through here is `lecture_2/flow_matching_unet_lecture.py` in the course repository at `https://huggingface.co/ChatterjeeLab/CIS6270`.

2.20.1 Imports and configuration

The script opens with the imports.

```python
from pathlib import Path  # Dataset and output paths.

import torch  # Tensors, gradients, and optimization.
import torch.nn as nn  # Layers and model classes.
import torch.nn.functional as F  # Resize feature maps in the decoder.
from torch.utils.data import DataLoader  # Shuffle and batch real images.
from torchvision import datasets
from torchvision.transforms import v2  # Convert and normalize image pixels.
from torchvision.utils import save_image  # Export tensors as PNG grids.
```

We take tensors and automatic differentiation from `torch`, the layer types from `nn`, and the stateless operations from `F`, whose only use here is resizing feature maps in the decoder. We use `torchvision` to load MNIST, convert and normalize pixels with `v2`, and save batches of tensors as PNG grids. `DataLoader` shuffles and batches the real images.

We specify the training settings, output paths, and device in one configuration block.

```python
# %% 02. Configuration
BATCH_SIZE = 128  # Images per training batch.
EPOCHS = 20  # Passes through the training set.
LEARNING_RATE = 2e-4  # Optimizer step size.
BASE_CHANNELS = 32  # U-Net widths: 32, 64, 128.
SAMPLE_STEPS = 100  # Euler steps from t=0 to t=1.
NUM_SAMPLES = 16  # Generate a 4-by-4 image grid.
SEED = 7  # Seed for initialization and training.
DATA_DIR = Path("data")  # MNIST download location.
OUTPUT_DIR = Path("flow_matching_outputs")  # Checkpoint and generated image
    folder.

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()  # NVIDIA GPU.
    else "mps" if torch.backends.mps.is_available()  # Apple GPU.
    else "cpu"  # CPU fallback.
)
```

We set the batch size, number of training epochs, learning rate, and U-Net width here. We also choose the number of Euler sampling steps, sample-grid size, and random seed. `BASE_CHANNELS` fixes the three U-Net widths at 32, 64 and 128, and `SAMPLE_STEPS` fixes the integration grid at $\Delta t = 1/100$. We select the device once, using an NVIDIA GPU if available, an Apple GPU otherwise, and the CPU as the final fallback.

2.20.2 Loading the images

For the data endpoint X_1, we rescale a real MNIST image to a range comparable to the Gaussian prior.

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 ->
            floats 0-1.
```

```
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] ->
            [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use
            the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1,
            28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main
            process.
    )
```

We apply three transformations to each image. With `ToImage`, we obtain the channel-first shape $[1, 28, 28]$. With `ToDtype` and `scale=True`, we convert integer pixels from 0-255 to floats in $[0, 1]$. Finally, `Normalize` subtracts 0.5 and divides by 0.5, mapping the pixels to $[-1, 1]$. This zero-centered, unit-scale range makes the data comparable to the standard Gaussian prior. The loader downloads the training split, transforms each retrieved image, and returns shuffled batches of 128 images with shape $[B, 1, 28, 28]$. We load in the main process and enable pinned memory on CUDA.

2.20.3 Sampling the conditional path

Given a batch of data endpoints, we sample prior endpoints and times, calculate the intermediate states, and return the target velocities.

```
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image;
        [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t
        =1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

We implement the conditional path from Section 2.12 using `randn_like` to draw the prior endpoint X_0. This independent standard Gaussian noise has the same shape and device as the real-image batch. We draw one uniform time per image, using `x1.shape[0]` for the batch size. Each image therefore occupies its own position along its path. Indexing with `[:, None, None, None]` adds three trailing dimensions to broadcast the time across channels, height, and width. We evaluate $X_t = (1 - t)X_0 + tX_1$ pixel by pixel and calculate the target $U_t = X_1 - X_0$, one signed velocity per pixel. The function returns the intermediate images, times, and target velocities.

2.20.4 The convolution block

We build the network for v_θ from a repeated convolution block.

```
# %% 05. Define a simple convolution block
def conv_block(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1), nn.SiLU(),  # Keep
            H and W.
```

```
        nn.Conv2d(out_channels, out_channels, 3, padding=1), nn.SiLU(),  # Refine features.
    )
```

In each U-Net block, we apply a 3×3 convolution from `in_channels` to `out_channels`, then a `SiLU` activation. Setting `padding=1` preserves the image height and width. We follow this with a second convolution and activation at the same channel width and resolution. Each encoder, bottleneck, and decoder stage uses this block.

2.20.5 The U-Net

In the constructor, we define the encoder, bottleneck, decoder, and output convolution with their respective channel widths.

```
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.
```

Subclassing `nn.Module` and calling `super().__init__()` starts PyTorch's tracking of layers and parameters. With $c = 32$, the encoder converts the two input channels, the image and the time, into 32 feature maps and then into 64. The bottleneck processes the lowest-resolution representation at 128 channels. Each decoder block receives the upsampled features concatenated with the encoder features saved at that resolution, so its input width is the sum of the two: $4c + 2c = 192$ channels become $2c = 64$, then $2c + c = 96$ become $c = 32$. A 1×1 convolution reduces the final 32 channels to one signed velocity per pixel. Max pooling over each 2×2 region halves the height and width. The whole model holds 471,265 parameters, small enough to train on a laptop CPU.

In the forward method, we apply these layers to the input state and time.

```
    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
```

```
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

Calling `model(xt, t)` executes this forward method. We first expand each scalar time into a constant channel with the same dimensions as `xt`, allowing the predicted field to vary with time. Concatenating the image and time channels produces the shape $[B, 2, 28, 28]$. We retain the first encoder features as `skip1` at 28×28, pool to 14×14, and retain the second encoder features as `skip2`. Another pooling step reduces the resolution to 7×7 for the bottleneck.

To decode, we resize the bottleneck features to the height and width of `skip2` using nearest-neighbor interpolation. We concatenate them with the saved encoder features and apply the first decoder block. We repeat this at full resolution with `skip1`. The output convolution produces the predicted velocity $v_\theta(X_t, t)$, matching the input image's shape. Through the skip connections, each pixel's velocity depends on both the digit's coarse layout and its local detail.

2.20.6 The training loop

In one pass over the dataset we carry out the five steps of Section 2.19 once per batch.

```
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for
            this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel
            .
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average
            images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch
            size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We select training mode with `model.train()` and initialize two counters for the reported loss. For each batch, we discard the digit labels for unconditional training and move the images to the selected hardware. Calling `sample_conditional_path` constructs intermediate images and exact target velocities from fresh noise and times, so the same training pair never appears in two epochs. After predicting one velocity per pixel, we compute each pixel's squared error with `(v_pred - ut).square()`. Flattening each image and summing over its pixels gives the squared Euclidean norm $\|v_\theta(X_t, t) - U_t\|^2$ in $\mathbb{R}^{784}$. Averaging over the batch estimates $\mathcal{L}_{\mathrm{CFM}}(\theta)$. Before backpropagation, we clear the old gradients because PyTorch otherwise accumulates them. We then call `backward` to differentiate the loss with respect to the trainable parameters and `step` to update them. Weighting the running loss by the actual batch size keeps a smaller final batch from distorting the epoch average.

2.20.7 Euler sampling

To generate, we run the solver of Section 2.5 on the trained field, keeping a few intermediate states along the way.

```
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

We disable gradient recording with the decorator to reduce memory use and select evaluation mode with `model.eval()`. Before allocating tensors, we check that both the sample count and step count are positive. We obtain the device from the model parameters, then initialize each sample with a Gaussian noise image, corresponding to $X_0 \sim p_0$. Dividing the unit interval into `steps` equal increments gives $\Delta t = 1/100$ with the configuration above. We keep an independent CPU copy of the starting noise for plotting. At each loop iteration, we assign the current time to every sample, evaluate the velocity at the current state and time, and advance by

$$x \leftarrow x + v_\theta(x, t)\,\Delta t,$$

which is one Euler step of the generation ODE. We save a snapshot every `steps // 4` steps. The second condition also saves the final state when the step count is not a multiple of four. With 100 steps the saved times are $t = 0$, 0.25, 0.5, 0.75, 1, five states per sample. The function returns the final images and the snapshots stacked along a new time dimension, shape $[B, 5, 1, 28, 28]$, both unclipped so that the raw ODE states remain available for inspection. For one sample we therefore run 100 forward passes through the U-Net, a hundredfold multiple of the single forward pass behind one training example.

2.20.8 Running training and sampling

We assemble the data loader, model, and optimizer in the main function, then train for the configured number of epochs and save a checkpoint.

```
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
```

```
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")
```

We set the random seed to make the run reproducible. Then we create the output folder if needed and initialize the data loader. We instantiate the U-Net on the selected hardware and pass its parameters to `AdamW`. Setting weight decay to zero avoids shrinking the parameters when the regression target has a fixed scale. Counting parameters with `numel()`, which returns the number of entries in each parameter tensor, confirms the 471,265 figure. The training loop runs `EPOCHS` passes over the dataset, prints the epoch loss, and keeps the history. The checkpoint stores the weights, the loss curve, the base width and the seed, which is everything needed to rebuild the model and repeat the run.

We complete the function by generating and saving images, then call it when the script runs.

```
    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )


# %% 13. Run the script
if __name__ == "__main__":  # Execute when launched directly.
    main()  # Load data, train, sample, and save.
```

We use a second seed to draw the same 16 initial noise images each time the script runs. For both saved grids, we reverse the normalization from $[-1, 1]$ to $[0, 1]$ and clamp the display values. This affects the PNGs; the tensors returned by `sample_images` retain the ODE states. We save the 16 generated digits with four per row. For the trajectory grid, we select the first four samples, flatten the sample and time dimensions, and set `nrow` to the number of snapshots. Each row then follows one sample across times $t = 0$, 0.25, 0.5, 0.75, 1.

The script's default is 20 epochs. The lecture's own run used the same configuration with five epochs on a CPU, which is already enough to produce recognizable digits. Along a trajectory the image begins as pure noise, acquires a bright blob in the right region of the frame by $t = 0.5$, and sharpens into a stroke pattern over the last quarter of the interval. We trained a velocity field in pixel space using linear interpolation and squared error. Integrating the learned field transports Gaussian noise to the data distribution. Training required neither likelihood evaluation nor a divergence trace or ODE solve.

2.21 Path choice and stochastic sampling

To train with flow matching, we choose a coupling and an interpolation that determine the density path p_t. Its transporting velocity is a conditional expectation, which we recover by least-squares regression against sampled conditional velocities. We can therefore learn the marginal field without evaluating that expectation directly. Theorem 2.2 establishes the equivalence of the training objectives, and Corollary 2.3 connects the learned field to the terminal distribution.

We specified the density path by hand. The interpolants in Section 2.18 produce different regression targets, and we have not established a principle for choosing among them. We generated samples with an ODE. Each prior sample therefore follows a deterministic, continuous trajectory, and trajectories cannot cross. This deterministic map constrains the terminal distribution. If a step reaches a region where the network is inaccurate, the dynamics contain no recovery mechanism.

We can add noise by specifying a stochastic process with a Gaussian conditional law whose mean and variance follow a chosen schedule. The dynamics then obey a stochastic differential equation, with a reverse-time drift that depends on the score of the marginal density. The posterior mean of the clean sample given a noisy one is a conditional expectation, so we can recover it with the same least-squares argument as in Section 2.14.1.

In Chapter 3, we will build this reverse process from a corruption rule that we can simulate. Because many clean samples can produce the same noisy state, reversal depends on their relative probabilities. We will derive how the score determines the reverse motion and learn it through denoising. We can then generate samples without evaluating the noisy data density.

Chapter 3

Score Matching and Diffusion Models

3.1 Stochastic generative paths

In Chapter 2, we generated samples by learning a velocity field. We draw an endpoint pair from a coupling, interpolate between them, and regress a network v_θ on the velocity of that interpolation. We then move a fresh prior sample toward data by solving $dX_t/dt = v_\theta(X_t, t)$ with the trained field. Once we draw the initial state, the trajectory is determined. At every step, we evaluate the same trained field at the current point. All of the randomness in a generated sample comes from the initial draw.

We can also connect a data distribution to a simple prior by adding noise. Starting at a clean sample, we repeatedly add small Gaussian perturbations and shrink the remaining signal according to a schedule. The state eventually has a standard Gaussian distribution and retains no trace of where it began. Because we choose the perturbations ourselves, we can generate a corrupted training example at any level of corruption and record exactly which noise produced it.

Generation runs this corruption backwards. One noisy state is consistent with many clean samples, so the backward direction at a given state is not determined by that state alone. To choose a backward direction, we need to know how the density of noisy states changes near the current position. We obtain this local information from the score, the gradient of the log density.

3.1.1 Time convention

We set $t = 0$ at a clean sample $X_0 \sim p_{\text{data}}$, and the state becomes noisier as t grows, reaching an approximately standard Gaussian at the terminal time T. Generation traverses the same interval from T back to 0. This follows the convention of the diffusion literature. In Chapter 2, we used the opposite direction, with the prior at $t = 0$ and the data at $t = 1$.

Two symbols also change meaning from Chapter 2. We write the drift of a stochastic process as $f(x, t)$ and its noise scale as $g(t)$, following the diffusion literature. The symbols v_θ and σ_t denote the learned velocity and the corruption path's noise standard deviation, respectively. The diffusion coefficient of a general stochastic differential equation remains $g(t)$. We can write $\nabla^2\varphi = \sum_i \partial^2\varphi/\partial x_i^2$ for the Laplacian, leaving the symbol Δ free for increments.

A stochastic differential equation describes one corrupted trajectory through random increments. To describe the density of these trajectories, we use Itô's lemma and integrate by parts. With a suitable drift and noise scale, the conditional distribution at every time is Gaussian, so we can sample it in a single draw.

3.2 Drift and Brownian increments

We specify a deterministic trajectory through its velocity at every point and time. For a random trajectory, we also specify the perturbation applied over each small interval. We can write both parts of the motion in a single differential,

$$dX_t = f(X_t, t)\, dt + g(t)\, dW_t.$$

"The state increment dX_t equals the drift $f(X_t, t)$, evaluated at the current state X_t and time t, multiplied by the time increment dt, plus the noise scale $g(t)$ multiplied by the Brownian increment dW_t."

- $X_t \in \mathbb{R}^d$ is the state at time t, a random vector with d coordinates.
- $f : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$ is the drift, the average velocity of the state at position x and time t, in units of state per unit time.
- $g : [0, T] \to \mathbb{R}_{>0}$ is the diffusion coefficient, a scalar that sets the strength of the noise, in units of state per square root of time.
- W_t is a standard Brownian motion in $\mathbb{R}^d$, and dW_t is its increment over an infinitesimal interval.

An equation of this form is a stochastic differential equation. The drift fixes the average displacement over a short interval, and the Brownian increment spreads the possible next states around that average. If we set $g \equiv 0$, we recover the ordinary differential equation of Chapter 2, with f in the role of the velocity field.

Brownian motion starts at the origin, $W_0 = 0$, and has continuous paths. Increments over disjoint intervals are independent. An increment over an interval of length h is Gaussian with mean zero and covariance hI, where I is the $d \times d$ identity.

3.3 Brownian increment scaling

Let's fix a time t and an interval length $h > 0$, and write $\Delta W = W_{t+h} - W_t$ for the increment over that interval. Its law is Gaussian, so we can produce it from a standard Gaussian draw,

$$\Delta W = \sqrt{h}\, \varepsilon, \qquad \varepsilon \sim N(0, I).$$

"The Brownian displacement ΔW over a time interval of length h equals the square root of h times a standard Gaussian vector ε, whose mean is zero and whose covariance is the identity matrix I."

Multiplying a random vector by the scalar $\sqrt{h}$ multiplies its covariance by h, so

$$\mathrm{Cov}(\Delta W) = h\ \mathrm{Cov}(\varepsilon) = hI.$$

"The covariance of the Brownian displacement ΔW equals the elapsed time h times the identity matrix I, so each coordinate has variance h and different coordinates are uncorrelated."

The typical increment therefore has size $\sqrt{h}$ in each coordinate. Halving the interval halves the variance and divides the typical displacement by $\sqrt{2} \approx 1.414$. At $h = 0.04$, a coordinate of ΔW has standard deviation $\sqrt{0.04} = 0.2$. At $h = 0.02$, its standard deviation is $\sqrt{0.02} = 0.1414$, smaller by a factor of 1.414. A tenfold grid refinement shrinks the typical increment by $\sqrt{10} \approx 3.162$.

Brownian motion W_t accumulates increments from the origin. At time t, it is Gaussian with mean zero and covariance tI, so its typical distance from the starting point grows as $\sqrt{t}$. The envelope $\pm\sqrt{t}$ marks one standard deviation. Roughly a third of the paths lie outside it at any fixed time. The eight sampled paths in Figure 3.1 wander within this widening envelope and occasionally cross it.

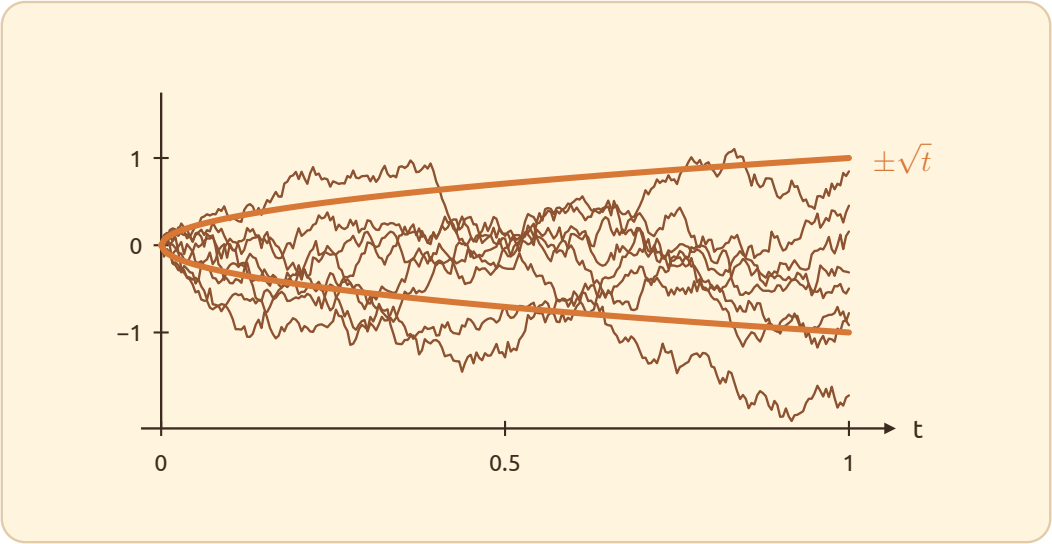


Figure 3.1: Eight sampled paths of a one-dimensional standard Brownian motion on the interval $0 \le t \le 1$, simulated with 240 increments of width $1/240$. The horizontal axis is time, the vertical axis is the position W_t, and all eight paths start at $W_0 = 0$. The two heavy curves are the envelope $\pm\sqrt{t}$, which is one standard deviation of W_t, so roughly two thirds of the paths lie inside it at any fixed time.

A smooth path moves a distance of order h over an interval of length h. The ratio of displacement to elapsed time therefore stays bounded, and the derivative exists. A Brownian path moves a distance of order $\sqrt{h}$, and the ratio $\sqrt{h}/h = 1/\sqrt{h}$ grows without bound as the interval shrinks: at $h = 10^{-4}$ the ratio is already 100. No derivative dW_t/dt exists, which is why the increment dW_t appears in the equation and never a velocity.

3.4 The Euler-Maruyama step

In Chapter 2, we solved $dx_t/dt = v_t(x_t)$ by replacing the differential with a finite step. We can make the same replacement here and add a noise term that scales as the square root of the step size. Starting from the state x_k at time t_k and taking a step of length h,

$$x_{k+1} = x_k + f(x_k, t_k)\, h + g(t_k)\sqrt{h}\, \varepsilon_k, \qquad \varepsilon_k \sim N(0, I) \text{ independent across } k.$$

"The next state x_{k+1} equals the current state x_k, plus the drift $f(x_k, t_k)$ at time t_k times the step size h, plus the noise scale $g(t_k)$ times the square root of h and a fresh standard Gaussian draw ε_k."

- $k = 0, 1, 2, \ldots$ indexes the steps, and $t_k = t_0 + kh$ is the time at step k.
- $h > 0$ is the step size, a time.
- $\varepsilon_k \sim N(0, I)$ is the Gaussian draw used at step k, independent of the draws at every other step and of the state.

This is the Euler-Maruyama scheme. We retain the Euler drift step from Chapter 2 and add the noise term. That term has the exact law of a Brownian increment over an interval of length h. The scheme's discretization error therefore comes from freezing f and g at the left endpoint.

3.4.1 Euler-Maruyama example

Let's take a one-dimensional state at $x_k = 2$, with drift $f = -1$ and noise scale $g = 0.5$, and step forward by $h = 0.04$ using the Gaussian draw $\varepsilon_k = 1.5$. The drift contributes

$$fh = (-1)(0.04) = -0.04,$$

and the noise contributes

$$g\sqrt{h}\, \varepsilon_k = (0.5)(0.2)(1.5) = 0.15,$$

so the step lands at

$$x_{k+1} = 2 - 0.04 + 0.15 = 2.11.$$

If we had followed the drift alone, the state would have reached 1.96, a move of 0.04 toward the origin. With this particular draw, the state rose to 2.11. A single path can therefore move against the average direction. The noise scale $g\sqrt{h}$ determines the size of this departure. Comparing the two contributions,

$$\frac{g\sqrt{h}}{|f|\, h} = \frac{0.5 \times 0.2}{1 \times 0.04} = \frac{0.1}{0.04} = 2.5,$$

the typical noise displacement is 2.5 times the drift displacement at this step size. The ratio $g/(|f|\sqrt{h})$ grows as the step shrinks. Over one step the noise dominates. Over many steps, the drift accumulates linearly in elapsed time and the noise accumulates as its square root, so the drift eventually dominates.

3.5 Noise accumulation

Each Euler-Maruyama step draws a new Gaussian noise vector ε_k. If we run the scheme twice from the same initial state, we obtain two different trajectories. In Chapter 2, the starting point determined the entire path. Here it determines a distribution of possible paths, so each sample path is one draw from that distribution.

We can repeat the step of Section 3.4.1 with three different draws. Holding $x_k = 2$, $f = -1$, $g = 0.5$ and $h = 0.04$ fixed, the draws $\varepsilon_k = 1.5$, -0.5 and 0.2 land at

$$2.11, \qquad 1.91, \qquad 1.98,$$

so we obtain three endpoints from the same starting state and update rule. Their average is 2.00, above the drift prediction 1.96, because these three noise draws average to 0.4. With more draws, the endpoint average settles on 1.96. Continue the first path for a second step with $\varepsilon_{k+1} = -0.8$. It lands at $2.11 - 0.04 - 0.08 = 1.99$. The drift has contributed the same displacement twice. The two noise contributions have partly cancelled.

To see how the population behaves, we start every particle at the same point x and take one step. The next state is Gaussian,

$$x + f(x,t)h + g(t)\sqrt{h}\,\varepsilon \sim N\Big(x + f(x,t)h,\ g(t)^2 h\, I\Big),$$

"After one step from state x, the new state is Gaussian with mean $x + f(x,t)h$, the starting point plus drift over a time interval h, and covariance $g(t)^2 hI$, the squared noise scale times elapsed time and the identity matrix I. The drift moves the center, and the noise sets the width."

With the numbers of Section 3.4.1, a cloud of particles started at $x = 2$ has mean 1.96 and standard deviation $g\sqrt{h} = 0.1$ after one step of size 0.04. The center has moved by 0.04 and the cloud has acquired a width of 0.1 around it.

When we repeat the step without a drift, the variance accumulates additively, because the increments are independent. After N steps of size h with constant g, the variance of the state is $Ng^2h = g^2T$ with $T = Nh$ the elapsed time. At $g = 0.5$ and $T = 1$ the variance is 0.25 and the standard deviation is 0.5, and both grow without bound as T increases. A corruption process built this way would spread the data over all of $\mathbb{R}^d$ and never settle on a fixed distribution. To bound the spread, we pull the state back toward the origin at a rate matched to the injected noise.

3.6 A restoring drift

We can take the drift proportional to the negative of the current state, with a constant noise scale,

$$dX_t = -\kappa X_t\, dt + g\, dW_t.$$

"The state increment dX_t equals the negative restoring rate κ times the current state X_t and time increment dt, plus the constant diffusion coefficient g times the Brownian increment dW_t."

Here $\kappa > 0$ is the restoring rate, in units of inverse time, and $g > 0$ is a constant noise scale. The drift points from X_t toward the origin and its magnitude grows with the distance, so the restoring displacement is large far from the origin and small near it. This is the Ornstein-Uhlenbeck process.

3.6.1 Derivation of the transition law

We multiply the state by the growing factor $e^{\kappa t}$ and differentiate the product. The factor is deterministic and smooth, so the ordinary product rule applies,

$$d\Big(e^{\kappa t}X_t\Big) = \kappa e^{\kappa t}X_t\,dt + e^{\kappa t}\,dX_t = \kappa e^{\kappa t}X_t\,dt + e^{\kappa t}\left(-\kappa X_t\,dt + g\,dW_t\right) = e^{\kappa t}g\,dW_t.$$

The drift has cancelled, leaving a differential with no dt term at all. Now, we can integrate from 0 to t and divide by $e^{\kappa t}$,

$$X_t = e^{-\kappa t}x_0 + g\int_0^t e^{-\kappa(t-s)}\,dW_s.$$

"The state X_t at time t equals the initial state x_0 damped by $e^{-\kappa t}$, plus the Brownian increments dW_s accumulated from zero to t, each scaled by the noise coefficient g and damped by $e^{-\kappa(t-s)}$ from its arrival time s."

The integral against dW_s is a sum of independent Gaussian increments with deterministic coefficients, so it is Gaussian with mean zero. To calculate its variance, approximate the integral by $\sum_j e^{-\kappa(t-s_j)}\Delta W_j$. The increments are independent, so these terms are uncorrelated. The term at s_j has variance $e^{-2\kappa(t-s_j)}\Delta s$. We can therefore add the variances and take the limit to obtain an ordinary integral,

$$\mathrm{Var}\left(g\int_0^t e^{-\kappa(t-s)}dW_s\right) = g^2\int_0^t e^{-2\kappa(t-s)}\,ds = \frac{g^2}{2\kappa}\left(1-e^{-2\kappa t}\right).$$

By the Itô isometry, the variance of this stochastic integral equals the ordinary integral of the squared integrand. We can now collect the mean and variance,

$$X_t \mid x_0 \;\sim\; N\left(e^{-\kappa t}x_0,\; \frac{g^2}{2\kappa}\left(1-e^{-2\kappa t}\right)I\right).$$

"Given the initial state x_0, the state X_t is Gaussian with mean $e^{-\kappa t}x_0$, the damped initial state, and covariance equal to the stationary variance $g^2/(2\kappa)$ times $1-e^{-2\kappa t}$ and the identity matrix I. Here κ is the restoring rate and g is the noise scale."

The contribution of the initial state decays geometrically at rate κ. The variance rises from zero toward the stationary value $g^2/(2\kappa)$. At this variance, the inward drift exactly offsets the outward spreading caused by noise.

3.6.2 Signal and noise variance

Let's start at $x_0 = 2$ with restoring rate $\kappa = 1$ and noise scale $g = \sqrt{2}$, and evaluate at $t = \log 2 \approx 0.6931$. The damping factor is

$$e^{-\kappa t} = e^{-\log 2} = \tfrac{1}{2},$$

so the conditional mean is

$$\mathbb{E}[X_t \mid x_0] = \tfrac{1}{2}\times 2 = 1,$$

and half of the initial signal remains. For the variance, $e^{-2\kappa t} = e^{-2\log 2} = 1/4$ and $g^2/(2\kappa) = 2/2 = 1$, so

$$\mathrm{Var}(X_t \mid x_0) = 1\times\left(1-\tfrac{1}{4}\right) = 0.75,$$

a standard deviation of $\sqrt{0.75} = 0.8660$. The stationary variance for these constants is 1, and at $t = \log 2$ the variance has covered three quarters of the distance to it. Letting t grow further, the mean approaches zero and the variance approaches one, so the state no longer depends on x_0 and settles into a standard Gaussian. The choice $g^2 = 2\kappa$ gives a limiting variance of one.

For generation, we need to describe how the density of the whole population evolves.

3.7 Changes in smooth measurements

The density p_t describes the whole population of trajectories. To calculate its evolution, we apply a smooth function to each state, average over the population, and track how that average changes.

We now choose a smooth test function $\varphi : \mathbb{R}^d \to \mathbb{R}$ with bounded derivatives and compact support, and call $\varphi(X_t)$ a probe of the state. The choice $\varphi(x) = x_i$ measures a coordinate, $\varphi(x) = x_i^2$ measures a squared coordinate, and other choices measure other smooth quantities. Averaging the probe over the population,

$$\mathbb{E}[\varphi(X_t)] = \int \varphi(x)\, p_t(x)\, dx.$$

"The expected probe value $\mathbb{E}[\varphi(X_t)]$ equals the integral of the measurement $\varphi(x)$ weighted by the state density $p_t(x)$, over all positions x."

The left side averages the probe over trajectories. The right side computes the same average by integrating against the density. We can differentiate the probe along one random path using Itô's lemma, then average the result. Moving the derivatives off φ gives an equation for the density p_t. To differentiate along a random path, we first need the squared increments of Brownian motion.

3.8 Quadratic variation

We can divide an elapsed time T into N intervals of width $h = T/N$ and write ΔW_k for the one-dimensional Brownian increment in interval k. Each increment is $\sqrt{h}\,\varepsilon_k$ with ε_k standard Gaussian, so its square is

$$(\Delta W_k)^2 = h\,\varepsilon_k^2.$$

"The squared Brownian increment $(\Delta W_k)^2$ in interval k equals the interval width h times the square of a standard Gaussian draw ε_k. Because ε_k^2 has mean one, the squared increment has the same order as h."

At $h = 0.01$ the typical displacement is $\sqrt{h} = 0.1$, and squaring 0.1 returns 0.01, which is h again. The square of the increment is on the same scale as the time increment, even as we refine the time grid.

The sum of squared increments has mean $\mathbb{E}[\sum_k (\Delta W_k)^2] = Nh = T$. Its fluctuation around this mean vanishes as we refine the grid. A standard Gaussian has $\mathbb{E}[\varepsilon^4] = 3$, so $\mathrm{Var}(\varepsilon^2) = 3 - 1 = 2$, and the N terms are independent, giving

$$\mathrm{Var}\left(\sum_{k=0}^{N-1} (\Delta W_k)^2\right) = Nh^2\,\mathrm{Var}(\varepsilon^2) = 2Nh^2 = 2Th.$$

At $T = 1$ and $h = 0.01$ the sum has mean 1 and variance 0.02, a standard deviation of 0.1414; at $h = 10^{-4}$ the standard deviation falls to 0.01414. The sum concentrates on the elapsed time,

$$\sum_{k=0}^{N-1} (\Delta W_k)^2 \;\longrightarrow\; T \qquad \text{as } N \to \infty.$$

"The sum of squared Brownian increments $(\Delta W_k)^2$ over the N intervals approaches the elapsed time T as N grows without bound. Each squared increment has typical size h, and there are $N = T/h$ intervals."

For comparison, consider a smooth path x_t with bounded velocity. Over an interval of width h, its displacement is of order h, so its squared displacement is of order h^2. Summing $N = T/h$ such squares gives a quantity of order Th. This sum vanishes as the intervals shrink. The second-order term of a Taylor expansion therefore disappears for a smooth path. It remains for a Brownian path, so we must retain it when differentiating a function along that path.

3.9 Itô's lemma

We can take $\varphi(x) = x^2$ as the simplest case in which the difference is visible. Changing X by ΔX changes the probe by

$$\varphi(X + \Delta X) - \varphi(X) = 2X\Delta X + (\Delta X)^2,$$

where the first term comes from the slope and the second records the curvature. For a general smooth φ, expanding to second order,

$$\Delta\varphi \approx \varphi_x\, \Delta X + \tfrac{1}{2}\varphi_{xx}(\Delta X)^2,$$

"The change in the probe $\Delta\varphi$ approximately equals its slope φ_x times the state increment ΔX, plus one half of its curvature φ_{xx} times the squared increment $(\Delta X)^2$."

where $\varphi_x = d\varphi/dx$ and $\varphi_{xx} = d^2\varphi/dx^2$ are evaluated at the current state. Insert the increment of the stochastic differential equation, written with the square-root time scale of Section 3.3,

$$\Delta X = f\Delta t + g\sqrt{\Delta t}\,\varepsilon,$$

and square it:

$$(\Delta X)^2 = f^2(\Delta t)^2 \;+\; 2fg\,(\Delta t)^{3/2}\varepsilon \;+\; g^2\Delta t\,\varepsilon^2.$$

"The squared state increment $(\Delta X)^2$ contains the squared drift contribution $f^2(\Delta t)^2$, the cross term $2fg(\Delta t)^{3/2}\varepsilon$, and the squared noise contribution $g^2\Delta t\varepsilon^2$, where f is the drift, g is the noise scale, and ε is a standard Gaussian draw. Only the noise square remains at first order in the time interval Δt."

Let Δt shrink and keep the terms of order Δt. The first two terms vanish faster than Δt, while ε^2 stays of order one and sums to its mean by the concentration of Section 3.8, leaving

$$(\Delta X)^2 \;\longrightarrow\; g^2\,\Delta t.$$

The second-order Taylor term is therefore as large as the ordinary first-order drift term, and it belongs in the dt part of the differential.

Theorem 3.1 (Itô's lemma)**.** *Let X_t solve $dX_t = f(X_t, t)\,dt + g(t)\,dW_t$ in one dimension, and let $\varphi(x, t)$ have continuous derivatives φ_t, φ_x and φ_{xx}. Then*

$$d\varphi(X_t, t) = \Big(\varphi_t + f\,\varphi_x + \tfrac{1}{2}g^2\varphi_{xx}\Big)\,dt \;+\; g\,\varphi_x\,dW_t,$$

with every derivative of φ and every coefficient evaluated at (X_t, t).

"The probe increment $d\varphi$ equals its time derivative φ_t, plus the drift f times its slope φ_x, plus one half of the squared noise scale g^2 times its curvature φ_{xx}, all multiplied by the time increment dt. We then add the noise scale g times the slope φ_x and the Brownian increment dW_t."

The stochastic chain rule contains an additional term, $\frac{1}{2}g^2\varphi_{xx}$. This term is proportional to the curvature of φ. Averaging a curved function over a symmetric spread can change its value. The term therefore changes the average probe value even though the noise has mean zero. In d dimensions the slope becomes the gradient and the curvature becomes the Laplacian,

$$d\varphi(X_t, t) = \Big(\partial_t\varphi + f \cdot \nabla\varphi + \tfrac{1}{2}g^2\nabla^2\varphi\Big)\,dt \;+\; g\,\nabla\varphi \cdot dW_t,$$

where $\nabla\varphi \in \mathbb{R}^d$ collects the first partial derivatives and $\nabla^2\varphi = \sum_i \partial^2\varphi/\partial x_i^2$ sums the second ones. With a scalar noise scale, the increment has covariance $g^2 hI$. Only the diagonal of the Hessian receives weight, so the full matrix of second derivatives reduces to the Laplacian.

3.9.1 Squaring Brownian motion

We set $X_t = W_t$, which is the stochastic differential equation with $f = 0$ and $g = 1$, and take $\varphi(x) = x^2$, so that $\varphi_t = 0$, $\varphi_x = 2x$ and $\varphi_{xx} = 2$. Substituting into Theorem 3.1, we obtain

$$d\Big(W_t^2\Big) = \Big(0 + 0 + \tfrac{1}{2} \cdot 1 \cdot 2\Big)\,dt + 1 \cdot 2W_t\,dW_t = dt + 2W_t\,dW_t.$$

"The change in the squared Brownian position $d(W_t^2)$ equals the elapsed time increment dt, plus twice the current position W_t times the Brownian increment dW_t."

The extra dt is the curvature correction, and it accounts for the growth of the squared distance even though the increment dW_t has mean zero.

3.9.2 Expected Brownian variance

Now, we can take expectations of both sides. The stochastic integral $\int_0^t 2W_s\,dW_s$ has mean zero, because each increment dW_s is independent of the position W_s that multiplies it and has mean zero. Only the deterministic part remains,

$$\frac{d}{dt}\,\mathbb{E}\Big[W_t^2\Big] = 1, \qquad \mathbb{E}\Big[W_0^2\Big] = 0 \quad\Longrightarrow\quad \mathbb{E}\Big[W_t^2\Big] = t.$$

"The expected squared Brownian position $\mathbb{E}[W_t^2]$ increases at rate one. Starting from $W_0 = 0$, its value therefore equals the elapsed time t."

At $t = 0.25$ the mean squared position is 0.25 and the standard deviation is $\sqrt{0.25} = 0.5$, which agrees with the covariance tI stated in Section 3.2. If we dropped the curvature term from Itô's lemma, we would be left with $d(W_t^2) = 2W_t\,dW_t$, whose expectation is constant, and we would conclude that $\mathbb{E}[W_t^2] = 0$ for all time. All of the variance growth comes from the curvature term, and the same term becomes the diffusion term of the density equation.

3.10 The Fokker-Planck equation

Itô's lemma holds along one path at a time. Averaged over paths, it becomes a statement about the density. Taking the expectation of the d-dimensional form, we drop the stochastic integral, which has mean zero, and read off the rate of change of the averaged probe,

$$\frac{d}{dt}\,\mathbb{E}[\varphi(X_t)] = \mathbb{E}\Big[f(X_t,t)\cdot\nabla\varphi(X_t) + \tfrac{1}{2}g(t)^2\,\nabla^2\varphi(X_t)\Big]\,.$$

"The rate of change of the expected measurement $\mathbb{E}[\varphi(X_t)]$ equals the expectation of the drift $f(X_t,t)$ dotted with the probe gradient $\nabla\varphi(X_t)$, plus one half of the squared noise scale $g(t)^2$ times the probe Laplacian $\nabla^2\varphi(X_t)$."

3.10.1 Derivation: moving the derivatives onto the density

We can write both sides as integrals against p_t. Since the probe does not depend on time, we can move the time derivative on the left under the integral and apply it to the density,

$$\int \varphi(x)\,\partial_t p_t(x)\,dx = \int \Big[\,f(x,t)\cdot\nabla\varphi(x) + \tfrac{1}{2}g(t)^2\,\nabla^2\varphi(x)\Big]\,p_t(x)\,dx.$$

Both terms on the right contain derivatives of φ. We can integrate by parts to transfer these derivatives to p_t.

Apply it once to the drift term. In d dimensions the boundary term vanishes by the divergence theorem, because φ has compact support, leaving

$$\int p_t\, f\cdot\nabla\varphi\,dx = -\int \varphi\,\nabla\cdot(fp_t)\,dx.$$

"The integral of the density p_t times the drift f dotted with the probe gradient $\nabla\varphi$ equals the negative integral of the probe φ times the divergence $\nabla\cdot(fp_t)$ of the probability flux."

Now integrate the Laplacian term by parts twice. The two minus signs cancel, and compact support again removes the boundary terms,

$$\int p_t\,\nabla^2\varphi\,dx = -\int \nabla p_t\cdot\nabla\varphi\,dx = \int \varphi\,\nabla^2 p_t\,dx.$$

"Integrating by parts twice transfers the Laplacian from the probe φ to the density p_t. The two sign changes cancel, so integrating $p_t\nabla^2\varphi$ gives the same result as integrating $\varphi\nabla^2 p_t$."

Substituting both into the identity and collecting everything on one side,

$$\int \varphi(x)\,\Big[\,\partial_t p_t + \nabla\cdot(fp_t) - \tfrac{1}{2}g^2\,\nabla^2 p_t\,\Big]\,dx = 0.$$

This holds for every smooth compactly supported φ. A continuous function whose integral against every such φ vanishes is itself zero, so the bracket vanishes pointwise.

Theorem 3.2 (Fokker-Planck equation)**.** *Let X_t solve $dX_t = f(X_t, t)\,dt + g(t)\,dW_t$ with f and g sufficiently regular, and let p_t be the density of X_t. Then*

$$\boxed{\partial_t p_t(x) = -\nabla \cdot (f(x,t)\,p_t(x)) + \tfrac{1}{2} g(t)^2\,\nabla^2 p_t(x)}$$

for all $x \in \mathbb{R}^d$ and all t in the interval on which the process is defined.

"The rate of change $\partial_t p_t(x)$ of the density at position x equals the negative divergence of the drift flux $f(x,t)p_t(x)$, plus one half of the squared noise scale $g(t)^2$ times the density Laplacian $\nabla^2 p_t(x)$."

3.10.2 Drift and diffusion terms

The first term is the continuity equation of Chapter 2, with the drift as the velocity field. Probability moves along f. The density at a point falls when more flux flows out than in. The second term comes from the surviving squared increment and applies the heat operator to p_t. Its sign depends on the density's curvature. At a peak, the Laplacian is negative and the density falls. In a trough, the Laplacian is positive and the density rises.

Both terms are divergences, since $\nabla^2 p_t = \nabla \cdot \nabla p_t$, so the total probability is conserved,

$$\frac{d}{dt}\int p_t(x)\,dx = \int \left[-\nabla \cdot (f p_t) + \tfrac{1}{2} g^2 \nabla \cdot \nabla p_t\right] dx = 0,$$

because the integral of this divergence over $\mathbb{R}^d$ vanishes for a decaying density. Collecting the two terms into a single flux,

$$J_t(x) = f(x,t)p_t(x) - \tfrac{1}{2} g(t)^2 \nabla p_t(x), \qquad \partial_t p_t = -\nabla \cdot J_t,$$

"The probability flux $J_t(x)$ equals the drift $f(x,t)$ times the density $p_t(x)$, minus one half of the squared noise scale $g(t)^2$ times the density gradient $\nabla p_t(x)$. The density changes at the negative divergence of this flux, so it increases where the flux converges."

The second term in J_t describes diffusion from high to low density. Writing diffusion as a flux also allows us to construct a deterministic velocity field with the same marginals. We call its evolution equation the probability-flow ordinary differential equation.

3.10.3 Diffusion at the peak of a Gaussian

We take a one-dimensional standard Gaussian density at $t = 0$, with no drift and unit noise scale, so $f = 0$ and $g = 1$ and the Fokker-Planck equation reduces to $\partial_t p_t = \frac{1}{2}\partial_x^2 p_t$. Let's evaluate both sides at the peak, $x = 0$. The density there is

$$p_0(0) = \frac{1}{\sqrt{2\pi}} = 0.3989,$$

and differentiating $p_0(x) = (2\pi)^{-1/2} e^{-x^2/2}$ twice produces $p_0''(x) = (x^2 - 1)p_0(x)$, so at the peak

$$p_0''(0) = -p_0(0) = -0.3989.$$

Multiplying the curvature by one half of the squared noise scale,

$$\partial_t p_t(0)\big|_{t=0} = \tfrac{1}{2} \times 1 \times (-0.3989) = -0.1995,$$

so the peak density falls at about 0.1995 per unit time. Over a short interval of 0.1 the first-order estimate is

$$p_{0.1}(0) \approx 0.3989 - 0.1 \times 0.1995 = 0.3790.$$

We can also calculate the exact answer. A driftless diffusion with $g = 1$ started from $N(0, 1)$ stays Gaussian with variance $1 + t$. To check this, substitute $p_t = N(0, 1 + t)$ into $\partial_t p_t = \frac{1}{2}\partial_x^2 p_t$. A Gaussian of variance v satisfies $\partial_v p = \frac{1}{2}\partial_x^2 p$. Since $dv/dt = 1$, both sides match. At $t = 0.1$,

$$p_{0.1}(0) = \frac{1}{\sqrt{2\pi(1.1)}} = 0.3804,$$

so the linear estimate is low by 0.0014, which is the second-order error of the one-step extrapolation. The peak has dropped and the probability has moved outward, with the total still equal to one.

We can now calculate how the density evolves for a given drift and noise scale. For generation, we choose a pair whose conditional density is available in closed form at every time.

3.11 Variance-preserving diffusion

A corruption process useful for training has three properties. Its terminal distribution is a fixed simple prior, independent of the data, so that generation has somewhere to start. Its conditional law at any intermediate time is available in closed form, so that a training example at a chosen noise level costs one Gaussian draw. Its scale also stays bounded, so that a network receives inputs of comparable magnitude at every noise level.

We can choose a positive noising rate $\beta(t)$, in units of inverse time, and set the drift and the noise scale from it,

$$f(x, t) = -\tfrac{1}{2}\beta(t)\, x, \qquad g(t) = \sqrt{\beta(t)},$$

so that the stochastic differential equation reads

$$dX_t = -\tfrac{1}{2}\beta(t)\, X_t\, dt + \sqrt{\beta(t)}\, dW_t.$$

"The state increment dX_t equals negative one half of the noising rate $\beta(t)$ times the current state X_t and time increment dt, plus the square root of $\beta(t)$ times the Brownian increment dW_t."

This is the Ornstein-Uhlenbeck process of Section 3.6, now with a time-dependent restoring rate $\kappa(t) = \beta(t)/2$ and noise scale $g(t) = \sqrt{\beta(t)}$. These choices give the instantaneous stationary variance $g(t)^2/(2\kappa(t)) = \beta(t)/\beta(t) = 1$. A single rate controls signal decay and noise injection. Matching these two contributions holds the variance at one. This is the variance-preserving process [Song et al., 2021]. It is the continuous-time form of the corruption used by DDPM [Ho et al., 2020] and by the earlier diffusion probabilistic model [Sohl-Dickstein et al., 2015].

3.11.1 Stationarity of the standard Gaussian

We substitute $p = N(0, I)$ into Theorem 3.2 with this drift and noise scale. For the standard Gaussian in $\mathbb{R}^d$, $\nabla p = -x\, p$, hence $\nabla^2 p = (\|x\|^2 - d)\, p$ and $\nabla \cdot (xp) = d\, p + x \cdot \nabla p = (d - \|x\|^2)\, p$. The two terms of the equation are

$$-\nabla \cdot \left(-\tfrac{1}{2}\beta x\, p\right) = \tfrac{1}{2}\beta\, (d - \|x\|^2)\, p, \qquad \tfrac{1}{2}\beta\, \nabla^2 p = \tfrac{1}{2}\beta\, (\|x\|^2 - d)\, p,$$

and they cancel, so $\partial_t p = 0$. A state already distributed as a standard Gaussian stays standard Gaussian for all time. The structure of any other distribution is erased as the drift pulls it in and the noise pushes it out, and the distribution approaches the same limit.

3.11.2 The conditional law at each noise level

The same integrating-factor argument as in Section 3.6.1 works with a time-dependent rate. We then accumulate the noising rate and define the signal coefficient from it,

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}.$$

"The accumulated noising $B(t)$ equals the integral of the noising rate $\beta(s)$ from time zero to time t. The signal coefficient a_t equals the exponential of negative one half of $B(t)$."

We now multiply the state by $e^{B(t)/2}$ and differentiate. The factor is deterministic, so the product rule applies with no correction, and $\frac{d}{dt} e^{B(t)/2} = \frac{1}{2}\beta(t) e^{B(t)/2}$:

$$d\Big(e^{B(t)/2} X_t\Big) = \tfrac{1}{2}\beta e^{B/2} X_t\, dt + e^{B/2}\left(-\tfrac{1}{2}\beta X_t\, dt + \sqrt{\beta}\, dW_t\right) = e^{B(t)/2}\sqrt{\beta(t)}\, dW_t.$$

Integrating from the data time to t,

$$e^{B(t)/2} X_t = X_0 + \int_0^t e^{B(s)/2}\sqrt{\beta(s)}\, dW_s.$$

The integral is Gaussian with mean zero, and by the Itô isometry of Section 3.6.1 its variance is the ordinary integral of the squared integrand,

$$\int_0^t e^{B(s)}\beta(s)\, ds = \int_0^t \frac{d}{ds} e^{B(s)}\, ds = e^{B(t)} - 1,$$

because $B'(s) = \beta(s)$. Dividing by $e^{B(t)/2}$ multiplies the variance by $e^{-B(t)} = a_t^2$, leaving $a_t^2(e^{B(t)} - 1) = 1 - a_t^2$. Collecting the mean and the variance,

$$\boxed{X_t \mid X_0 \;\sim\; N\Big(a_t X_0,\; \big(1 - a_t^2\big)\, I\Big), \qquad a_t = e^{-B(t)/2}}$$

We can sample from this conditional distribution by writing

$$X_t = a_t X_0 + \sigma_t\, \varepsilon, \qquad \sigma_t = \sqrt{1 - a_t^2}, \qquad \varepsilon \sim N(0, I).$$

"The corrupted state X_t at noise level t equals the signal coefficient a_t times the clean sample X_0, plus the noise standard deviation σ_t times an independent standard Gaussian draw ε."

- $a_t = e^{-B(t)/2} \in (0, 1]$ is the signal coefficient, the fraction of the clean sample that remains at time t. It is dimensionless and decreases from $a_0 = 1$.
- $\sigma_t = \sqrt{1 - a_t^2}$ is the noise *standard deviation* at time t, and $\sigma_t^2 = 1 - a_t^2$ is the corresponding *variance*. Both are dimensionless and σ_t increases from $\sigma_0 = 0$.

- $\varepsilon \sim N(0, I)$ is the Gaussian draw that produced this particular corrupted state, and it is recorded during training.

The two coefficients satisfy $a_t^2 + \sigma_t^2 = 1$ at every time, which is the identity behind the name of the process. We can write $q_t(x \mid x_0) = N(x; a_t x_0, \sigma_t^2 I)$ for the conditional corruption density and

$$p_t(x) = \int q_t(x \mid x_0)\, p_{\text{data}}(x_0)\, dx_0$$

for the marginal density of corrupted states, which is the density that appears in Theorem 3.2.

In training, a clean sample, a time, and one Gaussian vector produce a corrupted state at any noise level, with no simulation of the intermediate steps.

3.11.3 A check from Itô's lemma

We can check the closed form using Itô's lemma from Section 3.9. This calculation separates the effects of drift and diffusion on the first two moments. Let's apply Itô's lemma with $\varphi(x) = x$ in one dimension. The curvature vanishes, so the dt bracket is the drift alone, and taking expectations removes the stochastic integral,

$$\frac{d}{dt}\,\mathbb{E}[X_t] = -\tfrac{1}{2}\beta(t)\,\mathbb{E}[X_t] \quad \Longrightarrow \quad \mathbb{E}[X_t] = e^{-B(t)/2}\,\mathbb{E}[X_0] = a_t\,\mathbb{E}[X_0],$$

since $B'(t) = \beta(t)$. Next, we take $\varphi(x) = x^2$, for which $\varphi_x = 2x$ and $\varphi_{xx} = 2$. The dt bracket is $2xf + g^2 = -\beta x^2 + \beta$, so the second moment $m_2(t) = \mathbb{E}[X_t^2]$ obeys

$$\frac{d}{dt}\, m_2(t) = -\beta(t)\, m_2(t) + \beta(t).$$

"The rate of change of the second moment $m_2(t)$ equals the negative noising rate $\beta(t)$ times $m_2(t)$, plus $\beta(t)$ from the injected noise. The moment therefore approaches one from either side."

The drift subtracts a fraction of the second moment and the diffusion adds back the same fraction of one, which is the balance that holds the scale fixed. Solving with the integrating factor $e^{B(t)}$,

$$m_2(t) = e^{-B(t)} m_2(0) + \left(1 - e^{-B(t)}\right) = a_t^2\, m_2(0) + \sigma_t^2,$$

which is the second moment of $N(a_t X_0, \sigma_t^2)$ averaged over the data, in agreement with the closed form derived above. Data with $\mathbb{E}[X_0] = 0$ and $m_2(0) = 1$ has $m_2(t) = 1$ at every time.

Let's take the clean sample $x_0 = 10$. A point mass there has $\mathbb{E}[X_0] = 10$ and $m_2(0) = 100$, so at the level $a_t = 0.8$,

$$\mathbb{E}[X_t] = (0.8)(10) = 8, \qquad m_2(t) = (0.64)(100) + 0.36 = 64.36,$$

so the conditional variance is $64.36 - 8^2 = 0.36$, matching σ_t^2 exactly.

3.11.4 Preservation of the covariance

Since X_0 and ε are independent, their covariances add under the scaling,

$$\mathrm{Cov}(X_t) = a_t^2 \ \mathrm{Cov}(X_0) + \left(1 - a_t^2\right) I.$$

"The covariance of the corrupted state X_t equals the squared signal coefficient a_t^2 times the covariance of the clean state X_0, plus the accumulated noise variance $1 - a_t^2$ times the identity matrix I."

Data standardized to identity covariance therefore has $\mathrm{Cov}(X_t) = a_t^2 I + (1 - a_t^2)I = I$ at every noise level, which is where the name of the process comes from. For data with another covariance, the expression is a convex combination of $\mathrm{Cov}(X_0)$ and I, with weight a_t^2. As the signal coefficient decays, the covariance approaches the identity. In both cases, the network receives inputs of comparable magnitude at all noise levels.

3.11.5 The schedule and its noise levels

To calculate the noise levels, let's use a linear schedule on $t \in [0, 1]$,

$$\beta(t) = \beta_{\min} + (\beta_{\max} - \beta_{\min})\, t, \qquad \beta_{\min} = 0.1, \quad \beta_{\max} = 20,$$

so that $\beta(t) = 0.1 + 19.9\, t$, and integrating from zero,

$$B(t) = \beta_{\min} t + \tfrac{1}{2}\,(\beta_{\max} - \beta_{\min})\, t^2 = 0.1\, t + 9.95\, t^2, \qquad a_t = e^{-(0.1t + 9.95t^2)/2}.$$

These constants correspond to 1000 discrete steps with per-step variances from 10^{-4} to 0.02. Dividing each endpoint by the step size $h = 1/1000$ gives 0.1 and 20.

If we invert a_t, we can locate a chosen noise level. To set $a_t = 0.8$ we need $B(t) = -2 \log 0.8 = 0.4463$, so $9.95\, t^2 + 0.1\, t - 0.4463 = 0$ and the positive root is

$$t = \frac{-0.1 + \sqrt{0.01 + 4(9.95)(0.4463)}}{2(9.95)} = \frac{-0.1 + 4.2157}{19.9} = 0.2068.$$

At that time $\sigma_t = \sqrt{1 - 0.64} = 0.6$ exactly. Three further levels follow the same calculation and are collected in Table 3.1. The two curves cross at $a_t = \sigma_t = 1/\sqrt{2} = 0.7071$, which the same inversion places at $t = 0.2589$, and the conditional cloud widens along the whole schedule (Figure 3.2).

Table 3.1: Four noise levels of the linear variance-preserving schedule $\beta(t) = 0.1 + 19.9t$, with $B(t) = 0.1t + 9.95t^2$. Each row fixes a signal coefficient a_t, solves $B(t) = -2 \log a_t$ for the time, and reports the noise standard deviation $\sigma_t = \sqrt{1 - a_t^2}$ and the noising rate there. The last row is the terminal time of the schedule.

t	a_t	σ_t	$\beta(t)$
0.0589	0.980	0.199	1.272
0.2068	0.800	0.600	4.216
0.4544	0.350	0.937	9.142
0.7994	0.040	0.999	16.007
1.0000	0.0066	1.000	20.000

At the terminal time, $B(1) = 0.1 + 9.95 = 10.05$ and $a_1 = e^{-5.025} = 0.0066$, so the surviving signal is below one percent and $\sigma_1 = 0.99998$. The terminal law is a standard Gaussian to four decimal places for any clean sample, so generation can start from pure noise with no knowledge of the data.

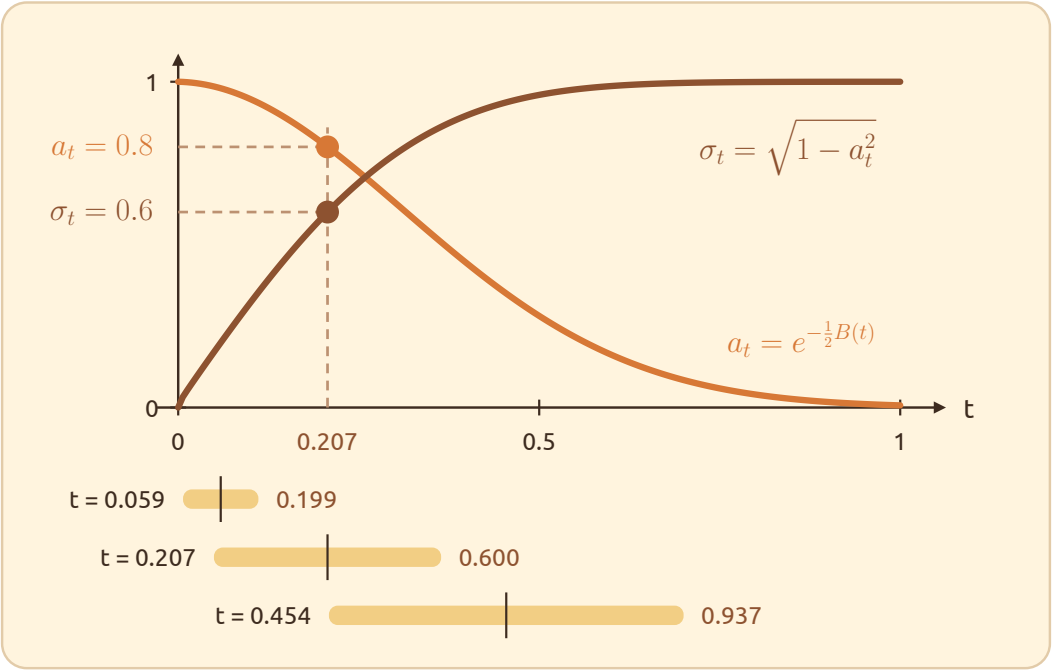


Figure 3.2: The variance-preserving schedule $\beta(t) = 0.1 + 19.9t$ on $0 \le t \le 1$. The falling curve is the signal coefficient $a_t = e^{-B(t)/2}$ and the rising curve is the noise standard deviation $\sigma_t = \sqrt{1 - a_t^2}$; both are dimensionless and run between 0 and 1. The dashed lines mark $t = 0.207$, where $a_t = 0.8$ and $\sigma_t = 0.6$. Below the axis, three horizontal bars stand for the conditional cloud at $t = 0.059$, 0.207 and 0.454; each bar is drawn at its own time with width proportional to σ_t there, the vertical tick marks the center of the cloud, and the number at the right of each bar is the value of σ_t, namely 0.199, 0.600 and 0.937.

3.11.6 Corruption example

We can fix a one-dimensional clean sample $x_0 = 10$ and corrupt it at the level $a_t = 0.8$, $\sigma_t = 0.6$ located above, using the Gaussian draw $\varepsilon = -1$. The corrupted state is

$$x_t = a_t x_0 + \sigma_t \varepsilon = (0.8)(10) + (0.6)(-1) = 8 - 0.6 = 7.4,$$

and its conditional corruption density is

$$q_t(\cdot \mid x_0) = N(8,\ 0.36)\,,$$

with mean $a_t x_0 = 8$, variance $\sigma_t^2 = 0.36$ and standard deviation $\sigma_t = 0.6$. The draw lands 0.6 below its conditional mean, which is exactly one standard deviation of the corruption at this level. That displacement,

$$x_t - a_t x_0 = \sigma_t \varepsilon = -0.6,$$

is the displacement we aim to estimate through denoising.

3.12 The forward corruption in code

We store the schedule on a grid of K noise levels and retrieve the two coefficients for each training example's level. The step index k runs from 1 to K, and step k has noise variance β_k. We recover

the continuous time of Section 3.11 through $t_k = k/K$. The per-step quantities are

$$\alpha_k = 1 - \beta_k, \qquad \bar{\alpha}_k = \prod_{j=1}^{k} \alpha_j.$$

"The one-step signal retention α_k equals one minus the noise variance β_k added at step k. The cumulative retention $\bar{\alpha}_k$ equals the product of the retentions α_j from step $j = 1$ through step k."

The corruption coefficients are the square roots of these,

$$a_k = \sqrt{\bar{\alpha}_k}, \qquad \sigma_k = \sqrt{1 - \bar{\alpha}_k},$$

and the Gaussian transition from the clean state to step k is

$$q(x_k \mid x_0) = N\Big(x_k;\ a_k x_0,\ \sigma_k^2 I\Big), \qquad x_k = a_k x_0 + \sigma_k \varepsilon.$$

"Given the clean state x_0, the state x_k is Gaussian with mean $a_k x_0$, the signal coefficient times the clean state, and covariance $\sigma_k^2 I$, the noise variance times the identity matrix. We sample it by adding $a_k x_0$ to the noise standard deviation σ_k times a standard Gaussian tensor ε."

3.12.1 Discrete and continuous schedules

The two descriptions agree when β_k is the continuous rate multiplied by the step size. Setting $\beta_k = \beta(t_k)h$ with $h = 1/K$ and taking logarithms of the product,

$$\log \bar{\alpha}_k = \sum_{j=1}^{k} \log(1 - \beta_j) \approx -\sum_{j=1}^{k} \beta(t_j)\, h \approx -B(t_k),$$

using $\log(1 - \beta) \approx -\beta$ for small β and recognizing the Riemann sum of β. Hence $\bar{\alpha}_k \approx e^{-B(t_k)} = a_{t_k}^2$, so $a_k \approx a_{t_k}$ and the discrete coefficients track the continuous curve.

With $K = 1000$ levels and β_k spaced linearly from 10^{-4} to 0.02, the cumulative retention at step 207 is $\bar{\alpha}_{207} = 0.6402$, hence

$$a_{207} = \sqrt{0.6402} = 0.8001, \qquad \sigma_{207} = \sqrt{1 - 0.6402} = 0.5999,$$

against the continuous values $a_t = 0.7997$ and $\sigma_t = 0.6004$ at $t = 0.207$. The worked example of Section 3.11.6 therefore corresponds to step 207 of the thousand-step schedule. At the last step, $\bar{\alpha}_{1000} = 4.036 \times 10^{-5}$ and $a_{1000} = 0.0064$, against 0.0066 from the continuous schedule.

3.12.2 The implementation

We can implement the schedule in PyTorch [Paszke et al., 2019]. The `ForwardDiffusion` class stores it once and draws a corrupted batch at any requested level. Image batches have shape (batch, channels, height, width).

```
class ForwardDiffusion:
    """Gaussian corruption of a clean batch at any of K stored levels."""

    def __init__(self, num_steps=1000, beta_start=1e-4, beta_end=0.02):
```

```
        self.num_steps = num_steps
        self.betas = torch.cat([
            torch.zeros(1),  # Placeholder so that index k holds beta_k.
            torch.linspace(beta_start, beta_end, num_steps),
        ])
        self.alphas = 1.0 - self.betas  # One-step retention alpha_k.
        self.alphas_cumprod = torch.cumprod(self.alphas, dim=0)
        self.a = torch.sqrt(self.alphas_cumprod)  # Signal a_k.
        self.sigma = torch.sqrt(1.0 - self.alphas_cumprod)  # Noise sigma_k.

    def q_sample(self, x_0, k):
        """Corrupt the clean batch x_0 at the per-item noise indices k."""
        noise = torch.randn_like(x_0)  # One epsilon per batch item.
        a_k = self.a[k].view(-1, 1, 1, 1)  # Broadcast over C, H, W.
        sigma_k = self.sigma[k].view(-1, 1, 1, 1)
        x_k = a_k * x_0 + sigma_k * noise
        return x_k, noise  # The noisy batch and the exact noise.
```

The constructor stores K noise levels, named `num_steps` in the code. We prefix a zero to the linear schedule so that the Python index equals the step index. The entry `betas[k]` holds β_k. Entry 0 is a placeholder for the clean state before any steps. Because $\beta_0 = 0$, the cumulative product starts at $\bar{\alpha}_0 = 1$, which is the right value for the clean state. The call to `torch.linspace` builds the K per-step variances between `beta_start` and `beta_end`, each one controlling the fresh noise injected at a single step.

We then precompute the sampling coefficients. Subtracting each variance from one gives the one-step retention α_k. The call to `torch.cumprod` multiplies these retentions along the step axis to obtain $\bar{\alpha}_k$. We store its square root as the signal coefficient $a_k = \sqrt{\bar{\alpha}_k}$. The other square root gives the noise standard deviation $\sigma_k = \sqrt{1 - \bar{\alpha}_k}$. We build each tensor once, with length $K + 1$. Corrupting a batch then costs two table lookups and one Gaussian draw.

The method `q_sample` applies the Gaussian transition at the requested level. `torch.randn_like` draws a tensor with the same shape, dtype and device as `x_0`, each coordinate an independent standard Gaussian with mean zero and variance one. The index tensor `k` holds one noise index per batch item. The lookup `self.a[k]` gathers the corresponding coefficient for each item. The call to `.view(-1, 1, 1, 1)` appends three singleton dimensions. Each item's scalar then broadcasts across its own channels, rows and columns. The last two lines form $x_k = a_k x_0 + \sigma_k \varepsilon$ and return the pair (x_k, ε), the corrupted batch together with the exact noise that produced it. We use that noise as the denoising regression target.

3.12.3 Single-pixel example

We can take a single normalized pixel at $x_0 = 0.5$, request the noise index $k = 207$, and suppose the Gaussian draw at that coordinate is $\varepsilon = -1$. The stored coefficients are $a_{207} = 0.8001$ and $\sigma_{207} = 0.5999$ from Section 3.12.1, so the corrupted pixel is

$$x_{207} = (0.8001)(0.5) + (0.5999)(-1) = 0.4000 - 0.5999 = -0.1998.$$

The pixel has crossed zero, since a corruption of six tenths applied to a surviving signal of four tenths changes the sign of the value. At this level, the noise standard deviation already exceeds the surviving signal for any pixel with $|x_0| < 0.750$. By step 500, the signal coefficient has fallen to $a_k = 0.2803$. The noise then exceeds the signal for nearly every pixel of a normalized image.

A denoiser therefore has less and less systematic signal to work with as the index grows, and the denoising problem becomes monotonically harder along the schedule.

3.13 Forward corruption and the marginal density

A stochastic differential equation describes one corrupted trajectory through its drift and Brownian increments. We simulate that trajectory on a grid with Euler-Maruyama steps. When differentiating along the trajectory, we use Itô's lemma to retain the squared increment that vanishes in ordinary calculus. Averaging over trajectories gives the Fokker-Planck equation of Theorem 3.2, which describes the evolution of the whole density. Our choice of the variance-preserving pair $f = -\frac{1}{2}\beta(t)x$ and $g = \sqrt{\beta(t)}$ gives that density in closed form. We can corrupt a sample to any noise level in one draw, $X_t = a_t X_0 + \sigma_t \varepsilon$. The conditional density is $q_t(x \mid x_0) = N(x; a_t x_0, \sigma_t^2 I)$. Averaging this density over the data gives the marginal p_t.

To reverse this motion, we need a direction at every state. We can obtain a local direction from the score $\nabla_x \log p_t(x)$, the gradient of the log density.

3.14 The score of a density

The variance-preserving corruption has a closed form at every noise level. To reverse it at a particular state, we need a local direction determined by the corrupted density. We can calculate that direction from the gradient of the log density.

We now fix a noise level t and write $p_t(x)$ for the density of the noisy state X_t evaluated at a point $x \in \mathbb{R}^d$. The quantity we need is the gradient of its logarithm,

$$s_t(x) = \nabla_x \log p_t(x).$$

"The score $s_t(x)$ equals the gradient of the log density $\log p_t(x)$ with respect to position x at fixed noise level t. This vector in $\mathbb{R}^d$ points in the direction of steepest log-density increase, and its length gives the rate of that increase."

- $p_t(x)$ is the density of the noisy state at noise level t, evaluated at $x \in \mathbb{R}^d$. It is a mixture over all clean states the corruption could have started from.
- ∇_x is the gradient in the state variable, a column of d partial derivatives $\partial/\partial x_1, \ldots, \partial/\partial x_d$, taken at fixed t.
- $s_t(x) \in \mathbb{R}^d$ is the score, in units of inverse length. One score field exists for each noise level, so s is a time-indexed family of vector fields over $\mathbb{R}^d$.

A small step taken along $s_t(x)$ raises $\log p_t$, and therefore raises p_t, faster than a step of the same length in any other direction. At a local maximum of the density the gradient vanishes and the score is the zero vector, and the same holds at any other stationary point. The score at x depends only on how p_t behaves in a neighborhood of x. Multiplying the density by a constant leaves its score unchanged, because the logarithm converts that constant into an additive term with zero gradient. The score is defined wherever the density is positive and differentiable, even if we cannot evaluate p_t itself.

3.14.1 The Gaussian score

Our conditional corruption densities are Gaussian, so let's calculate a Gaussian score explicitly. Take a Gaussian in one coordinate with mean $\mu \in \mathbb{R}$ and variance $v > 0$,

$$p(x) = \frac{1}{\sqrt{2\pi v}} \exp\left(-\frac{(x-\mu)^2}{2v}\right), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v},$$

where $C = -\frac{1}{2}\log(2\pi v)$ collects every factor with no dependence on x.

"The Gaussian log density $\log p(x)$ equals a constant C, minus the squared displacement $(x-\mu)^2$ from the mean μ, divided by twice the variance v. It is a downward parabola with curvature set by the inverse variance $1/v$."

Differentiating the log density with respect to x, the constant drops out and one linear term remains,

$$s(x) = \frac{d}{dx}\log p(x) = -\frac{x-\mu}{v}.$$

"The Gaussian score $s(x)$ equals the negative displacement $x - \mu$ from the mean μ, divided by the variance v. It points toward the mean, with magnitude proportional to distance and inversely proportional to variance."

Take $\mu = 2$ and $v = 4$, and evaluate at $x = 5$. The displacement from the mean is $5 - 2 = 3$, and the score is

$$s(5) = -\frac{5-2}{4} = -\frac{3}{4} = -0.75.$$

The sign is negative, so the direction of increasing density at $x = 5$ points left, back toward $\mu = 2$. If we widen the Gaussian while keeping the displacement at 3, the score weakens. With $v = 9$, it becomes $-3/9 \approx -0.333$. A wider density changes more slowly over the same distance.

Dropping the constant, $\log p(5) = -(5-2)^2/8 = -1.125$. Stepping 0.1 to the left, along the sign of the score, moves the log density to $\log p(4.9) = -(2.9)^2/8 = -1.0513$, a rise of 0.0737. Stepping 0.1 to the right lowers it to -1.2013, a fall of 0.0763. The first-order prediction from the score is $s(5) \times (-0.1) = (-0.75)(-0.1) = 0.075$. This falls between the two measured magnitudes. The small gaps on either side result from the parabola's curvature.

The standard Gaussian is the special case $\mu = 0$ and $v = 1$, where the score reduces to $s(x) = -x$. Its arrows point inward from both sides and grow in length with distance from the origin. The values $s(-1) = 1$ and $s(2) = -2$ have opposite signs because these points lie on opposite sides of the mean (Figure 3.3).

3.14.2 Cancellation of the normalizing constant

We can evaluate the score of a density even when we know the density only up to a constant. Let $E_\theta(x)$ be an energy function with parameters θ, and define the model density by exponentiating the negative energy and normalizing the result to have total mass one,

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}, \qquad Z_\theta = \int_{\mathbb{R}^d} e^{-E_\theta(y)}\, dy.$$

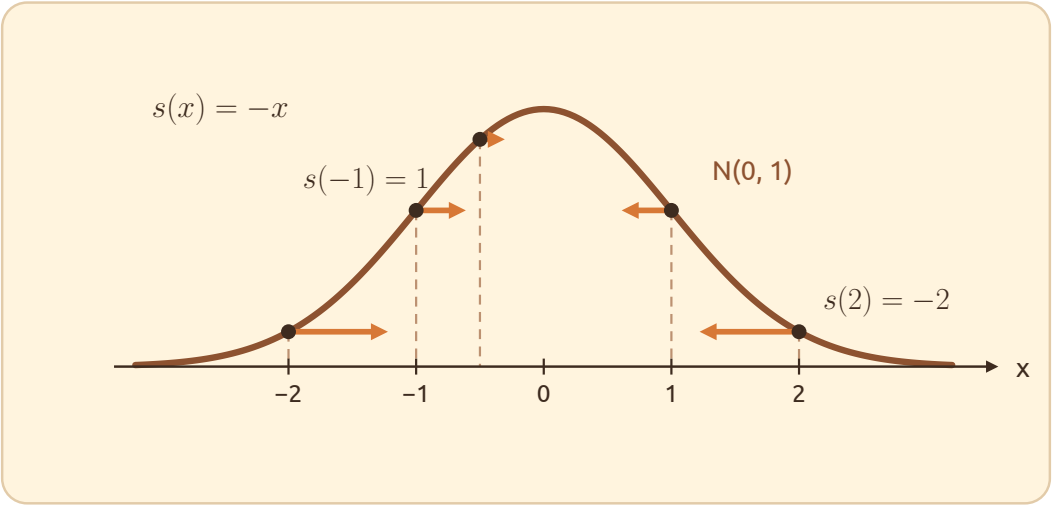


Figure 3.3: The standard Gaussian density $\mathcal{N}(0, 1)$ with its score $s(x) = -x$ drawn as a horizontal arrow at five positions along the axis. Each arrow starts at the point of the curve above that position, and its length is proportional to $|s(x)| = |x|$. Left of the mean the arrows point right and right of the mean they point left, so every arrow points toward $x = 0$, where the score vanishes. Two values are labeled, $s(-1) = 1$ and $s(2) = -2$.

"The model density $p_\theta(x)$ equals the exponential of the negative energy $E_\theta(x)$, divided by the normalizing constant Z_θ. We obtain Z_θ by integrating the exponential over the state space; it depends on the model parameters θ and is constant in the state x."

Computing the normalizing integral Z_θ is usually intractable in d dimensions. To calculate the score, we first take the logarithm of the density. We then differentiate in x, removing the constant term,

$$\nabla_x \log p_\theta(x) = \nabla_x \big[- E_\theta(x) - \log Z_\theta \big] = -\nabla_x E_\theta(x).$$

"The model score $\nabla_x \log p_\theta(x)$ equals the negative gradient of the energy $E_\theta(x)$. The log normalizer $\log Z_\theta$ is constant with respect to the state x, so its gradient is zero."

With $E_\theta(x) = x^2/(2v)$ in one coordinate, the normalizing constant is $Z_\theta = \sqrt{2\pi v}$ and the score is $-x/v$, which is the Gaussian score of Section 3.14.1 with mean zero. The value $\sqrt{2\pi v}$ never appeared in the calculation. This cancellation motivated score matching as a way to fit models whose normalizing constants are out of reach [Hyvärinen, 2005].

We can therefore represent the score directly without evaluating a normalizing integral. To train this model, we need a loss that we can evaluate using a finite sample from the true density.

3.15 Score matching

We can write p for the density we want to model and $s_\theta(x) \in \mathbb{R}^d$ for a network that predicts its score at the point x, with parameters θ. The natural criterion is the squared distance between the prediction and the true score, averaged over the data,

$$J(\theta) = \frac{1}{2} \mathbb{E}_{x \sim p} \Big[\big\| s_\theta(x) - \nabla_x \log p(x) \big\|^2 \Big].$$

"The score-matching loss $J(\theta)$ averages one half of the squared distance between the predicted score $s_\theta(x)$ and the data score $\nabla_x \log p(x)$, over samples x drawn from the data density p."

- p is the data density. We never evaluate it; we only draw samples from it.
- $s_\theta(x)$ is the model score, a vector field $\mathbb{R}^d \to \mathbb{R}^d$ with parameters θ.
- $\nabla_x \log p(x)$ is the target, the score of the data density at the sampled point.

We have data samples for the expectation, but we do not have the data score needed as its target. Since we cannot estimate this score pointwise from samples, we cannot compute the loss directly. We can integrate by parts to remove the unknown target and express the loss through the model alone.

3.15.1 Derivation: removing the data score

If we expand the square, the loss separates into three expectations,

$$J(\theta) = \mathbb{E}_p\Big[\tfrac{1}{2}\big\|s_\theta(x)\big\|^2\Big] - \mathbb{E}_p\Big[s_\theta(x) \cdot \nabla_x \log p(x)\Big] + \mathbb{E}_p\Big[\tfrac{1}{2}\big\|\nabla_x \log p(x)\big\|^2\Big].$$

The third expectation depends only on the data. We denote it by C and omit it during optimization because an additive constant does not change the minimizer.

The middle expectation contains the unknown target. Let's write it as an integral against p and use the identity $p\,\nabla_x \log p = \nabla_x p$ to cancel the density,

$$\mathbb{E}_p\big[s_\theta(x) \cdot \nabla_x \log p(x)\big] = \int p(x)\, s_\theta(x) \cdot \frac{\nabla_x p(x)}{p(x)}\, dx = \int s_\theta(x) \cdot \nabla_x p(x)\, dx.$$

Write $s_{\theta,i}$ for the model's i-th coordinate and ∂_i for the partial derivative in x_i. We can now integrate the i-th coordinate of the dot product by parts in x_i,

$$\int s_{\theta,i}(x)\, \partial_i p(x)\, dx = \Big[\, s_{\theta,i}(x)\, p(x) \,\Big]_{x_i=-\infty}^{x_i=+\infty} - \int p(x)\, \partial_i s_{\theta,i}(x)\, dx.$$

We assume that p is differentiable and that $s_{\theta,i}(x)\, p(x) \to 0$ as $|x_i| \to \infty$, so the boundary term vanishes. This condition holds for a density with exponential or Gaussian tails together with a model whose growth is at most polynomial. If we sum the remaining integrals over the d coordinates, the partial derivatives collect into the divergence of the model,

$$\mathbb{E}_p\big[s_\theta(x) \cdot \nabla_x \log p(x)\big] = -\,\mathbb{E}_p\big[\nabla_x \cdot s_\theta(x)\big].$$

Substituting this back, the minus sign in front of the middle term flips, and the unknown target is gone from the expression,

$$J(\theta) = \mathbb{E}_{x\sim p}\Big[\, \tfrac{1}{2}\big\|s_\theta(x)\big\|^2 + \nabla_x \cdot s_\theta(x) \,\Big] + C.$$

"The score-matching loss $J(\theta)$ averages one half of the squared magnitude of the predicted score $s_\theta(x)$, plus its divergence $\nabla_x \cdot s_\theta(x)$, over data samples x. The remaining constant C does not depend on the parameters θ."

Both terms on the right are computable from a data sample and from derivatives of the model. The first term grows with the magnitude of the predicted score. The second falls as the predicted field contracts, as a true score field does around a mode. The balance between the two terms determines the minimizer.

The two forms differ by the constant C, so they have the same minimizer. In the original form, the squared distance is zero exactly when $s_\theta(x) = \nabla_x \log p(x)$. A model flexible enough to represent the data score therefore attains its minimum at that function. No other function on the support of p attains the minimum. We can thus fit the data score without evaluating it. The equivalent loss requires one derivative of the model with respect to its input.

3.15.2 Fitting a Gaussian score with one parameter

Let's carry the identity through on a model with a single parameter, where we can evaluate every expectation by hand. We take the data to be standard Gaussian in one coordinate, so that $\mathbb{E}[x^2] = 1$, and the model to be a linear inward field

$$s_a(x) = -a\,x,$$

where the coefficient $a \in \mathbb{R}$ controls how strongly the predicted score pulls toward the origin. The squared magnitude is a^2x^2, and the divergence in one coordinate is the ordinary derivative $ds_a/dx = -a$. Substituting both and dropping the constant C,

$$\tilde{J}(a) = \mathbb{E}\Big[\tfrac{1}{2}a^2x^2 - a\Big] = \tfrac{1}{2}a^2\,\mathbb{E}[x^2] - a = \tfrac{1}{2}a^2 - a,$$

where the last step used $\mathbb{E}[x^2] = 1$ for standard Gaussian data.

Next, we can evaluate this at two candidate values. At $a = 0.5$ the loss is

$$\tilde{J}(0.5) = \tfrac{1}{2}(0.5)^2 - 0.5 = 0.125 - 0.5 = -0.375,$$

and at $a = 1$ it is

$$\tilde{J}(1) = \tfrac{1}{2}(1)^2 - 1 = 0.5 - 1 = -0.5,$$

which is the lower of the two. If we differentiate and set the derivative to zero, we locate the minimum exactly,

$$\frac{d\tilde{J}}{da} = a - 1 = 0 \quad \Longrightarrow \quad a = 1,$$

and the second derivative is $1 > 0$, which confirms a minimum. At that value the fitted field is $s_1(x) = -x$, the score of the standard Gaussian computed directly in Section 3.14.1. We fitted it using the sample average of x^2 and the derivative of the model with respect to its input, without evaluating the true score.

Nothing in the calculation required the data variance to be one. For centered data with variance v, the only change is $\mathbb{E}[x^2] = v$. Substituting gives the loss $\tilde{J}(a) = \frac{1}{2}a^2v - a$. Its derivative is $av - 1$, which vanishes at the minimizer $a = 1/v$. At $v = 4$ that is $a = 0.25$, with loss value

$$\tilde{J}(0.25) = \tfrac{1}{2}(0.25)^2 \times 4 - 0.25 = 0.125 - 0.25 = -0.125,$$

and the fitted field $s(x) = -0.25\,x = -x/v$ reproduces the Gaussian score of Section 3.14.1 with mean zero. The coefficient we obtain is the inverse variance, which is the one quantity about the data that the score of a centered Gaussian encodes.

3.15.3 Random directions for the divergence

To compute the divergence $\nabla_x \cdot s_\theta(x)$ in d dimensions, we sum the d diagonal entries of the Jacobian $\nabla_x s_\theta(x)$. Extracting these entries from a network costs d separate backward passes. A 64-dimensional descriptor needs 64 passes per training sample. A colour image on a 32×32 grid has $3 \times 32 \times 32 = 3072$ coordinates and needs 3072 passes. A sequence of 50 residue embeddings of width 1280 has 64,000 coordinates and needs 64,000 passes. We can estimate this sum without bias along random directions. To do so, draw a random vector $v \in \mathbb{R}^d$ with $\mathbb{E}[vv^\top] = I$, independently of the data, and average

$$J_{\mathrm{SSM}}(\theta) = \mathbb{E}_{x,\,v}\Big[\tfrac{1}{2}\big(v^\top s_\theta(x)\big)^2 + v^\top(\nabla_x s_\theta(x))v\Big].$$

"The sliced score-matching loss $J_{\mathrm{SSM}}(\theta)$ averages one half of the squared projection of the predicted score $s_\theta(x)$ onto a random direction v, plus the directional derivative $v^\top(\nabla_x s_\theta(x))v$ along that direction, over data samples x and random directions v."

Both terms recover their exact counterparts in expectation. For any fixed matrix A, the identity $\mathbb{E}[v^\top A v] = \mathrm{tr}(A\,\mathbb{E}[vv^\top]) = \mathrm{tr}(A)$ applies to the Jacobian and returns the divergence, and the same identity applied to $A = s_\theta s_\theta^\top$ returns $\|s_\theta\|^2$. The saving is computational, since the product $(\nabla_x s_\theta)\,v$ is one Jacobian-vector product, available in a single pass, and the full Jacobian is never formed.

Both losses require model derivatives at every data point and target the score of the data density. Gaussian corruption gives us another target. When we generate a noisy sample from a known clean state and noise draw, we can calculate the conditional corruption score in closed form without differentiating the model.

3.16 Denoising score matching

We can fix a clean sample x_0 and a noise level t. Under variance-preserving corruption, the noisy state is Gaussian, centered on a shrunken copy of x_0,

$$q_t(x \mid x_0) = \mathcal{N}(x;\, a_t x_0,\ \sigma_t^2 I).$$

"The corruption density $q_t(x \mid x_0)$ is Gaussian in the noisy state x, with mean $a_t x_0$, the signal coefficient times the clean state, and covariance $\sigma_t^2 I$, the noise variance times the identity matrix. Each coordinate receives independent Gaussian noise with variance σ_t^2."

- $a_t \in (0, 1]$ is the surviving signal coefficient at noise level t, a scalar that shrinks the clean state toward the origin.
- $\sigma_t > 0$ is the noise standard deviation at the same level, so σ_t^2 is the variance added to each coordinate. The variance-preserving schedule relates them through $\sigma_t^2 = 1 - a_t^2$.
- $\varepsilon \sim \mathcal{N}(0, I)$ is the standard Gaussian draw that realizes the corruption, through $X_t = a_t X_0 + \sigma_t \varepsilon$.

This density is the Gaussian of Section 3.14.1 in d coordinates, with mean $a_t x_0$ and variance σ_t^2 in each, so its logarithm is a constant minus a squared distance,

$$\log q_t(x \mid x_0) = C_t - \frac{\|x - a_t x_0\|^2}{2\sigma_t^2}, \qquad C_t = -\frac{d}{2}\log\left(2\pi\sigma_t^2\right).$$

The constant C_t depends on the noise level and on the dimension, and it has no dependence on x, so it vanishes under the gradient. If we differentiate the squared distance in x, a factor of two comes down and cancels the two in the denominator,

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t},$$

where the second equality substitutes the displacement $x - a_t x_0 = \sigma_t \varepsilon$ from the corruption equation and cancels one factor of σ_t.

"The conditional score $\nabla_x \log q_t(x \mid x_0)$ equals the negative displacement $x - a_t x_0$ from the conditional mean $a_t x_0$, divided by the noise variance σ_t^2. Equivalently, it is the negative sampled noise ε divided by the noise standard deviation σ_t."

During training we draw x_0 and ε ourselves, so both expressions are known numbers for every synthetic pair we create. The second form is the cheaper one, because we already have the noise tensor.

3.16.1 The conditional score of one noisy sample

Let's use the clean sample $x_0 = 10$ at a noise level with signal coefficient $a_t = 0.8$ and noise standard deviation $\sigma_t = 0.6$. Take the Gaussian draw $\varepsilon = -1$. From the corruption equation, the noisy state is

$$x_t = 0.8 \times 10 + 0.6 \times (-1) = 8 - 0.6 = 7.4,$$

and the corruption density at this level is Gaussian with conditional mean $a_t x_0 = 8$ and variance $\sigma_t^2 = 0.36$. The draw is 0.6 below its conditional mean.

Computing the conditional score from the displacement, we negate that -0.6 and divide by the variance,

$$\nabla_x \log q_t(7.4 \mid 10) = -\frac{7.4 - 8}{0.36} = \frac{0.6}{0.36} = \frac{5}{3} \approx 1.667.$$

Computing it from the sampled noise instead, we negate $\varepsilon = -1$ and divide by the standard deviation,

$$-\frac{\varepsilon}{\sigma_t} = -\frac{-1}{0.6} = \frac{1}{0.6} = \frac{5}{3} \approx 1.667.$$

The two calculations agree because the displacement is $\sigma_t \varepsilon$ by construction. The sign is positive, so the conditional score at 7.4 points to the right, back toward the conditional mean 8. Its magnitude $5/3$ is large because the conditional variance 0.36 is small, and a narrow Gaussian falls off quickly.

3.16.2 The marginal score as a posterior average

A sampler starting from noise has no clean state to condition on, so the conditional score is not available to it. The density it must reverse is the marginal over all clean states,

$$p_t(x) = \int q_t(x \mid x_0)\, p_0(x_0)\, dx_0,$$

where $p_0 = p_{\text{data}}$ is the clean data density. Many clean states can produce the same noisy point, and the marginal at that point averages over all of them.

First, differentiate under the integral sign. Then multiply and divide each term by its corruption density. This converts the gradient into a log gradient,

$$\nabla_x p_t(x) = \int \nabla_x q_t(x \mid x_0)\, p_0(x_0)\, dx_0 = \int q_t(x \mid x_0)\, \nabla_x \log q_t(x \mid x_0)\, p_0(x_0)\, dx_0.$$

Next, divide both sides by $p_t(x)$. The left side becomes the marginal score. On the right, Bayes' rule identifies $q_t(x \mid x_0)\, p_0(x_0)/p_t(x)$ as the posterior density of the clean state given the observation,

$$p(x_0 \mid X_t = x) = \frac{q_t(x \mid x_0)\, p_0(x_0)}{p_t(x)}.$$

We are left with an average of conditional scores against that posterior,

$$\boxed{s_t(x) = \mathbb{E}\Big[\, \nabla_x \log q_t(x \mid X_0) \;\Big|\; X_t = x \,\Big]}$$

"The marginal score $s_t(x)$ averages the conditional score $\nabla_x \log q_t(x \mid X_0)$ over clean states X_0, using their posterior probabilities after observing the noisy state $X_t = x$."

Each candidate clean state has its own direction, pointing toward its own corruption mean, and the posterior weights fix how much of each direction remains in the average. A candidate that explains the observation well is close to it in units of σ_t and has a large weight; a candidate far away has almost none.

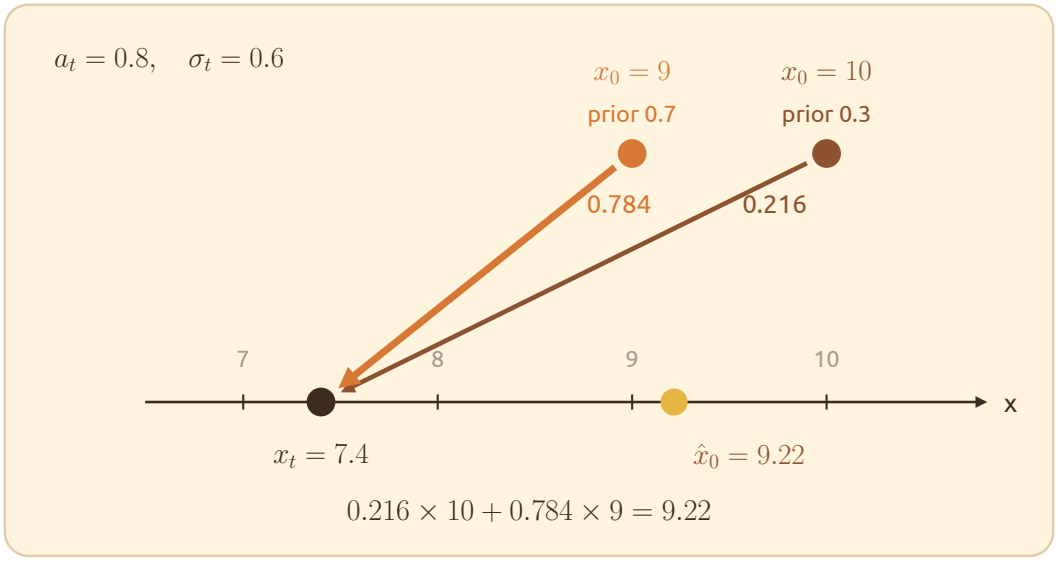


Figure 3.4: One noisy observation $x_t = 7.4$ on the horizontal axis, with the two clean states that could have produced it at the schedule $a_t = 0.8$, $\sigma_t = 0.6$. The state $x_0 = 10$ carries prior weight 0.3 and the state $x_0 = 9$ carries prior weight 0.7; the arrows are labeled with the posterior weights 0.216 and 0.784 these states receive after the observation, rounded from 0.2156 and 0.7844. The weighted average of the two clean states, $0.2156 \times 10 + 0.7844 \times 9 = 9.2156$, is the clean estimate $\hat{x}_0$ marked on the axis between 9 and 10.

Let the data take the value 10 with prior probability 0.3 and the value 9 with prior probability 0.7 (Figure 3.4). Keep the schedule at $a_t = 0.8$ and $\sigma_t = 0.6$. Suppose we observe $x_t = 7.4$. The two conditional means are $0.8 \times 10 = 8$ and $0.8 \times 9 = 7.2$. For each candidate, evaluate the Gaussian density at 7.4 around its conditional mean. The shared factor $1/\sqrt{2\pi \times 0.36}$ cancels in the ratio, leaving the exponentials,

$$q_t(7.4 \mid 10) \propto \exp\left(-\frac{(7.4-8)^2}{2 \times 0.36}\right) = e^{-0.5} = 0.6065,$$

$$q_t(7.4 \mid 9) \propto \exp\left(-\frac{(7.4-7.2)^2}{2 \times 0.36}\right) = e^{-0.0556} = 0.9460.$$

Multiplying each by its prior and normalizing by the sum $0.3{\times}0.6065{+}0.7{\times}0.9460 = 0.1820{+}0.6622 = 0.8441$,

$$p(10 \mid x_t = 7.4) = \frac{0.1820}{0.8441} = 0.2156, \qquad p(9 \mid x_t = 7.4) = \frac{0.6622}{0.8441} = 0.7844.$$

The candidate 9 receives the larger share because its corruption mean 7.2 is closer to the observation 7.4 than the mean 8 is, and that proximity outweighs its smaller prior. The posterior mean of the clean state follows,

$$\mathbb{E}\big[X_0 \mid x_t = 7.4\big] = 0.2156 \times 10 + 0.7844 \times 9 = 2.156 + 7.060 = 9.2156.$$

The marginal score at 7.4 averages the two conditional scores with the same weights. For the candidate 10 that score is $5/3 \approx 1.667$, as computed above, and for the candidate 9 it is

$$-\frac{7.4-7.2}{0.36} = -\frac{0.2}{0.36} = -\frac{5}{9} \approx -0.5556,$$

which points left, back toward 7.2. Weighting and summing,

$$s_t(7.4) = 0.2156 \times \tfrac{5}{3} + 0.7844 \times \left(-\tfrac{5}{9}\right) = 0.3593 - 0.4358 = -0.0765.$$

The average points left even though the larger of the two conditional scores points right, because the corruption mean of the dominant candidate is below the observation. We can reach the same number a second way, through the posterior mean. If we take the posterior average inside the conditional score, X_0 is replaced by its posterior mean,

$$s_t(x) = \frac{a_t\,\mathbb{E}[X_0 \mid X_t = x] - x}{\sigma_t^2} = \frac{0.8 \times 9.2156 - 7.4}{0.36} = \frac{-0.02756}{0.36} = -0.0765,$$

which agrees to the displayed precision.

3.16.3 Squared error and the conditional mean

The marginal score is a conditional expectation, which minimizes the corresponding least-squares loss. We can therefore learn it from training pairs containing individual clean states.

Lemma 3.3 (Conditional mean of a squared-error minimizer)**.** *Let X and Y be random variables with $\mathbb{E}\|Y\|^2 < \infty$. For any measurable function f,*

$$\mathbb{E}\big\|f(X) - Y\big\|^2 = \mathbb{E}\big\|f(X) - \mathbb{E}[Y \mid X]\big\|^2 + \mathbb{E}\big\|Y - \mathbb{E}[Y \mid X]\big\|^2,$$

so the minimizer over all such f is $f^\star(X) = \mathbb{E}[Y \mid X]$, and the minimum value is the conditional variance of Y given X.

Proof. Add and subtract $\mathbb{E}[Y \mid X]$ inside the norm and expand. The cross term is

$$2\,\mathbb{E}\Big[\big(f(X) - \mathbb{E}[Y \mid X]\big) \cdot \big(\mathbb{E}[Y \mid X] - Y\big)\Big],$$

and conditioning on X makes the second factor zero while the first factor is fixed, so the cross term vanishes by the tower property. Only the first term on the right depends on f, and it is nonnegative and attains zero at $f = \mathbb{E}[Y \mid X]$. □

The network receives the pair $X = (X_t, t)$. We choose the regression target $Y = -\varepsilon/\sigma_t$, the conditional score for the clean state and noise that produced the sample. The denoising score matching loss regresses the network onto that target, weighted across noise levels [Vincent, 2011],

$$\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{t,\,X_0,\,\varepsilon}\left[\, w(t)\, \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right], \qquad X_t = a_t X_0 + \sigma_t \varepsilon.$$

"The denoising loss $\mathcal{L}_{\mathrm{DSM}}$ averages the weight $w(t)$ times the squared distance between the predicted score $s_\theta(X_t, t)$ and the target, negative Gaussian noise ε divided by the noise standard deviation σ_t, over noise levels t, clean samples X_0, and Gaussian draws ε."

- t is drawn from a distribution over noise levels, uniform in the implementations of this chapter, so every level receives training signal.
- $w(t) > 0$ is a weight that controls how much each noise level contributes.
- $s_\theta(X_t, t)$ is the network, which receives the noisy state and the noise level and nothing else. The clean state and the noise that produced the pair are withheld from it.

Apply Lemma 3.3 at each fixed t. The minimizer over all functions of (x, t) is the posterior average of the target. Section 3.16.2 identifies this average as the marginal score $s_t(x)$. A network with enough capacity, trained to optimality, therefore approximates the score of the noisy-data density at every noise level. We only need training pairs generated from individual clean states and noise draws.

Return to the two-candidate example at $x_t = 7.4$. A pair from $x_0 = 10$ has target 1.667 and posterior weight 0.2156. A pair from $x_0 = 9$ has target -0.5556 and posterior weight 0.7844. A network that outputs a single number c at this input incurs

$$f(c) = 0.2156\,(c - 1.667)^2 + 0.7844\,(c + 0.5556)^2,$$

a quadratic in c whose minimizer is the weighted mean -0.0765 found in Section 3.16.2. Its minimum value is $f(-0.0765) = 0.835$, the posterior variance of the target. That residual stays strictly positive because two clean states remain compatible with the observation, and no function of x_t alone can separate them. A training loss that plateaus above zero is the expected behavior here.

3.16.4 A Gaussian check of the averaging

For standard Gaussian data, we can calculate the conditional expectation in closed form. We take $X_0 \sim \mathcal{N}(0, 1)$ in one coordinate and keep the schedule $a_t = 0.8$, $\sigma_t = 0.6$. Because the path is variance preserving, $a_t^2 + \sigma_t^2 = 0.64 + 0.36 = 1$, so the noisy state

$$X_t = 0.8\,X_0 + 0.6\,\varepsilon$$

is again standard Gaussian, and the marginal score we are trying to learn is $s_t(x) = -x$ at this level and at every other.

The pair (X_0, ε) is jointly Gaussian with X_t, so the conditional law of the noise given the observation follows from the covariance. With $\mathrm{Cov}(\varepsilon, X_t) = \sigma_t = 0.6$ and $\mathrm{Var}(X_t) = 1$,

$$\varepsilon \mid X_t = x \;\sim\; \mathcal{N}\big(0.6\,x,\; 1 - 0.36\big) = \mathcal{N}\big(0.6\,x,\; 0.64\big),$$

a Gaussian with mean $0.6x$ and variance 0.64, so a standard deviation of 0.8. If we average the denoising target over that law, we get

$$\mathbb{E}\left[-\frac{\varepsilon}{\sigma_t} \;\middle|\; X_t = x \right] = -\frac{0.6\,x}{0.6} = -x = s_t(x),$$

which is the marginal score, exactly as Lemma 3.3 requires.

The individual targets scatter widely around that mean. We can fix the observation at $x_t = 1.5$, where the correct score is -1.5. Three training pairs that produce this same observation are

$$\begin{aligned}
(x_0, \varepsilon) = (1.5,\ 0.5): &\quad 0.8 \times 1.5 + 0.6 \times 0.5 = 1.5, &&\text{target} = -0.5/0.6 = -0.833,\\
(x_0, \varepsilon) = (1.125,\ 1.0): &\quad 0.9 + 0.6 = 1.5, &&\text{target} = -1.0/0.6 = -1.667,\\
(x_0, \varepsilon) = (0.75,\ 1.5): &\quad 0.6 + 0.9 = 1.5, &&\text{target} = -1.5/0.6 = -2.5.
\end{aligned}$$

The three targets lie on either side of the marginal score, and none equals it. The conditional law above weights them to give the average -1.5. A network minimizing squared error at this input converges to that average without access to the weights.

By Lemma 3.3 the smallest attainable value of the noise-regression error at a fixed level is the conditional variance of the noise, and here

$$\mathrm{Var}\left(\varepsilon \mid X_t\right) = 1 - \sigma_t^2 = 1 - 0.36 = 0.64,$$

a number that depends on the schedule alone. At convergence, the loss remains positive because the observation cannot determine all of the noise. The amount of undetermined noise sets this residual loss at every level.

The posterior mean of the clean state is $\mathbb{E}[X_0 \mid X_t = x] = a_t x = 0.8 \times 1.5 = 1.2$ by the same covariance argument. If we solve the corruption equation $x_t = a_t x_0 + \sigma_t \varepsilon$ for the clean state, with $-\sigma_t s_t(x_t)$ in place of the noise, we recover that value from the score alone,

$$\hat{x}_0 = \frac{x_t + \sigma_t^2\, s_t(x_t)}{a_t} = \frac{1.5 + 0.36 \times (-1.5)}{0.8} = \frac{1.5 - 0.54}{0.8} = \frac{0.96}{0.8} = 1.2.$$

3.17 Noise prediction and the clean estimate

In Section 3.16, we trained a network to predict the score. We can also train it to predict the noise tensor sampled during corruption, since the two predictions contain the same information. Let $\varepsilon_\theta(x, t) \in \mathbb{R}^d$ be a network that predicts the noise added to the sample, and define the score prediction by dividing out the known scale,

$$s_\theta(x, t) = -\,\frac{\varepsilon_\theta(x, t)}{\sigma_t}.$$

"The predicted score $s_\theta(x,t)$ equals the negative predicted noise $\varepsilon_\theta(x,t)$ divided by the noise standard deviation σ_t, which we obtain directly from the schedule."

To choose the weight $w(t)$, consider the scale of the score target. The score target $-\varepsilon/\sigma_t$ has coordinates of size $1/\sigma_t$, so its magnitude grows without bound as the noise level falls. At $\sigma_t = 0.6$ a unit of sampled noise amounts to $1/0.6 \approx 1.667$ in the target; at $\sigma_t = 0.05$ the same unit amounts to 20. The smallest noise levels have the largest squared targets, so they dominate an unweighted average of squared score errors. The large-noise levels then receive almost no gradient, even though a sampler visits these levels first. If we choose $w(t) = \sigma_t^2$, that factor cancels exactly and the contributions are equalized.

If we substitute the noise prediction and the weight $w(t) = \sigma_t^2$, the scale clears from both terms,

$$\sigma_t^2 \left\| -\frac{\varepsilon_\theta(X_t,t)}{\sigma_t} + \frac{\varepsilon}{\sigma_t} \right\|^2 = \sigma_t^2 \cdot \frac{1}{\sigma_t^2} \left\| \varepsilon - \varepsilon_\theta(X_t,t) \right\|^2 = \left\| \varepsilon_\theta(X_t,t) - \varepsilon \right\|^2,$$

so we obtain a squared-error loss for predicting the sampled noise,

$$\boxed{\mathcal{L}(\theta) = \mathbb{E}_{t,\,X_0,\,\varepsilon}\Big[\, \left\| \varepsilon_\theta(X_t,t) - \varepsilon \right\|^2 \Big]}$$

"The noise-prediction loss $\mathcal{L}(\theta)$ averages the squared difference between the predicted noise $\varepsilon_\theta(X_t,t)$ and the Gaussian noise ε used to create the noisy state X_t, over noise levels t, clean samples X_0, and Gaussian draws."

This is the objective trained in practice [Ho et al., 2020]. It contains no schedule factors, no divergence, and no derivative of the network with respect to its input. By Lemma 3.3 its minimizer is $\mathbb{E}[\varepsilon \mid X_t = x]$, the average noise consistent with the observation, which is $-\sigma_t s_t(x)$ by the same substitution run backwards.

The same output also estimates the clean state. The corruption equation $x_t = a_t x_0 + \sigma_t \varepsilon$ is linear in x_0. For $a_t \neq 0$, solve it for the clean state. Then substitute the network's prediction for the unknown noise,

$$\hat{x}_0 = \frac{x_t - \sigma_t\, \varepsilon_\theta(x_t,t)}{a_t} = \frac{x_t + \sigma_t^2\, s_\theta(x_t,t)}{a_t}.$$

"The clean-state estimate $\hat{x}_0$ equals the noisy state x_t minus the noise standard deviation σ_t times the predicted noise $\varepsilon_\theta(x_t,t)$, divided by the signal coefficient a_t. Equivalently, we add the noise variance σ_t^2 times the predicted score $s_\theta(x_t,t)$ to x_t, then divide by a_t."

At a fixed interior noise level, with $a_t > 0$ and $\sigma_t > 0$, the three predictions contain identical information. Each is an invertible affine function of the others once x_t is fixed, so a network trained to output any one of them determines the other two. We can choose any of the three network outputs. Noise prediction is common because its target is a standard Gaussian draw at every level, so its coordinates have unit scale throughout. The score target grows like $1/\sigma_t$ and diverges as the corruption vanishes. The clean-state target is bounded by the data, and its reconstruction from a noise prediction is divided by a_t, which shrinks toward zero at the noisy end. A network that emits the noise has a target of constant scale across the whole range, and the schedule converts that prediction into either of the other two.

3.17.1 Comparing prediction types

We return to $x_t = 7.4$ with $a_t = 0.8$ and $\sigma_t = 0.6$, and suppose the trained network emits $\varepsilon_\theta = -0.8$ for this input. The Gaussian draw that actually produced the sample was $\varepsilon = -1$, so the prediction errs by 0.2. If we divide by the negative noise scale, it becomes a score,

$$\hat{s}_t = -\frac{-0.8}{0.6} = \frac{0.8}{0.6} \approx 1.333,$$

against the conditional score $5/3 \approx 1.667$ that the true noise would have produced. Reconstructing the clean state from the noise prediction,

$$\hat{x}_0 = \frac{7.4 - 0.6 \times (-0.8)}{0.8} = \frac{7.4 + 0.48}{0.8} = \frac{7.88}{0.8} = 9.85,$$

and reconstructing it from the score prediction instead returns the same value,

$$\hat{x}_0 = \frac{7.4 + 0.36 \times 1.333}{0.8} = \frac{7.4 + 0.48}{0.8} = 9.85,$$

which is close to the clean value 10 that produced this sample.

Part of that gap would remain even after perfect training. Under the two-candidate prior of Section 3.16.2, an optimal network at this input emits $\mathbb{E}[\varepsilon \mid x_t = 7.4] = -\sigma_t s_t(7.4) = -0.6 \times (-0.0765) = 0.0459$, and the clean estimate built from it is

$$\hat{x}_0 = \frac{7.4 - 0.6 \times 0.0459}{0.8} = \frac{7.3724}{0.8} = 9.22,$$

the posterior mean marked in Figure 3.4. A perfectly trained denoiser returns that average. The individual clean state that happened to produce the observation is not recoverable from x_t alone, and the averaging is a property of the problem.

3.18 The corruption, the density, and the score

We can connect the corruption process to its density and score directly. Starting from a clean sample $X_0 \sim p_0$, we add noise according to a stochastic differential equation of the form

$$dX_t = f(X_t, t)\, dt + g(t)\, dW_t,$$

with $f(x, t) \in \mathbb{R}^d$ the systematic drift, $g(t) > 0$ the noise amplitude, and W_t a standard Brownian motion in $\mathbb{R}^d$.

"The state increment dX_t equals the drift $f(X_t, t)$ at the current state X_t and time t times the time increment dt, plus the noise amplitude $g(t)$ times the Brownian increment dW_t."

The variance-preserving path of this chapter is the case $f(x, t) = -\frac{1}{2}\beta(t)\, x$ and $g(t) = \sqrt{\beta(t)}$, with the schedule $\beta(t) > 0$ controlling how fast signal decays [Sohl-Dickstein et al., 2015, Song et al., 2021]. Across many clean samples, the generated paths form a density p_t at each time. We choose the schedule to bring the terminal density close to a standard Gaussian that we can sample.

We can locate our numerical example on this path through its signal coefficient $a_t = \exp\Big(-\frac{1}{2}B(t)\Big)$. Here, $B(t) = \int_0^t \beta(u)\, du$ is the accumulated noising. The value $a_t = 0.8$ therefore corresponds to

$B(t) = -2\log 0.8 = 0.4463$. Under the linear schedule of Section 3.11.5, we reach this accumulation at $t = 0.2068$. The matching noise standard deviation is $\sigma_t = \sqrt{1 - 0.8^2} = \sqrt{0.36} = 0.6$. This is the value used in the worked examples above. The pair $(0.8, 0.6)$ describes one instant of the continuous corruption using the coefficients in the training code.

To connect the path to the score, recall that over a short interval of length h the Brownian displacement is

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

so each coordinate has standard deviation $\sqrt{h}$ and mean squared displacement h. At $h = 0.01$, the standard deviation is 0.1 and the mean square is 0.01. The sum of squared increments over an interval approaches its elapsed time. We write this property as $(dW_t)^2 = dt$. Following a smooth measurement $\varphi(x, t)$ along the path, that surviving square adds a curvature term to the ordinary chain rule, which is Itô's lemma in one coordinate,

$$d\varphi = \Big(\varphi_t + f\,\varphi_x + \tfrac{1}{2}g^2\,\varphi_{xx}\Big)dt + g\,\varphi_x\,dW_t.$$

If we average over paths, the Brownian term drops out, because the stochastic integral has mean zero, and for a measurement with no explicit time dependence we are left with

$$\frac{d}{dt}\,\mathbb{E}\big[\varphi(X_t)\big] = \mathbb{E}\Big[f \cdot \nabla\varphi + \tfrac{1}{2}g^2\nabla^2\varphi\Big],$$

where ∇^2 is the Laplacian, the sum of the second derivatives of φ in the coordinates. Write this average as an integral against p_t. Integration by parts transfers the derivatives from φ to the density. Discarding the boundary terms as before gives the Fokker-Planck equation,

$$\partial_t p_t = -\,\nabla \cdot (f\,p_t) + \tfrac{1}{2}g(t)^2\,\nabla^2 p_t.$$

"The time derivative $\partial_t p_t$ of the density equals the negative divergence of the drift flux $f p_t$, plus one half of the squared noise amplitude $g(t)^2$ times the density Laplacian $\nabla^2 p_t$. The first term transports probability with the drift, and the second spreads it through diffusion."

The density p_t describes the samples generated by the stochastic differential equation. We estimated its score in Sections 3.16 and 3.17. To generate data from noise, we use the time-reversal result for diffusions. The reverse process is a diffusion with the same noise amplitude $g(t)$. Its drift equals the forward drift minus $g(t)^2\,s_t(x)$. We run it with a negative time increment [Anderson, 1982, Song et al., 2021]. We fix f and g ourselves when we choose the schedule. The score s_t is the one unknown in the reverse drift. We estimate it with the denoising loss of Section 3.16, or its equivalent noise regression form in Section 3.17.

3.19 Training a denoiser

At each training step, we draw a batch of clean samples, a noise level for each item, and a Gaussian tensor with the same shape as the data. We corrupt the batch and retain the sampled noise as the target. The network receives the noisy batch and its noise levels. We then differentiate the squared error between the predicted and sampled noise to update the parameters [Paszke et al., 2019].

```python
def training_step(model, diffusion, x_0, optimizer, num_steps):
    # one noise level per batch item, drawn uniformly over the schedule
    k = torch.randint(1, num_steps + 1, (x_0.shape[0],), device=x_0.device)

    # the forward process returns both the noisy batch and the noise it used
    x_k, noise = diffusion.q_sample(x_0, k)

    noise_pred = model(x_k, k)
    loss = F.mse_loss(noise_pred, noise)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()
```

The first line draws one index per batch item, uniformly over steps 1 through K. A single training step therefore covers many noise levels. No level appears only in rare batches. We pass these indices to `q_sample`, the method of Section 3.12.2. It returns the noisy batch x_k and the tensor `noise`. This tensor contains the exact regression target. The call `model(x_k, k)` passes the noise index alongside the state, so one set of weights serves every level. The loss is the mean squared error of the boxed objective in Section 3.17. The three optimizer lines clear the old gradients, accumulate the new ones, and apply the update.

A step costs one forward pass and one backward pass through the network, which is the same cost as a step of ordinary supervised training. The corruption adds two multiplications and one addition per coordinate, and a table lookup for the two schedule coefficients. Each draw of (X_0, t, ε) produces an exact regression target without integrating the stochastic differential equation or evaluating a density.

3.19.1 One training batch at step $k = 50$

Let's run one step with numbers small enough to check by hand. We can take a mini-batch of three clean scalars,

$$x_0^{(1)} = 2.0, \qquad x_0^{(2)} = -1.0, \qquad x_0^{(3)} = 0.5,$$

and suppose the drawn step index is $k = 50$ for all three, where the cumulative signal retention of the discrete schedule is $\bar{\alpha}_{50} = 0.8$. The two corruption coefficients at that step are the square roots

$$a_{50} = \sqrt{\bar{\alpha}_{50}} = \sqrt{0.8} = 0.8944, \qquad \sigma_{50} = \sqrt{1 - \bar{\alpha}_{50}} = \sqrt{0.2} = 0.4472.$$

The sampled noise values are

$$\varepsilon^{(1)} = 0.5, \qquad \varepsilon^{(2)} = -0.2, \qquad \varepsilon^{(3)} = 1.0,$$

and each noisy sample is built by the same rule, $x_{50}^{(i)} = \sqrt{0.8}\, x_0^{(i)} + \sqrt{0.2}\, \varepsilon^{(i)}$. Substituting item by item,

$$\begin{aligned}
x_{50}^{(1)} &= 0.8944 \times 2.0 + 0.4472 \times 0.5 = 1.7889 + 0.2236 = 2.0125, \\
x_{50}^{(2)} &= 0.8944 \times (-1.0) + 0.4472 \times (-0.2) = -0.8944 - 0.0894 = -0.9839, \\
x_{50}^{(3)} &= 0.8944 \times 0.5 + 0.4472 \times 1.0 = 0.4472 + 0.4472 = 0.8944.
\end{aligned}$$

To two decimals the noisy batch is $x_{50} = (2.01,\ -0.98,\ 0.89)$. Each entry has been pulled toward zero by the factor 0.894 and then displaced by its own noise draw.

The network receives the whole noisy batch together with the step index, and we suppose it emits

$$\varepsilon_\theta(x_{50}, 50) = (0.4,\ -0.1,\ 0.8).$$

Comparing with the drawn noise $(0.5, -0.2, 1.0)$ and averaging the squared errors over the three items,

$$\mathcal{L} = \frac{1}{3}\Big[(0.5-0.4)^2 + (-0.2+0.1)^2 + (1.0-0.8)^2\Big] = \frac{1}{3}[0.01 + 0.01 + 0.04] = \frac{0.06}{3} = 0.02.$$

We differentiate this scalar loss during the backward pass. The predictions are close to their targets, so the errors are small. At initialization, the network would produce a loss near the mean squared noise, which is 1 for standard Gaussian draws.

The same batch also has its score targets, which the noise regression never forms explicitly. Dividing each drawn noise value by $-\sigma_{50} = -0.4472$,

$$-\frac{\varepsilon^{(1)}}{\sigma_{50}} = -1.118, \qquad -\frac{\varepsilon^{(2)}}{\sigma_{50}} = 0.447, \qquad -\frac{\varepsilon^{(3)}}{\sigma_{50}} = -2.236.$$

Each points from the noisy sample back toward its own conditional mean, and their magnitudes exceed the noise magnitudes by the factor $1/\sigma_{50} = 2.236$. With the noise regression that factor never appears in the gradient, and the weight $w(t) = \sigma_t^2$ of Section 3.17 makes the two losses equivalent.

The same three predictions also reconstruct the clean batch, through the estimate of Section 3.17. For the first item,

$$\hat{x}_0^{(1)} = \frac{2.0125 - 0.4472 \times 0.4}{0.8944} = \frac{2.0125 - 0.1789}{0.8944} = \frac{1.8336}{0.8944} = 2.05,$$

and if we repeat for the other two we get -1.05 and 0.60, against the clean values 2.0, -1.0 and 0.5. A noise error of 0.1 at this step moves the clean estimate by $\sigma_{50}/a_{50} \times 0.1 = 0.05$. The ratio of the schedule coefficients therefore amplifies the noise error in the clean reconstruction. That ratio grows as the signal decays, which is why reconstructions from heavily noised states are poor even when the noise prediction itself is accurate.

3.19.2 Conditioning on the noise level

We use one network at every noise level. Its target at a given state depends on the level, since the same x can represent lightly perturbed data at small k or almost pure noise at large k. We therefore pass the step index as a second input [Song and Ermon, 2019]. First, encode the index into a vector whose width matches the layers. Then add this vector within the layers. Every layer can then use the current noise level.

The encoding is itself a small network, and the injection is an addition into the layer input. For data small enough that a multilayer perceptron suffices, both operations fit in a few lines.

3.19.3 An MLP denoiser for small vectors

For vectors in $\mathbb{R}^{64}$, which is the size of a learned descriptor or a small latent code, a two-layer perceptron suffices for noise prediction.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # step embedding: input is a scalar k, output has the width of x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim),
        )

        # main network: predicts the noise from x_k plus the step embedding
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim),
        )

    def forward(self, x_k, k):
        # normalize the step index from [0, K] to [0, 1]
        k = k.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(k)          # embed the noise level
        return self.net(x_k + t_emb)      # predict the noise
```

The stack `time_mlp` maps a scalar through a hidden layer of width `time_embed_dim` to a vector of width `data_dim`. The output width matches the data, so we can add the embedding to the state. The second stack, `net`, predicts the noise. It maps a vector of width `data_dim` through a hidden layer of 128 units and back to width `data_dim`. This restores the data shape required for the noise prediction.

The step index arrives as an integer tensor with one entry per batch item. The expression `k.float().unsqueeze(1) / 1000` casts this tensor and adds the feature axis required by the linear layer. It also divides by the number of steps so the input lies in $[0, 1]$ whatever the schedule length. Passing the result through `time_mlp` produces one embedding vector per batch item. We add this vector to `x_k`, shifting the input by an amount that depends only on the noise level. The final stack maps the shifted input to the predicted noise. Addition is the cheapest way to inject the embedding. This network adds it once at the input. Larger networks repeat the addition inside every block.

We choose the architecture to match the data. For a small vector, the perceptron above is enough. For an image, we use the U-Net architecture [Ronneberger et al., 2015] adapted by DDPM for diffusion [Ho et al., 2020]. This convolutional network has downsampling and upsampling paths connected by skip connections. For a sequence of residue embeddings we use a transformer encoder with attention across positions [Vaswani et al., 2017]. The loss, the target and the time conditioning are identical in all three, and only the network body changes.

With the noise regression objective of Section 3.17, we train a denoiser to estimate $\varepsilon_\theta(x, t)$ at every state and noise level. Dividing its prediction by $-\sigma_t$ gives the score estimate $s_\theta(x, t)$ at the same state and level. We can use this estimate to run the corruption backwards from the terminal noise distribution. At each step, we substitute the trained score for the unknown score in the sampling dynamics.

3.20 Noise-prediction network

A denoiser trained with noise regression estimates the score at every noise level. During generation, we evaluate it once per reverse step. Its architecture must match the data shape.

The network predicts $\varepsilon_\theta(x_k, k)$, an estimate of the standard Gaussian draw mixed into the clean sample to produce x_k. It has exactly the shape of x_k. For a 28×28 grayscale image that is 784 numbers in and 784 numbers out, one predicted noise value per pixel. The network needs local texture to estimate noise on individual pixels. It also needs global layout because the digit's identity constrains the possible clean pixel values. We use the U-Net architecture [Ronneberger et al., 2015] to incorporate both kinds of information. Its encoder lowers spatial resolution and widens the channel dimension, ending at a bottleneck with the coarsest resolution. The decoder restores resolution. Skip connections carry encoder features to the matching decoder stage. DDPM uses this architecture for the noise prediction on images [Ho et al., 2020].

We also condition the network on the step index k, because the required correction depends on the noise level. This conditioning must reach every spatial location, since the entire tensor has the same noise level. We can implement these operations in a convolutional block.

```
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                              padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb
```

In the constructor, `GroupNorm(8, in_channels)` splits the incoming channels into eight groups and normalizes within each group. This keeps the statistics stable across the input scales produced by different noise levels. `SiLU` is the smooth activation $x \mapsto x/(1 + e^{-x})$, the state multiplied by its own logistic value. The 3×3 convolution with `padding=1` maps `in_channels` feature maps to `out_channels` feature maps at unchanged spatial size. The linear layer `time_proj` maps the time embedding $e_k \in \mathbb{R}^{d_{\text{time}}}$ to one real number per output channel.

During the forward pass, we normalize the input, apply the activation and convolution, and add

the projected step embedding. Writing A for the weight matrix of `time_proj` and e_k for the step embedding, the block computes

$$h = \mathrm{Conv}\big(\mathrm{SiLU}(\mathrm{GN}(x))\big), \qquad \text{output} = h + Ae_k,$$

with the second term broadcast across height and width. The two calls to `unsqueeze(-1)` give the projected embedding the dimensions required for broadcasting. With a batch of $B = 128$ images and $d_{\text{time}} = 128$, the embedding `t_emb` has shape $[128, 128]$. For a block with 64 output channels, `time_proj(t_emb)` projects it to shape $[128, 64]$. The two unsqueezes then produce shape $[128, 64, 1, 1]$. PyTorch then adds it to `h` of shape $[128, 64, 28, 28]$ by repeating the same 64 offsets at all $28 \times 28 = 784$ locations. Each channel of each image receives one offset that depends only on the diffusion step. The network can therefore predict different corrections for the same feature pattern at different noise levels.

Three of these blocks and a final projection make a complete noise predictor.

```
class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, k):
        # Embed the step index k.
        k = k.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(k)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

The two-layer `time_mlp` turns the scalar step into a vector in $\mathbb{R}^{128}$. Its input dimension is 1 because the step index is a single number. The forward pass first divides this index by 1000. The input therefore lies in $[0, 1]$ for the thousand-step schedule used here. The three blocks run at full spatial resolution with channel counts $1 \to 32 \to 64 \to 32$, widening to let the middle block hold more feature types and narrowing again. The final 1×1 convolution maps the 32 channels back to the single image channel. The output now has the input shape, so we can compare it directly with the sampled noise. Each block receives the same `t_emb`, which is how the step index reaches all three stages.

This three-block network preserves the input shape, conditions on a scalar step, and applies that conditioning at every spatial location.

Peptides are sequences over twenty amino acids. We cannot add Gaussian noise directly to these discrete tokens, so Gaussian diffusion requires continuous coordinates for each peptide.

3.21 Latent diffusion on protein language model embeddings

Gaussian corruption and denoising require a state in $\mathbb{R}^d$, where we can add noise and subtract a noise estimate. In latent diffusion, we first map each amino acid sequence to continuous coordinates with a pretrained encoder. We then run diffusion in this encoder output space.

For proteins the encoder is a protein language model. ESM-2 is a masked language model over amino acid sequences [Lin et al., 2023], trained with the masked-language objective used by BERT [Devlin et al., 2019]. We hide a subset $\mathcal{M}$ of positions and train the model to recover them from the visible residues,

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\setminus \mathcal{M}}).$$

"The masked-language loss $\mathcal{L}_{\text{MLM}}$ sums the negative log probability of the correct amino acid x_i at each hidden position i in the masked set $\mathcal{M}$, conditioned on the visible residues $x_{\setminus \mathcal{M}}$."

ESM-2 was trained on roughly 70 million sequences with this objective. Predicting hidden residues requires sequence representations that capture constraints on folding, function, and biophysical behavior. These representations also support structure prediction. To use them as diffusion inputs, we pass a peptide of length L through the encoder and obtain a tensor

$$Z_0 \in \mathbb{R}^{B \times L \times D},$$

with B sequences in the batch, L token positions per sequence, and D embedding coordinates per position. For the 650-million-parameter ESM-2 used here, $D = 1280$. Each residue receives its own D-dimensional vector, and that vector depends on the whole sequence, because attention has mixed the positions.

AMP-Diffusion runs DDPM on exactly this tensor to generate antimicrobial peptides [Chen et al., 2023]. The corruption is the Gaussian path of Section 3.12, applied coordinatewise to the embeddings,

$$z_k = \sqrt{\bar{\alpha}_k}\, z_0 + \sqrt{1 - \bar{\alpha}_k}\, \varepsilon, \qquad \bar{\alpha}_k = \prod_{j=1}^{k} (1 - \beta_j).$$

"The noisy latent z_k equals the clean latent z_0 times the square root of the cumulative retention $\bar{\alpha}_k$, plus a same-shaped standard Gaussian draw ε times the square root of the accumulated noise variance $1 - \bar{\alpha}_k$. The retention $\bar{\alpha}_k$ is the product of the one-step retentions $1 - \beta_j$ through step k."

The AMP-Diffusion paper writes the cumulative product without a bar and indexes it by t. We retain the bar to distinguish the one-step retention $\alpha_k = 1 - \beta_k$ from the product $\bar{\alpha}_k$. We also retain the step index k, reserving t for continuous time in the stochastic differential equation.

The network output is the clean latent, and the loss is the squared distance between that prediction and the original embedding,

$$\mathcal{L}(\theta) = \mathbb{E}_{z_0, k, \varepsilon}\Big[\, \big\| \hat{z}_\theta(z_k, k) - z_0 \big\|_2^2 \,\Big].$$

"The clean-latent loss $\mathcal{L}(\theta)$ averages the squared Euclidean distance between the denoiser prediction $\hat{z}_\theta(z_k, k)$, computed from the noisy latent z_k and step k, and the clean latent z_0, over clean latents, step indices, and noise draws."

Both parameterizations contain the same information. We can convert a clean-latent prediction into a noise estimate and apply the DDPM reverse step unchanged.

We can take one coordinate of one residue's standardized embedding, $z_0 = 1.2$, and corrupt it at step $k = 500$ of the thousand-step schedule, where $a_{500} = \sqrt{\bar{\alpha}_{500}} = 0.2803$ and $\sigma_{500} = \sqrt{1 - \bar{\alpha}_{500}} = 0.9599$. With a noise draw $\varepsilon = 0.4$,

$$z_{500} = 0.2803 \times 1.2 + 0.9599 \times 0.4 = 0.3364 + 0.3840 = 0.7204.$$

At this halfway step, the clean contribution is 0.3364 and the noise contribution is 0.3840, so the two are already comparable. Every one of the $L \times 1280$ coordinates of the peptide is corrupted this way, with the same two coefficients and an independent noise draw.

3.21.1 The latent denoiser

We use a Transformer encoder [Vaswani et al., 2017] to denoise the sequence of residue embeddings. This matches the architecture that produced the embeddings. To condition it on the noise level, we first encode the step index.

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, k):
        # k: [B] -> normalize -> [B, 1] -> [B, D]
        k = k.float().unsqueeze(1) / 1000
        return self.time_mlp(k)
```

A two-layer network with a ReLU between the layers turns the scalar step into a vector in $\mathbb{R}^D$. The output width equals the embedding width, so we can add the result directly to a residue embedding without any further projection. Dividing the step index by 1000 puts it in $[0, 1]$ for the thousand-step schedule. This keeps the input to the first linear layer on the same scale as the rest of the network.

```
class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
```

```
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_k, k):
        """
        z_k: [B, L, D] - noised ESM-2 embeddings
        k:   [B]       - diffusion step
        """
        # Step conditioning
        t_emb = self.time_embed(k).unsqueeze(1)  # [B, 1, D]
        h = z_k + t_emb  # inject the step globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted clean latent or noise
```

The constructor builds one `TransformerEncoderLayer` with model width 1280, eight attention heads and a feed-forward width of $4 \times 1280 = 5120$, then stacks four copies of it. `batch_first=True` fixes the tensor layout as $[B, L, D]$, which is the layout the ESM-2 encoder produces. The output head is a layer normalization followed by one linear map back to the embedding width, so the prediction is in the same space as the input.

During the forward pass, `unsqueeze(1)` adds a middle axis to the time embedding, changing its shape from $[B, D]$ to $[B, 1, D]$. We then add this embedding to every one of the L residue positions. Every position therefore receives the same offset from the step index. This matches the broadcasting across height and width in the convolutional block of Section 3.20. The stacked self-attention layers then mix the positions, so the correction at one residue depends on the whole peptide. The final linear head produces the prediction, one D-dimensional vector per residue.

We train both the image network and the latent network by the same regression, and evaluate both once per reverse step. Neither network contains the rule for the reverse step itself.

3.22 Langevin dynamics across noise levels

Suppose we know the score of a fixed density exactly. We can use it to draw samples by Langevin dynamics. Let's write p for the target density, $s = \nabla \log p$ for its score, and τ for sampler time at this density,

$$dX_\tau = s(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau.$$

"The state increment dX_τ equals the target score $s(X_\tau)$ at the current state X_τ times the sampler-time increment $d\tau$, plus the constant noise scale $\sqrt{2}$ times the Brownian increment dW_τ."

- τ is sampler time, measured at one fixed density, independently of the noising time t of the corruption path.

- $s(x) = \nabla \log p(x)$ is the score of the target density, with the same dimension as the state.
- $\sqrt{2}$ is the diffusion coefficient, a constant here, so the injected noise has variance $2\,d\tau$ per coordinate.

The drift moves the state uphill in log density and the noise pushes it off the peak. Those two effects balance exactly when the density is p. Putting the drift $f = s = \nabla \log p$ and the diffusion coefficient $g = \sqrt{2}$ into the Fokker-Planck equation of Theorem 3.2,

$$\partial_\tau p = -\nabla \cdot (p \nabla \log p) + \nabla^2 p = -\nabla \cdot (\nabla p) + \nabla^2 p = 0,$$

where the middle step uses $p \nabla \log p = \nabla p$. Under suitable regularity conditions p is therefore stationary, and a long run of the dynamics produces samples from p.

We take $p = \mathcal{N}(0, 1)$, whose score is $s(x) = -x$, start at $x = 2$, use a step size $\eta = 0.01$, and draw $z = 0.5$. The Euler-Maruyama update of the Langevin dynamics is $x' = x + \eta\, s(x) + \sqrt{2\eta}\, z$,

$$x' = 2 + 0.01 \times (-2) + \sqrt{0.02} \times 0.5 = 2 - 0.02 + 0.1414 \times 0.5 = 2.0507.$$

The drift pulls the state 0.02 toward the mean and this particular noise draw pushes it 0.0707 away, so a single step need not move toward the peak. Averaged over draws, the noise adds nothing and only the restoring 0.02 remains, which keeps the chain from running off to infinity.

Scores estimated from data are poor in regions with almost no data mass. A Langevin chain initialized in such a region therefore takes a long time to reach the modes. We can improve sampling by annealing the noise level. We can learn a score network at a decreasing sequence of M noise levels $\sigma_1 > \cdots > \sigma_M > 0$ and run a few Langevin iterations at each level before moving down [Song and Ermon, 2019]. Indexing the levels by i and the iterations within a level by j, with a step size η_i at level i,

$$x^{j+1} = x^j + \eta_i\, s_\theta(x^j, \sigma_i) + \sqrt{2\eta_i}\, z^j, \qquad \sigma_1 > \cdots > \sigma_M > 0.$$

"The next sample x^{j+1} equals the current sample x^j, plus the step size η_i times the predicted score $s_\theta(x^j, \sigma_i)$ at noise level σ_i, plus a fresh standard Gaussian draw z^j times the square root of twice η_i."

At a large noise level the smoothed density has shallow valleys between its modes, so the chain crosses between them easily. At a small noise level the density is close to the data density and the chain resolves fine structure. If we run the levels in decreasing order, the chain settles into a mode first and sharpens inside it afterwards, which is annealed Langevin sampling.

We hold the density fixed at each level for several iterations, with no rule connecting one level to the next beyond the choice of the ladder. The corruption path of this chapter is already a continuum of densities p_t indexed by a single time. We still need a set of dynamics that tracks that continuum backwards in one pass.

3.23 The reverse-time SDE

The forward corruption is an SDE with drift f and noise amplitude g,

$$dX_t = f(X_t, t)\, dt + g(t)\, dW_t,$$

and running it from $t = 0$ to $t = T$ moves the data density $p_0 = p_{\text{data}}$ to a terminal density p_T close to a standard Gaussian. To reverse it we have to find a second SDE whose marginals at every time are the same p_t, traversed from T down to 0. Such an SDE exists, and it is the forward one with a score correction added to the drift [Anderson, 1982, Song et al., 2021]. Generation runs backward in noising time, so $dt < 0$, and the process $\overline{W}_t$ has Brownian increments for that reversed clock,

$$\boxed{dX_t = \Big[f(X_t, t) - g(t)^2 s_t(X_t)\Big]dt + g(t)\, d\overline{W}_t, \qquad dt < 0.}$$

"The backward state increment dX_t equals the forward drift $f(X_t, t)$ minus the squared noise amplitude $g(t)^2$ times the marginal score $s_t(X_t)$, all multiplied by the negative time increment dt. We then add the noise amplitude $g(t)$ times the reverse Brownian increment $d\overline{W}_t$."

- $f(x, t)$ is the drift of the forward corruption, reused unchanged.
- $g(t)$ is the forward noise amplitude, also reused unchanged, so the reverse process injects noise at the same rate the forward process did.
- $s_t(x) = \nabla \log p_t(x)$ is the score of the corrupted density at noising time t, and it is the only term the forward process does not already contain. The trained denoiser estimates it as $s_\theta(x, t)$.
- $\overline{W}_t$ is a Brownian motion for the reversed clock.

Because dt is negative, the term $-g(t)^2 s_t(X_t)\, dt$ points along $+s_t$, so the score correction moves the sample toward increasing density at every step. This correction concentrates the Gaussian cloud back onto the data.

A negative time increment through a numerical implementation is an easy source of sign errors, so we measure time from the start of generation instead. We can set $\tau = T - t$ and $Y_\tau = X_t$, which makes $d\tau = -dt$ positive. Substituting into the reverse SDE,

$$dY_\tau = \Big[- f(Y_\tau, T - \tau) + g(T - \tau)^2 s_{T-\tau}(Y_\tau)\Big]d\tau + g(T - \tau)\, d\overline{W}_\tau.$$

"The generation increment dY_τ equals the negative forward drift f, plus the squared noise amplitude g^2 times the score $s_{T-\tau}$, evaluated at state Y_τ and noising time $T - \tau$, all multiplied by the positive generation-time increment $d\tau$. The random contribution is $g(T - \tau)$ times the reverse Brownian increment $d\overline{W}_\tau$."

Every step size in this form is positive.

3.23.1 The reverse drift at Gaussian equilibrium

Let's check the score correction for the variance-preserving process at equilibrium. For that process $f(x, t) = -\frac{1}{2}\beta(t)x$ and $g(t) = \sqrt{\beta(t)}$, and the terminal density is a standard Gaussian, whose score is $s(x) = -x$. Evaluating the generation drift at that equilibrium, and writing β for $\beta(T - \tau)$,

$$-f + g^2 s = \tfrac{1}{2}\beta x + \beta(-x) = -\tfrac{1}{2}\beta x.$$

"The generation drift is the negative forward drift $-f$ plus the squared noise scale g^2 times the score s. At Gaussian equilibrium, these contributions are one half of the noising rate β times the state x, and β times the score $-x$, leaving the restoring drift $-\frac{1}{2}\beta x$."

The reverse drift is the same restoring drift the forward process has, so the reverse process preserves the standard Gaussian exactly as the forward process does. With $\beta = 5$ and $x = 2$, the first term is $\frac{1}{2} \times 5 \times 2 = 5$, the second is $5 \times (-2) = -10$, and the sum is -5, which is $-\frac{1}{2} \times 5 \times 2$. If we dropped the score term and negated the drift alone, we would be left with $+\frac{1}{2}\beta x$, a repelling drift, and the equilibrium would spread outward. The score term flips that sign back.

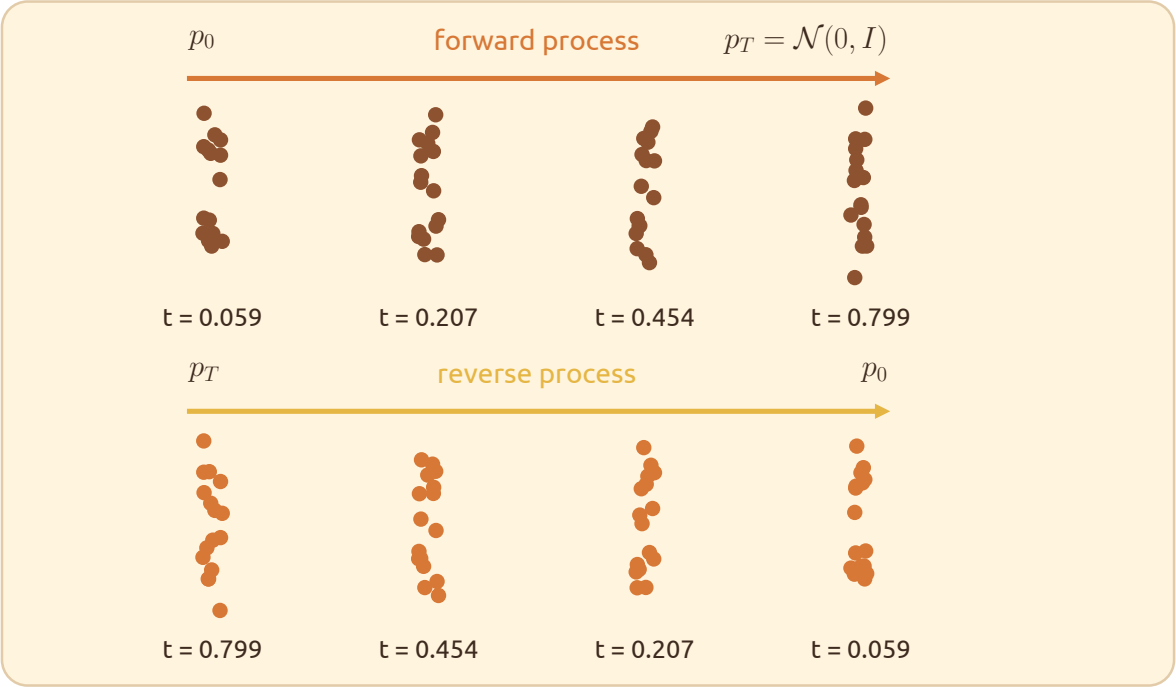


Figure 3.5: A one-dimensional population under the two directions of the same path. Each column of dots is one set of sampled states at the labeled time, with the state on the vertical axis. The top row runs the forward corruption of the variance-preserving process from the two-cluster data density p_0 at $t = 0.059$, where the signal coefficient is $a_t = 0.98$, to $p_T = \mathcal{N}(0, I)$ at $t = 0.799$, where $a_t = 0.04$; the clusters merge as the signal coefficient decays through 0.80 and 0.35. The bottom row runs the reverse-time SDE over the same four times in the opposite order, and the score correction $-g(t)^2 s_t$ reseparates the clusters.

In Figure 3.5, the forward and reverse processes have the same marginals in opposite time directions. To sample numerically, we run the path on a finite grid of steps.

3.24 DDPM discretization

To implement corruption and generation, we need both processes on a finite grid. We use explicit Gaussian transitions whose coefficients we can compute before training. Denoising diffusion probabilistic models use such a grid [Ho et al., 2020, Sohl-Dickstein et al., 2015]. The index k runs over the grid, with $k = 0$ the clean data and $k = K$ the noisiest state; the continuous time t stays reserved for the SDE description.

Each step retains a fraction of the signal and adds fresh Gaussian noise. Let $\beta_k \in (0,1)$ be the variance of the noise added at step k, and define the one-step signal retention

$$\alpha_k = 1 - \beta_k.$$

The transition itself scales the previous state and adds an independent standard Gaussian draw ε_k,

$$X_k = \sqrt{\alpha_k}\, X_{k-1} + \sqrt{\beta_k}\, \varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, I).$$

"The next state X_k equals the previous state X_{k-1} times the square root of the one-step retention α_k, plus an independent standard Gaussian draw ε_k times the square root of the step noise variance β_k."

We choose the two coefficients so that their squares add to one, $\alpha_k + \beta_k = 1$. An input with unit variance therefore comes out of the step with unit variance, which is the discrete version of the variance-preserving property established earlier in this chapter.

If we compose the steps, they collapse into a single Gaussian, because a sum of independent Gaussians is Gaussian. We define the cumulative retention

$$\bar{\alpha}_k = \prod_{j=1}^{k} \alpha_j,$$

and the composition of the first k steps is

$$X_k = \sqrt{\bar{\alpha}_k}\, X_0 + \sqrt{1 - \bar{\alpha}_k}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

"The noisy state X_k equals the clean state X_0 times the square root of the cumulative retention $\bar{\alpha}_k$, plus a standard Gaussian draw ε times the square root of the accumulated noise variance $1 - \bar{\alpha}_k$. The retention $\bar{\alpha}_k$ is the product of all one-step retentions through step k."

This is the Gaussian corruption of the variance-preserving SDE with a_t replaced by $\sqrt{\bar{\alpha}_k}$ and σ_t replaced by $\sqrt{1 - \bar{\alpha}_k}$. Here $\bar{\alpha}_k$ and $1 - \bar{\alpha}_k$ are variances and their square roots are standard deviations. During training we can therefore sample a step index k directly and jump to that noise level in one line, with no simulation of the intermediate steps.

With a two-step schedule and deliberately large noise we can inspect every coefficient. Let's set $\beta_1 = \beta_2 = 0.2$, so $\alpha_1 = \alpha_2 = 0.8$ and

$$\bar{\alpha}_2 = 0.8 \times 0.8 = 0.64, \qquad \sqrt{\bar{\alpha}_2} = 0.8, \qquad \sqrt{1 - \bar{\alpha}_2} = \sqrt{0.36} = 0.6.$$

With the clean value $x_0 = 10$ and the noise draw $\varepsilon = -1$ used throughout this chapter,

$$x_2 = 0.8 \times 10 + 0.6 \times (-1) = 8 - 0.6 = 7.4,$$

which is the same noisy sample the continuous path produced with $a_t = 0.8$ and $\sigma_t = 0.6$. The discrete and continuous descriptions agree on this example by construction.

3.24.1 The schedule used in practice

The schedule actually used for images is linear in k. With $K = 1000$ steps, $\beta_1 = 10^{-4}$ and $\beta_K = 0.02$, the betas are the thousand equally spaced values produced by `torch.linspace(1e-4, 0.02, 1000)`. This is the discretization of the continuous rate $\beta(t) = 0.1 + 19.9t$ at step size $h = 1/1000$, because $\beta(0)h = 10^{-4}$ and $\beta(1)h = 0.02$, with linear interpolation in between.

The continuous signal coefficient is $a_t = e^{-B(t)/2}$ with $B(t) = 0.1t + 9.95t^2$, and the discrete one is $\sqrt{\bar{\alpha}_k}$ at $k = 1000t$. At $t = 0.25$ they are 0.7237 and 0.7239; at $t = 0.5$ they are 0.2812 and 0.2803; at $t = 1$ they are 0.00657 and 0.00635. The terminal state retains well under one percent of the signal, which is why the sampler can start from a standard Gaussian with no visible error.

The continuous value $a_t = 0.8$ of the running example occurs at $t = 0.2068$, and the nearest discrete step is $k = 207$, where $\sqrt{\bar{\alpha}_{207}} = 0.8001$ and $\sqrt{1 - \bar{\alpha}_{207}} = 0.5999$. Two steps at $\beta = 0.2$ therefore reach the noise level that the practical schedule reaches only after 207 steps.

3.24.2 Reverse posterior derivation

To generate we need the distribution of X_{k-1} given X_k. That distribution is not available, because it depends on the data density through the unknown marginal $q(x_{k-1})$. Conditioning on the clean state x_0 gives a Gaussian distribution whose coefficients we can calculate. Applying Bayes' rule to the forward step and the earlier corruption,

$$q(x_{k-1} \mid x_k, x_0) \;\propto\; q(x_k \mid x_{k-1})\, q(x_{k-1} \mid x_0),$$

as functions of x_{k-1}, with the normalizer $q(x_k \mid x_0)$ absorbed into the proportion because it does not depend on x_{k-1}.

"The reverse density $q(x_{k-1} \mid x_k, x_0)$ of the previous state x_{k-1}, given the current noisy state x_k and clean state x_0, is proportional to the forward-transition density $q(x_k \mid x_{k-1})$ times the earlier corruption density $q(x_{k-1} \mid x_0)$."

Both factors are Gaussian in x_{k-1}. The forward-step density over x_k has mean $\sqrt{\alpha_k}x_{k-1}$ and variance β_k, and the earlier corruption has mean $\sqrt{\bar{\alpha}_{k-1}}x_0$ and variance $1 - \bar{\alpha}_{k-1}$. Writing both exponents and keeping only the terms that involve x_{k-1},

$$\log q(x_{k-1} \mid x_k, x_0) = -\frac{\|x_k - \sqrt{\alpha_k}\, x_{k-1}\|^2}{2\beta_k} - \frac{\|x_{k-1} - \sqrt{\bar{\alpha}_{k-1}}\, x_0\|^2}{2(1 - \bar{\alpha}_{k-1})} + \text{const.}$$

Expanding both squares and collecting the quadratic and linear terms in x_{k-1},

$$= -\frac{1}{2}\left(\frac{\alpha_k}{\beta_k} + \frac{1}{1 - \bar{\alpha}_{k-1}}\right)\|x_{k-1}\|^2 + \left\langle x_{k-1},\; \frac{\sqrt{\alpha_k}\, x_k}{\beta_k} + \frac{\sqrt{\bar{\alpha}_{k-1}}\, x_0}{1 - \bar{\alpha}_{k-1}} \right\rangle + \text{const.}$$

This quadratic is a Gaussian exponent. The coefficient of $-\frac{1}{2}\|x_{k-1}\|^2$ gives the inverse variance. Multiplying that variance by the vector in the inner product gives the mean. Adding the two inverse variances and taking the reciprocal,

$$\tilde{\beta}_k = \left(\frac{\alpha_k}{\beta_k} + \frac{1}{1 - \bar{\alpha}_{k-1}}\right)^{-1} = \frac{\beta_k(1 - \bar{\alpha}_{k-1})}{\alpha_k(1 - \bar{\alpha}_{k-1}) + \beta_k}.$$

The denominator simplifies, because $\alpha_k(1-\bar{\alpha}_{k-1})+\beta_k = \alpha_k - \bar{\alpha}_k + 1 - \alpha_k = 1-\bar{\alpha}_k$, giving

$$\tilde{\beta}_k = \frac{\beta_k\,(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k}.$$

"The reverse variance $\tilde{\beta}_k$ equals the step noise variance β_k times one minus the previous cumulative retention $\bar{\alpha}_{k-1}$, divided by one minus the current cumulative retention $\bar{\alpha}_k$."

The factor $(1-\bar{\alpha}_{k-1})/(1-\bar{\alpha}_k)$ is below one, so $\tilde{\beta}_k < \beta_k$ at every interior step. Once we know the clean state, the uncertainty about the previous noisy state falls below the variance that the forward step injected.

If we multiply the variance by the linear term, we get the mean. Using $\tilde{\beta}_k$ in the form derived above,

$$\tilde{\mu}_k(x_k, x_0) = \tilde{\beta}_k\left(\frac{\sqrt{\alpha_k}\,x_k}{\beta_k} + \frac{\sqrt{\bar{\alpha}_{k-1}}\,x_0}{1-\bar{\alpha}_{k-1}}\right) = \frac{\sqrt{\bar{\alpha}_{k-1}}\,\beta_k}{1-\bar{\alpha}_k}\,x_0 + \frac{\sqrt{\alpha_k}\,(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k}\,x_k.$$

"The reverse mean $\tilde{\mu}_k$ is a weighted sum of the clean state x_0 and current noisy state x_k. The clean-state weight is the square root of the previous cumulative retention $\bar{\alpha}_{k-1}$ times the step variance β_k, divided by the current noise variance $1-\bar{\alpha}_k$. The noisy-state weight is the square root of the one-step retention α_k times the previous noise variance $1-\bar{\alpha}_{k-1}$, divided by the same denominator."

The reverse step is therefore an interpolation between the clean state and the current noisy state, with weights fixed entirely by the schedule. At generation time the clean state is the one thing we do not have, so we substitute the denoiser's clean estimate for it.

3.24.3 The reverse mean through the noise prediction

Earlier in this chapter we wrote the clean estimate in terms of the noise prediction. On the discrete grid that identity reads

$$\hat{x}_0 = \frac{x_k - \sqrt{1-\bar{\alpha}_k}\;\varepsilon_\theta(x_k, k)}{\sqrt{\bar{\alpha}_k}},$$

which is the corruption equation solved for X_0 with the predicted noise in place of the unknown draw. If we substitute this into $\tilde{\mu}_k$ and collect the terms in x_k, the clean state x_0 drops out entirely. The coefficient of x_k becomes

$$\frac{\sqrt{\bar{\alpha}_{k-1}}\,\beta_k}{(1-\bar{\alpha}_k)\sqrt{\bar{\alpha}_k}} + \frac{\sqrt{\alpha_k}\,(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k} = \frac{\beta_k + \alpha_k - \bar{\alpha}_k}{\sqrt{\alpha_k}\,(1-\bar{\alpha}_k)} = \frac{1}{\sqrt{\alpha_k}},$$

using $\sqrt{\bar{\alpha}_{k-1}}/\sqrt{\bar{\alpha}_k} = 1/\sqrt{\alpha_k}$ in the first fraction and $\beta_k + \alpha_k = 1$ in the numerator. The coefficient of ε_θ collapses the same way, and what is left is

$$\boxed{\mu_\theta(x_k, k) = \frac{1}{\sqrt{\alpha_k}}\left(x_k - \frac{\beta_k}{\sqrt{1-\bar{\alpha}_k}}\,\varepsilon_\theta(x_k, k)\right)}$$

"The learned reverse mean $\mu_\theta(x_k, k)$ equals the noisy state x_k minus the predicted noise $\varepsilon_\theta(x_k, k)$ times the step variance β_k divided by the accumulated noise standard deviation $\sqrt{1-\bar{\alpha}_k}$, all divided by the square root of the one-step retention α_k."

One forward pass of the network therefore produces everything we need for the reverse step. From that one output we compute the clean estimate $\hat{x}_0$, the score $s_\theta = -\varepsilon_\theta/\sqrt{1-\bar{\alpha}_k}$ and the reverse mean μ_θ, and the sampler works with whichever of the three is convenient.

3.25 The variational objective

We obtained the noise-regression loss by reweighting the score objective. DDPM derives it from a likelihood bound on the discrete chain. This derivation also explains the squared error and the schedule weights omitted from the practical loss.

We define the reverse chain $p_\theta(x_{0:K}) = p(x_K)\prod_{k=1}^{K} p_\theta(x_{k-1} \mid x_k)$ with prior $p(x_K) = \mathcal{N}(0, I)$. The forward corruption $q(x_{1:K} \mid x_0)$ serves as the approximate posterior. Applying Jensen's inequality to the marginal likelihood produces the variational bound [Kingma and Welling, 2014],

$$-\log p_\theta(x_0) \;\le\; \mathbb{E}_q\left[-\log \frac{p_\theta(x_{0:K})}{q(x_{1:K} \mid x_0)}\right].$$

If we rewrite the forward chain through the posteriors $q(x_{k-1} \mid x_k, x_0)$ of Section 3.24.2, the right-hand side splits into a prior-matching term, a reconstruction term, and one Kullback-Leibler divergence per interior step,

$$\underbrace{D_{\mathrm{KL}}(q(x_K \mid x_0)\,\|\,p(x_K))}_{\text{prior matching}} + \sum_{k=2}^{K} \underbrace{D_{\mathrm{KL}}(q(x_{k-1} \mid x_k, x_0)\,\|\,p_\theta(x_{k-1} \mid x_k))}_{L_{k-1}} - \underbrace{\log p_\theta(x_0 \mid x_1)}_{\text{reconstruction}}.$$

The first term has no parameters, because both the forward chain and the prior are fixed, and we train the network through the interior terms.

We can fix the reverse variance at a schedule value v_k, so that $p_\theta(x_{k-1} \mid x_k) = \mathcal{N}(\mu_\theta(x_k, k), v_k I)$. The symbol v_k is kept separate from $\sigma_k = \sqrt{1-\bar{\alpha}_k}$, which is the corruption noise level of Section 3.12 and belongs to the forward process. Two Gaussians with the same covariance have a divergence that depends only on the gap between their means,

$$L_{k-1} = \frac{1}{2v_k}\big\|\tilde{\mu}_k(x_k, x_0) - \mu_\theta(x_k, k)\big\|^2.$$

"The interior loss term L_{k-1} equals the squared distance between the exact reverse posterior mean $\tilde{\mu}_k$ and the model mean μ_θ, divided by twice the fixed reverse variance v_k."

We calculate the exact mean using the sampled noise ε and the model mean using the predicted noise. Subtracting the two means gives

$$\tilde{\mu}_k - \mu_\theta = \frac{1}{\sqrt{\alpha_k}} \cdot \frac{\beta_k}{\sqrt{1-\bar{\alpha}_k}}\big(\varepsilon_\theta(x_k, k) - \varepsilon\big).$$

Squaring this and dividing by $2v_k$ turns the divergence into a weighted noise regression,

$$L_{k-1} = \frac{\beta_k^2}{2v_k\,\alpha_k\,(1-\bar{\alpha}_k)}\big\|\varepsilon - \varepsilon_\theta(x_k, k)\big\|^2.$$

Every factor in front of the squared error is a schedule constant. If we set it to one and draw the step index uniformly in place of summing over it, we are left with the loss that DDPM actually trains,

$$\boxed{\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{k,X_0,\varepsilon}\Big[\,\big\|\varepsilon - \varepsilon_\theta(X_k,k)\big\|^2\Big], \qquad X_k = \sqrt{\bar\alpha_k}\,X_0 + \sqrt{1-\bar\alpha_k}\,\varepsilon.}$$

"The simplified loss $\mathcal{L}_{\text{simple}}$ averages the squared distance between the sampled Gaussian noise ε and predicted noise $\varepsilon_\theta(X_k, k)$, over uniformly drawn step indices k, clean samples X_0, and Gaussian draws used to form the noisy states X_k."

Dropping the weights changes each step's contribution to training without changing the pointwise minimizer. At every fixed (x_k, k), the conditional mean $\mathbb{E}[\varepsilon \mid X_k = x_k]$ minimizes the squared error. Multiplying the error by a positive constant leaves that minimizer unchanged. The weighted and unweighted objectives therefore have the same optimal predictor in the ideal regression limit.

The weights that get dropped are not uniform across the chain. With $v_k = \tilde\beta_k$ and the thousand-step linear schedule, the weight at $k = 500$ is

$$\frac{\beta_k^2}{2\tilde\beta_k\alpha_k(1-\bar\alpha_k)} = \frac{(0.01004)^2}{2\times 0.01003\times 0.98996\times 0.92141} = 0.005508,$$

while on the two-step hand schedule with $\beta_2 = 0.2$, $\alpha_2 = 0.8$, $\bar\alpha_2 = 0.64$ and $\tilde\beta_2 = 1/9$, the same expression is

$$\frac{0.04}{2\times\frac{1}{9}\times 0.8\times 0.36} = 0.625.$$

Weighting by these constants concentrates the gradient on the few steps with the largest coefficients. The unweighted loss spreads it evenly over noise levels, which is the behavior that trains well in practice.

3.26 Drawing the reverse step

The network estimates the center of the reverse transition, and the sampler has to draw a state around that center. With the posterior variance of Section 3.24.2 as the spread, one reverse step is

$$x_{k-1} = \mu_\theta(x_k,k) + \sqrt{\tilde\beta_k}\;z_k, \qquad z_k \sim \mathcal{N}(0,I),$$

with no noise added at the final step $k = 1$.

"The sampled previous state x_{k-1} equals the learned reverse mean $\mu_\theta(x_k, k)$, evaluated at the current state x_k and step k, plus a fresh standard Gaussian draw z_k times the reverse standard deviation $\sqrt{\tilde\beta_k}$."

At $k = 1$, we omit the noise and return the mean as the generated sample. This avoids adding a final random increment to the clean state.

3.26.1 One complete reverse transition

Let's run one step of the sampler with numbers. We work on the two-step schedule $\beta_1 = \beta_2 = 0.2$ and start from the noisy state $x_2 = 7.4$ built in Section 3.24. Suppose the network predicts $\varepsilon_\theta = -1$ at $k = 2$. The coefficients are $\alpha_2 = 0.8$, $\bar{\alpha}_2 = 0.64$, $\sqrt{1-\bar{\alpha}_2} = 0.6$ and $\bar{\alpha}_1 = 0.8$.

The reverse mean follows from the noise form,

$$\mu_\theta = \frac{1}{\sqrt{0.8}}\left(7.4 - \frac{0.2}{0.6} \times (-1)\right) = \frac{7.4 + 0.3333}{0.8944} = \frac{7.7333}{0.8944} = 8.646.$$

The clean-state form of the same mean is a useful check. The clean estimate is $\hat{x}_0 = (7.4+0.6)/0.8 = 10$, and the two interpolation weights are

$$\frac{\sqrt{\bar{\alpha}_1}\,\beta_2}{1-\bar{\alpha}_2} = \frac{0.8944 \times 0.2}{0.36} = 0.4969, \qquad \frac{\sqrt{\alpha_2}\,(1-\bar{\alpha}_1)}{1-\bar{\alpha}_2} = \frac{0.8944 \times 0.2}{0.36} = 0.4969,$$

so that $\tilde{\mu}_2 = 0.4969 \times 10 + 0.4969 \times 7.4 = 0.4969 \times 17.4 = 8.646$. The two routes agree, in line with the substitution of Section 3.24. At this step the two weights happen to be equal, so the reverse mean is halfway between the clean estimate and the current state.

The reverse variance is

$$\tilde{\beta}_2 = \frac{\beta_2(1-\bar{\alpha}_1)}{1-\bar{\alpha}_2} = \frac{0.2 \times 0.2}{0.36} = \frac{0.04}{0.36} = \frac{1}{9} = 0.1111,$$

with standard deviation $\sqrt{1/9} = 1/3$. A Gaussian draw $z_2 = 0.3$ then adds $0.3 \times 0.3333 = 0.1$, and

$$x_1 = 8.646 + 0.1 = 8.746.$$

The state has moved from 7.4 toward the clean value 10. One more reverse step remains, and at $k = 1$ the sampler returns the mean with no added noise.

3.27 The shrinking-step limit

We connect the discrete chain to the continuous equation by setting $\beta_k = \beta(t_k)h$ for a small interval h. The DDPM transition then becomes the Euler-Maruyama update for the variance-preserving process.

3.28 Implementing DDPM

We implement DDPM in PyTorch [Paszke et al., 2019] and train on MNIST digits. During training, we draw a step index, corrupt the batch, predict the noise, and minimize the squared error. To generate images, we start from Gaussian noise and apply the reverse step K times.

3.28.1 The schedule tensors

We fix all training and sampling coefficients before training, so we store the schedule once in tensors indexed by the step.

```python
import math

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2
from torchvision.utils import save_image

NUM_STEPS = 1000  # Diffusion steps K.
BETA_START = 1e-4  # Noise variance added at step 1.
BETA_END = 0.02  # Noise variance added at step K.

betas = torch.cat([
    torch.zeros(1),  # Placeholder so that index k holds beta_k.
    torch.linspace(BETA_START, BETA_END, NUM_STEPS),  # beta_1 ... beta_K.
])
alphas = 1.0 - betas  # One-step signal retention alpha_k.
alphas_cumprod = torch.cumprod(alphas, dim=0)  # Cumulative retention.
alphas_cumprod_prev = torch.cat([
    torch.ones(1),  # abar_0 = 1.
    alphas_cumprod[:-1],  # Index k now holds abar_{k-1}.
])
posterior_variance = (
    betas * (1.0 - alphas_cumprod_prev) / (1.0 - alphas_cumprod)
)  # Reverse variance beta-tilde_k.
```

We store the tables from the `ForwardDiffusion` class of Section 3.12.2 at module level and add two tables for the reverse step. We prefix a zero to the linear schedule so that the Python index equals the diffusion step. The entry `betas[k]` contains β_k. The entry `betas[0]` is a placeholder for the clean state before any steps. With $\beta_0 = 0$ the cumulative product starts at $\bar{\alpha}_0 = 1$, which is the right value for the clean state. The shifted tensor `alphas_cumprod_prev` stores $\bar{\alpha}_{k-1}$ at index k, so we can compute the posterior variance for the whole schedule in one vectorized line. Entry 0 of `posterior_variance` is undefined, because both its numerator and its denominator vanish there, and we never access it. Training draws $k \geq 1$, and sampling runs from $k = K$ down to $k = 1$.

3.28.2 Forward corruption

The helper `extract` reads one schedule value per batch item. We use these values in `q_sample` to apply the Gaussian transition.

```python
def extract(schedule, k, shape):
    # One schedule value per item, reshaped to broadcast over C, H, W.
    out = schedule.to(k.device).gather(0, k)
    return out.reshape(k.shape[0], *((1,) * (len(shape) - 1)))
```

The helper `extract` selects the schedule value for each image's own step index with `gather`, then appends singleton axes so the result broadcasts across channel, height and width. For a batch of 128 images the gathered tensor has shape [128] and the reshaped tensor has shape $[128, 1, 1, 1]$, which multiplies against $[128, 1, 28, 28]$.

```python
def q_sample(x_0, k, noise=None):
    if noise is None:
```

```
        noise = torch.randn_like(x_0)  # Mean 0, covariance I, shape of x_0.
    abar = extract(alphas_cumprod, k, x_0.shape)
    a_k = abar.sqrt()  # Signal coefficient a_k.
    sigma_k = (1.0 - abar).sqrt()  # Noise standard deviation sigma_k.
    return a_k * x_0 + sigma_k * noise, noise  # x_k and the exact noise.
```

The function `q_sample` implements the corruption equation of Section 3.24. It returns the same pair (x_k, ε) as the method in Section 3.12.2. An additional optional argument allows us to reuse an existing noise tensor. `randn_like` draws independent standard Gaussian noise with the shape, dtype and device of the clean batch. The two coefficients come from the same cumulative retention, so we simulate no intermediate steps and can place a batch at any noise level in one operation. Because the function returns the noise alongside the noisy image, the training target is available, and the loss regresses against the exact draw used to build `x_k`.

3.28.3 Sinusoidal step embedding

The scalar step index has to become a vector before the network can use it. Dividing by K and feeding a single number into a linear layer, as the three-block network of Section 3.20 does, works at small scale. A sinusoidal encoding separates nearby steps much better.

```
def timestep_embedding(k, dim):
    half = dim // 2  # Half the coordinates sine, half cosine.
    freqs = torch.exp(
        -math.log(10000.0)
        * torch.arange(half, device=k.device).float() / half
    )
    args = k.float()[:, None] * freqs[None, :]  # [B, half].
    return torch.cat([torch.sin(args), torch.cos(args)], dim=1)  # [B, dim].


def num_groups(channels):
    for g in (8, 4, 2, 1):  # Largest valid GroupNorm group count.
        if channels % g == 0:
            return g
```

The frequencies are geometrically spaced, $\omega_i = 10000^{-i/(\dim/2)}$ for $i = 0, \ldots, \dim/2 - 1$, and from each one we take a sine and a cosine of $\omega_i k$. With $\dim = 128$ the fastest coordinate has $\omega_0 = 1$ and a period of $2\pi \approx 6.28$ steps, and the slowest has $\omega_{63} = 1.155 \times 10^{-4}$ and a period of about 54 410 steps. Over the whole thousand-step schedule the slowest coordinate traverses only $1000 \times \omega_{63}/2\pi = 0.0184$ of a cycle, so it varies monotonically and separates early steps from late ones. The fastest coordinate separates neighbors: between $k = 50$ and $k = 51$ its sine changes by 0.933, while the slowest coordinate's sine changes by 1.15×10^{-4}. The embedding therefore encodes both a coarse position in the schedule and a fine distinction between adjacent steps.

The helper `num_groups` returns the largest count among 8, 4, 2 and 1 that divides the channel count. Every `GroupNorm` is therefore valid for the widths used in the architecture.

3.28.4 The time-conditioned residual block

At every resolution, we use a block with two convolutions, add the step embedding between them, and include a skip path around both.

```python
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(num_groups(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels,
                               kernel_size=3, padding=1)
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.norm2 = nn.GroupNorm(num_groups(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels,
                               kernel_size=3, padding=1)
        self.skip = (
            nn.Identity() if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, kernel_size=1)
        )
        self.activation = nn.SiLU()

    def forward(self, x, t_emb):
        h = self.conv1(self.activation(self.norm1(x)))  # GN, SiLU, Conv_1.
        h = h + self.time_proj(t_emb)[:, :, None, None]  # Offset per channel.
        h = self.conv2(self.activation(self.norm2(h)))
        return h + self.skip(x)  # Residual connection.
```

The block applies normalization, activation and convolution, injects the step, repeats the three operations, and adds the input back. Written out, the first stage computes

$$h = \mathrm{Conv}_1(\mathrm{SiLU}(\mathrm{GN}(x))), \qquad h \leftarrow h + Ae_k,$$

with e_k the step embedding and A the weight matrix of `time_proj`, which maps $e_k \in \mathbb{R}^{d_{\text{time}}}$ to one value per output channel. The indexing `[:, :, None, None]` adds the two spatial axes, so the same channel offset lands at every pixel. The block then normalizes, activates and convolves a second time and adds the skip path,

$$\text{output} = \mathrm{Conv}_2(\mathrm{SiLU}(\mathrm{GN}(h))) + \mathrm{skip}(x).$$

The skip is the identity when the channel counts agree and a 1×1 convolution when they differ, which is the only role of that convolution. If we keep a direct path from input to output, a block can learn a small correction to its input. At low noise levels a denoiser has to make exactly that kind of correction, because the right answer is close to the input already.

3.28.5 The U-Net

We can assemble the encoder, the bottleneck and the decoder from those blocks, with the channel count rising as the spatial resolution falls.

```python
class UNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_embed_dim=128):
        super().__init__()
        self.time_embed_dim = time_embed_dim
        self.time_mlp = nn.Sequential(  # Learned representation of step k.
            nn.Linear(time_embed_dim, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim),
        )
        c1 = base_channels  # 64 channels at 28 x 28.
```

```
        c2 = base_channels * 2  # 128 channels at 14 x 14.
        c3 = base_channels * 4  # 256 channels at 7 x 7.

        self.in_conv = nn.Conv2d(in_channels, c1, kernel_size=3, padding=1)
        self.down1 = ResidualBlock(c1, c1, time_embed_dim)
        self.pool1 = nn.Conv2d(c1, c1, kernel_size=3, stride=2, padding=1)
        self.down2 = ResidualBlock(c1, c2, time_embed_dim)
        self.pool2 = nn.Conv2d(c2, c2, kernel_size=3, stride=2, padding=1)
        self.mid = ResidualBlock(c2, c3, time_embed_dim)
        self.up1 = nn.ConvTranspose2d(c3, c2, kernel_size=4,
                                      stride=2, padding=1)
        self.dec1 = ResidualBlock(c2 + c2, c2, time_embed_dim)
        self.up2 = nn.ConvTranspose2d(c2, c1, kernel_size=4,
                                      stride=2, padding=1)
        self.dec2 = ResidualBlock(c1 + c1, c1, time_embed_dim)
        self.out_conv = nn.Conv2d(c1, in_channels, kernel_size=1)
```

The step embedding passes through a two-layer perceptron before reaching any block, so the network learns its own representation of the schedule position on top of the raw sinusoids. The three widths are 64, 128 and 256, one per resolution.

The encoder runs a residual block at 28×28 and then halves the resolution with a stride-two convolution, $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$. A second residual block widens to 128 channels at 14×14, and a second stride-two convolution reaches 7×7. The bottleneck block processes the lowest resolution with 256 channels, where each unit has a receptive field covering a large part of the digit. The decoder reverses the two reductions with transposed convolutions of kernel size 4, stride 2 and padding 1, each of which doubles the spatial size exactly, $7 \to 14 \to 28$. Each decoder block has twice as many input channels as its own width, because the encoder features at the matching resolution are concatenated onto the upsampled ones. The final 1×1 convolution maps the 64 channels back to the single image channel.

3.28.6 The forward pass

One batch runs down through the encoder and back up through the decoder, with the encoder features held aside for the two skip connections.

```
    def forward(self, x, k):
        t_emb = self.time_mlp(
            timestep_embedding(k, self.time_embed_dim)
        )  # Learned representation of diffusion step k.
        h = self.in_conv(x)
        h1 = self.down1(h, t_emb)  # 28 x 28 encoder features.
        h = self.pool1(h1)  # 28 -> 14.
        h2 = self.down2(h, t_emb)  # 14 x 14 encoder features.
        h = self.pool2(h2)  # 14 -> 7.
        h = self.mid(h, t_emb)  # Compressed representation.
        h = self.up1(h)  # 7 -> 14.
        h = self.dec1(torch.cat([h, h2], dim=1), t_emb)  # Join 14 x 14 skip.
        h = self.up2(h)  # 14 -> 28.
        h = self.dec2(torch.cat([h, h1], dim=1), t_emb)  # Join 28 x 28 skip.
        return self.out_conv(h)  # [B, 1, 28, 28] predicted noise.
```

We retain the encoder outputs `h1` and `h2` in local variables and concatenate them with the decoder features at matching resolutions. These skip connections preserve spatial detail that downsampling

would discard, supporting a noise prediction at full resolution. The output is $\varepsilon_\theta(x_k, k) \in \mathbb{R}^{B\times 1\times 28\times 28}$, the same shape as the input batch, and the loss of Section 3.25 compares it against the sampled noise.

3.28.7 One reverse step and the sampler

At each reverse step, we evaluate the network once, compute the Gaussian mean, and draw a sample. Starting from pure noise, we repeat this step K times.

```
@torch.no_grad()
def p_sample(model, x_k, k):
    eps_theta = model(x_k, k)  # Predicted noise at step k.
    beta_k = extract(betas, k, x_k.shape)
    alpha_k = extract(alphas, k, x_k.shape)
    abar_k = extract(alphas_cumprod, k, x_k.shape)
    mean = (
        x_k - beta_k / (1.0 - abar_k).sqrt() * eps_theta
    ) / alpha_k.sqrt()  # Reverse mean mu_theta.
    var = extract(posterior_variance, k, x_k.shape)  # beta-tilde_k.
    z = torch.randn_like(x_k)  # Fresh Gaussian draw.
    keep = (k > 1).float().reshape(-1, *((1,) * (x_k.dim() - 1)))
    return mean + keep * var.sqrt() * z  # No noise at the final step.


@torch.no_grad()
def sample(model, num_images, device):
    x = torch.randn(num_images, 1, 28, 28, device=device)  # X_K ~ N(0, I).
    for step in range(NUM_STEPS, 0, -1):  # k = K down to k = 1.
        k = torch.full((num_images,), step, device=device, dtype=torch.long)
        x = p_sample(model, x, k)
    return x  # Generated images in the training range [-1, 1].
```

The decorator `@torch.no_grad()` switches off gradient storage, because sampling never updates θ and the activation graph would otherwise be held for the whole chain. The first line is the only network call in the step. The three schedule lookups select β_k, α_k and $\bar{\alpha}_k$ for every image in the batch. We use these coefficients to compute `mean` from the boxed reverse mean in Section 3.24. The mask `keep` is 1 wherever $k > 1$ and 0 at $k = 1$, so the last step returns the mean with no added randomness.

The sampler draws the starting state from the standard Gaussian that the forward process converges to, then applies the reverse step once per index from K down to 1. Each iteration builds a tensor of identical step indices, one per image, because every image in the batch is at the same point in the chain. A thousand steps means a thousand network evaluations per batch of samples, which dominates the cost of generation.

3.28.8 Training on MNIST

We set up the data, network and optimizer once. For each batch of each epoch, the loop runs the four lines of the regression.

```
torch.manual_seed(SEED)  # Reproducible initialization and sampling.

transform = v2.Compose([
    v2.ToImage(),  # Channel-first image: [1, 28, 28].
```

```
    v2.ToDtype(torch.float32, scale=True),  # Pixels 0-255 -> floats 0-1.
    v2.Normalize(mean=(0.5,), std=(0.5,)),  # Then [0, 1] -> [-1, 1].
])
dataset = datasets.MNIST(root=DATA_DIR, train=True, download=True,
                         transform=transform)
loader = DataLoader(dataset, batch_size=128, shuffle=True)

model = UNet(in_channels=1, base_channels=64,
             time_embed_dim=128).to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=2e-4)

for epoch in range(EPOCHS):  # 20 passes over the training set.
    for x_0, _ in loader:  # Labels unused: generation is unconditional.
        x_0 = x_0.to(DEVICE)
        k = torch.randint(1, NUM_STEPS + 1, (x_0.shape[0],), device=DEVICE)
        x_k, noise = q_sample(x_0, k)  # Corrupt directly to step k.
        eps_theta = model(x_k, k)  # Predict the noise.
        loss = F.mse_loss(eps_theta, noise)  # L_simple on this batch.
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet.pt")
```

The normalization maps each pixel through $x_{\text{normalized}} = (x - 0.5)/0.5$, so $[0, 1]$ becomes $[-1, 1]$. The terminal corruption distribution is a standard Gaussian centered at zero. Centering the data at zero with unit scale therefore gives comparable scales at both ends of the path, as in the flow-matching implementation in Chapter 2.

We draw a step index uniformly from 1 to K for every image independently, following Section 3.25. Each mini-batch of 128 images therefore covers many noise levels without favoring any level. The call to `q_sample` returns the corrupted image and the exact noise. We compare the prediction against that noise with `mse_loss`, then update θ with Adam. Because we sampled the noise ourselves, we know the exact regression target.

3.28.9 Generating

With the weights trained, we can draw a grid of digits in three lines.

```
images = sample(model, num_images=16, device=DEVICE)  # Reverse all K steps.
images = (images.clamp(-1.0, 1.0) + 1.0) / 2.0  # [-1, 1] -> [0, 1].
save_image(images, OUTPUT_DIR / "samples.png", nrow=4)  # 4-by-4 grid.
```

Sixteen Gaussian images go into the reverse chain and sixteen digits come out after $K = 1000$ reverse steps. An imperfect denoiser can produce small excursions outside $[-1, 1]$. To display the images, we undo the training normalization and clamp these excursions to the range $[0, 1]$.

The convolutional network and the 28×28 shape are specific to images. We can apply the same schedule, corruption, loss, and reverse step to any continuous tensor, including peptide embeddings.

3.29 AMP-Diffusion end to end

We can generate antimicrobial peptides by combining the latent space and denoiser of Section 3.21 with the DDPM schedule, reverse step, and training objective. We make four changes to the image model.

- The clean state X_0 becomes the ESM-2 latent $Z_0 \in \mathbb{R}^{B\times L\times D}$.
- A Transformer replaces the U-Net, because the state is a sequence of residue embeddings.
- The network output is the clean latent $\hat{Z}_0$ in place of the noise.
- The ESM-2 language-model head converts a generated latent back into amino acids.

Peptides also have different lengths. We pad their tensors to a common length L and use an attention mask M to mark positions containing residues.

3.29.1 Setup and masked corruption

We specify the encoder checkpoint and tensor shapes in the configuration. The corruption function also receives a mask marking the positions that contain residues.

```
from transformers import AutoModel, AutoModelForMaskedLM, AutoTokenizer

MODEL_NAME = "facebook/esm2_t33_650M_UR50D"  # Pretrained ESM-2 encoder.
EMBED_DIM = 1280  # ESM-2 residue embedding width D.
MAX_LEN = 64  # Token positions L per padded peptide.
NUM_LAYERS = 6  # Transformer layers in the denoiser.
BATCH_SIZE = 32
LEARNING_RATE = 1e-4


def q_sample_latent(z0, k, mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0)  # [B, L, D] standard Gaussian.
    abar = alphas_cumprod.to(z0.device)[k].reshape(-1, 1, 1)
    z_k = abar.sqrt() * z0 + (1.0 - abar).sqrt() * noise
    return z_k * mask.unsqueeze(-1), noise  # Padding stays exactly zero.
```

We reuse the image model's schedule tensors. The reshape to $[B, 1, 1]$ broadcasts one coefficient per sequence across all L positions and D coordinates. The image version used $[B, 1, 1, 1]$ to broadcast across channel, height and width. The corruption itself is the same equation,

$$Z_k = \left(\sqrt{\bar{\alpha}_k}\, Z_0 + \sqrt{1-\bar{\alpha}_k}\, \varepsilon\right) \odot M,$$

"The noisy latent Z_k equals the clean latent Z_0 times the square root of the cumulative retention $\bar{\alpha}_k$, plus standard Gaussian noise ε times the square root of $1-\bar{\alpha}_k$, all multiplied elementwise by the attention mask M. This mask equals one at valid residues and zero at padding."

The mask prevents us from adding Gaussian noise to padding and training the denoiser on those positions.

3.29.2 The latent denoiser and its loss

The denoiser is the transformer of Section 3.21 extended with three pieces: the sinusoidal step embedding, a learned position embedding, and the padding mask.

```
class LatentDenoiser(nn.Module):
    def __init__(self, embed_dim=EMBED_DIM, num_layers=NUM_LAYERS,
                 num_heads=8, max_len=MAX_LEN):
        super().__init__()
        self.embed_dim = embed_dim
        self.time_mlp = nn.Sequential(  # Same sinusoidal step embedding.
            nn.Linear(embed_dim, embed_dim),
            nn.SiLU(),
            nn.Linear(embed_dim, embed_dim),
        )
        self.pos_embed = nn.Embedding(max_len, embed_dim)  # Residue index.
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=4 * embed_dim, batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers)
        self.out = nn.Sequential(nn.LayerNorm(embed_dim),
                                 nn.Linear(embed_dim, embed_dim))

    def forward(self, z_k, k, attention_mask):
        t_emb = self.time_mlp(timestep_embedding(k, self.embed_dim))
        positions = torch.arange(z_k.shape[1], device=z_k.device)
        h = (z_k
             + t_emb.unsqueeze(1)  # Shared across positions.
             + self.pos_embed(positions).unsqueeze(0))  # Shared across batch.
        h = self.transformer(h, src_key_padding_mask=~attention_mask.bool())
        return self.out(h)  # Predicted clean latent Z0-hat.
```

We use the image model's sinusoidal step embedding. We also add a learned position embedding because order matters in a peptide and self-attention by itself is permutation-equivariant. This denoiser uses six layers, compared with four in Section 3.21.

We add the three inputs before passing them through the Transformer,

$$h = Z_k + e_k \mathbf{1}^\top + P,$$

with e_k the step embedding broadcast across the L positions and P the table of position embeddings broadcast across the B sequences. The argument `src_key_padding_mask` follows the opposite convention to `attention_mask`, and a `True` entry means ignore that position. The code converts the mask to Boolean and negates it with `~`. Every valid residue then attends to every other valid residue. Padded positions receive no attention weight.

```
def latent_loss(z0_hat, z0, mask):
    error = (z0_hat - z0) ** 2  # [B, L, D] squared error.
    error = error * mask.unsqueeze(-1)  # Zero out padded positions.
    return error.sum() / (mask.sum() * z0.shape[-1])  # Valid tokens only.
```

The loss is the masked mean squared error between the predicted clean latent and the target,

$$\mathcal{L}(\theta) = \frac{1}{D \sum_{b,\ell} M_{b\ell}} \sum_{b,\ell,d} M_{b\ell} \left(\hat{Z}_{0,b\ell d} - Z_{0,b\ell d}\right)^2.$$

"The loss $\mathcal{L}(\theta)$ sums squared errors between the predicted clean latent $\hat{Z}_0$ and target latent Z_0 over sequences b, token positions ℓ, and embedding coordinates d, weighted by the validity mask $M_{b\ell}$. We divide by the embedding width D times the number of valid tokens, giving the mean error over valid coordinates."

The denominator $D \sum_{b\ell} M_{b\ell}$ counts only valid coordinates. Padding contributes neither error nor weight, including for short peptides padded to $L = 64$.

3.29.3 Reverse latent sampling

We derived the reverse step for a noise-predicting network, while this network predicts the clean latent. Rearranging the corruption equation converts one into the other,

$$\hat{\varepsilon} = \frac{Z_k - \sqrt{\bar{\alpha}_k}\,\hat{Z}_0}{\sqrt{1-\bar{\alpha}_k}}.$$

"The inferred noise $\hat{\varepsilon}$ equals the noisy latent Z_k minus the predicted clean latent $\hat{Z}_0$ times the square root of the cumulative retention $\bar{\alpha}_k$, divided by the accumulated noise standard deviation $\sqrt{1-\bar{\alpha}_k}$."

A clean-state prediction therefore suffices for the DDPM sampler, and the two parametrizations are interchangeable at any interior noise level. At the level with $\sqrt{\bar{\alpha}_k} = 0.8$ and $\sqrt{1-\bar{\alpha}_k} = 0.6$, the chapter's noisy sample is $z_k = 7.4$ and a noise-predicting network that outputs -0.8 implies the clean estimate $\hat{z}_0 = (7.4 + 0.6 \times 0.8)/0.8 = 7.88/0.8 = 9.85$. Running the conversion the other way recovers the noise,

$$\hat{\varepsilon} = \frac{7.4 - 0.8 \times 9.85}{0.6} = \frac{7.4 - 7.88}{0.6} = \frac{-0.48}{0.6} = -0.8,$$

which is the prediction we started from.

```
@torch.no_grad()
def p_sample_latent(model, z_k, k, mask):
    z0_hat = model(z_k, k, mask)  # Predict the clean latent.
    abar = alphas_cumprod.to(z_k.device)[k].reshape(-1, 1, 1)
    eps_hat = (z_k - abar.sqrt() * z0_hat) / (1.0 - abar).sqrt()
    beta_k = betas.to(z_k.device)[k].reshape(-1, 1, 1)
    alpha_k = alphas.to(z_k.device)[k].reshape(-1, 1, 1)
    mean = (z_k - beta_k / (1.0 - abar).sqrt() * eps_hat) / alpha_k.sqrt()
    var = posterior_variance.to(z_k.device)[k].reshape(-1, 1, 1)
    xi = torch.randn_like(z_k)
    keep = (k > 1).float().reshape(-1, 1, 1)
    return (mean + keep * var.sqrt() * xi) * mask.unsqueeze(-1)


@torch.no_grad()
def sample_latents(model, num_sequences, length, device):
    mask = torch.ones(num_sequences, length, device=device)
    z = torch.randn(num_sequences, length, EMBED_DIM, device=device)
    for step in range(NUM_STEPS, 0, -1):  # Z_K down to Z_0.
        k = torch.full((num_sequences,), step, device=device,
                       dtype=torch.long)
        z = p_sample_latent(model, z, k, mask)
    return z
```

After converting to $\hat{\varepsilon}$, we calculate the reverse mean with the image sampler's formula. The final multiplication by the mask holds padded positions at zero through the whole chain, so the noise added at each step never leaks into them. The outer loop starts from a standard Gaussian latent Z_K and runs the reverse step once per index down to 1, exactly as the image sampler does.

For a worked step at $k = 500$ we take the latent coordinate of Section 3.21. There the clean coordinate $z_0 = 1.2$ was corrupted to $z_{500} = 0.7204$. Suppose the denoiser predicts $\hat{z}_0 = 1.05$. The inferred noise is

$$\hat{\varepsilon} = \frac{0.7204 - 0.2803 \times 1.05}{0.9599} = \frac{0.7204 - 0.2943}{0.9599} = 0.4438,$$

against the draw 0.4 that actually made the sample. The reverse mean is then

$$\mu_\theta = \frac{1}{\sqrt{0.98996}} \left(0.7204 - \frac{0.01004}{0.9599} \times 0.4438 \right) = 0.7193,$$

and with $\tilde{\beta}_{500} = 0.01003$ the sampler adds a Gaussian draw of standard deviation 0.1002. One step out of a thousand moves the coordinate by about 0.001 on average. The random displacement is about 0.1. We need this long reverse chain to accumulate the denoising movement from the Gaussian to a latent that the language-model head can decode.

3.29.4 Encoding and decoding

We use an encoder to map peptides into latent space and a decoder to convert generated latents back into residues.

```
@torch.no_grad()
def encode(sequences, tokenizer, esm, device):
    batch = tokenizer(sequences, padding="max_length", truncation=True,
                      max_length=MAX_LEN, return_tensors="pt").to(device)
    output = esm(**batch)  # Frozen ESM-2 forward pass.
    return output.last_hidden_state, batch["attention_mask"]
```

The tokenizer converts amino acid strings into ESM-2 token identifiers and produces the attention mask in the same call. The frozen encoder then returns `last_hidden_state`, the clean latent $Z_0 \in \mathbb{R}^{B \times L \times D}$ with one contextual D-dimensional embedding per residue.

```
AA = list("ACDEFGHIKLMNPQRSTVWY")  # The 20 standard amino acids.


@torch.no_grad()
def decode(z0_standardized, z_mean, z_std, lm_head, tokenizer):
    z0 = z0_standardized * z_std + z_mean  # Undo standardization.
    logits = lm_head(z0)  # Amino-acid logits at each position.
    keep = torch.tensor(
        [tokenizer.convert_tokens_to_ids(a) for a in AA],
        device=logits.device,
    )
    ids = logits[..., keep].argmax(dim=-1)  # Highest-scoring residue.
    return ["".join(AA[i] for i in row) for row in ids]
```

To decode a generated latent, we first undo the standardization,

$$Z_0 = Z_0^{\text{std}} \odot \sigma_z + \mu_z,$$

with μ_z and σ_z the coordinatewise training mean and standard deviation of the latent coordinates, written separately from the schedule's σ_k. The ESM-2 language-model head, which was trained to score amino acids from exactly these embeddings, maps each position to logits over the full token vocabulary. We restrict the columns to the twenty standard amino acids, removing the scores for special tokens. Taking the argmax at each position then gives one residue per token. Joining those produces a peptide sequence.

3.29.5 Standardization, training and generation

We can load and freeze the encoder first, and standardize the latents it produces coordinate by coordinate before any diffusion runs on them.

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
esm = AutoModel.from_pretrained(MODEL_NAME).to(DEVICE)
lm_head = AutoModelForMaskedLM.from_pretrained(MODEL_NAME).lm_head.to(DEVICE)
for parameter in esm.parameters():
    parameter.requires_grad = False  # Train only the diffusion denoiser.

z0_all, mask_all = encode(train_sequences, tokenizer, esm, DEVICE)
valid = mask_all.bool().unsqueeze(-1)  # [B, L, 1].
count = valid.sum()
z_mean = (z0_all * valid).sum(dim=(0, 1)) / count  # Per-coordinate mean.
z_std = (
    ((z0_all - z_mean) ** 2 * valid).sum(dim=(0, 1)) / count
).sqrt()  # Per-coordinate standard deviation.
z0_all = (z0_all - z_mean) / z_std  # Standardized clean latents.
```

ESM-2 remains frozen while the denoiser trains, so the latent space and regression targets stay fixed. Standardizing each of the 1280 coordinates to mean near zero and variance near one lets the corruption end at a standard Gaussian. Raw ESM-2 embeddings have their own coordinate means and scales. Corrupting these embeddings toward $\mathcal{N}(0, I)$ would have to remove a systematic offset as well as their structure. At $k = K$, this leaves a mismatch between the terminal corrupted density and the Gaussian used to initialize the sampler.

```
denoiser = LatentDenoiser().to(DEVICE)
optimizer = torch.optim.Adam(denoiser.parameters(), lr=LEARNING_RATE)

for epoch in range(EPOCHS):
    for z0, mask in latent_loader:
        z0, mask = z0.to(DEVICE), mask.to(DEVICE)
        k = torch.randint(1, NUM_STEPS + 1, (z0.shape[0],), device=DEVICE)
        z_k, _ = q_sample_latent(z0, k, mask)  # Corrupt the clean latent.
        z0_hat = denoiser(z_k, k, mask)  # Predict Z0-hat.
        loss = latent_loss(z0_hat, z0, mask)  # Valid residues only.
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

z0_generated = sample_latents(denoiser, num_sequences=64,
                              length=MAX_LEN, device=DEVICE)
peptides = decode(z0_generated, z_mean, z_std, lm_head, tokenizer)
```

We adapt the MNIST training loop by replacing images with latents and `q_sample` with masked corruption. The loss measures the masked clean-latent error in place of the noise error. We draw

one diffusion step per sequence, corrupt the clean latent to that step, let the denoiser predict $\hat{Z}_0$, and compare over valid residues only.

Generation takes two calls. The sampler produces standardized clean latents from Gaussian noise, and the decoder undoes the standardization and reads out amino acids,

$$Z_K \sim \mathcal{N}(0, I) \;\longrightarrow\; Z_0 \;\longrightarrow\; \text{peptide sequence.}$$

Peptides generated this way are physicochemically consistent with known antimicrobial peptides [Chen et al., 2023].

We can train and sample diffusion models in pixel space and in protein language model latent space, adding fresh noise at each reverse step. A deterministic velocity field can also produce the same marginals p_t without adding noise along the trajectory.

3.30 Continuous limit of the DDPM step

The DDPM sampler uses Gaussian updates on a finite grid indexed by k. We can recover the continuous-time variance-preserving dynamics by relating the per-step variance β_k to the continuous noise rate. This also connects the coded sampler to a deterministic field with the same marginals.

Let one discrete step span a small continuous interval Δt, so that step k falls at time $t_k = k\,\Delta t$. The per-step noise variance is the continuous noise rate evaluated at that time, multiplied by the length of the interval,

$$\beta_k \approx \beta(t_k)\,\Delta t.$$

"The discrete noise variance β_k at step k approximately equals the continuous noise rate $\beta(t_k)$, measured in inverse time at time t_k, multiplied by the step interval Δt. Shorter intervals therefore receive less noise variance."

The DDPM forward update [Ho et al., 2020] scales the previous state by $\sqrt{\alpha_k} = \sqrt{1-\beta_k}$ and adds an independent Gaussian draw,

$$X_k = \sqrt{1-\beta_k}\,X_{k-1} + \sqrt{\beta_k}\,\varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, I).$$

For a small nonnegative number u, the square root expands as

$$\sqrt{1-u} = 1 - \tfrac{1}{2}u - \tfrac{1}{8}u^2 - \cdots,$$

so if we keep the first-order term alone the error is $u^2/8$. With $u = \beta(t_k)\,\Delta t$ the signal coefficient becomes

$$\sqrt{1-\beta_k} \approx 1 - \tfrac{1}{2}\beta(t_k)\,\Delta t.$$

Substituting both small-step approximations into the forward update,

$$X_k \approx \Big(1 - \tfrac{1}{2}\beta(t_k)\,\Delta t\Big)X_{k-1} + \sqrt{\beta(t_k)\,\Delta t}\;\varepsilon_k.$$

Subtracting X_{k-1} from both sides isolates the displacement produced by one step,

$$X_k - X_{k-1} \approx -\tfrac{1}{2}\beta(t_k)\,X_{k-1}\,\Delta t + \sqrt{\beta(t_k)}\,\sqrt{\Delta t}\;\varepsilon_k.$$

"The state displacement $X_k - X_{k-1}$ approximately equals negative one half of the noise rate $\beta(t_k)$ times the previous state X_{k-1} and interval Δt, plus a standard Gaussian draw ε_k times the square roots of $\beta(t_k)$ and Δt. The drift displacement scales with Δt, and the random displacement scales with $\sqrt{\Delta t}$."

This is the Euler-Maruyama update for the variance-preserving stochastic differential equation,

$$dX_t = -\tfrac{1}{2}\beta(t)\, X_t\, dt + \sqrt{\beta(t)}\, dW_t,$$

with drift $f(x,t) = -\frac{1}{2}\beta(t)x$ and diffusion coefficient $g(t) = \sqrt{\beta(t)}$. The coded chain and the continuous equation therefore describe one process. In the continuous equation there are infinitely many infinitesimal corruption updates, and the DDPM chain groups them into K finite Gaussian transitions. On the reverse side the same correspondence holds: the learned reverse transition of the DDPM chain is the Euler-Maruyama discretization of the reverse-time stochastic differential equation [Anderson, 1982, Song et al., 2021].

3.30.1 Accuracy of the small-step approximation

We hold the noise rate at $\beta(t) = 2$ and split the unit interval into $K = 200$ steps, so $\Delta t = 0.005$ and $\beta_k = 2 \times 0.005 = 0.01$. The exact signal coefficient is

$$\sqrt{1 - 0.01} = 0.994\,9874,$$

and the first-order approximation is $1 - \frac{1}{2}(0.01) = 0.995$. The two differ by 1.256×10^{-5}, which matches the next term of the expansion, $u^2/8 = 1.25 \times 10^{-5}$, to three figures. Over the whole chain of 200 steps the accumulated discrepancy stays of order $Ku^2/8 = 2.5 \times 10^{-3}$, which shrinks as $1/K$ when $\beta(t)\Delta t$ is held to $\beta(t)/K$.

The deterministic part of the displacement is the current state multiplied by $-\frac{1}{2}(2)(0.005) = -0.005$, and the noise standard deviation is $\sqrt{2 \times 0.005} = 0.1$. One step of this chain moves the center by half a percent and scatters the sample by a tenth. These sizes follow the drift's Δt scaling and the Brownian increments' $\sqrt{\Delta t}$ scaling.

At $t = 0.5$, the linear rate in Section 3.11.5 is $\beta(0.5) = 0.1 + 19.9 \times 0.5 = 10.05$. For our thousand-step schedule, $\Delta t = 1/1000$, giving a predicted per-step variance $\beta_k = 0.01005$. The call to `torch.linspace(1e-4, 0.02, 1000)` stores the close value 0.010040 at $k = 500$. The expansion of the signal coefficient is as close, with $\sqrt{1 - 0.01005} = 0.994962$ against $1 - \frac{1}{2}(0.01005) = 0.994975$, a gap of 1.3×10^{-5} in a coefficient of order one. The DDPM chain of Section 3.24 is therefore the variance-preserving equation integrated on a grid, and it converges to the continuous description as the grid refines.

Both the chain and the equation it approximates are driven by Brownian increments, so both assign a different trajectory to the same starting point on every run. We can construct deterministic trajectories through the same densities by comparing their evolution equations.

3.31 Stochastic and deterministic density evolution

In Chapter 2 we moved a density with a velocity field alone. The deterministic field $v_t(x)$ specifies a particle's instantaneous velocity at state x and time t. The density then obeys the continuity

equation,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (p_t(x)\, v_t(x)).$$

"The time derivative of the density $p_t(x)$ equals the negative divergence of the probability flux $p_t(x)v_t(x)$, the density times the velocity $v_t(x)$ at position x and time t. Probability accumulates where this flux converges and decreases where it spreads."

A diffusion moves individual states through a drift and a random increment,

$$dX_t = f(X_t, t)\, dt + g(t)\, dW_t,$$

and its density obeys the Fokker-Planck equation derived earlier in this chapter,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (f(x,t)\, p_t(x)) + \tfrac{1}{2} g(t)^2\, \nabla^2 p_t(x).$$

"The time derivative of the density $p_t(x)$ equals the negative divergence of the drift flux $f(x,t)p_t(x)$, plus one half of the squared diffusion coefficient $g(t)^2$ times the density Laplacian $\nabla^2 p_t(x)$. These terms describe drift transport and Brownian spreading."

Diffusion adds a Laplacian term to the divergence that describes deterministic transport. We can rewrite this Laplacian as the divergence of a flux and combine both terms into one divergence. The coefficient of $p_t(x)$ inside it then gives a deterministic velocity field with the same density evolution.

3.32 The probability-flow ODE

We can express the Laplacian through the score,

$$s_t(x) = \nabla_x \log p_t(x).$$

Differentiating the logarithm by the chain rule relates the gradient of the density to the score,

$$\nabla p_t(x) = p_t(x)\, \nabla_x \log p_t(x) = p_t(x)\, s_t(x).$$

"The density gradient $\nabla p_t(x)$ equals the density $p_t(x)$ times the score $s_t(x)$, so multiplying the log-density gradient by the density recovers the direction and magnitude of the density gradient."

The Laplacian is the divergence of the gradient, so substituting the identity once gives

$$\nabla^2 p_t(x) = \nabla \cdot (\nabla p_t(x)) = \nabla \cdot (p_t(x)\, s_t(x)).$$

Multiplying by $\frac{1}{2} g(t)^2$, which does not depend on x and therefore passes inside the divergence,

$$\tfrac{1}{2} g(t)^2\, \nabla^2 p_t(x) = \nabla \cdot \Big(\tfrac{1}{2} g(t)^2\, p_t(x)\, s_t(x)\Big).$$

Next, we put that back into the Fokker-Planck equation,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (f(x,t)\, p_t(x)) + \nabla \cdot \Big(\tfrac{1}{2} g(t)^2\, p_t(x)\, s_t(x)\Big).$$

Both terms are now divergences of a flux proportional to $p_t(x)$, so we collect them into one,

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot \Big(p_t(x) \Big[f(x,t) - \tfrac{1}{2} g(t)^2 \, s_t(x) \Big] \Big).$$

Comparing with the continuity equation term by term, we can identify the bracket as a velocity field,

$$\boxed{v_t(x) = f(x,t) - \tfrac{1}{2} \, g(t)^2 \, \nabla_x \log p_t(x)}$$

"The probability-flow velocity $v_t(x)$ equals the forward drift $f(x,t)$ minus one half of the squared diffusion coefficient $g(t)^2$ times the marginal score $\nabla_x \log p_t(x)$. This deterministic score correction reproduces the spreading in the density equation."

We follow this field by solving the *probability-flow ODE* [Song et al., 2021],

$$\frac{dX_t}{dt} = f(X_t, t) - \tfrac{1}{2} \, g(t)^2 \, \nabla_x \log p_t(X_t).$$

The symbols in the construction are:

- $f(x,t)$ is the drift of the forward stochastic differential equation, a vector field on $\mathbb{R}^d$ with units of state per unit time;
- $g(t)$ is the diffusion coefficient, a scalar function of time alone, and $g(t)^2$ is the rate at which variance accumulates;
- $s_t(x) = \nabla_x \log p_t(x)$ is the score of the marginal density at time t, pointing toward states of higher probability;
- $v_t(x)$ is the probability-flow velocity, the deterministic motion assigned to a particle at x at time t.

Start the stochastic differential equation and the probability-flow ODE from the same distribution at the same time. With the exact score, both produce the same density p_t at every later time. Their individual trajectories stay different. The stochastic process injects fresh randomness at every instant, so a single starting point spreads into a cloud of possible paths. The ODE assigns one curve to each starting point, and two such curves never cross. Spreading in the stochastic picture comes from the Brownian increments; in the deterministic picture it comes from neighboring trajectories moving apart along the score.

3.33 Probability flow for Brownian marginals

We can take the forward process to be Brownian motion alone,

$$dX_t = dW_t, \qquad X_0 \sim \mathcal{N}(0,1),$$

so that $f(x,t) = 0$ and $g(t) = 1$. Brownian motion adds variance at unit rate, and the starting variance is one, so the marginal at time t is Gaussian with variance $1 + t$,

$$p_t = \mathcal{N}(0, 1 + t).$$

The score of a centered Gaussian with variance v is $-x/v$, as computed in Section 3.14.1, so here

$$s_t(x) = \nabla_x \log p_t(x) = -\frac{x}{1+t}.$$

Substituting $f(x,t) = 0$, $g(t) = 1$ and this score into the probability-flow velocity,

$$v_t(x) = 0 - \tfrac{1}{2}(1)^2 \left(-\frac{x}{1+t}\right) = \frac{x}{2(1+t)}.$$

"The probability-flow velocity $v_t(x)$ equals the position x divided by twice $1+t$, where t is elapsed time. Each state moves away from the origin at a speed proportional to its distance, with the proportionality factor decreasing as the cloud widens."

The resulting ODE separates,

$$\frac{dX_t}{dt} = \frac{X_t}{2(1+t)} \quad \Longrightarrow \quad \frac{d}{dt}\log X_t = \frac{1}{2(1+t)},$$

and integrating both sides from 0 to t, we get $\log X_t - \log X_0 = \frac{1}{2}\log(1+t)$, so

$$X_t = X_0\sqrt{1+t}.$$

"The state X_t equals the initial state X_0 times the square root of one plus elapsed time t. This deterministic rescaling expands the line at the rate needed to match the variance accumulated by Brownian motion."

3.33.1 Marginal equivalence

We can put $X_0 = 2$ and run to $t = 3$. The map then produces

$$X_3 = 2\sqrt{1+3} = 2 \times 2 = 4,$$

a single number with no randomness in it. If $X_0 \sim \mathcal{N}(0,1)$, then $X_3 = 2X_0$ is Gaussian with mean 0 and variance $2^2 \times 1 = 4$,

$$X_3 \sim \mathcal{N}(0,4).$$

Brownian motion from the same start reaches variance $1+3=4$ at the same time, so it also produces $\mathcal{N}(0,4)$. Individual paths differ completely, and the two populations are identical at every fixed time (Figure 3.6).

3.33.2 Probability inside a moving interval

Because the deterministic map is monotone, it maps intervals to intervals, and their probability goes with them. The transport is $X_3 = 2X_0$, so

$$\mathbb{P}(-2 \le X_3 \le 2) = \mathbb{P}(-1 \le X_0 \le 1) = 0.6827.$$

"The probability that the transported state X_3 lies between -2 and 2 equals the probability that the initial state X_0 lies between -1 and 1, about 68.27 percent. The map doubles each initial position, carrying exactly the samples in the initial interval into the transported interval."

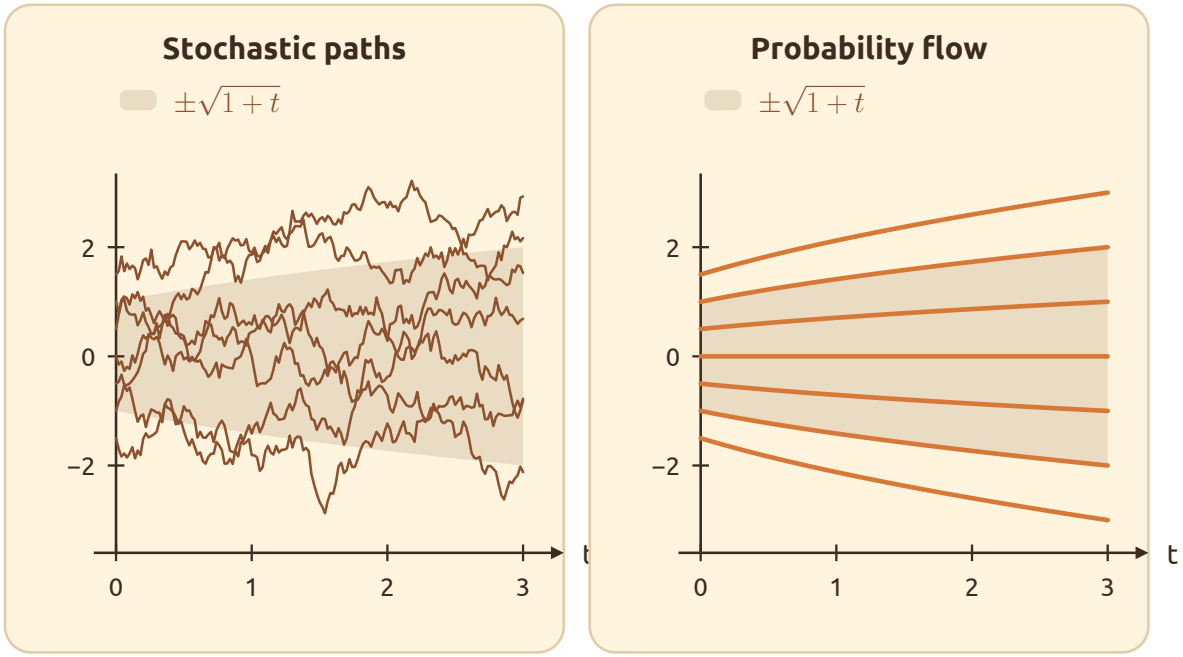


Figure 3.6: The same density evolution carried two ways from $X_0 \sim \mathcal{N}(0,1)$ under $dX_t = dW_t$. Left, seven Brownian sample paths, each built from independent increments, which cross one another and wander in and out of the shaded band. Right, the same seven starting points followed under the probability-flow velocity $v_t(x) = x/(2(1+t))$, giving the smooth curves $X_t = X_0\sqrt{1+t}$, which never cross and preserve the order of the starting points. The shaded band in both panels is $\pm\sqrt{1+t}$, one standard deviation of the shared marginal, equal to 1 at $t = 0$ and 2 at $t = 3$. The horizontal axis is time and the vertical axis is the one-dimensional state.

The interval widens from 2 to 4 units while holding the same probability, so its probability per unit length halves. If we check that against the densities directly, with $x = 0$ at the center,

$$p_3(2x) = \frac{p_0(x)}{2}, \qquad p_0(0) = \frac{1}{\sqrt{2\pi}} = 0.3989, \qquad p_3(0) = \frac{0.3989}{2} = 0.1995.$$

The peak falls from 0.3989 to 0.1995, which is the density of $\mathcal{N}(0,4)$ at the origin, $1/\sqrt{2\pi \times 4} = 0.1995$. A map that doubles every length in one dimension halves every density, and that factor of two is the one-dimensional Jacobian of Section 1.24.

For pure Brownian motion, the probability-flow velocity contains only the score correction. With a nonzero drift, both terms contribute.

3.34 Probability flow for the variance-preserving path

The forward process behind DDPM is the variance-preserving equation of this chapter,

$$dX_t = -\tfrac{1}{2}\beta(t)\, X_t\, dt + \sqrt{\beta(t)}\, dW_t,$$

so its drift and diffusion coefficient are

$$f(x,t) = -\tfrac{1}{2}\beta(t)\, x, \qquad g(t) = \sqrt{\beta(t)}, \qquad g(t)^2 = \beta(t).$$

If we substitute both into the probability-flow velocity, we get

$$v_t(x) = -\tfrac{1}{2}\beta(t)\, x - \tfrac{1}{2}\beta(t)\, \nabla_x \log p_t(x) = -\tfrac{1}{2}\beta(t)\Big[x + \nabla_x \log p_t(x)\Big].$$

"The variance-preserving probability-flow velocity $v_t(x)$ equals negative one half of the noise rate $\beta(t)$ times the sum of the current position x and marginal score $\nabla_x \log p_t(x)$. The same rate scales the contraction and score correction."

3.34.1 A consistency check at the terminal distribution

We build the variance-preserving path so that the terminal distribution is a standard Gaussian, $p_T = \mathcal{N}(0, I)$, whose score is

$$\nabla_x \log p_T(x) = -x.$$

Putting that score into the velocity,

$$v_T(x) = -\tfrac{1}{2}\beta(T)\big[x + (-x)\big] = 0.$$

The velocity vanishes everywhere once the distribution has reached the standard Gaussian. Contraction pulls each state toward the origin at rate $\frac{1}{2}\beta(t)x$. The score correction pushes it outward at the same rate. The two cancel exactly, as stationarity requires. We reach the same conclusion from the stochastic process by a different route, with the Brownian increments balancing the contraction only in distribution.

3.34.2 The velocity through the noise prediction

The denoising network predicts the Gaussian noise added to the clean state. For the variance-preserving corruption $X_t = a_t X_0 + \sigma_t \varepsilon$, we convert that prediction into a score using the identity from Section 3.17,

$$s_\theta(x, t) = -\frac{\varepsilon_\theta(x, t)}{\sigma_t},$$

"The learned score $s_\theta(x, t)$ equals the negative predicted noise $\varepsilon_\theta(x, t)$ divided by the corruption standard deviation σ_t. Flipping the sign and dividing by this known scale converts the noise estimate into a direction of increasing log density."

Here a_t is the surviving signal and σ_t the standard deviation of the accumulated noise at time t, the two coefficients of Section 3.11.

Substituting the score model into the variance-preserving probability-flow velocity,

$$v_t(x) = -\tfrac{1}{2}\beta(t)\, x - \tfrac{1}{2}\beta(t)\left(-\frac{\varepsilon_\theta(x, t)}{\sigma_t}\right),$$

and collecting the two terms,

$$\boxed{v_t(x) = -\tfrac{1}{2}\beta(t)\, x + \frac{\beta(t)}{2\,\sigma_t}\, \varepsilon_\theta(x, t)}$$

"The probability-flow velocity $v_t(x)$ equals negative one half of the noise rate $\beta(t)$ times the state x, plus the predicted noise $\varepsilon_\theta(x,t)$ times $\beta(t)$ divided by twice the corruption standard deviation σ_t. The first term contracts toward the origin, and the second supplies the denoising correction."

The DDPM U-Net and the latent transformer of this chapter already output ε_θ, and the same weights serve the deterministic sampler. The sampling procedure changes, becoming numerical integration of an ordinary differential equation.

3.34.3 Deterministic generation

Generation starts from the same prior as the stochastic sampler,

$$X_T \sim \mathcal{N}(0, I),$$

and integrates the probability-flow ODE backward from T to 0. For a small positive step Δt, one explicit Euler update is

$$X_{t-\Delta t} = X_t - \Delta t\, v_t(X_t),$$

where the minus sign appears because v_t is a forward-time velocity and generation runs toward decreasing t. The procedure is four operations per step:

- evaluate $\varepsilon_\theta(X_t, t)$, or equivalently $s_\theta(X_t, t)$, at the current state;
- build the probability-flow velocity $v_t(X_t)$ from it;
- move the state backward by one solver step;
- return X_0 after the last step as the generated sample.

Once we draw X_T, the rest of the trajectory is fixed. Different draws of X_T produce different samples, so all of the model's randomness comes from the initial draw. We can replace the Euler step with any solver for ordinary differential equations.

3.35 Deterministic backward-step example

At the current time t, we take a one-dimensional state and a network output

$$X_t = 1.2, \qquad \beta(t) = 2, \qquad \sigma_t = 0.5, \qquad \varepsilon_\theta(X_t, t) = 0.8.$$

The score follows from the noise prediction,

$$s_\theta(X_t, t) = -\frac{0.8}{0.5} = -1.6.$$

The state is at 1.2 and the score points back toward the origin with magnitude 1.6, as a Gaussian-like density centered near zero requires. We can now assemble the velocity from the two terms,

$$v_t(X_t) = \underbrace{-\tfrac{1}{2}(2)(1.2)}_{\text{contraction} = -1.2} + \underbrace{\frac{2}{2(0.5)}(0.8)}_{\text{denoising correction} = +1.6} = 0.4.$$

We obtain the same number from the collected form, $v_t = -\frac{1}{2}(2)[1.2 + (-1.6)] = -(-0.4) = 0.4$. Contraction alone would pull the state toward zero at speed 1.2. The outward score correction is

larger, giving a net forward-time velocity of 0.4 away from the origin. This outward motion increases the spread during noising.

Generation runs the other way. With a positive step $\Delta t = 0.1$,

$$X_{t-\Delta t} = 1.2 - (0.1)(0.4) = 1.16.$$

"The estimated earlier state $X_{t-\Delta t}$ equals the current state X_t minus the positive step size Δt times the forward-time velocity $v_t(X_t)$. This backward step moves the sample from 1.2 to 1.16."

We re-evaluate the network at 1.16 and at time $t - \Delta t$ before the next step, and the only error in the update is the finite Euler step.

3.35.1 The same state under the stochastic sampler

Next, we can run the identical state through the reverse-time stochastic differential equation of this chapter for comparison. In its reverse drift the score appears with the full squared diffusion coefficient,

$$f(X_t, t) - g(t)^2\, s_\theta(X_t, t) = -1.2 - (2)(-1.6) = -1.2 + 3.2 = 2.0,$$

and one backward Euler-Maruyama step with the same $\Delta t = 0.1$ is

$$X_{t-\Delta t} = 1.2 - (0.1)(2.0) + \sqrt{\beta(t)\,\Delta t}\; z = 1.0 + 0.4472\, z, \qquad z \sim \mathcal{N}(0, 1).$$

The stochastic step moves the center five times as far, to 1.0 against the deterministic 1.16. It then adds a random displacement with standard deviation 0.4472, exceeding the whole deterministic move. Both updates reproduce the same family of marginals through different combinations of drift and noise. The stochastic sampler over-corrects along the score, and the Brownian increment restores the spread. The deterministic velocity contains exactly half that correction and produces the spread by separating neighboring trajectories.

Table 3.2: One backward step from $X_t = 1.2$ with $\beta(t) = 2$, $\sigma_t = 0.5$, $\varepsilon_\theta = 0.8$ and $\Delta t = 0.1$. The score coefficient is the only structural difference between the two samplers, and the stochastic sampler compensates for the larger coefficient with the Brownian increment it retains.

	Reverse-time SDE	Probability-flow ODE
Score coefficient in the drift	$g(t)^2 = 2$	$\frac{1}{2}g(t)^2 = 1$
Drift at $X_t = 1.2$	2.0	0.4
Noise term	$\sqrt{\beta(t)\Delta t}\, z = 0.4472\, z$	none
Next state	$1.0 + 0.4472\, z$	1.16
Trajectory from a fixed X_T	a different one each run	one, fixed
Marginal at every t	p_t	p_t

A deterministic invertible map transforms the prior density into the data density as it moves samples. The velocity field determines the rate of this density change.

3.36 The density along a deterministic trajectory

Along a trajectory of the probability-flow ODE, the continuity equation becomes an ordinary differential equation for the log density evaluated at the moving state,

$$\frac{d}{dt}\log p_t(X_t) = -\nabla \cdot v_t(X_t).$$

"The rate of change of the log density $\log p_t(X_t)$ along the moving state X_t equals the negative divergence $\nabla \cdot v_t(X_t)$ of the probability-flow velocity. Density decreases along a trajectory at the rate of local expansion."

Integrating from data time 0 to prior time T and rearranging,

$$\log p_0(X_0) = \log p_T(X_T) + \int_0^T \nabla \cdot v_t(X_t)\, dt.$$

"The data log density $\log p_0(X_0)$ equals the prior log density $\log p_T(X_T)$ at the transported endpoint X_T, plus the divergence $\nabla \cdot v_t(X_t)$ of the probability-flow velocity accumulated along the trajectory from time zero to time T."

This is the continuous-time form of the change-of-variables correction of Section 1.24, with the accumulated divergence playing the role of the log-determinant of the Jacobian. With an exact score, we can calculate the likelihood of any data point. We integrate the ODE forward and accumulate the divergence along the path [Song et al., 2021]. Forming the full Jacobian in high dimensions is impractical, and the trace is instead estimated from a few random directions, which costs one vector-Jacobian product per direction.

3.36.1 Accumulated divergence on the expanding Gaussian

We return to the Brownian example of Section 3.33, where the velocity is $v_t(x) = x/[2(1+t)]$. In one dimension the divergence is the spatial derivative,

$$\nabla \cdot v_t = \frac{\partial}{\partial x}\frac{x}{2(1+t)} = \frac{1}{2(1+t)}.$$

It is positive at every time, so nearby trajectories separate and the density along each trajectory falls. The divergence here depends only on time, so the integral along a trajectory is a plain integral in t,

$$\int_0^3 \frac{dt}{2(1+t)} = \tfrac{1}{2}\Big[\log(1+t)\Big]_0^3 = \tfrac{1}{2}\log 4 = \log 2 = 0.6931.$$

We take the sample $X_0 = 2$, which the flow moves to $X_3 = 4$. The final density is $\mathcal{N}(0, 4)$, and evaluating its logarithm at the transported sample,

$$\log p_3(4) = -\tfrac{1}{2}\log(2\pi \times 4) - \frac{4^2}{2 \times 4} = -1.6121 - 2 = -3.6121.$$

If we add the accumulated divergence, we recover the starting log density,

$$\log p_0(2) = -3.6121 + 0.6931 = -2.9189.$$

Checking against the standard Gaussian directly, $-\frac{1}{2}\log(2\pi) - \frac{1}{2}(2)^2 = -0.9189 - 2 = -2.9189$, so the two agree to four decimal places. The correction $\log 2$ is exactly the twofold expansion the flow applied between the two times, in agreement with the halving of the peak density computed in Section 3.33.2.

3.36.2 Common structure of the two samplers

One learned network now supports two generative procedures. The forward corruption

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

and the probability-flow ODE

$$\frac{dX_t}{dt} = f(X_t, t) - \tfrac{1}{2}g(t)^2\,\nabla_x \log p_t(X_t)$$

produce the same marginals p_t whenever the score is exact. The reverse-time stochastic differential equation generates by stochastic denoising, the probability-flow ODE generates by deterministic transport, and both read the same s_θ or ε_θ. For the variance-preserving path used by DDPM the complete chain of substitutions is

$$f(x,t) = -\tfrac{1}{2}\beta(t)x, \qquad g(t)^2 = \beta(t), \qquad s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}, \qquad v_t(x) = -\tfrac{1}{2}\beta(t)x + \frac{\beta(t)}{2\sigma_t}\varepsilon_\theta(x,t).$$

Both samplers use the unconditional score to move toward regions supported by the training distribution. To generate a peptide with a particular function, a molecule with a target property, or a sample that satisfies a structural requirement, we need a term that changes which part of that distribution the trajectory reaches.

3.37 Conditioning the score

Let c describe a desired condition. It may be an image class, a protein family, a binding target, a measured activity, or a biological phenotype. The unconditional model describes the density of states at time t,

$$p_t(x),$$

and for controlled generation we need the density of states that are both plausible under the corruption process and compatible with c,

$$p_t(x \mid c).$$

Both dynamics of Section 3.36.2 require a score, so we need $\nabla_x \log p_t(x \mid c)$.

Bayes' rule relates the two densities,

$$p_t(x \mid c) = \frac{p_t(x)\; p_t(c \mid x)}{p_t(c)}.$$

"The conditional density $p_t(x \mid c)$ equals the unconditional state density $p_t(x)$ times the probability $p_t(c \mid x)$ of condition c at state x and time t, divided by the overall condition probability $p_t(c)$."

Taking logarithms turns the product into a sum,

$$\log p_t(x \mid c) = \log p_t(x) + \log p_t(c \mid x) - \log p_t(c),$$

and when we differentiate with respect to x the last term drops out, because $p_t(c)$ is a number at each time and does not vary with the state,

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x).$$

Writing $s_t(x) = \nabla_x \log p_t(x)$ for the unconditional score and $s_t(x \mid c)$ for the conditional score,

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x).$$

"The conditional score $s_t(x \mid c)$ equals the unconditional score $s_t(x)$ plus the gradient $\nabla_x \log p_t(c \mid x)$ of the condition log probability. The first term points toward states supported by the data, and the second points toward states more compatible with condition c."

The generative network produces $s_\theta(x, t)$. To calculate the conditional correction, we use a separate classifier $p_\phi(c \mid x_t, t)$ with parameters ϕ. We differentiate its log-probability with respect to its input. The trajectory passes through noisy states, so we train the classifier on the same corrupted samples as the denoiser [Dhariwal and Nichol, 2021]. The unconditional score keeps the trajectory within the region occupied by the training data. The classifier gradient selects which part of that region it approaches.

3.38 The guidance scale and the tilted target

The classifier gradient can produce a correction that is too weak or too strong. We can control its magnitude with a scalar guidance strength,

$$s_{\text{guided}}(x, t) = s_\theta(x, t) + \gamma \, \nabla_x \log p_\phi(c \mid x, t).$$

"The guided score $s_{\text{guided}}(x, t)$ equals the unconditional score prediction $s_\theta(x, t)$, plus the classifier log-probability gradient $\nabla_x \log p_\phi(c \mid x, t)$ times the nonnegative guidance scale γ, which we choose at sampling time."

The scale has three reference points. At $\gamma = 0$ generation is unconditional. At $\gamma = 1$, and with both models exact, the expression is the ordinary conditional score derived above. At $\gamma > 1$ the condition receives more weight than Bayes' rule assigns it.

For $\gamma > 1$, we obtain a sharper density than $p_t(x \mid c)$. Integrating the guided score $s_t(x) + \gamma \nabla_x \log p_t(c \mid x)$ gives

$$p_t^\gamma(x) \;\propto\; p_t(x)\, p_t(c \mid x)^\gamma.$$

"The guided density $p_t^\gamma(x)$ is proportional to the unconditional density $p_t(x)$ times the condition probability $p_t(c \mid x)$ raised to the guidance power γ. Increasing γ concentrates more probability on states with high classifier probability."

Larger γ concentrates probability around the states the classifier scores well, which narrows the sample diversity and places weight on whatever errors the classifier makes. A classifier that assigns high probability to a region the data never covers will pull trajectories there, and the unconditional term opposes movement into those regions.

3.38.1 A worked guided score

Let's take a two-dimensional state and read the two directions at one point. The unconditional score and the classifier gradient are

$$s_\theta(x_t) = (-1.0,\ -0.1), \qquad \nabla_x \log p_\phi(c \mid x_t) = (1.3,\ 1.1).$$

The unconditional direction points left and slightly down, back toward the bulk of the data density. The classifier gradient points up and to the right, toward the states compatible with c. At $\gamma = 1$ the two add,

$$s_\theta + \nabla_x \log p_\phi = (-1.0 + 1.3,\ -0.1 + 1.1) = (0.3,\ 1.0).$$

The sum has turned almost entirely upward, at 73.3 degrees from the positive horizontal axis, with length 1.0440. The classifier gradient cancels most of the unconditional term's first coordinate. The remaining direction points toward the condition. The leftover 0.3 retains some of the pull toward the data. Doubling the scale,

$$s_\theta + 2\,\nabla_x \log p_\phi = (-1.0,\ -0.1) + 2\,(1.3,\ 1.1) = (1.6,\ 2.1),$$

with length 2.6401 at 52.7 degrees. The guided direction is now two and a half times as long as the unconditional score and tilted toward the condition (Figure 3.7). Every further unit of γ adds the same vector $(1.3, 1.1)$. The guided direction therefore converges to the classifier gradient as γ grows. The unconditional score remains a fixed offset whose relative size diminishes.

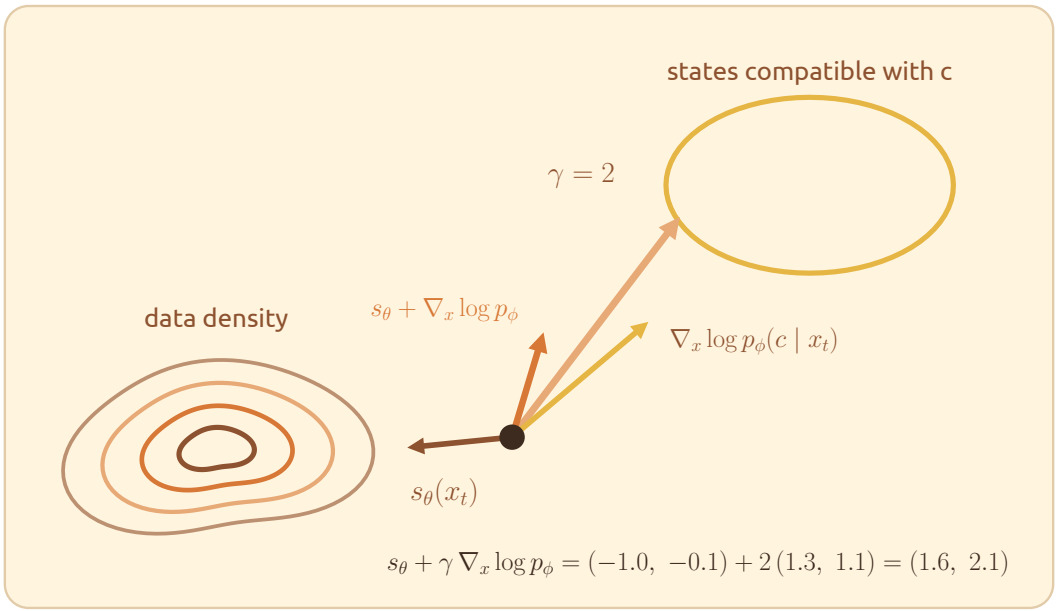


Figure 3.7: The three directions at one state x_t, drawn as arrows from the black dot. The dark arrow pointing left is the unconditional score $s_\theta(x_t) = (-1.0, -0.1)$, aimed at the contours of the data density on the left. The gold arrow is the classifier gradient $\nabla_x \log p_\phi(c \mid x_t) = (1.3, 1.1)$, aimed at the gold region of states compatible with c. The short orange arrow is their sum at $\gamma = 1$, equal to $(0.3, 1.0)$, and the long pale arrow is the guided direction at $\gamma = 2$, equal to $(1.6, 2.1)$. Arrow lengths are proportional to vector magnitudes.

3.39 Reward guidance

We can use a classifier to specify categorical conditions. For continuous properties such as binding affinity, solubility, thermal stability, or measured activity, we define a reward $R_t(x)$. This scalar function is differentiable in the state x at time t. Larger values indicate more desirable states.

To turn a reward into a distribution we give high-reward states more probability while keeping the model's own density as the base. Exponential reweighting does this,

$$p_t^{(R)}(x) = \frac{p_t(x)\, \exp\left[\eta\, R_t(x)\right]}{Z_t}, \qquad Z_t = \int p_t(x)\, \exp\left[\eta\, R_t(x)\right] dx.$$

"The reward-guided density $p_t^{(R)}(x)$ equals the base density $p_t(x)$ times the exponential of the guidance strength η multiplied by the reward $R_t(x)$, divided by the normalizer Z_t. The strength η controls how reward differences change probability ratios."

Taking the logarithm and differentiating with respect to x,

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta\, \nabla_x R_t(x),$$

because $\log Z_t$ depends on time alone. Writing the result as a score,

$$s_t^{(R)}(x) = s_t(x) + \eta\, \nabla_x R_t(x).$$

"The reward-guided score $s_t^{(R)}(x)$ equals the unconditional score $s_t(x)$ plus the reward gradient $\nabla_x R_t(x)$ times the guidance strength η. Because only the reward gradient enters, adding a constant to the reward has no effect."

Classifier guidance is the special case in which the reward is the classifier log-probability,

$$R_t(x) = \log p_\phi(c \mid x, t), \qquad \eta = \gamma,$$

which recovers the guided score of Section 3.38 term for term. One construction therefore covers categorical conditions and continuous properties, and the only requirement on R_t is that we can take gradients through it.

3.40 Guidance inside the generative dynamics

We can guide either sampler by substituting the guided score for the unconditional score. The unconditional reverse-time stochastic differential equation is

$$dX_t = \Big[f(X_t, t) - g(t)^2\, s_\theta(X_t, t)\Big] dt + g(t)\, d\bar{W}_t,$$

integrated from T down to 0, where $\bar{W}_t$ is a Brownian motion for the reversed clock. The score is the only term in it that refers to where probability is concentrated, so we insert the guidance there,

$$dX_t = \Big[f(X_t, t) - g(t)^2 \big(s_\theta(X_t, t) + \gamma\, \nabla_x \log p_\phi(c \mid X_t, t)\big)\Big] dt + g(t)\, d\bar{W}_t.$$

"The guided state increment dX_t equals the forward drift $f(X_t, t)$ minus the squared noise amplitude $g(t)^2$ times the sum of the learned score $s_\theta(X_t, t)$ and the classifier gradient $\nabla_x \log p_\phi(c \mid X_t, t)$ scaled by γ, all multiplied by the negative time increment dt. We then add $g(t)$ times the reverse Brownian increment $d\bar{W}_t$."

Generation runs from T toward 0, so dt is negative. Multiplying the negative classifier term in the drift by this increment gives a positive classifier-gradient update. The learned score denoises toward the data distribution. The classifier gradient selects the destination within it. Brownian increments preserve variation between trajectories.

3.40.1 Guidance in the noise parameterization

Our coded sampler uses the noise prediction ε_θ, which relates to the score through $s_\theta = -\varepsilon_\theta/\sigma_t$. Requiring the same relation to hold for the guided quantities,

$$-\frac{\varepsilon_{\text{guided}}(x, t, c)}{\sigma_t} = -\frac{\varepsilon_\theta(x, t)}{\sigma_t} + \gamma \nabla_x \log p_\phi(c \mid x, t),$$

and multiplying both sides by $-\sigma_t$,

$$\boxed{\varepsilon_{\text{guided}}(x, t, c) = \varepsilon_\theta(x, t) - \gamma \sigma_t \nabla_x \log p_\phi(c \mid x, t)}$$

"The guided noise prediction $\varepsilon_{\text{guided}}$ equals the original prediction $\varepsilon_\theta(x, t)$ minus the classifier gradient $\nabla_x \log p_\phi(c \mid x, t)$ times the noise standard deviation σ_t and guidance scale γ. The factor σ_t makes the correction largest at high noise and zero when corruption vanishes."

At each step, predict ε_θ from the current state. Evaluate the classifier on that state and differentiate its log-probability with respect to the state. Use this gradient to form $\varepsilon_{\text{guided}}$, then substitute it for ε_θ in the reverse-mean formula. Guidance thus changes the neural prediction supplied to the same sampler.

3.40.2 Guidance in the deterministic sampler

We can make the same substitution in the probability-flow velocity,

$$v_t^c(x) = f(x, t) - \tfrac{1}{2} g(t)^2 \Big[s_\theta(x, t) + \gamma \nabla_x \log p_\phi(c \mid x, t) \Big],$$

and if we separate out the unconditional velocity $v_t(x) = f(x, t) - \frac{1}{2} g(t)^2 s_\theta(x, t)$, we get

$$v_t^c(x) = v_t(x) - \tfrac{1}{2} \gamma\, g(t)^2 \nabla_x \log p_\phi(c \mid x, t).$$

"The guided probability-flow velocity $v_t^c(x)$ equals the unconditional velocity $v_t(x)$ minus one half of the guidance scale γ times the squared noise amplitude $g(t)^2$ and classifier gradient $\nabla_x \log p_\phi(c \mid x, t)$. The factor one half converts the score correction into a deterministic velocity correction."

In the reverse stochastic differential equation the guidance term appears with the full $g(t)^2$, because that equation retains the Brownian increment. In the probability-flow ODE it appears with half of that, because the deterministic velocity reproduces the corresponding density evolution without any noise. Mixing the two coefficients produces a sampler whose marginals match neither.

3.41 Guided backward-step example

We continue from the state of Section 3.35, where $X_t = 1.2$, $\beta(t) = 2$, $\sigma_t = 0.5$, $\varepsilon_\theta = 0.8$ and the unguided velocity came out at $v_t = 0.4$. Next, we can add a classifier whose log-probability increases to the right at this state, with

$$\nabla_x \log p_\phi(c \mid X_t, t) = 0.6, \qquad \gamma = 2, \qquad g(t)^2 = \beta(t) = 2.$$

The guided velocity subtracts half of $\gamma g(t)^2$ times that gradient,

$$v_t^c(X_t) = 0.4 - \tfrac{1}{2}(2)(2)(0.6) = 0.4 - 1.2 = -0.8.$$

The forward-time velocity has reversed sign. Taking the backward step with $\Delta t = 0.1$,

$$X_{t-\Delta t} = 1.2 - (0.1)(-0.8) = 1.2 + 0.08 = 1.28.$$

Against the unguided result of 1.16, guidance has displaced the sample by 0.12 in the positive direction, which is $\Delta t \times \frac{1}{2}\gamma g(t)^2 \times 0.6$. The negative classifier term inside the forward-time velocity becomes positive movement along the classifier gradient once the step is taken backward.

The same state comes out of the noise parameterization. The guided noise prediction is

$$\varepsilon_{\text{guided}} = 0.8 - (2)(0.5)(0.6) = 0.8 - 0.6 = 0.2,$$

whose implied score is $-0.2/0.5 = -0.4$. Feeding that score into the collected velocity,

$$v_t^c = -\tfrac{1}{2}(2)\big[1.2 + (-0.4)\big] = -0.8,$$

which is the number computed above. The two routes agree because they express the same substitution in different variables. A numerical disagreement in an implementation indicates a missing σ_t or a missing factor of one half.

3.42 Guiding a flow-matching model

Diffusion guidance starts from an unconditional score and adds a correction. We can train a flow-matching model of Chapter 2 with the condition attached from the start, because the condition accompanies the endpoint data during training. The network then learns

$$v_\theta(X_t, t, c),$$

the velocity of the conditional probability path, and sampling integrates

$$\frac{dX_t}{dt} = v_\theta(X_t, t, c)$$

forward from the prior at $t = 0$ to data at $t = 1$ [Lipman et al., 2023]. The condition may be a class label, a property vector, a target embedding, a protein representation, or any other input available at training time. The two clocks run in opposite directions. Under our diffusion convention, the data is at $t = 0$ and the noise at $t = T$, so corruption moves forward in t and generation moves backward. The flow model of Chapter 2 has its own clock, with the prior at $t = 0$ and the data at $t = 1$, so for it generation moves forward in t. The sign of a gradient correction must agree with the direction in which we integrate the model.

3.42.1 Reward-guided flow

Retraining is not always available. Given an unconditional flow $v_\theta(X_t, t)$ and a differentiable reward $R_t(X_t)$, we can bend the trajectory toward increasing reward by adding the reward gradient to the velocity,

$$v_t^R(X_t) = v_\theta(X_t, t) + \kappa(t)\, \nabla_{X_t} R_t(X_t).$$

"The guided flow velocity $v_t^R(X_t)$ equals the learned velocity $v_\theta(X_t, t)$ plus the reward gradient $\nabla_{X_t} R_t(X_t)$ times the time-dependent guidance strength $\kappa(t)$. This added direction increases reward at the current state X_t."

For the probability-flow velocity in Section 3.40.2, the Fokker-Planck derivation fixes the coefficient converting a score correction to velocity at $-\frac{1}{2}g(t)^2$. A general flow-matching path has no diffusion coefficient $g(t)$ that fixes this number. We therefore choose $\kappa(t)$ ourselves to control how strongly the reward changes the learned flow. One possible schedule is

$$\kappa(t) = 4\,\eta\, t\,(1-t),$$

which vanishes at both ends of the interval, peaks at η in the middle, and does not change the endpoints of the learned path.

3.42.2 Terminal rewards

Many properties are defined only for completed samples. A partially denoised protein latent does not correspond to a valid sequence, and a solubility predictor applied to it reports a number with no interpretation. We can estimate the endpoint from the flow model. For the linear interpolation path of Chapter 2, where $X_t = (1-t)X_0 + tX_1$ and the velocity is the endpoint difference, the endpoint estimate from the current state is

$$\hat{X}_1 = X_t + (1-t)\, v_\theta(X_t, t),$$

and the reward model evaluates that estimate, $R(\hat{X}_1)$. Differentiating through the endpoint prediction, we get the guidance direction at the current state,

$$\nabla_{X_t} R(\hat{X}_1(X_t)) = \left(\frac{\partial \hat{X}_1}{\partial X_t}\right)^\top \nabla_{\hat{X}_1} R(\hat{X}_1).$$

"The reward gradient at the current state X_t equals the transpose of the endpoint Jacobian $\partial \hat{X}_1/\partial X_t$ times the reward gradient $\nabla_{\hat{X}_1} R(\hat{X}_1)$ at the predicted endpoint $\hat{X}_1$. It measures how changing X_t changes the completed samples reward, requiring differentiation through the reward model and velocity network."

We now work it in one dimension. At $t = 0.6$ we take $X_t = 0.4$ and $v_\theta(X_t, t) = 1.5$, so the endpoint estimate is

$$\hat{X}_1 = 0.4 + (1 - 0.6)(1.5) = 1.0.$$

Let the reward be $R(x) = -(x - 1.3)^2$, which peaks at 1.3, so $R'(\hat{X}_1) = -2(1.0 - 1.3) = 0.6$, a positive number because the estimate still falls short of the target. If the velocity network's own derivative at this state is $\partial v_\theta/\partial x = 0.5$, the endpoint derivative is

$$\frac{\partial \hat{X}_1}{\partial X_t} = 1 + (1-t)\frac{\partial v_\theta}{\partial x} = 1 + (0.4)(0.5) = 1.2,$$

and the guidance direction is $0.6 \times 1.2 = 0.72$. With $\eta = 1$ the schedule value is $\kappa(0.6) = 4(1)(0.6)(0.4) = 0.96$, so the guided velocity is

$$v_t^R = 1.5 + (0.96)(0.72) = 2.1912,$$

and an Euler step of $\Delta t = 0.1$ moves the state to $0.4 + (0.1)(2.1912) = 0.6191$ against the unguided 0.55. The reward has advanced the trajectory by 0.0691 toward the region where $\hat{X}_1$ lands nearer 1.3.

A property predictor defined on clean samples can therefore influence states long before they are clean, at the cost of two backward passes per step.

3.43 Classifier-free guidance

Both constructions so far require a gradient from a separate model. Training a classifier on noisy states is extra work, and some conditions have no classifier at all. To remove the separate model we change how the generative network is trained. During training, we replace the condition c by a null condition $\varnothing$ on a randomly chosen fraction of examples. One network then learns both fields,

$$F_\theta(x, t, c) \quad \text{and} \quad F_\theta(x, t, \varnothing),$$

where F_θ stands for whichever quantity the model predicts: a score, a noise, or a velocity [Ho and Salimans, 2021].

We obtain the guidance direction by subtracting the unconditional prediction from the conditional prediction. Bayes' rule explains this relation. The conditional and unconditional scores satisfy

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x),$$

and rearranging isolates the classifier gradient without ever building a classifier,

$$\nabla_x \log p_t(c \mid x) = \nabla_x \log p_t(x \mid c) - \nabla_x \log p_t(x).$$

"The classifier gradient $\nabla_x \log p_t(c \mid x)$ for condition c equals the conditional score $\nabla_x \log p_t(x \mid c)$ minus the unconditional score $\nabla_x \log p_t(x)$. We can calculate this direction from both scores without training a separate classifier."

Guidance then begins with the unconditional field and amplifies the conditional difference,

$$F_{\mathrm{CFG}}(x, t, c) = F_\theta(x, t, \varnothing) + w\Big[F_\theta(x, t, c) - F_\theta(x, t, \varnothing)\Big].$$

"The classifier-free guided field $F_{\mathrm{CFG}}(x, t, c)$ equals the unconditional prediction $F_\theta(x, t, \varnothing)$, plus the guidance strength w times the difference between the conditional prediction $F_\theta(x, t, c)$ and the unconditional prediction. The null condition $\varnothing$ removes conditioning, and w scales the change induced by condition c."

At $w = 0$ the field is unconditional, at $w = 1$ it is the ordinary conditional field, and at $w > 1$ it extrapolates past the conditional field along the direction the condition induces. The same formula applies to both model families. For DDPM noise prediction,

$$\varepsilon_{\mathrm{CFG}} = \varepsilon_\theta(x, t, \varnothing) + w\Big[\varepsilon_\theta(x, t, c) - \varepsilon_\theta(x, t, \varnothing)\Big],$$

and for flow matching,

$$v_{\mathrm{CFG}} = v_\theta(x, t, \varnothing) + w\Big[v_\theta(x, t, c) - v_\theta(x, t, \varnothing)\Big].$$

3.43.1 A worked classifier-free step

Suppose that at one state the two predictions are

$$\varepsilon_\theta(x, t, \varnothing) = 0.50, \qquad \varepsilon_\theta(x, t, c) = 0.10,$$

so the condition changes the predicted noise by $0.10 - 0.50 = -0.40$. With $w = 2$,

$$\varepsilon_{\mathrm{CFG}} = 0.50 + 2\,(-0.40) = -0.30.$$

Guidance adds twice the conditional change to the unconditional prediction. The result is -0.30, beyond the conditional prediction of 0.10. Reading the same numbers as scores with $\sigma_t = 0.5$, the unconditional score is $-0.50/0.5 = -1.0$ and the conditional score is $-0.10/0.5 = -0.2$, so the guided score is $-1.0 + 2(0.8) = 0.6$, which equals $-\varepsilon_{\mathrm{CFG}}/\sigma_t = 0.30/0.5 = 0.6$. The linear relation between the two parameterizations still holds after the guidance step, so guidance may be applied in either one.

3.43.2 Selection without a gradient

Classifier-free guidance still needs a model trained with the condition. An external objective $R(x)$ that returns a value with no gradient, such as a docking score or a wet-lab assay, requires a different sampling procedure. We can generate J proposals and convert their rewards into probabilities of being retained,

$$\pi_j = \frac{\exp\left[\eta\, R(X^{(j)})\right]}{\sum_{i=1}^{J} \exp\left[\eta\, R(X^{(i)})\right]}.$$

"The retention probability π_j of proposal $X^{(j)}$ equals the exponential of its reward $R(X^{(j)})$ times the strength η, divided by the sum of these exponential weights over all J proposals. Larger rewards give larger retention probabilities, with η controlling their separation."

Take three proposals with rewards 1.2, 0.8 and 0.3, and set $\eta = 2$. Their unnormalized weights are $e^{2.4} = 11.023$, $e^{1.6} = 4.953$ and $e^{0.6} = 1.822$. The sum is 17.798, giving retention probabilities 0.6193, 0.2783 and 0.1024. The reward gap of 0.4 between the first two becomes a probability ratio of $e^{0.8} = 2.23$. We can use these weights for ranking, resampling, rejection and iterative refinement of a candidate pool. Gradient guidance changes a trajectory using a local direction. Selection compares candidates after we produce them. Without an infinitesimal direction of improvement, selection needs more samples.

3.44 Multi-objective guidance

When we design a peptide, we often need it to bind a target, stay soluble, avoid toxicity, survive in circulation, and resist nonspecific adsorption. Suppose there are M objectives,

$$r_1(x),\ r_2(x),\ \ldots,\ r_M(x),$$

each a scalar function of the state. One may be a probability between zero and one; another may be an energy spanning tens of units. Adding these raw values would let their units and numerical ranges determine the tradeoff. To remove that, we normalize each objective against a reference,

$$\tilde{r}_m(x) = \frac{r_m(x) - \mu_m}{\tau_m}.$$

"The normalized objective $\tilde{r}_m(x)$ equals the raw objective $r_m(x)$ minus its reference mean μ_m, divided by its reference scale τ_m. We therefore measure each objective in standard deviations of the reference population."

For a property that should be minimized, such as toxicity, the sign is reversed so that larger normalized values consistently mean more desirable states. Choosing nonnegative weights that sum to one,

$$\lambda_m \geq 0, \qquad \sum_{m=1}^{M} \lambda_m = 1,$$

defines one scalar reward,

$$R_\lambda(x) = \sum_{m=1}^{M} \lambda_m \, \tilde{r}_m(x).$$

"The scalarized reward $R_\lambda(x)$ equals the sum of each normalized objective $\tilde{r}_m(x)$ times its importance weight λ_m, over the M objectives. The weights sum to one, keeping the reward on the scale of a single normalized objective."

We choose the weights to set the tradeoff between properties. The reward gradient is linear in these weights,

$$\nabla_x R_\lambda(x) = \sum_{m=1}^{M} \lambda_m \, \nabla_x \tilde{r}_m(x),$$

so each property adds a direction in proportion to its weight.

3.44.1 Candidate rankings under different weights

We take two candidate peptides with normalized binding and solubility values

$$\tilde{r}(x) = (1.2,\ 0.2), \qquad \tilde{r}(x') = (0.5,\ 1.1).$$

Candidate x has stronger binding and lower solubility than candidate x'. Weighting binding at 0.7 and solubility at 0.3,

$$R_\lambda(x) = 0.7(1.2) + 0.3(0.2) = 0.84 + 0.06 = 0.90,$$
$$R_\lambda(x') = 0.7(0.5) + 0.3(1.1) = 0.35 + 0.33 = 0.68,$$

and candidate x receives the larger reward. Reversing the weights to 0.3 and 0.7,

$$R_\lambda(x) = 0.3(1.2) + 0.7(0.2) = 0.36 + 0.14 = 0.50,$$
$$R_\lambda(x') = 0.3(0.5) + 0.7(1.1) = 0.15 + 0.77 = 0.92,$$

and candidate x' is now ahead. Neither candidate dominates the other on both properties, so the preference between them rests entirely on λ. The Pareto frontier contains candidates that we cannot improve on one property without sacrificing another. Generating with several weight vectors produces different compromises along this frontier.

3.44.2 Conflicting objectives

Scalarization also resolves conflicts between directions. Suppose two normalized objectives have gradients

$$\nabla_x \tilde{r}_1 = (1.0,\ 0.8), \qquad \nabla_x \tilde{r}_2 = (0.5,\ -1.2),$$

with positive first components and second components of opposite sign. With $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$,

$$\nabla_x R_\lambda = 0.6\,(1.0,\ 0.8) + 0.4\,(0.5,\ -1.2) = (0.60 + 0.20,\ 0.48 - 0.48) = (0.80,\ 0).$$

At these weights, the second coordinate cancels exactly, so the scalarized direction moves entirely along the first. Different weights would leave a nonzero second component. We can use this combined gradient to guide either diffusion or flow matching.

3.44.3 Multi-objective guidance in each sampler

Substituting R_λ for R_t in the reward-guided score of Section 3.39 produces a guided score built from all M objectives,

$$s_\theta^{(\lambda)}(x,t) = s_\theta(x,t) + \eta \sum_{m=1}^{M} \lambda_m \nabla_x \tilde{r}_m(x).$$

For the diffusion probability-flow ODE the same reward appears in the velocity,

$$v_t^{(\lambda)}(x) = v_t(x) - \tfrac{1}{2}\,\eta\, g(t)^2 \sum_{m=1}^{M} \lambda_m \nabla_x \tilde{r}_m(x),$$

which moves backward generation toward larger scalarized reward, and for a flow that generates forward from prior to data,

$$v_t^{(\lambda)}(x) = v_\theta(x,t) + \kappa(t) \sum_{m=1}^{M} \lambda_m \nabla_x \tilde{r}_m(x).$$

Classifier-free guidance accepts the same weights. With F_u the unconditional field and F_m the field conditioned on objective m,

$$F^{(\lambda)} = F_u + \eta \sum_{m=1}^{M} \lambda_m (F_m - F_u),$$

where F again stands for a predicted noise, a score or a velocity. When we combine separately conditioned fields this way, we assume their contributions can be meaningfully added. Objectives that interact strongly are better handled by a model conditioned on all of them jointly, which learns the interaction that a scalarized combination assumes away.

3.45 Constraints and scheduled guidance

Scalarization ranks the allowed outcomes by preference. A constraint fixes which outcomes are allowed at all. Let the valid set be described by inequalities,

$$\mathcal{C} = \{x : h_j(x) \le 0,\ j = 1, \dots, J\}.$$

A soft constraint appears in the reward as a penalty on violation,

$$R_{\mathcal{C}}(x) = R_\lambda(x) - \sum_{j=1}^{J} \rho_j\, \max\left(0, h_j(x)\right)^2.$$

"The constrained reward $R_{\mathcal{C}}(x)$ equals the scalarized reward $R_{\lambda}(x)$ minus the sum of squared positive constraint violations $\max(0, h_j(x))^2$, each multiplied by its penalty weight ρ_j. Each penalty is zero when its constraint $h_j(x) \leq 0$ is satisfied."

Suppose we require a polar fraction of at least 0.4, written as $h(x) = 0.4 - f_{\text{polar}}(x)$. A twelve-residue candidate with three polar residues has $f_{\text{polar}} = 0.25$, so $h = 0.15$, and with $\rho = 10$ the penalty is $10(0.15)^2 = 0.225$. The scalarized reward falls from 0.90 to 0.675. The competing candidate in Section 3.44.1 had reward 0.68. The constraint therefore reverses their ranking at these settings. When an exact projection onto $\mathcal{C}$ is available, we can map the state back after each step,

$$x \leftarrow \text{Proj}_{\mathcal{C}}(x),$$

which enforces the constraint exactly at the cost of a discontinuity in the trajectory. Constraints that arise in sequence design include fixed positions, required motifs, composition and net charge, total length, structural geometry, maximum predicted toxicity and minimum predicted activity. For discrete sequences they can also be enforced at decoding time through token masks, fixed residues, constrained decoding, rejection, or post-generation filtering.

A property predictor is not equally trustworthy at every noise level. At high noise a detailed property predictor has little reliable signal to work with, and at low noise the emerging structure supports stronger property guidance. We can handle this by making the strength depend on time,

$$\gamma(t) = \gamma_{\max} \zeta(t),$$

where $\zeta(t)$ is a schedule between zero and one that determines when the condition becomes influential. The schedule $\kappa(t) = 4\eta t(1-t)$ of Section 3.42.1 is one such choice for a forward-generating flow.

Guidance maximizes predicted reward, so errors in the property model can produce samples with high predictions and poor measured behavior. To reduce this risk, we normalize objectives before combining them and clip unusually large guidance gradients. We use weaker guidance at high noise, evaluate rewards on predicted clean endpoints, and compare several guidance strengths. Retaining an unconditional contribution helps keep samples in regions supported by the data.

3.46 Guided generation in latent space

We apply these constructions to the ESM-2 residue embeddings used by the latent diffusion model [Lin et al., 2023]. They also apply to any other continuous state. We implement both model families on the same latents and train them with the same two property heads. For each model, we sample with classifier-free conditioning alone, guidance toward one property, and guidance toward a weighted combination. We then decode the generated latents into amino acids. During decoding, we enforce a minimum count of polar or charged residues [Chen et al., 2023].

We configure the device, ESM-2 checkpoint, sequence length, batch size, and number of training epochs. Setting the condition-dropout probability to 0.2 gives the null condition to one fifth of the training examples. We load sequences and labels from a comma-separated file, uppercase and strip the sequences, and read or calculate their property values.

To encode each batch, we load the tokenizer and pretrained ESM-2 model, freeze the parameters, and set evaluation mode. We remove the leading and trailing special tokens from the encoder

outputs and stack them into an $N \times L \times D$ tensor containing N sequences, L residues, and D embedding features. We then calculate each feature's mean and standard deviation across sequences and positions, along with separate statistics for each property. After clamping the denominators away from zero, we standardize the features and properties and save the statistics for decoding.

The two properties are composition proxies computed directly from the sequence.

```
POLAR_RESIDUES = set("STNQYCHKRDE")

def composition_proxies(seq):
    """Net charge per residue and polar/charged fraction."""
    L = max(len(seq), 1)
    charge = (seq.count("K") + seq.count("R")
              - seq.count("D") - seq.count("E")) / L
    polar = sum(1 for a in seq if a in POLAR_RESIDUES) / L
    return charge, polar
```

The first proxy counts the positively charged lysine and arginine residues, subtracts the negatively charged aspartate and glutamate, and divides by length, giving a net charge per residue. The second counts residues in the polar or charged set and divides by length. Both are bounded and cheap to compute, and neither has a gradient with respect to a latent, because both read a discrete sequence. We therefore train a small neural reward model to predict them from the latent and differentiate this model during sampling.

3.46.1 The reward model and its gradient

For guidance we need $\nabla_z R_\lambda(z)$ at the current latent, so the reward must be a differentiable function of the latent and of time. A small network predicts both standardized properties from the flattened latent and the scalar time.

```
class RewardModel(nn.Module):
    def __init__(self, latent_dim, hidden=HIDDEN):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim + 1, hidden), nn.SiLU(),
            nn.Linear(hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, 2))

    def forward(self, z, t):
        h = torch.cat([z.flatten(1), t.view(-1, 1)], dim=1)
        return self.net(h)          # two standardized properties


def normalize_weights(lambdas, device):
    w = torch.as_tensor(lambdas, dtype=torch.float32, device=device)
    w = w.clamp_min(0.0)
    return w / w.sum().clamp_min(1e-8)


def weighted_reward(reward_model, z, t, lambdas):
    r_hat = reward_model(z, t)                          # (B, M)
    return (r_hat * normalize_weights(lambdas, z.device)).sum(dim=1)


def reward_gradient(reward_model, z, t, lambdas):
```

```
    with torch.enable_grad():
        state = z.detach().requires_grad_(True)
        R = weighted_reward(reward_model, state, t, lambdas).sum()
        grad, = torch.autograd.grad(R, state)
    return grad.detach()
```

The forward method concatenates the flattened latent with the scalar time and returns two numbers per sequence, the standardized charge and the standardized polar fraction. The helper `normalize_weights` clamps the weights to be nonnegative and divides them by their sum. This enforces the simplex condition $\lambda_m \geq 0$ with $\sum_m \lambda_m = 1$ from Section 3.44, whatever the caller passes in. The reward helper `weighted_reward` forms R_λ as the weighted sum across the property axis, one scalar per sequence in the batch.

We compute the reward gradient inside a sampler that otherwise runs with gradients disabled. Within `torch.enable_grad()`, we detach a copy of the state and mark it with `requires_grad_` to start a fresh graph without previous sampling history. We then call `torch.autograd.grad` to obtain $\nabla_z R_\lambda(z)$ without accumulating parameter gradients. Summing over the batch before differentiating is correct because each sequence's reward depends only on its own latent, so the gradient of the sum equals the stacked per-sequence gradients. The final `detach` drops the graph so that the sampler's own arithmetic stays gradient-free.

We train each generative model together with its reward model. For the flow model, we draw a Gaussian z_0 and a uniform time, then form the interpolant $z_t = (1-t)z_0 + tz_1$ using a data latent z_1. We train the velocity head against $z_1 - z_0$ and the reward head against the standardized property targets, summing both squared-error losses before a single backward pass. Before the forward pass, condition dropout replaces a random fifth of the condition indices with the null index. The same network therefore learns the conditional and unconditional fields used in Section 3.43. For the diffusion model, we draw an index k uniformly from 1 to K and set $t = k/K$. We draw ε, form $z_k = \sqrt{\bar{\alpha}_k}\, z_0 + \sqrt{1-\bar{\alpha}_k}\, \varepsilon$, and regress against ε, adding the same reward loss and using the same condition dropout. This denoiser predicts the noise. The unconditional denoiser in Section 3.29 predicted the clean latent. We also pass this denoiser one extra argument, the condition index. The schedule `sched` holds the tensors of Section 3.28 unchanged, under the same names and with the same zero-prefixed indexing: the per-step variances β_k, the retentions $\alpha_k = 1 - \beta_k$, their cumulative products $\bar{\alpha}_k$, the shifted products $\bar{\alpha}_{k-1}$, and the posterior variances $\tilde{\beta}_k$.

3.46.2 The guided flow step

We can sample the flow by integrating forward from $t = 0$ to $t = 1$, applying classifier-free guidance to the velocity and then adding the scheduled reward direction.

```
@torch.no_grad()
def sample_flow(model, reward_model, n, shape, cond,
                w=2.0, eta=0.0, lambdas=(1.0, 0.0), steps=100):
    z = torch.randn(n, *shape, device=DEVICE)
    null = torch.full((n,), NULL_INDEX, device=DEVICE, dtype=torch.long)
    dt = 1.0 / steps
    for i in range(steps):
        t = torch.full((n,), i * dt, device=DEVICE)
        v = model(z, t, null)                          # unconditional field
        if cond is not None:
            v = v + w * (model(z, t, cond) - v)        # classifier-free guidance
```

```
        if eta > 0.0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = 4.0 * eta * (i * dt) * (1.0 - i * dt)
            v = v + kappa * grad_R                    # reward steering
        z = z + dt * v                                # Euler step
    return z
```

We initialize the latents with standard Gaussian noise and prepare the null condition index once for the whole batch. Inside the loop, the first network call produces the unconditional velocity. When a condition is supplied, a second call produces the conditional velocity. We then update `v` using $v_\varnothing + w(v_c - v_\varnothing)$ from Section 3.43, with $w = 2$ by default. The reward block computes $\nabla_z R_\lambda$ through the helper above, evaluates the schedule $\kappa(t) = 4\eta t(1-t)$, and adds the scaled gradient to the velocity, which is the guided flow velocity of Section 3.42.1. The final line is one explicit Euler step of size $\Delta t = 1/\text{steps}$, taken in the positive time direction because this model generates forward.

3.46.3 The guided diffusion step

The diffusion sampler runs the step index backward and applies the same two guidance terms to the predicted noise.

```
@torch.no_grad()
def sample_diffusion(model, reward_model, sched, n, shape, cond,
                     w=2.0, eta=0.0, lambdas=(1.0, 0.0)):
    z = torch.randn(n, *shape, device=DEVICE)
    null = torch.full((n,), NULL_INDEX, device=DEVICE, dtype=torch.long)
    for step in range(NUM_STEPS, 0, -1):  # k = K down to k = 1.
        t = torch.full((n,), step / NUM_STEPS, device=DEVICE)
        eps = model(z, t, null)                          # unconditional noise
        if cond is not None:
            eps = eps + w * (model(z, t, cond) - eps)    # classifier-free
        beta_k = sched.betas[step]
        alpha_k = sched.alphas[step]
        abar = sched.alphas_cumprod[step]
        if eta > 0.0:
            sigma_k = (1.0 - abar).sqrt()
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma_k * grad_R           # reward guidance
        mean = (z - beta_k / (1.0 - abar).sqrt() * eps) / alpha_k.sqrt()
        if step > 1:
            var = sched.posterior_variance[step]
            z = mean + var.sqrt() * torch.randn_like(z)
        else:
            z = mean
    return z
```

The loop runs from K down to 1. At each step, we make the same three schedule lookups as the image sampler in Section 3.28. We pass the network the normalized time $t = k/K$ used during training. The first two network calls apply classifier-free guidance to the predicted noise exactly as the flow sampler applied it to the velocity. The line `eps = eps - eta * sigma_k * grad_R` computes the guided noise prediction from Section 3.40.1. The corruption standard deviation $\sigma_k = \sqrt{1 - \bar{\alpha}_k}$ converts the score correction into a noise correction. Its effect is small at low noise and large at high noise, which is the time-dependent behavior called for in Section 3.45, obtained here without a separate schedule. At a step where $\bar{\alpha}_k = 0.36$, the standard deviation is $\sqrt{1 - 0.36} = 0.8$. A reward

gradient of 0.25 with $\eta = 1$ then shifts the noise prediction by 0.2. At a late step where $\bar{\alpha}_k = 0.99$, the same gradient shifts it by 0.025.

We compute `mean` using the reverse mean from Section 3.24.2, substituting guided noise for the raw prediction. This is the same formula used by `p_sample_latent` in Section 3.29. The final branches add Gaussian noise with the posterior standard deviation at each step except the last, where we return the mean. A deterministic variant replaces those branches with one probability-flow Euler step built from the same guided noise, using the velocity of Section 3.34.2.

3.46.4 Decoding latents back to amino acids

Generated latents lie in the standardized embedding space. To recover a sequence, we undo the normalization and pass the embeddings through the pretrained language-model head.

```
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"

@torch.no_grad()
def decode(z, esm, tokenizer, z_mean, z_std):
    latent = z * z_std + z_mean                    # undo normalization
    logits = esm.lm_head(latent)                   # full vocabulary logits
    keep = torch.tensor(
        [tokenizer.convert_tokens_to_ids(a) for a in AMINO_ACIDS],
        device=logits.device)
    logits = logits.index_select(-1, keep)         # 20 canonical residues
    scores, choices = logits.max(dim=-1)           # argmax at each position
    return logits, choices
```

The first line reverses the standardization using the statistics saved at load time, returning the latent to the scale ESM-2 produced. The call to `esm.lm_head` is the masked-language-model head of ESM-2, which maps a residue embedding to a logit over the full tokenizer vocabulary [Devlin et al., 2019, Lin et al., 2023]. That vocabulary contains special tokens and ambiguity codes that no generated sequence should contain, so `index_select` retains only the twenty canonical amino-acid columns. The final `max` selects the highest-scoring residue at each position. It returns both the retained logits and the chosen indices. The constraint step uses these logits to rank substitutions by cost.

3.46.5 Enforcing the composition constraint at decoding

Guidance makes a high polar fraction more likely without guaranteeing it. We can impose the hard constraint afterwards, by substituting residues at the positions where substitution costs the least logit.

```
POLAR_COLUMNS = [AMINO_ACIDS.index(a) for a in "STNQYCHKRDE"]

def enforce_polar(logits, choices, minimum):
    cols = torch.tensor(POLAR_COLUMNS, device=logits.device)
    polar_scores, pick = logits.index_select(-1, cols).max(dim=-1)
    polar_choices = cols[pick]                     # best polar residue
    already = torch.isin(choices, cols)            # positions already polar
    sequences = []
    for b in range(choices.shape[0]):
        row = choices[b].clone()
        missing = minimum - int(already[b].sum())
        if missing > 0:
            chosen = logits[b].gather(1, row[:, None]).squeeze(1)
```

```
            cost = chosen - polar_scores[b]          # logit lost by swapping
            cost = cost.masked_fill(already[b], float("inf"))
            _, swap = cost.topk(missing, largest=False)
            row[swap] = polar_choices[b][swap]
        sequences.append("".join(AMINO_ACIDS[i] for i in row.tolist()))
    return sequences
```

At each position, we find the highest-scoring polar or charged residue. We store the logits and residue choices in `polar_scores` and `polar_choices`. The mask `already` marks the positions whose unconstrained choice is already in the set. The count `missing` is how many further positions the constraint requires, and when it is zero or negative the sequence passes through unchanged. Otherwise, `cost` measures the logit lost at each position when we replace the argmax residue with the best polar one. We assign infinite cost to positions that are already polar, preventing a second selection. Calling `topk` with `largest=False` selects the `missing` lowest-cost substitutions. We update those positions and convert the integer rows to sequence strings.

We take a twelve-residue sequence whose argmax decoding contains three polar residues and a required minimum of five, so missing $= 2$. Suppose the nine non-polar positions have substitution costs $0.9, 1.4, 2.6, 3.1, 3.3, 4.0, 4.2, 5.5$ and 6.1 in units of logit. The two cheapest are 0.9 and 1.4, so we replace those two residues with a total logit reduction of 2.3. The constraint is now satisfied exactly, and the sequence differs from the unconstrained decoding at two positions chosen to be the least costly ones available. The same mechanism extends to fixed positions, required motifs and length limits. We perform this exact projection once at the end in place of the gradient-based penalty in Section 3.45.

In the driver, we load the data, train both networks, and generate three latent batches from each model. We use classifier-free conditioning at $w = 2$, guidance toward the first property with $\lambda = (1, 0)$, and guidance toward both properties with $\lambda = (0.7, 0.3)$. We decode each batch, enforce the polar constraint, and write the sequences to FASTA files. We also save the latents, parameters, and normalization statistics for reuse [Paszke et al., 2019].

3.47 Discrete state spaces

We added noise to deterministic transport and obtained a stochastic differential equation. The corresponding density evolved according to the Fokker-Planck equation. We learned the score through denoising and used it to reverse the dynamics. Predicting the noise was equivalent to estimating a rescaled score.

To obtain DDPM, we discretized the reverse-time equation on a grid. We also rewrote the Laplacian as a divergence to obtain the probability-flow ODE. This ODE reproduced the same marginal distributions deterministically, giving a continuous normalizing flow with an explicit likelihood. For conditional generation, we used a gradient derived from Bayes' rule. Depending on the conditioning method, we calculated this gradient from a classifier, a reward, the difference between two network predictions, or a weighted combination of property objectives.

We can gradually corrupt either a vector or a string. In both cases, we can train a denoiser to reverse a corruption step using the posterior average as its regression target. We can also use Bayes' rule to derive reverse transitions from forward transitions. To guide samples toward several properties, we combine the property objectives in a scalarized reward.

For continuous dynamics, we used the score $\nabla_x \log p_t(x)$. Computing this gradient requires a state space with infinitesimal displacements. A sequence instead belongs to $\mathcal{S} = \mathcal{V}^L$, the set of length-$L$ strings over a finite alphabet $\mathcal{V}$ with $|\mathcal{V}|$ symbols. This space has no infinitesimal displacements. Changing a symbol requires a discrete replacement. We therefore cannot directly use the continuous score, velocity field, Laplacian, or Gaussian corruption kernel on this finite set. The integration-by-parts derivation of implicit score matching also requires continuous space.

To describe motion on $\mathcal{V}^L$, we use a jump process. A particle stays at one state for a random waiting time and then jumps to another. We specify the motion through the rate $R_t(x, y)$, the probability per unit time of jumping from state x to a different state y at time t. We track the density with the master equation, balancing probability flow into each state against probability flow out. For corruption, we replace the Gaussian transition with a Markov kernel constructed from masking or uniform replacement. We then apply Bayes' rule to the forward rate and the denoiser to derive the reverse rate. To guide sampling, we reweight those reverse rates using a classifier evaluated on the sequence.

In Chapter 4, let's build a generative model that changes tokens directly. We will corrupt complete sequences and train a denoiser to predict the original tokens from the corrupted sequence. A prediction at one position can then depend on the rest of the sequence. By converting these predictions into reverse transition rates, we can sample strings while accounting for dependencies between positions.

Chapter 4

Discrete Diffusion

4.1 Finite-vocabulary diffusion

In Chapter 3 we constructed a generative model by corrupting data and learning to reverse the corruption. We added Gaussian increments to a continuous state and trained a network to estimate the score at every noise level. We then generated samples by solving a reverse stochastic differential equation from noise toward the data distribution. Every intermediate state belonged to $\mathbb{R}^d$, so we could corrupt it through small continuous displacements.

Each position in a DNA sequence contains one of four letters: A, C, G or T. A peptide contains one of twenty residues at each position, and a sentence contains tokens from a fixed vocabulary. We cannot move continuously from A to C because the alphabet contains no intermediate letter. To keep every state inside the vocabulary, we change a sequence by erasing, revealing or replacing its tokens.

We use $t = 0$ for clean data and $t = 1$ for noise. A sequence at $t = 0$ is a data sample; at $t = 1$ it contains no information about that sample. This is the diffusion convention from Chapter 3 and the masked-diffusion papers. In Chapter 2, we used the opposite convention, with the source at $t = 0$ and the data at $t = 1$. Generation therefore runs backward in time, from $t = 1$ down to $t = 0$, and a single reverse step always moves from a later time t to an earlier time $s < t$, where s is the less corrupted of the two.

4.1.1 Autoregressive and bidirectional prediction

We train autoregressive models to predict each token from the tokens before it. Large-scale training of these models supports many current language, reasoning and code-generation capabilities [Brown et al., 2020]. During generation, we complete the prefix before sampling the next token, so every prediction uses a fixed left context.

We can also predict a token using context on its right. In “The [MASK] was too hot to drink”, we can use the words on both sides of the gap to narrow the missing token to a handful of candidates. To train a bidirectional predictor, we hide a subset of tokens in a real sentence and train a network to recover them [Devlin et al., 2019]. We can use this predictor for text infilling and partially observed sequences. When several positions are hidden, we predict them together using every visible token.

Training on different hidden subsets produces prediction tasks ranging from one missing token to a completely hidden sequence. To generate complete sequences, we need to connect these tasks. First, we specify a corruption process that determines the probability of each hidden subset. We then train one network across these subsets, using a loss that weights each task appropriately. Finally, we sample reverse transitions from a fully corrupted sequence to a clean one. In masked diffusion, we can derive the corruption, loss and reverse transitions from a single survival schedule.

4.2 Vocabulary and sequence-space size

For a fixed alphabet and length, we write a sequence as a tuple of tokens, one per position,

$$x = (x^1, \dots, x^L) \in \mathcal{V}^L, \qquad |\mathcal{V}^L| = K^L.$$

"The sequence x contains L tokens, x^1 through x^L, from the alphabet $\mathcal{V}$ of K symbols. Choosing one symbol at each position gives K^L sequences in the state space $\mathcal{V}^L$."

- $\mathcal{V}$ is the alphabet, a finite set of symbols.
- $K = |\mathcal{V}|$ is the alphabet size.
- L is the sequence length, the number of positions.
- $x^i \in \mathcal{V}$ is the token at position i, with a superscript for the position so that subscripts stay available for time.
- $\mathcal{S} = \mathcal{V}^L$ is the state space, the set of all sequences the model can occupy.

For the worked calculations, use the DNA alphabet $\mathcal{V} = \{A, C, G, T\}$, with $K = 4$, so we can list each token probability. Even four letters give $4^{10} = 1{,}048{,}576$ sequences at length $L = 10$. A 12-residue peptide over twenty amino acids has $20^{12} = 4.096 \times 10^{15}$ possible sequences. We cannot enumerate all sequence probabilities at either size. We therefore need a generative model that concentrates probability on the useful sequences without listing the entire state space.

The positions also depend on each other, so the problem does not split into L separate ones. Let's take length two and suppose the sequences we want are AA, CC, GG and TT. A model that samples the two bases independently and uniformly produces a matching pair with probability $4 \times \frac{1}{4} \times \frac{1}{4} = \frac{1}{4}$, so three quarters of its samples fall outside the target set even though its marginal at each position is correct. The second base has to agree with the first, and only a model that represents the dependence between positions can enforce that.

We can train this model using each clean sequence as its own target. We corrupt the sequence and train a network to recover the removed information, repeating the procedure at noise levels from nearly clean to almost completely corrupted. During generation, we repeatedly use the network predictions to update a corrupted sequence until every position contains a clean token.

4.3 Two ways to corrupt a sequence

We can corrupt a discrete sequence by erasing tokens or replacing them. Each choice requires a different reverse process.

For erasure, we replace a token with a special symbol m outside the clean alphabet, written [MASK] in text models. In AmGT, every visible letter is original, so we need to infer only the second base. The mask identifies the corrupted position directly.

For replacement, we change a token to another letter in the same alphabet. In AGGT, the second G could be original or a replacement for C. We need to infer both which positions require revision and their correct tokens. Uniform replacement allows us to edit a position repeatedly during generation. Its reverse posterior and likelihood loss differ from those for masking because no absorbing symbol identifies the corrupted positions.

For masking, we can separate the reverse step into two independent decisions and reduce the loss to a weighted cross-entropy over tokens.

4.4 Continuous time for a discrete state

Both forms of corruption change a sequence at discrete moments. We can still use continuous time, as in Chapters 2 and 3, to choose reverse steps of any size and integrate the loss over noise levels. A changing path through a finite state space, however, cannot be continuous,

$$\gamma : [0,1] \to \mathcal{S} \ \text{ continuous } \implies \gamma(t) = \gamma(0).$$

"If the sequence-valued path γ is continuous on the time interval from 0 to 1, then the sequence $\gamma(t)$ at every time t equals the initial sequence $\gamma(0)$, so a continuous path never changes a single token."

A finite state space has the discrete topology, so every subset is open. The interval $[0,1]$ is connected, and a continuous map must have a connected image. The only connected subsets of a discrete space contain one element. The image of the entire interval must therefore be a single sequence.

The probabilities can change continuously even when each sequence changes through jumps. At every instant, we specify the probability of a token changing and the distribution of its replacement. A sampled sequence stays fixed between jumps, but the probability of each sequence varies smoothly with t. For example, one trajectory might pass from ACGT to AmGT to AmGm. Across many independent trajectories, the fraction in each state approximates a smooth function of time.

4.5 Masked diffusion on one token

In MDLM we train a single network to reverse progressive masking, using a weighted cross-entropy at masked positions [Sahoo et al., 2024]. We can define the corruption at one position first.

We introduce the mask symbol m, outside the clean alphabet, and write $\mathcal{V}_m = \mathcal{V} \cup \{m\}$ for the enlarged alphabet with $K+1$ symbols. During corruption a clean token either remains visible or becomes m, and once masked it stays masked for the rest of the forward process.

Let's follow one position whose clean token is $a \in \mathcal{V}$. Corruption at time t begins with a draw on the survival of the original letter,

$$B_t \sim \text{Bernoulli}(\alpha_t).$$

"The survival draw B_t is Bernoulli with survival probability α_t, so it equals one with probability α_t and zero with probability one minus α_t."

The draw selects between two outcomes,

$$Z_t = \begin{cases} a, & B_t = 1, \\ m, & B_t = 0. \end{cases}$$

"The noisy token Z_t equals the clean token a when the survival draw B_t is one, and equals the mask m when B_t is zero."

If we add the probabilities of the two events, we get the forward kernel at one position,

$$q_t(z \mid a) = \alpha_t \mathbf{1}\{z = a\} + (1 - \alpha_t)\, \mathbf{1}\{z = m\}.$$

"The probability $q_t(z \mid a)$ of the noisy token z, given the clean token a, is the sum of two contributions: the survival probability α_t when $z = a$, and the masking probability $1 - \alpha_t$ when $z = m$, where m is the mask symbol."

- $\alpha_t \in [0, 1]$ is the survival probability, the probability that a token is still clean at time t.
- $Z_t \in \mathcal{V}_m$ is the noisy token at time t, a random variable.
- m is the mask symbol, absorbing under the forward process.
- $\mathbf{1}\{\cdot\}$ is the indicator, one when its argument holds and zero otherwise.

Every other outcome in $\mathcal{V}_m$ has probability zero. A masked token remains masked during forward corruption, so it can never become a different visible base. This absorbing property simplifies the reverse posterior.

The schedule α_t is a design choice, subject to $\alpha_0 = 1$, $\alpha_1 = 0$ and α_t decreasing in t. At $t = 0$ every token survives and the sequence is clean; at $t = 1$ no token survives and the sequence is entirely mask. In the worked examples here we use the linear schedule $\alpha_t = 1 - t$, for which the arithmetic stays readable.

Let's take the clean token $a = C$ and list $q_t(\cdot \mid C)$ over the five outcomes (A, C, G, T, m) at three times of the linear schedule,

$$\begin{aligned} q_{0.25}(\cdot \mid C) &= (0,\ 0.75,\ 0,\ 0,\ 0.25), \\ q_{0.5}(\cdot \mid C) &= (0,\ 0.5,\ 0,\ 0,\ 0.5), \\ q_{0.75}(\cdot \mid C) &= (0,\ 0.25,\ 0,\ 0,\ 0.75). \end{aligned}$$

Only the clean letter and the mask have positive probability. As t increases, probability moves from the clean letter to the mask, with the other three bases remaining at zero. Uniform replacement would also assign probability to A, G and T, so we would need to consider every letter as a possible source of the observed token.

4.5.1 Independence across positions

Corruption applies the same draw at each position, independently,

$$q_t(z \mid x) = \prod_{i=1}^{L} q_t(z^i \mid x^i).$$

"The probability $q_t(z \mid x)$ of the entire corrupted sequence z given the clean sequence x equals the product over positions $i = 1$ through L of the probability $q_t(z^i \mid x^i)$ of the noisy token at that position given the clean token there."

We draw independently at each position with the same survival probability α_t. The number of surviving positions is therefore Binomial with L trials and success probability α_t, and the expected number of masked positions is $L(1-\alpha_t)$. These corruption draws are independent conditional on the clean sequence x. Positions in the data distribution p_{data} can still depend on each other arbitrarily, as in the AA, CC, GG, TT example in Section 4.2.

A forward trajectory under the linear schedule, drawn on the grid $t = 0, 0.3, 0.7, 1$, might run

$$\underbrace{\text{ACGT}}_{t=0} \longrightarrow \underbrace{\text{AmGT}}_{t=0.3} \longrightarrow \underbrace{\text{Ammm}}_{t=0.7} \longrightarrow \underbrace{\text{mmmm}}_{t=1}.$$

Following the same initial sequence through time, we erase more positions while retaining the identity of every visible letter. A masked position never returns to a letter during forward corruption.

4.5.2 The transition between two noise levels

The kernel $q_t(z \mid a)$ relates the noisy token to the clean one. A reverse step relates the token at two noisy times, so we also need the law of Z_t given Z_s for $s < t$. Survival to t requires survival to s first, and the forward process has no memory beyond the current state, so the two survivals multiply,

$$\mathbb{P}(Z_t = a \mid a) = \mathbb{P}(Z_s = a \mid a)\, \mathbb{P}(Z_t = a \mid Z_s = a).$$

"The probability $\mathbb{P}(Z_t = a \mid a)$ of retaining the clean token a at time t equals the survival probability $\mathbb{P}(Z_s = a \mid a)$ at the earlier time s times the probability $\mathbb{P}(Z_t = a \mid Z_s = a)$ of surviving the interval from s to t."

Substituting the two marginal survival probabilities turns this into a statement about the schedule,

$$\alpha_t = \alpha_s\, \mathbb{P}(Z_t = a \mid Z_s = a),$$

and taking the complement of the surviving event gives the masking probability over the interval,

$$\mathbb{P}(Z_t = m \mid Z_s = a) = 1 - \frac{\alpha_t}{\alpha_s}.$$

"The probability $\mathbb{P}(Z_t = m \mid Z_s = a)$ of masking a visible token a between times s and t equals one minus the later survival probability α_t divided by the earlier survival probability α_s, for $s < t$ and $\alpha_s > 0$."

A masked token stays masked, so $\mathbb{P}(Z_t = m \mid Z_s = m) = 1$. We therefore need only the marginal survival probabilities α_s and α_t to determine the interval transition. With $\alpha_s = 0.7$ and $\alpha_t = 0.4$, a token that is still visible at s is masked by t with probability $1 - 0.4/0.7 = 3/7 \approx 0.4286$.

4.6 Reversing one step of the corruption

To generate, we run the interval rule backward. Given a position that is masked at time t, we need the law of the same position at the earlier time s. We first condition on the clean token a to calculate this probability.

Bayes' rule turns the reverse conditional into forward quantities,

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a,\, Z_t = m \mid a)}{q(Z_t = m \mid a)}.$$

"The reveal probability $q(Z_s = a \mid Z_t = m, a)$ is the joint probability $q(Z_s = a, Z_t = m \mid a)$ of being visible at the earlier time s and masked at the later time t, divided by the probability $q(Z_t = m \mid a)$ of being masked at t. Both probabilities condition on the clean token a."

We can factor the numerator along the forward path. The token survives to s with probability α_s and is then masked during the interval with probability $1 - \alpha_t/\alpha_s$, while the denominator is the marginal masking probability $1 - \alpha_t$,

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s\,(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}.$$

Cancelling the survival factor α_s inside the numerator,

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} \;=:\; r_{s,t}.$$

"The reveal probability $r_{s,t}$, written as $q(Z_s = a \mid Z_t = m, a)$, equals the decrease in survival probability from α_s to α_t, divided by the current masking probability $1 - \alpha_t$. Here a is the clean token and m is the mask."

We write $r_{s,t}$ for this number throughout, and drop the subscripts to r where the two times are fixed by context. It depends on the schedule and on nothing else.

4.6.1 Reverse probability by path counting

We can derive $r_{s,t}$ a second time, by counting. Let's start a large population of independent positions, all with the same clean token a, and follow them forward.

A fraction α_s of the population is still visible at time s,

$$q(Z_s = a \mid a) = \alpha_s.$$

Of those, a fraction $1 - \alpha_t/\alpha_s$ is masked by time t, so the fraction that is visible at s and masked at t is

$$q(Z_s = a,\, Z_t = m \mid a) = \alpha_s \left(1 - \frac{\alpha_t}{\alpha_s}\right) = \alpha_s - \alpha_t.$$

The fraction masked at t in total, whenever the masking happened, is $1 - \alpha_t$. Normalizing the first count within the second,

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t},$$

which agrees with the Bayes derivation. The numerator is the survival mass lost between s and t, corresponding to positions restored in the reverse step. The denominator includes every position masked at t, regardless of when it became masked. We condition on its current mask status because its masking time is unknown.

4.6.2 The probability of staying masked

A position that is masked at t either reveals its letter at s or stays masked, and those two events exhaust the possibilities,

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a).$$

Substituting the reveal probability and combining over a common denominator,

$$q(Z_s = m \mid Z_t = m, a) = 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{1 - \alpha_s}{1 - \alpha_t}.$$

"The probability $q(Z_s = m \mid Z_t = m, a)$ of remaining masked at the earlier time s, given a mask at time t and clean token a, equals the earlier masking probability $1 - \alpha_s$ divided by the current masking probability $1 - \alpha_t$."

The two probabilities add to one,

$$\frac{\alpha_s - \alpha_t}{1 - \alpha_t} + \frac{1 - \alpha_s}{1 - \alpha_t} = \frac{1 - \alpha_t}{1 - \alpha_t} = 1,$$

as they must, since the position is either revealed or still masked at s.

The reveal probability $r_{s,t}$ depends only on the schedule at the two times. It is the same for every clean token a. The identity of a matters only through which letter appears once the reveal has happened. That separation lets the learned reverse step factor into a schedule-determined probability of acting at all, multiplied by a network-determined choice of letter.

4.7 The denoiser and the learned reverse step

The exact reverse step conditions on the clean token a, which we do not have at generation time. We can replace it by a network estimate of that posterior,

$$d_\theta^i(a \mid z_t, t) \approx q(x_0^i = a \mid z_t).$$

"The denoiser probability $d_\theta^i(a \mid z_t, t)$ estimates the posterior probability that the clean token x_0^i at position i was a, given the full noisy sequence z_t and the noise time t."

- θ are the network parameters.
- $z_t \in \mathcal{V}_m^L$ is the realized noisy sequence at time t, with z_t^i its token at position i.

- $x_0 \in \mathcal{V}^L$ is the clean sequence, with x_0^i its token at position i.
- $d_\theta^i(\cdot \mid z_t, t)$ is a categorical distribution over clean tokens for position i.

We input the entire noisy sequence to the denoiser and obtain one categorical distribution per position. Each masked-position prediction can therefore depend on every visible token, to its left and right. This is the bidirectional predictor from Section 4.1, with the noise level supplied as its second argument.

Averaging the exact reverse step over this prediction defines the learned reverse step,

$$p_\theta(Z_s^i = a \mid z_t) = \sum_{b \in \mathcal{V}} q(Z_s^i = a \mid z_t^i = m,\, b)\; d_\theta^i(b \mid z_t, t).$$

"The learned probability $p_\theta(Z_s^i = a \mid z_t)$ of revealing token a at position i averages the exact reverse probability over candidate clean tokens b, weighted by their denoiser probabilities $d_\theta^i(b \mid z_t, t)$ given the noisy sequence z_t at time t."

A revealed base must equal the clean token it came from, so the exact reverse probability inside the sum is $r_{s,t}\,\mathbf{1}\{a = b\}$,

$$p_\theta(Z_s^i = a \mid z_t) = \sum_{b \in \mathcal{V}} r_{s,t}\,\mathbf{1}\{a = b\}\, d_\theta^i(b \mid z_t, t),$$

and only the term $b = a$ remains,

$$\boxed{p_\theta(Z_s^i = a \mid z_t) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}\, d_\theta^i(a \mid z_t, t)}$$

"The learned reveal probability $p_\theta(Z_s^i = a \mid z_t)$ equals the survival difference $\alpha_s - \alpha_t$ divided by the current masking probability $1 - \alpha_t$, multiplied by the denoiser probability $d_\theta^i(a \mid z_t, t)$ of clean token a at position i given the noisy sequence z_t."

To sample a reverse step at a masked position, first draw a Bernoulli variable with reveal probability $r_{s,t}$. This probability comes from the schedule and is the same at every masked position. If the position is revealed, draw its letter from $d_\theta^i(\cdot \mid z_t, t)$. Otherwise, keep the mask, which has probability $1 - r_{s,t}$. The first draw depends on the step size; the second uses the network prediction.

4.8 Reverse-step example

We can take $\alpha_s = 0.7$ and $\alpha_t = 0.4$, which under the linear schedule $\alpha_t = 1 - t$ are the times $s = 0.3$ and $t = 0.6$. The reveal probability follows from the schedule alone,

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5.$$

Among the positions still masked at time t, the recovered survival mass $\alpha_s - \alpha_t = 0.3$ is half of the total mask mass $1 - \alpha_t = 0.6$, so half of those positions are expected to become visible in this step.

Suppose the denoiser at one such position predicts A with probability 0.6 and C with probability 0.4, and gives G and T no mass. The reveal mass splits in that proportion,

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \qquad p(C \mid z_t) = 0.5(0.4) = 0.2.$$

Including the probability of keeping the mask completes the distribution over the five outcomes,

$$p(A, C, G, T, m \mid z_t) = (0.3,\ 0.2,\ 0,\ 0,\ 0.5), \qquad \sum_{b \in \mathcal{V}_m} p(b \mid z_t) = 1.$$

"The reverse distribution $p(A, C, G, T, m \mid z_t)$ assigns probability 0.5 to retaining the mask m, 0.3 to revealing A and 0.2 to revealing C, given the noisy sequence z_t. The probabilities of G and T are zero, and the five probabilities sum to one."

We reveal a token with probability one half. Conditional on revealing it, we recover the denoiser probability for A by dividing $0.3/0.5 = 0.6$. With the remaining probability one half, we keep the position masked. We can then predict it again in a subsequent step using any additional tokens revealed elsewhere. The schedule controls how quickly we reveal positions.

4.9 Masked DNA generation

We can generate a length-four DNA sequence by applying these two draws at every masked position. We can use the linear schedule and the reverse grid $t = 1, 0.7, 0.3, 0$, so that $\alpha_1 = 0$, $\alpha_{0.7} = 0.3$, $\alpha_{0.3} = 0.7$ and $\alpha_0 = 1$.

Generation starts from mmmm at $t = 1$, where every position is masked and the sequence holds no information.

The first step moves to $s = 0.7$. Its reveal probability is

$$r_{0.7,\,1} = \frac{\alpha_{0.7} - \alpha_1}{1 - \alpha_1} = \frac{0.3 - 0}{1 - 0} = 0.3,$$

applied independently at each of the four masked positions, for an expected $4 \times 0.3 = 1.2$ reveals. In this sample, we reveal positions 1 and 3 and draw A and G from the corresponding denoiser distributions, giving AmGm. This reveal pattern has probability $0.3^2 \times 0.7^2 = 0.0441$. The complete step includes both the reveal pattern and the sampled letters, with 2^4 possible patterns and all compatible letter assignments.

The second step moves to $s = 0.3$, with two positions still masked,

$$r_{0.3,\,0.7} = \frac{\alpha_{0.3} - \alpha_{0.7}}{1 - \alpha_{0.7}} = \frac{0.7 - 0.3}{1 - 0.3} = \frac{0.4}{0.7} \approx 0.5714,$$

for an expected $2 \times 0.5714 \approx 1.14$ reveals. The sampled draw reveals position 2 as C, giving ACGm. We keep A and G fixed because the learned reverse distribution assigns probability one to the observed letter at each visible position.

For the last step, we set $s = 0$ and calculate

$$r_{0,\,0.3} = \frac{\alpha_0 - \alpha_{0.3}}{1 - \alpha_{0.3}} = \frac{1 - 0.7}{1 - 0.7} = 1,$$

so every remaining mask is revealed. The sampled final letter is T, and the trace ends at ACGT,

$$\underbrace{\text{mmmm}}_{t=1} \longrightarrow \underbrace{\text{AmGm}}_{t=0.7} \longrightarrow \underbrace{\text{ACGm}}_{t=0.3} \longrightarrow \underbrace{\text{ACGT}}_{t=0}.$$

The number of visible positions never decreases because masking is absorbing in the forward process. As s approaches zero, the reveal probability approaches one, so we reveal all remaining masks by the final step. We sampled each letter of ACGT from the denoiser, conditional on the tokens visible before that step.

We assumed an accurate denoiser when constructing these reverse steps. To train it, we need to quantify the difference between its predictions and the target distribution.

4.10 Comparing two predictions with KL divergence

We can compare two distributions over the same finite set of outcomes by the Kullback-Leibler divergence,

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \;\geq\; 0.$$

"The divergence $D_{\mathrm{KL}}(p \,\|\, q)$ equals the sum over outcomes x of the reference probability $p(x)$ times the logarithm of the reference probability divided by the comparison probability $q(x)$, and it is never negative."

- p is the reference distribution, the one whose probabilities are the weights in the sum.
- q is the comparison distribution, the one being measured against p.

The divergence is zero exactly when the distributions agree on the support of p. It is infinite if q assigns zero probability to any outcome with positive probability under p. Because the divergence is asymmetric, we specify the exact law as the first argument and the model as the second. For reverse steps, these laws are q and p_θ, giving $D_{\mathrm{KL}}(q \,\|\, p_\theta)$.

4.10.1 A two-token example

We can take an alphabet with two tokens, a target $q = (0.75, 0.25)$ and a model $p = (0.5, 0.5)$, and compute the mismatch,

$$q = (0.75,\, 0.25), \qquad p = (0.5,\, 0.5), \qquad q/p = (1.5,\, 0.5).$$

The ratio is above one at the token the model underestimates and below one at the token it overestimates, which already identifies the direction of each error.

Each token contributes its log ratio, weighted by the target probability,

$$0.75 \log 1.5 \approx 0.30410, \qquad 0.25 \log 0.5 \approx -0.17329.$$

An individual KL term is negative when the model assigns more probability to a token than the target does. Here the two contributions have opposite signs, and their sum is nonnegative.

Adding them gives the divergence,

$$D_{\mathrm{KL}}(q \,\|\, p) = 0.30410 - 0.17329 \approx 0.13081 \text{ nats}.$$

"The divergence $D_{\mathrm{KL}}(q \,\|\, p)$ between the target distribution q and model distribution p is the sum of their target-weighted log ratios, giving approximately 0.13081 nats of mismatch."

Had the model matched the target, both ratios would equal one, both logarithms would vanish, and the divergence would be zero. For a fixed target, we therefore minimize the KL divergence by matching the model distribution to it.

4.11 Pathwise variational bound

We want to maximize the probability of real sequences under the model. To justify training with reverse-step divergences, we need to relate them to this sequence probability.

Discretize time on the grid $0 = t_0 < t_1 < \cdots < t_N = 1$, and write x_n for the state at time t_n. The state x_0 is clean, and x_N is all masks. We generate by taking reverse steps from x_N to x_0, with joint path probability $p_\theta(x_{0:N})$. Summing over every intermediate noisy path gives the clean-sequence probability,

$$p_\theta(x_0) = \sum_{x_{1:N}} p_\theta(x_{0:N}).$$

"The probability $p_\theta(x_0)$ of the clean sequence x_0 equals the sum over every noisy path $x_{1:N}$ of the joint model probability $p_\theta(x_{0:N})$ of the full path from clean step zero to noise step N."

That sum has as many terms as there are paths, up to $(K+1)^{LN}$ of them, which is beyond evaluation at any realistic L and N. If we multiply and divide by the forward path law, which we can sample from, the sum becomes an average,

$$p_\theta(x_0) = \sum_{x_{1:N}} q(x_{1:N} \mid x_0)\, \frac{p_\theta(x_{0:N})}{q(x_{1:N} \mid x_0)} = \mathbb{E}_q\left[\frac{p_\theta(x_{0:N})}{q(x_{1:N} \mid x_0)}\right].$$

"The clean-sequence probability $p_\theta(x_0)$ equals the average, under the forward path law q, of the joint model probability $p_\theta(x_{0:N})$ divided by the forward conditional path probability $q(x_{1:N} \mid x_0)$."

The forward law has to cover the model paths for this to be valid, meaning $q(x_{1:N} \mid x_0) > 0$ wherever $p_\theta(x_{0:N}) > 0$. Masking satisfies this for any schedule with $0 < \alpha_{t_n} < 1$ at the interior times, and sampling from it costs one Bernoulli draw per position per step. The quantity inside the expectation is the path weight, and it is positive.

4.11.1 Jensen's inequality

The log probability contains the logarithm of an average. We can estimate an average of log weights from sampled paths and use concavity to relate these two quantities.

A function is concave when its graph lies above every chord joining two of its points,

$$f(\lambda a + (1-\lambda)b) \;\geq\; \lambda f(a) + (1-\lambda) f(b) \quad\Longrightarrow\quad f(\mathbb{E}W) \;\geq\; \mathbb{E} f(W).$$

"For a concave function f, the value at the weighted mean $\lambda a + (1-\lambda)b$ is at least the weighted mean of the values at a and b, where λ is the weight on a. For a random input W, this gives a function value $f(\mathbb{E}W)$ at the mean that is at least the mean function value $\mathbb{E}f(W)$."

The logarithm is concave. Let W take the values 1 and 4 with equal probability. If we average first, we get $\mathbb{E}W = 2.5$ and $\log 2.5 \approx 0.91629$. If we take logarithms first, we get $\frac{1}{2}\log 1 + \frac{1}{2}\log 4 = \frac{1}{2}(0) + \frac{1}{2}(1.38629) \approx 0.69315$. The first number is the larger of the two, as concavity requires.

4.11.2 Variational bound and gap

Consider two equally likely path weights. One path has half its forward-law probability under the model; the other has one and a half times its forward-law probability,

$$\mathbb{P}(W = 0.5) = \mathbb{P}(W = 1.5) = \tfrac{1}{2}.$$

Averaging the weights and then taking the logarithm gives

$$\log \mathbb{E}[W] = \log\Big[\tfrac{1}{2}(0.5) + \tfrac{1}{2}(1.5)\Big] = \log 1 = 0,$$

and taking the logarithm first and averaging afterward gives

$$\mathbb{E}[\log W] = \tfrac{1}{2}\log 0.5 + \tfrac{1}{2}\log 1.5 \approx -0.14384 \;\le\; 0.$$

"The mean log weight $\mathbb{E}[\log W]$ is about -0.14384, and the log mean weight $\log \mathbb{E}[W]$ is zero. Concavity of the logarithm makes the mean log weight no larger than the log mean weight."

The gap between the two is 0.14384 nats, and it is caused entirely by the spread of W. Had both weights equalled one, the two sides would coincide. The bound is tight when the path weight is constant across paths, which depends on how closely the model's reverse process matches the forward process.

4.11.3 The evidence lower bound

We can write the path weight as

$$W = \frac{p_\theta(x_{0:N})}{q(x_{1:N} \mid x_0)}, \qquad \text{so that} \qquad \log p_\theta(x_0) = \log \mathbb{E}_q W.$$

Applying concavity of the logarithm,

$$\log \mathbb{E}_q W \;\ge\; \mathbb{E}_q \log W,$$

and expanding the log ratio inside the expectation defines the bound,

$$\mathrm{ELBO} = \mathbb{E}_q[\log p_\theta(x_{0:N}) - \log q(x_{1:N} \mid x_0)]\,.$$

"The evidence lower bound ELBO averages the joint path log probability $\log p_\theta(x_{0:N})$ minus the forward conditional path log probability $\log q(x_{1:N} \mid x_0)$, over noisy paths $x_{1:N}$ drawn from the forward law q for the clean sequence x_0."

For each sampled path, evaluate the two log probabilities after one forward corruption. Averaging their difference estimates the lower bound, which we can differentiate for training.

4.11.4 Exact variational gap

To calculate the gap, start with the KL divergence between the forward path law and the model's path posterior,

$$D_{\mathrm{KL}}(q \,\|\, p_\theta(\cdot \mid x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} \mid x_0)}{p_\theta(x_{1:N} \mid x_0)}.$$

"The path divergence equals the average under the forward law q of the logarithm of the forward path probability $q(x_{1:N} \mid x_0)$ divided by the model's posterior path probability $p_\theta(x_{1:N} \mid x_0)$, given the clean sequence x_0."

Next, we can replace the posterior by the joint divided by the marginal, $p_\theta(x_{1:N} \mid x_0) = p_\theta(x_{0:N})/p_\theta(x_0)$,

$$D_{\mathrm{KL}}(q \,\|\, p_\theta(\cdot \mid x_0)) = \mathbb{E}_q[\log q(x_{1:N} \mid x_0) - \log p_\theta(x_{0:N}) + \log p_\theta(x_0)] \,.$$

The last term does not depend on the path, so the expectation does not change it, and the first two are exactly the negative of the bound,

$$D_{\mathrm{KL}}(q \,\|\, p_\theta(\cdot \mid x_0)) = \log p_\theta(x_0) - \mathrm{ELBO} \;\geq\; 0.$$

"The path divergence $D_{\mathrm{KL}}(q \,\|\, p_\theta(\cdot \mid x_0))$ equals the clean-sequence log probability $\log p_\theta(x_0)$ minus the evidence lower bound ELBO. Its nonnegativity makes the ELBO a lower bound on the log probability."

The bound is tight when the conditional path laws agree on their support, matching the equality condition in the two-weight example. Maximizing it increases the data log-likelihood and brings the model reverse-path law closer to the reversed forward law.

4.11.5 Splitting the bound across steps

Both the forward corruption and reverse sampler are Markov in the step index n, so their path laws factor across the grid. We can therefore split the bound into one term per step. Each term compares the exact reverse law $q(x_{n-1} \mid x_n, x_0)$ with the learned law $p_\theta(x_{n-1} \mid x_n)$ at the same times, using the KL divergence from Section 4.10. Both reverse laws also factor across positions. Each step term is therefore a sum of L per-position divergences.

At the boundary $t = 1$, the schedule has $\alpha_1 = 0$, so the forward law puts all of its mass on the all-mask sequence and the sampler starts from exactly that sequence, leaving nothing to compare. At $t = 0$ the schedule has $\alpha_0 = 1$, so the final reverse step reveals every remaining position with probability one, and its reconstruction term takes the same form as the step terms. We obtain one KL divergence per step and per position.

4.12 Constraints on the denoiser

The denoiser must assign zero probability to the mask and preserve every visible token. We can enforce these consequences of the corruption process directly in the network output,

$$d_\theta^i(m \mid z_t, t) = 0, \qquad z_t^i \neq m \implies d_\theta^i(a \mid z_t, t) = \mathbf{1}\{a = z_t^i\}.$$

"The denoiser probability $d_\theta^i(m \mid z_t, t)$ of the mask m is zero. At a visible position i, its clean-token prediction $d_\theta^i(a \mid z_t, t)$ assigns probability one to the observed token z_t^i and zero to every other candidate a."

Together, these constraints define the substitution parameterization, SUBS, of MDLM [Sahoo et al., 2024].

The clean sequence contains no mask, so its token posterior is supported on $\mathcal{V}$ and assigns zero probability to m. We enforce this constraint by setting the mask logit to $-\infty$ before the softmax.

The second constraint is carry-over unmasking. Masking is absorbing under the forward process, so a token that is visible at time t was never masked, and it is still the original token. Its posterior is therefore a point mass on the observed letter. We can enforce it exactly by copying the input token at every visible position.

We need network predictions only at masked positions.

4.13 The reverse divergence at one position

Let's take a position that is masked at time t and write out the exact and learned reverse laws at that position side by side. Both are mixtures of the same two pieces, with the same mixing weight,

$$q_s = (1-r)\,\delta_m + r\,\delta_a, \qquad p_{\theta,s} = (1-r)\,\delta_m + r\,d_\theta, \qquad r = \frac{\alpha_s - \alpha_t}{1-\alpha_t}.$$

"The exact reverse distribution q_s combines the point mass δ_m at the mask with weight $1-r$ and the point mass δ_a at the clean token a with reveal weight r. The learned distribution $p_{\theta,s}$ keeps the same mask mass and distributes its reveal mass according to the denoiser d_θ."

- δ_m is the point mass on the mask symbol.
- δ_a is the point mass on the true clean token a at this position.
- d_θ abbreviates $d^i_\theta(\cdot \mid z_t, t)$, supported on $\mathcal{V}$ by the first SUBS constraint.

The two laws share their mask mass exactly, since r comes from the schedule and both use it. The only place they can disagree is in how the reveal mass is spread over the clean letters, where the exact law concentrates everything on a and the learned law follows the network.

We now expand the categorical divergence over the $K+1$ outcomes. Every outcome other than m and a has $q_s = 0$ there and contributes nothing, because the weight in each term of $D_{\mathrm{KL}}(q \,\|\, p)$ is the reference probability. Two terms remain,

$$D_{\mathrm{KL}}(q \,\|\, p) = (1-r)\log\frac{1-r}{1-r} \;+\; r\log\frac{r}{r\,d_\theta(a)}.$$

"The reverse divergence $D_{\mathrm{KL}}(q \,\|\, p)$ has a mask contribution weighted by the remaining-mask probability $1-r$ and a clean-token contribution weighted by the reveal probability r. Their log ratios compare identical mask probabilities and the exact reveal probability r with the model probability $r\,d_\theta(a)$ of the true token a."

The mask contribution vanishes because the first logarithm is zero. In the second term the common factor r cancels inside the logarithm, leaving

$$D_{\mathrm{KL}}(q \,\|\, p) = -\,r\log d_\theta(a).$$

"The reverse divergence $D_{\mathrm{KL}}(q \,\|\, p)$ equals the reveal probability r times the negative log probability $-\log d_\theta(a)$ that the denoiser assigns to the correct clean token a."

We have reduced the categorical divergence over $K+1$ outcomes to cross-entropy for the correct token, weighted by the schedule. With $r = 0.5$ and predicted probability 0.6 for the correct letter, the divergence is $0.5 \times 0.51083 = 0.25541$ nats. Raising that prediction to 0.9 lowers the divergence to $0.5 \times 0.10536 = 0.05268$ nats. It reaches zero when the denoiser assigns probability one to the correct letter.

4.13.1 The contribution of visible positions

At a position that is already visible at time t, both reverse laws collapse to the same point mass,

$$z_t^i = a \neq m \implies q(z_s^i \mid z_t^i, a) = p_\theta(z_s^i \mid z_t) = \delta_a \implies D_{\mathrm{KL}}(q \,\|\, p_\theta) = 0.$$

"When the observed token z_t^i equals the clean token a and is not the mask, the exact reverse law and the learned reverse law both put probability one on a through the point mass δ_a, so their KL divergence is zero."

The exact side holds because masking is absorbing, so a token visible at t was visible with the same identity at every earlier time. The learned side holds because of carry-over unmasking, the second SUBS constraint of Section 4.12, which makes the denoiser a point mass at a visible position and hence makes the reverse step copy the observed token. All of the loss comes from masked positions, which is also the property that made the letters A and G stay fixed through the trace of Section 4.9.

4.14 The masked diffusion loss

Summing the per-position result over the positions that are masked at time t defines the error of one corruption,

$$\ell_\theta(z_t, x_0) = \sum_{i \,:\, z_t^i = m} -\log d_\theta^i(x_0^i \mid z_t, t).$$

"The masked-token loss $\ell_\theta(z_t, x_0)$ adds the negative log probabilities $-\log d_\theta^i(x_0^i \mid z_t, t)$ of the true clean tokens x_0^i across positions i where the noisy sequence z_t contains the mask m."

We know the training target because we corrupt a clean sequence drawn from the data. The closed-form marginal $q_t(z \mid x)$ lets us sample the corruption directly at any time t, without simulating a trajectory.

4.14.1 The weight on a short step

The per-step divergence has the factor $r_{s,t}$ in it, which depends on the size of the step. To collect the step terms of Section 4.11 into an integral over time, we need the behaviour of $r_{s,t}$ as the step shrinks. We now put $s = t - h$ and expand the schedule, $\alpha_{t-h} = \alpha_t - h\dot{\alpha}_t + O(h^2)$,

$$r_{t-h,\,t} = \frac{\alpha_{t-h} - \alpha_t}{1 - \alpha_t} = \frac{-h\dot{\alpha}_t}{1 - \alpha_t} + O(h^2) = h\, w(t) + O(h^2), \qquad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}.$$

"For a reverse step of length h from time t to $t-h$, the reveal probability $r_{t-h,t}$ equals h times the reveal rate $w(t)$, up to an error of order h^2. The rate $w(t)$ is the negative survival derivative $-\dot{\alpha}_t$ divided by the current masking probability $1-\alpha_t$."

For the linear schedule $\alpha_t = 1-t$, the expansion has no remainder term at all, since $\dot{\alpha}_t = -1$ and $1-\alpha_t = t$ give $r_{t-h,\,t} = h/t = h\,w(t)$ exactly. At $t = 0.6$ and $h = 0.01$ both sides equal 0.016667.

The sum over a fine grid of steps therefore converges to an integral in t whose density is $w(t)$, and the per-corruption error appears in that integral multiplied by the reveal rate,

$$w(t)\,\ell_\theta = \frac{-\dot{\alpha}_t}{1-\alpha_t}\,\ell_\theta(z_t, x_0).$$

4.14.2 The loss

Averaging over sampled times, data sequences and corruptions completes the objective,

$$\boxed{\mathcal{L}_{\text{mask}} = \mathbb{E}_{t,\,x_0,\,z_t}[w(t)\,\ell_\theta(z_t, x_0)]}$$

"The training loss $\mathcal{L}_{\text{mask}}$ averages the reveal-rate weight $w(t)$ times the masked-token prediction loss $\ell_\theta(z_t, x_0)$, over noise times t, clean sequences x_0 and corrupted sequences z_t."

To estimate the loss, draw a time t uniformly on $[0,1]$ and a clean sequence x_0 from the data. Mask each position independently with probability $1-\alpha_t$ to obtain z_t, then evaluate the network once. Sum $-\log d_\theta^i(x_0^i \mid z_t, t)$ over the masked positions and multiply by $w(t)$. This requires one network evaluation per example, as in masked prediction, with the diffusion weighting supplied by $w(t)$.

Visible positions contribute zero divergence under SUBS, so the sum over positions runs only over masked ones. The endpoints of the schedule are exact, $\alpha_0 = 1$ and $\alpha_1 = 0$, so the boundary terms of the bound disappear. After those two simplifications we are left with a cross-entropy, reweighted by the schedule.

4.14.3 Loss-term example

Let's run the recipe once on DNA. We draw the time $t = 0.5$, so that the linear schedule has $\alpha_{0.5} = 0.5$ and the weight is

$$w(0.5) = \frac{-\dot{\alpha}_{0.5}}{1-\alpha_{0.5}} = \frac{1}{0.5} = 2.$$

Then we draw the clean sequence $x_0 = \text{ACGT}$ and corrupt it by masking each of the four positions independently with probability $1-\alpha_{0.5} = 0.5$. The expected number of masked positions is $4 \times 0.5 = 2$, and the sampled corruption masks positions 2 and 4, giving $z_t = \text{AmGm}$.

We now evaluate the network once on AmGm. At position 2 it returns a categorical distribution over (A, C, G, T) equal to (0.1, 0.7, 0.1, 0.1), and at position 4 it returns (0.2, 0.2, 0.1, 0.5). The true clean tokens at those two positions are C and T, so the probabilities that enter the loss are 0.7 and 0.5,

$$\ell_\theta(z_t, x_0) = -\log 0.7 - \log 0.5 = 0.35667 + 0.69315 = 1.04982 \text{ nats}.$$

Multiplying by the weight completes one sample of the objective,

$$w(0.5)\,\ell_\theta(z_t, x_0) = 2 \times 1.04982 = 2.09964 \text{ nats}.$$

Positions 1 and 3 contribute zero loss. Under carry-over unmasking, their predicted distributions are point masses on the visible letters A and G.

Gradient descent on this term raises the two probabilities 0.7 and 0.5. A later evaluation of the same corruption with 0.9 and 0.8 would give $\ell_\theta = 0.10536 + 0.22314 = 0.32850$ and a weighted term of $2 \times 0.32850 = 0.65701$ nats, less than a third of the original. The loss reaches zero only in the limit where the network assigns probability one to the correct letter at every masked position, which no model attains on real data, because several letters remain possible at most masked positions and the data itself is not deterministic.

4.14.4 Positivity of the weight

The survival probability decreases in t, so its derivative is nonpositive and the weight is nonnegative,

$$\dot{\alpha}_t \leq 0 \quad \Longrightarrow \quad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \;\geq\; 0.$$

"Because the survival probability α_t decreases, its derivative $\dot{\alpha}_t$ is nonpositive, so the loss weight $w(t)$, equal to minus that derivative divided by the masking probability one minus α_t, is nonnegative."

Minimizing positively weighted cross-entropy increases the predicted probability of the correct clean token. The weight $w(t)$ determines how strongly an error at time t contributes. For the linear schedule, $w(t) = 1/t$, giving weights 4 at $t = 0.25$, 2 at $t = 0.5$ and $4/3$ at $t = 0.75$. Low-noise examples have larger weights and fewer masked positions. These effects cancel in expectation: the expected mask count is $L(1 - \alpha_t) = Lt$, so the expected weighted error is $w(t) \cdot Lt \cdot \overline{\mathrm{CE}}(t) = L\,\overline{\mathrm{CE}}(t)$. Here $\overline{\mathrm{CE}}(t)$ is the mean cross-entropy at one masked position. Each noise level therefore contributes according to its prediction error after accounting for the mask count.

4.14.5 General corruption schedules

We now have a corruption process, a reverse step and a training loss. We corrupt a sequence by masking each position independently with probability $1 - \alpha_t$. In reverse, we reveal each masked position with probability $r_{s,t} = (\alpha_s - \alpha_t)/(1 - \alpha_t)$ and sample its letter from the denoiser. We train the denoiser using $\mathcal{L}_{\text{mask}}$, a weighted cross-entropy at masked positions.

We can specify a general continuous-time process on a finite state space by its jump rates between states. Masking, uniform replacement and block generation correspond to different rate choices, with the survival schedule α_t summarizing one aspect of the process. The reverse rates depend on both the forward rates and the marginals. We train them with a divergence between rates, which reduces to the masked loss for masking rates.

We can describe these alternatives using continuous-time Markov chains. Masked diffusion corresponds to jump rates that send every token to a single absorbing symbol.

4.15 Continuous-time transitions

We derived masked diffusion by choosing a survival schedule, reversing an interval with Bayes' rule and training a denoiser with weighted cross-entropy. Each calculation used the fact that a corrupted token equals m. To include replacement by other bases and corruption in blocks, we

need a description that allows different destinations. We can specify each discrete diffusion model through a rate matrix and a schedule. These choices distinguish MDLM, D3PM, uniform diffusion, SEDD, block diffusion, per-token schedules and simplex diffusion.

In each case, a token occupies one state of a finite alphabet, remains there for a random time and then jumps to another state. Jump rates specify what can happen over a short interval. The master equation describes how the probabilities change, and the matrix exponential solves that equation. Stationarity and detailed balance describe the equilibrium distribution.

In Chapter 1 we moved probability one step at a time with a transition matrix P, an array of nonnegative entries whose rows each sum to one. Writing probability vectors as rows,

$$p_{n+1}(y) = \sum_x p_n(x)\, P(x, y).$$

"The probability $p_{n+1}(y)$ of state y after one step equals the sum over current states x of the current probability $p_n(x)$ times the transition probability $P(x, y)$ from x to y."

That description has a built-in unit of time, namely one step. Our tokens change in continuous time, so a single matrix is not enough and we need one for every ordered pair of times,

$$P_{s,t}(x, y) = \mathbb{P}\big(X_t = y \mid X_s = x\big), \qquad s < t.$$

"The transition probability $P_{s,t}(x, y)$ is the probability that the process is at state y at time t, given that it was at state x at time s."

- X_t is the random state at time t. For one token it takes values in the alphabet $\mathcal{V}$, enlarged to the masked alphabet $\mathcal{V}_m$ of Section 4.5 when masking is in play; for a whole sequence it takes values in $\mathcal{S} = \mathcal{V}^L$.
- x, y, z are particular states, elements of that same set.
- $P_{s,t}$ is the matrix whose (x, y) entry is $P_{s,t}(x, y)$, one matrix for each pair $s < t$. Each of its rows is a probability distribution over the states reachable from x.
- p_t is the row vector of probabilities at time t, with $p_t(y) = \mathbb{P}(X_t = y)$.

We can specify this uncountable family of transition matrices through its short-interval behavior. Over an interval of length h, the state usually stays unchanged, so $P_{t,t+h}$ is close to the identity matrix I. Its leading correction is proportional to h,

$$P_{t,t+h} = I + h\, R_t + o(h).$$

"The transition matrix $P_{t,t+h}$ over a short interval h equals the identity matrix I plus h times the rate matrix R_t, up to a remainder $o(h)$ that shrinks faster than h."

We call R_t the rate matrix at time t. For an off-diagonal entry, the expansion gives

$$\mathbb{P}\big(X_{t+h} = y \mid X_t = x\big) = h\, R_t(x, y) + o(h), \qquad y \neq x.$$

"The probability of moving from the current state $X_t = x$ to a different state $X_{t+h} = y$ over a short interval h equals h times the jump rate $R_t(x, y)$, plus a remainder $o(h)$ that shrinks faster than h."

A rate is a probability change per unit time and can exceed one. Multiplying it by a short interval gives the approximate jump probability. At rate 2 over an interval of length 0.01, that probability is approximately 0.02. For a single exponential clock, the exact value is $1 - e^{-2\times 0.01} = 0.0198$, with discrepancy 0.0002 from the higher-order remainder.

4.16 Rate-matrix constraints

Every entry of R_t is constrained by the fact that $P_{t,t+h}$ has to be a transition matrix for every small h. This imposes two constraints on R_t.

Jump probabilities cannot be negative, and for $y \neq x$ the probability of landing on y is $hR_t(x, y)+o(h)$ with $h > 0$, so

$$R_t(x, y) \geq 0 \qquad \text{for } y \neq x.$$

"Every rate $R_t(x, y)$ from a state x to a different state y is greater than or equal to zero."

Each row of $P_{t,t+h}$ sums to one for every h, and each row of the identity already sums to one, so the row sums of the correction term hR_t must vanish. Dividing by h and separating the diagonal entry,

$$R_t(x, x) = -\sum_{y\neq x} R_t(x, y).$$

"The diagonal rate $R_t(x, x)$ equals negative the sum of the rates $R_t(x, y)$ from x to every other state y."

The diagonal entry is therefore minus the total rate of leaving x, and it adds no information of its own; it is fixed once the off-diagonal row is chosen. We write

$$\lambda_t(x) = \sum_{y\neq x} R_t(x, y) = -R_t(x, x)$$

for the total exit rate, which is also the rate of the exponential waiting time at x.

Definition 4.1 (Rate matrix)**.** A rate matrix on a finite state space $\mathcal{S}$ is a family of $|\mathcal{S}| \times |\mathcal{S}|$ matrices R_t, one for each time t, with $R_t(x, y) \geq 0$ for every pair of distinct states $x \neq y$ and $\sum_y R_t(x, y) = 0$ for every state x. The entry $R_t(x, y)$ with $y \neq x$ is the rate of jumping from x to y at time t, in units of probability per unit time, and $-R_t(x, x) = \lambda_t(x)$ is the total exit rate out of x.

Any array of nonnegative off-diagonal numbers becomes a rate matrix once the diagonal is filled in this way, so to build a corruption process we choose which jumps are allowed and how fast.

4.16.1 DNA rate-matrix example

Let's take a single DNA base and suppose that an A jumps to C at rate 2, and to G and to T at rate 1 each. We enter these rates off the diagonal and set the diagonal entry to their negative sum,

$$R(\mathrm{A}, \cdot\,) = \begin{array}{cccc} \mathrm{A} & \mathrm{C} & \mathrm{G} & \mathrm{T} \\ -4 & 2 & 1 & 1 \end{array}$$

"The rate-matrix row $R(\mathrm{A}, \cdot)$ contains jump rates 2, 1 and 1 from A to C, G and T. Its diagonal entry is the negative total exit rate, -4, so the row sums to zero."

The total exit rate out of A is $\lambda(\mathrm{A}) = 2 + 1 + 1 = 4$. Over the interval $h = 0.1$, the short-interval expansion turns this row of rates into a row of probabilities,

$$P_{t,t+0.1}(\mathrm{A}, \cdot\,) \approx (1, 0, 0, 0) + 0.1\,(-4, 2, 1, 1) = (0.6,\ 0.2,\ 0.1,\ 0.1).$$

"The row $P_{t,t+0.1}(\mathrm{A}, \cdot)$ of transition probabilities out of A is approximately the identity row $(1, 0, 0, 0)$ plus 0.1 times the rate row $(-4, 2, 1, 1)$, which is 0.6 for staying at A and 0.2, 0.1 and 0.1 for C, G and T."

The four probabilities sum to one. Conditional on changing from A, C is twice as likely as G or T because its rate is twice theirs. The $o(h)$ term accounts for the approximation error. For an interval of length 0.1, the exact probability of never leaving A is $e^{-4\times 0.1} = 0.6703$. The first-order value 0.6 is lower because the expansion applies the full exit rate across the interval and omits paths that leave A and return.

4.16.2 Bounds on the step size

The same first-order step breaks down once the interval grows. At $h = 0.3$, the staying probability of A would be

$$1 - 0.3 \times 4 = -0.2,$$

which is not a probability at all. The exact survival probability over that interval is $e^{-1.2} = 0.3012$, so the negative value comes from the linear approximation. If we require the diagonal entry of $I + hR_t$ to stay nonnegative at every state, we get a bound on the step,

$$h \sum_{y \neq x} R_t(x, y) \leq 1.$$

"The step size h times the sum of the rates $R_t(x, y)$ from x to every other state y is at most one, so that the probability of staying at x is not negative."

For this row, the total rate is 4, so we require $h \leq 0.25$. A fixed step size must satisfy the condition at every state. The state with the largest exit rate imposes the tightest bound.

4.17 Waiting times and destinations

With very small time steps, we repeatedly sample an unchanged state. We can simulate the jumps directly by first drawing the time until the next change.

Let $S(s)$ be the probability that an A started at time 0 has not jumped by time s. To stay until $s + h$, the base has to stay until s and then survive the interval h, which it does with probability $1 - 4h$ up to $o(h)$, since 4 is its total exit rate. The two requirements are independent given survival to s, so they multiply,

$$S(s + h) = S(s)\,(1 - 4h) + o(h).$$

"The survival probability $S(s+h)$ of staying at A through time $s+h$ equals the probability $S(s)$ of staying through time s, multiplied by the short-interval staying probability $1-4h$, up to the remainder $o(h)$."

We then subtract $S(s)$, divide by h, and let h go to zero,

$$\frac{dS}{ds} = -4\,S(s) \quad \Longrightarrow \quad S(s) = e^{-4s}.$$

"The rate of change dS/ds of the survival probability $S(s)$ equals negative 4 times $S(s)$, so the probability $S(s)$ that A has not jumped by time s equals e^{-4s}."

The waiting time in A is exponential with rate 4 and mean $1/4$ of a time unit. Repeating the calculation for a state x with exit rate $\lambda(x)$ gives an exponential waiting time with rate $\lambda(x)$ and mean $1/\lambda(x)$.

When the jump finally happens, the destination is drawn in proportion to the rates leaving the current state,

$$\mathbb{P}(\text{next state} = y) = \frac{R(\mathrm{A}, y)}{4} = \tfrac{1}{2},\ \tfrac{1}{4},\ \tfrac{1}{4} \quad \text{for C, G, T.}$$

"The probability that the next state is y equals the rate $R(\mathrm{A}, y)$ from A to y divided by 4, the total rate of leaving A, which is $1/2$ for C and $1/4$ each for G and T."

To simulate a trajectory, draw an exponential waiting time using the current state's exit rate and advance the clock by that amount. Then draw a destination from the normalized rates and repeat at the new state. This direct simulation method gives exact paths for constant rates without choosing a time step [Gillespie, 1976]. The first-order method in Section 4.16 approximates the path on a time grid, including many intervals with no jump.

We can generate each draw from a uniform random variable. Inverting $S(s) = e^{-4s}$ turns a uniform draw $u \in (0,1)$ into the waiting time $-\log(u)/4$. To choose the destination, divide the unit interval at the cumulative rate fractions 0.5, 0.75 and 1, then draw a second uniform variable v. With $u = 0.42$, the waiting time is $-\log(0.42)/4 = 0.8675/4 = 0.2169$. With $v = 0.31$, we select the first interval and jump to C. The base remains A until $t = 0.2169$. After the jump, use the exit rate and cumulative probabilities from row C of R.

The first waiting time has mean $1/4 = 0.25$ and median $\log(2)/4 = 0.1733$. More than half of the waits therefore end before the mean. The probability of jumping before $t = 0.25$ is $1 - e^{-1} = 0.632$. The probability of staying at A for the entire unit interval is $e^{-4} = 0.0183$. One base started at A and followed to $t = 1$ waits, jumps to C, waits again with the row of R for C, and ends at T (Figure 4.1).

4.17.1 Path distributions

Each trajectory consists of waiting times and jumps. For training, we need the distribution over states at each time. Let's simulate fourteen independent bases with the same rate matrix, starting most of them at A. We can estimate the distribution at any fixed time by counting the bases at A, C, G and T (Figure 4.2).

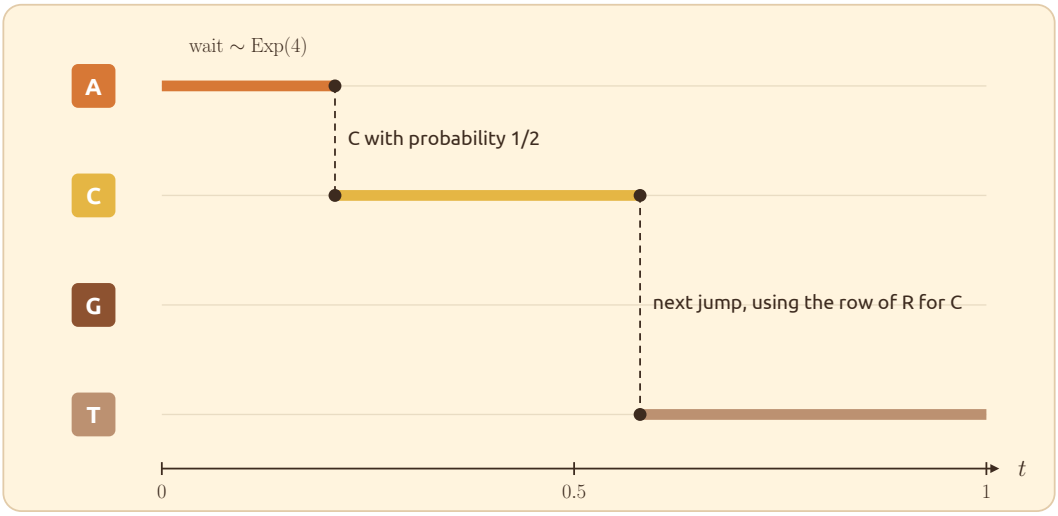


Figure 4.1: One sampled base under the rate matrix of Section 4.16, with time on the horizontal axis and the four bases as four horizontal lines. The base starts at A and holds it for a waiting time drawn from Exp(4), the exponential distribution with rate equal to the total exit rate 4 of A. At that first jump the destination is drawn in proportion to the rates out of A, and C has probability 1/2. The second waiting time and destination use the row of R for C, and the base finishes the interval at T. Dashed vertical lines mark the two jump times.

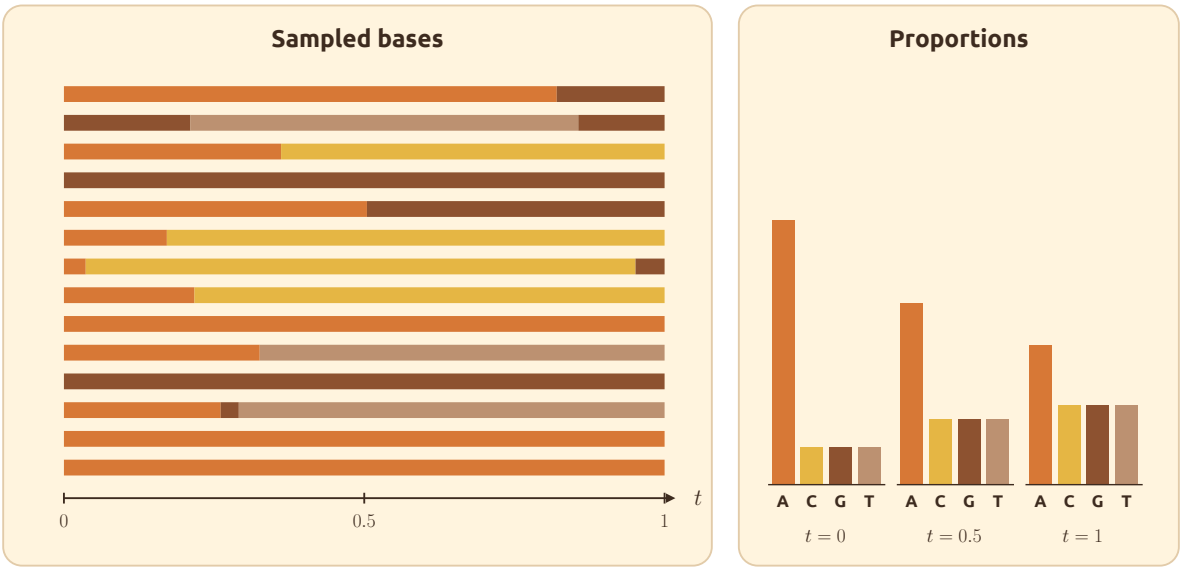


Figure 4.2: Left: fourteen bases sampled independently from the same rate matrix over $0 \le t \le 1$, one per horizontal bar, with the color of a bar segment giving the base occupied during that segment. Seven of every ten start at A. Right: the proportion of the fourteen samples at each of A, C, G and T, read off at $t = 0$, $t = 0.5$ and $t = 1$. The individual bars change by jumps at random times, while the proportions move smoothly and flatten toward equal occupancy of the four bases by $t = 1$.

Individual trajectories change through jumps, while their state probabilities change smoothly. We can train against this probability evolution.

4.18 The master equation

We need a differential equation for the probability vector p_t, whose entries are the limiting proportions in Figure 4.2. In Chapter 2, we related density changes to the divergence of the probability flux $p_t(x)v_t(x)$. For jumps, we define a flux on each ordered pair of states,

$$j_t(x \to y) = p_t(x)\, R_t(x, y).$$

"The probability flux $j_t(x \to y)$ from x to y equals the probability $p_t(x)$ of being at x times the jump rate $R_t(x, y)$ from x to y."

The flux from x to y is zero if either $p_t(x) = 0$ or $R_t(x, y) = 0$. Summing the incoming and outgoing fluxes gives the change in probability at a state.

4.18.1 Derivation from the short-interval transition

Let's write the probability of y after a short interval as a sum over every current state, and substitute the expansion $P_{t,t+h} = I + hR_t + o(h)$ of Section 4.15,

$$p_{t+h}(y) = \sum_x p_t(x)\, P_{t,t+h}(x, y) = p_t(y) + h \sum_x p_t(x)\, R_t(x, y) + o(h).$$

"The probability $p_{t+h}(y)$ of state y after an interval h sums the current-state probabilities $p_t(x)$ times the transition probabilities $P_{t,t+h}(x, y)$. Expanding the transition matrix gives the current probability $p_t(y)$ plus h times the sum of probability-weighted rates $p_t(x)R_t(x, y)$, up to the remainder $o(h)$."

The term $p_t(y)$ came from the identity, since its (x, y) entry vanishes unless $x = y$. Subtracting that term from both sides, dividing by h and letting h go to zero, we get

$$\frac{d}{dt}\, p_t(y) = \sum_x p_t(x)\, R_t(x, y).$$

"The rate of change of the state probability $p_t(y)$ equals the sum over states x of their probabilities $p_t(x)$ times the rate-matrix entries $R_t(x, y)$ into y, including the diagonal entry when $x = y$."

This is the Kolmogorov forward equation, also called the master equation for jump processes. Its right-hand side is linear in p_t.

4.18.2 Arrivals and departures

The sum over x on the right includes the term $x = y$, whose coefficient is the diagonal rate. If we separate that term and substitute $R_t(y, y) = -\sum_{x \neq y} R_t(y, x)$, the equation splits into two pieces with opposite signs,

$$\frac{d}{dt}\, p_t(y) = \underbrace{\sum_{x \neq y} p_t(x)\, R_t(x, y)}_{\text{arrives at } y} \; - \; \underbrace{\sum_{x \neq y} p_t(y)\, R_t(y, x)}_{\text{leaves } y}$$

"The rate of change of the probability $p_t(y)$ of being in state y equals the sum over other states x of their probabilities $p_t(x)$ times their jump rates $R_t(x, y)$ into y, minus the probability $p_t(y)$ times the sum of its outgoing jump rates $R_t(y, x)$."

Each term is one of the fluxes $j_t(x \to y)$ or $j_t(y \to x)$ defined above, so the probability at a state grows at the total flux in and shrinks at the total flux out. If we stack the equations for all states and write p_t as a row vector, the whole system collapses into one matrix equation,

$$\boxed{\dot{p}_t = p_t\, R_t}$$

"The time derivative $\dot{p}_t$ of the probability row vector p_t equals that vector multiplied by the rate matrix R_t, combining all incoming and outgoing probability fluxes."

We multiply R_t on the left by the row vector p_t. The yth entry of this product therefore uses column y of R_t, containing the rates into y. The density equation in Chapter 2 balanced flux through a region's boundary. Here we balance flux along the edges entering and leaving each state.

4.18.3 Conservation of total probability

A differential equation on a vector of probabilities has to preserve their sum. Summing the master equation over every state y and exchanging the order of the two sums,

$$\frac{d}{dt}\sum_y p_t(y) = \sum_x p_t(x) \sum_y R_t(x,y) = 0.$$

"The total probability changes at the sum over states x of the probability $p_t(x)$ times the row sum $\sum_y R_t(x,y)$ of the rate matrix. Every row sum is zero, so the total $\sum_y p_t(y)$ stays constant."

The zero-row-sum condition in Section 4.16 ensures conservation of probability. Each flux $j_t(x \to y)$ contributes positively to the equation for y and negatively to the equation for x. Summing over states cancels these contributions, so probability moves between states without being created or destroyed.

4.18.4 DNA probability evolution

Let's take a single base with the four states A, C, G, T, and let every base jump to each other base at rate 1. Each off-diagonal entry of R is then 1, and the row-sum rule fixes each diagonal entry at -3. We start from

$$p_t = (0.1,\ 0.2,\ 0.3,\ 0.4) \quad \text{for (A, C, G, T).}$$

"The probability vector p_t assigns probabilities 0.1, 0.2, 0.3 and 0.4 to A, C, G and T, respectively."

We now compute the balance at A. Probability $0.2 + 0.3 + 0.4 = 0.9$ is at the other three bases and each part of it flows toward A at rate 1, while the probability 0.1 at A flows out at its total rate 3,

$$\dot{p}_t(\mathrm{A}) = (0.2 + 0.3 + 0.4)(1) - (0.1)(3) = 0.6.$$

"The rate of change $\dot{p}_t(\mathrm{A})$ of the probability of A equals its incoming flux, $(0.2 + 0.3 + 0.4)$ times the rate 1, minus its outgoing flux, the probability 0.1 times the total exit rate 3. The net rate is 0.6."

The same arithmetic at every base has a compact form, because the flux into y is $1 - p_t(y)$ and the flux out of y is $3p_t(y)$,

$$\dot{p}_t(y) = 1 - 4\,p_t(y), \qquad \dot{p}_t = (0.6,\ 0.2,\ -0.2,\ -0.6).$$

"For each base y, its probability changes at rate $\dot{p}_t(y) = 1 - 4p_t(y)$, where $p_t(y)$ is its current probability. For A, C, G and T, these rates are 0.6, 0.2, -0.2 and -0.6."

Taking one Euler step with $h = 0.1$, inside the bound $h \le 1/3$ set by the total exit rate 3, we get

$$p_t + 0.1\,\dot{p}_t = (0.16,\ 0.22,\ 0.28,\ 0.34).$$

"An Euler step of length 0.1 adds the probability-change vector $0.1\dot{p}_t$ to the current vector p_t, giving probabilities 0.16, 0.22, 0.28 and 0.34 for A, C, G and T."

The four changes are $+0.06$, $+0.02$, -0.02 and -0.06, which sum to zero, so the stepped vector still sums to one. Rare bases gain probability and common bases lose it, and the fixed point of $\dot{p}_t(y) = 1 - 4p_t(y)$ is $p_t(y) = 1/4$ at every base. Each coordinate obeys a scalar linear equation with that fixed point, whose exact solution is

$$p_t(y) = \tfrac{1}{4} + \left(p_0(y) - \tfrac{1}{4}\right) e^{-4t},$$

so each coordinate approaches $1/4$ with time constant $1/4$. At $t = 0.1$ the exact values are $(0.1495,\ 0.2165,\ 0.2835,\ 0.3505)$, which the Euler step reproduced to within 0.011, and by $t = 1$ they have reached $(0.2473,\ 0.2491,\ 0.2509,\ 0.2527)$, which is the flattening visible in the proportions of Figure 4.2.

We can represent the balance at A on a state graph by labeling each edge with its flux (Figure 4.3).

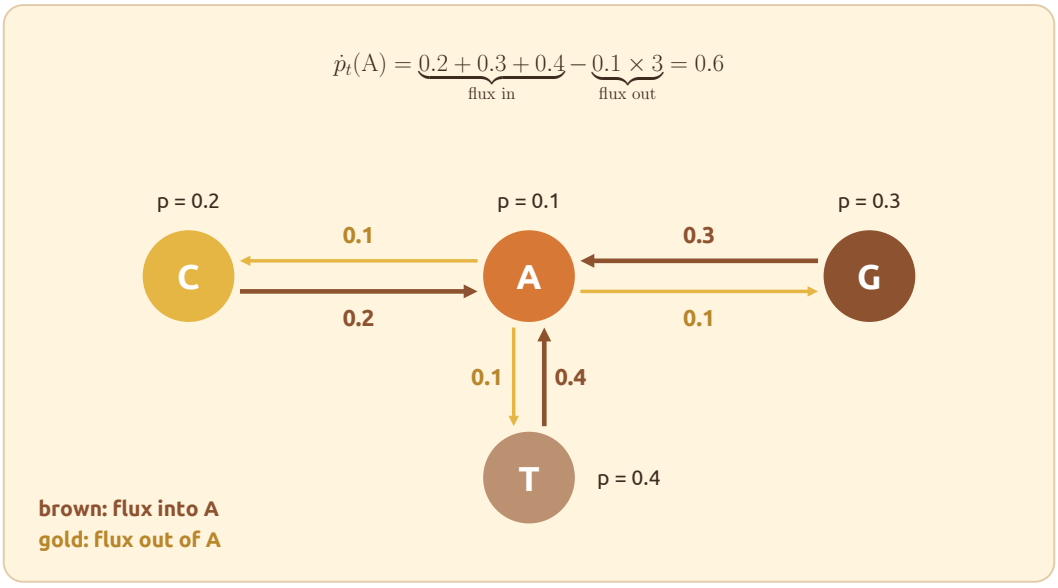


Figure 4.3: The master equation at A for a single base with all pairwise rates equal to 1 and probabilities $p = (0.1, 0.2, 0.3, 0.4)$ at A, C, G and T. Each node carries its probability and each arrow its flux $j_t(x \to y) = p_t(x)R_t(x, y)$. Brown arrows carry flux into A and total $0.2 + 0.3 + 0.4 = 0.9$; gold arrows carry flux out of A and total $3 \times 0.1 = 0.3$, since the probability 0.1 at A leaves toward each of the three other bases at rate 1. The equation above the graph is the resulting $\dot{p}_t(\mathrm{A}) = 0.6$.

During training, we draw a random corruption time t for each example. A closed-form kernel lets us sample the corrupted state directly at that time.

4.19 The matrix exponential

The master equation is a linear differential equation in p_t, which is the kind we solved in Chapter 1 with the matrix exponential. We can hold the rate matrix fixed in time, so that

$$\dot{p}_t = p_t \, R,$$

and the solution follows from multiplying the initial vector by one matrix,

$$p_t = p_0 \, e^{tR}, \qquad e^{tR} = \sum_{k=0}^{\infty} \frac{t^k R^k}{k!}.$$

"The probability vector p_t equals the initial vector p_0 times the matrix exponential e^{tR}, which is the sum over k from zero to infinity of t^k times R^k divided by k factorial."

The series converges for every square matrix, since the norms of its terms are bounded by those of the scalar series for $e^{t\|R\|}$. If we differentiate it term by term, we can confirm that it solves the equation,

$$\frac{d}{dt} \, e^{tR} = \sum_{k=1}^{\infty} \frac{t^{k-1} R^k}{(k-1)!} = e^{tR} R.$$

"The time derivative of the transition matrix e^{tR} equals that matrix multiplied by the fixed rate matrix R. Differentiating the series gives the sum of $t^{k-1}R^k/(k-1)!$ over positive integers k, where t is elapsed time."

If we take p_0 to be the indicator row vector of a single state x, we pick out one row, so the transition matrix over an interval of length t is

$$P_{0,t} = e^{tR},$$

which is the two-time transition matrix defined in Section 4.15.

4.19.1 The entries of the matrix

To use e^{tR} as a transition matrix, we need nonnegative entries and row sums of one. We can check these properties even though R has negative diagonal entries. Let $\mathbf{1}$ be the column vector of ones. Every row of R sums to zero, which says $R\mathbf{1} = 0$, and then every power satisfies $R^k\mathbf{1} = R^{k-1}(R\mathbf{1}) = 0$ for $k \geq 1$. If we apply the series to $\mathbf{1}$, only its first term remains,

$$e^{tR}\mathbf{1} = \mathbf{1} + \sum_{k \geq 1} \frac{t^k R^k \mathbf{1}}{k!} = \mathbf{1}.$$

"Multiplying the transition matrix e^{tR} by the all-ones vector $\mathbf{1}$ gives the vector of row sums. In the exponential series, every term $R^k\mathbf{1}$ vanishes for $k \geq 1$, leaving $\mathbf{1}$, so every row sum is one."

The argument for nonnegativity is different, because R itself has negative diagonal entries. We can choose a number λ at least as large as every total exit rate $\lambda(x)$, so that $R + \lambda I$ has no negative entry anywhere. The identity commutes with R, so the exponential splits,

$$e^{tR} = e^{-\lambda t} \, e^{t(R + \lambda I)}.$$

"The transition matrix e^{tR} equals the positive scalar $e^{-\lambda t}$ times the exponential $e^{t(R+\lambda I)}$ of the shifted rate matrix. Here t is elapsed time, I is the identity and λ is at least the largest total exit rate."

The exponential of $t(R + \lambda I)$ has nonnegative entries because every power in its series does. Multiplying by the positive scalar $e^{-\lambda t}$ preserves nonnegativity. Together with the row sums of one, this proves that e^{tR} is a transition matrix for every $t \geq 0$.

4.19.2 Composing two intervals

We also need transitions between arbitrary noise levels. For a transition from x at time s to z at time u, we can sum over the state y at an intermediate time t,

$$P_{s,u}(x, z) = \sum_{y} P_{s,t}(x, y)\, P_{t,u}(y, z).$$

"The transition probability $P_{s,u}(x, z)$ from x at time s to z at time u equals the sum over every intermediate state y of $P_{s,t}(x, y)$, the probability of reaching y at time t, times $P_{t,u}(y, z)$, the probability of then reaching z at time u."

The sum over y is a matrix product, so this is the Chapman-Kolmogorov equation, and for a fixed rate matrix it is the familiar law of exponents,

$$P_{s,u} = P_{s,t}\, P_{t,u}, \qquad e^{(u-s)R} = e^{(t-s)R}\, e^{(u-t)R}.$$

"The transition matrix $P_{s,u}$ from time s to time u equals the product of the transition matrices $P_{s,t}$ and $P_{t,u}$ for the two intervals separated by time t. For a fixed rate matrix R, the corresponding exponentials multiply to $e^{(u-s)R}$."

For the masking kernel in Section 4.5, we multiplied survival probabilities across intervals. The Chapman-Kolmogorov equation extends this composition to arbitrary rate matrices.

4.19.3 Rates that change with time

A diffusion model corrupts quickly at some times and slowly at others, so the rate matrix has to depend on t. In the simplest way to achieve that, we keep one fixed matrix R and scale it by a positive scalar schedule $\beta(t)$, accumulating that scale into a clock,

$$R_t = \beta(t)\, R, \qquad \tau(t) = \int_0^t \beta(s)\, ds.$$

"The time-dependent rate matrix R_t equals the fixed matrix R scaled by the rate schedule $\beta(t)$. Integrating that schedule from time zero to t gives the accumulated rate $\tau(t)$."

- R is a fixed rate matrix, whose nonzero off-diagonal entries are the jumps the process may make and in what proportion.
- $\beta(t) > 0$ is the schedule, a scalar function of time with no matrix structure of its own.

- $\tau(t)$ is the accumulated rate, with $\tau(0) = 0$ and $\dot{\tau}(t) = \beta(t)$. It is the amount of corruption applied by time t, measured in the natural units of R.

We choose the allowed jumps and their relative rates through R, then set their overall speed through $\beta(t)$. We can make these choices independently.

Because R_t is a scalar multiple of one fixed matrix, it commutes with itself at every pair of times, and the solution of $\dot{p}_t = p_t R_t$ is still a single exponential with the clock in place of the elapsed time,

$$P_{0,t} = e^{\tau(t)\, R}.$$

"The transition matrix $P_{0,t}$ from time 0 to time t equals the matrix exponential of the clock $\tau(t)$ times the fixed rate matrix R."

If $R_s R_t \neq R_t R_s$ for some times, we cannot use this single-exponential formula. The transition matrix is the limit of the ordered product $\prod_k (I + h R_{t_k})$, with factors arranged from left to right in increasing time. This limit is the time-ordered exponential. A scalar schedule avoids this complication because all rate matrices commute.

The step-size condition from Section 4.16 becomes $h\beta(t)\lambda(x) \leq 1$, where $\lambda(x)$ is the exit rate of the fixed matrix. Faster corruption therefore requires smaller steps. For the linear schedule $\alpha_t = 1 - t$, we have $\beta(t) = 1/(1-t)$. At $t = 0.75$, this gives $\beta = 4$. If the largest fixed exit rate is 0.75, the total rate is 3 and we need $h \leq 1/3$, compared with $h \leq 4/3$ at $t = 0$.

4.20 Rate matrices with a reference distribution

To choose R, consider a corruption in which we redraw the token from a fixed distribution π at rate one. This draw is independent of the current token. Each row of the rate matrix equals π, with one subtracted from its diagonal entry,

$$\boxed{R_\pi = \mathbf{1}\pi^\top - I}$$

"The rate matrix R_π equals the redraw matrix $\mathbf{1}\pi^\top$ minus the identity I. The all-ones column $\mathbf{1}$ repeats the probability row $\pi^\top$ in every row of the redraw matrix."

- π is a probability distribution over the states, written as a column so that $\pi^\top$ is a row; it is the only freedom in the family.
- $\mathbf{1}\pi^\top$ is the rank-one matrix whose every row equals $\pi^\top$.
- $R_\pi(x, y) = \pi(y)$ for $y \neq x$, and $R_\pi(x, x) = \pi(x) - 1$.

Both rules of Section 4.16 hold. The off-diagonal entries $\pi(y)$ are nonnegative because π is a distribution, and each row sums to $1 - 1 = 0$. The total exit rate out of x is $\lambda(x) = 1 - \pi(x)$, which is at most one, so a step of size $h \leq 1$ is always safe under this family.

To simplify the exponential, multiply the redraw matrix $M = \mathbf{1}\pi^\top$ by itself,

$$M^2 = \mathbf{1}(\pi^\top \mathbf{1})\, \pi^\top = M.$$

"The redraw matrix $M = \mathbf{1}\pi^\top$ satisfies $M^2 = M$, because the inner product $\pi^\top \mathbf{1}$ sums the redraw probabilities to one. Applying the redraw matrix twice therefore gives the same distribution as applying it once."

A matrix with $M^2 = M$ is idempotent, and by induction every power M^k equals M. Drawing from π twice in a row has the same law as drawing once, which is the same statement.

The redraws themselves form a Poisson process of rate one in clock time, since the rate of a redraw is one at every state. A token is still clean at clock time τ when no redraw has occurred, and that has probability $e^{-\tau}$, which gives the survival probability directly. The expected number of redraws by $\tau = \log 2 = 0.693$ is 0.693, and exactly half of the tokens have had none.

The example in Section 4.18 belongs to this family. Its off-diagonal rates are 1 and its diagonal entries are -3, giving the matrix $4R_\pi$ for uniform π over four bases. Equivalently, use R_π with constant schedule $\beta(t) = 4$. The accumulated rate is $\tau(t) = 4t$, so the survival probability is $\alpha_t = e^{-4t}$. At $t = 0.1$, this is 0.6703, reproducing the exact solution.

4.20.1 The matrix exponential of R_π

We can now compute $e^{\tau R_\pi}$ with $R_\pi = M - I$. The identity commutes with M, so the exponential of the difference splits into a scalar factor and a matrix factor,

$$e^{\tau(M-I)} = e^{-\tau}\, e^{\tau M}.$$

"The transition matrix $e^{\tau(M-I)}$ at accumulated rate τ factors into the scalar $e^{-\tau}$ and the matrix exponential $e^{\tau M}$, because the redraw matrix M commutes with the identity I."

Every power M^k equals M, so the series for $e^{\tau M}$ collapses onto a single matrix with a scalar coefficient,

$$e^{\tau M} = I + \sum_{k \geq 1} \frac{\tau^k}{k!}\, M = I + (e^\tau - 1)\, M.$$

"The exponential $e^{\tau M}$ of the redraw matrix M equals the identity I plus M multiplied by the scalar series $\sum_{k \geq 1} \tau^k / k!$. This series equals $e^\tau - 1$, where τ is the accumulated rate."

Multiplying the two results and using $e^{-\tau}(e^\tau - 1) = 1 - e^{-\tau}$,

$$e^{\tau R_\pi} = e^{-\tau}\, I + (1 - e^{-\tau})\, \mathbf{1}\pi^\top.$$

"The matrix exponential of τ times R_π, the transition matrix after accumulated rate τ, equals $e^{-\tau}$ times the identity matrix I, plus one minus $e^{-\tau}$ times $\mathbf{1}\pi^\top$, the matrix whose rows all equal π."

Each row is a mixture of two distributions. With probability $e^{-\tau}$, we retain the initial token; with the remaining probability, we draw from π. We can therefore calculate the exponential as a convex combination of two known matrices.

4.20.2 The noise kernel and its schedule

Write the survival probability as $\alpha_t = e^{-\tau(t)}$, using the notation from Section 4.5. Each row of the transition matrix is then

$$\boxed{q_t(x \mid x_0) = \alpha_t\, \delta(x, x_0) + (1 - \alpha_t)\, \pi(x)}$$

"The probability $q_t(x \mid x_0)$ of the noisy token x given the clean token x_0 equals the survival probability α_t times $\delta(x, x_0)$, which is one when x equals x_0 and zero otherwise, plus one minus α_t times $\pi(x)$, the probability that π assigns to x."

- x_0 is the clean token at a position, the value the sequence has at $t = 0$.
- x is the noisy token at that position at time t.
- $\alpha_t = e^{-\tau(t)}$ is the survival probability, decreasing from $\alpha_0 = 1$ because τ increases.
- $\delta(x, x_0)$ is the Kronecker delta on the state space.

This is the corruption of the masked derivation with π in place of the mask. Since the clock grows at rate $\beta(t)$ and $\alpha_t = e^{-\tau(t)}$, differentiating the logarithm converts any schedule for α_t into the rate that produces it,

$$\beta(t) = -\frac{\dot{\alpha}_t}{\alpha_t}.$$

"The corruption rate $\beta(t)$ equals the negative time derivative $-\dot{\alpha}_t$ of the survival probability, divided by the survival probability α_t."

Every survival schedule α_t corresponds to exactly one rate function $\beta(t)$, so the two descriptions are equivalent and we may design in whichever is convenient. The linear schedule $\alpha_t = 1 - t$ is the common choice, and it has $\beta(t) = 1/(1 - t)$ and $\tau(t) = -\log(1 - t)$. At $t = 0.5$ the survival probability is 0.5, the instantaneous rate is 2, and the accumulated rate is $\tau = \log 2 = 0.6931$.

The composition rule of Section 4.19 has a clean form in this family. Here $Q(\alpha) = \alpha I + (1 - \alpha)\mathbf{1}\pi^\top$ is the kernel at survival probability α. Using $I\mathbf{1}\pi^\top = \mathbf{1}\pi^\top$ and $M^2 = M$, the cross terms all collapse onto M,

$$Q(\alpha_1)\, Q(\alpha_2) = \alpha_1\alpha_2\, I + \big(1 - \alpha_1\alpha_2\big)\, \mathbf{1}\pi^\top = Q(\alpha_1\alpha_2).$$

"Composing the kernels $Q(\alpha_1)$ and $Q(\alpha_2)$, with survival probabilities α_1 and α_2, gives the kernel $Q(\alpha_1\alpha_2)$. Survival probabilities therefore multiply across successive intervals."

To obtain the transition from s to t, divide the survival probabilities: $\alpha_{t|s} = \alpha_t/\alpha_s$. This agrees with the earlier calculation for masking. For $\alpha_s = 0.8$ and $\alpha_t = 0.5$, the interval survival probability is $0.5/0.8 = 0.625$. With uniform π over the four bases, the two A-to-A entries are 0.85 and 0.71875. Multiplying the matrices gives $0.85 \times 0.71875 + 3 \times (0.05 \times 0.09375) = 0.625$, matching the A-to-A entry of $Q(0.5)$.

4.20.3 Consistency with the master equation

We can verify the kernel by substituting into $\dot{q}_t = q_t R_t$, with $R_t = \beta(t) R_\pi$ and $\beta = -\dot{\alpha}_t/\alpha_t$. First, differentiate $q_t(x \mid x_0) = \alpha_t \delta(x, x_0) + (1 - \alpha_t)\pi(x)$ with respect to t, holding x_0 fixed,

$$\dot{q}_t(x \mid x_0) = \dot{\alpha}_t \left(\delta(x, x_0) - \pi(x)\right).$$

"The time derivative $\dot{q}_t(x \mid x_0)$ of the noisy-token probability equals the survival derivative $\dot{\alpha}_t$ times the difference between the clean-token indicator $\delta(x, x_0)$ and the redraw probability $\pi(x)$. Probability moves from the clean token toward the redraw distribution as survival decreases."

For the right-hand side, write $q_t(\cdot \mid x_0)$ as a row vector. Its entries sum to one, so $q_t M = (q_t \mathbf{1})\pi^\top = \pi^\top$, and the product with the rate matrix collapses,

$$q_t\, R_t = \beta(t)\, (q_t M - q_t) = \beta(t)\, (\pi^\top - q_t) = \beta(t)\, \alpha_t\, (\pi^\top - \delta(\cdot, x_0)),$$

where in the last step we substituted $q_t = \alpha_t \delta(\cdot, x_0) + (1 - \alpha_t)\pi^\top$ and cancelled the common part of the two $\pi^\top$ terms. Since $\beta(t)\alpha_t = -\dot{\alpha}_t$, the two sides agree. The kernel in terms of α_t and the rate matrix in terms of $\beta(t)$ therefore describe the same process. Probability moves from the clean token to π, with speed determined by $\dot{\alpha}_t$ and proportions determined by π.

4.20.4 Masking and uniform replacement

We recover masking and uniform replacement through two choices of π. For masking, we put all of π on the mask token m, which enlarges the alphabet to $\mathcal{V}_m$,

$$\pi = \delta_m : \qquad q_t(x \mid x_0) = \alpha_t\, \delta(x, x_0) + (1 - \alpha_t)\, \delta(x, m).$$

"For masking, the redraw distribution $\pi = \delta_m$ assigns probability one to the mask m. The noisy-token probability $q_t(x \mid x_0)$ equals the survival probability α_t times the clean-token indicator $\delta(x, x_0)$, plus the masking probability $1 - \alpha_t$ times the mask indicator $\delta(x, m)$."

The corresponding rate matrix has $R_\pi(x, m) = 1$ for every token $x \neq m$, zero rate between any two tokens, and $R_\pi(m, m) = \pi(m) - 1 = 0$, so the mask is absorbing and a masked token stays masked. This is the corruption of Section 4.5, and the loss built on it is that of MDLM [Sahoo et al., 2024].

For uniform replacement, we spread π over all K tokens of the vocabulary and keep the alphabet as it is,

$$\pi = \tfrac{1}{K}\mathbf{1} : \qquad q_t(x \mid x_0) = \alpha_t\, \delta(x, x_0) + \frac{1 - \alpha_t}{K}.$$

"For a uniform redraw distribution π over K tokens, the noisy-token probability $q_t(x \mid x_0)$ equals the survival probability α_t times the clean-token indicator $\delta(x, x_0)$, plus the redraw probability $1 - \alpha_t$ divided by the vocabulary size K."

Here every pair of distinct tokens is joined by a rate $1/K$, and the diagonal is $1/K - 1 = -(K-1)/K$. With masking we erase the identity of a token, and the corrupted sequence marks which positions are unknown. With uniform replacement we hide that identity among the other tokens, and the corrupted sequence looks like an ordinary sequence with errors in it. Both kernels follow the same α_t, and the lost probability spreads according to π (Figure 4.4).

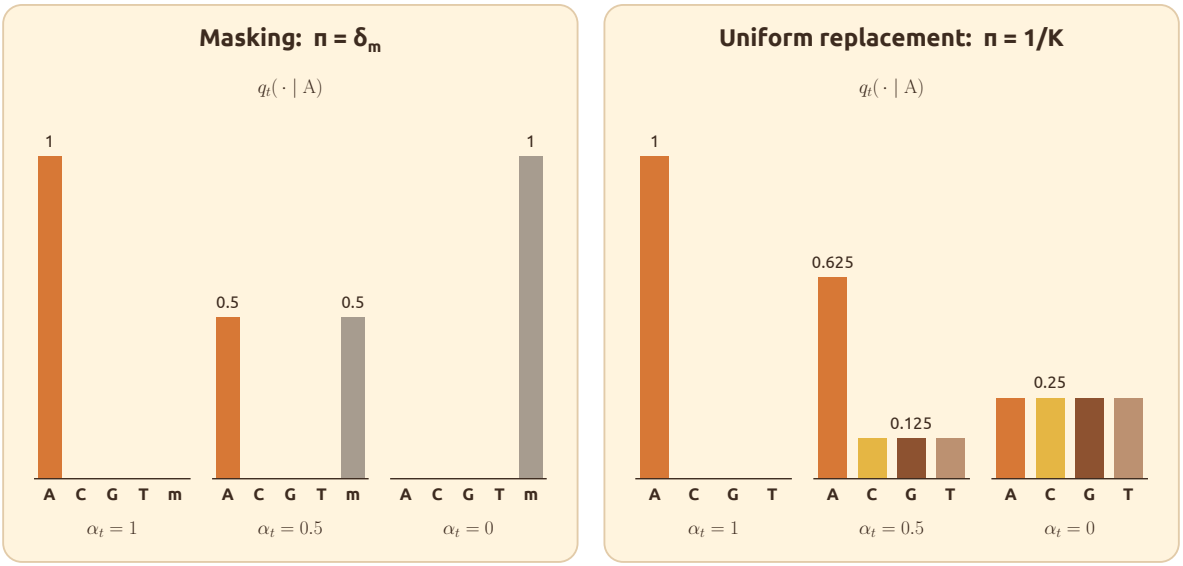


Figure 4.4: The kernel $q_t(\cdot \mid \mathrm{A})$ out of a clean A, drawn as bars over the states, at three survival probabilities. Left, masking with $\pi = \delta_m$ on the alphabet A, C, G, T, m: the bar at A has height α_t and the bar at m has height $1 - \alpha_t$, giving 1 at A when $\alpha_t = 1$, 0.5 and 0.5 when $\alpha_t = 0.5$, and all the mass at m when $\alpha_t = 0$. Right, uniform replacement with $\pi = \mathbf{1}/K$ and $K = 4$: the bar at A has height $\alpha_t + (1 - \alpha_t)/4$ and each other bar $(1 - \alpha_t)/4$, giving 0.625 at A against 0.125 elsewhere when $\alpha_t = 0.5$, and 0.25 everywhere when $\alpha_t = 0$.

4.20.5 DNA matrix exponential example

Let's carry out the calculation once with numbers. Take $\mathcal{V} = \{A, C, G, T\}$, so $K = 4$, uniform $\pi = \mathbf{1}/4$, and accumulated rate $\tau = \log 2$, the value the linear schedule reaches at $t = 0.5$, for which $\alpha = e^{-\tau} = 1/2$. Substituting into the closed form,

$$e^{\tau R_\pi} = \tfrac{1}{2} I + \tfrac{1}{2} \cdot \tfrac{1}{4} \mathbf{1}\mathbf{1}^\top = \begin{pmatrix} 0.625 & 0.125 & 0.125 & 0.125 \\ 0.125 & 0.625 & 0.125 & 0.125 \\ 0.125 & 0.125 & 0.625 & 0.125 \\ 0.125 & 0.125 & 0.125 & 0.625 \end{pmatrix}$$

"With survival probability $\alpha = 1/2$, the transition matrix $e^{\tau R_\pi}$ is an equal mixture of the identity I and the uniform redraw matrix $\mathbf{1}\mathbf{1}^\top/4$. Its diagonal entries are 0.625, and every off-diagonal entry is 0.125."

A clean A stays A with probability 0.625. It survives half the time, contributing 0.5, and one quarter of the other half consists of fresh draws that land on A again, contributing $0.5 \times 0.25 = 0.125$. Each row sums to $0.625 + 3 \times 0.125 = 1$, and the matrix is symmetric because the four bases have equal probability under uniform π.

If we split the same accumulated rate into n Euler steps of size τ/n and multiply the resulting transition matrices, we approach this value from below,

n	1	2	10	∞
$(I + \frac{\tau}{n} R_\pi)^n(\mathrm{A}, \mathrm{A})$	0.480	0.570	0.616	0.625

"The entry from A to A of $(I+\tau R_\pi/n)^n$, the product of n Euler steps of size τ/n, is 0.480 for $n=1$, 0.570 for $n=2$ and 0.616 for $n=10$, and it approaches the exact value 0.625."

One step of size $\log 2 = 0.693$ is already near the stability bound of Section 4.16, and it misses the exact entry by 0.145; ten steps miss it by 0.009. We can calculate kernels using either discrete-time transitions or continuous-time rates. In D3PM, we multiply a chosen transition matrix once per discrete step [Austin et al., 2021]. For continuous-time models, we exponentiate a rate matrix to calculate the kernel at any t in one evaluation.

With $\pi = \delta_m$ on the five states A, C, G, T, m and $\tau = \log 2$, the same closed form gives a matrix with 0.5 on the first four diagonal entries, 0.5 in the last column of those four rows, and zeros in every other off-diagonal position,

$$e^{\tau R_{\delta_m}} = \tfrac{1}{2} I + \tfrac{1}{2}\,\mathbf{1}\delta_m^\top = \begin{pmatrix} 0.5 & 0 & 0 & 0 & 0.5 \\ 0 & 0.5 & 0 & 0 & 0.5 \\ 0 & 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

"At accumulated rate $\tau = \log 2$ the masking kernel keeps each of the four bases with probability 0.5 and sends the other 0.5 to the mask, and its last row is the indicator of the mask, so a masked position stays masked."

The last row keeps a masked token at the absorbing state. Every other column receives probability only from its own starting token. At the same accumulated rate τ, masking moves probability 0.5 from the clean base to m. Uniform replacement moves 0.375 from the clean base, assigning 0.125 to each alternative. Another 0.125 comes from redraws that return to A. Since corruption is independent across positions, as in Section 4.5, a sequence of length $L = 100$ has an expected $L(1-\alpha_t) = 50$ masks at this noise level. Under uniform replacement, we expect 50 redraws, of which about 12.5 return to the original base and are indistinguishable from unchanged positions.

4.21 Eigenvalues and the stationary distribution

We start generation from the corruption endpoint, which we can determine from the eigenvalues of R_π. Because $M^2 = M$, each eigenvalue μ of M satisfies $\mu^2 = \mu$, so it is either 0 or 1. The rank-one matrix M has eigenvalue 1 once, with left eigenvector $\pi^\top$. Subtracting the identity shifts each eigenvalue down by one,

$$\text{eigenvalues of } R_\pi: \qquad 0 \ \ \text{(once)}, \qquad -1 \ \ (K-1 \text{ times}).$$

"The rate matrix R_π has the eigenvalue 0 once and the eigenvalue -1 repeated $K-1$ times, where K is the number of tokens."

In Chapter 1 we established how a linear system behaves along each eigendirection. A component with a zero eigenvalue stays constant, and one with a negative eigenvalue μ decays like $e^{\mu\tau}$. The zero direction here holds the distribution the process ends at, and every other direction decays like $e^{-\tau}$, which is the survival probability α_t. The entire approach to the endpoint therefore depends on one number, and α_t alone parameterizes this family.

Let the survival probability go to zero. Every row of $e^{\tau R_\pi}$ approaches π, whatever token we started from, so the corrupted token at the end of the forward process holds no information about the clean one. A distribution that does not change under the dynamics is called stationary,

$$\pi R = 0 \quad \Longleftrightarrow \quad \sum_x \pi(x)\, R(x,y) = 0 \;\text{ for every } y.$$

"The stationary distribution π satisfies $\pi R = 0$, where R is the rate matrix. At every state y, summing the probabilities $\pi(x)$ times the rate entries $R(x,y)$ over states x gives zero net probability change."

From the master equation, stationarity requires $\dot{p}_t = 0$ at $p_t = \pi$, so π is a left eigenvector with eigenvalue zero. For this family, every row of M equals $\pi^\top$, giving $\pi M = \pi$. Hence the chosen redraw distribution is stationary,

$$\pi M = \pi \quad \Longrightarrow \quad \pi R_\pi = \pi M - \pi = 0.$$

"The redraw matrix M preserves the distribution π, so $\pi M = \pi$. Since the rate matrix is $R_\pi = M - I$, the net change $\pi R_\pi = \pi M - \pi$ is zero."

Masking ends at the all-mask sequence, uniform noise ends at uniformly random tokens, and generation starts from those. By choosing π, we specify both the corruption endpoint and the prior for generation.

4.22 Detailed balance

At stationarity, the total incoming and outgoing fluxes at each state are equal. Detailed balance requires equality on each edge. A rate matrix R satisfies detailed balance with respect to π when

$$\pi(x)\, R(x,y) = \pi(y)\, R(y,x) \qquad \text{for all } x, y.$$

"The stationary probability $\pi(x)$ times the rate $R(x,y)$ from x to y equals the stationary probability $\pi(y)$ times the rate $R(y,x)$ from y back to x, for every pair of states x and y."

In the flux notation of Section 4.18, the condition reads $j(x \to y) = j(y \to x)$ when the process is at π, so that as much probability crosses each edge in one direction as in the other. We can state the two conditions explicitly.

Definition 4.2 (Stationarity and detailed balance)**.** Let R be a rate matrix on a finite state space and let π be a probability distribution on that space. The distribution π is stationary for R when $\pi R = 0$, so that a process started at π keeps the marginal π at every later time. The pair (R, π) is in detailed balance when $\pi(x)R(x,y) = \pi(y)R(y,x)$ for every pair of states x and y, and the process is then called reversible with respect to π.

Detailed balance implies stationarity, but stationarity alone does not imply detailed balance. To prove the implication, sum over x, substitute the detailed-balance identity and use the zero row sums,

$$\sum_x \pi(x)\, R(x,y) = \pi(y) \sum_x R(y,x) = 0.$$

"The net probability change $\sum_x \pi(x)R(x,y)$ at state y equals its stationary probability $\pi(y)$ times the row sum $\sum_x R(y,x)$, after applying detailed balance. The row sum is zero, so the net change vanishes."

At stationarity a process in detailed balance looks the same run forward or backward in time, and we call it reversible; a stationary process without detailed balance is distinguishable from its own reverse.

4.22.1 A cycle of three bases

Take three bases A, C and G with unit-rate jumps A $\to$ C, C $\to$ G and G $\to$ A, and no reverse jumps. Each diagonal entry is -1. At the uniform distribution $(1/3, 1/3, 1/3)$, each base receives flux $1/3$ from its upstream neighbor and loses flux $1/3$ to its downstream neighbor. The distribution is stationary because each entry of πR equals $(1/3)(1) + (1/3)(-1) = 0$.

Detailed balance fails on every edge. The flux from A to C is $\pi(\mathrm{A})R(\mathrm{A},\mathrm{C}) = 1/3$, while the flux from C back to A is $\pi(\mathrm{C})R(\mathrm{C},\mathrm{A}) = 0$, since that rate is zero. Probability circulates around the cycle at stationarity, and run backward in time it would circulate the other way, so the forward and reverse processes are different even though neither changes the marginal. We can compare this cycle with a chain whose fluxes match in both directions on every edge (Figure 4.5).

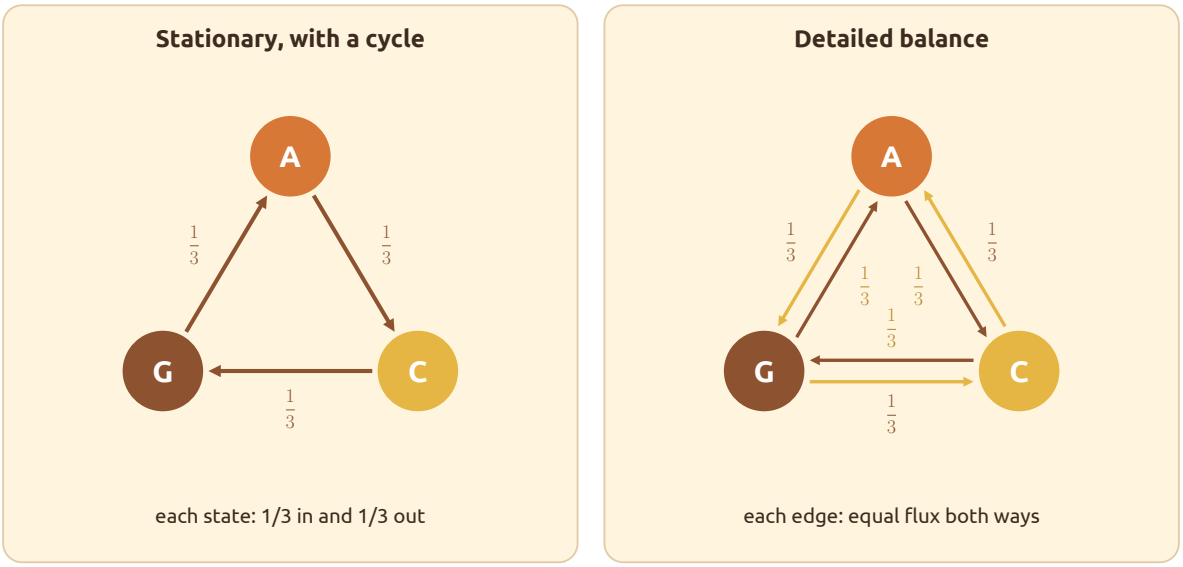


Figure 4.5: Two three-state processes on the bases A, C and G, with the uniform distribution $(1/3, 1/3, 1/3)$ stationary in both. Each arrow is labeled by its flux $\pi(x)R(x,y)$. Left: jumps run only A $\to$ C $\to$ G $\to$ A at rate 1, so each state receives $1/3$ and loses $1/3$ and the distribution is stationary, while each edge carries $1/3$ one way and nothing back. Right: every edge carries rate 1 in both directions, so each edge carries flux $1/3$ each way and detailed balance holds. Stationarity balances the flux at each state; detailed balance balances it on each edge.

4.22.2 Comparing corruption processes

We can check detailed balance for uniform replacement and masking. For uniform π over K tokens, every rate between two distinct tokens equals $1/K$, and the stationary probability of every token

also equals $1/K$, so

$$\pi(x)\, R_\pi(x, y) = \tfrac{1}{K} \cdot \tfrac{1}{K} = \pi(y)\, R_\pi(y, x).$$

"For uniform π over K tokens, the flux $\pi(x)R_\pi(x, y)$ from x to y equals $1/K$ times $1/K$, which is also the flux $\pi(y)R_\pi(y, x)$ from y back to x."

With $K = 4$ each edge of the DNA alphabet has flux $1/16 = 0.0625$ in each direction at stationarity. Uniform noise is reversible with respect to the uniform distribution, and the reverse process can move a token to any other token, so a sampler under uniform replacement can correct a token it has already written.

For masking, each token jumps to m at rate one, and the reverse-direction forward rate is zero. Equal fluxes on the edge (x, m) therefore require $\pi(x) = 0$. The only stationary distribution places all probability on m, where every flux is zero. Forward corruption uses one-way masking jumps, so its reverse can only reveal tokens. We compare both corruptions in Table 4.1.

Table 4.1: The two members of the family $R_\pi = \mathbf{1}\pi^\top - I$ used in this chapter, with $K = |\mathcal{V}|$ and α_t the survival probability. Rates are quoted for a pair of distinct states, and the stationary distribution is the limit of the kernel as $\alpha_t \to 0$.

	masking	uniform replacement
π	δ_m	$\mathbf{1}/K$
alphabet	$\mathcal{V}_m$	$\mathcal{V}$
$q_t(x \mid x_0)$	$\alpha_t\delta(x, x_0) + (1 - \alpha_t)\delta(x, m)$	$\alpha_t\delta(x, x_0) + (1 - \alpha_t)/K$
forward rates $R_\pi(x, y)$	1 into m, 0 between tokens	$1/K$ between any two tokens
state at $\alpha_t = 0$	all masks	uniform tokens
stationary distribution	δ_m	$\mathbf{1}/K$
detailed balance at π	fails	holds
reverse jumps	only out of m	between any two tokens

We specify the allowed jumps through R_π and their speed through $\beta(t)$, or equivalently α_t. We then compute the kernel $q_t(x \mid x_0)$ with a matrix exponential. Masking and uniform replacement differ only in π. The marginals satisfy the master equation and converge to π, as determined by the eigenvalues of R_π. We can also describe D3PM, SEDD and block diffusion through their choices of π, schedule and grouping of positions.

To generate a sample we run the same jumps backwards, from π at $t = 1$ to data at $t = 0$, passing through the same marginals p_t in reverse order. The reversed process is also a jump process, with rates $\hat{R}_t$.

4.23 Reverse jump processes

We choose the forward rates to carry clean data at $t = 0$ to the noise distribution π at $t = 1$. To generate, we start from π and pass through the same marginals p_t in reverse order. The reversed process is also a jump process, so we need its rates $\hat{R}_t$. For SDE reversal in Chapter 3, we estimated the score $\nabla \log p_t$; here we need relative state probabilities.

Let's take two states x and y with $x \neq y$, and watch the probability that crosses the edge between them during a short interval of length h. In the forward direction the process is at x with probability $p_t(x)$ and moves toward y at rate $R_t(x, y)$, so the mass moved from x to y is $p_t(x)R_t(x, y)h$. Watched backward, that same mass moves from y to x. The reverse process is at y with the same probability $p_t(y)$, since the two processes share their marginals by construction, so matching the two amounts of mass fixes its rate:

$$p_t(y)\,\hat{R}_t(y, x) = p_t(x)\,R_t(x, y)$$

"The probability $p_t(y)$ of being at y times the reverse rate $\hat{R}_t(y, x)$ back to x equals the probability $p_t(x)$ of being at x times the forward rate $R_t(x, y)$ toward y, so the two directions move the same probability across the edge in the same length of time."

This is a detailed balance condition between the forward process and its reversal, and it holds edge by edge. It differs from the detailed balance of a stationary chain in one respect, since the two rates belong to two different processes and the common weight p_t changes with time.

4.23.1 Solving for the reverse rate

Dividing the flux balance by $p_t(y)$ isolates the unknown:

$$\boxed{\hat{R}_t(y, x) = R_t(x, y)\,\frac{p_t(x)}{p_t(y)}}$$

"The reverse rate $\hat{R}_t(y, x)$ from y to x equals the forward rate $R_t(x, y)$ from x to y, which we chose, times the probability $p_t(x)$ of x divided by the probability $p_t(y)$ of y, both at the same time t."

- $R_t(x, y)$ is the forward rate from x to y, in probability per unit time, with $x \neq y$ and $R_t(x, x) = -\sum_{y \neq x} R_t(x, y)$.
- $\hat{R}_t(y, x)$ is the reverse rate from y to x, in the same units, and its diagonal is fixed the same way, $\hat{R}_t(y, y) = -\sum_{x \neq y} \hat{R}_t(y, x)$.
- p_t is the marginal of the forward process at time t, a row vector of probabilities over the state space.

The same expression follows from Bayes' rule, and the second derivation expresses the reverse rate as a probability we could in principle estimate by counting. We can condition on the state at the later time and ask where the process came from. For $x \neq y$,

$$\mathbb{P}(X_t = x \mid X_{t+h} = y) = \frac{\mathbb{P}(X_{t+h} = y \mid X_t = x)\,p_t(x)}{p_t(y)} = \frac{h\,R_t(x, y)\,p_t(x)}{p_t(y)} + o(h)$$

"The probability of an earlier state $X_t = x$, given the later state $X_{t+h} = y$, equals the interval length h times the forward jump rate $R_t(x, y)$ times the marginal probability ratio $p_t(x)/p_t(y)$, up to the remainder $o(h)$."

The coefficient of h is the reverse rate $\hat{R}_t(y, x)$. The ratio comes from the prior probability $p_t(x)$ divided by the observation probability $p_t(y)$, so generation requires estimating this ratio.

4.23.2 Recovery of the forward marginals

We can check that these reverse rates recover the marginals p_t when initialized at p_1. Substitute the flux identities $p_t(x)\hat{R}_t(x, y) = p_t(y)R_t(y, x)$ and $p_t(y)\hat{R}_t(y, x) = p_t(x)R_t(x, y)$ into the reverse net flow at state y:

$$\sum_{x \neq y} \Big[p_t(x)\hat{R}_t(x, y) - p_t(y)\hat{R}_t(y, x) \Big] = -\sum_{x \neq y} \Big[p_t(x)R_t(x, y) - p_t(y)R_t(y, x) \Big] = -\frac{d}{dt}\, p_t(y)$$

"The sum over every other state x of the reverse flux $p_t(x)\hat{R}_t(x, y)$ into y minus the reverse flux $p_t(y)\hat{R}_t(y, x)$ out of y equals the negative of the forward net flux into y, which by the master equation is $-\frac{d}{dt}p_t(y)$."

The reverse process runs from $t = 1$ down to $t = 0$, so its own clock is $s = 1 - t$ and its master equation reads $\frac{d}{ds}p = p\hat{R}$. Written in t, that is exactly the identity above. Every forward arrival at y has become a reverse departure from y, and every forward departure has become a reverse arrival. If we start the reverse rates at p_1 and run the clock down, we therefore pass through every intermediate marginal p_t and end at the data distribution.

The forward process moves the mass $p_t(x)R_t(x, y)$ from x to y, and the reverse process moves the mass $p_t(y)\hat{R}_t(y, x)$ from y to x (Figure 4.6). If we set the two equal on every edge, at every time, we retrace p_t from noise back to data.

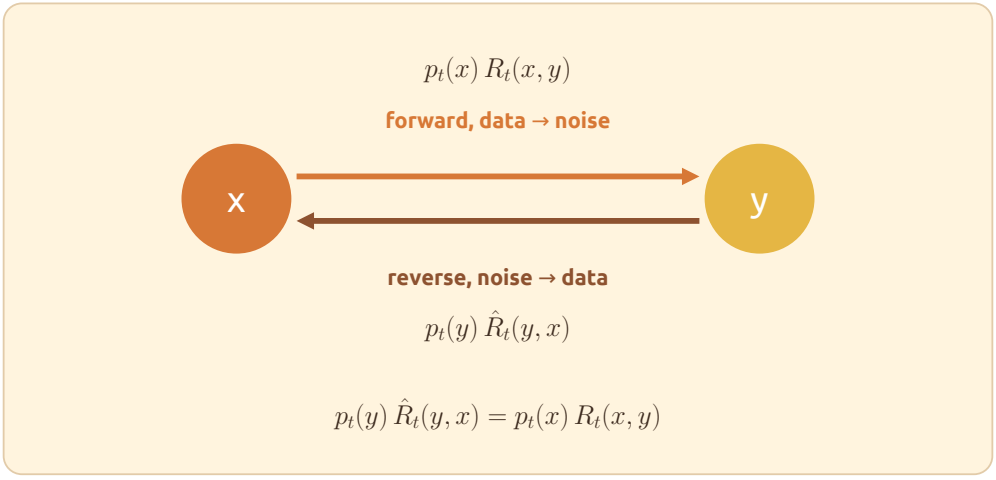


Figure 4.6: One edge of the state space, joining the states x and y. The upper arrow is the forward process, which runs from data to noise and carries probability at the flux $p_t(x)R_t(x, y)$. The lower arrow is the reverse process, which runs from noise to data and carries probability at the flux $p_t(y)\hat{R}_t(y, x)$. Equating the two fluxes on this edge, at the same time t, is the condition that determines $\hat{R}_t(y, x)$.

4.23.3 Learning probability ratios

We chose the forward rate $R_t(x, y)$, so we can evaluate it directly. The ratio $p_t(x)/p_t(y)$ depends on the data distribution through the marginals and has no available formula. We therefore train a network to estimate it, just as we estimated $\nabla \log p_t$ for SDE reversal in Chapter 3. The collection of ratios over state pairs is the concrete score [Lou et al., 2024].

The state space $\mathcal{S} = \mathcal{V}^L$ has K^L elements, making a full table of probability ratios impractical. We need only ratios between sequences connected by a forward jump. These processes change one position at a time, so connected sequences differ at one index. Write $z^{i \leftarrow a}$ for sequence z with token a at position i. The required ratios are

$$s_t(z)_{i,a} = \frac{p_t(z^{i \leftarrow a})}{p_t(z)}$$

"The concrete score $s_t(z)_{i,a}$ at position i and token a equals the probability $p_t(z^{i \leftarrow a})$ of the sequence z with position i changed to a, divided by the probability $p_t(z)$ of z itself."

Each sequence has $L(K-1)$ such numbers, one per position and per alternative token, so a DNA sequence of length 10 has $10 \times 3 = 30$ of them. We can output all of them with a network that has L heads of K logits each.

4.24 The posterior average of conditional reverse rates

During training the clean sequence x_0 is in hand, and conditioned on it the forward process has a closed form. Corruption acts on every position independently, so the conditional law of the noisy sequence z factorizes:

$$q_t(z \mid x_0) = \prod_{i=1}^{L} q_t(z^i \mid x_0^i)$$

"The probability $q_t(z \mid x_0)$ of the noisy sequence z given the clean sequence x_0 equals the product over positions i from 1 to L of $q_t(z^i \mid x_0^i)$, the probability of the noisy token z^i given the clean token x_0^i."

Conditioning on the starting point does not change the rates, since x_0 constrains the initial state and not the dynamics. The conditioned process is therefore a jump process with the same rates R_t and with marginals $q_t(\cdot \mid x_0)$, and the reversal rule of Section 4.23.1 applies to it verbatim:

$$\hat{R}_t(y, x \mid x_0) = R_t(x, y)\, \frac{q_t(x \mid x_0)}{q_t(y \mid x_0)}$$

"The conditional reverse rate $\hat{R}_t(y, x \mid x_0)$ from y to x equals the forward rate $R_t(x, y)$ times the ratio of the conditional probabilities $q_t(x \mid x_0)$ and $q_t(y \mid x_0)$, given the clean sequence x_0."

Every quantity on the right is a product of per-position kernels we wrote in closed form, so we can evaluate this rate for any training sequence at any noise level. During generation, x_0 is unknown, so we need to express the marginal rates using these conditional rates.

4.24.1 Posterior averaging over clean sequences

We can calculate the noisy marginal by corrupting the data distribution,

$$p_t(x) = \sum_{x_0} p_{\text{data}}(x_0)\, q_t(x \mid x_0)$$

"The marginal probability $p_t(x)$ of the noisy state x equals the sum over clean sequences x_0 of the data probability $p_{\text{data}}(x_0)$ times the conditional probability $q_t(x \mid x_0)$."

Next, we can divide both sides by $p_t(y)$, then multiply and divide each term of the sum by $q_t(y \mid x_0)$. The factors regroup into a posterior and a conditional ratio:

$$\frac{p_t(x)}{p_t(y)} = \sum_{x_0} \underbrace{\frac{p_{\text{data}}(x_0)\, q_t(y \mid x_0)}{p_t(y)}}_{p(x_0 \mid X_t = y)} \frac{q_t(x \mid x_0)}{q_t(y \mid x_0)}$$

"The marginal probability ratio $p_t(x)/p_t(y)$ averages the conditional ratio $q_t(x \mid x_0)/q_t(y \mid x_0)$ over clean sequences x_0. Each weight is the posterior probability $p(x_0 \mid X_t = y)$, obtained by dividing the data probability $p_{\text{data}}(x_0)$ times the corruption probability $q_t(y \mid x_0)$ by the noisy marginal $p_t(y)$."

The underbraced factor is Bayes' rule for the clean sequence given the observed noisy one. If we multiply both sides by the forward rate $R_t(x, y)$, which does not depend on x_0 and passes inside the sum, each term becomes a conditional reverse rate:

$$\boxed{\hat{R}_t(y, x) = \sum_{x_0} p(x_0 \mid X_t = y)\; \hat{R}_t(y, x \mid x_0)}$$

"The marginal reverse rate $\hat{R}_t(y, x)$ averages the conditional rates $\hat{R}_t(y, x \mid x_0)$ over clean sequences x_0, weighted by their posterior probabilities $p(x_0 \mid X_t = y)$ given the current state y."

Every clean sequence that could have produced y has its own target rate, and the posterior weights combine those targets into one number per edge. This is the discrete counterpart of the marginal velocity of Chapter 2, where a marginal velocity field was the posterior average of the conditional velocities passing through a point. Here we average conditional rates over the posterior distribution of clean sequences.

4.24.2 Single-base posterior average

Consider one DNA position under uniform noise with $\alpha_t = 1/2$ and unit forward rates between distinct bases. We obtain these rates by setting $\beta_t = 4$ and choosing uniform π over $K = 4$ bases, since

each off-diagonal rate is $\beta_t \pi(y)$. The schedule $\alpha_t = e^{-4t}$ reaches one half at $t = (\log 2)/4 \approx 0.1733$. Substituting into $q_t(x \mid x_0) = \alpha_t \delta(x, x_0) + (1 - \alpha_t)/K$ gives

$$q_t(x_0 \mid x_0) = \tfrac{1}{2} + \tfrac{1}{2} \cdot \tfrac{1}{4} = 0.625, \qquad q_t(x \mid x_0) = \tfrac{1}{2} \cdot \tfrac{1}{4} = 0.125 \quad (x \neq x_0)$$

Let the clean base be A with probability 0.75 and C with probability 0.25, and suppose the process is at C at time t. We want the reverse rate of the jump from C back to A.

The targets attached to the two possible clean bases differ by a factor of 25:

$$\hat{R}_t(\mathrm{C}, \mathrm{A} \mid \mathrm{A}) = 1 \cdot \frac{0.625}{0.125} = 5, \qquad \hat{R}_t(\mathrm{C}, \mathrm{A} \mid \mathrm{C}) = 1 \cdot \frac{0.125}{0.625} = 0.2$$

"For a clean A, the reverse rate $\hat{R}_t(\mathrm{C}, \mathrm{A} \mid \mathrm{A})$ equals the forward rate 1 times the conditional probability ratio 0.625/0.125, giving 5. For a clean C, the ratio is 0.125/0.625, giving a reverse rate of 0.2."

Conditional on a clean A, the reverse rate is five times the noise rate. Conditional on a clean C, it is one fifth of that rate. We can average these rates using posterior weights from Bayes' rule. The likelihood of observing C is 0.125 under a clean A and 0.625 under a clean C, so

$$p(\mathrm{A} \mid \mathrm{C}) = \frac{0.75 \times 0.125}{0.75 \times 0.125 + 0.25 \times 0.625} = \frac{0.09375}{0.09375 + 0.15625} = \frac{0.09375}{0.25} = 0.375$$

and $p(\mathrm{C} \mid \mathrm{C}) = 0.15625/0.25 = 0.625$. Observing C reduces the probability of a clean A from the prior value 0.75 to the posterior value 0.375. Averaging the two targets with these weights,

$$\hat{R}_t(\mathrm{C}, \mathrm{A}) = 0.375 \times 5 + 0.625 \times 0.2 = 1.875 + 0.125 = 2$$

"The marginal reverse rate $\hat{R}_t(\mathrm{C}, \mathrm{A})$ from C to A averages the conditional rates 5 and 0.2 with posterior probabilities 0.375 and 0.625, giving the rate 2."

The posterior over the clean base and the two conditional rates it averages are drawn side by side in Figure 4.7.

We can check the same number directly through the marginals. Corrupting the data distribution at this noise level,

$$p_t(\mathrm{A}) = 0.75 \times 0.625 + 0.25 \times 0.125 = 0.5, \qquad p_t(\mathrm{C}) = 0.75 \times 0.125 + 0.25 \times 0.625 = 0.25$$

with $p_t(\mathrm{G}) = p_t(\mathrm{T}) = 0.125$, which sum to one. The reverse rate from the definition is $\hat{R}_t(\mathrm{C}, \mathrm{A}) = 1 \times p_t(\mathrm{A})/p_t(\mathrm{C}) = 0.5/0.25 = 2$, matching the posterior average. If we repeat the arithmetic on the other two bases, $\hat{R}_t(\mathrm{C}, \mathrm{G}) = \hat{R}_t(\mathrm{C}, \mathrm{T}) = 0.125/0.25 = 0.5$, so the total rate of leaving C in the reverse direction is $2 + 0.5 + 0.5 = 3$ and the process waits an exponential time of mean $1/3$ before its next jump.

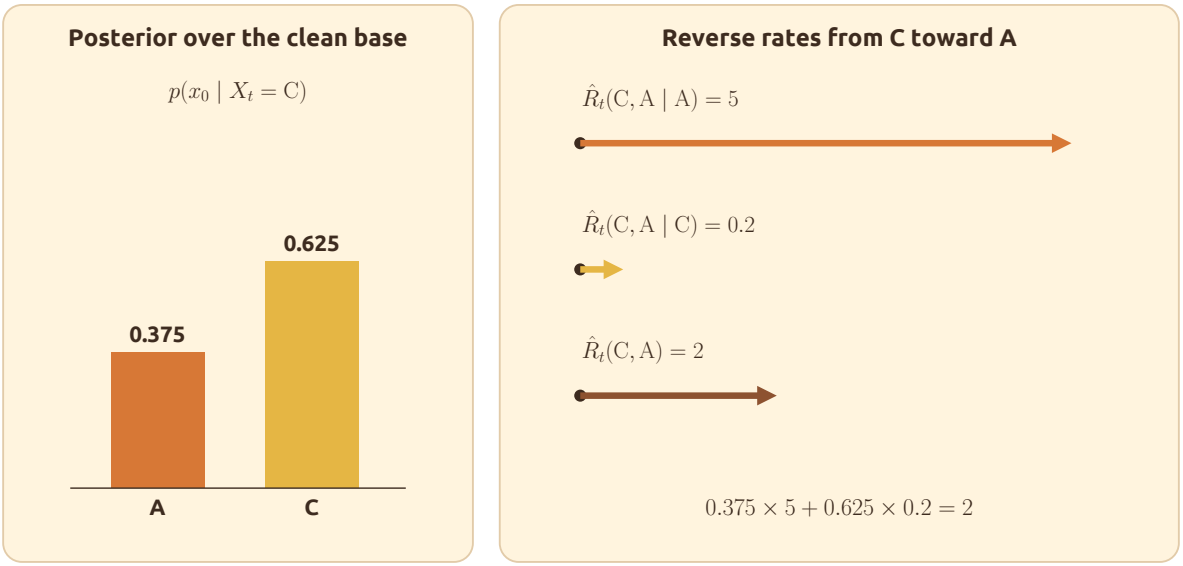


Figure 4.7: Left, the posterior $p(x_0 \mid X_t = \mathrm{C})$ over the clean base after observing a C at $\alpha_t = 1/2$, with weight 0.375 on A and 0.625 on C. Right, the two conditional reverse rates for the jump from C toward A, drawn as arrows whose length is the rate: 5 under a clean A, 0.2 under a clean C, and the posterior average 2 that the marginal reverse process uses.

4.24.3 Reduction to one position at a time

The posterior $p(x_0 \mid X_t = y)$ is a distribution over all K^L sequences, which is far too large to store or to predict. We can shrink it using two properties of the forward process. It changes one position at a time, and it corrupts positions independently.

We can change position i of the noisy sequence z to the token a. In the ratio of conditional probabilities, every factor except the i-th appears identically in numerator and denominator and cancels:

$$\frac{q_t(z^{i \leftarrow a} \mid x_0)}{q_t(z \mid x_0)} = \frac{q_t(a \mid x_0^i)}{q_t(z^i \mid x_0^i)}$$

"For a sequence z with position i changed to token a, the conditional sequence-probability ratio $q_t(z^{i \leftarrow a} \mid x_0)/q_t(z \mid x_0)$ equals the single-position ratio $q_t(a \mid x_0^i)/q_t(z^i \mid x_0^i)$. All other position factors cancel because those tokens are unchanged."

Only the clean token at position i remains. Substituting this into the posterior average and collapsing the sum over sequences to a sum over the token b at that one position,

$$\frac{p_t(z^{i \leftarrow a})}{p_t(z)} = \sum_b p(x_0^i = b \mid z) \, \frac{q_t(a \mid b)}{q_t(z^i \mid b)}$$

"The probability ratio $p_t(z^{i \leftarrow a})/p_t(z)$ for changing position i of sequence z to token a averages the single-position corruption ratio $q_t(a \mid b)/q_t(z^i \mid b)$ over possible clean tokens b, weighted by the

posterior probabilities $p(x_0^i = b \mid z)$."

We only need to predict L distributions over K tokens, one per position, each conditioned on the whole noisy sequence. That is the denoiser $d_\theta(a \mid z, t)$ of Section 4.7, and we write $d_\theta^i(b \mid z, t)$ for its output at position i. A single network with L softmax heads produces the concrete score at every position.

We can check the cancellation with two positions. Assign data probability 0.75 to AG and 0.25 to CG, set $\alpha_t = 1/2$, and observe $z = \text{CG}$. The joint posterior weights are $0.75 \times 0.125 \times 0.625 = 0.05859$ for AG and $0.25 \times 0.625 \times 0.625 = 0.09766$ for CG. They sum to $p_t(\text{CG}) = 0.15625$. Dividing by this sum gives posterior probabilities 0.375 and 0.625. These match the single-base calculation because the common factor $q_t(\text{G} \mid \text{G})$ cancels. Substituting into the per-position formula gives $p_t(\text{AG})/p_t(\text{CG}) = 0.375 \times 5 + 0.625 \times 0.2 = 2$, matching the direct ratio $0.3125/0.15625 = 2$.

We can therefore calculate reverse rates from clean-token predictions. We can also reweight jump rates to generate sequences with a desired property.

4.25 Conditioning a jump process: Doob's h-transform

We often want to sample sequences with a specific property, such as a binding peptide, an active promoter or a folded protein. We can condition a jump process on this property at its endpoint using an h-transform, which Doob developed for Brownian motion [Doob, 1957].

Let E be the event we want, and keep only the paths that end in E. We define the value function

$$h_t(x) = \mathbb{P}(E \mid X_t = x)$$

"The value $h_t(x)$ is the probability of the event E given that the state X_t at time t equals x."

The value $h_t(x)$ is a number in $[0, 1]$ and summarizes everything about the state x that matters for E. Conditioning a single short step on E is Bayes' rule with E as the evidence:

$$\mathbb{P}(X_{t+h} = y \mid X_t = x,\, E) = \mathbb{P}(X_{t+h} = y \mid X_t = x)\, \frac{h_{t+h}(y)}{h_t(x)}$$

"The transition probability from state x to state y over an interval h, conditional on event E, equals the unconditioned transition probability times the ratio of event probabilities $h_{t+h}(y)/h_t(x)$ at the destination and current state."

For $y \neq x$ the unconditioned probability is $hR_t(x, y) + o(h)$, and $h_{t+h}(y) \to h_t(y)$ as $h \to 0$. Reading off the coefficient of h,

$$\boxed{R_t^h(x, y) = R_t(x, y)\, \frac{h_t(y)}{h_t(x)}}$$

"The conditioned rate $R^h_t(x, y)$ from x to y equals the original rate $R_t(x, y)$ times $h_t(y)$, the probability of E from the destination, divided by $h_t(x)$, the probability of E from the current state."

This is Doob's h-transform. Jumps that raise the chance of E speed up by the factor by which they raise it, and jumps that lower it slow down by the same factor. A jump into a state from which E is impossible has $h_t(y) = 0$ and never happens. Under the transform the state space and the graph of allowed jumps stay the same, and only the speeds along each edge change.

We know h at both ends of the time interval. At the final time of the conditioning, h is the indicator of E, equal to one on the states that satisfy it and zero elsewhere. At earlier times $h_t(x)$ is a smooth interpolation between those extremes, and it is the probability that a path started at x at time t and run under the original rates lands in E. A constant h changes nothing, since the ratio $h_t(y)/h_t(x)$ is then one on every edge, which is the statement that conditioning on an event of probability one does not change the process.

4.25.1 Backward equation for h

To normalize the conditioned transition probabilities, we need a relation between the values of h at successive times. Starting from x, we can calculate the probability of E by averaging over the possible states one short step later:

$$h_t(x) = \sum_y P_{t,t+dt}(x, y)\, h_{t+dt}(y)$$

"The event probability $h_t(x)$ from state x at time t equals the average of the later event probabilities $h_{t+dt}(y)$, weighted by the transition probabilities $P_{t,t+dt}(x, y)$ over the interval dt."

If we divide this relation by $h_t(x)$, the left side becomes one while the right side becomes the total conditioned transition probability out of x. The relation is therefore exactly the normalization that Section 4.25 needed. If we now substitute $P_{t,t+dt} = I + dt\, R_t + o(dt)$ and let dt go to zero, we get a differential equation:

$$\frac{\partial}{\partial t}\, h_t(x) = -\sum_y R_t(x, y)\, h_t(y)$$

"The time derivative of the event probability $h_t(x)$ equals the negative sum over states y of the rate-matrix entry $R_t(x, y)$ times the event probability $h_t(y)$. The sum includes the diagonal contribution $R_t(x, x)h_t(x)$."

This is the Kolmogorov backward equation. It is linear in h and it runs backward from the final time, where h_1 is the indicator of E. The master equation $\dot{p}_t = p_t R_t$ runs the other way, propagating p forward from $t = 0$. Solving the backward equation exactly requires enumerating the state space, so in practice we estimate h.

4.25.2 The reverse process as an h-transform

For a forward process in detailed balance with π, we can write the reverse rate from Section 4.23.1 as an h-transform. Detailed balance gives

$$R_t(x, y) = R_t(y, x)\,\frac{\pi(y)}{\pi(x)}$$

so the forward rate in one direction is the rate in the other direction reweighted by the stationary masses. Substituting it into the reverse rate and grouping the factors by state,

$$\hat{R}_t(y, x) = R_t(x, y)\,\frac{p_t(x)}{p_t(y)} = R_t(y, x)\,\frac{p_t(x)/\pi(x)}{p_t(y)/\pi(y)}$$

which with the definition $h_t = p_t/\pi$ reads

$$\boxed{\hat{R}_t(y, x) = R_t(y, x)\,\frac{h_t(x)}{h_t(y)}, \qquad h_t = \frac{p_t}{\pi}}$$

"The reverse rate $\hat{R}_t(y, x)$ equals the forward rate $R_t(y, x)$ from y to x times the value $h_t(x)$ at the destination divided by the value $h_t(y)$ at the current state, where h_t is the marginal p_t divided by the stationary distribution π."

We can therefore generate data by applying an h-transform to the noise process, increasing the rates toward states that are more probable under the data than under pure noise. The ratio p_t/π measures that excess, and it equals one everywhere at $t = 1$, where the two distributions coincide.

The same reading applies to the conditional rates. We condition on a clean token x_0 under uniform noise and take $h_t(x) = q_t(x \mid x_0)/\pi(x)$. The constant π cancels from the ratio and

$$\hat{R}_t(y, x \mid x_0) = R_t(y, x)\,\frac{q_t(x \mid x_0)}{q_t(y \mid x_0)}$$

which agrees with Section 4.24 because the rate matrix is symmetric under a uniform π, $R_t(x, y) = R_t(y, x)$.

For uniform $\pi = 1/4$ and marginals $p_t = (0.5, 0.25, 0.125, 0.125)$ over A, C, G and T, divide each marginal by $1/4$ to obtain $h_t = (2, 1, 0.5, 0.5)$. The reverse rate from C to A is $1 \times 2/1 = 2$; the rates to G and T are each $1 \times 0.5/1 = 0.5$. Conditioning on a clean A gives $h_t = (2.5, 0.5, 0.5, 0.5)$ and rate $2.5/0.5 = 5$ toward A. Conditioning on a clean C gives $h_t = (0.5, 2.5, 0.5, 0.5)$ and rate $0.5/2.5 = 0.2$ toward A. Averaging these two h-transforms with the posterior weights gives the marginal reverse process.

To rewrite the rates in terms of $h_t = p_t/\pi$ we need a π that is positive everywhere, which uniform noise has and masking does not. Masking moves every token to m and never brings anything back, so its stationary distribution $\pi = \delta_m$ has zero mass on every real token and the quotient p_t/π is undefined there. The general rule of Section 4.23.1 still applies, since it assumed nothing about the forward process beyond its rates and its marginals.

From that rule we can already read off the direction of the masked reverse process. The reverse rate $\hat{R}_t(y, x)$ has the factor $R_t(x, y)$ in it, so it vanishes on every edge the forward process does not use. Masking moves along only the edges that replace a token by m, so the reverse process can move only along those edges backward, replacing an m by a token. Remasking is impossible under the exact reversal, and the masked reverse step derived by hand in Section 4.6 is that statement in closed form.

4.25.3 Guiding generation toward a property

We can apply the transform a second time, now to a property c we want the finished sequence to have. The event is c observed at the end of generation, so the value function is $h_t(x) = p(c \mid X_t = x)$, the probability that a sequence continuing from x ends up with the property. Transforming the reverse rates,

$$\hat{R}_t^c(y, x) = \hat{R}_t(y, x)\, \frac{p(c \mid X_t = x)}{p(c \mid X_t = y)}$$

"The guided reverse rate $\hat{R}_t^c(y, x)$ from y to x equals the unguided reverse rate $\hat{R}_t(y, x)$ times $p(c \mid X_t = x)$, the probability of the property from the destination, divided by $p(c \mid X_t = y)$, the probability of the property from the current state."

In Discrete Guidance, we estimate $p(c \mid x)$ with a classifier trained on noisy sequences at every noise level, and raise the ratio to a power $\gamma \geq 1$ to sharpen the effect [Nisonoff et al., 2025]. The diffusion model stays fixed throughout, and guidance changes only the rates at sampling time, so one trained model works for many properties.

Let's guide one jump with the rates computed above. From C, the unguided reverse rates are 2 toward A and 0.5 each toward G and T. Suppose the classifier assigns $p(c \mid \cdot) = 0.2$ at C, 0.1 at A, 0.6 at G, and 0.2 at T. Each rate is then multiplied by the ratio of the classifier probability at its destination to the classifier probability at C:

$$\hat{R}^c(\mathrm{C}, \cdot) = \Big(2 \cdot \frac{0.1}{0.2},\ \ 0.5 \cdot \frac{0.6}{0.2},\ \ 0.5 \cdot \frac{0.2}{0.2}\Big) = (1,\ 1.5,\ 0.5)$$

"The guided rates out of C toward A, G and T equal the unguided rates 2, 0.5 and 0.5 multiplied by the classifier ratios 0.1/0.2, 0.6/0.2 and 0.2/0.2, and the results are 1, 1.5 and 0.5."

The rate toward G triples because its property probability is three times that at C. The rate toward A halves because its property probability is half that at C. To calculate destination probabilities, normalize each set of rates by its total. The unguided total is $2 + 0.5 + 0.5 = 3$, giving probabilities 2/3, 1/6 and 1/6 for A, G and T. The guided total is also $1 + 1.5 + 0.5 = 3$, giving 1/3, 1/2 and 1/6. The holding time remains 1/3 for these values, and G becomes the most likely destination (Figure 4.8).

Raising the classifier ratios to $\gamma = 2$ strengthens the preference for G. The rates become $2 \times 0.25 = 0.5$ toward A, $0.5 \times 9 = 4.5$ toward G and $0.5 \times 1 = 0.5$ toward T. Their sum is 5.5, giving destination probabilities 0.0909, 0.8182 and 0.0909. The mean holding time decreases to $1/5.5 = 0.1818$, and

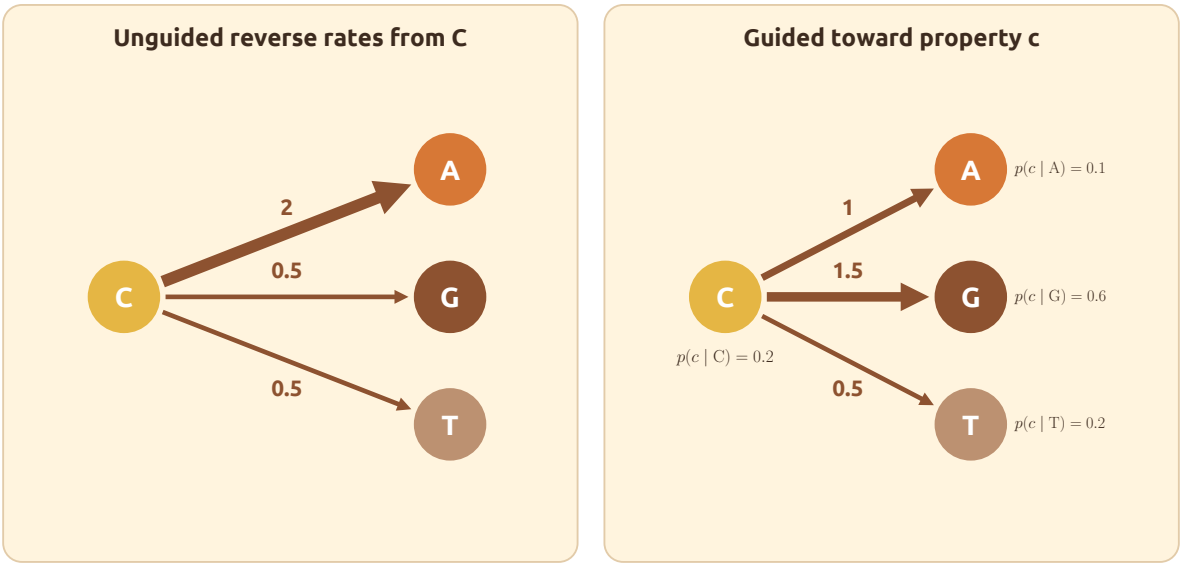


Figure 4.8: Left, the unguided reverse rates out of C at $\alpha_t = 1/2$: 2 toward A, 0.5 toward G and 0.5 toward T, with arrow thickness proportional to the rate. Right, the same jumps after the h-transform by a classifier with $p(c \mid \cdot)$ equal to 0.2 at C, 0.1 at A, 0.6 at G and 0.2 at T, giving the guided rates 1, 1.5 and 0.5. The jump toward G triples and the jump toward A halves.

over four fifths of jumps end at G. Increasing γ improves property satisfaction while moving the samples farther from the data distribution. We choose the exponent to control this trade-off.

4.26 A loss for the reverse rates

We cannot evaluate the marginal reverse rates, but we can calculate conditional rates for every training pair. We need a loss whose minimizer equals the correct average of these conditional targets. We can derive such a loss from the KL divergence between path distributions: for processes with the same starting distribution, any difference in the path laws comes from the rates.

We can take one short step of length h from a state x, and write $A(y)$ for the target rate toward $y \neq x$ and $B(y)$ for the model rate, with $A_{\text{out}} = \sum_{y \neq x} A(y)$ and $B_{\text{out}} = \sum_{y \neq x} B(y)$ the total rates of leaving x. The step has one outcome per destination plus the outcome of staying, with probabilities $hA(y)$ and $1 - hA_{\text{out}}$ under the target, so

$$\text{KL} = \sum_{y \neq x} hA(y) \log \frac{A(y)}{B(y)} \; + \; (1 - hA_{\text{out}}) \log \frac{1 - hA_{\text{out}}}{1 - hB_{\text{out}}} + o(h)$$

"The short-step KL divergence sums the target jump probabilities $hA(y)$ times the log ratios of target rates $A(y)$ to model rates $B(y)$, over destinations $y \neq x$. It also includes the target staying probability $1 - hA_{\text{out}}$ times its log ratio to the model staying probability $1 - hB_{\text{out}}$, up to $o(h)$, where h is the interval length and the subscript denotes total exit rates."

The factor h cancels inside the first logarithm, which is why only the rates appear there. Expanding the staying term with $\log(1 - u) = -u + O(u^2)$,

$$(1 - hA_{\text{out}}) \log \frac{1 - hA_{\text{out}}}{1 - hB_{\text{out}}} = (1 - hA_{\text{out}})(hB_{\text{out}} - hA_{\text{out}}) + O(h^2) = h\,(B_{\text{out}} - A_{\text{out}}) + o(h)$$

Collecting both pieces, dividing by h and integrating along the path,

$$\text{KL(paths)} = \mathbb{E} \int_0^1 \sum_{y \neq X_t} \Big[A \log \frac{A}{B} - A + B \Big]\, dt$$

"The path KL divergence averages, along target-process paths, the time integral of the rate error $A \log(A/B) - A + B$, summed over destinations y different from the current state X_t. Here A and B are the target and model jump rates to y, and time t runs from zero to one."

The expectation runs over paths of the target process, and its state X_t at each time selects the row of rates being compared. The term $B - A$ comes entirely from the staying probability. Without it, nothing would penalize a model that adds jumps the target never makes, because the logarithmic term alone contributes nothing at a destination with $A(y) = 0$.

4.26.1 The rate error and its minimizer

For one destination, define the rate error

$$\ell(A, B) = A \log \frac{A}{B} - A + B$$

"The rate error $\ell(A, B)$ between a target rate A and a model rate B, both positive, equals A times the log of A divided by B, minus A, plus B."

This is the Bregman divergence generated by $\varphi(u) = u \log u - u$ on the positive half-line. If we write out $\varphi(A) - \varphi(B) - \varphi'(B)(A - B)$ with $\varphi'(u) = \log u$, we get $A \log A - A - B \log B + B - (\log B)(A - B) = A \log(A/B) - A + B$, which is ℓ. We can prove that the error is nonnegative and equals zero only at $B = A$ by applying $\log u \leq u - 1$ with $u = B/A$:

$$A \log \frac{A}{B} = -A \log \frac{B}{A} \;\geq\; -A\Big(\frac{B}{A} - 1\Big) = A - B$$

so $\ell(A, B) \geq 0$, with equality exactly when $B/A = 1$. The two tails behave differently, which matters while a model is still far from the target. For $B \to 0$ with A fixed, ℓ grows like $A \log(1/B)$ and diverges, so the error is unbounded for a model that assigns a near-zero rate to a jump the target makes. For $B \to \infty$, ℓ grows like B, only linearly. An imperfect fit is therefore tolerated more readily when it overestimates a rate than when it underestimates one.

At the state C of the worked example the marginal reverse rates are 2, 0.5 and 0.5 toward A, G and T. If the denoiser predicts a uniform distribution over the four bases, averaging the conditional rates gives $0.25 \times 5 + 0.25 \times 0.2 + 0.25 \times 1 + 0.25 \times 1 = 1.8$ toward each of the three destinations. The rate

error at this state is then $\ell(2, 1.8) + 2\,\ell(0.5, 1.8) = 0.0107 + 2 \times 0.6595 = 1.330$ per unit time, almost all of it from the two destinations whose rate the uniform prediction inflates by a factor of 3.6.

The rate error, like the squared velocity error in Chapter 2, is minimized by a conditional mean. To verify this, let the target rate A vary randomly and differentiate the expected error with respect to the model rate B:

$$\frac{\partial}{\partial B}\,\mathbb{E}[\ell(A, B)] = 1 - \frac{\mathbb{E}[A]}{B} = 0 \quad \Longrightarrow \quad B^* = \mathbb{E}[A]$$

"The derivative of the expected rate error $\mathbb{E}[\ell(A, B)]$ with respect to the model rate B equals one minus the expected target rate $\mathbb{E}[A]$ divided by B, so the optimal model rate B^* equals $\mathbb{E}[A]$."

The second derivative is $\mathbb{E}[A]/B^2 > 0$, so the stationary point is the minimum. If we train a model rate against random conditional targets we therefore recover the mean of those targets, and Section 4.24 identified that mean as the marginal reverse rate.

4.26.2 The minimum of the posterior mixture

The two conditional targets for the jump from C to A were 5 and 0.2, with posterior weights 0.375 and 0.625. Each error curve $\ell(A, B)$ is convex in B, falls steeply as B approaches A from below, reaches zero at $B = A$, and grows linearly afterward. The error we actually minimize is the posterior mixture

$$0.375\,\ell(5, B) + 0.625\,\ell(0.2, B)$$

Differentiating, the derivative is $0.375(1 - 5/B) + 0.625(1 - 0.2/B) = 1 - 2/B$, which vanishes at

$$B^* = 0.375 \times 5 + 0.625 \times 0.2 = 2$$

the posterior-average rate. Substituting back, $\ell(5, 2) = 5\log(2.5) - 3 = 1.581$ and $\ell(0.2, 2) = 0.2\log(0.1) + 1.8 = 1.339$, so the mixture at its minimum equals $0.375 \times 1.581 + 0.625 \times 1.339 = 1.430$. Neither individual target is matched, and that residual error is irreducible, since one model rate cannot equal two different targets. If we move B to either target the mixture rises: at $B = 5$ it is 2.598 and at $B = 0.2$ it is 4.235 (Figure 4.9).

4.26.3 Conditional rate matching

We can now compare losses based on marginal and conditional rates. If the marginal reverse rates were available, we could compare them directly with the model rates $\hat{R}_t^\theta$ at every noisy state and time:

$$\mathcal{L}_{\text{marg}}(\theta) = \int_0^1 \mathbb{E}_{Z_t} \sum_y \ell\big(\hat{R}_t(Z_t, y),\ \hat{R}_t^\theta(Z_t, y)\big)\,dt$$

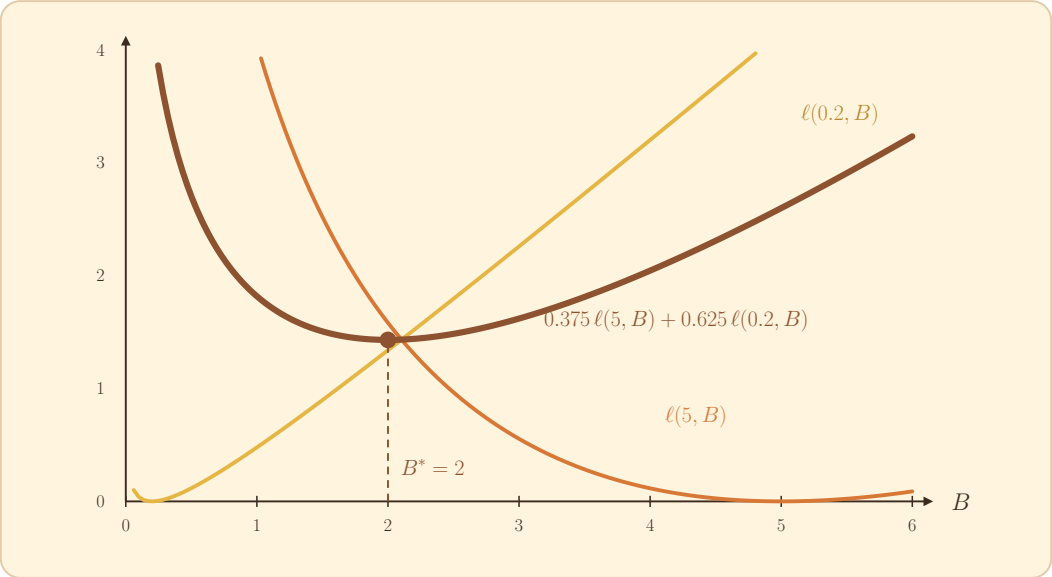


Figure 4.9: The rate error $\ell(A, B) = A \log(A/B) - A + B$ as a function of the model rate B on the horizontal axis. The two thin curves are the errors against the conditional targets $A = 5$ and $A = 0.2$, each touching zero at its own target. The thick curve is their posterior mixture $0.375\,\ell(5, B) + 0.625\,\ell(0.2, B)$, whose minimum is marked at $B^* = 2$, the posterior-average rate, with value 1.430.

"The marginal loss $\mathcal{L}_{\text{marg}}(\theta)$ equals the integral over time t from 0 to 1 of the expectation over noisy states Z_t of the sum over destinations y of the rate error ℓ between the marginal reverse rate $\hat{R}_t(Z_t, y)$ and the model rate $\hat{R}^\theta_t(Z_t, y)$."

The sum over y runs over the states reachable from Z_t in one jump, and $\hat{R}_t(Z_t, y)$ is out of reach because it averages over every clean sequence that could have produced Z_t. The conditional rates are available for every training pair, so we replace the target:

$$\mathcal{L}_{\text{cond}}(\theta) = \int_0^1 \mathbb{E}_{x_0,\, Z_t} \sum_y \ell\big(\hat{R}_t(Z_t, y \mid x_0),\ \hat{R}^\theta_t(Z_t, y)\big)\, dt$$

"The conditional loss $\mathcal{L}_{\text{cond}}(\theta)$ integrates over time the expected sum of rate errors between conditional target rates $\hat{R}_t(Z_t, y \mid x_0)$ and model rates $\hat{R}^\theta_t(Z_t, y)$. The expectation averages over clean sequences x_0 and noisy states Z_t, and the sum covers destinations y."

To sample this loss once we need one clean sequence, one time drawn uniformly, one corruption of that sequence, and one evaluation of the network.

Theorem 4.3 (Conditional rate matching)**.** *Let the forward process have rates R_t and conditional marginals $q_t(\cdot \mid x_0)$, and let the model rates $\hat{R}^\theta_t(z, y)$ be strictly positive and free to vary at each pair (z, y). Then $\mathcal{L}_{\text{marg}}$ and $\mathcal{L}_{\text{cond}}$ differ by a constant independent of θ. They have the same minimizer and the same gradient in θ, and at the minimum $\hat{R}^\theta_t(z, y) = \hat{R}_t(z, y)$ at every z, y and t that the forward process reaches.*

To prove the result, apply the calculation from Section 4.26.1 at each (z, y, t). First condition $\mathcal{L}_{\text{cond}}$

on $Z_t = z$. The clean sequence then follows $p(x_0 \mid Z_t = z)$, and the model rate is independent of x_0. The inner expectation is therefore $\mathbb{E}[\ell(A, B)]$, with random target $A = \hat{R}_t(z, y \mid x_0)$ and fixed prediction $B = \hat{R}_t^\theta(z, y)$. Its minimizer is $\mathbb{E}[A] = \sum_{x_0} p(x_0 \mid Z_t = z)\hat{R}_t(z, y \mid x_0)$. By Section 4.24, this equals the marginal reverse rate $\hat{R}_t(z, y)$. Now expand ℓ and separate the terms involving θ. The difference between the losses is $\mathbb{E}[A \log A] - \mathbb{E}[A] \log \mathbb{E}[A]$, which is independent of θ. Their gradients therefore agree term by term.

We can estimate the conditional loss from corrupted training sequences and their conditional rates. Optimizing it recovers the marginal reverse rates without evaluating the posterior average explicitly.

4.26.4 The bound on the likelihood

The same loss also controls the likelihood that the model assigns to data. Adding the mismatch at the final time between the fully noised data and the prior we actually sample from,

$$-\log p_\theta(x_0) \le \mathcal{L}_{\text{cond}}(x_0; \theta) + D_{\text{KL}}(q_1(\cdot \mid x_0) \,\|\, p_1)$$

"The negative log-likelihood of a clean sequence x_0 is at most its conditional rate loss $\mathcal{L}_{\text{cond}}(x_0; \theta)$ plus the KL divergence between $q_1(\cdot \mid x_0)$, the distribution of x_0 after full corruption, and the prior p_1 that generation starts from."

The derivation follows the discrete-time argument of Section 4.11. We write the likelihood as an expectation over paths of the model, apply Jensen's inequality to move the logarithm inside, and recognize the result as the path divergence computed above. When the forward process ends at its stationary distribution and we take $p_1 = \pi$, the second term is zero and the loss bounds the negative log-likelihood on its own. The continuous-time form of this bound belongs to the CTMC construction of discrete denoising [Campbell et al., 2022], and in SEDD the same quantity is written in terms of ratios [Lou et al., 2024].

Minimizing the loss therefore moves the model rates toward the marginal reverse rates and raises the lower bound on the data likelihood. We still need a parameterization, a way to turn a network output into nonnegative rates.

4.27 Sampling with a denoiser

In Section 4.24.3, we expressed the marginal reverse rate as a posterior average over the clean token at one position. Substituting the denoiser for that posterior gives the model rate:

$$\hat{R}_t^\theta(z, z^{i \leftarrow a}) = R_t(z^{i \leftarrow a}, z) \sum_b d_\theta^i(b \mid z, t) \, \frac{q_t(a \mid b)}{q_t(z^i \mid b)}$$

"The model rate $\hat{R}_t^\theta(z, z^{i \leftarrow a})$ for changing position i of sequence z to token a equals the opposite-direction forward rate $R_t(z^{i \leftarrow a}, z)$ times a denoiser-weighted average of corruption ratios. For each candidate clean token b, multiply its denoiser probability $d_\theta^i(b \mid z, t)$ by the conditional probability ratio $q_t(a \mid b)/q_t(z^i \mid b)$, then sum over b."

Every factor on the right is nonnegative, so every model rate is nonnegative without any constraint on the network, and the denoiser is already normalized by its softmax. When the denoiser matches the true posterior, every model rate equals the marginal reverse rate and the loss attains its minimum. The network output is one categorical distribution per position, and the masked construction of Section 4.14 already trains exactly that.

Let's run the worked example through this formula. At $\alpha_t = 1/2$ with uniform noise, suppose the denoiser at the position holding C predicts $d_\theta = 0.375$ for A and 0.625 for C. For the jump to A,

$$\hat{R}_t^\theta(\mathrm{C}, \mathrm{A}) = 1 \cdot \Big(0.375 \cdot \frac{0.625}{0.125} + 0.625 \cdot \frac{0.125}{0.625}\Big) = 0.375 \times 5 + 0.625 \times 0.2 = 2$$

and for the jump to G, $0.375 \times (0.125/0.125) + 0.625 \times (0.125/0.625) = 0.375 + 0.125 = 0.5$, with the same value for T. From the denoiser output, we recover the same rates calculated directly from the marginals in Section 4.24.2.

4.27.1 Sampling with short reverse steps

We can generate sequences by applying Euler integration to the jump process. We can start at $t = 1$ from π, pick a step size h, and over each step let every position jump independently with probability proportional to its reverse rate:

$$\mathbb{P}(Z_{t-h}^i = a \mid Z_t = z) \approx h\, \hat{R}_t^\theta(z, z^{i \leftarrow a}), \qquad a \neq z^i$$

"The probability that position i holds the token a at the earlier time $t - h$, given the sequence z at time t, is approximately h times the model rate $\hat{R}_t^\theta(z, z^{i \leftarrow a})$, for every token a other than the current z^i."

The remaining probability keeps the position unchanged. The step is valid when h times the total reverse rate at every position is at most one, the same condition that bounded the forward step. Several positions can change within a single step, so one network evaluation advances the entire sequence, and the approximation becomes exact as h shrinks. This scheme is called τ-leaping. Its error per step is of order h^2 at each position, and it comes from treating the rates as frozen over the step and from ignoring the chance of two jumps at the same position.

We take the three rates 2, 0.5 and 0.5 out of C, whose total is 3. The step is valid for $h \leq 1/3$. At $h = 0.05$ the position moves to A with probability 0.1, to G with probability 0.025, to T with probability 0.025, and stays at C with probability 0.85. The exact holding probability over an interval of that length at constant rates is $e^{-3 \times 0.05} = 0.8607$, so the linear step understates it by 0.0107, and the error shrinks with h^2. For the π family we can avoid the approximation entirely by taking an exact posterior step over any interval, as Section 4.6 did for masking.

4.27.2 A corrector step at a fixed time

Finite steps and denoiser errors can accumulate during sampling, so the sample distribution can depart from the intended marginal p_t. We can reduce these errors using a fixed-time process that preserves p_t. Add the forward and reverse rate matrices at the same time:

$$C_t = R_t + \hat{R}_t$$

"The corrector rate matrix C_t equals the forward rate matrix R_t plus the reverse rate matrix $\hat{R}_t$, both taken at the same time t."

To check the fluxes under C_t, apply the reversal identity $p_t(x)R_t(x,y) = p_t(y)\hat{R}_t(y,x)$ in each direction:

$$p_t(x)\, C_t(x,y) = p_t(x)R_t(x,y) + p_t(x)\hat{R}_t(x,y) = p_t(y)\hat{R}_t(y,x) + p_t(y)R_t(y,x) = p_t(y)\, C_t(y,x)$$

"The probability $p_t(x)$ times the corrector rate $C_t(x,y)$ from x to y equals the probability $p_t(y)$ times the corrector rate $C_t(y,x)$ from y back to x."

Thus C_t satisfies detailed balance with p_t, so p_t is stationary under these rates. We can hold the noise time t fixed and simulate C_t to reduce deviations from this marginal. During sampling, we alternate predictor steps that change the noise time with corrector steps at a fixed noise time [Campbell et al., 2022].

Let's check the balance on the DNA numbers. At $\alpha_t = 1/2$ with $p_t = (0.5, 0.25, 0.125, 0.125)$ and unit forward rates, the reverse rates are $\hat{R}_t(\mathrm{C},\mathrm{A}) = 2$ and $\hat{R}_t(\mathrm{A},\mathrm{C}) = p_t(\mathrm{C})/p_t(\mathrm{A}) = 0.5$. The corrector rates on that edge are $C_t(\mathrm{C},\mathrm{A}) = 1 + 2 = 3$ and $C_t(\mathrm{A},\mathrm{C}) = 1 + 0.5 = 1.5$, and the two fluxes agree: $0.25 \times 3 = 0.75$ and $0.5 \times 1.5 = 0.75$. On the edge between C and G the rates are 1.5 and 3, and both fluxes equal 0.375.

4.28 Discrete diffusion requirements

For short intervals, the transition matrix satisfies $P_{t,t+h} = I + hR_t + o(h)$. Probability follows the master equation $\dot{p}_t = p_t R_t$, which the matrix exponential $P_{0,t} = e^{\tau(t)R}$ solves for a constant rate matrix with clock τ. Detailed balance holds edge by edge against the stationary distribution, $\pi(x)R(x,y) = \pi(y)R(y,x)$. Reversal is a reweighting by probability ratios, $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$, and those ratios come from a posterior average over clean data, $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$. Conditioning on anything observed later, including the reversal itself, is an h-transform $R_t^h(x,y) = R_t(x,y)\, h_t(y)/h_t(x)$. Training minimizes the rate loss $\int_0^1 \mathbb{E} \sum_y \ell(\hat{R}_t(\cdot \mid x_0), \hat{R}_t^\theta)\, dt$, whose minimizer is the marginal reverse rate and which bounds the negative log-likelihood.

For generative modeling, the construction must satisfy these conditions:

1. The forward rates are nonnegative off the diagonal, with zero row sums, so that R_t is a rate matrix and p_t stays a probability vector.

2. The conditional marginal $q_t(z \mid x_0)$ has a closed form, so that training can sample any noise level directly without simulating the corruption.

3. The forward process ends at a distribution we can sample from, ideally its stationary distribution, so that generation has a place to start.

4. The reverse rates follow from the posterior over clean tokens, one position at a time, so that a single network output determines them.

5. The loss bounds the likelihood and is minimized at the true reverse rates.

The two π-family processes of the previous section satisfy all five. In masking, $\pi = \delta_m$ on the masked alphabet $\mathcal{V}_m$, the process reaches that distribution through one-way jumps, and the reverse process can only reveal. In uniform replacement, π is uniform on $\mathcal{V}$, the process is in detailed balance with it, and the reverse process can revise a token it has already written. Both share the kernel $q_t(x \mid x_0) = \alpha_t \delta(x, x_0) + (1 - \alpha_t)\pi(x)$, the denoiser, and the rate loss, and the choice between them is the choice of π.

We can change the rate matrix R_t, survival schedule α_t and reveal rate $w(t) = -\dot{\alpha}_t/(1 - \alpha_t)$. We can use the sequence space $\mathcal{V}^L$, the masked alphabet $\mathcal{V}_m$, blocks of positions or the interior of a simplex. The network can predict clean-token probabilities $d_\theta(a \mid z, t)$, probability ratios or per-token schedules. We also choose the loss weights and the sampler, including its step rule, number of steps and any corrector.

4.29 Discrete diffusion variants

A discrete diffusion model is specified by the five facts of Section 4.28. Each published model meets that specification with one choice of rate matrix, and the forward marginal, the distribution generation starts from, the reverse rates and the training loss all follow from that choice.

Three models use the π family from Section 4.20, with different choices of π and continuous or discrete time. Other models change the network prediction, assign a schedule to each token, generate blocks of a sequence or use probability vectors on the simplex. We can also vary the reveal order, loss weighting or target distribution while keeping the rates fixed.

4.30 Masked diffusion with $\pi = \delta_m$

We can recover the masked model from Sections 4.5 to 4.14 by choosing a π-family distribution with all its probability on the mask token.

The alphabet is $\mathcal{V}_m$, the vocabulary enlarged by the mask token m in Section 4.5, so that a position holds one of the $K = |\mathcal{V}|$ clean tokens or the mask. We can set $\pi = \delta_m$, the distribution with $\pi(m) = 1$ and $\pi(a) = 0$ for every clean token a. The rate matrix $R_t = \beta(t)R_\pi$ of the family then has only one kind of nonzero off-diagonal entry:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

"The rate $R_t(a, m)$ at which a position holding the clean token a jumps to the mask token m equals the schedule $\beta(t)$, and the rate $R_t(m, a)$ of the jump back from the mask to any clean token a is zero, so once a position is masked it stays masked."

- $\beta(t) \geq 0$ is the masking schedule, in units of probability per unit time, shared by every clean token.

- m is the mask token, which is in the state space of the corrupted sequence and outside the vocabulary of the data.
- a ranges over the clean tokens $\mathcal{V}$, which for DNA are A, C, G and T.

Written out for DNA, with the states ordered A, C, G, T, m, the rate matrix is

$$R_t = \beta(t) \begin{pmatrix} -1 & 0 & 0 & 0 & 1 \\ 0 & -1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Every off-diagonal entry is $\beta(t)$ or zero, and every row sums to zero, including the all-zero mask row. The matrix therefore satisfies the rate constraints.

To calculate the forward marginal at each position, substitute $\pi = \delta_m$ into the matrix exponential from Section 4.20:

$$q_t(z^i \mid x_0^i) = \alpha_t \, \delta(z^i, x_0^i) + (1 - \alpha_t) \, \delta(z^i, m)$$

"The probability $q_t(z^i \mid x_0^i)$ that position i holds the noisy token z^i, given that it held the clean token x_0^i, equals the survival probability α_t when z^i is the clean token itself, and one minus α_t when z^i is the mask."

This is the corruption kernel we wrote down by hand in Section 4.5, so the two derivations agree. The survival probability depends on the schedule through $\alpha_t = e^{-\tau(t)}$ with $\tau(t) = \int_0^t \beta(s)\, ds$, and we can sample any noise level directly from this one line, with no trajectory in between.

As $\alpha_t \to 0$ the kernel places all of its probability on m at every position, so the forward process ends at the all-mask sequence, and δ_m is stationary for R_t because the mask row is zero. Generation starts there.

4.30.1 The reverse rates for masking

To calculate the reverse rate at a masked position, first hold the clean token fixed and apply the reversal identity:

$$\hat{R}_t(m, a \mid x_0) = R_t(a, m) \, \frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t) \, \frac{\alpha_t \, \delta(a, x_0)}{1 - \alpha_t}$$

"The conditional reveal rate $\hat{R}_t(m, a \mid x_0)$ equals the forward masking rate $R_t(a, m) = \beta(t)$ times the conditional token probability $q_t(a \mid x_0) = \alpha_t \delta(a, x_0)$, divided by the mask probability $q_t(m \mid x_0) = 1 - \alpha_t$. The indicator $\delta(a, x_0)$ restricts the reveal to the true clean token x_0."

The indicator $\delta(a, x_0)$ is zero at every destination except the true clean token, so conditioned on x_0 the position can only be revealed as x_0 itself. If we substitute $\beta(t) = -\dot{\alpha}_t/\alpha_t$, the survival probability cancels,

$$\hat{R}_t(m, a \mid x_0) = \frac{-\dot{\alpha}_t}{\alpha_t} \cdot \frac{\alpha_t \, \delta(a, x_0)}{1 - \alpha_t} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \, \delta(a, x_0) = w(t) \, \delta(a, x_0),$$

with $w(t) = -\dot{\alpha}_t/(1 - \alpha_t)$ the reveal rate of Section 4.14. We then average over the posterior and put the denoiser $d^i_\theta(a \mid z, t)$ in its place, which produces the model rate:

$$\hat{R}^\theta_t(m, a) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \, d^i_\theta(a \mid z, t)$$

"The model reveal rate $\hat{R}^\theta_t(m, a)$ at masked position i equals the negative survival derivative $-\dot{\alpha}_t$ divided by the masking probability $1 - \alpha_t$, times the denoiser probability $d^i_\theta(a \mid z, t)$ of clean token a given the noisy sequence z and time t."

The total reveal rate at a masked position is $w(t) \sum_a d^i_\theta(a \mid z, t) = w(t)$, independent of the denoiser, and the denoiser splits that total over the clean tokens.

This is the learned reverse step of Section 4.7 written as a rate, and the two agree term for term. Over a step from t down to $s = t - h$ the rate $w(t) \, d^i_\theta(a \mid z, t)$ integrates to the reveal probability $r_{s,t} \, d^i_\theta(a \mid z, t)$, because $r_{t-h,\,t} = h \, w(t) + O(h^2)$ by the expansion of Section 4.14. We recover the same reverse step from Bayes' rule on a finite interval and from the rate matrix.

Now consider a visible position containing the clean token a in sequence z. The reverse rate that changes it to b is

$$\hat{R}_t(z, z^{i \leftarrow b}) = R_t(z^{i \leftarrow b}, z) \, \frac{p_t(z^{i \leftarrow b})}{p_t(z)},$$

and the first factor is the forward rate of a jump from b into a at position i. Masking moves clean tokens to the mask and nowhere else, so every forward rate into a clean token is zero, and every reverse rate out of a visible position is zero with it. A revealed token is never revised. This is the carry-over unmasking rule of MDLM [Sahoo et al., 2024], which follows directly from the rate matrix. The second rule of the same parameterization follows from the posterior. The mask is never the clean token at any position, so $p(x^i_0 = m \mid z) = 0$, and the denoiser must put no probability on m.

A half-masked sequence therefore has activity only at its masked positions, and at each of them the reveal rate is divided over the alphabet in proportion to the denoiser probabilities (Figure 4.10).

4.30.2 A worked reveal step

We can take the sequence of Figure 4.10 with the linear schedule $\alpha_t = 1 - t$, so that $\dot{\alpha}_t = -1$ and

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} = \frac{1}{t}.$$

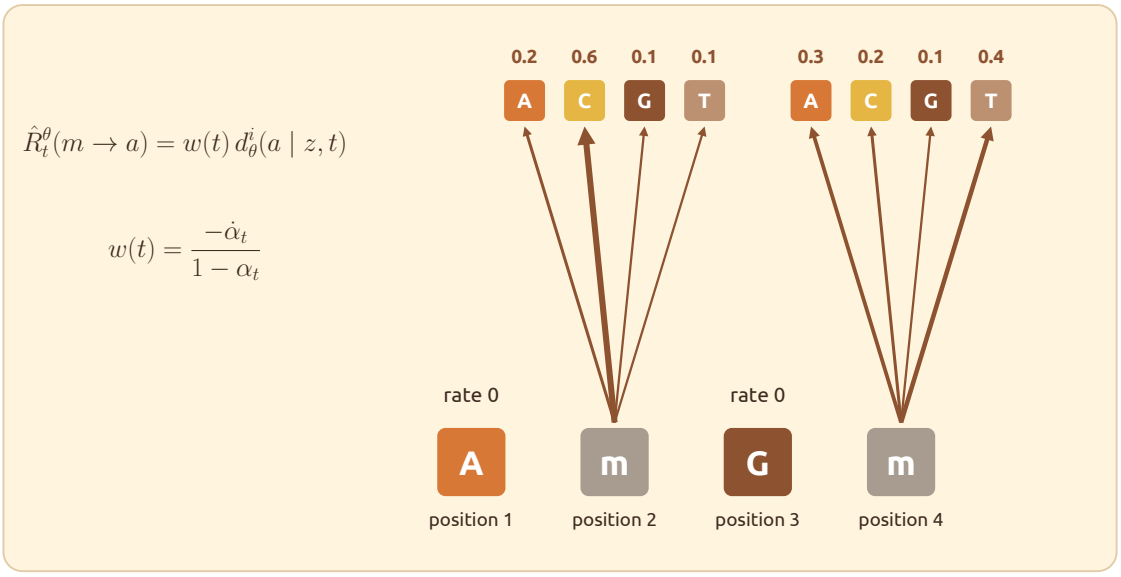


Figure 4.10: A four-position DNA sequence halfway through generation, holding A, m, G, m. Each masked position leaves at the total rate $w(t) = -\dot{\alpha}_t/(1-\alpha_t)$, split over the four bases in proportion to the denoiser probabilities printed above the arrows, which are $(0.2, 0.6, 0.1, 0.1)$ over (A, C, G, T) at position 2 and $(0.3, 0.2, 0.1, 0.4)$ at position 4. Arrow thickness is proportional to the rate. The visible A at position 1 and the visible G at position 3 have rate zero in every direction.

At $t = 0.5$ the reveal rate is $w(0.5) = 2$. Position 2 has the denoiser probabilities $(0.2, 0.6, 0.1, 0.1)$ over (A, C, G, T), so its four reveal rates are

$$2 \times (0.2,\ 0.6,\ 0.1,\ 0.1) = (0.4,\ 1.2,\ 0.2,\ 0.2),$$

which sum to 2. Position 4 has $(0.3, 0.2, 0.1, 0.4)$ and its rates are $(0.6, 0.4, 0.2, 0.8)$, again summing to 2. Over a reverse step of length $h = 0.02$, each masked position is revealed with probability about $h\, w(t) = 0.04$, and when position 2 is revealed it becomes C with probability $1.2/2 = 0.6$. The exact step over the interval from $t = 0.5$ down to $s = 0.48$ reveals with probability

$$\frac{\alpha_s - \alpha_t}{1-\alpha_t} = \frac{t-s}{t} = \frac{0.02}{0.5} = 0.04,$$

which agrees with the short-step approximation to the digits shown, because the reveal rate barely moves over an interval this short. Positions 1 and 3 hold A and G and have rate zero, so the probability that either changes in this step is zero exactly, at any step length.

4.30.3 The loss for masking

We can derive the masking loss from the path-KL rate error $\ell(A, B) = A \log(A/B) - A + B$, where A is the target rate and B is the model rate. At a masked position, substitute the target rates $w(t)\delta(a, x_0^i)$ and model rates $w(t) d_\theta^i(a \mid z, t)$, then sum over destinations a:

$$\sum_a \ell = w(t)\Big[-\log d_\theta^i(x_0^i \mid z,t) - 1 + \sum_a d_\theta^i(a \mid z,t)\Big] = w(t)\big[-\log d_\theta^i(x_0^i \mid z,t)\big]$$

"The total rate error at a masked position equals the reveal rate $w(t)$ times the negative log probability $-\log d_\theta^i(x_0^i \mid z,t)$ of the correct clean token x_0^i. The additional terms cancel because the denoiser probabilities $d_\theta^i(a \mid z,t)$ sum to one over candidate tokens a."

At the true destination $a = x_0^i$, the target rate is $w(t)$ and the model rate is $w(t)d_\theta^i(x_0^i \mid z,t)$. Its contribution is $w(t)[-\log d_\theta^i - 1 + d_\theta^i]$. Every other destination has target rate zero, so $\ell(0,B) = B$ contributes its model rate $w(t)d_\theta^i(a \mid z,t)$. Adding these model probabilities gives $\sum_a d_\theta^i = 1$, which cancels the -1. Thus probabilities assigned to wrong tokens enter through the $+B$ staying term and complete the normalization.

At a visible position every target rate and every model rate is zero, so the error there is zero. Adding over the masked positions and integrating over time,

$$\mathcal{L}_{\text{MDLM}} = \int_0^1 \frac{-\dot\alpha_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i(x_0^i \mid Z_t, t)\Big]\, dt$$

"The MDLM loss $\mathcal{L}_{\text{MDLM}}$ integrates the reveal-rate weight $-\dot\alpha_t/(1-\alpha_t)$ times the expected sum of negative log probabilities $-\log d_\theta^i(x_0^i \mid Z_t,t)$ of the true clean tokens. We average over clean sequences x_0 and noisy sequences Z_t, sum over masked positions i, and integrate over times t from zero to one."

This is $\mathcal{L}_{\text{mask}}$, the weighted cross-entropy of Section 4.14. We reached it there through the variational bound of Section 4.11 one reverse step at a time, and here it follows from the rate loss in a single substitution. Both derivations give the same weight $w(t)$, sum over masked positions and normalization cancellation. Masked diffusion therefore satisfies the reverse-rate and loss conditions with $\pi = \delta_m$.

Suppose the clean sequence is ACGT, the noisy sequence at $t = 0.5$ is $\mathrm{A}\, m\, \mathrm{G}\, m$, and the denoiser probabilities are the ones in Figure 4.10, so $d_\theta^2(\mathrm{C}) = 0.6$ and $d_\theta^4(\mathrm{T}) = 0.4$. At position 2 the four rate errors are

$$\ell(2,\, 1.2) = 0.2217, \qquad \ell(0,\, 0.4) = 0.4, \qquad \ell(0,\, 0.2) = 0.2, \qquad \ell(0,\, 0.2) = 0.2,$$

and they sum to 1.0217, which is $w(t)(-\log 0.6) = 2 \times 0.5108$ to four figures. The cancellation is visible in the arithmetic: the three wrong destinations contribute 0.8, which is $w(t)$ times the probability 0.4 the denoiser placed off the clean token, and the -1 cancels it exactly. Position 4 contributes $2 \times (-\log 0.4) = 1.8326$. The two visible positions contribute nothing, and the integrand at this draw is 2.8542 nats.

Trained this way on LM1B, the masked-diffusion likelihood bound reached a perplexity of 23.00 against 20.86 for an autoregressive transformer retrained on the same budget of 327B tokens [Sahoo et al., 2024]. Masked diffusion is a few points of perplexity behind autoregression at that budget, and it fills positions in any order.

The reverse rates are zero at visible positions, so an early token error remains in the final sequence.

4.31 D3PM in discrete time

D3PM [Austin et al., 2021], the first general discrete diffusion model, uses one transition matrix per discrete step. It preceded the continuous-time construction by two years. We can connect its finite-step transitions to the rate-based description.

A D3PM forward process has a finite number of steps $t = 1, \dots, T$ and a transition matrix Q_t for each of them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}(x_t;\ x_{t-1}Q_t), \qquad q(x_t \mid x_0) = \mathrm{Cat}(x_t;\ x_0\bar{Q}_t)$$

"The one-step law $q(x_t \mid x_{t-1})$ is categorical with probability vector $x_{t-1}Q_t$, obtained by multiplying the one-hot token row x_{t-1} by the transition matrix Q_t. Starting from the clean token x_0, the law $q(x_t \mid x_0)$ uses $x_0\bar{Q}_t$, where $\bar{Q}_t = Q_1Q_2 \cdots Q_t$ combines the first t transitions."

- x_t is a one-hot row vector of length K holding the token at step t, so that $x_{t-1}Q_t$ selects a row of Q_t.
- Q_t is a $K \times K$ matrix of transition probabilities, with nonnegative entries and rows summing to one.
- $\bar{Q}_t$ is the cumulative transition matrix, the product of the first t matrices.

To evaluate $\bar{Q}_t$ in closed form, choose a family closed under multiplication. Set $Q_t = (1-\beta_t)I + \beta_t \mathbf{1}\pi^\top$, where $\beta_t \in [0, 1]$ is the redraw probability at step t. This is the one-step version of the π-family rate matrix. Write $M = \mathbf{1}\pi^\top$, whose rows equal π and which satisfies $M^2 = M$. Multiplying two family members gives

$$\big(aI + (1-a)M\big)\big(cI + (1-c)M\big) = ac\, I + (1-ac)\, M$$

"Multiplying transition matrices with survival probabilities a and c gives a matrix with survival probability ac. The result is ac times the identity I, plus the redraw probability $1 - ac$ times the common redraw matrix M."

Writing the product out, we have $ac\, I + a(1-c)M + (1-a)c\, M + (1-a)(1-c)M^2$, and $M^2 = M$ collects the last three terms into a single multiple of M with coefficient

$$a(1-c) + (1-a)c + (1-a)(1-c) = a - ac + c - ac + 1 - a - c + ac = 1 - ac.$$

Applying this to the whole product,

$$\bar{Q}_t = \bar{\alpha}_t\, I + (1 - \bar{\alpha}_t)\, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

"The cumulative transition matrix $\bar{Q}_t$ combines the identity I with weight $\bar{\alpha}_t$ and the redraw matrix $\mathbf{1}\pi^\top$ with weight $1-\bar{\alpha}_t$. The cumulative survival probability $\bar{\alpha}_t$ multiplies the per-step survival probabilities $1-\beta_s$ over steps $s=1$ through t."

The cumulative survival probability $\bar{\alpha}_t$ determines this closed-form marginal. We take uniform π on DNA, $\beta_1 = 0.2$ and $\beta_2 = 0.3$. Then $\bar{\alpha}_2 = 0.8 \times 0.7 = 0.56$ and

$$\bar{Q}_2 = 0.56\, I + 0.44 \cdot \tfrac{1}{4}\mathbf{1}\mathbf{1}^\top = \begin{pmatrix} 0.67 & 0.11 & 0.11 & 0.11 \\ 0.11 & 0.67 & 0.11 & 0.11 \\ 0.11 & 0.11 & 0.67 & 0.11 \\ 0.11 & 0.11 & 0.11 & 0.67 \end{pmatrix},$$

which agrees entry by entry with multiplying Q_1 and Q_2 directly. A clean A is still A after two steps with probability 0.67, made of the 0.56 chance of never being redrawn and the $0.44 \times 0.25 = 0.11$ chance of being redrawn onto A.

The continuous-time limit now follows from the same product. We can put $\beta_s = \beta(s)\Delta t$ with $\Delta t = 1/T$. The cumulative survival probability is $\prod_s (1-\beta(s)\Delta t)$, whose logarithm is $-\sum_s \beta(s)\Delta t + O(\Delta t)$, so $\bar{\alpha}_t \to e^{-\tau(t)}$ and $\bar{Q}_t \to e^{\tau(t) R_\pi}$. With constant $\beta = 1$ and $\tau = 1$, the products are $0.9^{10} = 0.3487$, $0.99^{100} = 0.3660$ and $0.999^{1000} = 0.3677$, climbing toward $e^{-1} = 0.3679$. Absorbing D3PM, with $\pi = \delta_m$, is masked diffusion, and uniform D3PM is uniform replacement.

4.31.1 Reversing one finite step

For a finite reverse step, we apply Bayes' rule using the transition probability, earlier marginal and later marginal:

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\Big(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\Big)$$

"The categorical posterior $q(x_{t-1} \mid x_t, x_0)$ gives the probability of each earlier token x_{t-1}, conditional on the observed token x_t and clean token x_0. Multiply the candidate-to-observation probabilities $x_t Q_t^\top$ elementwise by the clean-to-candidate probabilities $x_0 \bar{Q}_{t-1}$, then divide by the observation probability $x_0 \bar{Q}_t x_t^\top$."

For each candidate token at step $t-1$, the numerator multiplies its probability given the clean token by the probability of reaching the observed token from it. The denominator normalizes by the probability of the observation.

Use uniform π on DNA with $\beta_1 = 0.2$ and $\beta_2 = 0.3$. Suppose the clean base is A and the observed base at step 2 is C. We want the posterior at step 1. Column C of Q_2, written $x_2 Q_2^\top$, is $(0.075, 0.775, 0.075, 0.075)$ over A, C, G and T. The matrix has diagonal entries 0.775 and off-diagonal entries 0.075. Row A of $\bar{Q}_1$ is $(0.85, 0.05, 0.05, 0.05)$. Their elementwise product is

$$(0.06375,\ 0.03875,\ 0.00375,\ 0.00375),$$

with sum 0.11, which is $\bar{Q}_2(\mathrm{A}, \mathrm{C})$ as it must be. Dividing,

$$q(x_1 \mid x_2 = \mathrm{C},\, x_0 = \mathrm{A}) = (0.5795,\ 0.3523,\ 0.0341,\ 0.0341).$$

The most likely history is that the base was still A after the first step and was redrawn onto C at the second, which has probability 0.580. Being redrawn onto C at the first step and surviving the second has probability 0.352, and the two paths through G and T have 0.034 each.

At generation time the clean token is unknown, so the model averages this posterior over the denoiser's prediction:

$$p_\theta(x_{t-1} \mid x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t)$$

"The reverse-step probability $p_\theta(x_{t-1} \mid x_t)$ is proportional to the joint probability $q(x_{t-1}, x_t \mid \tilde{x}_0)$ of the earlier and current tokens, averaged over candidate clean tokens $\tilde{x}_0$ with denoiser weights $\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$."

This is the finite-step posterior average. For training, we use the discrete-time variational bound from Section 4.11, with steps $t = 1, \ldots, T$ in place of the grid $0 = t_0 < \cdots < t_N = 1$ and clean data at the same endpoint. We add an auxiliary clean-token cross-entropy with coefficient λ_{aux}. The continuous-time limit is the rate loss.

4.31.2 Rate matrices chosen for the data

We can also choose D3PM rates that depend on the relationship between tokens. For pixel intensities a discretized Gaussian kernel moves probability to nearby intensities first, so a corrupted image stays an image for longer. On five intensity levels $0, \ldots, 4$ with $\sigma = 1$, the off-diagonal weights out of level 2 are proportional to $e^{-(2-j)^2/2}$, which is 0.6065 for levels 1 and 3 and 0.1353 for levels 0 and 4. Normalized to a total rate of one, the row holds 0.4088 to each neighbour and 0.0912 to each far level, so a mid-grey pixel is four and a half times more likely to move one step than four.

For text, a nearest-neighbour graph A built from word embeddings plays the same role, and exponentiating its rate matrix produces the transition matrix:

$$Q_t = \exp\left(\tau_t\, R\right), \qquad R(i,j) = A_{ij} \quad (i \neq j), \qquad R(i,i) = -\sum_{l \neq i} A_{il}$$

"The transition matrix Q_t equals the exponential of the accumulated rate τ_t times the rate matrix R. For distinct tokens i and j, the rate $R(i,j)$ is the graph weight A_{ij}; the diagonal entry $R(i,i)$ is the negative sum of all outgoing graph weights."

We take four tokens with weights $A_{\mathrm{AC}} = 1$, $A_{\mathrm{CG}} = 1$, $A_{\mathrm{GT}} = 1$ and $A_{\mathrm{AG}} = 0.5$, the rest zero. The rate matrix is

$$R = \begin{pmatrix} -1.5 & 1 & 0.5 & 0 \\ 1 & -2 & 1 & 0 \\ 0.5 & 1 & -2.5 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix},$$

whose rows sum to zero, as required for a rate matrix. At $\tau_t = 0.3$ its exponential has first row

$$(0.6747,\ 0.1964,\ 0.1122,\ 0.0167),$$

so the token reaches its graph neighbour with probability 0.196, the half-weight neighbour with 0.112, and the token two hops away with 0.017. Because A is symmetric, $\pi(i)R(i,j) = \pi(j)R(j,i)$ holds for the uniform π, so this rate matrix satisfies detailed balance with the uniform distribution, as defined in Section 4.22. It also satisfies the other conditions of the construction.

On LM1B, uniform corruption reached a perplexity bound of 137.9 and absorbing corruption 76.9 with the same model and the same budget [Austin et al., 2021]. We put the structure of the data into the model through the choice of rate matrix, and those two choices of π separate the bound by a factor of nearly two with the network held fixed.

4.32 Uniform replacement rates

With masking, we identify every corrupted position directly from the mask symbols. Under uniform replacement, a noisy token may be original or replaced, and we cannot distinguish these cases from the token alone. We therefore evaluate possible changes at every position, including those already updated. UDLM [Schiff et al., 2025] is this member of the family, with π uniform over the K tokens.

Choose $R_t = \beta(t)(\mathbf{1}\pi^\top - I)$ with $\pi = \mathbf{1}/K$, giving rate $\beta(t)/K$ between distinct tokens. The forward marginal is $q_t(x \mid x_0) = \alpha_t \delta(x, x_0) + (1 - \alpha_t)/K$. Its endpoint and stationary distribution are both uniform over the K tokens, so we begin generation with uniformly random tokens.

Apply the reversal identity to this kernel to obtain the conditional rate from the current token c to a different token a:

$$\hat{R}_t(c, a \mid x_0) = \frac{\beta(t)}{K} \cdot \frac{\alpha_t\, \delta(a, x_0) + \frac{1-\alpha_t}{K}}{\alpha_t\, \delta(c, x_0) + \frac{1-\alpha_t}{K}}$$

"The conditional reverse rate $\hat{R}_t(c, a \mid x_0)$ from the current token c to token a equals the forward rate $\beta(t)/K$ times the ratio of their noisy-token probabilities given the clean token x_0. Each probability includes the uniform redraw mass $(1-\alpha_t)/K$, with survival mass α_t added at the clean token, where K is the vocabulary size."

Divide by the noise rate $\beta(t)/K$ to find the factor for a jump toward the clean token, away from it, or between two wrong tokens:

$$\frac{\hat{R}_t(c, a \mid x_0)}{\beta(t)/K} = \begin{cases} 1 + \frac{K\alpha_t}{1-\alpha_t} & a = x_0 \\ \left(1 + \frac{K\alpha_t}{1-\alpha_t}\right)^{-1} & c = x_0 \\ 1 & \text{otherwise} \end{cases}$$

"The conditional reverse rate $\hat{R}_t(c, a \mid x_0)$, divided by the noise rate $\beta(t)/K$, equals $1 + K\alpha_t/(1-\alpha_t)$ for a jump to the clean token $a = x_0$. It equals the reciprocal for a jump away from the clean token $c = x_0$, and one when neither token is clean. Here K is the vocabulary size and α_t is the survival probability."

The three cases are a jump toward the clean token, a jump away from it, and a jump between two tokens that are both wrong. The last of these runs at the bare noise rate, because the posterior favours neither of its endpoints.

For $K = 4$ and $\alpha_t = 1/2$, the factor is $1 + 4(0.5)/(0.5) = 5$, giving rates 5, 0.2 and 1 when the forward rates are one. This matches Section 4.24.2: the kernel assigns 0.625 to the clean base and 0.125 elsewhere, with ratio $0.625/0.125 = 5$.

4.32.1 The rate error on one base

Every token can jump to every other token, so all $K - 1$ destinations contribute to the rate error at a position, and the arithmetic is longer than for masking. Let's take the clean base to be A, the current base to be C, $\alpha_t = 1/2$, unit forward rates, and a denoiser that predicts $(0.5, 0.3, 0.1, 0.1)$ over (A, C, G, T).

We can start with the jump from C to A. Its conditional rate depends on which base was clean: 5 if the clean base was A, 0.2 if it was C, and 1 if it was G or T. Averaging those four conditional rates with the denoiser probabilities,

$$\hat{R}^\theta(\mathrm{C}, \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

"The model jump rate $\hat{R}^\theta(\mathrm{C}, \mathrm{A})$ from C to A averages the conditional rates 5, 0.2, 1 and 1 for clean bases A, C, G and T, using their denoiser probabilities 0.5, 0.3, 0.1 and 0.1. The resulting rate is 2.76."

For the jump from C to G, permute the conditional rates to 1, 0.2, 5 and 1 for clean bases A, C, G and T. Their denoiser-weighted average is $0.5(1) + 0.3(0.2) + 0.1(5) + 0.1(1) = 1.16$. The rate from C to T is also 1.16, since the denoiser assigns equal probabilities to G and T.

The targets are the conditional rates at the true clean base A, which are 5 toward A and 1 toward each of G and T. The rate error is

$$\ell(5,\, 2.76) + 2\,\ell(1,\, 1.16) = 0.731 + 2(0.0116) = 0.754$$

"The rate error $\ell(5, 2.76)$ for the jump back to A plus twice the rate error $\ell(1, 1.16)$ for the jumps to G and T equals 0.731 plus two times 0.0116, which is 0.754."

Almost all of the loss comes from the first term. Underestimating the rate of returning to the clean base by a factor of about two costs 0.731 nats. Overestimating the two uninformative rates by 16% costs 0.0116 each. The rate error is asymmetric in exactly this way, because the $A\log(A/B)$ term grows without bound as B falls toward zero. Too little rate on the right token raises the error far more than a little extra rate spread over the wrong ones.

By the conditional rate-matching theorem, the posterior-weighted error is minimized at the posterior-average rate. The uniform kernel has no closed-form collapse like the one that reduced the masking loss to a single cross-entropy, so in UDLM we evaluate the bound as written, summed over the $K-1$ destinations at every position.

The relative performance of masking and uniform replacement depends on the alphabet. With a small alphabet a uniform replacement is easy to spot, because a wrong token has few plausible alternatives around it and the denoiser can correct it. With a large alphabet a replacement is almost invisible. Uniform diffusion was ahead of masking on Species10 with 12 tokens and on QM9 with 40 tokens, and behind it on LM1B with 30,522 tokens [Schiff et al., 2025]. In these results, uniform replacement performed better for the smaller vocabularies.

Both members of the family reach the reverse rates through a denoiser over clean tokens. The rates depend only on the ratio $p_t(z^{i\leftarrow a})/p_t(z)$, and a model can predict it directly.

4.33 Predicting the ratios directly

In SEDD [Lou et al., 2024], we train a network to predict the ratio $p_t(z^{i\leftarrow a})/p_t(z)$ directly. The output $s_\theta(z)_{i,a}$ is the concrete score for changing position i to token a. A sequence of length L requires $L(K-1)$ such numbers. This avoids computing the ratio from a clean-token denoiser and the forward kernel.

We now ask what the rate error becomes when we write both the target and the model as a forward rate times a ratio. Let λ be the forward rate of the jump, r the true ratio and s the predicted ratio, so the target rate is λr and the model rate is λs. Substituting into $\ell(A,B) = A\log(A/B) - A + B$,

$$\ell(\lambda r,\ \lambda s) = \lambda r \log\frac{\lambda r}{\lambda s} - \lambda r + \lambda s = \lambda\big[s - r\log s + r(\log r - 1)\big]$$

"The rate error $\ell(\lambda r, \lambda s)$ equals the forward rate λ times the predicted ratio s, minus λ times the true ratio r times the log predicted ratio $\log s$, plus the target-only term $\lambda r(\log r - 1)$. The target rate is λr, and the model rate is λs."

- $\lambda = R_t(z^{i\leftarrow a}, z)$ is the forward rate of the jump that would undo the reverse jump, in probability per unit time.
- $r = p_t(z^{i\leftarrow a})/p_t(z)$ is the true ratio, which the construction calls the concrete score.
- $s = s_\theta(z)_{i,a}$ is the network's prediction of that ratio, a positive number with no normalization constraint.

This is score entropy. The prediction appears only in the linear term s and logarithmic term $-r \log s$, whose sum is minimized at $s = r$. The target-only term $r(\log r - 1)$ has no effect on the gradient. It makes the error zero at the optimum, where the bracket is $r - r \log r + r \log r - r = 0$.

The likelihood bound and its minimizer are unchanged. With this parameterization, the network outputs $L(K - 1)$ positive ratios. We obtain the rates directly from these predictions without summing over clean tokens.

Return to the DNA example with uniform noise, $\alpha_t = 1/2$ and unit jump rates. The clean base is A with prior probability 0.75 and C with probability 0.25, and we observe C. In Section 4.24.2, we calculated posterior probabilities 0.375 for A and 0.625 for C, with marginal ratio $0.5/0.25 = 2$. Averaging the conditional ratios with the denoiser probabilities gives

$$\sum_b p(x_0 = b \mid \mathrm{C}) \, \frac{q_t(\mathrm{A} \mid b)}{q_t(\mathrm{C} \mid b)} = 0.375 \times \frac{0.625}{0.125} + 0.625 \times \frac{0.125}{0.625} = 0.375(5) + 0.625(0.2) = 2,$$

so the target ratio for the A direction at this position is $r = 2$, and the forward rate is $\lambda = 1$. Suppose the network predicts $s = 1.8$. Then

$$\ell(2,\ 1.8) = 1.8 - 2 \log 1.8 + 2(\log 2 - 1) = 1.8 - 1.1756 - 0.6137 = 0.0107.$$

A 10% error in the ratio costs about one hundredth of a nat on this jump. A network that predicted $s = 1$, treating the two bases as equally likely, would incur $2 \log 2 - 1 = 0.3863$, and one that predicted $s = 3$ would incur 0.1891. The asymmetry of Section 4.32 is visible again, because the same factor of 1.5 costs more when the prediction is too small.

SEDD uses the forward process, closed-form marginal and noise endpoint of its chosen π-family member. The change is in the network output.

Generating a whole sequence in parallel makes exact likelihood evaluation intractable. Splitting the sequence into blocks allows us to vary the degree of parallel generation and likelihood evaluation.

4.34 Blocks between diffusion and autoregression

Masked diffusion generates every position in parallel and bounds the likelihood. Autoregression evaluates the likelihood exactly, generates sequences of any length, and reuses a key-value cache across steps. In block diffusion [Arriola et al., 2025] we keep the chain rule between blocks and run diffusion inside each one, so both properties hold in the same model.

We can split a sequence of length L into B blocks of length L', writing x^b for block b and $x^{<b}$ for every block before it. The chain rule over blocks is exact:

$$\log p_\theta(x) = \sum_{b=1}^{B} \log p_\theta(x^b \mid x^{<b})$$

"The log-likelihood $\log p_\theta(x)$ of a sequence equals the sum over blocks b from 1 to B of $\log p_\theta(x^b \mid x^{<b})$, the log probability of block b given every earlier block."

Each factor is a distribution over one block of L' tokens, conditioned on a clean prefix, which is exactly the kind of distribution masked diffusion models. Applying the rate loss to each factor and adding,

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i(x^i \mid Z_t^b,\, x^{<b}) \Big] dt$$

"The negative log-likelihood of sequence x is bounded by the sum of block losses. Each block loss integrates the reveal-rate weight $-\dot{\alpha}_t/(1-\alpha_t)$ times the expected negative log probabilities of clean tokens x^i at masked positions i in block b. The denoiser conditions on the noisy block Z_t^b and the clean prefix $x^{<b}$."

The rate matrix acts on the current block, with the prefix held fixed. Masking on the L' positions gives the forward rates, closed-form marginal and all-mask endpoint. We compute the reveal rates with the clean prefix included in the denoiser input, then sum the MDLM bounds across blocks.

Generate the first block from L' masks, revealing positions in any order (Figure 4.11). Once complete, append it to the clean prefix and generate the next block in the same way. Caching the prefix keys and values keeps the conditioning cost independent of the number of diffusion steps within a block.

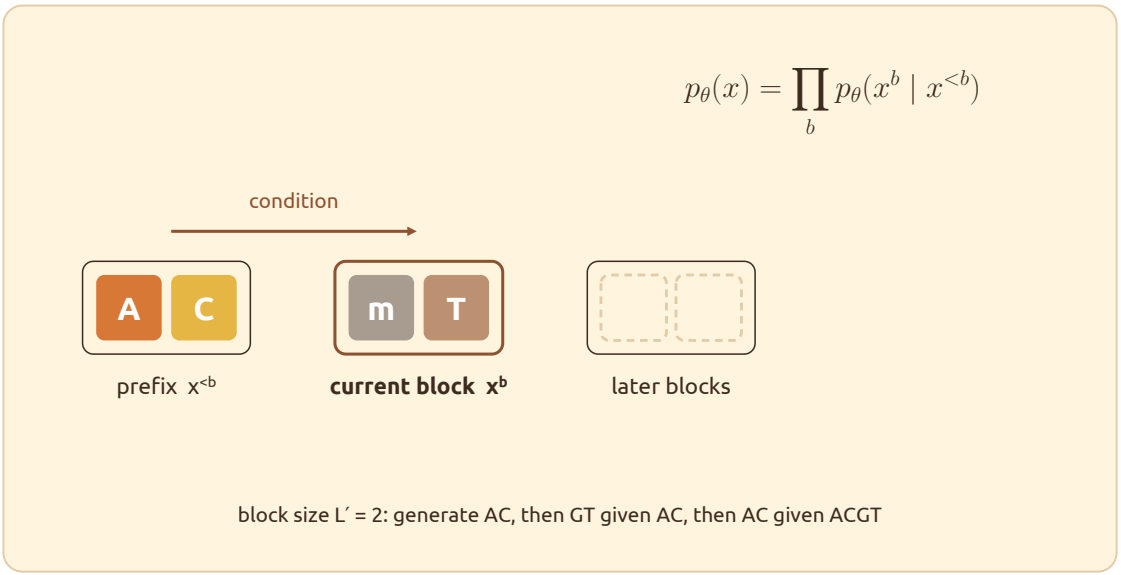


Figure 4.11: Generating ACGTAC in blocks of $L' = 2$. The prefix $x^{<b} = \text{AC}$ is already clean and conditions everything that follows; the current block x^b still holds one mask and one revealed T, and is filled by masked diffusion in any order; the later blocks are untouched. Each completed block joins the prefix for the next one, and the product over blocks is the chain-rule factorization printed above.

4.34.1 Block size and autoregression

With $L' = L$ there is one block, the prefix is empty, and the bound is the MDLM loss. With $L' = 1$ each block holds a single token, and the bound becomes the exact autoregressive log-likelihood.

A one-position block contributes loss only when masked. In that case, its input consists of one mask token and the clean prefix, independent of t. Write its prediction as $d_\theta(x^b \mid x^{<b})$. The masking probability is $1 - \alpha_t$, giving the time integral for block b:

$$\int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\,(1-\alpha_t)\,\big(-\log d_\theta(x^b \mid x^{<b})\big)\,dt = \big(-\log d_\theta(x^b \mid x^{<b})\big)\int_0^1 (-\dot{\alpha}_t)\,dt,$$

and the remaining integral is $\alpha_0 - \alpha_1 = 1 - 0 = 1$. Each block adds exactly $-\log d_\theta(x^b \mid x^{<b})$, and the sum over blocks is the autoregressive negative log-likelihood with no slack. The two extremes of the block size are the two models, and every intermediate value interpolates between them.

Let's work one draw of the bound. Take ACGTAC with $L' = 2$, the linear schedule, and the time $t = 0.5$ drawn for all three blocks, so $w(t) = 2$. Suppose block 1 is corrupted to (m, m) with denoiser probabilities 0.5 on A and 0.8 on C; block 2 is corrupted to (m, T) with probability 0.7 on G; and block 3 is corrupted to (A, m) with probability 0.9 on C. The three contributions are

$$2(0.6931 + 0.2231) = 1.8326, \qquad 2(0.3567) = 0.7134, \qquad 2(0.1054) = 0.2107,$$

and the bound at this draw is 2.7567 nats for the whole sequence. Most of that comes from block 1, because both of its positions were masked and the denoiser had no left context at all.

On LM1B, an autoregressive transformer reached a test perplexity of 22.83, the MDLM bound reached 31.78 at the same budget, and blocks of four tokens reached 28.23, between the two [Arriola et al., 2025]. Smaller blocks move the bound toward autoregression, which the $L' = 1$ calculation above predicts exactly.

We can also assign different masking or replacement rates to different tokens.

4.35 Per-token schedules

The schedule $\beta(t)$ has been the same for A, C, G and T, and for every word in a vocabulary of thirty thousand. Common tokens and rare tokens are therefore erased at the same speed, and the same denoiser budget goes to recovering both. In MD4 and its generalization GenMD4 [Shi et al., 2024], each token has its own rate:

$$R_t(a, m) = \beta_a(t)$$

"The masking rate $R_t(a, m)$ from clean token a to mask m equals the token-specific rate schedule $\beta_a(t)$, allowing different tokens to be erased at different speeds."

Each row still has one nonnegative off-diagonal entry and sums to zero, so the matrix remains a valid rate matrix. The rate matrix is diagonal in the clean tokens, so the forward marginal factorizes over them and the matrix exponential is taken token by token:

$$\alpha_t(a) = \exp\Big(-\int_0^t \beta_a(s)\,ds\Big), \qquad q_t(z^i \mid x_0^i) = \alpha_t(x_0^i)\,\delta(z^i, x_0^i) + \big(1-\alpha_t(x_0^i)\big)\,\delta(z^i, m),$$

which is a closed-form marginal with a token-dependent survival probability. Every $\alpha_t(a)$ falls to zero, so the process still ends at the all-mask sequence.

The reverse rate acquires a token-dependent weight. If we repeat the masking derivation with β_a in place of β, the conditional reverse rate at a masked position is

$$\hat{R}_t(m, a \mid x_0) = \beta_a(t)\,\frac{\alpha_t(a)\,\delta(a, x_0)}{1-\alpha_t(x_0)} = \frac{-\dot{\alpha}_t(a)}{1-\alpha_t(a)}\,\delta(a, x_0) = w_a(t)\,\delta(a, x_0),$$

so the reveal rate $w_a(t) = -\dot{\alpha}_t(a)/(1-\alpha_t(a))$ now depends on which token is being revealed. The model rate is $\hat{R}_t^\theta(m, a) = w_a(t)\,d_\theta^i(a \mid z, t)$, and the training objective is the MDLM loss with the weight inside the sum over tokens.

We now take two bases with constant rates $\beta_{\mathrm{A}} = 1$ and $\beta_{\mathrm{G}} = 2$, so $\alpha_t(\mathrm{A}) = e^{-t}$ and $\alpha_t(\mathrm{G}) = e^{-2t}$. At $t = 0.5$ a clean A survives with probability 0.6065 and a clean G with probability 0.3679, so G has usually already been erased when A has not. The reveal rates there are

$$w_{\mathrm{A}}(0.5) = \frac{0.6065}{1-0.6065} = 1.5415, \qquad w_{\mathrm{G}}(0.5) = \frac{2(0.3679)}{1-0.3679} = 1.1640.$$

A masked position is revealed as A at $1.5415\,d_\theta^i(\mathrm{A})$ and as G at $1.1640\,d_\theta^i(\mathrm{G})$, so the total reveal rate is no longer independent of the denoiser. A token that survives longer in the forward process is revealed faster in reverse, because less time remains in which it can appear.

In GenMD4, we optimize the schedules β_a as model parameters using the same bound. Any rate matrix satisfying facts 1 to 3 may be optimized inside the bound, and the schedule is part of the rate matrix. On Text8, absorbing D3PM reached a bound of 1.45 bits per character, MD4 reached 1.37 and GenMD4 with learned per-token rates reached 1.34, all with the same transformer [Shi et al., 2024]. The whole of that gap comes from the rate matrix, with the network held fixed.

After drawing a token, we discard the rest of the denoiser distribution. We can retain this information by expanding the state space.

4.36 Simplex diffusion

At each reverse step, we predict a distribution over clean tokens and sample a token from it. The stored state contains that token, so it does not record whether the prediction was concentrated or uncertain. This loss of uncertainty information is called information collapse.

Simplex Diffusion Models [Deschenaux et al., 2026] enlarge the state at each position from a token to a probability vector $P_t \in \Delta^{K-1}$, a belief over the K clean tokens. The sampler holds the belief from one step to the next, so the uncertainty is not lost.

Token-valued diffusion visits only the corners of the simplex and moves along its edges. A belief moves through the interior and reaches a corner only at the end of generation (Figure 4.12). The corners are the one-hot vectors $\mathbf{e}_a$, so the earlier models are the restriction of this one to the vertices.

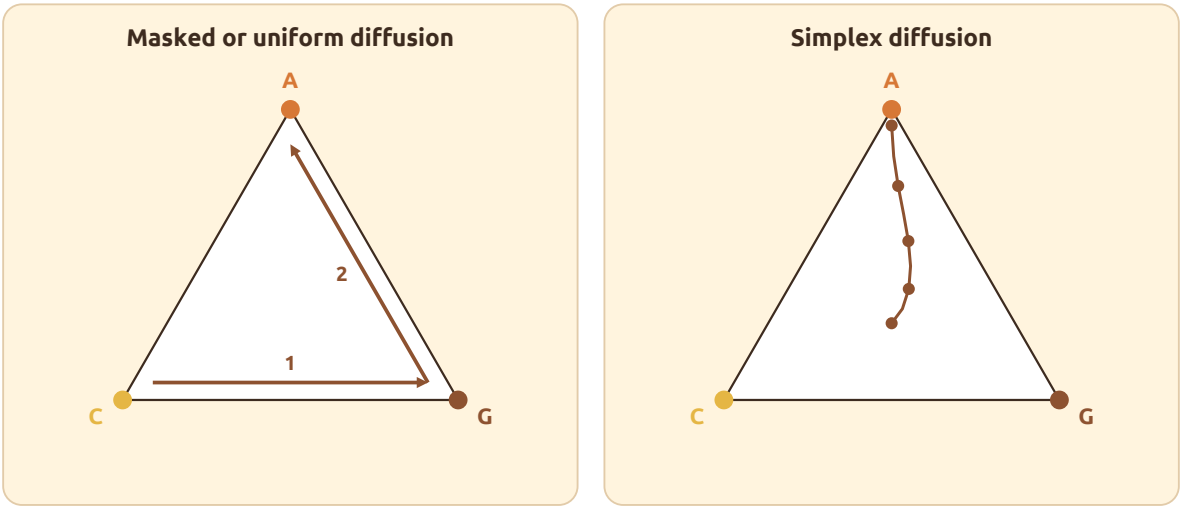


Figure 4.12: The state space of one position over three tokens A, C, G, drawn as the probability simplex. On the left, masked and uniform diffusion occupy only the corners, which are the one-hot vectors, and each reverse step is a jump along an edge, here C to G and then G to A. On the right, simplex diffusion moves through the interior, and the marked points are the belief at successive reverse steps, converging on the corner A.

To define a closed-form forward marginal, we center the noisy belief on the token-model marginal:

$$\mathbb{E}[P_t \mid x_0] = \alpha_t \, \mathbf{e}_{x_0} + (1 - \alpha_t) \, \pi$$

"The expected noisy belief P_t at a position, given that the clean token there was x_0, equals the survival probability α_t times the one-hot vector $\mathbf{e}_{x_0}$ of the clean token, plus one minus α_t times the target distribution π."

The mean of the belief is exactly the uniform-replacement kernel of Section 4.32. The beliefs themselves follow a Dirichlet distribution around that mean, with a concentration c_t that controls how far they spread, so

$$P_t \mid x_0 \;\sim\; \mathrm{Dir}(c_t \, \mathbb{E}[P_t \mid x_0]).$$

As $c_t \to 0$ a Dirichlet concentrates on the corners, with the probability of each corner equal to its share of the mean, so the belief process degenerates to uniform diffusion in that limit. The process is Markov in t, and its final beliefs are distributed around π.

4.36.1 Thinning, an innovation, and a mixture

For the reverse step, fix the clean token x_0 and choose times $s < t$, so that $\alpha_s > \alpha_t$ and the belief at s is sharper. A Dirichlet with parameter vector $\mathbf{a} + \mathbf{b}$ has a mixture representation: for independent $V \sim \mathrm{Dir}(\mathbf{a})$, $U \sim \mathrm{Dir}(\mathbf{b})$ and $W \sim \mathrm{Beta}(\sum_k a_k, \sum_k b_k)$,

$$W\,V + (1 - W)\,U \;\sim\; \mathrm{Dir}(\mathbf{a} + \mathbf{b}).$$

If we apply this with $\mathbf{a} = c_t \mathbb{E}[P_t \mid x_0]$ and $\mathbf{b} = c_s \mathbb{E}[P_s \mid x_0] - c_t \mathbb{E}[P_t \mid x_0]$, the reverse step becomes three operations: thin the current belief by the factor W, draw an innovation U from the Dirichlet with parameter $\mathbf{b}$, and mix the two. The step is valid whenever $\mathbf{b}$ is nonnegative in every coordinate, which fixes a compatibility condition between the concentration schedule c_t and the survival schedule α_t.

We can satisfy this condition by choosing $c_t = \gamma K/(1 - \alpha_t)$ for a constant $\gamma > 0$. Every coordinate other than x_0 then has parameter $c_t(1 - \alpha_t)/K = \gamma$, constant in time, and the x_0 coordinate has parameter

$$c_t\Big(\alpha_t + \frac{1 - \alpha_t}{K}\Big) = \gamma\Big(1 + \frac{K\alpha_t}{1 - \alpha_t}\Big),$$

in which the factor $1 + K\alpha_t/(1-\alpha_t)$ is the one that governed the uniform reverse rates in Section 4.32. The increment $\mathbf{b}$ then has all of its mass on the x_0 coordinate, so the innovation U is the one-hot vector $\mathbf{e}_{x_0}$ and the reverse step is a mixture of the current belief with the clean token.

Take $K = 4$, $\gamma = 1$, uniform π and clean base A. At $\alpha_t = 0.5$, the concentration is $c_t = 8$, the mean is $(0.625, 0.125, 0.125, 0.125)$, and the Dirichlet parameters are $(5, 1, 1, 1)$. The A coordinate has mean 0.625 and standard deviation 0.161, so a typical draw assigns between one half and four fifths of its mass to A. At $\alpha_s = 0.8$, the concentration is $c_s = 20$, the mean is $(0.85, 0.05, 0.05, 0.05)$, and the parameters are $(17, 1, 1, 1)$. Subtract the parameter vectors to obtain the increment $(12, 0, 0, 0)$, giving

$$P_s = W\,P_t + (1 - W)\,\mathbf{e}_{\mathrm{A}}, \qquad W \sim \mathrm{Beta}(8,\, 12).$$

The thinning weight has mean $8/20 = 0.4$, and the step moves the belief four tenths of the way from where it is toward the clean token on average: $0.4(0.625, 0.125, 0.125, 0.125) + 0.6(1, 0, 0, 0) = (0.85, 0.05, 0.05, 0.05)$, which is the mean at α_s. If we simulate the mixture four hundred thousand times, we reproduce the standard deviations of $\mathrm{Dir}(17, 1, 1, 1)$ to three decimals, so the representation holds numerically as well as algebraically.

During generation the clean token is unknown, and the model averages the step over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

"The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to the earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ that the position was x_0, times the conditional reverse step $q(P_s \mid P_t, x_0)$ available in closed form once x_0 is fixed."

In practice the sum is realized by drawing $\tilde{X}_0 \sim d_\theta(\cdot \mid P_t, t)$ and using $\mathbf{e}_{\tilde{X}_0}$ as the innovation, so one step costs one forward pass, one Beta draw and one categorical draw. The only architectural change is at the input, where the denoiser reads a belief of K numbers in place of a single token.

We train with cross-entropy, whose optimum is the clean-token posterior:

$$\mathcal{L}(\theta) = \mathbb{E}[-\log d_\theta(x_0 \mid P_t, t)]$$

"The training loss $\mathcal{L}(\theta)$ averages the negative log probability $-\log d_\theta(x_0 \mid P_t, t)$ assigned to the true clean token x_0, given its noisy probability vector P_t and noise time t."

The forward marginal, the posterior average and the cross-entropy are the same three pieces as in masked diffusion, with a belief in place of a token at each position. On TinyGSM at 512 function evaluations, simplex diffusion solved 49.0% of the problems against 45.8% for masked diffusion at the same budget [Deschenaux et al., 2026]. The enlarged state preserves uncertainty between successive steps.

4.37 DNA diffusion implementation

To implement masked diffusion in PyTorch, encode A, C, G and T as tokens 0 through 3, and the mask m as token 4. Use the linear schedule $\alpha_t = 1 - t$, with clean data at $t = 0$. The network `model` takes a noisy sequence and a time as inputs and returns five logits per position.

```
MASK = 4                              # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                   # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)
```

The forward marginal $q_t(z^i \mid x_0^i) = \alpha_t \delta(z^i, x_0^i) + (1 - \alpha_t)\delta(z^i, m)$ lets us corrupt each position directly at a chosen noise level. Keep the original token with probability α_t and replace it with the mask otherwise, independently across positions. The tensor `keep` contains these Bernoulli draws, and `torch.where` selects the original token or mask at each position. Broadcasting `t` allows each sequence in a batch to use its own noise level without simulating intermediate states.

```
def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)             # [B, L, 5]
    logits[..., MASK] = -1e9          # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                       # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()
```

We estimate the masked diffusion loss from Section 4.30 using one sampled time and corruption per sequence. Draw t uniformly on $(0, 1)$ to estimate the time integral, then corrupt each sequence at its sampled time. Reshaping with `t[:, None]` broadcasts the time across that sequence's positions. We pass the noisy sequence and time to the denoiser to obtain five logits per position for the categorical prediction $d^i_\theta(\cdot \mid z, t)$.

Set the mask logit to -10^9 to implement the zero-mask rule. Since clean sequences contain no m, the clean-token posterior assigns it zero probability; the softmax approximation assigns about e^{-10^9}. Compute the cross-entropy against `x0` at each position to obtain $-\log d^i_\theta(x^i_0 \mid z, t)$. For the weight `w = 1.0 / t`, substitute $\alpha_t = 1 - t$ and $\dot\alpha_t = -1$ into the reveal rate,

$$w(t) = \frac{-\dot\alpha_t}{1 - \alpha_t} = \frac{1}{1 - (1 - t)} = \frac{1}{t}.$$

Finally, multiply the cross-entropy by the weight, select masked positions with `(zt == MASK)`, sum over positions and average over the batch. Visible positions contribute zero because their target and model rates are zero. As $t \to 0$, the weight grows without bound while almost every position is visible, leaving the few masked positions to contribute to the integrand.

```
@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Start generation from the all-mask state, the stationary distribution for masking. Initialize a tensor of `MASK` tokens with shape $B \times L$ for B sequences of length L. The time grid `ts` runs backward from $t = 1$ to $t = 10^{-3}$, and each loop iteration processes an interval $[s, t]$.

Inside the loop, evaluate the denoiser, remove the mask logit and sample `x0` from each position's clean-token distribution. These draws implement the posterior average in the reverse step: the reveal rate for token a is $w(t)\, d^i_\theta(a \mid z, t)$, so conditioned on a reveal happening the token drawn is distributed as d^i_θ. Each masked position is then revealed with probability $(t - s)/t$, which is the reveal rate integrated over the step:

$$\frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{(1 - s) - (1 - t)}{t} = \frac{t - s}{t}.$$

With 32 uniform steps the first interval runs from $t = 1$ to $s = 0.9688$, and the reveal probability there is 0.0312, so about 3% of the positions are filled on the first pass. The last interval, from $t = 0.0322$ to $s = 0.001$, reveals with probability 0.969, so whatever remains is almost certainly finished. The condition `(z == MASK) & unmask` is carry-over unmasking, derived in Section 4.30

from the forward rates into clean tokens all being zero. A revealed base is never touched again, so a step only sets which masks open and what fills them.

We can adapt the implementation to each model. For uniform noise, replace tokens with draws from π, keep the mask logit and use all $K-1$ reverse rates from Section 4.32. For SEDD, output $L(K-1)$ positive ratios and train with score entropy. For block diffusion, loop over blocks, include the clean prefix in the denoiser input and sum the block losses. For per-token schedules, replace the scalar `w` with a vector indexed by clean token. For simplex diffusion, store `z` as beliefs with shape $B \times L \times K$ and replace the reveal draw with a Beta draw and mixture.

4.38 Reveal-order planning

The standard sampler reveals every masked position with probability $(t-s)/t$, regardless of whether its denoiser prediction is concentrated or nearly uniform. Once revealed, a token remains fixed. In P2 [Peng et al., 2026a], we choose reveal positions using confidence and allow revision through remasking.

The reverse rate out of a masked position is $w(t)\, d_\theta^i(a \mid z, t)$ at every masked position, so the total rate is the same everywhere and the choice of which positions to reveal within a step is free. If we reveal the positions the denoiser is most confident about, the marginal reverse rates are untouched and only the order in which the sequence is filled changes.

To revise tokens, we use the corrector $C_t = R_t + \hat{R}_t$ from Section 4.27.2. It satisfies detailed balance with p_t, so we can use it to reduce sampling errors while preserving the correct marginal. Under masking, the forward part returns a visible token to the mask at rate $\beta(t)$, and the reverse part reveals a masked position at rate $w(t)d_\theta^i$. Masking a token again and revealing it is one corrector step.

The flux balance is exact for any schedule. A fraction α_t of positions is visible and moves out at rate $\beta(t) = -\dot{\alpha}_t/\alpha_t$, and a fraction $1-\alpha_t$ is masked and moves out at rate $w(t) = -\dot{\alpha}_t/(1-\alpha_t)$, so the two fluxes are

$$\alpha_t\, \beta(t) = -\dot{\alpha}_t, \qquad (1-\alpha_t)\, w(t) = -\dot{\alpha}_t,$$

and they are equal at every t. With the linear schedule at $t = 0.5$, both β and w equal 2, and the flux each way is 1. At $t = 0.25$ the rates are $\beta = 4/3$ and $w = 4$, and the flux each way is still 1, because the three quarters of positions that are visible are remasked slowly and the one quarter that is masked is revealed quickly. The masked fraction does not drift.

4.38.1 One sampling step with a planner

Both changes fit inside one reverse step, replacing the three lines of the sampler of Section 4.37 that drew a clean token and opened a random set of masks.

```
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()
```

```
    # planner score: how sure the denoiser is here
    score = p.max(-1).values                    # or -entropy, or a
                                                # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)                # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)              # remask the worst
    return z
```

The first four lines evaluate the denoiser, apply the zero-mask rule, compute a softmax and draw a token at each position. These operations match the sampler in Section 4.37, so we can apply them to any trained MDLM checkpoint without retraining.

We summarize each position's prediction with a planner score. The value `p.max(-1).values` is the largest clean-token probability, so concentrated predictions score higher. We can also use negative entropy of $d_\theta^i(\cdot \mid z, t)$, or train a separate network to score which position to reveal next.

Use the first `topk_where` call to rank masked positions by score and reveal the highest-scoring `n_unmask` positions. Write their sampled tokens into the sequence. Use the second call to rank visible positions by negative score and remask the lowest-scoring `n_remask` positions. We choose both counts for each step. Setting `n_remask = 0` recovers confidence-ordered sampling without the corrector.

Let's take a six-position sequence. Positions 1, 3, 4 and 6 are masked, with planner scores 0.42, 0.91, 0.55 and 0.78, and positions 2 and 5 are visible, with scores 0.88 and 0.31. With `n_unmask = 2`, the step reveals positions 3 and 6, whose scores are 0.91 and 0.78, leaving positions 1 and 4 masked for a later step when more context is available. With `n_remask = 1`, position 5 goes back to the mask, because a score of 0.31 is weak support for the token committed there. The sequence after the step holds four visible positions and two masked ones, and the next forward pass has the two newly revealed bases as context for the two that were deferred.

On a protein benchmark, a masked diffusion model folded 48.1% of its samples, and sampling the same checkpoint with a planner raised that to 58.9%, with mean pLDDT rising from 80.2 to 83.5 [Peng et al., 2026a]. The weights are identical in the two rows, and only the order of the reveals and the corrector steps differ.

The planner changes the distribution of states visited during sampling. We can adjust the training weights to account for this change.

4.39 Planner-weighted training

The MDLM loss weights masked positions equally, matching a uniformly random reveal order. At time t, each masked position is then equally likely to be filled next. A planner changes the distribution of masking patterns visited during sampling. Errors at the positions selected next therefore matter most. In PAPL [Peng et al., 2026b], we incorporate these priorities by inserting the planner weights into the rate loss:

$$\mathcal{L}_{\text{PAPL}} = \int_0^1 w(t)\, \mathbb{E}\Big[\sum_{i:\, Z_t^i = m} \lambda_i(Z_t)\, \big(-\log d_\theta^i(x_0^i \mid Z_t, t) \big) \Big] dt$$

"The PAPL loss $\mathcal{L}_{\text{PAPL}}$ integrates the reveal rate $w(t)$ times the expected sum of planner-weighted prediction errors. At each masked position i in the noisy sequence Z_t, multiply the planner weight $\lambda_i(Z_t)$ by the negative log probability $-\log d_\theta^i(x_0^i \mid Z_t, t)$ of the clean token x_0^i."

- $\lambda_i(Z_t) \geq 0$ is the planner weight at position i, a function of the whole noisy sequence Z_t and not of the clean data.
- $w(t) = -\dot{\alpha}_t/(1-\alpha_t)$ is the reveal rate, unchanged.

Setting $\lambda_i \equiv 1$ recovers the MDLM loss. The weights come from the planner's own distribution over which masked position to reveal next. If that distribution is $\rho_i(Z_t)$ over the M currently masked positions, the natural choice is the ratio to the uniform order,

$$\lambda_i(Z_t) = \frac{\rho_i(Z_t)}{1/M} = M\, \rho_i(Z_t),$$

which averages to one over the masked positions, so the total weight of a state is unchanged. The forward rates, marginals and reverse rates are the same as in masked diffusion. The reweighted bound is still minimized at the true reverse rates because each term is minimized there and the weights are nonnegative.

We take a state with four masked positions, a planner distribution $(0.10, 0.50, 0.15, 0.25)$ over them, and per-position cross-entropies $(1.2, 0.3, 0.9, 0.5)$ nats. The uniform baseline is $1/4$, so the weights are

$$\lambda = 4 \times (0.10,\ 0.50,\ 0.15,\ 0.25) = (0.4,\ 2.0,\ 0.6,\ 1.0),$$

which sum to 4 as required. The MDLM inner sum is $1.2 + 0.3 + 0.9 + 0.5 = 2.9$ nats, and the PAPL inner sum is

$$0.4(1.2) + 2.0(0.3) + 0.6(0.9) + 1.0(0.5) = 0.48 + 0.60 + 0.54 + 0.50 = 2.12$$

nats. The second position, where the planner is five times more likely to act than chance, has 0.60 of the total in place of 0.30. The first position, which the planner will probably defer, is discounted from 1.2 to 0.48. The gradient goes to the positions the sampler will reach first.

On the same protein benchmark, a 150M-parameter diffusion model folded 42.4% of its samples, and planner-aware training raised that to 59.4%, with mean predicted alignment error falling from 12.00 to 8.97 [Peng et al., 2026b]. The rates and the bound are the ones derived in this chapter, with a different weighting inside the expectation.

Reweighting the loss changes which sequences the model produces on average. To steer it toward sequences with a measured property, we need the h-transform.

4.40 Guiding generation by search

For a therapeutic peptide, we may require target binding, solubility and membrane permeability. Only a small subset of sequences satisfy all three. We can favor these sequences with the h-transform from Section 4.25, reweighting each rate using $h_t(x) = p(c \mid X_t = x)$. This value is the probability of obtaining property c at the end of generation, given state x at time t.

Guided sampling requires an estimate of this value. In Section 4.25.3, we trained a classifier at every noise level. External scorers, such as docking scores or chemical filters, may only accept completed sequences. In PepTune [Tang et al., 2025a], we estimate the value by completing the sequence and scoring the result.

In PepTune, we guide a masked diffusion model over peptide SMILES strings. Each token represents a chemical fragment. A partially unmasked state therefore specifies part of a molecule. From such a state z at time t, we run the sampler to completion N times and score each completed sequence with the reward r. The average is an estimate of the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

"The value $h_t(z)$ at the partially unmasked state z is approximately the average over N rollouts of the reward $r(x_0^{(n)})$ of the completed sequence, where each $x_0^{(n)}$ is drawn by running the model's own reverse process $p_\theta(\cdot \mid z)$ from z to the end of generation."

The estimate is unbiased by construction, because the reverse process started at z produces exactly the distribution over endings that $h_t(z)$ averages over. Its variance falls like $1/N$, so a handful of rollouts is enough to order the children of a state even when each individual estimate is rough.

With the value in hand, we evaluate the guided rate of Section 4.25.3 on that estimate, one reverse jump at a time:

$$\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \frac{h_t(z')}{h_t(z)}$$

"The guided reverse rate $\hat{R}_t^c(z, z')$ equals the model reverse rate $\hat{R}_t(z, z')$ multiplied by the ratio of estimated completion values $h_t(z')/h_t(z)$ at the destination z' and current state z."

Consider a state z and a child z' obtained by revealing one fragment. Four rollouts from z have rewards 0.1, 0.8, 0.3 and 0.2, giving $h_t(z) \approx 0.35$. Four rollouts from z' have rewards 0.7, 0.9, 0.6 and 0.8, giving $h_t(z') \approx 0.75$. The ratio is $0.75/0.35 = 2.143$, so a model rate of 1.2 becomes a guided rate of 2.571. This makes revealing the fragment more than twice as likely. A sibling with mean rollout reward 0.2 receives the factor $0.2/0.35 = 0.571$, reducing its rate.

4.40.1 Search over the tree of partial sequences

We organize partially unmasked sequences in a tree, with each edge corresponding to one reveal. To extend the search, select a path from the root to a leaf, add a child and score a completed rollout from it. Use the reward to update the value estimate at every node on the selected path. The rollout count N at each node is therefore its visit count across the search.

To balance mean reward with exploration, select the child maximizing the upper-confidence score $\bar{r}_j + c_{\text{uct}}\sqrt{\log n / n_j}$, where $\bar{r}_j$ is the mean reward at child j, n_j its visit count and n the parent's. The exploration bonus is large for a child visited only once and decreases with repeated visits. To expand a node, perturb the denoiser predictions with Gumbel noise. Taking the maximum samples from $d^i_\theta(\cdot \mid z, t)$; taking several sorted outcomes produces distinct children.

Combining binding affinity, solubility and permeability into one reward requires choosing their relative weights in advance. In PepTune, we retain nondominated sequences in a Pareto archive. Consider binding and solubility scores $(0.8, 0.3)$ for P_1, $(0.5, 0.7)$ for P_2 and $(0.4, 0.2)$ for P_3. Discard P_3 because P_1 scores higher on both properties. Keep P_1 and P_2 because each scores higher on one property. Both remain in the archive.

The peptide model generated valid SMILES 45% of the time on its own, and searching under the h-transform reached 100% validity at the same diversity [Tang et al., 2025a]. The model weights are unchanged throughout, and only its reverse rates are reweighted.

When generating another peptide, we start a new search from the all-mask state.

4.41 Distilling search results

In TR2-D2 [Tang et al., 2025c], we retain the search trajectories and fine-tune the model on them. We incorporate the reweighting into the model parameters, so subsequent sampling costs the same as sampling from the base model.

The target is the reward-tilted distribution:

$$p^*(x) \propto p_\theta(x)\, \exp\left(r(x)/\lambda\right)$$

"The target distribution $p^*(x)$ is proportional to the model probability $p_\theta(x)$ of the sequence x times the exponential of its reward $r(x)$ divided by the temperature λ."

- $r(x)$ is the reward of a completed sequence, in whatever units the scorer produces.
- $\lambda > 0$ is a temperature. With large λ the model is nearly unchanged, and small λ concentrates p^* on the highest-reward sequences.

This is the h-transform once more, with $h(x) = \exp(r(x)/\lambda)$ evaluated at the end of generation. Fine-tuning toward p^* therefore means learning the reverse rates of the h-transformed process, which the rate loss already covers once the samples are weighted correctly.

Two rollouts can reach the same sequence through paths with different probabilities under the model and the search process. To train toward the tilted model distribution, we weight each path using the reward and the ratio of model to search rates:

$$\log W = \frac{r(x_0)}{\lambda} + \sum_{\text{jumps}} \log \frac{\hat{R}_t^\theta(z, z')}{\hat{R}_t^{\text{search}}(z, z')}$$

"The log weight $\log W$ of a path equals the reward $r(x_0)$ of its final sequence divided by the temperature λ, plus the sum over the jumps of the path from z to z' of the log of the model rate $\hat{R}_t^\theta(z, z')$ divided by the search rate $\hat{R}_t^{\text{search}}(z, z')$ under which that jump was actually drawn."

The second term corrects for sampling with the search rates when the target uses model probabilities. A jump with a higher search rate than model rate contributes a negative log ratio and reduces the weight. A higher model rate contributes a positive log ratio.

Take two paths ending at the same sequence with reward $r = 2.0$ and temperature $\lambda = 1$. Path 1 has model-to-search rate ratios 1.2, 0.9 and 1.1. Their logarithms, 0.1823, -0.1054 and 0.0953, sum to 0.1723, giving $\log W_1 = 2.1723$. Path 2 has ratios 0.5, 0.6 and 0.7, whose logarithms sum to -1.5606, giving $\log W_2 = 0.4394$. Exponentiating gives unnormalized weights 8.778 and 1.552. Normalizing assigns 0.850 of the training weight to path 1 and 0.150 to path 2. Path 1 receives almost six times the weight because its model-to-search rate ratios are larger.

Training on these weighted trajectories is a trajectory-aware version of the rate loss, the same $\ell(A, B)$ summed along each path, with the path weight multiplying its contribution. On regulatory DNA, the pre-trained model produced enhancers with median predicted activity 0.17, and the fine-tuned model reached 9.74 while keeping 99.9% of samples in accessible chromatin [Tang et al., 2025c]. After fine-tuning we sample with no rollouts at all.

We can also allow the sequence length to change during generation.

4.42 Variable-length diffusion

The state space $\mathcal{S} = \mathcal{V}^L$ requires a fixed length, but generated peptides and sentences can have different lengths. Padding to a maximum length requires modeling positions that we eventually discard. A2D2 [Tang et al., 2026c] allows the length to change by adding insertion jumps alongside unmasking.

We include both jump types in one rate matrix over sequences of any length. An unmasking jump replaces m with a clean token at an existing position. An insertion jump adds a masked position at a gap. A length-ℓ sequence has $\ell + 1$ gaps, so we index insertion rates by gap and unmasking rates by position. The master equation, reversal and rate loss apply because their derivations require only a countable state space. During generation, alternate an insertion policy that selects how many positions to add at each gap with an unmasking policy that fills them. A quality head scores the reliability of each decision.

The total rate of leaving a state now depends on its length, so two paths that make the same jumps can still differ in how long they waited between them. The path KL divergence derived earlier has

exactly the two terms we need:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^\theta}{\hat{R}} \; - \; \int_0^1 (\hat{R}^\theta_{\text{out}} - \hat{R}_{\text{out}})\, dt$$

"The log weight $\log W$ of a path equals the sum over its jumps of the log of the model rate $\hat{R}^\theta$ divided by the reference rate $\hat{R}$, minus the integral over time of the difference between $\hat{R}^\theta_{\text{out}}$, the model's total rate of leaving the current state, and $\hat{R}_{\text{out}}$, the reference total rate."

The first term depends on the observed jumps, and the second accounts for the waiting intervals between them. The second is the $-A + B$ part of $\ell(A, B) = A \log(A/B) - A + B$, the contribution from staying in the current state. With a fixed length and a fixed number of reveals, the total out-rate is the same for every path and the term cancels. With insertions it does not cancel, because a path that inserted early has more positions, and therefore more out-rate, for the rest of its run.

We take a state of length 8 with 3 masked positions and 9 gaps, an unmasking rate of $w(t) = 2$ per masked position and an insertion rate of $\nu(t) = 0.5$ per gap. Its total out-rate is

$$3(2) + 9(0.5) = 6 + 4.5 = 10.5.$$

One insertion raises the length to 9, the masked count to 4 and the gap count to 10, so the out-rate becomes $4(2) + 10(0.5) = 13$. Delaying the insertion leaves the out-rate at 10.5 over the intervening interval. The time integral accounts for this difference from the out-rate 13 after insertion. For a concrete weight, suppose a path makes two jumps with model-to-reference rate ratios 1.5 and 0.8, and its model out-rate averages 3.2 over $[0, 1]$ against 3.0 for the reference. Then

$$\log W = \log 1.5 + \log 0.8 - (3.2 - 3.0) = 0.4055 - 0.2231 - 0.2 = -0.0177,$$

so $W = 0.982$. The jump terms nearly cancel. The higher model out-rate contributes -0.2 through the waiting term, making the total log weight negative.

On small molecules, the pre-trained any-length model reached 44.2% quality, and fine-tuning on reweighted search paths raised that to 76.1%, with mean QED rising from 0.641 to 0.795 [Tang et al., 2026c]. The length and the tokens come out of the same rate matrix, so one bound covers both.

4.43 Model comparison

Table 4.2 compares the models by their forward process, marginal, endpoint, reverse step and loss. Choosing the forward process in the first row determines the remaining entries in each column.

Each column specifies a model, and each row compares the consequences of choosing π or the state space. The three π-family members have closed-form marginals, allowing training with one forward pass at a sampled noise level. Substituting the respective rates into the loss gives the weighted cross-entropy for MDLM, the block sum and the simplex cross-entropy.

Table 4.2: The five facts of the construction, answered by five of the models of this chapter. Rows are the five facts in the order they are checked; columns are the models. MDLM, D3PM and Uniform differ only in π and in whether time is discrete. Block diffusion repeats the MDLM column inside a block of length L' with a clean prefix as context. Simplex diffusion replaces the token at each position by a probability vector whose mean follows the uniform-replacement marginal.

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1-\beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs
Closed-form marginal	$\alpha_t \delta_{x_0} + (1-\alpha_t)\delta_m$	$x_0 \bar{Q}_t$	$\alpha_t \delta_{x_0} + (1-\alpha_t)/K$	MDLM marginal per block	mean $\alpha_t \mathbf{e}_{x_0} + (1-\alpha_t)\pi$
Where it ends	all masks	π	uniform tokens	all-mask block	beliefs around π
Reverse via posterior	reveal at rate $w(t)d_\theta$	Bayes step averaged over $\tilde{p}_\theta$	ratios averaged over d_θ	MDLM rates given the prefix	kernel averaged over d_θ
Loss	weighted cross-entropy	$\mathcal{L}_{\text{vb}} + \lambda_{\text{aux}}\,\text{CE}$	full rate loss	sum of block losses	cross-entropy

SEDD uses the MDLM or uniform forward process and predicts the reverse probability ratios directly. MD4 uses the MDLM construction with a separate survival schedule $\alpha_t(a)$ for each token.

We can modify sampling and training within these constructions. P2 changes reveal order, and PAPL reweights the loss. PepTune and TR2-D2 use an h-transform to change the sampled distribution. A2D2 expands the state space while retaining the reversal and rate loss. These changes adjust the sampler or objective associated with the models in the table.

4.44 Discrete diffusion construction

1. Choose a rate matrix that encodes which replacements make sense for the data. For DNA this might join bases that substitute for one another in a codon; for molecules, fragments with similar valence; for text, nearest neighbours in an embedding, as in Section 4.31. The only requirement is fact 1: nonnegative off the diagonal, rows summing to zero.

2. Compute its matrix exponential to obtain the forward marginal in closed form. A rate matrix built from a projection, a graph Laplacian or a circulant structure usually has one, and without one every training step must simulate a trajectory.

3. Find where the process settles, and check that the limit can be sampled directly. Generation starts from that distribution, so a stationary distribution we cannot draw from is of no use.

4. Write the reverse rates through the posterior over clean tokens, one position at a time, using the conditional rate and the posterior average. We train the denoiser to predict a categorical distribution for the π family and a ratio for SEDD, as required by the reverse-rate formula.

5. Train a denoiser with the rate loss, which bounds the likelihood and is minimized at the true reverse rates. Check the minimizer on a two-point example, as we did for the posterior average.

6. Condition generation on the properties required, with Doob's h-transform, estimating the

value by a classifier, by rollouts, or by fine-tuning on reweighted paths.

A rate matrix satisfying these conditions determines a discrete diffusion model, its sampler and its training loss.

We chose a corruption process, computed its forward marginals and obtained generation rates by time reversal. The rate matrix determined the marginal path p_t, reverse rates, posterior average and training loss.

We can also begin with a desired probability path and solve the master equation for rates that generate it. In Chapter 5, we choose distributions at intermediate times, then learn a posterior over data tokens to estimate the corresponding rates. This is discrete flow matching. By choosing the path directly, we can control how probability moves from source to data and derive the training objective and sampler for that choice.

Chapter 5

Discrete Flow Matching

5.1 Choosing the probability path

In Chapter 4, we fixed a corruption process and solved for its reverse. We masked or replaced tokens at chosen rates, then obtained the reverse rates by averaging conditional reverse rates under a posterior estimated with a denoiser. Choosing the corruption process also fixed the probability path.

We can also choose the probability path first, derive a velocity that generates it, and construct a sampler from that velocity. We used this approach in continuous space in Chapter 2. Here the chosen path determines the corruption process.

Time runs from the source at $t = 0$ to the data at $t = 1$, so p_0 is the noise distribution and p_1 is the data distribution. In Chapter 4, we placed clean data at $t = 0$ and generated samples backward from $t = 1$. The time t here corresponds to $1 - t$ in Chapter 4, and the schedule κ_t equals the masked-diffusion survival probability α_{1-t}. We write $u_t(y, x)$ for the rate of moving from the current state x to state y at time t, with the destination first. This equals $R_t(x, y)$ in Chapter 4, with the arguments exchanged.

5.1.1 Continuous flow matching

For continuous flow matching, draw a source sample x_0 and a data sample x_1 [Lipman et al., 2023]. Interpolate along the straight path $x_t = (1 - \kappa_t)x_0 + \kappa_t x_1$, with conditional velocity

$$u_t(x \mid x_1) = \frac{\dot{\kappa}_t}{1 - \kappa_t}\,(x_1 - x).$$

"The conditional velocity $u_t(x \mid x_1)$ at the current point x equals the schedule derivative $\dot{\kappa}_t$ divided by the remaining source weight $1 - \kappa_t$, multiplied by the displacement $x_1 - x$ to the data sample x_1."

For each data sample, we have a conditional velocity. Several conditional trajectories can pass through the same point x. To obtain the velocity of the marginal density, we average over the data samples that could have produced x,

$$u_t(x) = \mathbb{E}\big[\, u_t(x \mid X_1) \;\big|\; X_t = x \,\big].$$

"The marginal velocity $u_t(x)$ averages the conditional velocity $u_t(x \mid X_1)$ over possible data samples X_1, given that the current state X_t equals x."

We trained a network by regressing on the conditional velocities. At the minimum of the regression loss, its output equals this posterior average. For tokens, we also need a conditional path for each endpoint pair, a velocity that generates that path, and a loss whose minimizer recovers the posterior average.

5.1.2 Token velocities and continuous relaxations

A continuous velocity measures displacement per unit time. There is no token halfway between A and C, so a letter-valued trajectory x_t has no derivative dx_t/dt. We also cannot take the divergence required by the continuity equation of Chapter 2. We can address this in two ways.

First, we can keep the tokens discrete and describe jumps through their rates. We write the velocity as a rate matrix with the destination first and use the master equation of Chapter 4 in place of the continuity equation.

Second, we can represent each token by a corner of the probability simplex. Points in its interior can move continuously, so we can apply continuous flow matching inside the simplex.

5.1.3 Probability paths, training, and sampling

Discrete flow matching connects the probability path, training loss and sampler through seven results.

1. A probability velocity is a rate matrix read from the destination's side.
2. The master equation is the condition under which a velocity generates a probability path.
3. With a coupling and a mixture path, each position follows a simple conditional path.
4. A conditional velocity generates each conditional path.
5. The posterior average of the conditional velocities generates the marginal path.
6. Cross-entropy on the data token recovers that posterior.
7. Euler steps sample from the path, and velocities with zero net flux correct the samples.

Discrete Flow Matching, discrete flow models, kinetic-optimal paths and Edit Flows differ in their choices of paths, velocities and training procedures within this construction.

5.2 Probability velocities

In Chapter 4, we denoted the jump rate from state x to state y by $R_t(x, y)$, for $x \neq y$, in units of probability per unit time. In discrete flow matching, we write the destination first,

$$u_t(y, x) = R_t(x, y).$$

"The probability velocity $u_t(y, x)$ is the rate of jumping from the current state x to the state y, which is the entry $R_t(x, y)$ of the rate matrix of Chapter 4."

- $u_t(y, x)$ is the probability velocity, the rate at which probability leaves x toward y at time t, in probability per unit time.
- x is the state the process occupies now, written in the second argument.
- y is a candidate destination, written in the first argument.

Fixing x and varying y gives a vector $u_t(\cdot, x)$ with one component for the rate toward each destination. This associates a vector with every state in a finite set, just as a continuous velocity field associates a vector with every spatial point.

5.2.1 Velocity constraints

For a valid velocity, the column of rates leaving x must satisfy the rate-matrix conditions from Chapter 4,

$$u_t(y, x) \geq 0 \quad \text{for } y \neq x, \qquad \sum_y u_t(y, x) = 0.$$

"Every rate $u_t(y, x)$ from x to a different state y is nonnegative, and the rates $u_t(y, x)$ summed over every state y, including x itself, equal zero."

Under the first rule probabilities cannot go negative, since a rate at which probability flows out of a state cannot itself be negative. The second rule conserves total probability, since any probability flowing out of one state has to flow into another. Both are the conditions a rate matrix of a continuous-time Markov chain satisfies, with the column convention in place of the row convention.

Once we choose the off-diagonal entries, we can solve the second rule for the diagonal entry $u_t(x, x)$,

$$u_t(x, x) = -\sum_{y \neq x} u_t(y, x).$$

"The diagonal entry $u_t(x, x)$ equals negative the sum of the rates $u_t(y, x)$ from x to every other state y, which is the total rate of leaving x."

We can choose one nonnegative rate per ordered pair of distinct states; the column-sum condition then determines the diagonal. Its magnitude $|u_t(x, x)|$ is the total exit rate from x. The reciprocal is the mean waiting time before a jump, as we derived from exponential waiting times in Chapter 4.

5.3 The master equation in velocity form

We derive the master equation by balancing the probability arriving at a state against the probability leaving it, as in Chapter 4. With the destination first, we obtain

$$\boxed{\frac{d}{dt}\, p_t(y) = \sum_x u_t(y, x)\, p_t(x)}$$

"The rate of change of the probability $p_t(y)$ of being at state y equals the sum over every state x of the velocity $u_t(y, x)$ from x to y, times the probability $p_t(x)$ of currently being at x."

- p_t is a probability vector on the state space at time t, with one nonnegative entry per state summing to one.
- $p_t(y)$ is the probability of the single state y at time t.
- The sum runs over all states x, including $x = y$, whose term contains the negative diagonal.

We can write the matrix equation as $\dot{p}_t = u_t\, p_t$, where p_t is a column vector and u_t has entry $u_t(y, x)$ at row y, column x. Under the column convention, we multiply the probability vector by u_t from the left, without transposing the matrix.

Definition 5.1 (A velocity generates a path)**.** Let p_t be a family of probability vectors on a finite state space, differentiable in t, and let u_t satisfy the two rules of Section 5.2. We say that u_t generates the path p_t when $\frac{d}{dt}p_t(y) = \sum_x u_t(y, x)p_t(x)$ holds at every state y and every time $t \in [0, 1]$.

In discrete flow matching, we seek a velocity that satisfies the master equation for a chosen probability path.

5.3.1 Separating the diagonal term

We can separate incoming and outgoing probability by expanding the diagonal term. By the second rule of Section 5.2, $u_t(y, y)$ equals minus the total rate out of y. Substituting this value gives

$$\frac{d}{dt}\, p_t(y) = \underbrace{\sum_{x \neq y} u_t(y, x)\, p_t(x)}_{\text{arrives at } y} \; - \; \underbrace{\sum_{x \neq y} u_t(x, y)\, p_t(y)}_{\text{leaves } y}.$$

"The rate of change of the state probability $p_t(y)$ equals the total incoming flux $u_t(y, x)p_t(x)$, summed over all other states x, minus the total outgoing flux $u_t(x, y)p_t(y)$ summed over those states."

Each term is the probability flux along one edge, measured in probability per unit time. To calculate incoming flux along (x, y), multiply the probability at x by the rate from x to y. For outgoing flux, multiply the probability at y by the reverse rate. In Chapter 2, we integrated flux across a region's boundary to calculate its density change. Here, we sum fluxes along the edges entering and leaving a single state.

5.3.2 DNA jump-velocity example

Let the token be a single DNA base, so the state space is $\mathcal{V} = \{A, C, G, T\}$ with $K = 4$, and suppose that at time t the probabilities are

$$p_t = (0.1,\ 0.2,\ 0.3,\ 0.4) \quad \text{for (A, C, G, T).}$$

"The probabilities of A, C, G and T at time t are 0.1, 0.2, 0.3 and 0.4, and they sum to one."

Let each base other than G jump to G at rate 1, and let G remain fixed. Set $u_t(\mathrm{G}, x) = 1$ for $x \in \{A, C, T\}$ and all other off-diagonal rates to zero. The column-sum rule then gives $u_t(x, x) = -1$

for $x \neq \mathrm{G}$ and $u_t(\mathrm{G}, \mathrm{G}) = 0$. With destinations in rows and current states in columns, both ordered A, C, G, T, we obtain

$$u_t = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & -1 \end{pmatrix}.$$

Every column sums to zero and every off-diagonal entry is nonnegative, so both rules hold. To calculate the change in probability at G, we multiply its row of three incoming rates by p_t,

$$\dot{p}_t(\mathrm{G}) = (0.1 + 0.2 + 0.4)(1) = 0.7.$$

"The probability of G, $p_t(\mathrm{G})$, increases by the incoming probabilities 0.1, 0.2, and 0.4 at A, C, and T, each multiplied by its rate 1 toward G. Their sum gives a probability growth rate of 0.7."

Each of the other three bases only loses its own probability, at rate 1, so its rate of change is the negative of its current probability. The four rates of change are

$$\dot{p}_t = (-0.1,\ -0.2,\ 0.7,\ -0.4),$$

which sum to zero by the two rules. Taking an Euler step of size $h = 0.1$, we obtain

$$p_t + 0.1\,\dot{p}_t = (0.09,\ 0.18,\ 0.37,\ 0.36).$$

"An Euler step of size $h = 0.1$ adds the change $0.1\,\dot{p}_t$ to the current probability vector p_t, giving probabilities 0.09, 0.18, 0.37, and 0.36, which sum to one."

The probability of G has increased, and total probability remains one. As t becomes large, all probability accumulates at G, the only state with no outgoing rate.

5.4 Position-wise sequence dynamics

A DNA sequence of length N has 4^N states, one for every assignment of a letter to each position. At $N = 10$, there are $4^{10} = 1{,}048{,}576$ sequences. Specifying a rate for every ordered pair of distinct sequences would require about 1.1×10^{12} numbers, which is impractical for network output and sampling.

We can reduce this cost by allowing each jump to change only one position. Combining the per-position velocities gives the sequence velocity

$$u_t(y, x) = \sum_{i=1}^{N} \delta_{x^{\bar{i}}}(y^{\bar{i}})\; u_t^i(y^i, x).$$

"The sequence velocity $u_t(y, x)$ sums the per-position rates $u_t^i(y^i, x)$ over positions i, counting a rate only when the current sequence x and destination sequence y agree at all other positions."

- $x^i \in \mathcal{V}$ is the token of x at position i, and y^i the token of y there.

- $x^{\bar{i}}$ is every position of x except i, so $\delta_{x^{\bar{i}}}(y^{\bar{i}})$ equals one when x and y agree off position i and zero otherwise.
- $u_t^i(y^i, x)$ is the per-position velocity at position i, the rate at which position i moves to the token y^i while the whole current sequence is x.

For a pair of sequences differing at two or more positions, every indicator in the sum vanishes and the rate is zero. For a pair differing at exactly one position j, only the term $i = j$ remains and the rate is $u_t^j(y^j, x)$. For $y = x$ all N indicators equal one, and the diagonal entry is $\sum_i u_t^i(x^i, x)$, the sum of the per-position diagonals, which is the total rate at which any position of x changes.

We need K rates per position, or NK rates for the current sequence. At $N = 10$, this is $10 \times 4 = 40$ numbers compared with about 10^{12} for all sequence pairs. Each per-position velocity can still depend on the whole sequence x through its second argument. One network pass therefore uses the full context to predict the $N \times K$ rates.

5.4.1 One Euler step at one position

A first-order expansion gives the transition probability over a short interval, as in Chapter 4. For a step of size h, we draw the next token at position i from

$$\Pr\left(X_{t+h}^i = y^i \mid X_t = x\right) = \delta_{x^i}(y^i) + h\, u_t^i(y^i, x).$$

"The probability that position i holds the token y^i after a step of size h, given that the current sequence is x, equals the indicator that y^i is the current token x^i, plus h times the per-position velocity $u_t^i(y^i, x)$."

For sufficiently small h, this expression is a probability distribution over $\mathcal{V}$. Its entries sum to one because the per-position velocities sum to zero. To keep every entry nonnegative, we need the current-token entry to satisfy

$$h\,|u_t^i(x^i, x)| \le 1.$$

The step size must be at most the reciprocal of the total exit rate from the current token, as for the reverse steps in Chapter 4.

We draw all N positions in parallel, conditioned on the same current sequence x. This update can change multiple positions within one step, whereas each jump changes only one position.

5.4.2 The product over positions

Let's multiply the per-position step probabilities and expand the product,

$$\prod_{i=1}^{N} \left[\delta_{x^i}(y^i) + h\, u_t^i(y^i, x)\right] = \delta_x(y) + h\, u_t(y, x) + O(h^2).$$

"The product over positions of the per-position step probabilities equals the indicator that y is x, plus h times the sequence velocity $u_t(y, x)$, up to terms of order h squared."

We check three cases. When $y = x$, every factor is $1 + h\, u_t^i(x^i, x)$ and the product is $1 + h \sum_i u_t^i(x^i, x) + O(h^2)$, which is $1 + h\, u_t(x, x) + O(h^2)$ by the diagonal computed above. When y differs from x at

exactly one position j, factor j contributes $h\,u_t^j(y^j, x)$ and every other factor contributes $1 + O(h)$, so the product is $h\,u_t(y, x) + O(h^2)$. When y differs at two or more positions, at least two factors contain a power of h and the product is $O(h^2)$.

The h^2 terms are the events in which two positions change inside the same step, and the sequence velocity has no rate at all for those events. The parallel update and the one-position-at-a-time process therefore agree to first order in h, and a sampler may update every position at once.

Suppose two positions each have a total outgoing rate of 1.6, and take $h = 0.1$. Each position changes with probability $h \times 1.6 = 0.16$, and both change in the same step with probability $0.16^2 = 0.0256$, which is h^2 times the product of the two rates. Exactly one of the two changes with probability $0.16 \times 0.84 = 0.1344$. The double change is about one sixth as likely as a single change at this step size. Halving h halves the probability of a single change and quarters the probability of a double one, so the discarded term shrinks faster than the term we keep.

To specify the rates, we first need a probability path.

5.5 Source-data probability paths

In flow matching we choose a probability path with the right endpoints,

$$p_0 = p, \qquad p_1 = q,$$

where p is an easily sampled source distribution and q is the data distribution. Running a velocity that generates this path from $t = 0$ to $t = 1$ transforms source samples into data samples. We cannot tabulate either q or a full path on $\mathcal{V}^N$. As in Chapter 2, we therefore construct p_t by averaging simple conditional paths over endpoint pairs.

5.5.1 A coupling of the source and the data

The pairs come from a joint distribution over source and data sequences,

$$(X_0, X_1) \sim \pi, \qquad \sum_{x_1} \pi(x_0, x_1) = p(x_0), \qquad \sum_{x_0} \pi(x_0, x_1) = q(x_1).$$

"The pair of a source sequence X_0 and a data sequence X_1 is drawn from the coupling π, whose sum over data sequences x_1 returns the source distribution $p(x_0)$ and whose sum over source sequences x_0 returns the data distribution $q(x_1)$."

- $\pi(x_0, x_1)$ is the coupling, a probability distribution on pairs of sequences.
- p is the source distribution, the first marginal of π.
- q is the data distribution, the second marginal of π.

Any joint distribution with those two marginals is admissible, and the simplest one draws the two ends independently,

$$\pi(x_0, x_1) = p(x_0)\,q(x_1).$$

To sample from the independent coupling, we draw once from the source and once from the training set. One possible source is the all-mask sequence $(m, \ldots, m)$, with p a point mass and m an extra

symbol outside $\mathcal{V}$. Another is a sequence of tokens drawn independently and uniformly from $\mathcal{V}$. Pairing each source sequence with a nearby data sequence changes the velocity we must learn.

5.5.2 The mixture path at each position

Given the pair (x_0, x_1), the conditional path at position i holds one of two tokens,

$$\boxed{p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i}(x^i) + \kappa_t\, \delta_{x_1^i}(x^i)}$$

"The conditional token probability $p_t(x^i \mid x_0, x_1)$ assigns weight $1 - \kappa_t$ to the source token x_0^i and weight κ_t to the data token x_1^i, where κ_t is the schedule at time t."

- κ_t is the schedule, a function of time with $\kappa_0 = 0$ and $\kappa_1 = 1$, increasing in t.
- $\dot{\kappa}_t = d\kappa_t/dt$ is its derivative, nonnegative on $[0, 1]$.
- x_0^i and x_1^i are the source and data tokens at position i.
- $\delta_a(b)$ equals one when $a = b$ and zero otherwise.

The mixture has two point masses. The data token has weight κ_t, and the source token has weight $1 - \kappa_t$. At $t = 0$, the data weight is zero and the position holds its source token. At $t = 1$, the data weight is one and the position holds its data token. Every other alphabet token has probability zero.

Positions are independent once the pair is fixed, so the conditional path on sequences is a product,

$$p_t(x \mid x_0, x_1) = \prod_{i=1}^{N} p_t(x^i \mid x_0, x_1).$$

"The conditional probability $p_t(x \mid x_0, x_1)$ of the whole sequence x equals the product of its position probabilities $p_t(x^i \mid x_0, x_1)$, with the endpoint pair (x_0, x_1) fixed."

The dependence between positions is therefore entirely in the coupling and in the data distribution, never in the conditional path. This is the discrete counterpart of the conditional Gaussian of Chapter 2, whose coordinates were independent given the two endpoints.

5.5.3 Single-position path example

Let's follow a single position whose source token is T and whose data token is C, under the linear schedule $\kappa_t = t$. At $t = 0$ all of the probability is on T. At $t = 0.25$ a quarter has moved to C, leaving 0.75 on T. Halfway through, T and C are equally likely at 0.50 each. At $t = 0.75$ the weights are 0.75 on C and 0.25 on T, and at $t = 1$ the position holds C with probability one. Across all five times the bars on A and on G stay at zero, so the entire motion is along the single edge from T to C (Figure 5.1).

For this path, probability moves along one edge, so we can use a velocity supported on that edge. We obtain the full probability path by averaging over endpoint pairs.

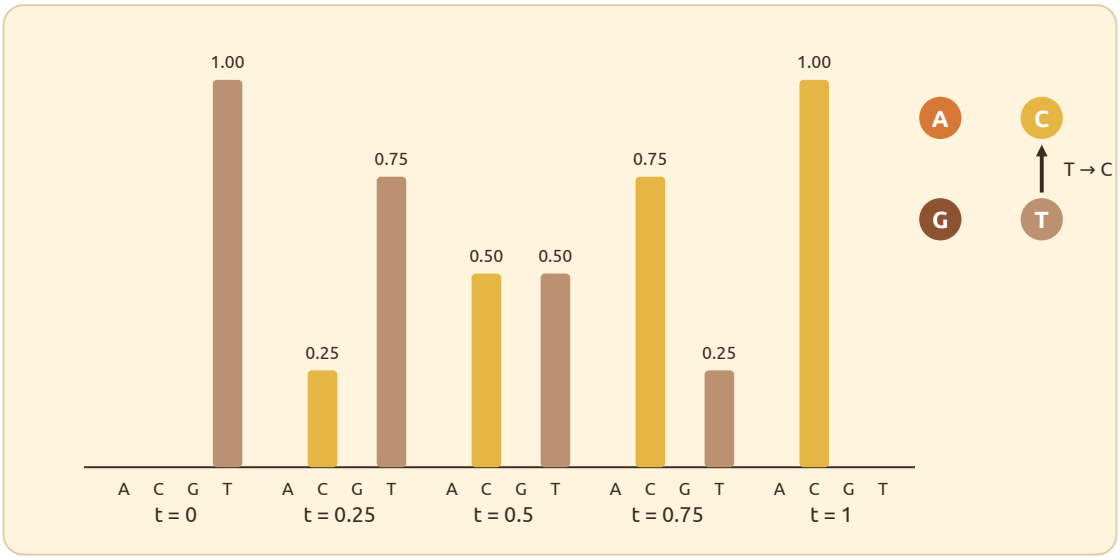


Figure 5.1: One position under the conditional mixture path $p_t(x^i \mid x_0, x_1)$, with source token T and data token C, drawn at $t = 0$, 0.25, 0.5, 0.75 and 1 under the linear schedule $\kappa_t = t$. Each group of four bars is a probability distribution over A, C, G and T, and the number above a bar is its probability. The bar on T falls from 1.00 to zero as $1 - t$ and the bar on C rises from zero to 1.00 as t; A and G stay empty at every time, so all of the probability travels along the single edge from T to C marked at the right.

5.5.4 Averaging the conditional paths over the coupling

The marginal path is the coupling-weighted average of the conditional paths,

$$p_t(x) = \sum_{x_0,\, x_1} p_t(x \mid x_0, x_1)\, \pi(x_0, x_1).$$

"The marginal probability $p_t(x)$ of the sequence x is the sum over every source sequence x_0 and every data sequence x_1 of the conditional probability $p_t(x \mid x_0, x_1)$, weighted by the coupling $\pi(x_0, x_1)$."

At $t = 0$ every conditional path is the point mass at its own source sequence, so the average is $\sum_{x_0,x_1} \delta_{x_0}(x)\pi(x_0, x_1) = p(x)$, the first marginal of the coupling. At $t = 1$ every conditional path is the point mass at its data sequence, so the average is $q(x)$, the second marginal. The path therefore satisfies $p_0 = p$ and $p_1 = q$ for any coupling and any schedule with $\kappa_0 = 0$ and $\kappa_1 = 1$.

Evaluating $p_t(x)$ requires an intractable sum over every pair of sequences. For training, we only need samples from p_t, which we can draw directly.

5.5.5 Sampling from the marginal path

Draw a pair (x_0, x_1) from π, then flip an independent coin at every position with heads probability κ_t. Keep the data token for heads and the source token for tails. This produces a draw from p_t: we sample the pair from the coupling and each position from its conditional mixture.

Now take a data sequence $x_1 = \text{ACGT}$, a uniform source draw $x_0 = \text{GTTA}$, and $\kappa_t = 0.5$. Suppose the four coins land heads, tails, heads, tails. Position 1 takes its data token A, position 2 keeps its

source token T, position 3 takes its data token G, and position 4 keeps its source token A, so

$$x_t = \text{ATGA}.$$

Positions 1 and 3 hold their data tokens, and positions 2 and 4 hold source tokens. We use ATGA as the noisy training input and ACGT as the target.

5.5.6 Two choices of source

With the all-mask source $x_0 = (m, \ldots, m)$, every position is either masked or holding its data token,

$$p_t(x^i \mid m, x_1) = (1 - \kappa_t)\, \delta_m(x^i) + \kappa_t\, \delta_{x_1^i}(x^i).$$

"The conditional token probability $p_t(x^i \mid m, x_1)$ assigns the remaining source weight $1 - \kappa_t$ to the mask m and the data weight κ_t to the data token x_1^i."

This is the forward marginal of masked diffusion with the clock reversed [Sahoo et al., 2024]. The schedule κ_t here equals the survival probability α_{1-t} of Chapter 4, so a position masked with probability $1 - \alpha_s$ at Chapter 4's time s is masked with probability $1 - \kappa_t$ at time $t = 1 - s$ here.

With a uniform source, a position that has not yet moved holds a random token from $\mathcal{V}$. A visible token may therefore be the data token or an uninformative replacement, as in uniform diffusion. Both sources give mixture paths of the same form, differing only in x_0. We can derive their velocities without specifying which source we use.

5.6 The velocity of the mixture path

To generate samples along the mixture path, we need its velocity. We can begin by differentiating the conditional path at one position. The two indicators are constant in t, so

$$\frac{d}{dt}\, p_t(a \mid x_0, x_1) = \dot{\kappa}_t \left[\delta_{x_1}(a) - \delta_{x_0}(a)\right].$$

"The conditional probability $p_t(a \mid x_0, x_1)$ changes at the schedule rate $\dot{\kappa}_t$, multiplied by the difference between the indicator that a is the data token x_1 and the indicator that it is the source token x_0."

Here a is a candidate token at the position under study, and x_0 and x_1 are that position's source and data tokens, with the position index suppressed. Probability flows out of the source token at rate $\dot{\kappa}_t$ and into the data token at the same rate, so the entire motion runs along one edge, exactly as in Figure 5.1.

5.6.1 The conditional velocity

The rates must satisfy the master equation for the conditional path. For a position currently at z, we need an outgoing rate toward the data token. Dividing the required flux $\dot{\kappa}_t$ by the remaining source probability $1 - \kappa_t$, we obtain

$$u_t(a, z \mid x_1) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\delta_{x_1}(a) - \delta_z(a)\right].$$

"The conditional jump rate $u_t(a, z \mid x_1)$ from the current token z to token a equals the schedule derivative $\dot{\kappa}_t$ divided by the remaining source weight $1 - \kappa_t$, multiplied by the difference between the data-token indicator $\delta_{x_1}(a)$ and current-token indicator $\delta_z(a)$."

- $z \in \mathcal{V}$ is the token the position holds now, the second argument of the velocity.
- $a \in \mathcal{V}$ is the candidate destination, the first argument.
- x_1 is the data token at this position, which is the only part of the pair the velocity depends on.
- $\dot{\kappa}_t/(1 - \kappa_t)$ is the schedule factor, a positive number with units of inverse time.

Summing over a cancels the bracket, since each indicator sums to one, so the entries sum to zero. For $a \neq z$ the bracket equals $\delta_{x_1}(a)$, which is nonnegative, and the schedule factor is nonnegative whenever κ_t is increasing and $\kappa_t < 1$, so both rules hold. The velocity is zero everywhere except at $a = x_1$ and $a = z$, so a position jumps straight to its data token or stays where it is.

If a position already holds its data token, $z = x_1$ and the bracket vanishes for every a. The velocity is then zero, so the position never moves again. If the source and data tokens agree, the conditional path is constant and the velocity is zero throughout. The velocity depends on x_1 and the current token z, with no direct dependence on x_0.

5.6.2 Checking the master equation

Let's substitute the conditional velocity and path into the master equation. Only $z = x_0$ and $z = x_1$ have probability, so the sum over z contains two terms:

$$\sum_z u_t(a, z \mid x_1)\, p_t(z \mid x_0, x_1) = (1 - \kappa_t)\, \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\delta_{x_1}(a) - \delta_{x_0}(a)\right] + \kappa_t \cdot 0.$$

"The right side of the master equation equals one minus κ_t, the probability of sitting at the source token, times the velocity out of x_0, plus κ_t, the probability of sitting at the data token, times the velocity out of x_1, which is zero."

Now, we can cancel the factors of $1 - \kappa_t$:

$$= \dot{\kappa}_t \left[\delta_{x_1}(a) - \delta_{x_0}(a)\right] = \frac{d}{dt}\, p_t(a \mid x_0, x_1).$$

"The net conditional flux is the schedule derivative $\dot{\kappa}_t$ times the difference between the data-token indicator $\delta_{x_1}(a)$ and source-token indicator $\delta_{x_0}(a)$, equal to the time derivative of $p_t(a \mid x_0, x_1)$."

The conditional velocity satisfies the master equation and therefore generates the conditional path, as in Definition 5.1. We can also see why its denominator is $1 - \kappa_t$. The required flux is $\dot{\kappa}_t$, and only probability $1 - \kappa_t$ remains at the source. We must divide the flux by that remaining probability to obtain the jump rate. As the source probability decreases, the rate per remaining sample increases.

5.6.3 Survival and hazard

The schedule factor is a hazard rate in the sense of Chapter 4, and the mixture path is its survival curve. A position still at its source token at time t jumps at rate $\dot\kappa_t/(1-\kappa_t)$, so its survival probability $S(t)$, the probability that it has not yet jumped, obeys

$$\frac{dS}{dt} = -\frac{\dot\kappa_t}{1-\kappa_t}\,S(t), \qquad S(0)=1 \quad\Longrightarrow\quad S(t) = 1-\kappa_t.$$

"The rate of change of the survival probability $S(t)$ equals negative $\dot\kappa_t$ divided by one minus κ_t, times $S(t)$ itself, and with $S(0)=1$ the solution is $S(t)=1-\kappa_t$."

Separating variables and integrating,

$$S(t) = \exp\!\Big(-\int_0^t \frac{\dot\kappa_s}{1-\kappa_s}\,ds\Big) = \exp\big(\log(1-\kappa_t) - \log(1-\kappa_0)\big) = 1-\kappa_t,$$

since the integrand is the derivative of $-\log(1-\kappa_s)$ and $\kappa_0 = 0$. Under this rate the probability $1-\kappa_t$ remains on the source token at time t, the same weight the mixture path has there.

With the linear schedule $\kappa_t = t$ the derivative is $\dot\kappa_t = 1$ and the rate is

$$\kappa_t = t: \qquad \frac{\dot\kappa_t}{1-\kappa_t} = \frac{1}{1-t} = 2 \text{ at } t=0.5, \qquad 10 \text{ at } t=0.9.$$

"With the linear schedule $\kappa_t = t$, the jump rate is one divided by one minus t, which equals 1 at $t=0$, 2 at $t=0.5$ and 10 at $t=0.9$."

As t approaches one, less time remains for a position to leave its source token and the jump rate increases (Figure 5.2). The rate diverges at $t=1$, so every position reaches its data token by the end. The accumulated hazard $\int_0^1 \dot\kappa_s/(1-\kappa_s)\,ds$ is infinite, and the survival probability reaches zero at $t=1$.

The conditional velocity contains x_1, the data token, and at generation time that token is the quantity we are trying to produce.

5.7 Posterior-averaged velocities

During training, we know the data sample x_1 because we draw the endpoint pair from the coupling. During generation, we know only the current sequence x. We can average the conditional velocity over the posterior of data samples consistent with x, as we did for continuous velocities in Chapter 2 and reverse rates in Chapter 4.

Theorem 5.2 (Marginal velocity)**.** *Let π be a coupling of the source and data distributions, let $p_t(\cdot \mid x_0, x_1)$ be a conditional path for each pair with a conditional velocity $u_t(\cdot,\cdot \mid x_0, x_1)$ that generates it, and let p_t be the marginal path of Section 5.5.4. Then the posterior average*

$$u_t^i(y^i, x) = \sum_{x_0,\,x_1} u_t^i(y^i, x \mid x_0, x_1)\; p_t(x_0, x_1 \mid x)$$

generates the marginal path p_t, where $p_t(x_0, x_1 \mid x)$ is the posterior over pairs given the current sequence x.

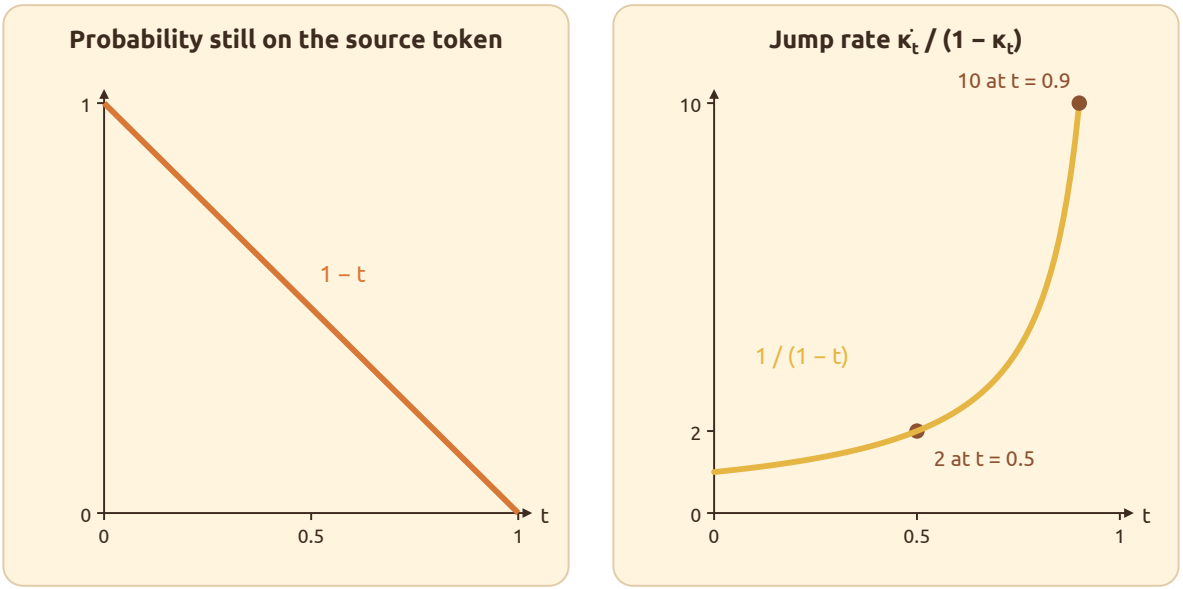


Figure 5.2: The linear schedule $\kappa_t = t$, drawn as a survival probability and as a jump rate. Left: the probability $1 - \kappa_t = 1 - t$ that a position still holds its source token, falling in a straight line from 1 at $t = 0$ to 0 at $t = 1$. Right: the rate $\dot{\kappa}_t/(1 - \kappa_t) = 1/(1 - t)$ at which such a position jumps to its data token, marked at the two values 2 at $t = 0.5$ and 10 at $t = 0.9$, and growing without bound as t approaches 1. The horizontal axis of both panels is time t from 0 to 1.

"The marginal rate $u_t^i(y^i, x)$ of replacing position i by token y^i averages the conditional rate $u_t^i(y^i, x \mid x_0, x_1)$ over source-data pairs (x_0, x_1), weighted by their posterior probability $p_t(x_0, x_1 \mid x)$ given the current sequence x."

This result appears in Discrete Flow Matching [Gat et al., 2024] and the discrete flow models used in Multiflow [Campbell et al., 2024]. It is the discrete counterpart of the marginal-velocity result from Chapter 2.

5.7.1 The posterior weights

The weights are a straight application of Bayes' rule,

$$p_t(x_0, x_1 \mid x) = \frac{p_t(x \mid x_0, x_1)\, \pi(x_0, x_1)}{p_t(x)}.$$

"The posterior probability of the pair (x_0, x_1) given the current sequence x equals the conditional path probability $p_t(x \mid x_0, x_1)$ times the coupling $\pi(x_0, x_1)$, divided by the marginal probability $p_t(x)$."

The numerator is the probability of drawing the pair and then observing x at time t. The denominator sums this quantity over every pair and normalizes the weights. We can sum this posterior over pairs to obtain a token distribution for network training.

5.7.2 Marginal-path derivation

Let's differentiate the marginal path of Section 5.5.4 with respect to t. The coupling does not depend on t, so the derivative passes through the sum onto each conditional path, where each conditional master equation applies:

$$\frac{d}{dt}\,p_t(y) = \sum_{x_0,\,x_1} \pi(x_0, x_1) \sum_x u_t(y, x \mid x_0, x_1)\; p_t(x \mid x_0, x_1).$$

"The state probability $p_t(y)$ changes by the conditional incoming flux $u_t(y, x \mid x_0, x_1)p_t(x \mid x_0, x_1)$, summed over current states x and averaged over endpoint pairs (x_0, x_1) under their coupling."

Next, we can exchange the two sums, then multiply and divide by $p_t(x)$, which is positive wherever the inner sum is nonzero:

$$\frac{d}{dt}\,p_t(y) = \sum_x p_t(x)\Big[\sum_{x_0,\,x_1} u_t(y, x \mid x_0, x_1)\,\frac{p_t(x \mid x_0, x_1)\,\pi(x_0, x_1)}{p_t(x)}\Big] = \sum_x u_t(y, x)\,p_t(x).$$

"After we factor $p_t(x)$ out of the inner sum, the bracket contains the posterior average of the conditional velocities. This average equals the marginal velocity $u_t(y, x)$, so the expression satisfies the marginal master equation."

The fraction inside the bracket is the posterior from Section 5.7.1. Because the master equation is linear in the velocity, we obtain the same result by averaging the fluxes or averaging the velocities. A nonlinear condition would not permit this exchange.

A convex combination of nonnegative off-diagonal entries is nonnegative, and a convex combination of columns that sum to zero again sums to zero, so the marginal velocity satisfies both rules of Section 5.2.

5.7.3 Data-token posterior velocity

We can simplify the sum over sequence pairs in Theorem 5.2. The conditional velocity depends on a pair only through its data token at position i, so all pairs sharing that token contribute the same velocity. Grouping those pairs reduces the sum to four token values:

$$\boxed{u_t^i(y^i, x) = \frac{\dot\kappa_t}{1-\kappa_t}\,\big[\,p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i)\,\big]}$$

"The marginal rate $u_t^i(y^i, x)$ equals the schedule factor $\dot\kappa_t/(1-\kappa_t)$ multiplied by the difference between the posterior probability $p_{1|t}(y^i \mid x)$ of data token y^i and the indicator $\delta_{x^i}(y^i)$ that it is the current token at position i."

The one quantity left is the denoiser, the posterior probability that the data token at position i equals y^i,

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\,x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x).$$

"The denoiser probability $p_{1|t}(y^i \mid x)$ sums the posterior weights of endpoint pairs whose data token x_1^i equals y^i, conditioned on the current sequence x."

- $p_{1|t}(\cdot \mid x)$ is a probability distribution over $\mathcal{V}$ for each position i and each current sequence x, the posterior over the clean token at that position.
- $\delta_{x^i}(\cdot)$ is the point mass on the token the position currently holds.
- The difference of the two is a signed vector summing to zero, which the schedule factor scales into a velocity.

We therefore train the network to predict the data token, as in Chapter 4. One forward pass over the current sequence gives a distribution over $\mathcal{V}$ at each position. To obtain each velocity, we subtract a point mass and multiply by a scalar depending only on t. The velocity has no explicit dependence on the source distribution or coupling, and the schedule enters through $\dot{\kappa}_t/(1-\kappa_t)$.

5.7.4 Averaging the jumps out of G

Let's take one position currently holding G, with the denoiser predicting

$$p_{1|t}(\cdot \mid x) = (0.1,\ 0.6,\ 0.2,\ 0.1) \quad \text{for (A, C, G, T)},$$

so the network puts most of its mass on C as the clean token at this position. Under the linear schedule at $t = 0.5$ the factor is $\dot{\kappa}_t/(1-\kappa_t) = 1/0.5 = 2$, and the marginal velocity is

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2).$$

"At the current token G, the schedule factor 2 multiplies the difference between the denoiser probabilities and a point mass on G. The resulting jump rates toward A, C, and T are 0.2, 1.2, and 0.2, with diagonal entry -1.6 at G."

The three off-diagonal rates are nonnegative and the four entries sum to zero, so this is a valid velocity. The total rate of leaving G is 1.6, and the rate toward C is six times the rate toward A, in the same proportion as the denoiser probabilities 0.6 and 0.1. One Euler step of size $h = 0.1$ at this position moves the probabilities to

$$\delta_{\mathrm{G}} + 0.1\, u_t(\cdot, x) = (0.02,\ 0.12,\ 0.84,\ 0.02).$$

"After one step of size 0.1, the position becomes A with probability 0.02, becomes C with probability 0.12, stays at G with probability 0.84, and becomes T with probability 0.02."

The four numbers sum to one, and C, the token the denoiser scores highest, is the most likely replacement. The position keeps its current token with probability $0.84 = 1 - 0.1 \times 1.6$, one minus the step size times the total rate of leaving.

5.8 The Euler sampling step

If we substitute the marginal velocity into the per-position Euler step of Section 5.4.1 and collect the two terms, we get

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1-\lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\,\dot{\kappa}_t}{1-\kappa_t}.$$

"The Euler step distribution at position i equals one minus λ_t times the point mass on the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is the step size h times $\dot{\kappa}_t$ divided by one minus κ_t."

- $\lambda_t \in [0, 1]$ is the redraw probability, the product of the step size and the schedule factor.
- h is the step size of the sampler, chosen at generation time.

At each position, redraw from the denoiser distribution with probability λ_t, or keep the current token with probability $1 - \lambda_t$. We need only one Bernoulli draw and one categorical draw per position, without forming the velocity or normalizing rates.

At $t = 0.5$ with $h = 0.1$ the redraw probability is $\lambda_t = 0.1 \times 2 = 0.2$, and a redraw returns G with probability 0.2, so the total probability of still holding G after the step is

$$0.8 + 0.2 \times 0.2 = 0.84,$$

which is the third entry computed in Section 5.7.4. Each of the other three entries is λ_t times its denoiser probability: $0.2 \times 0.6 = 0.12$ for C, and $0.2 \times 0.1 = 0.02$ for A and for T.

The rule is valid when $\lambda_t \leq 1$. As κ_t approaches one, λ_t increases, so we must shrink the step size near $t = 1$ or clip a redraw probability above one to one.

5.8.1 Continuous and discrete velocities

We can compare the continuous and discrete velocities directly. For the straight-line path of Chapter 2, the continuous velocity is

$$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right],$$

"The continuous marginal velocity $u_t(x)$ equals the schedule factor $\dot{\kappa}_t/(1-\kappa_t)$ times the displacement from the current point x to the posterior mean data point $\hat{x}_{1|t}(x)$."

and on tokens, at each position,

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right].$$

"The discrete marginal velocity $u_t^i(\cdot, x)$ equals the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ times the difference between the data-token posterior $p_{1|t}(\cdot \mid x)$ and the point mass on the current token x^i."

The posterior mean of a vector corresponds to a posterior distribution over tokens. The current point corresponds to a point mass on the current token. We compare the terms in Table 5.1; this correspondence also appears in Discrete Flow Matching [Gat et al., 2024].

To compute these velocities, we need the denoiser $p_{1|t}(\cdot \mid x)$. We cannot evaluate this posterior directly, so we must estimate it from samples.

Table 5.1: The continuous construction of Chapter 2 and the discrete construction of this chapter, written with the same schedule factor $\dot\kappa_t/(1-\kappa_t)$. Each row names one ingredient of the velocity and its two forms. In the last row, $\lambda_t = h\dot\kappa_t/(1-\kappa_t)$ is the redraw probability of Section 5.8.

	continuous	discrete
state	a point $x \in \mathbb{R}^d$	a token $x^i \in \mathcal{V}$ at each position
path	$(1-\kappa_t)x_0 + \kappa_t x_1$	$(1-\kappa_t)\delta_{x_0^i} + \kappa_t \delta_{x_1^i}$
network output	posterior mean $\hat{x}_{1\|t}(x)$	posterior distribution $p_{1\|t}(\cdot \mid x)$
current state enters as	the point x	the point mass δ_{x^i}
velocity	$\frac{\dot\kappa_t}{1-\kappa_t}[\hat{x}_{1\|t}(x) - x]$	$\frac{\dot\kappa_t}{1-\kappa_t}[p_{1\|t}(\cdot \mid x) - \delta_{x^i}]$
one step of size h	displace by $h\, u_t(x)$	redraw from $p_{1\|t}$ with probability λ_t

5.9 Training the denoiser with cross-entropy

The posterior in Section 5.7.1 requires a sum over all sequence pairs. We can train using three accessible draws: a time, an endpoint pair from the coupling, and a noisy sequence from the conditional path. With these samples, we define the loss

$$\boxed{\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot\mid X_0,X_1)} \sum_{i=1}^{N} \log p^\theta_{1|t}(X_1^i \mid X_t)}$$

"The denoising loss $\mathcal{L}(\theta)$ averages the summed negative log probabilities $p^\theta_{1|t}(X_1^i \mid X_t)$ assigned to the true data tokens X_1^i at positions i, over uniform times t, source-data pairs (X_0, X_1) from the coupling π, and noisy sequences X_t from the conditional path."

- θ are the network parameters.
- $p^\theta_{1|t}(\cdot \mid X_t)$ is the network's predicted distribution over the data token at a position, given the noisy sequence and the time.
- t is drawn uniformly on $[0, 1]$, so every noise level is trained on equally.

Each term is the cross-entropy between the network prediction and the true data token at one position. For each training step, we sample a time, an endpoint pair and a noisy sequence, then score every true data token. The loss contains no schedule factor or rates. After training, we can therefore use the same network with different step sizes and correctors.

5.9.1 The minimizer of the loss

To identify the minimizer we average over the data token before averaging over anything else. Given X_t, the true data token at position i is distributed as the denoiser $p_{1|t}(\cdot \mid X_t)$, so

$$\mathbb{E}\big[-\log p^\theta_{1|t}(X_1^i \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^\theta_{1|t}(y \mid X_t)\big)\Big].$$

"The expected loss at position i equals the expectation over X_t of the cross-entropy between the true denoiser $p_{1|t}(\cdot \mid X_t)$ and the network's prediction $p^\theta_{1|t}(\cdot \mid X_t)$, because the data token given X_t follows the true denoiser."

Splitting the cross-entropy into an entropy and a divergence,

$$= \mathbb{E}_{X_t}\Big[\, H\big(p_{1|t}(\cdot \mid X_t)\big) + D_{\mathrm{KL}}\Big(p_{1|t}(\cdot \mid X_t) \,\Big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\Big) \,\Big].$$

"The expected denoising loss equals the true posterior entropy H, averaged over noisy sequences, plus the average KL divergence from that posterior to the network prediction. Only the divergence depends on the network parameters θ."

Only the divergence depends on θ. It is nonnegative and equals zero exactly when the network matches the true denoiser at every current sequence. Minimizing cross-entropy on observed tokens therefore recovers the posterior from Section 5.7.3, which we need to compute the velocity. The minimum loss is the expected entropy, a positive number. Similarly, in Chapter 2, the minimum flow-matching regression loss was the conditional variance of the velocity.

5.9.2 Scoring one training example

Let's return to the pair of Section 5.5.5, with noisy input x_t = ATGA and target x_1 = ACGT. Suppose a partly trained network assigns the true tokens the probabilities 0.7, 0.2, 0.6 and 0.1 at the four positions. The loss for this example is the sum of the four negative logs,

$$-\big(\log 0.7 + \log 0.2 + \log 0.6 + \log 0.1\big) = 0.357 + 1.609 + 0.511 + 2.303 = 4.780.$$

"The loss for this example is the sum of the negative logarithms of 0.7, 0.2, 0.6 and 0.1, which equals 0.357 plus 1.609 plus 0.511 plus 2.303, or 4.780 nats."

Positions 2 and 4 contribute the largest losses, 1.609 and 2.303 nats. Both hold source tokens, so we must infer C and T from the remaining sequence without information from the visible token. Positions 1 and 3 already hold their data tokens and receive high predicted probabilities, but still contribute 0.357 and 0.511 nats. We sum over every position, including those holding data tokens. In the masked-diffusion loss of Chapter 4, we summed only over masked positions.

5.9.3 Mask-source posterior example

With the all-mask source, a visible token is always the data token at that position, so the posterior over data sequences is supported on those that agree with every visible token. Each surviving candidate has the same time factor,

$$p_t(x \mid m, x_1) = (1-\kappa_t)^{\#\mathrm{masked}(x)}\ \kappa_t^{\#\mathrm{visible}(x)} \prod_{i\ \mathrm{visible}} \delta_{x_1^i}(x^i).$$

"The conditional probability of x given the mask source and the data sequence x_1 equals one minus κ_t raised to the number of masked positions of x, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches the token of x_1."

Proposition 5.3 (The mask denoiser does not depend on time)**.** *With the all-mask source and the independent coupling, the posterior $p_{1|t}(\cdot \mid x)$ at a masked position depends on the current sequence x alone, and not on t.*

The time factor $(1-\kappa_t)^{\#\text{masked}}\kappa_t^{\#\text{visible}}$ depends on x through its counts of masked and visible positions, and those counts are the same for every candidate x_1. The factor is therefore common to every term of the numerator in Bayes' rule and cancels against the same factor in the denominator. We observed the same cancellation for masked diffusion in Chapter 4, where the denoiser also depended on the masked sequence alone. The result is Proposition 6 of Discrete Flow Matching [Gat et al., 2024].

Let's take length two, the mask source, and a data distribution over three sequences,

$$q(\text{AC}) = 0.5, \qquad q(\text{AG}) = 0.3, \qquad q(\text{GC}) = 0.2,$$

and suppose the current state at time t is $x = (\text{A}, m)$, with position 1 visible and position 2 masked. At $\kappa_t = 0.5$ the time factor is $(1-0.5)^1(0.5)^1 = 0.25$ for every candidate, and the unnormalized posterior weights are

$$0.25 \times 0.5 = 0.125 \text{ (AC)}, \qquad 0.25 \times 0.3 = 0.075 \text{ (AG)}, \qquad 0 \text{ (GC)},$$

where GC is excluded because its first token disagrees with the visible A. Normalizing by the total 0.2,

$$p_t(x_1 = \text{AC} \mid x) = 0.625, \qquad p_t(x_1 = \text{AG} \mid x) = 0.375.$$

At masked position 2, we sum the posterior weights of candidates with each second token to obtain $p_{1|t}(\text{C} \mid x) = 0.625$ and $p_{1|t}(\text{G} \mid x) = 0.375$, with A and T at zero. If we repeat the computation at $\kappa_t = 0.8$, the common factor becomes $(0.2)^1(0.8)^1 = 0.16$, so the weights become 0.08 and 0.048 with total 0.128, and the normalized posterior is again 0.625 and 0.375. The denoiser is unchanged at the new time, as stated in Proposition 5.3.

The velocity at the masked position does change with time, through the schedule factor alone. At $\kappa_t = 0.5$ the factor is 2 and the velocity out of the mask symbol is

$$u_t^2(\text{C}, x) = 2 \times 0.625 = 1.25, \qquad u_t^2(\text{G}, x) = 2 \times 0.375 = 0.75, \qquad u_t^2(m, x) = -2,$$

whose entries sum to zero. After a step of size $h = 0.1$, the position remains masked with probability 0.8, becomes C with probability 0.125, and becomes G with probability 0.075. At $\kappa_t = 0.8$, the schedule factor increases to 5, giving redraw probability $\lambda_t = 0.5$. The same step then leaves probability 0.5 on the mask, 0.3125 on C, and 0.1875 on G. We use the same denoiser at both times, with a higher redraw rate at the later time.

The entropy of this posterior also bounds the loss from below, by Section 5.9.1. With probabilities 0.625 and 0.375,

$$H = -\big(0.625 \log 0.625 + 0.375 \log 0.375\big) = 0.625 \times 0.470 + 0.375 \times 0.981 = 0.6616,$$

so no network can score better than 0.6616 nats on average at this position. A network predicting 0.5 and 0.5 scores 0.6931 nats, and the gap $0.6931 - 0.6616 = 0.0316$ is exactly the KL divergence from the true posterior to that prediction. By training to the optimum, we reduce this gap to zero. The remaining entropy measures the ambiguity about the second token when only the first is visible.

5.9.4 Comparison with the masked-diffusion bound

We can rewrite the masked-diffusion likelihood bound from Chapter 4 using the time correspondence in Section 5.1,

$$\int_0^1 \frac{\dot\kappa_t}{1-\kappa_t}\,\mathbb{E}\Big[\sum_{i:\,X_t^i=m} -\log p_{1|t}^\theta(X_1^i \mid X_t)\Big]\,dt.$$

"The masked-diffusion loss integrates the schedule weight $\dot\kappa_t/(1-\kappa_t)$ times the expected negative log probability $p_{1|t}^\theta(X_1^i \mid X_t)$ of each true data token X_1^i, summed only over positions i where the noisy token X_t^i is the mask m."

Two differences separate it from $\mathcal{L}(\theta)$. The bound has the time weight $\dot\kappa_t/(1-\kappa_t)$, which grows without bound near $t=1$, and it sums over masked positions alone. The cross-entropy loss has no such weight and sums over every position.

Both losses have the same minimizer. Under the mask source, the true denoiser at a visible position is a point mass on its token, so the additional visible-position terms equal zero at the optimum. The pointwise minimizer also remains unchanged under any strictly positive time weight, because the loss decomposes into a separate divergence at each time and noisy sequence. We therefore recover the same denoiser by minimizing either loss [Sahoo et al., 2024, Gat et al., 2024]. The weight changes how we distribute training effort across noise levels. With the weight, times near $t=1$ dominate the gradient; without it, we train equally at every masking level.

5.10 Two sources and two samplers

We can sample with the two-part rule from Section 5.8:

1. Draw x from the source distribution p, and set $t=0$.
2. Predict $p_{1|t}^\theta(\cdot \mid x)$ at every position with one network call.
3. At each position independently, with probability $\lambda_t = h\dot\kappa_t/(1-\kappa_t)$, redraw the token from that position's prediction; otherwise keep it.
4. Set $t \leftarrow t+h$ and return to step 2 until $t=1$.

Each iteration uses one forward pass and N categorical draws. A run with $1/h$ steps therefore requires $1/h$ forward passes, independently of sequence length. If the redraw probability λ_t exceeds one near $t=1$, we can reduce the step size to $h \le (1-\kappa_t)/\dot\kappa_t$, or redraw every position on the final step, using the limit of the two-part rule at $\lambda_t = 1$. This is Algorithm 1 of Discrete Flow Matching [Gat et al., 2024] with the denoiser velocity.

5.10.1 Sampling from the mask source

At a visible position under the mask source, the true denoiser puts all of its probability on the visible token, so the bracket in the marginal velocity cancels:

$$u_t^i(\cdot, x) = \frac{\dot\kappa_t}{1-\kappa_t}\left[\delta_{x^i}(\cdot) - \delta_{x^i}(\cdot)\right] = 0.$$

"At a visible position i, the data-token posterior equals the point mass δ_{x^i} on the current token. Their difference is zero, so the marginal velocity is zero."

Once revealed, a token remains fixed for the rest of the run. At a masked position, we unmask with probability λ_t and draw the new token from the denoiser distribution over $\mathcal{V}$. Starting from all masks reproduces the carry-over unmasking of masked diffusion [Sahoo et al., 2024], using the first-order probability λ_t in place of the exact reveal probability $(\kappa_{t+h} - \kappa_t)/(1 - \kappa_t)$ derived in Chapter 4.

5.10.2 Sampling from the uniform source

With the uniform source, we cannot distinguish a data token from a random source token by observing its value alone. The denoiser assigns probability to all four bases, so the exit rate from the current token is nonzero. In Section 5.7.4, the posterior probability of the current token G is 0.2. With $\lambda_t = 0.2$, the replacement probability is

$$\lambda_t \left(1 - p_{1|t}(\mathrm{G} \mid x)\right) = 0.2 \times 0.8 = 0.16$$

in a single step. We can therefore revise an early token using the updated sequence context. With the mask source, each unmasking is permanent; with the uniform source, later replacements remain possible (Figure 5.3).

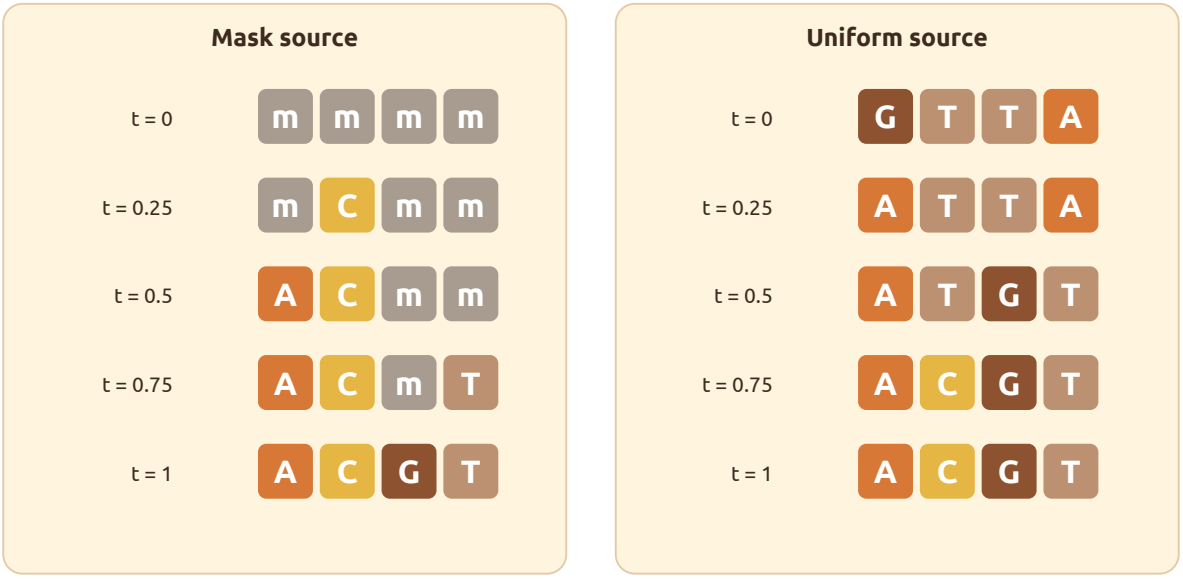


Figure 5.3: Four positions generated by the sampler of this section at $t = 0$, 0.25, 0.5, 0.75 and 1, ending at the same sequence ACGT. Each row is one time and each tile is the token at one position. Left, the mask source: every position starts at the mask symbol m (gray), a position is revealed at most once, and a revealed base keeps its token to the end. Right, the uniform source: every position starts at a random base, and any base can be redrawn at any step. The second position holds T from $t = 0$ through $t = 0.5$ and is revised to the data token C at $t = 0.75$.

Both runs end at the same sequence and use the same trained denoiser. The source distribution determines whether we sample each token once or allow repeated replacements. We choose the source before training, when defining the coupling.

5.11 Discrete flow-matching implementation

We can implement the source, path, loss and sampling step in a small class. Import `torch`, `torch.nn.functional as F`, and `Categorical` from `torch.distributions`.

```
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform
```

In the constructor, we store the vocabulary size `V`, which is 4 for DNA, and pass the schedule κ_t and its derivative $\dot{\kappa}_t$ as functions of a time tensor. The defaults are $\kappa_t = t$ and $\dot{\kappa}_t = 1$. We use `torch.ones_like` so that the derivative has the same shape as the batch of times. To change the schedule, pass two different functions; the rest of the class stays unchanged.

In `source`, we draw each token uniformly, giving the uniform source from Section 5.5.6. For a mask source, we would return a tensor filled with the mask index and increase `V` by one to include that symbol.

```
    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture
```

In `corrupt`, we sample the conditional path with the coin flips from Section 5.5.5. We draw a source sequence for each data sequence in the batch, giving the independent coupling. At every position, we compare an independent uniform random number with κ_t, so `heads` is true with probability κ_t. Using `[:, None]`, we broadcast each sequence's time across its positions. All positions within a sequence therefore share κ_t, and each sequence has its own value. With `torch.where`, we select the data token for heads and the source token for tails, producing a draw from $p_t(\cdot \mid x_0, x_1)$.

```
    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)
```

We compute the cross-entropy from Section 5.9, using the true data token as the target at every position. The network output `logits` has shape $[B, N, K]$, with one score per token per position. We transpose it to place the class dimension second, as required by `F.cross_entropy`. The loss contains no time weight or rate, and the network receives information about t only through the noisy input. Averaging over the batch and positions gives $\mathcal{L}(\theta)/N$, up to the batch's Monte Carlo error.

```
    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

For one sampling step, we apply the two-part rule from Section 5.8. We call the network once and apply softmax to its logits to obtain $p^{\theta}_{1|t}(\cdot \mid x)$ at every position. We then compute $\dot\kappa_t/(1-\kappa_t)$, clamping the denominator at 10^{-4} to keep the rate finite near $\kappa_t = 1$. At each position, we draw a Bernoulli with probability $\lambda_t = h\,\dot\kappa_t/(1-\kappa_t)$. If this value exceeds one, the comparison returns true everywhere and we redraw every position, including on the final step. Next, we sample a candidate from each position's denoiser. We keep the candidate where the Bernoulli is true and the current token elsewhere. The decorator `@torch.no_grad()` disables gradient tracking because we do not differentiate during sampling.

The implementation contains the source, path, loss and sampler in twenty-three lines. We sample the mixture path with `torch.where` in `corrupt`, compute denoiser probabilities with softmax in `step`, evaluate the schedule factor in `rate`, and apply the two-part update with the final `torch.where`.

5.11.1 Training and generation

We can use this class in training and generation loops.

```
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()
```

For each training step, we draw one time per sequence, so a batch contains several noise levels. We pass the corrupted batch and times to the network, compute cross-entropy against the true tokens, and take one optimizer step. Under the mask source, we could omit the time input because the denoiser is independent of t, by Proposition 5.3. The number of masked positions already contains the required information about the noise level.

```
@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

To generate samples, we draw B source sequences of length L, then take 64 equal steps from $t = 0$ to $t = 1$. At each iteration, we pass the current time t, broadcast across the batch, and step size $s - t$, equal to $1/64$ on this grid. Training does not fix the step size. We can therefore run the same network for 16 or 512 steps, balancing sample quality against forward-pass cost. We can also use a nonuniform grid with smaller steps near $t = 1$, where λ_t is largest, without retraining.

To change the schedule, we pass different functions to the constructor. To change the source or sampler, we edit `source` or `step`, respectively. The loss and training loop remain unchanged.

5.11.2 Discrete flow-matching construction

We generate the path by averaging the per-position mixture $(1-\kappa_t)\delta_{x_0^i} + \kappa_t \delta_{x_1^i}$ over a coupling. At each position, the velocity is the schedule factor times the difference between the denoiser and a

point mass on the current token. We train only the denoiser, $p_{1|t}(\cdot \mid x)$, by minimizing cross-entropy on the data tokens. Sampling then requires one network call and one Bernoulli per position at each step. These expressions follow from the master equation and the mixture path.

5.12 Discrete Flow Matching and the choice of schedule

Discrete Flow Matching [Gat et al., 2024] uses the mixture path, denoiser parameterization and Euler sampler. To combine token-level transitions into sequence transitions, the authors use the factorized velocity

$$u_t(y, x) = \sum_{i=1}^{N} \delta_{x^{\bar{i}}}(y^{\bar{i}})\; u_t^i(y^i, x),$$

in which $x^{\bar{i}}$ is every position of x except i, so the only jumps with positive rate are the ones that change a single position. A sampler that updates every position at once therefore runs a different process from the one this velocity describes, and the two agree only to first order in the step size.

5.12.1 Per-token jump composition

Let's take a two-position sequence $x = (\mathrm{G}, \mathrm{A})$ at $t = 0.5$ with the linear schedule, so the schedule factor $\dot{\kappa}_t/(1-\kappa_t)$ equals 2. Let the denoiser predict $(0.1, 0.6, 0.2, 0.1)$ over $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$ at the first position and $(0.3, 0.1, 0.5, 0.1)$ at the second. The two per-position velocities are

$$u_t^1(\cdot, x) = 2\big[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\big] = (0.2,\ 1.2,\ -1.6,\ 0.2),$$

$$u_t^2(\cdot, x) = 2\big[(0.3,\ 0.1,\ 0.5,\ 0.1) - (1,\ 0,\ 0,\ 0)\big] = (-1.4,\ 0.2,\ 1.0,\ 0.2).$$

With $h = 0.05$, the first position draws its next token from $(0.01,\ 0.06,\ 0.92,\ 0.01)$ and the second from $(0.93,\ 0.01,\ 0.05,\ 0.01)$, each summing to one. If we draw them independently and multiply, the sequence stays at (G, A) with probability $0.92 \times 0.93 = 0.8556$, while under the sequence velocity that probability is $1 + h\,[\,-1.6 + (-1.4)\,] = 1 - 0.15 = 0.85$. The gap is 0.0056, which is exactly $h^2 \times 1.6 \times 1.4$, the probability that both positions move in the same step. The parallel step also lands on the sequence (C, G) with probability $0.06 \times 0.05 = 0.0030$, where two positions changed at once, and that transition has rate zero under the sequence velocity. Again the discrepancy is second order, $h^2 \times 1.2 \times 1.0$. Every error introduced by updating all positions together is of this form, so it shrinks quadratically as the number of steps grows.

5.12.2 Sampling schedule selection

The training loss has no explicit schedule factor. We use κ_t to corrupt X_t and to compute sampling rates. With a mask source, the denoiser is independent of t, so we can choose any schedule after training a single network.

The marginal path p_t depends on time only through κ_t. Different schedules therefore follow the same curve of distributions but place uniform time steps at different points on that curve. With eight equal steps and $\kappa_t = t$, the grid is $\kappa = 0, 0.125, 0.250, \ldots, 1$. For the quadratic schedule $\kappa_t = t^2$, it is

$$\kappa = 0,\ 0.0156,\ 0.0625,\ 0.1406,\ 0.25,\ 0.3906,\ 0.5625,\ 0.7656,\ 1.$$

Table 5.2: The linear schedule and the quadratic schedule at four times. κ_t is the probability that a position already holds its data token, and $\dot\kappa_t/(1-\kappa_t)$ is the rate at which a position still holding its source token jumps. Both schedules satisfy $\kappa_0 = 0$ and $\kappa_1 = 1$, and both leave exactly one expected unmasking per position.

	$t = 0.25$	$t = 0.5$	$t = 0.75$	$t = 0.9$
$\kappa_t = t$, schedule	0.2500	0.5000	0.7500	0.9000
$\kappa_t = t$, jump rate	1.333	2.000	4.000	10.00
$\kappa_t = t^2$, schedule	0.0625	0.2500	0.5625	0.8100
$\kappa_t = t^2$, jump rate	0.5333	1.333	3.429	9.474

The first step advances by 0.0156 and the last by 0.2344. We compare both schedules and their jump rates at four times in Table 5.2.

A position still at its source token survives with probability $1-\kappa_t$ and jumps at rate $\dot\kappa_t/(1-\kappa_t)$, so the expected number of jumps it makes over the whole path is

$$\int_0^1 (1-\kappa_t)\,\frac{\dot\kappa_t}{1-\kappa_t}\,dt = \int_0^1 \dot\kappa_t\,dt = \kappa_1 - \kappa_0 = 1.$$

"The expected number of jumps per position is the time integral of the probability $1-\kappa_t$ of still being at the source token times the jump rate $\dot\kappa_t/(1-\kappa_t)$, which collapses to the integral of $\dot\kappa_t$ and therefore to one, for every schedule with $\kappa_0 = 0$ and $\kappa_1 = 1$."

The schedule determines when that single jump happens. The expected jump time is $\int_0^1 (1-\kappa_t)\,dt$, which is $1/2$ for the linear schedule and $2/3$ for the quadratic one, and the median jump time moves from 0.5 to $1/\sqrt{2} = 0.7071$. Under the quadratic schedule the sequence stays mostly masked through the first half of the path, and most of the revealing happens at the end. We can therefore train one denoiser and compare schedules at sampling time. With a 1.7B-parameter model trained on code, the authors solved 6.7% of HumanEval problems on the first try and 13.4% within ten tries [Gat et al., 2024] by filling in masked tokens.

We can also change the sampler because several velocities can satisfy the master equation for the same path.

5.13 Zero-flux velocities

The master equation is linear in the velocity. Suppose u_t generates p_t and a second velocity v_t has no net flux into any state. If we add the two, the right-hand side of the master equation is unchanged at every state, so the sum generates the same path:

$$\sum_x v_t(y,x)\,p_t(x) = 0 \ \text{ for every } y \quad\Longrightarrow\quad u_t + v_t \text{ also generates } p_t.$$

"If a velocity v_t has zero net flux into every state y, meaning the sum over x of $v_t(y,x)$ times $p_t(x)$ is zero, then the sum of v_t with a velocity u_t that generates p_t is another velocity that generates the same path."

The sum $u_t + v_t$ must still be a velocity, so its off-diagonal entries have to stay nonnegative. A term of this kind moves individual samples around without moving the marginals, and we can use it in a sampler to revisit positions that were filled in badly.

We could find v_t by solving a linear system at every time. An easier sufficient condition is detailed balance, introduced in Chapter 4. It requires equal flux in both directions along every edge:

$$v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y.$$

"The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states."

5.13.1 Derivation: detailed balance implies zero net flux

We can start from the net flux into y and replace every term using detailed balance, which lets us exchange the two arguments at the cost of exchanging the two probabilities:

$$\sum_x v_t(y, x)\, p_t(x) = \sum_x v_t(x, y)\, p_t(y) = p_t(y) \sum_x v_t(x, y) = 0.$$

"The net flux into y equals the sum over x of the reversed fluxes $v_t(x, y)p_t(y)$, which is $p_t(y)$ times the column of rates leaving y, and those rates sum to zero."

The final equality uses the zero column sum $\sum_x v_t(x, y) = 0$. As in Chapter 4, detailed balance makes p_t stationary for v_t, while u_t generates its time evolution.

5.13.2 A zero-flux velocity on three states

Let's build one by hand on the states $\{\mathrm{A}, \mathrm{C}, \mathrm{G}\}$ with $p_t = (0.2,\ 0.3,\ 0.5)$. We pick a rate $v_t(\mathrm{C}, \mathrm{A}) = 3$ from A to C, and detailed balance fixes the reverse rate at $v_t(\mathrm{A}, \mathrm{C}) = 3 \times 0.2/0.3 = 2$. Next, we pick $v_t(\mathrm{G}, \mathrm{A}) = 1$, and the reverse rate is $1 \times 0.2/0.5 = 0.4$. The edge between C and G stays unused. Filling in the diagonals so that each column sums to zero,

$$v_t = \begin{pmatrix} -4 & 2 & 0.4 \\ 3 & -2 & 0 \\ 1 & 0 & -0.4 \end{pmatrix},$$

with rows and columns ordered $(\mathrm{A}, \mathrm{C}, \mathrm{G})$ and the destination indexing the row. The net flux into A is $-4(0.2) + 2(0.3) + 0.4(0.5) = -0.8 + 0.6 + 0.2 = 0$, into C it is $3(0.2) - 2(0.3) = 0$, and into G it is $1(0.2) - 0.4(0.5) = 0$.

We now take a velocity that does move the path, u_t with every state jumping to G at rate 1, for which $\dot{p}_t = (-0.2,\ -0.3,\ 0.5)$. When we add the two, every off-diagonal entry is still nonnegative, and the sum

$$u_t + v_t = \begin{pmatrix} -5 & 2 & 0.4 \\ 3 & -3 & 0 \\ 2 & 1 & -0.4 \end{pmatrix}$$

produces $\dot{p}_t(\mathrm{A}) = -5(0.2) + 2(0.3) + 0.4(0.5) = -0.2$, $\dot{p}_t(\mathrm{C}) = 3(0.2) - 3(0.3) = -0.3$ and $\dot{p}_t(\mathrm{G}) = 2(0.2) + 1(0.3) - 0.4(0.5) = 0.5$, the same three numbers. The total rate of leaving A rose from 1 to 5, so a sample under $u_t + v_t$ moves five times as often while the distribution follows the identical curve. A corrector adds motion of exactly this kind.

5.14 The corrector

To construct v_t with detailed balance, we can use the symmetry between the mixture path's two endpoints. Running the path backward and replacing the denoiser with the source posterior $p_{0|t}$ gives

$$\check{u}_t^i(y^i, x) = \frac{\dot{\kappa}_t}{\kappa_t} \left[\delta_{x^i}(y^i) - p_{0|t}(y^i \mid x) \right].$$

"The backward velocity $\check{u}_t^i(y^i, x)$ equals the schedule derivative $\dot{\kappa}_t$ divided by the data weight κ_t, multiplied by the difference between the current-token indicator $\delta_{x^i}(y^i)$ and the source-token posterior $p_{0|t}(y^i \mid x)$."

- $\check{u}_t^i(y^i, x)$ is the backward velocity at position i, in units of probability per unit time. Its off-diagonal entries are negative, as they are for any jump process run backward in time.
- $p_{0|t}(\cdot \mid x)$ is the posterior over the source token at position i given the current sequence x, the mirror image of the denoiser $p_{1|t}$.
- $\dot{\kappa}_t/\kappa_t$ is the backward counterpart of the jump rate. It diverges as $t \to 0$, where $\kappa_t \to 0$, as the forward rate diverges at $t = 1$.

The two velocities have the same shape, with the source posterior in place of the denoiser, κ_t in place of $1 - \kappa_t$, and the two terms in the bracket exchanged. Let's write $\hat{u}_t$ for the forward velocity we have been using, and combine the two with weights:

$$\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1.$$

"The corrected velocity $\bar{u}_t$ is the forward velocity $\hat{u}_t$ weighted by α_t, minus the backward velocity $\check{u}_t$ weighted by β_t, with weights constrained by $\alpha_t - \beta_t = 1$."

The constraint $\alpha_t - \beta_t = 1$ lets us write $\bar{u}_t = \hat{u}_t + \beta_t(\hat{u}_t - \check{u}_t)$, which separates the part that moves the path from the part that does not. Each step then moves forward by $1 + \beta_t$ and backward by β_t, adding noise and removing it again.

5.14.1 Detailed balance for the velocity difference

We need to check that $\hat{u}_t - \check{u}_t$ satisfies detailed balance with the mixture path, which by Section 5.13.1 makes $\bar{u}_t$ generate p_t for any β_t. Consider one position, conditional on a pair (x_0, x_1) with $x_0 \neq x_1$, where the path puts mass $1 - \kappa_t$ on the source token and κ_t on the data token. Conditionally the source posterior is the point mass δ_{x_0} and the denoiser is the point mass δ_{x_1}, so the two velocities have one nonzero off-diagonal entry each:

$$\hat{u}_t(x_1, x_0) = \frac{\dot{\kappa}_t}{1 - \kappa_t}, \qquad \check{u}_t(x_0, x_1) = -\frac{\dot{\kappa}_t}{\kappa_t}, \qquad \hat{u}_t(x_0, x_1) = \check{u}_t(x_1, x_0) = 0.$$

Subtracting, the difference $v_t = \hat{u}_t - \check{u}_t$ has rate $\dot{\kappa}_t/(1 - \kappa_t)$ from x_0 to x_1 and rate $\dot{\kappa}_t/\kappa_t$ back the other way. Multiplying each by the probability of the state it leaves gives

$$v_t(x_1, x_0)\, p_t(x_0) = \frac{\dot{\kappa}_t}{1 - \kappa_t}\,(1 - \kappa_t) = \dot{\kappa}_t, \qquad v_t(x_0, x_1)\, p_t(x_1) = \frac{\dot{\kappa}_t}{\kappa_t}\,\kappa_t = \dot{\kappa}_t.$$

"The flux from the source token to the data token equals $\dot{\kappa}_t$, and the flux from the data token back to the source token equals $\dot{\kappa}_t$ as well, so the two match and detailed balance holds along the single edge the conditional path uses."

The two fluxes are equal at every time and for every schedule, so the extra term adds $\dot{\kappa}_t$ of circulation along the edge in each direction without changing p_t. If we average over the coupling the equality is preserved, because both sides are linear in the conditional path.

5.14.2 The corrector for the mask source

We take the linear schedule $\kappa_t = t$, the weights $\alpha_t = 1 + \eta t$ and $\beta_t = \eta t$, and the mask source, where the source posterior $p_{0|t}$ is the point mass on the mask token m. With $\beta_t = \eta t$ the backward term stays finite, since $\breve{u}_t$ has the factor $\dot{\kappa}_t/\kappa_t = 1/t$, which diverges at $t = 0$, and multiplying by ηt cancels it. Substituting both velocities,

$$\boxed{\bar{u}_t^i(\cdot, x) = \frac{1 + \eta t}{1 - t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot)\right] + \eta \left[\delta_m(\cdot) - \delta_{x^i}(\cdot)\right]}$$

"The corrected velocity $\bar{u}_t^i$ multiplies the difference between the data-token posterior $p_{1|t}$ and current-token point mass δ_{x^i} by $(1+\eta t)/(1-t)$, then adds the remasking rate η times the difference between the mask point mass δ_m and δ_{x^i}."

- $\eta \geq 0$ is the corrector strength, in units of inverse time. At $\eta = 0$ the second bracket vanishes and $\bar{u}_t$ is the marginal velocity built earlier in this chapter.
- $\delta_m(\cdot)$ is the point mass on the mask token, so the second bracket is a rate η from the current token back to the mask.

At a masked position, $\delta_{x^i} = \delta_m$, the second bracket is zero, and the position unmasks at total rate $(1 + \eta t)/(1 - t)$, split over tokens by the denoiser. At a visible position the true denoiser is the point mass on the visible token, the first bracket is zero, and the position remasks at rate η. This is the stochastic sampler of discrete flow models [Campbell et al., 2024], and η is the amount of stochasticity it adds.

5.14.3 Counting the jumps

At $t = 0.4$ with $\eta = 2$, a masked position unmasks at rate $(1 + 2 \times 0.4)/(1 - 0.4) = 1.8/0.6 = 3.0$, and a visible position remasks at rate 2. To calculate the resulting probability change, write $M(t)$ for the probability that the position is masked at time t and combine the incoming and outgoing fluxes:

$$\dot{M}(t) = -M(t)\,\frac{1 + \eta t}{1 - t} + \big(1 - M(t)\big)\,\eta.$$

Substituting the solution we expect, $M(t) = 1 - t$, the first term becomes $-(1-t)(1+\eta t)/(1-t) = -(1 + \eta t)$ and the second becomes $t\eta$, so

$$\dot{M}(t) = -(1 + \eta t) + \eta t = -1,$$

which is the derivative of $1 - t$ for every value of η. The marginal fraction of masked positions is $1 - t$ for any corrector strength. At one position, detailed balance reduces to this cancellation.

Integrating the unmasking rate against the probability of being masked, and the remasking rate against the probability of being visible,

$$\mathbb{E}[\text{unmaskings}] = \int_0^1 (1-t)\,\frac{1+\eta t}{1-t}\,dt = 1+\frac{\eta}{2}, \qquad \mathbb{E}[\text{remaskings}] = \int_0^1 t\,\eta\,dt = \frac{\eta}{2},$$

so that

$$\eta = 0,\ 2,\ 10: \qquad \mathbb{E}[\text{jumps per position}] = 1+\eta = 1,\ 3,\ 11.$$

"The expected jump count at each position is $1+\eta$, where η is the corrector strength, giving 1, 3, and 11 jumps for $\eta = 0$, 2, and 10."

At $\eta = 0$, each position unmasks once and remains fixed, matching Section 5.12.2. At $\eta = 10$, it unmasks six times and remasks five times on average. These repeated updates let us replace tokens when the sequence context changes. We can check both jump counts and marginals by simulating 200,000 independent positions at each strength. For $\eta = 0, 2, 10$, the masked fractions at $t = 0.5$ are 0.4987, 0.4996, and 0.5002. The corresponding mean jump counts are 1.000, 3.002, and 10.993. Individual paths can therefore differ at every intermediate token while retaining the same marginal at every time (Figure 5.4).

For a single Euler step, fix $t = 0.4$, a step of $h = 0.01$, and the denoiser $(0.1,\ 0.6,\ 0.2,\ 0.1)$ over $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$ at a masked position. The unmasking probability in that step is $h(1+\eta t)/(1-t)$, which splits over the four bases in proportion to the denoiser:

$$\begin{aligned}
\eta = 0: &\quad (0.00167,\ 0.01000,\ 0.00333,\ 0.00167), &&\text{stay masked } 0.98333,\\
\eta = 2: &\quad (0.00300,\ 0.01800,\ 0.00600,\ 0.00300), &&\text{stay masked } 0.97000,\\
\eta = 10: &\quad (0.00833,\ 0.05000,\ 0.01667,\ 0.00833), &&\text{stay masked } 0.91667.
\end{aligned}$$

Each row sums to one. A visible position remasks with probability ηh, equal to 0, 0.02, and 0.10 for the three settings. Raising η from 0 to 10 increases the per-step unmasking probability fivefold. It also gives a remasking probability of one in ten. The masked fraction at $t = 0.4$ remains 0.6 in all three runs.

5.14.4 The parallel-step error

A corrector can reduce errors from independent position updates in the parallel sampler of Section 5.12.1. Consider data uniform over AA and TT, starting from two masks. With both positions masked, each denoiser is $(0.5, 0.5)$ over $\{\mathrm{A}, \mathrm{T}\}$. Once either position is visible, the other position's posterior is a point mass on that same base.

If we reveal both positions in one step, we draw them independently from $(0.5, 0.5)$. AA, AT, TA, and TT each occur with probability 0.25, so half the outputs fall outside the data. If we reveal one position first, its token determines the posterior at the remaining position. The second draw then gives AA or TT with certainty. Under the linear schedule, each reveal time is uniform on $[0, 1]$. On a uniform grid with $1/h$ steps, the probability that both reveal times fall in the same step is

$$\sum_{k=1}^{1/h} \mathbb{P}(T_1 \in \text{cell } k)\,\mathbb{P}(T_2 \in \text{cell } k) = \frac{1}{h}\cdot h^2 = h,$$

which is $1/64 = 0.0156$ at 64 steps. Half of those draws are wrong, for an error rate of 0.78% at $\eta = 0$, and the only way to lower it is to take more steps.

With $\eta > 0$, we can correct a pair revealed jointly at time τ. Each of the two positions returns to the mask at rate η, so the first remasking of either happens at rate 2η and the pair survives untouched to $t = 1$ with probability $e^{-2\eta(1-\tau)}$. Whichever position remasks first is then redrawn with the other one visible, where the denoiser is a point mass, and the pair lands back on AA or TT. For a joint reveal at the midpoint $\tau = 0.5$, that survival probability is 0.135 at $\eta = 2$ and 4.54×10^{-5} at $\eta = 10$. This improvement requires extra network calls, whose cost we can choose after training without changing the grid.

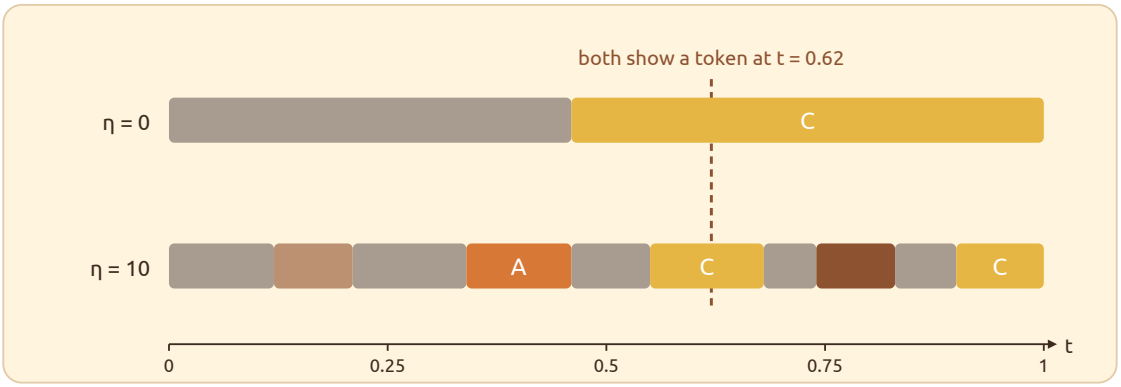


Figure 5.4: One position sampled from the mask source under the corrector of Section 5.14.2, at $\eta = 0$ (top) and $\eta = 10$ (bottom), with time running left to right from $t = 0$ to $t = 1$. Gray marks a masked position and a color marks a token, labeled with its base where the stretch is wide enough. At $\eta = 0$ the position unmasks once, near $t = 0.46$, and keeps C to the end. At $\eta = 10$ it makes nine jumps, five unmaskings and four remaskings, passing through A before settling on C. The dashed line at $t = 0.62$ marks a time at which both paths hold a token; at every time the probability of being masked is $1 - t$ for both.

An Euler step is valid as long as the redraw probability stays below one, so with the corrector we need $h \le (1-t)/(1+\eta t)$. At $t = 0.9$ this allows $h \le 0.1$ at $\eta = 0$, $h \le 0.0357$ at $\eta = 2$, and $h \le 0.01$ at $\eta = 10$. Near the end of the path the admissible step shrinks by roughly a factor of $1 + \eta$, so a corrector that makes eleven jumps per position also runs about eleven times as many steps there.

5.15 Joint structure and sequence generation

In discrete flow models [Campbell et al., 2024], we can derive the corrector from a general conditional path $p_t(\cdot \mid x_1)$. Solving the master equation for a velocity gives

$$u_t^\star(y, x \mid x_1) = \frac{\big[\, \dot{p}_t(y \mid x_1) - \dot{p}_t(x \mid x_1) \,\big]_+}{Z_t \; p_t(x \mid x_1)}, \qquad y \neq x.$$

"The conditional rate $u_t^\star(y, x \mid x_1)$ from x to y takes the positive part of the destination growth $\dot{p}_t(y \mid x_1)$ minus the source growth $\dot{p}_t(x \mid x_1)$, then divides by the product of the support size Z_t and current-state probability $p_t(x \mid x_1)$."

- $[\, a \,]_+ = \max(a, 0)$ keeps only the positive part, so that the rate stays admissible.

- Z_t counts the states with positive probability at time t, so the flux out of x is shared among them.

Probability flows from x to y only when the required growth of $p_t(y)$ exceeds that of $p_t(x)$. Dividing by the probability at x converts the required probability flux into a rate per sample.

Consider the mask path at one position with data token C, so $p_t(m \mid \mathrm{C}) = 1 - t$ and $p_t(\mathrm{C} \mid \mathrm{C}) = t$, the two states with nonzero probability, and $Z_t = 2$. Their derivatives are -1 and $+1$. From $x = m$ to $y = \mathrm{C}$,

$$u_t^\star(\mathrm{C}, m) = \frac{[\,1 - (-1)\,]_+}{2\,(1-t)} = \frac{2}{2(1-t)} = \frac{1}{1-t},$$

which is the mixture velocity built earlier in this chapter, with the linear schedule, equal to 2 at $t = 0.5$ and 10 at $t = 0.9$. In the other direction the bracket is $[-1 - 1]_+ = 0$, so no probability flows from C back to the mask. For the mask source and the uniform source, $u_t^\star$ coincides with the mixture velocity we derived, and among all velocities that generate the path it is the one with the fewest expected jumps: one per masked position.

5.15.1 Multiflow: two modalities on one path

The authors of Multiflow apply this construction to proteins [Campbell et al., 2024]. Each residue has a spatial position, an orientation and an amino acid, so the protein state has both continuous and discrete components. We use continuous flows for positions and orientations, as in Chapter 2, and masked jumps for amino acids. The conditional path factorizes over residues and modalities,

$$p_t(x \mid x_0, x_1) = \prod_{i=1}^{N} p_t\big(\mathrm{pos}^i \mid \cdot\big)\, p_t\big(\mathrm{rot}^i \mid \cdot\big)\, p_t\big(\mathrm{aa}^i \mid \cdot\big),$$

while every velocity and rate depends on the whole noisy protein. This factorization keeps the conditional path tractable. Predicting rates from the whole state lets us account for backbone geometry when assigning a hydrophobic residue.

We change one residue and one modality at a time. For the sequence component, we sum per-residue amino-acid velocities with their own schedule factors. We also use continuous velocities for positions and orientations. The amino-acid velocity at each residue is

$$u_t^i(\cdot, x) = \frac{\dot\kappa_{t_{\mathrm{seq}}}}{1 - \kappa_{t_{\mathrm{seq}}}}\,\big[\,p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot)\,\big],$$

with x the whole noisy protein, including its backbone. We therefore predict the twenty amino-acid probabilities at residue i using both coordinates and residue identities.

With separate times t_{str} and t_{seq}, we can use one trained model for several tasks. To design a sequence for a fixed backbone, set $t_{\mathrm{str}} = 1$ and advance the sequence clock. To fold a fixed sequence, set $t_{\mathrm{seq}} = 1$ and advance the structure clock. Advancing both generates a protein from noise. We keep the same training loss because we draw the clocks independently during training, and the amino-acid cross-entropy is unaffected by the other clock.

In Multiflow, we sample amino acids with corrector strength $\eta = 20$. At $t_{\mathrm{seq}} = 0.5$, the total unmasking rate is $(1 + 20 \times 0.5)/(1 - 0.5) = 11/0.5 = 22$. If the denoiser assigns leucine probability 0.4, the rate of unmasking to leucine is $22 \times 0.4 = 8.8$. A visible amino acid remasks at rate 20.

With step size $h = 0.005$, the masked residue becomes leucine with probability $0.005 \times 8.8 = 0.044$. It remains masked with probability $1 - 0.005 \times 22 = 0.89$, and a visible residue remasks with probability $0.005 \times 20 = 0.10$. Each residue undergoes $1 + \eta = 21$ updates on average. At $t = 0.9$, the step-size constraint is $h \leq 0.1/(1 + 20 \times 0.9) = 0.00526$. The extra updates therefore require a finer grid near the endpoint.

We can also choose the probability path by minimizing a cost.

5.16 Kinetic-optimal paths

Several velocities generate a given path, and several paths connect the same endpoints. We can select among them by minimizing a cost expressed in probability fluxes. This gives a convex optimization problem.

5.16.1 The flux and its energy

The flux along an edge is the velocity along it times the probability of the state it leaves:

$$j_t(y, x) = u_t(y, x)\, p_t(x).$$

"The flux $j_t(y, x)$ from x to y equals the velocity $u_t(y, x)$ times the probability $p_t(x)$ of the current state."

We can write the master equation as $\dot{p}_t(y) = \sum_{x \neq y} [\, j_t(y, x) - j_t(x, y)\,]$, which is linear in the flux. The master equation and off-diagonal nonnegativity are linear constraints on j. Minimizing a convex cost in j under these constraints is therefore a convex problem. Define the energy

$$\min_j \int_0^1 \sum_{y \neq x} \frac{w_t(y, x)}{p_t(x)}\, j_t(y, x)^2\, dt \quad \text{subject to the master equation and } j \geq 0.$$

"We choose nonnegative fluxes $j_t(y, x)$ satisfying the master equation to minimize their squared magnitudes, weighted by the edge cost $w_t(y, x)$ divided by the current-state probability $p_t(x)$, summed over edges and integrated over time."

- $w_t(y, x) > 0$ is a weight on the edge from x to y, chosen by us.
- $p_t(x)$ in the denominator makes the cost quadratic in the velocity when it is rewritten, since $j^2/p = u^2 p$.

In velocity form, the cost is $\sum_{y \neq x} w_t(y, x)\, u_t(y, x)^2\, p_t(x)$, the expected squared rate under p_t. At $w \equiv 1$, it is the discrete counterpart of the expected squared speed in continuous optimal transport. We call the resulting paths kinetic-optimal. With linear constraints and a convex cost, we obtain a closed-form minimizer.

5.16.2 Derivation: the velocity of least energy

Fix a time and minimize edge by edge, with a multiplier $\varphi_t(y)$ for the master equation at each state y. The Lagrangian is

$$\sum_{y\neq x} \frac{w_t(y,x)}{p_t(x)}\, j_t(y,x)^2 \;-\; \sum_y \varphi_t(y)\Big[\sum_{x\neq y}\big(j_t(y,x)-j_t(x,y)\big) - \dot p_t(y)\Big].$$

Let's differentiate with respect to $j_t(y,x)$. This edge has sign $+1$ in the constraint at y and sign -1 in the constraint at x, giving the stationarity condition

$$\frac{2\,w_t(y,x)}{p_t(x)}\, j_t(y,x) = \varphi_t(y) - \varphi_t(x),$$

and the nonnegativity constraint turns the solution into its positive part. Absorbing the factor of two into φ,

$$j_t(y,x) = \frac{p_t(x)}{w_t(y,x)}\,\big[\varphi_t(y)-\varphi_t(x)\big]_+.$$

Choosing the weight $w_t(y,x) = 1/p_t(y)$ makes the cost symmetric in the two endpoints of the edge, and the flux becomes $j_t(y,x) = p_t(y)\,p_t(x)\,[\varphi_t(y)-\varphi_t(x)]_+$. Inserting this into the master equation and using $a_+ - (-a)_+ = a$ to collapse the two positive parts, we get

$$\begin{aligned}\sum_{x\neq y}\big[j_t(y,x)-j_t(x,y)\big] &= p_t(y)\sum_{x\neq y} p_t(x)\,\big[\varphi_t(y)-\varphi_t(x)\big]\\ &= p_t(y)\Big[\varphi_t(y) - \sum_x p_t(x)\,\varphi_t(x)\Big].\end{aligned}$$

Set $\varphi_t(x) = \frac{d}{dt}\log p_t(x)$, the logarithmic probability growth rate. The average term vanishes because $\sum_x p_t(x)\,\dot p_t(x)/p_t(x) = \sum_x \dot p_t(x) = 0$, leaving $p_t(y)\,\varphi_t(y) = \dot p_t(y)$, as required by the master equation. Divide the flux by $p_t(x)$ to obtain the velocity:

$$\boxed{u_t^\star(y,x) = p_t(y)\,\Big[\,\frac{d}{dt}\log p_t(y) - \frac{d}{dt}\log p_t(x)\,\Big]_+}$$

"The kinetic-optimal jump rate $u_t^\star(y,x)$ equals the destination probability $p_t(y)$ times the positive part of the difference between the logarithmic probability growth rates at destination y and current state x."

Probability moves from x to y only when y is growing faster, relative to its own size, than x. The rate also vanishes when $p_t(x)$ or $p_t(y)$ is zero, so the formula stays finite on paths with empty states, which every mask path has.

5.16.3 Kinetic optimality of the mixture path

Let's apply the formula to the conditional mixture path at one position, where $p_t(x_0) = 1-\kappa_t$ and $p_t(x_1) = \kappa_t$. The two growth rates are $\dot\kappa_t/\kappa_t$ at the data token and $-\dot\kappa_t/(1-\kappa_t)$ at the source token, so

$$u_t^\star(x_1,x_0) = \kappa_t\Big[\frac{\dot\kappa_t}{\kappa_t} + \frac{\dot\kappa_t}{1-\kappa_t}\Big]_+ = \dot\kappa_t\Big[1+\frac{\kappa_t}{1-\kappa_t}\Big] = \frac{\dot\kappa_t}{1-\kappa_t},$$

which is the conditional velocity built earlier in this chapter, and the reverse rate $u_t^\star(x_0, x_1)$ has a negative bracket and is zero. The marginal path behaves the same way. We take the mask source at $t = 0.5$ with $\kappa_t = t$, and a data marginal $(0.1, 0.6, 0.2, 0.1)$ over $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$ at this position. The marginal path is

$$p_t = (\,0.5,\ 0.05,\ 0.30,\ 0.10,\ 0.05\,) \quad \text{for } (m, \mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}),$$

whose growth rates are $\dot{p}_t(m)/p_t(m) = -1/0.5 = -2$ at the mask and $\dot{\kappa}_t/\kappa_t = 2$ at every token. The rate from the mask to C is then $0.30 \times [\,2 - (-2)\,]_+ = 0.30 \times 4 = 1.2$, matching $\dot{\kappa}_t/(1-\kappa_t) \times p_{1|t}(\mathrm{C}) = 2 \times 0.6 = 1.2$. Between two visible tokens the growth rates are equal, the bracket is zero, and no probability moves, which is the no-revision behavior of the mask path. The mixture velocity we have been using all along is the minimum-energy velocity for the path it generates, for every source [Shaul et al., 2025].

5.16.4 A distance on the tokens

We can apply the formula for $u_t^\star$ to any path. Choose a distance d on the token set, then concentrate the conditional path on x_1 by cooling a softmax of negative distances:

$$p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty.$$

"The conditional probability of token y is proportional to the exponential of minus the inverse temperature β_t times its distance $d(y, x_1)$ to the data token x_1, with β_t increasing from zero to infinity."

- $d(y, x_1) \geq 0$ is the distance from the token y to the data token x_1, with $d(x_1, x_1) = 0$.
- β_t is the concentration of the path, an inverse temperature. This parameter is distinct from the corrector weight in Section 5.14, which uses the same symbol.

At $\beta_0 = 0$ the path is uniform over tokens, which makes the source the uniform distribution, and at $\beta_1 = \infty$ all the mass is on x_1. Substituting into the kinetic-optimal formula, the normalizing constant of the softmax cancels between the two logarithms, because $\log p_t(y \mid x_1) = -\beta_t d(y, x_1) - \log Z_t$ and the $\log Z_t$ term is common to both states:

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot{\beta}_t\ [\,d(x, x_1) - d(y, x_1)\,]_+.$$

"The kinetic-optimal rate $u_t^\star(y, x \mid x_1)$ multiplies the destination probability $p_t(y \mid x_1)$ by the inverse-temperature derivative $\dot{\beta}_t$ and the positive part of the distance reduction $d(x, x_1) - d(y, x_1)$ toward data token x_1."

Probability flows only toward tokens strictly closer to x_1, with higher rates for larger reductions in distance. Through the chosen distance, we can distinguish small token errors from large ones.

5.16.5 A worked metric path for DNA

We can define distances between DNA bases using their chemical classes. A transition exchanges purines A and G or pyrimidines C and T; a transversion crosses between these classes. Transitions are more common, so assign $d = 1$ to transitions and $d = 2$ to transversions. For data token $x_1 = \mathrm{G}$, the distance vector in order $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$ is $d(\cdot, \mathrm{G}) = (1, 2, 0, 2)$.

We now take $\beta_t = t/(1-t)$, which satisfies $\beta_0 = 0$ and $\beta_1 = \infty$, and evaluate at $t = 0.5$, where $\beta = 1$. The softmax of $(-1, -2, 0, -2)$ has unnormalized weights $e^{-1} = 0.3679$, $e^{-2} = 0.1353$, $e^{0} = 1$ and $e^{-2} = 0.1353$, summing to 1.6386, so

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826).$$

"With $\beta = 1$, the softmax of negative distances produces probabilities 0.2245 for A, 0.0826 for C, 0.6103 for G, and 0.0826 for T."

A has almost three times the probability of C or T at this time, because A is one transition from G while C and T are two steps away. The rate $\dot\beta_t = 1/(1-t)^2$ equals 4 at $t = 0.5$, and from a current token C the two allowed destinations are A, at a distance gain of $2 - 1 = 1$, and G, at a gain of $2 - 0 = 2$:

$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2-1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882.$$

"The rate from C to A is 0.2245 times 4 times the distance gain of one, or 0.898, and the rate from C to G is 0.6103 times 4 times the gain of two, or 4.882."

The total rate out of C is $0.898 + 4.882 = 5.780$, so one Euler step of size $h = 0.05$ from C lands on $(0.0449,\ 0.7110,\ 0.2441,\ 0)$ over the four bases, which sums to one. The reverse rate $u_t^\star(\mathrm{C}, \mathrm{A})$ has the bracket $[\,1 - 2\,]_+ = 0$, so probability never moves away from G. A pyrimidine can reach G directly or by way of the purine A, at rate $u_t^\star(\mathrm{G}, \mathrm{A}) = 0.6103 \times 4 \times 1 = 2.441$ (Figure 5.5).

If we collect all the rates into a matrix, we can check the master equation on this path. Ordering the bases $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$ with the destination indexing the row,

$$u_t^\star = \begin{pmatrix} -2.441 & 0.898 & 0 & 0.898 \\ 0 & -5.780 & 0 & 0 \\ 2.441 & 4.882 & 0 & 4.882 \\ 0 & 0 & 0 & -5.780 \end{pmatrix},$$

and multiplying it by $p_t = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$ produces

$$\dot p_t = (-0.3997,\ -0.4774,\ 1.3546,\ -0.4774),$$

which sums to zero. Differentiating the softmax path directly at $t = 0.5$, with $\beta_t = t/(1-t)$, we get the same four numbers, so this velocity generates this path. The largest entry is the 1.3546 flowing into G, and the two pyrimidines lose mass fastest because each has two outgoing transitions.

5.16.6 Path optimization

So far, we minimized energy over velocities while holding the path fixed. We can also optimize the path while keeping its endpoints p and q fixed. At the optimal velocity, the energy depends only on the square roots of the path probabilities:

$$\int_0^1 \sum_x \frac{\dot p_t(x)^2}{p_t(x)}\, dt = 4 \int_0^1 \sum_x \Big(\frac{d}{dt}\sqrt{p_t(x)}\Big)^2 dt.$$

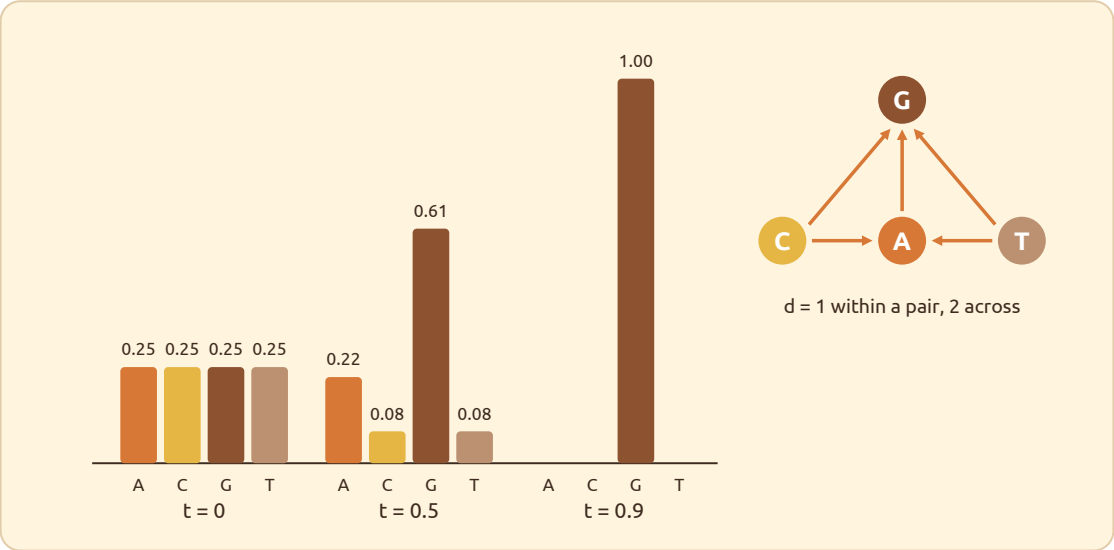


Figure 5.5: The metric path toward the data token G with $\beta_t = t/(1-t)$ and the transition distance $d(\cdot, \mathrm{G}) = (1, 2, 0, 2)$. The bars are the probability of each base at $t = 0$, $t = 0.5$ and $t = 0.9$, where β equals 0, 1 and 9. At $t = 0$ the path is uniform; at $t = 0.5$ the purine A holds 0.22 against 0.08 for each pyrimidine; by $t = 0.9$ the probability on G is 1.00 to two decimals. The graph at right draws the edges with positive rate, which run only toward bases closer to G.

"The path energy integrates the squared probability changes $\dot{p}_t(x)^2$, divided by the state probabilities $p_t(x)$ and summed over states x. It equals four times the integrated squared speed of the square-root probability vector $\sqrt{p_t}$."

Apply the chain rule to get $\frac{d}{dt}\sqrt{p_t} = \dot{p}_t/(2\sqrt{p_t})$, then square to obtain $\dot{p}_t^2/(4p_t)$. Since $\sum_x p_t(x) = 1$, the vector $\sqrt{p_t}$ has unit length and lies on the unit sphere. The energy measures its squared speed along that sphere. The least-energy curve between two sphere points is the great-circle arc.

For a target that is a point mass on x_1, reading the arc back as a probability path produces a mixture path whose schedule depends on the data token:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\left((1-t)\,\Omega\right)}{\sin^2 \Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}.$$

"The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1."

- $\Omega \in (0, \pi/2]$ is the angle between the source point $\sqrt{p}$ and the target corner $\sqrt{\delta_{x_1}}$ on the sphere, and $\cos\Omega = \sqrt{p(x_1)}$ is their inner product.

At $t = 0$ the fraction is $\sin^2\Omega/\sin^2\Omega = 1$ and $\kappa_0 = 0$, and at $t = 1$ the numerator is zero and $\kappa_1 = 1$. A token already likely under the source is at a small angle from the target, and its schedule rises early. With a uniform source over the four bases, $p(x_1) = 0.25$, so $\Omega = \arccos(0.5) = \pi/3$ and $\kappa_{0.5} = 1 - \sin^2(\pi/6)/\sin^2(\pi/3) = 1 - 0.25/0.75 = 0.6667$, ahead of the linear schedule at the same time. With the mask source the data token has source probability zero, $\Omega = \pi/2$, and the schedule

collapses to $\kappa_t = 1 - \sin^2((1-t)\pi/2) = \sin^2(\pi t/2)$, which equals 0.1464 at $t = 0.25$ and 0.8536 at $t = 0.75$ against 0.25 and 0.75 for the linear schedule. Its jump rate is $\dot\kappa_t/(1-\kappa_t) = \pi \tan(\pi t/2)$, which is 1.301 at $t = 0.25$ and 19.84 at $t = 0.9$, so the geodesic schedule has a lower initial jump rate and a higher final jump rate than the linear one.

5.16.7 The energy of two schedules

Let's calculate the energy of both schedules. For a two-state conditional mixture path, the probabilities are $1-\kappa_t$ and κ_t, with derivatives $-\dot\kappa_t$ and $\dot\kappa_t$. The energy from Section 5.16.6 is

$$\int_0^1 \sum_x \frac{\dot p_t(x)^2}{p_t(x)}\,dt = \int_0^1 \dot\kappa_t^2 \Big[\frac{1}{1-\kappa_t} + \frac{1}{\kappa_t}\Big] dt = \int_0^1 \frac{\dot\kappa_t^2}{\kappa_t\,(1-\kappa_t)}\,dt.$$

For the linear schedule the integrand is $1/(t(1-t))$, which has a non-integrable singularity at each endpoint, so the energy is infinite. With the integral truncated at 10^{-6} from each end the energy comes to 27.63, and tightening the truncation to 10^{-9} raises it to 41.45, consistent with logarithmic divergence. For the geodesic schedule $\kappa_t = \sin^2(\pi t/2)$, the numerator is $\dot\kappa_t^2 = (\pi/2)^2 \sin^2(\pi t)$ and the denominator is $\sin^2(\pi t/2)\cos^2(\pi t/2) = \frac{1}{4}\sin^2(\pi t)$, so the integrand is the constant π^2 and the energy is

$$\int_0^1 \pi^2\,dt = \pi^2 = 9.8696 = 4\,\Omega^2 \quad \text{with} \quad \Omega = \frac{\pi}{2},$$

matching the squared arc length of a great circle traversed at constant speed, exactly as in the square-root picture. The linear schedule traverses the same quarter circle on the sphere and differs only in its speed along it, which is infinite at both endpoints. Energy measures that speed, so the two schedules reach the same data through the same marginals at different costs.

5.16.8 A likelihood bound for any mixture path

Energy measures the cost of a path. To assess how well a trained model fits the data, we need its likelihood. The kinetic-optimal-path authors derive an evidence lower bound for any path and velocity, with a closed form for mixture paths. For the mask source, with one schedule shared by all tokens, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot\kappa_t}{1-\kappa_t}\,\mathbb{E}_{x_t}\Big[\sum_{i:\,x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t)\Big]\,dt.$$

"The model log-likelihood $\log p_1^\theta(x_1)$ is bounded below by the time integral of the schedule weight $\dot\kappa_t/(1-\kappa_t)$ times the expected log probabilities $p_{1|t}^\theta(x_1^i \mid x_t)$ of true data tokens, summed over masked positions i and averaged over noisy sequences x_t."

This is the MDLM bound from Chapter 4 with time reversed. Weighted cross-entropy over masked positions therefore bounds the likelihood. The unweighted loss has the same minimizer because a positive time weight does not change the optimal denoiser.

On a single position the schedule cancels out of the bound. A position is masked with probability $1-\kappa_t$ and has weight $\dot\kappa_t/(1-\kappa_t)$, so the product of the two is $\dot\kappa_t$ and the time integral is $\int_0^1 \dot\kappa_t\,dt = 1$ for every schedule. With the linear schedule the weight is $1/(1-t)$ and the masking probability is

$1 - t$, and their product is the constant 1. If the denoiser assigns the true base G a probability of 0.6 at every time, the bound comes to

$$\int_0^1 1 \cdot \log 0.6 \; dt = \log 0.6 = -0.511,$$

the negative cross-entropy at this position. We obtain the same value with $\kappa_t = t^2$ or any other admissible schedule, so this bound is independent of the training schedule.

The authors developed metric paths for image tokens, using distances from the tokenizer codebook [Shaul et al., 2025]. On face-blurred ImageNet at 256×256, they obtained FID 4.50 with a metric path in 100 sampling steps, compared with 5.72 for the masked path at the same sampling budget. We can thus choose the probability path as a model component, just as we chose the corruption process in Chapter 4.

With substitution-only jumps, p_t remains on sequences of a fixed length for all t, so we must choose that length before sampling. DNA, proteins, molecules and code vary in length, making fixed-length sampling restrictive.

5.17 Edit Flows with variable length

In Edit Flows we remove the constraint by enlarging the set of moves [Havasi et al., 2025]. At each position i of the current sequence x there are three edits,

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a),$$

where an insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a. The destination of a jump is now named by the edit applied to x, so we write the velocity as $u_t(\cdot \mid x)$ with the destination on the left of the bar. It is the same quantity as $u_t(y, x)$ with y the edited sequence. Each rate splits into an intensity and a token distribution:

$$\begin{aligned} u_t(\mathrm{ins}(x, i, a) \mid x) &= \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \\ u_t(\mathrm{del}(x, i) \mid x) &= \lambda^{\mathrm{del}}_{t,i}(x), \\ u_t(\mathrm{sub}(x, i, a) \mid x) &= \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x). \end{aligned}$$

"Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$."

- $\lambda^{\bullet}_{t,i}(x) \geq 0$ is the total rate of that kind of edit at position i, in units of edits per unit time, and it depends on the whole current sequence.
- $Q^{\bullet}_{t,i}(\cdot \mid x)$ is a distribution over the alphabet, so splitting the rate this way separates how often an edit happens from which token it writes.

For each position, we predict three intensities and two token distributions, since a deletion needs no token distribution. For an alphabet of size K, this requires $3 + 2K$ outputs per position. The diagonal entry is minus the sum of all intensities, as required for a velocity. Compared with the denoiser's K outputs per position, variable length increases the output-head size by a constant factor.

5.17.1 Alignments and the column-wise path

The mixture path pairs position i of x_0 with position i of x_1. For sequences of different lengths, we need an alignment to identify corresponding positions. We pad both sequences with a blank symbol ε until they have the same number of columns. For example, pad $x_0 = \text{ACGT}$ and $x_1 = \text{AGAC}$ to five columns as

$$z_0 = (\text{A},\ \text{C},\ \text{G},\ \text{T},\ \varepsilon), \qquad z_1 = (\text{A},\ \varepsilon,\ \text{G},\ \text{A},\ \text{C}).$$

A blank in the target column is a deletion, as in column 2, where C disappears. Different tokens in one column are a substitution, as in column 4, where T becomes A. A blank in the source column is an insertion, as in column 5, where C appears. Columns 1 and 3 agree, so there is no edit to make.

On the aligned pair, the mixture construction of this chapter applies without modification. Each column follows the mixture path independently, holding its source symbol with probability $1 - \kappa_t$ and its target symbol with probability κ_t. Removing the blanks from the aligned state z_t gives the input sequence x_t (Figure 5.6).

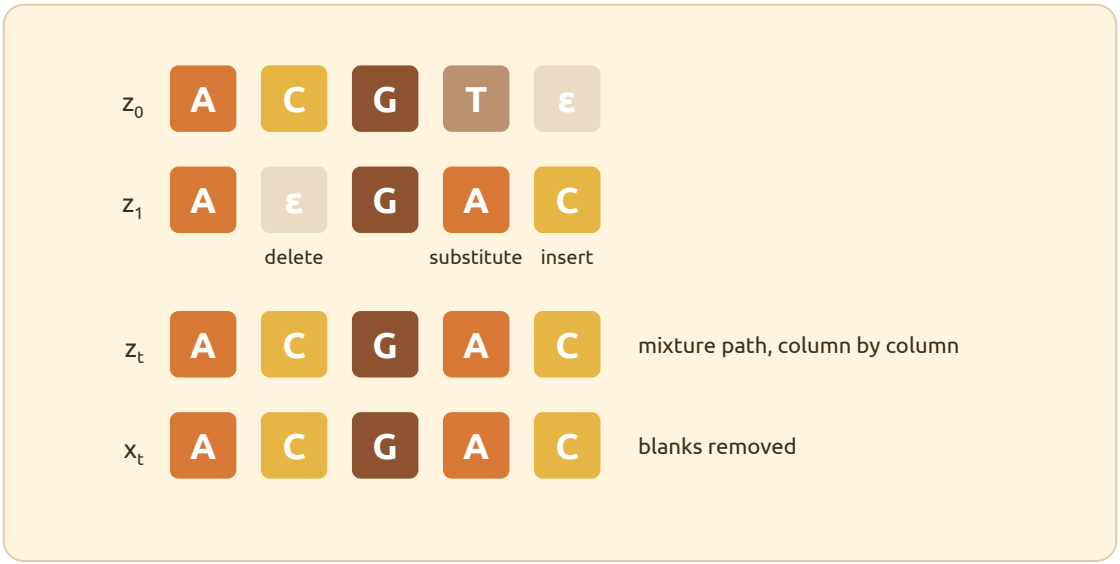


Figure 5.6: An alignment of $x_0 = \text{ACGT}$ with $x_1 = \text{AGAC}$ over five columns, with the blank written ε. The three columns that differ carry a deletion (column 2), a substitution (column 4) and an insertion (column 5). The third row is one draw of z_t at $\kappa_t = 0.5$, in which column 2 still holds its source token and columns 4 and 5 already hold their targets; this draw has probability $0.5^3 = 0.125$. The last row is x_t after the blanks are removed, which for this draw holds no blank at all and has length 5.

The length of x_t is a random variable, since a column produces a token only when it does not hold a blank. Column i holds a blank with probability $(1 - \kappa_t)\mathbf{1}[z_0^i = \varepsilon] + \kappa_t \mathbf{1}[z_1^i = \varepsilon]$, and summing over the N columns,

$$\mathbb{E}[\,|x_t|\,] = N - (1 - \kappa_t)(N - |x_0|) - \kappa_t(N - |x_1|) = (1 - \kappa_t)\,|x_0| + \kappa_t\,|x_1|,$$

so the expected length interpolates from $|x_0|$ to $|x_1|$. Both example sequences have length 4, so the expected length stays at 4. The sampled length can still vary. At $t = 0.5$, lengths 3, 4, and 5 occur with probabilities 0.25, 0.5, and 0.25. For a length-3 source and length-5 target, the expected length would be $3 + 2\kappa_t$, equal to 4 at the midpoint.

5.17.2 Averaging over latent alignments

Several alignments can connect the same sequence pair, and x_t does not specify which alignment produced it. We can marginalize this auxiliary variable by posterior averaging.

Theorem 5.4 (Marginalizing the alignment)**.** *Let the aligned pair (z_0, z_1) follow a column-wise mixture path on the padded state space, let x_t be the sequence obtained from z_t by deleting its blanks, and let $u_t(x, z \mid x_t, z_t)$ generate that path on the joint space. Then the velocity on sequences alone,*

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)}\big[\, u_t(x, z \mid x_t, z_t)\,\big],$$

generates the marginal path of x_t.

"The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t."

By linearity of the master equation, the posterior-averaging theorem also applies to an unseen alignment. Training on sampled alignments gives the same gradients as training with the averaged rate. We can therefore draw one alignment per example without enumerating the others.

The pair $x_0 = \mathrm{AC}$ and $x_1 = \mathrm{AAC}$ is small enough to enumerate, and it admits two alignments on three columns:

$$A_1 :\ z_0 = (\mathrm{A}, \mathrm{C}, \varepsilon),\ z_1 = (\mathrm{A}, \mathrm{A}, \mathrm{C}), \qquad A_2 :\ z_0 = (\mathrm{A}, \varepsilon, \mathrm{C}),\ z_1 = (\mathrm{A}, \mathrm{A}, \mathrm{C}).$$

The first replaces the C of column 2 with A and inserts C in column 3; the second inserts A in column 2 and does not touch column 3. Give them equal prior weight and ask which one produced the observed $x_t = \mathrm{AC}$ at time t. Under A_1 that means column 2 still holds its source token and column 3 still holds its blank, with probability $(1 - \kappa_t)^2$. Under A_2 it means only that column 2 still holds its blank, with probability $1 - \kappa_t$, since column 3 agrees in both sequences. Bayes' rule then produces

$$p_t(A_1 \mid x_t = \mathrm{AC}) = \frac{(1 - \kappa_t)^2}{(1 - \kappa_t)^2 + (1 - \kappa_t)} = \frac{1 - \kappa_t}{2 - \kappa_t},$$

which is 0.4286 at $\kappa_t = 0.25$, $1/3$ at $\kappa_t = 0.5$, and 0.2 at $\kappa_t = 0.75$. The shorter alignment has increasing posterior weight over time. The longer alignment requires two columns to retain their source symbols, whereas the shorter one requires only one.

Now, we can average the target rates at $t = 0.5$ with the linear schedule, where the weight $\dot\kappa_t/(1-\kappa_t)$ equals 2. Under A_1 two columns are unfinished, so two edits have rate 2 each; under A_2 one column is unfinished and one edit has rate 2. Weighting by the posterior $(1/3, 2/3)$,

$$\begin{aligned} u_t\big(\mathrm{sub}(x_t, 2, \mathrm{A}) \mid x_t\big) &= \tfrac{1}{3} \times 2 = 0.667, \\ u_t\big(\mathrm{ins}(x_t, 1, \mathrm{A}) \mid x_t\big) &= \tfrac{2}{3} \times 2 = 1.333, \\ u_t\big(\mathrm{ins}(x_t, 2, \mathrm{C}) \mid x_t\big) &= \tfrac{1}{3} \times 2 = 0.667, \end{aligned}$$

for total target rate 2.667. The shorter alignment's insertion has twice the rate of either edit from the longer alignment. We compute these rates by averaging over alignments without selecting one.

5.17.3 The training loss

We now predict rates directly, so we need a loss for those rates. Write u_t^θ for the predicted velocity and $x(z_t, i, z_1^i)$ for the sequence obtained by replacing column i of the current aligned state with its target symbol,

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbf{1}[z_1^i \neq z_t^i] \, \frac{\dot\kappa_t}{1-\kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

"The edit loss $\mathcal{L}(\theta)$ averages the predicted total exit rate $\sum_{x \neq x_t} u_t^\theta(x \mid x_t)$, minus the log rates of edits that replace each unfinished aligned symbol z_t^i with its target z_1^i, weighted by the schedule factor $\dot\kappa_t/(1-\kappa_t)$."

- z_t is the current aligned state and z_1 its target, so $\mathbf{1}[z_1^i \neq z_t^i]$ marks columns whose current and target symbols differ.
- $x(z_t, i, z_1^i)$ is the sequence that results from finishing column i, which is an insertion, a deletion or a substitution depending on which symbols the column holds.

The first term sums the predicted intensities and penalizes large rates. The weighted cross-entropy term favors higher rates for edits toward x_1. For one edit with predicted intensity λ and expected weight w, the contribution is $\lambda - w \log \lambda$. Differentiating gives

$$\frac{\partial}{\partial \lambda}\big[\lambda - w \log \lambda\big] = 1 - \frac{w}{\lambda} = 0 \quad \Longrightarrow \quad \lambda^\star = w,$$

with second derivative $w/\lambda^2 > 0$, so the stationary point is the minimum. The optimal intensity equals the rate at which the edit progresses toward the target. This calibrates the sampling rates through the same loss structure as the Bregman rate loss of Chapter 4. With substitutions only and a mask source, the second sum covers masked positions with weight $\dot\kappa_t/(1-\kappa_t)$, and the first sum is the total unmasking rate. The loss then reduces to masked flow matching.

Let's calculate the loss for Figure 5.6 at $t = 0.5$, using $\kappa_t = t$. The current sequence is $x_t = \text{ACGAC}$. Only column 2 is unfinished: its target is blank, so the required edit deletes its C. The schedule weight is $\dot\kappa_t/(1-\kappa_t) = 1/(1-0.5) = 2$. Suppose the predicted deletion intensity is $\lambda^{\text{del}} = 0.8$, and the intensities over all possible edits sum to 3.0. Substituting these values gives

$$\mathcal{L} = 3.0 - 2 \log 0.8 = 3.0 + 0.4463 = 3.446.$$

If we raise λ^{del} toward the stationary value $\lambda^\star = 2$, the second term falls faster than the first rises, and at $\lambda^{\text{del}} = 2$ the two effects balance.

5.17.4 One step of the sampler

For each sampling step, apply the Euler rule to each edit type. At position i, insert with probability $h\lambda_{t,i}^{\text{ins}}$, and delete or substitute with probability $h(\lambda_{t,i}^{\text{del}} + \lambda_{t,i}^{\text{sub}})$. Deletion and substitution at the same position are mutually exclusive, so choose between them in proportion to their intensities. Draw new tokens from Q^{ins} or Q^{sub}, then apply all selected edits simultaneously, as in Section 5.12.1.

Take $x_t = \text{ACG}$ with $\lambda^{\text{del}} = 0.9$ and $\lambda^{\text{sub}} = 0.3$ at the C, and a step of $h = 0.05$. The C is edited with probability $0.05 \times (0.9 + 0.3) = 0.06$, and given that an edit happens it is a deletion with

probability $0.9/1.2 = 0.75$, three times in four, and a substitution otherwise. An insertion intensity of $\lambda^{\text{ins}} = 0.4$ at the G would add a token to its right with probability $0.05 \times 0.4 = 0.02$ in the same step, independently.

With the rest of the intensities filled in, we can take one whole step of the sampler. Let the model output

$$\begin{aligned}
\text{position 1 (A)}: &\quad \lambda^{\text{ins}} = 0.2,\ \lambda^{\text{del}} = 0.1,\ \lambda^{\text{sub}} = 0.4,\\
\text{position 2 (C)}: &\quad \lambda^{\text{ins}} = 0.3,\ \lambda^{\text{del}} = 0.9,\ \lambda^{\text{sub}} = 0.3,\\
\text{position 3 (G)}: &\quad \lambda^{\text{ins}} = 0.4,\ \lambda^{\text{del}} = 0.2,\ \lambda^{\text{sub}} = 0.5,
\end{aligned}$$

for a total intensity of 3.3, which is the first term of the loss for this example. We draw six independent coins: insertions with probabilities 0.010, 0.015 and 0.020, and a deletion-or-substitution with probabilities 0.025, 0.060 and 0.035. The sequence is untouched with probability 0.8452, against the first-order estimate $1 - h \times 3.3 = 0.835$, and the 0.0102 difference is again the second-order term from steps that apply two edits at once. The expected change in length is

$$h\Big(\sum_i \lambda^{\text{ins}}_{t,i} - \sum_i \lambda^{\text{del}}_{t,i}\Big) = 0.05\,(0.9 - 1.2) = -0.015,$$

so this sequence drifts shorter at a rate of 0.3 tokens per unit time. The length is a running sum of insertions minus deletions, and nothing fixes it in advance.

With a model trained on code, the Edit Flows authors solved 12.8% of HumanEval problems on the first try. A mask model of the same size, supplied with the true length, solved 9.1% [Havasi et al., 2025]. Allowing variable length changes the state space while preserving the rate, path and loss construction.

5.18 Discrete flow variants

All four methods use probability velocities with nonnegative off-diagonal entries and columns that sum to zero,

$$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0,$$

it generates a path when it satisfies the master equation,

$$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x),$$

and the path we interpolate along is the mixture path,

$$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i} + \kappa_t\, \delta_{x_1^i},$$

where each position holds its source token with probability $1 - \kappa_t$ and its data token with probability κ_t. We compare the methods in Table 5.3.

We make three choices. First, we choose the path and thus the marginals at every time: a scheduled mixture, a softmax of negative distances, or a column-wise mixture on aligned sequences. Second, we choose what to predict: data-token distributions for the first three methods, or edit intensities and token distributions for Edit Flows. Third, we choose a sampler after training, since it does not appear in the loss. We can use the minimum-jump velocity or add circulation of strength η.

Changing the schedule changes reveal times without retraining. A corrector increases the average jump count to $1 + \eta$, with a corresponding increase in network calls and decrease in step size. A metric path requires distances between alphabet tokens. Variable-length editing requires rates for each edit at every position and marginalization over alignments.

Table 5.3: The four published constructions of this section, by the component each one changes. All four share the master equation, the posterior-averaged velocity and the Euler sampler built earlier in this chapter. η is the corrector strength of Section 5.14.2 and d is the token distance of Section 5.16.4.

Construction	Path	Network predicts	Sampler
Discrete Flow Matching	mixture, any κ_t	data token $p_{1\|t}$	Euler redraw
Discrete flow models	any $p_t(\cdot \mid x_1)$	data token $p_{1\|t}$	Euler, plus η
Kinetic-optimal	softmax of $-\beta_t d$	data token $p_{1\|t}$	Euler redraw
Edit Flows	mixture on columns	edit intensities, tokens	Euler per edit kind

In all four methods, we define rates on a graph whose vertices are sequences. Each jump changes a discrete state. The square-root construction in Section 5.16.6 also gives us a continuous geometry for probability distributions over those states.

5.19 Continuous token relaxations

After a discrete sampling step, a position contains either its original letter or another letter. To allow continuous motion, we can give each position a state that changes by small amounts before we select its final letter.

5.19.1 Motion on a finite set

On a finite alphabet without a specified distance, C is no closer to G than to A. A position changes through jumps, which we describe by rates. The master equation, posterior average and corrector all apply to this finite set. A mixture path, schedule κ_t and trained denoiser $p_{1|t}$ are sufficient to generate sequences.

Several continuous flow-matching operations require a differentiable state. An ODE solver needs a state that varies continuously, so that a small step in time produces a small change in the state. Classifier guidance needs a gradient of a log probability with respect to the state. Distillation into a single step needs a deterministic map from a source sample to a generated sample, so that we can train a student network on pairs of the two.

We can relax each letter to a probability vector. A DNA base then has four coordinates: certainty about A corresponds to $(1, 0, 0, 0)$, and complete uncertainty to $(0.25, 0.25, 0.25, 0.25)$. To generate a token, we follow a continuous velocity field from near the center to near a corner, then select the letter with the largest final coordinate.

5.19.2 The simplex as a state space

The set of probability vectors over an alphabet of K letters is the probability simplex,

$$\Delta^{K-1} \;=\; \Big\{\, x \in \mathbb{R}^K \;:\; \sum_{k=1}^{K} x_k = 1, \quad x_k \geq 0 \,\Big\}$$

"The probability simplex Δ^{K-1} is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative."

- $K = |\mathcal{V}|$ is the alphabet size, four for DNA.
- $x \in \Delta^{K-1}$ is one relaxed token, a probability vector over the letters, with x_k the mass it puts on letter k.
- The superscript records the dimension. The sum constraint fixes one degree of freedom, so Δ^{K-1} is a flat $(K-1)$-dimensional piece of $\mathbb{R}^K$.

For DNA, $K = 4$ and Δ^3 is a solid tetrahedron whose four corners are A, C, G and T. The corners are the one-hot vectors,

$$\mathbf{e}_k \in \Delta^{K-1}, \qquad (\mathbf{e}_k)_j = \delta_{jk},$$

"The corner $\mathbf{e}_k$ is the vector whose k-th coordinate is one and whose other coordinates are zero."

For three letters, the simplex is a triangle (Figure 4.12). Token-valued paths jump from corner to corner along its edges. Simplex-valued paths can move continuously through its interior.

We preserve the discrete data by placing it at the corners. A categorical distribution with letter probabilities q_k becomes a simplex distribution supported on its K corners,

$$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - \mathbf{e}_k),$$

"The data distribution $q(x)$ is the sum over letters k of the probability q_k times a point mass at the corner $\mathbf{e}_k$."

A sequence of L positions becomes a point in the product $(\Delta^{K-1})^L$, one simplex per position, and the data distribution on that product is supported on the K^L tuples of corners. The state space is now continuous, and its probability mass is confined to a finite set of points inside it.

5.19.3 The straight-line path and its collapsing support

We can apply the straight-line interpolation from Chapter 2 to a source point x_0 and a data corner x_1,

$$\psi_t(x_0 \mid x_1) = (1-t)\, x_0 + t\, x_1,$$

"The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1."

Using the same derivation as in Chapter 2, we obtain the velocity

$$u_t(x \mid x_1) = \frac{x_1 - x}{1 - t},$$

"The conditional velocity at point x is the displacement $x_1 - x$ toward the data corner x_1, divided by the remaining time $1 - t$."

A convex combination of simplex points remains in the simplex, so ψ_t stays in Δ^{K-1}. The displacement $x_1 - x$ has coordinate sum $1 - 1 = 0$, making the velocity tangent and preserving the probability sum under an Euler step.

Let the source distribution be uniform on the simplex. The interpolant then shrinks the whole simplex toward the corner $\mathbf{e}_i$ by the factor $1 - t$, and we can write the conditional path in closed form.

- The image of Δ^{K-1} under $x_0 \mapsto (1 - t)x_0 + t\,\mathbf{e}_i$ is exactly $\{x \in \Delta^{K-1} : x_i \geq t\}$. In one direction, $x_i = (1 - t)x_{0,i} + t \geq t$. In the other, given $x_i \geq t$, the point $x_0 = (x - t\,\mathbf{e}_i)/(1 - t)$ has nonnegative coordinates and sums to one.

- The map contracts each of the $K - 1$ chart directions by $1 - t$, so its Jacobian determinant is $(1 - t)^{K-1}$ and the density is multiplied by the reciprocal factor.

- The uniform density on Δ^{K-1} is the constant $\Gamma(K) = (K - 1)!$, since the simplex has volume $1/(K - 1)!$ in the chart $(x_1, \ldots, x_{K-1})$.

Combining the support, Jacobian and source density, we obtain

$$p_t(x \mid x_1 = \mathbf{e}_i) = \begin{cases} \dfrac{\Gamma(K)}{(1 - t)^{K-1}} & x_i \geq t, \\ 0 & x_i < t. \end{cases}$$

"The conditional density is constant on the region where the coordinate x_i is at least t, with value $\Gamma(K)$ divided by one minus t raised to the power $K - 1$, and it is zero where x_i falls below t."

The support is the shrunken simplex $\{x_i \geq t\}$, and the constant normalizes the total probability to one, since that region has volume $(1 - t)^{K-1}/\Gamma(K)$. For DNA the density at $t = 0$ is $\Gamma(4) = 6$, uniform over the whole tetrahedron; at $t = 0.5$ it is $6/0.5^3 = 48$, concentrated on the eighth of the tetrahedron where the data coordinate exceeds one half.

If $x_i < t$, the point x cannot have come from corner $\mathbf{e}_i$, so that corner has posterior probability zero. We can bound the number of possible corners by counting coordinates.

Proposition 5.5 (Support of the posterior under the straight-line path)**.** *Let the source be uniform on Δ^{K-1} and let the conditional path be the straight-line interpolant toward a corner. For any point $x \in \Delta^{K-1}$ and any time $t > 1/k$, the posterior over corners has support on at most $k-1$ corners.*

Proof. Suppose k corners $\mathbf{e}_{j_1}, \ldots, \mathbf{e}_{j_k}$ are all still possible at x at time t, which means $x_{j_m} \geq t$ for each m. Summing coordinates,

$$\mathbf{1}^\top x \;=\; \sum_{j=1}^{K} x_j \;\geq\; \sum_{m=1}^{k} x_{j_m} \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} \;=\; 1.$$

"The sum of all coordinates of x is at least the sum of the k coordinates that are still possible, each of which is at least t, so the total exceeds k times one over k, which is one."

A simplex point sums to exactly one, so the assumption fails. □

For DNA, a base becomes impossible after each threshold $t = 1/4$, $t = 1/3$, and $t = 1/2$. After the final threshold, at most one base remains possible at any simplex point. At $t = 0.3$, the point $x = (0.34, 0.33, 0.33, 0.00)$ has three coordinates at least 0.3. A, C, and G are therefore possible, attaining the bound $k - 1 = 3$ for $t > 1/4$. T has posterior probability zero. At $t = 0.6$, only the A coordinate of $x = (0.70, 0.30, 0, 0)$ exceeds the threshold. The posterior is then a point mass on A.

The resulting training target is discontinuous in time: a corner's posterior probability drops to zero when t crosses its threshold. It is also discontinuous in space, since nearby points can lie on opposite sides of $x_i = t$. Sampling becomes sensitive to step size because an Euler step can cross a threshold into a region with a collapsed posterior. We can avoid these problems with a path that retains every corner at every finite time.

5.20 Dirichlet Flow Matching

In Dirichlet Flow Matching [Stark et al., 2024] we use a family of Dirichlet distributions on the simplex, indexed by time. We choose the family so that the conditional path is positive on the whole interior for every finite t. Both discontinuities then disappear, and we recover the velocity of that path by balancing probability flux across a cut of the simplex.

5.20.1 The Dirichlet family as a conditional path

The Dirichlet distribution has one concentration parameter per letter and density

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}, \qquad \mathcal{B}(\alpha) = \frac{\prod_k \Gamma(\alpha_k)}{\Gamma\big(\sum_k \alpha_k\big)},$$

"The Dirichlet density at the probability vector x multiplies each coordinate x_k raised to its concentration parameter α_k minus one, then divides by the normalizing multivariate beta function $\mathcal{B}(\alpha)$."

- $\alpha = (\alpha_1, \ldots, \alpha_K)$ with $\alpha_k > 0$ are the concentrations, one per letter.
- $\mathcal{B}(\alpha)$ is the multivariate beta function, the normalizing constant.

If all the concentrations equal one, every exponent is zero and the density is constant, which is the uniform distribution on the simplex. Raising one concentration above one pulls mass toward that corner, and the larger it grows the more concentrated the distribution becomes there.

In Dirichlet Flow Matching we take the concentration to be the all-ones vector plus t in the data coordinate,

$$p_t(x \mid x_1 = \mathbf{e}_i) = \mathrm{Dir}(x;\ \alpha = \mathbf{1} + t\,\mathbf{e}_i)$$

"The conditional density $p_t(x \mid x_1 = \mathbf{e}_i)$ is Dirichlet with concentration $1 + t$ at the data corner's coordinate i and concentration one at every other coordinate."

The i-th concentration is $1 + t$ and all the others stay at 1, so only the data coordinate is favored, and more strongly as t grows. At $t = 0$ every concentration equals one and the path starts from the uniform distribution on the simplex, which is where generation begins.

Substitute $\alpha = \mathbf{1} + t\,\mathbf{e}_i$ into the density. All exponents $\alpha_k - 1$ vanish except at coordinate i, where the exponent equals t. The beta function becomes $\Gamma(1 + t)\,\Gamma(1)^{K-1}/\Gamma(K + t) = \Gamma(t + 1)/\Gamma(t + K)$, giving

$$p_t(x \mid x_1 = \mathbf{e}_i) = \frac{\Gamma(t + K)}{\Gamma(t + 1)}\, x_i^t$$

"The conditional density $p_t(x \mid x_1 = \mathbf{e}_i)$ equals the normalizing ratio $\Gamma(t + K)/\Gamma(t + 1)$, where K is the alphabet size, times the data coordinate x_i raised to time t."

At $t = 0$, the prefactor is $\Gamma(K)/\Gamma(1) = \Gamma(K)$ and the exponent is zero. We therefore recover the uniform simplex density $\Gamma(K)$, equal to 6 for DNA, just as for the straight-line source. For every finite t, the density remains positive wherever $x_i > 0$. Every corner therefore stays possible in the posterior. The support does not collapse as in Proposition 5.5, and the denoiser target remains continuous.

The concentration $1 + t$ gives a point mass at the corner only in the limit, so time must extend beyond one.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \ \text{ in training}, \qquad t_{\max} = 8 \ \text{ at inference}$$

"Time t is nonnegative and unbounded. During training, we draw t from an exponential distribution of rate one; during inference, we integrate to $t = 8$."

Here x_1 denotes the data sample without referring to time one. To sample, we take 100 Euler steps up to $t = 8$, then select the token from the final posterior.

5.20.2 Velocity derivation by flux matching

We need a velocity field that transports p_0 to p_t and satisfies the continuity equation on the simplex. We can keep the direction toward the data corner from the straight-line velocity and change the speed,

$$\boxed{u_t(x \mid x_1 = \mathbf{e}_i) = C(x_i, t)\,(\mathbf{e}_i - x)}$$

"The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and on the time, times the direction $\mathbf{e}_i - x$ from the current point to the data corner."

The straight-line velocity is the special case $C = 1/(1-t)$, a function of time alone. The direction $\mathbf{e}_i - x$ has coordinates summing to zero, so whatever C turns out to be, the velocity is tangent to the simplex and the flow stays on it.

To find C, we cut the simplex with the plane $x_i = c$ and let $S_1 = \{x \in \Delta^{K-1} : x_i < c\}$ be the region below the cut. Probability crossing that plane must account exactly for the rate at which S_1 loses mass.

First, compute the probability of S_1. Under the Dirichlet path, the coordinate x_i has distribution $\mathrm{Beta}(t+1, K-1)$. This is the standard Dirichlet marginal with parameter α_i for coordinate i and $\sum_{k \neq i} \alpha_k = K - 1$ for the remaining coordinates. We therefore need a one-dimensional integral,

$$p_t(S_1) = \frac{\mathcal{B}(c;\, t+1,\, K-1)}{\mathcal{B}(t+1,\, K-1)} = I_c(t+1,\, K-1),$$

"The probability $p_t(S_1)$ below the cut $x_i = c$ equals the Beta cumulative probability $I_c(t+1, K-1)$, obtained by dividing the incomplete beta function at c by the complete beta function with parameters $t+1$ and $K-1$."

Next, differentiate this probability in time. Only the first Beta parameter depends on t, so

$$\frac{d}{dt}\, p_t(S_1) = \tilde{I}_c(t+1,\, K-1), \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b),$$

where a and b are placeholders for the two Beta parameters.

To compute the flux through the cut, use the velocity on its plane. Every point on the plane $x_i = c$ has the same i-th velocity component, $C(c,t)(1-c)$, which is the only component that moves mass across. The density on the cut is the constant $\left[\Gamma(K+t)/\Gamma(1+t)\right] c^t$, and the slice $\{x \in \Delta^{K-1} : x_i = c\}$ is a copy of Δ^{K-2} scaled by $1-c$, so its volume in the chart is $(1-c)^{K-2}/\Gamma(K-1)$. No mass escapes through the other faces: at $x_j = 0$ for $j \neq i$ the velocity component $u_j = -Cx_j$ vanishes, and the same holds at $x_i = 0$.

Setting the loss rate equal to minus the outward flux,

$$\tilde{I}_c(t+1, K-1) \;=\; -\,C(c,t)\,(1-c)\,\frac{\Gamma(K+t)}{\Gamma(1+t)}\,c^t\,\frac{(1-c)^{K-2}}{\Gamma(K-1)}.$$

Solving for C and collecting the Gamma functions into $\mathcal{B}(t+1, K-1) = \Gamma(t+1)\Gamma(K-1)/\Gamma(t+K)$,

$$C(x_i, t) = -\,\tilde{I}_{x_i}(t+1,\, K-1)\,\frac{\mathcal{B}(t+1,\, K-1)}{(1-x_i)^{K-1}\,x_i^t}, \qquad \tilde{I}_x(a,b) = \frac{\partial}{\partial a} I_x(a,b)$$

"The scalar speed $C(x_i, t)$ equals minus the derivative $\tilde{I}_{x_i}(t+1, K-1)$ of the regularized incomplete beta function with respect to its first parameter, multiplied by the complete beta function $\mathcal{B}(t+1, K-1)$, and divided by the product $(1-x_i)^{K-1}x_i^t$."

Increasing the first Beta parameter moves probability toward larger coordinate values. The cumulative probability below a fixed cut decreases, so its derivative $\tilde{I}$ is negative. The speed C is therefore positive and points toward the data corner. For general K, this derivative has no closed form. We evaluate it numerically at each position and inference step using the current data-coordinate value.

Proposition 5.6 (Continuity for the Dirichlet path)**.** *With $p_t(x \mid \mathbf{e}_i) = \mathrm{Dir}(x; \mathbf{1} + t\,\mathbf{e}_i)$ and $u_t(x \mid \mathbf{e}_i) = C(x_i, t)(\mathbf{e}_i - x)$ with C as above, the pair satisfies the continuity equation on Δ^{K-1}, $\partial_t p_t + \nabla \cdot (p_t\, u_t) = 0$.*

We have established flux balance for one cut $x_i = c$. Varying c over $(0, 1)$ covers the simplex. Requiring the balance at every cut gives the differential continuity equation.

5.20.3 Two-letter closed-form solution

For two letters, we can calculate the speed in closed form. The second Beta parameter is $K - 1 = 1$, and there the regularized incomplete beta function is $I_x(a, 1) = x^a$, so $\tilde{I}_x(a, 1) = x^a \ln x$ and $\mathcal{B}(t+1, 1) = 1/(t+1)$. Substituting,

$$K = 2: \qquad C(x_1, t) = \frac{-\,x_1 \ln x_1}{(t+1)\,(1-x_1)}$$

"For two letters, the speed $C(x_1, t)$ equals minus the current coordinate x_1 times its logarithm, divided by the product of $t + 1$ and the remaining mass $1 - x_1$."

The logarithm of a number below one is negative, so the speed is positive on the whole open interval. At $x_1 = 0.3$ and $t = 1$ the closed form reads

$$C(0.3, 1) = \frac{-\,0.3 \times (-1.2040)}{2 \times 0.7} = \frac{0.3612}{1.4} = 0.2580,$$

and the general formula, with $\tilde{I}$ evaluated numerically, returns 0.2579942, matching the closed form to six decimals.

Writing x for the first coordinate, the marginal density of the Dirichlet path is $p_t(x) = (1+t)\,x^t$, the velocity in that coordinate is $u = C(x,t)(1-x)$, and the product simplifies because the factor $1-x$ cancels,

$$p_t(x)\,u(x) = (1+t)\,x^t \cdot \frac{-x\ln x}{(t+1)(1-x)} \cdot (1-x) = -\,x^{\,t+1}\ln x.$$

Differentiating the density in time and the flux in space,

$$\partial_t p + \partial_x(p\,u) = \left[x^t + (1+t)x^t \ln x\right] - \left[(t+1)x^t \ln x + x^t\right] = 0,$$

"The density derivative $\partial_t p_t$ is $x^t + (1+t)x^t \log x$. The spatial derivative of the probability flux $p_t u$ is its negative, so their sum is zero."

So the path and the velocity satisfy the continuity equation exactly, which is Proposition 5.6 at $K = 2$. If we evaluate the two terms numerically at $K = 3$, with $x = (0.5, 0.3, 0.2)$ and $t = 0.8$, we get $\partial_t p = 0.6355$ and $\nabla \cdot (pu) = -0.6355$, which agree to ten decimals.

5.20.4 Dirichlet velocity on DNA

We now take $K = 4$, time $t = 1$, and the relaxed base $x = (0.40,\ 0.30,\ 0.20,\ 0.10)$ for the letters $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$. The Beta parameters are $a = t + 1 = 2$ and $b = K - 1 = 3$, and $\mathcal{B}(2,3) = \Gamma(2)\Gamma(3)/\Gamma(5) = 2/24 = 1/12$. Evaluating $\tilde{I}_{x_k}(2,3)$ numerically at each coordinate and dividing by $(1-x_k)^3 x_k$,

$$C(0.40, 1) = 0.232, \quad C(0.30, 1) = 0.199, \quad C(0.20, 1) = 0.159, \quad C(0.10, 1) = 0.104.$$

The speeds are 0.232 toward A, 0.199 toward C, 0.159 toward G and 0.104 toward T. A coordinate that already holds a large share has a larger speed, so the flow concentrates gradually and the largest coordinate moves fastest toward its corner. The conditional velocity toward A is that speed times the direction to the A corner,

$$u_t(x \mid \mathbf{e}_{\mathrm{A}}) = 0.232 \times (0.60,\ -0.30,\ -0.20,\ -0.10) = (0.139,\ -0.070,\ -0.046,\ -0.023),$$

"The velocity toward A is 0.232 times the direction to the A corner, which equals 0.139 in the A coordinate, -0.070 in C, -0.046 in G and -0.023 in T, and these four numbers sum to zero."

The sum is zero to the digits shown, which confirms that the step stays on the simplex. The three other conditional velocities follow in the same way, each one pointing at its own corner:

$$\begin{aligned}
u_t(x \mid \mathbf{e}_{\mathrm{C}}) &= 0.199 \times (-0.40,\ 0.70,\ -0.20,\ -0.10) = (-0.0796,\ 0.1393,\ -0.0398,\ -0.0199),\\
u_t(x \mid \mathbf{e}_{\mathrm{G}}) &= 0.159 \times (-0.40,\ -0.30,\ 0.80,\ -0.10) = (-0.0636,\ -0.0477,\ 0.1272,\ -0.0159),\\
u_t(x \mid \mathbf{e}_{\mathrm{T}}) &= 0.104 \times (-0.40,\ -0.30,\ -0.20,\ 0.90) = (-0.0416,\ -0.0312,\ -0.0208,\ 0.0936).
\end{aligned}$$

5.20.5 The posterior average and the loss

During generation, we do not know the data corner. We therefore average the conditional velocities under the posterior over corners. Since the data lies on K corners, the integral reduces to a finite sum,

$$v_t(x) = \sum_{k=1}^{K} u_t(x \mid x_1 = \mathbf{e}_k)\; p_{1|t}(\mathbf{e}_k \mid x)$$

"The marginal velocity at x equals the sum over corners $\mathbf{e}_k$ of the conditional velocity toward that corner, weighted by the posterior probability $p_{1|t}(\mathbf{e}_k \mid x)$ that the data token is k."

Every conditional velocity is tangent to the simplex, and their weighted average is also tangent. The learned field therefore stays tangent without an additional constraint.

At this point in the worked base, let the denoiser output be $\hat{p} = (0.60, 0.25, 0.10, 0.05)$ over A, C, G and T. Weighting the four conditional velocities by those four numbers,

$$\begin{aligned}
v_{\mathrm{A}} &= 0.60(0.1392) + 0.25(-0.0796) + 0.10(-0.0636) + 0.05(-0.0416) = 0.0552,\\
v_{\mathrm{C}} &= 0.60(-0.0696) + 0.25(0.1393) + 0.10(-0.0477) + 0.05(-0.0312) = -0.0133,\\
v_{\mathrm{G}} &= 0.60(-0.0464) + 0.25(-0.0398) + 0.10(0.1272) + 0.05(-0.0208) = -0.0261,\\
v_{\mathrm{T}} &= 0.60(-0.0232) + 0.25(-0.0199) + 0.10(-0.0159) + 0.05(0.0936) = -0.0158.
\end{aligned}$$

The four components sum to zero. An Euler step of size $h = 0.1$ moves the base to

$$x + 0.1\, v = (0.4055,\ 0.2987,\ 0.1974,\ 0.0984),$$

which still sums to one. Mass moves toward A, the letter with the largest denoiser probability, and the other three coordinates shrink in proportion to how much the flow toward A draws from them.

We train the denoiser by minimizing cross-entropy on the data token, using the same loss as for jump paths,

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\,x_1,\,x}\Big[\log p^{\theta}_{1|t}(x_1 \mid x)\Big]$$

"The denoising loss $\mathcal{L}(\theta)$ averages the negative log probability $p^{\theta}_{1|t}(x_1 \mid x)$ assigned to the true data token x_1, over sampled times t, data tokens x_1, and noisy simplex points x from the conditional path."

At the cross-entropy minimum, we recover the true posterior. Substituting it into the average then gives the true marginal velocity. We evaluate the conditional velocity $C(x_i, t)(\mathbf{e}_i - x)$ only during sampling, avoiding numerical evaluations of $\tilde{I}$ during training. In each forward pass, we input all L positions and obtain logits per position. The denoiser therefore captures dependencies between positions, as it does for jump paths.

5.20.6 The score, guidance, and distillation into one step

We can differentiate the conditional log density to obtain its score. The Gamma prefactor is independent of x, leaving only the gradient of $t \log x_i$,

$$s_t(x \mid x_1 = \mathbf{e}_i) = \nabla_x \, t \log x_i = \frac{t}{x_i} \, \mathbf{e}_i,$$

"The conditional score $s_t(x \mid x_1 = \mathbf{e}_i)$ is the gradient of $t \log x_i$, with value t/x_i in data coordinate i and zero in all other coordinates."

If we average over corners with the denoiser output $\hat{p}$, whose k-th entry is $p^{\theta}_{1|t}(\mathbf{e}_k \mid x)$, the score becomes a matrix-vector product,

$$\hat{s} = D \, \hat{p}, \qquad D_{jk} = \delta_{jk} \, \frac{t}{x_j}$$

"The marginal score $\hat{s}$ equals the denoiser probability vector $\hat{p}$ multiplied by the diagonal matrix D, whose entry for coordinate j is time t divided by the current mass x_j."

The marginal velocity is linear in $\hat{p}$ as well, $\hat{v} = U\hat{p}$ with U the matrix whose k-th column is the conditional velocity $C(x_k, t)(\mathbf{e}_k - x)$. The two are therefore related by $\hat{v} = UD^{-1}\hat{s}$, so we can transfer a modification of either one to the other. For the worked base at $t = 1$, the diagonal of D is (2.5, 3.333, 5, 10) and the score is $\hat{s} = (1.5, 0.833, 0.5, 0.5)$.

We can use that linearity to lower the cost of guidance. In classifier-free guidance we combine a label-conditioned field and an unconditional one,

$$\hat{v}_{\mathrm{CFG}}(x, t, y) = \gamma \, \hat{v}(x, t, y) + (1 - \gamma) \, \hat{v}(x, t, \varnothing)$$

"The classifier-free velocity $\hat{v}_{\mathrm{CFG}}(x, t, y)$ combines the label-conditioned velocity $\hat{v}(x, t, y)$ with weight γ and the unconditional velocity $\hat{v}(x, t, \varnothing)$ with weight $1 - \gamma$, where y is the label and γ the guidance strength."

Here γ is the guidance strength, y the conditioning label and $\varnothing$ the null label the network is trained with some fraction of the time. Because both fields are linear in the denoiser, we can combine the two probability vectors first and compute the single velocity once, which halves the number of evaluations of $\tilde{I}$. A value $\gamma > 1$ extrapolates past the conditional field and can drive an entry of the combined vector below zero, so we project the result back onto the simplex before use. For classifier guidance, we add the gradient of a noisy classifier to the score $\hat{s}$, project onto the simplex tangent plane, and apply D^{-1} to recover a denoiser.

ODE integration gives a deterministic map from a source point to an output sequence. We can run the trained model for 100 steps to obtain source-output pairs, then train a student network to reproduce each output in one evaluation. In discrete-noise diffusion, a fixed starting state can produce different samples, so it does not define the deterministic pairs used here. On promoter

design, the authors reported mean squared error 0.0278 for the single-step student and 0.0269 for the 100-step teacher [Stark et al., 2024]. The student requires one forward pass to sample.

The Dirichlet path retains full support by using a chosen density family and its corresponding velocity. We still measure simplex distances with ordinary Euclidean distance, which weights absolute changes in probabilities equally.

5.21 Fisher-Flow: the simplex as a sphere

5.21.1 Euclidean distance near the simplex boundary

Let's compare two moves on the simplex over two letters. Moving a base from (0.5, 0.5, 0, 0) to (0.4, 0.6, 0, 0) shifts a tenth of the mass between two letters that were evenly matched, and its Euclidean distance is $\sqrt{2}\times 0.1 = 0.1414$. Moving from (0.01, 0.99, 0, 0) to (0.001, 0.999, 0, 0) divides the first probability by ten, a tenfold relative change, and its Euclidean distance is $\sqrt{2}\times 0.009 = 0.0127$, eleven times smaller. The straight-line distance measures absolute differences in probability, and a probability of 0.001 differs from one of 0.01 by only 0.009 on that scale.

In Fisher-Flow [Davis et al., 2024], we use the Fisher-Rao metric on probability distributions, then change coordinates to obtain the ordinary metric on a sphere. The two moves have lengths 0.201 and 0.137 under this metric. The tenfold change in the rare letter therefore has a distance comparable to the change between common letters.

5.21.2 The Fisher information as a local metric

We can quantify how distinguishable two nearby distributions are through their KL divergence. Expanding it to second order gives a quadratic form,

$$D_{\mathrm{KL}}(p_\theta \,\|\, p_{\theta+d\theta}) = \tfrac{1}{2}\sum_{j,k} g_{jk}(\theta)\, d\theta^j\, d\theta^k + o(|d\theta|^2)$$

"For a small parameter change $d\theta$, the KL divergence between the original and perturbed distributions equals one half of the quadratic form $d\theta^\top g(\theta) d\theta$, where $g(\theta)$ is the Fisher information matrix, up to higher-order terms."

The first-order term vanishes, since KL is nonnegative and zero at $d\theta = 0$, so the quadratic form is the leading behavior. The matrix g is the Fisher information, and the expansion above is its definition as a metric on a family of distributions.

For a categorical distribution, let the parameters be the probabilities themselves, so the log-likelihood of outcome i is $\log p^i$ and $\partial \log p^i / \partial p^j = \delta_{ij}/p^j$. Then

$$g_{jk}(p) = \mathbb{E}_p\Big[\frac{\partial \log p}{\partial p^j}\,\frac{\partial \log p}{\partial p^k}\Big] = \sum_i p^i\,\frac{\delta_{ij}}{p^j}\,\frac{\delta_{ik}}{p^k} = \frac{\delta_{jk}}{p^j},$$

"The Fisher information entry $g_{jk}(p)$ averages the product of log-probability derivatives with respect to probabilities p^j and p^k. It equals $1/p^j$ when $j = k$ and zero otherwise."

The expectation runs over outcomes i; the two Kronecker deltas are nonzero only when $i = j = k$, leaving a single term, and the factor p^i cancels one of the two denominators. We then obtain the inner product of two tangent vectors at a simplex point,

$$\langle u, v \rangle_p = \sum_{i=1}^{K} \frac{u^i v^i}{p^i}$$

"The Fisher-Rao inner product of two tangent vectors u and v at the point p is the sum over letters of u^i times v^i divided by p^i."

- $p \in \Delta^{K-1}$ is the point at which the inner product is taken.
- u, v are tangent vectors at p, meaning their coordinates sum to zero.
- The division by p^i makes a change in a rare coordinate count for more, which is the behavior we wanted from the two moves above.

By Chentsov's theorem, Fisher-Rao is the only metric on the simplex, up to a constant factor, under which merging two outcomes into one or relabeling the letters is an isometry. Any other metric would make the geometry depend on the names we happened to give the letters, and on whether we chose to distinguish two letters that behave alike. The metric diverges at the boundary where some p^i equals zero. We therefore smooth the data corners slightly before using them as endpoints.

5.21.3 The square-root map to the sphere

We can remove the division by probability from the metric by changing coordinates. Take the square root of every probability,

$$\varphi(p) = (\sqrt{p^1},\ \ldots,\ \sqrt{p^K}), \qquad \varphi^{-1}(s) = (s_1^2,\ \ldots,\ s_K^2)$$

"The map φ sends the probability vector p to the vector of square roots of its coordinates, and its inverse squares every coordinate back."

The simplex constraint becomes a norm constraint,

$$\sum_i p^i = 1 \quad \Longrightarrow \quad \|\varphi(p)\|_2^2 = \sum_i \left(\sqrt{p^i}\right)^2 = 1,$$

so $\varphi(p)$ lies on the unit sphere in $\mathbb{R}^K$. Probabilities are nonnegative, so the image is the part of the sphere with all coordinates nonnegative, one of its 2^K octants, and φ is a bijection between the simplex and that octant.

If we differentiate the square root, the effect on the metric becomes explicit,

$$ds^i = \frac{dp^i}{2\sqrt{p^i}} \quad \Longrightarrow \quad \|ds\|_2^2 = \sum_i \frac{(dp^i)^2}{4\,p^i} = \tfrac{1}{4}\,\langle dp, dp \rangle_p$$

"A sphere-coordinate change ds^i equals the probability change dp^i divided by $2\sqrt{p^i}$. Summing squared coordinate changes gives a squared Euclidean length $\|ds\|_2^2$ equal to one quarter of the squared Fisher-Rao length $\langle dp, dp\rangle_p$."

Multiplying the map by two gives an isometry, so Fisher-Flow uses a sphere of radius 2. On the sphere, the metric is Euclidean distance restricted to the surface, with no division by probability. We can therefore apply the Riemannian flow constructions from Chapter 2.

Distance between two probability vectors follows from the angle between their square-root vectors,

$$d_{\mathrm{FR}}(p, q) = 2 \arccos\left(\langle \sqrt{p},\ \sqrt{q}\rangle\right)$$

"The Fisher-Rao distance between p and q equals twice the arccosine of the inner product of their square-root vectors."

Square-root vectors of probability vectors have nonnegative entries, so their inner product lies in $[0, 1]$, the angle is at most a right angle, and the distance is at most π. Two distinct letters, as one-hot vectors, are orthogonal, so any two letters are the farthest apart at distance π, whichever letters they happen to be. The distance is therefore invariant to relabeling, as in Section 5.21.2.

Let's take a DNA base to the sphere. We can start from the uniform base $p_0 = (0.25,\ 0.25,\ 0.25,\ 0.25)$ and a smoothed A, $p_1 = (0.91,\ 0.03,\ 0.03,\ 0.03)$, where the smoothing holds the endpoint off the corner so that the metric stays finite. Their square roots are

$$x_0 = (0.5,\ 0.5,\ 0.5,\ 0.5), \qquad x_1 = (0.954,\ 0.173,\ 0.173,\ 0.173),$$

and both have length one, since the squared length of a square-root vector is the sum of the probabilities, here $4 \times 0.25 = 1$ and $0.91 + 3 \times 0.03 = 1$. Their inner product and the distance that follows are

$$\langle x_0, x_1\rangle = 0.737, \qquad \theta = \arccos(0.737) = 0.743, \qquad d_{\mathrm{FR}} = 1.485,$$

"The inner product of the two square-root vectors is 0.737, so the angle between them is 0.743 radians, and the Fisher-Rao distance is twice that angle, 1.485."

The uniform base and smoothed A are separated by about half the maximum distance π.

5.21.4 Geodesics on the sphere

On a sphere the shortest path between two points is the arc of the great circle through them, and the conditional path of Fisher-Flow is that arc traversed at constant speed,

$$x_t = \exp_{x_0}(t \log_{x_0}(x_1))$$

"The sphere point x_t lies a fraction t along the geodesic from source x_0 to data point x_1. We obtain it by scaling the initial tangent direction $\log_{x_0}(x_1)$ by t, then applying the exponential map at x_0."

- $\log_x(y)$ is the tangent vector at x that reaches y in unit time, with length equal to the arc length from x to y.
- $\exp_x(w)$ is the point reached by moving along the sphere from x in the direction w for a distance $\|w\|$.
- x_0 and x_1 are the source and data points, both on the unit sphere.

Both maps are elementary on a sphere,

$$\exp_x(w) = \cos(\|w\|)\, x + \sin(\|w\|)\, \frac{w}{\|w\|}, \qquad \log_x(y) = \frac{\arccos\langle x, y\rangle}{\sqrt{1 - \langle x, y\rangle^2}}\, (y - \langle x, y\rangle\, x),$$

"The exponential map at x in the direction w equals the cosine of the length of w times x, plus the sine of that length times the unit vector along w; the logarithm map from x to y equals the angle between them, divided by the sine of that angle, times the component of y orthogonal to x."

The cosine-scaled point x and sine-scaled orthogonal unit vector sum to a unit vector, so the exponential map remains on the sphere. In the logarithm map, $y - \langle x, y\rangle x$ is the component of y orthogonal to x. Rescaling it gives a tangent vector whose length equals the angle.

To compute the conditional velocity, divide the logarithm map toward the data point by the remaining time, as with displacement in Chapter 2,

$$\boxed{u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1 - t}}$$

"The conditional velocity at the sphere point x_t is the tangent vector $\log_{x_t}(x_1)$ toward the data endpoint x_1, divided by the remaining time $1 - t$."

The logarithm map has length equal to the remaining arc, which is $(1 - t)$ times the total arc, so after the division the speed is constant along the path. For the base of Section 5.21.3 that speed is 0.743 at every time, the same number as half the Fisher-Rao distance. Chapter 2's velocity, the displacement divided by the time remaining, is this formula with straight lines in place of arcs.

At $t = 0.5$ the midpoint of the arc between x_0 and x_1 is the normalized sum $(x_0 + x_1)/\|x_0 + x_1\|$, which equals

$$x_{0.5} = (0.780,\ 0.361,\ 0.361,\ 0.361) \quad \Longrightarrow \quad p_{0.5} = (0.609,\ 0.130,\ 0.130,\ 0.130),$$

corresponding to probability 0.609 on A and equal probabilities on the other letters. The logarithm map to x_1 is $(0.2323, -0.1672, -0.1672, -0.1672)$, with length 0.3713, half the total arc 0.7425.

Dividing by the remaining time $1 - t = 0.5$ gives velocity magnitude 0.743. For an exponential-map step of size $\Delta = 0.1$,

$$x_{t+\Delta} = \exp_{x_t}\Big(\frac{\Delta}{1-t}\,\log_{x_t}(x_1)\Big), \qquad \Delta = 0.1,$$

"To obtain the next sphere point, multiply the tangent direction $\log_{x_t}(x_1)$ by the step size Δ divided by the remaining time $1 - t$, then apply the exponential map at the current point x_t."

The factor $\Delta/(1-t) = 0.2$ covers a fifth of the remaining arc, an angle of 0.0743 radians, and lands at

$$x_{0.6} = (0.824,\ 0.327,\ 0.327,\ 0.327) \implies p_{0.6} = (0.680,\ 0.107,\ 0.107,\ 0.107),$$

"The step lands at 0.824 in the A coordinate and 0.327 in the other three, which squares to the distribution 0.680 for A and 0.107 for each of the rest."

The new point has length one to ten decimals, because the step followed the sphere. A straight Euclidean step of the same size would land off the sphere and require renormalization, and with the exponential map we avoid the error that renormalization would introduce.

5.21.5 The tangent projection and the regression loss

A linear network output can be any vector in $\mathbb{R}^K$, but a sphere velocity must be orthogonal to the current point. We enforce this by subtracting the output's component along x,

$$v_\theta(t,x) = \tilde{v}_\theta(t,x) - \big\langle x,\ \tilde{v}_\theta(t,x)\big\rangle\, x$$

"The tangent velocity $v_\theta(t,x)$ equals the raw prediction $\tilde{v}_\theta(t,x)$ minus its radial component, the inner product $\langle x, \tilde{v}_\theta(t,x)\rangle$ multiplied by the current sphere point x."

The result is orthogonal to x by construction, since $\langle x, v_\theta\rangle = \langle x, \tilde{v}_\theta\rangle - \langle x, \tilde{v}_\theta\rangle\|x\|^2$ and $\|x\| = 1$. At the midpoint $x = (0.780, 0.361, 0.361, 0.361)$, a raw output of $(0.3, -0.2, 0.5, 0.1)$ has inner product

$$\langle x, \tilde{v}_\theta\rangle = 0.780(0.3) + 0.361(-0.2) + 0.361(0.5) + 0.361(0.1) = 0.378,$$

and subtracting $0.378\,x$ from the raw output leaves

$$\begin{aligned} v_\theta &= (0.3,\ -0.2,\ 0.5,\ 0.1) - 0.378\,(0.780,\ 0.361,\ 0.361,\ 0.361) \\ &= (0.005,\ -0.337,\ 0.363,\ -0.037), \end{aligned}$$

whose inner product with x is zero to machine precision. We can use any architecture, because this projection replaces a restriction on the output layer.

In training we regress the projected output onto the geodesic target,

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\,(x_0,x_1),\,x_t}\left\| v_\theta(t, x_t) - \frac{\log_{x_t}(x_1)}{1-t} \right\|^2$$

"The flow-matching loss $\mathcal{L}(\theta)$ averages the squared distance between the predicted tangent velocity $v_\theta(t, x_t)$ and the target $\log_{x_t}(x_1)/(1-t)$, the remaining geodesic displacement divided by remaining time, over times t, source-data pairs (x_0, x_1), and their interpolated states x_t."

We average the loss over time t, source-data pairs (x_0, x_1) from the coupling, and points x_t along their arcs. We train by regression on velocities without a denoiser. The squared-loss minimizer is the target's conditional mean, so the learned field averages conditional velocities over pairs passing through x_t. This is the same posterior averaging used for jump and Dirichlet paths.

5.21.6 Choosing the coupling

We construct the path from a pair (x_0, x_1) and can choose any source distribution for x_0. The Dirichlet path uses a uniform Dirichlet source; the masked path uses the mask token. For Fisher-Flow, a learned source resembling the data can shorten the arcs between paired points.

Independently pairing source x_0 and data x_1 can produce intersecting arcs. At a crossing, the two regression targets differ, so their average minimizes squared error. The fitted field then generates a trajectory that bends away from both arcs, as with the straight-line paths in $\mathbb{R}^d$ from Chapter 2.

If we pair the samples by an optimal transport plan, the crossings disappear,

$$\pi \;=\; \arg\min_{\pi'} \; \mathbb{E}_{(x_0,x_1)\sim\pi'}\left[\, d^2(x_0, x_1) \,\right]$$

"The coupling π is the joint distribution over source and data pairs that minimizes the expected squared geodesic distance, among all couplings with the given marginals."

Under that plan the arcs do not cross, every point of the path belongs to one pair, and the collection of paths has the least kinetic energy. In practice we solve the plan on each minibatch with Sinkhorn's algorithm [Sinkhorn, 1964], which costs a few iterations on a matrix of pairwise distances and is negligible beside the forward pass.

Take three source bases centered on A, C, and G, each with probability 0.4672 on its selected letter and 0.1776 on the others. Pair them with three smoothed data one-hots ordered A, G, and C, each with probability 0.6040 on its selected letter and 0.1320 on the others. Computing squared Fisher-Rao distances gives the cost matrix

$$C = \begin{pmatrix} 0.076 & 0.963 & 0.963 \\ 0.963 & 0.963 & 0.076 \\ 0.963 & 0.076 & 0.963 \end{pmatrix},$$

with 0.076 wherever a source and a target favor the same base and 0.963 wherever they do not. If we pair the samples in the order they arrive we get the diagonal, $0.076 + 0.963 + 0.963$, and the best pairing matches each source to the target that shares its letter,

$$\text{pairing in order: } 2.001, \qquad \text{optimal pairing: } 0.227$$

"Pairing the samples in the order they arrive costs 2.001 in total, and the optimal pairing costs 0.227."

The mean arc length decreases from 0.746 to 0.275, reducing the target velocities to roughly one third of their previous magnitude and making them more consistent across pairs. We obtain this reduction by changing the coupling, which we can do for any path built from pairs.

5.21.7 Fisher-Flow in code

We can implement the construction in about twenty lines. First, define the two coordinate maps between the simplex and sphere.

```
def to_sphere(p):
    return p.clamp(min=1e-8).sqrt()             # simplex -> sphere

def to_simplex(x):
    return x ** 2                                # sphere -> simplex
```

Take square roots coordinatewise to map probabilities over A, C, G and T to a unit vector. Square the coordinates to map back. We clamp at 10^{-8} to keep zero probabilities away from the boundary, where the Fisher-Rao metric is undefined. This serves the same purpose as smoothing the data corner to $(0.91, 0.03, 0.03, 0.03)$ in the example.

Next, implement the logarithm and exponential maps for motion on the sphere.

```
def log_map(x, y):
    c = (x * y).sum(-1, keepdim=True).clamp(max=1 - 1e-6)
    v = y - c * x                                # orthogonal to x
    return v * torch.arccos(c) / v.norm(dim=-1, keepdim=True)

def exp_map(x, w):
    n = w.norm(dim=-1, keepdim=True).clamp(min=1e-8)
    return torch.cos(n) * x + torch.sin(n) * (w / n)
```

In `log_map`, we compute the inner product $\langle x, y\rangle$ in `c` and clamp it below one so that arccos remains differentiable and the orthogonal component is nonzero. We store that component, $y - \langle x, y\rangle x$, in `v`. Normalizing `v` and multiplying by $\arccos c$ gives a vector whose length equals the arc from x to y. This rescaling equals multiplication by $\arccos\langle x, y\rangle / \sqrt{1 - \langle x, y\rangle^2}$, since $\|v\| = \sqrt{1 - \langle x, y\rangle^2}$ for unit-sphere points.

In `exp_map`, we store the tangent-vector length in `n` and clamp it away from zero for safe division on small steps. We multiply the current point by the cosine of this length and the unit vector along w by its sine. Their sum has squared length $\cos^2 n + \sin^2 n = 1$, so the output lies on the sphere.

We can then implement the conditional path and loss.

```python
def geodesic(x0, x1, t):
    return exp_map(x0, t * log_map(x0, x1))      # the conditional path

def loss(model, x0, x1, t):
    xt = geodesic(x0, x1, t[:, None, None])
    u = log_map(xt, x1) / (1 - t[:, None, None])
    raw = model(xt, t)
    v = raw - (xt * raw).sum(-1, keepdim=True) * xt
    return ((v - u) ** 2).sum(-1).mean()
```

In `geodesic`, we compute x_t from Section 5.21.4 with one exponential-map call. In `loss`, we first place each sample on its arc at the sampled time. The two `None` indices broadcast one time per sequence across positions and letters. We then divide the remaining arc by the remaining time to obtain the velocity target. After calling the network, we subtract the component of its output along `xt` to obtain a tangent vector. Finally, we sum squared errors over the K letters and average over positions and batch samples.

These functions apply to any alphabet and any `model` architecture. The four geometry functions let us use the flow-matching procedure from Chapter 2 on the sphere.

5.22 Gumbel-Softmax Flow Matching

Sampling with the Dirichlet path requires a numerical Beta-function derivative at every step. Fisher-Flow requires exponential and logarithm maps on the sphere. We can also construct a simplex path by continuously relaxing the categorical sampling operation.

5.22.1 The Gumbel-max trick

We can sample a categorical distribution with a deterministic function of noise. Add independent Gumbel noise to each log probability, then select the largest value,

$$g_k \sim \text{Gumbel}(0,1), \qquad \arg\max_k \left[\log \pi_k + g_k\right] \;\sim\; \text{Cat}(\pi)$$

"Adding independent Gumbel noise g_k to each log probability $\log \pi_k$ and taking the index of the largest sum produces a sample from the categorical distribution π."

A Gumbel variable is the negative logarithm of an exponential clock: if $E \sim \text{Exp}(1)$ then $\mathbb{P}(-\log E \le x) = \mathbb{P}(E \ge e^{-x}) = e^{-e^{-x}}$, which is the Gumbel distribution function. The trick is therefore Chapter 4's race between exponential clocks, written with logarithms so that the winner is an argmax and the rates are logits. A quick check with four letters and $\pi = (0.5, 0.2, 0.2, 0.1)$ over 400 000 draws returns the frequencies (0.4996, 0.2005, 0.1998, 0.1000).

For a fixed noise draw, changing the logits changes the sample deterministically. Small logit changes usually preserve the largest entry; sufficiently large changes select a different letter.

5.22.2 The softmax relaxation and a cooling temperature

If we replace the argmax by a softmax, the discrete winner becomes a point of the simplex,

$$x = \text{softmax}\Big(\frac{\log \pi + g}{\tau}\Big), \qquad x_k = \frac{\exp\left((\log \pi_k + g_k)/\tau\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\tau\right)}$$

"The relaxed sample x applies softmax to the log probabilities $\log \pi$ plus Gumbel noise g, divided by temperature τ. Its coordinate x_k is the exponentiated noisy log probability of letter k, normalized over all letters."

- $\tau > 0$ is the temperature, which sets how sharply the softmax concentrates on the largest noisy logit.
- As $\tau \to 0$ the vector approaches the one-hot of the argmax, recovering the Gumbel-max trick exactly.
- As τ grows the vector approaches the uniform vector at the center of the simplex.

Varying temperature moves the relaxed point continuously between the simplex center and a corner. In Gumbel-Softmax Flow Matching [Tang et al., 2025b], we decrease temperature with an exponential time schedule,

$$\tau(t) = \tau_{\max}\, e^{-\lambda t}, \qquad \tau_{\max} = 10, \quad \lambda = 3$$

"The temperature $\tau(t)$ at time t equals the initial temperature $\tau_{\max} = 10$ multiplied by the exponential of minus the cooling rate $\lambda = 3$ times t."

At $t = 0$, temperature 10 gives nearly equal probability to every letter. At $t = 1$, it decreases to $10e^{-3} = 0.498$, concentrating most probability on one letter. The temperature remains positive, so the point stays inside the simplex and its density remains defined. The Dirichlet path also retains this property.

5.22.3 The annealed path toward a data token

To condition the path on a data token, add its one-hot vector to the logits and apply the cooling schedule,

$$\psi_t(x_1 = \mathbf{e}_k) = \text{softmax}\Big(\frac{\delta_{\cdot k} + g/\beta}{\tau(t)}\Big)$$

"The relaxed token $\psi_t(x_1 = \mathbf{e}_k)$ at time t is the softmax of the one-hot vector for data token k plus the fixed Gumbel noise g scaled by $1/\beta$, with the sum divided by the temperature $\tau(t)$."

- $\delta_{\cdot k}$ is the one-hot vector at the data letter k, serving as the logits.
- $g \in \mathbb{R}^K$ is one Gumbel draw, held fixed for the whole trajectory.
- $\beta > 0$ scales the noise down and is distinct from the schedule parameters β_t. A larger β makes paths from a given data token spread less.

Hold the Gumbel noise fixed throughout the trajectory. Only $\tau(t)$ then changes with time, producing a smooth curve from near the simplex center toward corner $\mathbf{e}_k$. The endpoint is approximate: at $t = 1$, its largest coordinate approaches one without reaching it.

Let's anneal a DNA base toward G. Fix the scaled Gumbel noise in A, C, G, T order at $g/\beta = (0, -0.45, 0.40, -0.05)$. With $\beta = 2$, this corresponds to $g = (0, -0.90, 0.80, -0.10)$. Add the data indicator at G to obtain logits $\ell = (0, -0.45, 1.40, -0.05)$. Evaluating the path at four times gives

$$\begin{array}{lll} t = 0.0: & \tau = 10.00 & (0.2438,\ 0.2331,\ 0.2805,\ 0.2426) \\ t = 0.5: & \tau = 2.231 & (0.2142,\ 0.1751,\ 0.4012,\ 0.2095) \\ t = 0.9: & \tau = 0.672 & (0.0955,\ 0.0489,\ 0.7669,\ 0.0887) \\ t = 1.0: & \tau = 0.498 & (0.0528,\ 0.0214,\ 0.8781,\ 0.0477) \end{array}$$

At time zero the four coordinates are near one quarter each, at $t = 0.5$ the G coordinate has risen to 0.4012, at $t = 0.9$ it holds 0.7669, and at $t = 1$ it holds 0.8781. We select the final token with an argmax. The other three letters shrink at different speeds, because each has its own Gumbel draw. C starts lowest at 0.2331 and ends lowest at 0.0214; A and T, whose noise values are close together, stay close to each other throughout.

5.22.4 Differentiating the path: the replicator equation

The conditional velocity is the time derivative of that curve. We can write $\ell_i = \delta_{ik} + g_i/\beta$ for the fixed logits. The only time dependence is in $1/\tau(t)$, and

$$\frac{d}{dt}\,\frac{\ell_i}{\tau(t)} = \ell_i\,\frac{\lambda}{\tau(t)},$$

since $\tau(t) = \tau_{\max} e^{-\lambda t}$ makes $1/\tau$ grow at relative rate λ. Composing with the softmax Jacobian $\partial x_i/\partial(\text{logit}_j) = x_i(\delta_{ij} - x_j)$ and using $\sum_j x_j = 1$,

$$u_{t,i}(x \mid x_1 = \mathbf{e}_k) = \frac{\lambda}{\tau(t)}\, x_i \sum_j x_j \Big[(\delta_{ik} + g_i/\beta) - (\delta_{jk} + g_j/\beta)\Big]$$

"The conditional velocity component $u_{t,i}$ multiplies the cooling rate λ divided by temperature $\tau(t)$, the current mass x_i, and the difference between letter i's noisy logit and the average noisy logit weighted by masses x_j. Each noisy logit is the data-token indicator plus Gumbel noise scaled by $1/\beta$."

The bracket is ℓ_i minus the mass-weighted average of all the logits, so a coordinate grows when its logit is above that average and shrinks when it is below. Summing over i cancels the two averages against each other, so the coordinates of the velocity sum to zero and the velocity is tangent to the simplex.

That form is the replicator equation of evolutionary dynamics, $\dot{x}_i = x_i(f_i - \bar{f})$ with $\bar{f} = \sum_j x_j f_j$, where the fitness of a type is $f_i = \lambda \ell_i/\tau(t)$ and the coordinates are population shares. The

identification is exact, and it accounts for the multiplicative factor x_i, since a letter with no mass has no velocity in the same way that a type with no population cannot grow.

We now evaluate this on the annealed base at $t = 0.5$, where $x = (0.2142, 0.1751, 0.4012, 0.2095)$ and $\lambda/\tau(t) = 3/2.231 = 1.345$. The mass-weighted average of the logits is

$$\bar{\ell} = 0.2142(0) + 0.1751(-0.45) + 0.4012(1.40) + 0.2095(-0.05) = 0.4724,$$

and multiplying each $x_i(\ell_i - \bar{\ell})$ by 1.345 produces the velocity $(-0.136, -0.217, 0.500, -0.147)$. Only G has a logit above the average, so only G grows, and C loses fastest because its logit is furthest below.

5.22.5 The inference-time velocity

During inference, we have the current point but not its original noise draw. If we set $g = 0$ in the formula above, then $\ell_i = \delta_{ik}$, the bracket becomes $\delta_{ik} - x_k$, and the velocity becomes a multiple of the direction toward the data corner,

$$u_t(x \mid x_1 = \mathbf{e}_k) = \frac{\lambda}{\tau(t)}\, x_k\, (\mathbf{e}_k - x)$$

"With Gumbel noise set to zero, the conditional velocity equals the cooling rate λ divided by temperature $\tau(t)$, times the data-coordinate mass x_k and the displacement $\mathbf{e}_k - x$ toward data corner $\mathbf{e}_k$."

The two expressions agree coordinate by coordinate: for $i = k$ both read $(\lambda/\tau)x_k(1 - x_k)$, and for $i \neq k$ both read $-(\lambda/\tau)x_i x_k$. The k-th component therefore has the factor $x_k(1-x_k)$, which is small near the corner and small when the data token has almost no mass yet, with its maximum halfway between. The velocity vanishes identically when $x_k = 0$, so a letter that has lost all of its mass cannot come back, which is the one place where this path behaves like an absorbing process. The prefactor $\lambda/\tau(t)$ grows as t approaches one, so the flow accelerates toward the end of the trajectory, compensating for the shrinking factor $x_k(1 - x_k)$ once x_k is large.

Let's take one step on the annealed base at $t = 0.5$, with $x = (0.2142, 0.1751, 0.4012, 0.2095)$ and $\lambda/\tau(t) = 1.345$,

$$u_t(x \mid x_1 = \mathbf{e}_{\mathrm{G}}) = 1.345 \times 0.4012 \times (\mathbf{e}_{\mathrm{G}} - x) = (-0.116,\ -0.094,\ 0.323,\ -0.113),$$

"The velocity toward G equals 1.345 times 0.4012 times the direction to the G corner, which is 0.323 for G, -0.116 for A, -0.094 for C and -0.113 for T."

The coordinates sum to zero, so an Euler step preserves the sum exactly. A large step can still drive a coordinate below zero, so the sampler projects onto the simplex after each step. The noisy velocity of Section 5.22.4 at the same point is $(-0.136, -0.217, 0.500, -0.147)$, about half again as large, because the noise pushes mass between the three non-target letters as well as toward G.

5.22.6 Training and the marginal velocity

For training we use cross-entropy on the data token, as we did for the mixture paths and for the Dirichlet path,

$$\mathcal{L} = \mathbb{E}\big[-\log\langle x_\theta(x_t, t),\, x_1\rangle\big]$$

"The denoising loss $\mathcal{L}$ averages the negative logarithm of the probability assigned to the true data token x_1, obtained by taking its one-hot inner product with the predicted probability vector $x_\theta(x_t, t)$ at noisy state x_t and time t."

We input a simplex point to the denoiser and obtain a probability distribution over letters. The training gradient does not require the conditional velocity. During sampling, we compute the posterior-averaged velocity,

$$u_t^\theta(x_t) = \sum_{k=1}^{K} u_t(x \mid x_1 = \mathbf{e}_k)\, \langle x_\theta(x_t, t),\, \mathbf{e}_k\rangle$$

"The predicted marginal velocity $u_t^\theta(x_t)$ averages each conditional velocity toward data corner $\mathbf{e}_k$, weighted by the probability of letter k in the denoiser output $x_\theta(x_t, t)$."

Given a data token, we can compute its conditional velocity exactly. We estimate the unknown token with the denoiser and average over its output probabilities, as for jump and Dirichlet paths. Training by cross-entropy and then averaging gives the same gradient as direct regression on the true marginal velocity [Tang et al., 2025b].

Let's run one Euler step late in the path. At $t = 0.8$ the base is at $x = (0.138,\, 0.084,\, 0.647,\, 0.131)$ and the denoiser outputs $(0.05,\, 0.10,\, 0.70,\, 0.15)$ over A, C, G, T. The temperature is $\tau(0.8) = 10e^{-2.4} = 0.9072$, so $\lambda/\tau = 3.307$. Each conditional velocity has the coefficient $(\lambda/\tau)\, x_k\, \hat{p}_k$, which equals 0.0228 for A, 0.0278 for C, 1.4978 for G and 0.0650 for T. Collecting the terms,

$$\begin{aligned} u_t^\theta(x_t) &= (-0.200,\ -0.108,\ 0.454,\ -0.146), \\ x_t + 0.01\, u_t^\theta &= (0.1360,\ 0.0829,\ 0.6515,\ 0.1295), \end{aligned}$$

"The marginal velocity is -0.200 for A, -0.108 for C, 0.454 for G and -0.146 for T, and one step of size 0.01 moves the G coordinate from 0.647 to 0.6515."

The coordinates still sum to one after the step. T has the second-largest denoiser probability, 0.15, but its coordinate decreases. Each conditional velocity is weighted by the existing mass x_k, which is only 0.131 for T compared with 0.647 for G. Late in the path, the largest coordinate also contributes most strongly to the velocity toward its corner.

5.22.7 The same path in log coordinates

We can also write the path in log coordinates, where the density of a Gumbel-softmax sample is the ExpConcrete distribution and the score has a closed form. Writing x now for the log coordinate and K for the alphabet size,

$$\nabla_{x_i} \log p_t(x \mid x_1 = \mathbf{e}_k) = -\tau(t) + \tau(t)\, K \cdot \mathrm{softmax}\left(\delta_{ik} - \tau(t)\, x_i\right)$$

"The log-density gradient at coordinate x_i equals minus the temperature $\tau(t)$, plus $\tau(t)$ times the alphabet size K and the softmax component computed from the data indicator δ_{ik} minus the scaled log coordinate $\tau(t)x_i$."

The softmax coordinates sum to one, so the positive score terms sum to $K\tau$. The K negative terms sum to $-K\tau$, leaving zero total. The score is therefore tangent to its constraint surface, just as the velocity is tangent to the simplex. We can use this closed-form score for score-based sampling and classifier gradients, as with the Dirichlet score in Section 5.20.6.

5.23 Three geometries

We can generate tokens on three state spaces. On a discrete alphabet, we describe jumps through rates. On the simplex, the Dirichlet velocity has a speed obtained from a Beta-function derivative. On the sphere, we follow great-circle arcs using the Fisher-Rao geometry. The Gumbel-softmax path also lies on the simplex, with a curve obtained by relaxing categorical sampling.

For each construction, we choose a conditional source-to-data path, derive its velocity and average conditional velocities over possible data tokens. For jump, Dirichlet and Gumbel-softmax paths, we train the denoiser with cross-entropy and then compute the posterior average. In Fisher-Flow, we regress directly on geodesic velocities and project the output onto the tangent space.

Table 5.4: The four constructions of this chapter, by state space. K is the alphabet size, $\mathbf{e}_i$ the corner at the data letter and $\tilde{I}$ the derivative of the regularized incomplete beta function with respect to its first parameter. The last column names what one sampler step costs beyond the network evaluation itself.

	state per position	conditional path	network target	extra cost
Jump rates	one of K letters	$(1-\kappa_t)\delta_{x_0} + \kappa_t \delta_{x_1}$	$p_{1\mid t}$	categorical draw
Dirichlet	Δ^{K-1}	$\mathrm{Dir}(\mathbf{1} + t\,\mathbf{e}_i)$	$p_{1\mid t}$	K values of $\tilde{I}$
Fisher-Flow	octant of S^{K-1}	great-circle arc	velocity	one exponential map
Gumbel-softmax	Δ^{K-1}	$\mathrm{softmax}\big((\mathbf{e}_k + g/\beta)/\tau(t)\big)$	$p_{1\mid t}$	simplex projection

We compare the four constructions in Table 5.4. ODE integration on the simplex and sphere gives a deterministic map from source to sample. Classifier guidance, optimal-transport couplings and single-step distillation use that map. A jump process has a random source-to-sample map, so these constructions do not apply in the same form.

Alphabet size affects the choice of path. Under a Dirichlet path, the data coordinate x_i follows $\mathrm{Beta}(t+1, K-1)$, with mean $(t+1)/(t+K)$. At the inference horizon $t = 8$, this mean is $9/12 = 0.75$ for DNA and $9/520 = 0.0173$ for an alphabet of 512 letters. Reaching mean 0.75 at $K = 512$ would

require $t = 1532$. For Gumbel-softmax, temperature $\tau(t)$ scales every logit equally without this dependence on K. On large alphabets, it reaches the data letter within the unit interval, while the required Dirichlet horizon grows with K [Tang et al., 2025b]. Below a few dozen letters, the Dirichlet horizon $t = 8$ is sufficient and the comparison favors the Dirichlet path. On the sphere, the Fisher-Rao distance between any two points of Δ^{K-1} is at most π, independently of K. We can further shorten arcs with the optimal-transport coupling from Section 5.21.6.

These models generate samples from the data distribution. For design tasks, we may also need a chosen promoter activity, binding to a specific target, or several sequence properties together. We can guide sampling toward those properties and reshape the paths to reduce the required number of steps.

5.24 The target distribution

When we design a peptide, we often need it to bind a chosen target, dissolve in water, resist proteases, and survive in serum. The training set may contain examples with individual properties. The subset satisfying all four is usually empty. We therefore keep the trained generative model and adjust its sampler using predictions from a property model.

Let c denote the desired property, such as binding a target above a specified affinity threshold, or let y be a sequence label. We want to sample sequences conditioned on that event. Applying Bayes' rule to the data distribution gives

$$p(x_1 \mid y) = \frac{p(y \mid x_1)\, p(x_1)}{\sum_{x'} p(y \mid x')\, p(x')}.$$

"The conditional probability $p(x_1 \mid y)$ of a data sequence x_1 given the property y equals the likelihood $p(y \mid x_1)$ of the property under that sequence, times the model probability $p(x_1)$, divided by the sum of that same product over every sequence x' in the state space."

We estimate the likelihood $p(y \mid x_1)$ with a classifier or regressor trained on labelled sequences, and use the flow model for $p(x_1)$. The denominator sums over $|\mathcal{V}|^L$ sequences. For a peptide with 100 residues and 20 possible amino acids, this means 20^{100} terms, so we cannot evaluate the normalizer directly.

We can avoid computing the normalizer by using ratios between states, in which shared constants cancel. Discrete Guidance [Nisonoff et al., 2025] uses this cancellation to express the conditioned transition rates without summing over all sequences.

5.25 Guidance as a tilt of the rates

Start from the trained, unguided jump rate $u_t(y, x)$ between current sequence x and destination y. To condition on a terminal sequence with property c, multiply this rate by the ratio of property probabilities at the two states,

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x.$$

"The guided rate $u_t(y \mid x, c)$ from the state x to a different state y equals the probability $p(c \mid y, t)$ of the property given the destination, divided by the probability $p(c \mid x, t)$ of the property given the current state, times the unguided rate $u_t(y, x)$."

- c is the property we condition on, an event on the terminal sequence X_1.
- $p(c \mid x, t)$ is the probability of that event given that the process is at the state x at time t. It is a function of a noisy intermediate state, not of a clean sequence.
- $u_t(y, x)$ is the velocity of the trained flow, the rate of jumping from x to y at time t, in units of probability per unit time.

If a destination has a higher predicted property probability than the current state, its ratio exceeds one and its jump rate increases. A lower-scoring destination has a ratio below one and a reduced rate. Multiplication preserves zero rates, so guidance changes probabilities within the trained model's support without adding destinations.

We evaluate the ratio at intermediate times t, so we train the property model on noisy sequences from the same path and use partly noisy inputs during sampling. The formula specifies only off-diagonal entries. After multiplying the rates by the ratios, we reset the diagonal to minus the row sum so the resulting rates satisfy both velocity conditions.

5.25.1 The guidance value function

Write the property probability as $h_t(x)$. We evaluate this same function at the current state and destination in the rate ratio:

$$h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x \,\big].$$

"The value $h_t(x)$ is the probability of the property c given that the process sits at the state x at time t, which is the expectation of the terminal property likelihood $p(c \mid X_1)$ under the unguided process started from x at time t."

Apply the tower property by first conditioning on the terminal state. Evaluate the property likelihood there, then average over terminal states reachable from current state x. Thus $h_t(x)$ is the probability of eventually reaching a sequence with the requested property. At $t = 1$, it equals the terminal likelihood $h_1(x) = p(c \mid x)$.

The conditional expectation of a fixed terminal quantity satisfies the process's backward equation, so h_t is space-time harmonic. We used this property to derive Doob's h-transform in Chapter 4. In destination-first notation, the transformed rate is

$$\boxed{u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x).}$$

"The h-transformed rate $u_t^h(y, x)$ equals the value $h_t(y)$ of the harmonic function at the destination, divided by its value $h_t(x)$ at the current state, times the original rate $u_t(y, x)$."

When h is harmonic for the unguided process, the transformed process is conditioned on the terminal event. The guided sampler is exact in continuous time with the true property probability $p(c \mid x, t)$. Finite Euler steps and property estimates trained on finite noisy datasets introduce approximation error.

Different guidance procedures use different functions in the rate adjustment. Discrete Guidance uses a single-property likelihood for h. In pCoMole, h is the expected exponentiated multi-objective utility subject to a feasibility check. MOG-DFM uses ranks and directions without a joint property model, so its rate adjustment does not give exact conditioning.

5.26 Single-position jumps and the reachable set

To evaluate the tilt we need $p(c \mid y, t)$ at every destination y, and the number of destinations sets the cost of one guided step. The conditional path factorizes over positions, and each position has an independent exponential clock, so the probability that two clocks ring in the same instant is zero. At any one time exactly one position changes, and the reachable set from a sequence of D positions over S letters has size

$$\#\{\text{destinations}\} = D\,(S-1) + 1.$$

"The reachable set has $D(S-1)+1$ states: for each of the D sequence positions, we can substitute any of the other $S-1$ letters, and we also include the current state."

For a peptide of 100 residues over 20 amino acids plus a mask token, $S = 21$ and $D = 100$, so the count is $100 \times 20 + 1 = 2001$. The state space itself has $21^{100} \approx 1.67 \times 10^{132}$ elements. We can guide a single sequence with 2001 property-model evaluations, avoiding the full sum in Bayes' rule.

5.27 Single-base guidance example

We take the mask source, the linear schedule $\kappa_t = t$, and a position that is still masked at $t = 0.6$. The denoiser predicts

$$p_{1|t}(\cdot \mid x) = (0.1,\ 0.2,\ 0.3,\ 0.4)$$

over A, C, G, T, and the unguided unmasking rate into base a is the schedule rate times that probability,

$$u_t(a, m) = \frac{\dot\kappa_t}{1-\kappa_t}\, p_{1|t}(a \mid x) = \frac{1}{1-0.6}\, p_{1|t}(a \mid x) = 2.5\, p_{1|t}(a \mid x).$$

Substituting the four probabilities,

$$u_t(\cdot, m) = (0.25,\ 0.50,\ 0.75,\ 1.00), \qquad \sum_a u_t(a, m) = 2.50.$$

We normalize the unguided rates by their sum. The resulting jump probabilities $(0.10, 0.20, 0.30, 0.40)$ equal the denoiser distribution.

Next, evaluate the property model on each of the four sequences with a base at this position, and on the current masked sequence,

$$p(c \mid \cdot, t) = (0.10,\ 0.35,\ 0.55,\ 0.25) \ \text{ for A, C, G, T,} \qquad p(c \mid x, t) = 0.20 \ \text{ for the masked state.}$$

Dividing each by the value at the current state,

$$\text{ratios} = (0.50,\ 1.75,\ 2.75,\ 1.25), \qquad \text{guided rates} = (0.125,\ 0.875,\ 2.063,\ 1.250).$$

"The likelihood ratios are $0.10/0.20 = 0.50$ for A, $0.35/0.20 = 1.75$ for C, $0.55/0.20 = 2.75$ for G and $0.25/0.20 = 1.25$ for T, and multiplying the unguided rates by them produces $0.25 \times 0.50 = 0.125$, $0.50 \times 1.75 = 0.875$, $0.75 \times 2.75 = 2.063$ and $1.00 \times 1.25 = 1.250$."

Two quantities change. The destination distribution moves from $(0.10, 0.20, 0.30, 0.40)$ to

$$\frac{(0.125,\ 0.875,\ 2.063,\ 1.250)}{4.3125} = (0.029,\ 0.203,\ 0.478,\ 0.290),$$

so G becomes the most likely base at this position. The total exit rate also increases from 2.50 to 4.3125, causing earlier unmasking. Multiplying by the likelihood ratio thus changes both the destination probabilities and jump timing. If most destinations have higher property scores than the current state, the total exit rate increases.

5.27.1 The destination of the guided jump

We can separate the destination probabilities from the exit rate. The denominator $p(c \mid x, t)$ is shared across destinations, so it cancels when we normalize the guided rates and affects only the exit rate. Dividing by the rate sum gives

$$\frac{u_t(a, m)\, p(c \mid a, t)/p(c \mid x, t)}{\sum_{a'} u_t(a', m)\, p(c \mid a', t)/p(c \mid x, t)} = \frac{p_{1|t}(a \mid x)\, p(c \mid a, t)}{\sum_{a'} p_{1|t}(a' \mid x)\, p(c \mid a', t)},$$

where the schedule rate $\dot\kappa_t/(1 - \kappa_t)$ cancels along with the denominator. The right side is Bayes' rule applied at one position, with the denoiser as the prior over the data token and the property model as the likelihood. The sum in it runs over four bases.

The products $p_{1|t}(a \mid x)\, p(c \mid a, t)$ are

$$(0.1 \times 0.10,\ 0.2 \times 0.35,\ 0.3 \times 0.55,\ 0.4 \times 0.25) = (0.010,\ 0.070,\ 0.165,\ 0.100),$$

which sum to 0.345 and normalize to $(0.029, 0.203, 0.478, 0.290)$. That is the guided destination distribution computed above, and it is the exact posterior over the data token given the property. The one-position version of the sum over sequences in Section 5.24 has four terms, and the guided jump samples the conditional distribution exactly.

The same sum also determines the exit rate. The exact value of h_t at the masked state is the posterior average of the property likelihood over the data token,

$$h_t(x) = \mathbb{E}\big[p(c \mid X_1) \mid X_t = x\big] = \sum_a p_{1|t}(a \mid x)\, p(c \mid a, t) = 0.345,$$

and substituting that value in place of the 0.20 used above changes every rate by the factor $0.345/0.20 = 1.725$. The guided rates become $(0.072, 0.507, 1.196, 0.725)$, whose sum is 2.500, the unguided exit rate exactly. A short calculation accounts for that,

$$\sum_a \frac{h_t(a)}{h_t(x)}\, \frac{\dot\kappa_t}{1 - \kappa_t}\, p_{1|t}(a \mid x) = \frac{\dot\kappa_t}{1 - \kappa_t}\, \frac{\sum_a p_{1|t}(a \mid x)\, h_t(a)}{h_t(x)} = \frac{\dot\kappa_t}{1 - \kappa_t},$$

because the numerator equals $h_t(x)$. With an exact property model, guidance preserves the unmasking time and changes the destination. If the predicted value at the current state differs from the posterior average at destinations, the exit rate changes. Here the current-state estimate 0.20 is below 0.345, causing early unmasking.

5.28 The guidance strength

The derived rate adjustment has no free parameter. Property estimates learned from finite data can give weak guidance, so we introduce a strength parameter by raising the likelihood ratio to a power,

$$u_t^{(\gamma)}(y \mid x,\, c) = \Big[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\Big]^{\gamma} u_t(y, x).$$

"The guided jump rate multiplies the original rate $u_t(y, x)$ by the destination-to-current property-probability ratio raised to the guidance strength γ."

At $\gamma = 0$ the exponent reduces every ratio to one and the rates are the unguided ones. At $\gamma = 1$ they are the conditional process of Section 5.25. At any other value the sampler targets a tempered version of the conditional distribution, because h^{γ} is not harmonic for the unguided process when $\gamma \neq 1$, and the exactness argument of Section 5.25.1 no longer applies. The strength γ is a hyperparameter chosen per target, and a value tuned on one property and one protein does not transfer to another.

Let's go back to the worked base at $t = 0.6$ and set the guidance strength to $\gamma = 2$. The squared ratios are $(0.25, 3.063, 7.563, 1.563)$, so the guided rates become

$$u_t^{(2)}(\cdot, m) = (0.0625,\ 1.531,\ 5.672,\ 1.563), \qquad \sum_a u_t^{(2)}(a, m) = 8.828,$$

and the destination distribution is $(0.007, 0.173, 0.642, 0.177)$. If we compare the three columns at $\gamma = 0$, 1 and 2, the probability of G climbs from 0.30 to 0.48 to 0.64 while the exit rate climbs from 2.50 to 4.31 to 8.83. Increasing guidance strength changes both the destination probabilities and the unmasking rate.

5.28.1 The classifier-free form

For classifier-free guidance, we train property-conditioned and unconditional velocities, then combine their rates geometrically,

$$u_t^{(\gamma)}(y \mid x, c) = u_t(y \mid x, c)^{\gamma}\ u_t(y, x)^{1-\gamma}.$$

"The guided rate is the conditional model's rate $u_t(y \mid x, c)$ raised to the power γ, times the unconditional rate $u_t(y, x)$ raised to the power one minus γ."

Taking logarithms gives $\log u^{(\gamma)} = \gamma \log u_c + (1 - \gamma) \log u_u$, the continuous-diffusion classifier-free expression applied to rates. To check its units, write a short-time transition probability as $u\Delta t$ to first order. The geometric mixture is $(u_c \Delta t)^{\gamma}(u_u \Delta t)^{1-\gamma} = u_c^{\gamma} u_u^{1-\gamma} \Delta t$. The powers of Δt sum to one, so the coefficient remains a rate.

Substituting the exact conditional velocity from Section 5.25 gives

$$\Big(\tfrac{h_t(y)}{h_t(x)} u_t(y, x)\Big)^{\gamma} u_t(y, x)^{1-\gamma} = \Big(\tfrac{h_t(y)}{h_t(x)}\Big)^{\gamma} u_t(y, x),$$

which is the γ-tilted rate written above. The classifier-free form and the classifier-guided form are the same one-parameter family, reached from two different sets of trained networks.

5.29 A first-order expansion of the log ratio

Exact guidance requires one property evaluation per destination, or 2001 for the 100-residue peptide in Section 5.26. To reduce this cost, we can approximate the ratios using derivatives at the current sequence.

We embed a sequence as a concatenation of one-hot vectors, so that $x \in \{0,1\}^{D \times S}$ and a single-position change is a sparse difference $y - x$. Expanding the log property probability to first order around the current sequence,

$$\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \;\approx\; (y - x)^\top \nabla_x \log p_\phi(c \mid x, t).$$

"The log ratio of property probabilities between the destination y and the current state x is approximated by the inner product of the one-hot difference $y - x$ with the gradient $\nabla_x \log p_\phi(c \mid x, t)$ of the log property probability at the current sequence, where ϕ are the parameters of the property network."

For a change from letter a to b at position i, the one-hot difference $y - x$ is $+1$ in slot (i, b) and -1 in slot (i, a). Every other entry is zero. The inner product therefore reduces to two gradient coordinates,

$$\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \approx \left[\nabla_x \log p_\phi\right]_{i,b} - \left[\nabla_x \log p_\phi\right]_{i,a}.$$

We compute the full gradient with one forward pass and one backward pass, then subtract two entries for each destination. The number of property-network calls per sampling step decreases from 2001 forward passes to one forward and one backward pass, independently of sequence length and alphabet size.

The approximation is exact when the log property probability is linear in the one-hot encoding, and otherwise it degrades with the size of the change, which here is one position. We go back once more to the masked base at $t = 0.6$ and suppose the gradient coordinates of $\log p_\phi$ at that position are $(-2.10, -1.15, -0.75, -1.30)$ for A, C, G, T and -1.61 at the mask slot. The approximate log ratios are then $(-0.49, 0.46, 0.86, 0.31)$ against the exact values $(-0.693, 0.560, 1.012, 0.223)$ obtained from the logarithms of the five property scores. If we multiply the unguided rates by the exponentials $(0.613, 1.584, 2.363, 1.363)$, we get guided rates $(0.153, 0.792, 1.772, 1.363)$ and a destination distribution $(0.038, 0.194, 0.434, 0.334)$, against the exact $(0.029, 0.203, 0.478, 0.290)$. The ordering of the four bases is unchanged, G is still the preferred destination, and the tilt is softer than the exact one.

The Discrete Guidance authors [Nisonoff et al., 2025] first combined these components for discrete states: noisy-state property training, exact ratios for small alphabets, Taylor approximations for large alphabets, and strength γ. In fixed-backbone inverse folding, they obtained a 94% predicted-stable fraction with the Taylor approximation at $\gamma = 100$, using one forward and one backward pass per step. Exact ratios gave 100% with 2001 property evaluations per step.

5.30 Multi-objective guidance

For therapeutic peptides, we often score binding affinity, solubility, non-fouling, hemolysis and serum half-life. Separate models trained on separate datasets predict these properties. The h-transform in Section 5.25.1 requires one function h_t, corresponding to the expected likelihood of their joint event. Estimating it would require sequences labelled for all five properties together, and such a dataset is unavailable.

With several objectives there is also no single best sequence. Let $s_n(x)$ be the score of sequence x on objective n, with larger values better. The set of candidates that cannot be improved on one objective without losing on another is

$$\mathrm{PF} = \{\, x \;:\; \nexists\, x' \text{ with } s_n(x') \geq s_n(x)\ \forall n \text{ and } s_m(x') > s_m(x) \text{ for some } m \,\}.$$

"The Pareto front PF is the set of sequences x for which no other sequence x' is at least as good on every objective n and strictly better on at least one objective m."

We take four candidate peptides scored on two objectives,

$$s(x_1) = (0.8,\ 0.3), \quad s(x_2) = (0.6,\ 0.7), \quad s(x_3) = (0.5,\ 0.2), \quad s(x_4) = (0.6,\ 0.3).$$

Candidate x_3 is dominated by x_4, which matches it or beats it on both coordinates and beats it on the first. Candidate x_4 is dominated by x_1, which ties on the second coordinate and beats it on the first. Neither x_1 nor x_2 dominates the other, so the front is $\{x_1, x_2\}$. With a weight vector ω we can single out one of them: at $\omega = (0.6, 0.4)$ the weighted scores are 0.60 for x_1 and 0.64 for x_2, and at $\omega = (0.8, 0.2)$ they are 0.70 and 0.62. The weights specify the requested trade-off. In MOG-DFM [Chen et al., 2025], we draw ω once per run from a simplex lattice and guide sampling toward the corresponding point on the front.

5.31 MOG-DFM: rank, direction and a cone

Without a joint likelihood, we cannot compute $h_t(y)/h_t(x)$. In MOG-DFM, we construct a rate adjustment from each objective's scores. We compute two quantities for every candidate letter at every position.

First, fix position i and objective n. Write $x^{(i \to y^i)}$ for the sequence obtained by substituting y^i at i. Rank the score change $s_n(x^{(i \to y^i)}) - s_n(x)$ among candidate letters, then divide by alphabet size $|\mathcal{T}|$:

$$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}.$$

"The rank score $I_n(y^i, x)$ of a candidate letter y^i is the rank of its score change on objective n among all candidate letters at that position, with rank one for the smallest change, divided by the alphabet size $|\mathcal{T}|$."

By ranking inside each objective we remove the units. A half-life measured in hours and a hemolysis probability in $[0, 1]$ have changes of incomparable size, and after ranking both land in $\{1/|\mathcal{T}|, \ldots, 1\}$.

To quantify the direction of improvement, collect all N score changes in $\Delta\mathbf{s}(y^i, x) \in \mathbb{R}^N$, then take the inner product with the weight vector,

$$D(y^i, x, \omega) = \Delta\mathbf{s}(y^i, x) \cdot \omega.$$

"The directional score $D(y^i, x, \omega)$ is the inner product of the vector $\Delta\mathbf{s}(y^i, x)$ of score changes with the weight vector ω that encodes the trade-off we asked for."

The two scores combine into one number per candidate,

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\tfrac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big],$$

and that number becomes an exponential tilt on the pretrained rate,

$$u^i_{\text{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big).$$

"The combined score $\Delta S(y^i, x, \omega)$ adds the z-scored rank average, with objective importances i_n, to the directional score weighted by λ and z-scored across candidate letters. The guided rate multiplies the pretrained rate $u^i_t(y^i, x)$ by the rate scale β and $\exp(\Delta S)$."

Here λ weights directional magnitudes relative to ranks, and β scales the exit rate. These differ from the redraw probability λ_t in Section 5.8 and metric-path parameter β_t in Section 5.16.4. The exponential is positive, so the adjusted off-diagonal rates remain nonnegative. We reset the diagonal to minus the row sum to obtain a valid velocity. Because the multiplier is not $h_t(y)/h_t(x)$ for one function h_t, the exact-conditioning argument of Section 5.25.1 does not apply.

5.31.1 Cone-based acceptance

After z-scoring, subtracting the mean puts roughly half the candidates above zero. The rate adjustment can therefore favor a candidate even when every candidate moves away from the requested trade-off. Before drawing the jump, we can filter candidates by the angle between their score change and the weight vector,

$$\alpha^i = \arccos\Big(\frac{\Delta\mathbf{s}(y^i, x) \cdot \omega}{\|\Delta\mathbf{s}(y^i, x)\|\,\|\omega\|}\Big) \;\le\; \Phi.$$

"The candidate's alignment angle α^i is the arccosine of its score-change vector's inner product with the requested weight vector ω, divided by their lengths. We retain the candidate when α^i is at most the cone angle Φ."

Proposition 5.7 (Accepted moves improve the weighted objective)**.** *Let $\Phi < \pi/2$ and let a candidate y^i satisfy $\alpha^i \le \Phi$ with $\Delta\mathbf{s}(y^i, x) \neq 0$. Then $D(y^i, x, \omega) = \Delta\mathbf{s}(y^i, x) \cdot \omega > 0$, so the weighted score $\omega \cdot \mathbf{s}$ increases along every accepted move.*

For $\Phi < \pi/2$, the cosine is positive on $[0, \Phi]$, and the inner product is $\cos\alpha^i$ times two positive lengths. Every accepted move therefore increases the weighted objective. This guarantee concerns individual steps and specifies no target distribution.

A fixed cone can reject every candidate if it is too narrow or accept everything if it is too wide. We can adapt its angle during sampling by controlling its logarithm,

$$\Phi_{t+h} = \mathrm{clip}\Big(\Phi_t \exp\left(\eta\left(\bar{r}_t - \tau\right)\right),\ \Phi_{\min},\ \Phi_{\max}\Big).$$

"The next cone angle Φ_{t+h} equals the current angle Φ_t multiplied by the exponential of the controller gain η times the rejection-rate error $\bar{r}_t - \tau$, then clipped between the limits $\Phi_{\min}$ and $\Phi_{\max}$."

Here η is the controller step size, distinct from the corrector strength. If the smoothed rejection rate exceeds its target, we widen the cone; if it falls below target, we narrow it. This proportional control on $\log \Phi$ keeps the surviving fraction near $1 - \tau$ without per-target tuning.

5.31.2 Guided-step example

We now take two objectives, a position currently holding A, and the three candidate letters C, G, T. The score changes at this position are

$$\Delta\mathbf{s}(\mathrm{A}) = (0.00,\ 0.00), \qquad \Delta\mathbf{s}(\mathrm{C}) = (+0.20,\ -0.05),$$
$$\Delta\mathbf{s}(\mathrm{G}) = (+0.05,\ +0.15), \qquad \Delta\mathbf{s}(\mathrm{T}) = (-0.10,\ +0.08),$$

with importances $i_1 = 1.2$, $i_2 = 0.8$, weights $\omega = (0.6, 0.4)$ and $\lambda = 1$.

We rank within objective 1, smallest change first: T, A, G, C, so $I_1 = (0.50, 1.00, 0.75, 0.25)$ for A, C, G, T after dividing by the alphabet size 4. Within objective 2 the order is C, A, T, G, so $I_2 = (0.50, 0.25, 1.00, 0.75)$. The importance-weighted average $\frac{1}{2}(1.2I_1 + 0.8I_2)$ is

$$(0.500,\ 0.700,\ 0.850,\ 0.450),$$

with mean 0.625 and population standard deviation 0.1601. Subtracting the mean and dividing by that deviation, the z-scored rank term is

$$\mathrm{Norm}\big[\tfrac{1}{2}(1.2I_1 + 0.8I_2)\big] = (-0.781,\ 0.469,\ 1.406,\ -1.093).$$

The directional scores are $\Delta\mathbf{s} \cdot \omega = (0.000, 0.100, 0.090, -0.028)$, with mean 0.0405 and standard deviation 0.0555, so the z-scored directional term is $(-0.730, 1.072, 0.892, -1.234)$. Adding them at $\lambda = 1$,

$$\Delta S = (-1.511,\ 1.541,\ 2.297,\ -2.327), \qquad \exp(\Delta S) = (0.221,\ 4.667,\ 9.948,\ 0.098).$$

Suppose the position is at $t = 0.7$ with the linear schedule and a denoiser posterior $(0.3, 0.3, 0.3, 0.1)$. The pretrained rates out of A are $\frac{1}{0.3}$ times the posterior at the three other letters, which is $(1.000, 1.000, 0.333)$ for C, G, T, with exit rate 2.333 and destination distribution $(0.429, 0.429, 0.143)$. Multiplying by the tilts at $\beta = 1$,

$$u_{\mathrm{guided}} = (4.667,\ 9.948,\ 0.033) \ \text{ for C, G, T.}$$

Apply the cone test with $\Phi = 60°$. The weight vector has length $\|\omega\| = \sqrt{0.52} = 0.7211$, and the candidate changes have lengths $\|\Delta\mathbf{s}\| = 0.2062$, 0.1581, 0.1281, so

$$\cos\alpha = \frac{0.100}{0.2062 \times 0.7211} = 0.6727, \quad \frac{0.090}{0.1581 \times 0.7211} = 0.7894, \quad \frac{-0.028}{0.1281 \times 0.7211} = -0.3032,$$

giving angles 47.7°, 37.9° and 107.7°. C and G are inside the cone and T is outside it, so T is removed. After renormalizing over the survivors the jump lands on C with probability 0.319 and on G with probability 0.681, against 0.429, 0.429 and 0.143 before guidance. We have removed the candidate that improves the second objective at the expense of the first. G, which improves both, now has the largest probability.

One of three candidates was rejected, so $\bar{r}_t = 0.333$. With a target $\tau = 0.2$ and gain $\eta = 0.5$, the next cone angle is $60^\circ \times \exp(0.5 \times 0.1333) = 60^\circ \times 1.0689 = 64.1^\circ$, clipped into $[30^\circ, 90^\circ]$ and left where it is. We widen the cone because the rejection rate exceeds the target.

5.32 pCoMole: a reward in place of a likelihood

Edit Flows generate sequences whose length changes along the path, and the same multi-objective question arises there. The destinations of an Edit Flow are insertions, deletions and substitutions, so the reachable set is larger than $D(S-1)+1$ and its size changes as the sequence grows. In pCoMole [Chen et al., 2026a] we use the h-transform of Section 5.25.1 and replace the property likelihood by a utility built from the objectives directly.

We can score a sequence by its weakest objective, with a small correction to break ties,

$$U(x) = \min_k \omega_k\big(f_k(x) - r_k\big) \;+\; \rho \sum_k \omega_k\big(f_k(x) - r_k\big).$$

"The utility $U(x)$ uses the smallest weighted gap $\omega_k(f_k(x) - r_k)$ across objectives k, where $f_k(x)$ is the objective value, r_k its reference and ω_k its weight, then adds ρ times the sum of all weighted gaps to break ties."

The minimum makes the score depend on the weakest property. For a strongly binding design with poor solubility, the solubility gap is near zero and determines the first term. The small ρ-weighted sum separates designs whose worst-objective gaps are equal.

Next, we can turn the utility into a terminal reward,

$$G(x) = \exp\big(\beta\, U(x)\big)\, \mathbb{1}\big[x \in \mathcal{F}\big].$$

"The terminal reward $G(x)$ is the exponential of inverse temperature β times utility $U(x)$, multiplied by the indicator that sequence x belongs to the feasible set $\mathcal{F}$."

We can impose feasibility through hard constraints such as a length window or a forbidden motif. An infeasible sequence receives zero reward regardless of utility. The inverse temperature β controls preference for high utility, similarly to γ in Section 5.28. Here β multiplies utility, while γ is a power on a likelihood ratio.

We then apply the h-transform,

$$h_t(x) = \mathbb{E}\big[\, G(X_1) \,\big|\, X_t = x \,\big], \qquad u_t^G(y \mid x) = u_t(y \mid x)\, \frac{h_t(y)}{h_t(x)}.$$

"The value $h_t(x)$ averages the terminal reward $G(X_1)$ over base-flow continuations from current state $X_t = x$. The guided rate $u_t^G(y \mid x)$ multiplies the base rate $u_t(y \mid x)$ by the value ratio $h_t(y)/h_t(x)$."

At the endpoint, the guided distribution is the base distribution weighted by G, as in Section 5.25.1 with a reward replacing the event likelihood. To derive this change of measure, use the martingale $h_t(X_t)$ along the base process. The corresponding Radon-Nikodym derivative has terminal value $G(X_1)/\mathbb{E}[G(X_1)]$, giving the endpoint weight.

5.32.1 Value estimation by rollouts

The expectation in h_t covers continuations of the base flow from the current state and has no closed form. We can estimate it by Monte Carlo simulation without retraining. From each candidate y_c, run R short continuations to $t = 1$, then average their rewards,

$$\hat{h}_t(y_c) = \frac{1}{R} \sum_{r=1}^{R} G(X_1^{(c,r)}).$$

"The estimate $\hat{h}_t(y_c)$ of the value at candidate y_c is the average of the terminal reward G over R rollouts, each started from that candidate and run forward to the end of the path."

A candidate whose rollouts all land outside $\mathcal{F}$ has estimate zero, and when no candidate has a positive estimate the sampler stays with the current sequence for that step.

The variable-length state space has too many destinations for full enumeration. In pCoMole, we draw a small candidate set from the base rates and select among its members,

$$\tilde{u}_t(y_c \mid x_t) \;\propto\; u_t(y_c \mid x_t)\, \hat{h}_t(y_c), \qquad c = 1, \ldots, C.$$

"The selection probability of candidate c is proportional to its base rate $u_t(y_c \mid x_t)$ times its estimated continuation value $\hat{h}_t(y_c)$, normalized over the C candidates drawn at this step."

5.32.2 Rollout-step example

Take two objectives with weights $\omega = (0.5, 0.5)$, reference levels $r = (0, 0)$, tie-breaking weight $\rho = 0.1$, and inverse temperature $\beta = 2$. Draw three candidate edits with base rates 0.6, 1.2 and 2.0, then run $R = 4$ continuations from each.

Candidate 1 produces terminal objective pairs $(0.8, 0.4)$, $(0.6, 0.6)$, $(0.9, 0.1)$ and $(0.5, 0.5)$, of which the third is infeasible. For the first, the weighted gaps are 0.40 and 0.20, so $U = 0.20 + 0.1 \times 0.60 = 0.26$ and $G = e^{0.52} = 1.682$. The second has gaps 0.30 and 0.30, so $U = 0.36$ and $G = e^{0.72} = 2.054$. The fourth has gaps 0.25 and 0.25, so $U = 0.30$ and $G = e^{0.60} = 1.822$. The third contributes zero, and the estimate is

$$\hat{h}(y_1) = \tfrac{1}{4}(1.682 + 2.054 + 0 + 1.822) = 1.390.$$

Candidate 2 has four feasible endpoints: $(1.0, 0.0)$, $(0.9, 0.1)$, $(0.95, 0.05)$, and $(1.0, 0.1)$. Their first-objective scores exceed those from candidate 1, but their second-objective scores are near zero. The minimum-weighted term therefore stays small. The utilities are 0.050, 0.100, 0.075, and 0.105, giving rewards 1.105, 1.221, 1.162, and 1.234. Averaging gives $\hat{h}(y_2) = 1.181$. Candidate 3 has four infeasible endpoints, so $\hat{h}(y_3) = 0$.

If we multiply each base rate by its estimate,

$$(0.6 \times 1.390,\ 1.2 \times 1.181,\ 2.0 \times 0) = (0.834,\ 1.417,\ 0),$$

and normalize to obtain selection probabilities $(0.371, 0.629, 0)$. Using base rates alone would give $(0.158, 0.316, 0.526)$. Candidate 3 has the highest base rate but receives zero probability because all its continuations are infeasible. The continuation values also change the relative probabilities of candidates 1 and 2. Candidate 2 has higher raw scores, but its low worst-objective value limits its advantage.

5.33 The guided sampler

We can add guidance to the sampling step without changing the trained model. The function takes the flow model, a property model, current sequence x_t, time t, step size h, and strength γ. It applies one Euler step with the Taylor approximation from Section 5.29. We use `V` for the vocabulary size and `k` and `kd` for κ_t and $\dot{\kappa}_t$.

```
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                      # property on noisy x
    g = torch.autograd.grad(logp, oh)[0]          # one backward pass
    adv = g - (oh * g).sum(-1, keepdim=True)      # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)             # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)    # every rate tilted
        u.scatter_(-1, xt[..., None], 0.0)        # no self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

First, one-hot encode the current sequence into a (B, D, V) tensor and enable gradients. This lets us differentiate with respect to every possible letter at every position, including unoccupied letters. Call `prop`, sum its output over the batch, and use `torch.autograd.grad` to compute the gradient in one backward pass. Batch elements do not interact, so the gradient of their sum stacks the individual gradients.

To compute `adv`, first multiply `oh * g` and sum over the last axis to select the gradient coordinate of each current letter. Subtract this from `g` to obtain $[\nabla_x \log p_\phi]_{i,b} - [\nabla_x \log p_\phi]_{i,a}$ in slot (i, b), the Taylor log ratio from Section 5.29. The current letter has log ratio zero and ratio one.

Next, compute the unguided rates from $p_{1|t}$ and $\dot{\kappa}_t/(1-\kappa_t)$, clamping the denominator to keep it finite at $t = 1$. Multiply by `torch.exp(gamma * adv)` to apply the first-order approximation $\exp(\gamma\,\mathrm{adv}) = [p_\phi(c \mid y, t)/p_\phi(c \mid x, t)]^\gamma$. At $\gamma = 0$, the factor is one and we recover the unguided step. At $\gamma = 1$, we approximate the conditional process.

Finally, set the current-letter entry to zero to remove the self-transition. Then `u.sum`-1 is the exit rate and `h * u.sum`-1 is the jump probability to first order in h. Use a uniform draw to decide whether each position jumps and a categorical draw from the adjusted rates to select its destination.

With `torch.where`, retain the current letter wherever no jump occurs. Updating positions in parallel gives the factorized Euler step, with guidance entering through the single rate-multiplication line.

5.34 The guidance term

For h-transform guidance, we multiply each rate by the ratio of destination and current-state values,

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big],$$

We specify G and estimate h according to the objective. In Discrete Guidance, G is a property likelihood and we estimate h with a model trained on noisy states. In pCoMole, G is an exponentiated multi-objective utility multiplied by a feasibility indicator, and we estimate h with base-flow continuations. MOG-DFM uses z-scored ranks and directions with a cone filter, requiring one property evaluation per candidate letter and no joint model. It has no specified target distribution. A strength parameter scales each adjustment; exact conditional sampling in the h-transform constructions requires strength one.

5.35 Sampling step counts

Guidance changes the sampled sequences but retains the base sampler's step count. Text sampling with discrete flow matching takes about 1024 steps, image metric paths take 100, and Edit Flows for code take up to 10,000. Each step requires a network forward pass. An autoregressive model of the same size uses one pass per token, or L passes for length L. To reduce this cost with non-autoregressive sampling, we need fewer than L steps.

5.36 The pairing of source and data

We construct paths from source-data pairs drawn from $\pi(x_0, x_1)$. Independent pairing, $\pi(x_0, x_1) = p_0(x_0)p_1(x_1)$, is convenient, but can increase the number of sampling steps.

As in Chapter 2, straight lines between independent endpoint pairs can cross. At an intersection, we regress on different velocities using the same state and time inputs, giving

$$v(x, t) = \mathbb{E}\big[X_1 - X_0 \,\big|\, X_t = x\big].$$

"The learned velocity $v(x, t)$ at the point x at time t is the expectation of the direction $X_1 - X_0$ over all pairs whose interpolation passes through x at that time."

The average of two different directions is a third direction that agrees with neither, so the integral curve of v bends away from both of the straight lines that produced it. We cannot follow a bent trajectory with one Euler step of size one, and the number of steps needed is set by how much it bends. A straight trajectory needs a single step.

5.37 Rectification

We can change the coupling to remove crossings. Train on the current pairs, follow the learned ODE from each source sample, and pair that source with the resulting endpoint,

$$(Z_0, Z_1) = \mathrm{RectFlow}\big((X_0, X_1)\big), \qquad \boldsymbol{Z}^{k+1} = \mathrm{RectFlow}\big((Z_0^k, Z_1^k)\big).$$

"To apply RectFlow, we train on endpoint pairs (X_0, X_1), integrate the learned flow from each source, and form new pairs (Z_0, Z_1) using those generated endpoints. Repeating this operation on round k's pairs gives round $k+1$'s coupling $\boldsymbol{Z}^{k+1}$."

Two trajectories of an ODE cannot cross, because the velocity at a point at a time is a single vector and the solution through that point is unique. The new trajectories therefore do not cross. The repeated application is called reflow in rectified flow [Liu et al., 2023], where errors in the trained velocity accumulate across rounds, so a small number of rounds is used in practice.

Rectification preserves the generated marginal distributions,

$$\mathrm{Law}(Z_t) = \mathrm{Law}(X_t) \quad \text{for every } t.$$

"The rectified state Z_t and original interpolated state X_t have the same probability distribution at every time t between zero and one."

Both processes satisfy the same continuity equation with the same velocity field v and the same initial density, so they have the same density at every time. In particular the terminal law is unchanged, so sample quality is untouched by rectification at the level of the marginals.

It also reduces or preserves the transport cost,

$$\mathbb{E}\big[c(Z_1 - Z_0)\big] \;\le\; \mathbb{E}\big[c(X_1 - X_0)\big] \quad \text{for every convex } c.$$

"For every convex transport cost c, the expected cost of the rectified displacement $Z_1 - Z_0$ is no greater than the expected cost of the original displacement $X_1 - X_0$."

Apply Jensen's inequality twice. The endpoint displacement $Z_1 - Z_0 = \int_0^1 v(Z_t, t)\, dt$ averages velocities along the rectified trajectory. Each velocity also averages the original endpoint directions. A convex function of either average is at most the average of that function, giving the cost bound. Rectification preserves or reduces cost, with a strict decrease when original trajectories cross.

5.38 Rectification for a set of tokens

A discrete path consists of jumps, so we cannot directly apply the trajectory-straightening argument from Section 5.37. To adapt rectification, we need to identify why a single large discrete step fails.

In a parallel Euler step, we approximate the joint distribution of the next sequence by a product of per-position marginals,

$$p_{s|t}(x_s \mid x_t) \;\approx\; \prod_{i=1}^{N} p_{s|t}(x_s^i \mid x_t).$$

"The distribution $p_{s|t}(x_s \mid x_t)$ of the whole next state given the current one is approximated by the product over the N positions of the per-position conditional distributions, each of which conditions on the current sequence x_t and on nothing else."

For small steps, the probability of two simultaneous position changes is $O(h^2)$, and the conditioning sequence x_t changes little. A single large step draws all positions from a product distribution, losing their joint dependence.

Let π be the coupling and let $\pi(X_1 \mid x_0)$ be the distribution of the data sequence it pairs with the source x_0. The error of a one-step factorized sampler is the divergence between that joint and the product of its per-position marginals, averaged over sources,

$$TC_\pi(X_1 \mid X_0) = \mathbb{E}_{x_0}\Big[D_{\mathrm{KL}}\Big(\pi(X_1 \mid x_0)\,\Big\|\,\prod_{i=1}^{N}\pi(X_1^i \mid x_0)\Big)\Big].$$

"The conditional total correlation $TC_\pi(X_1 \mid X_0)$ of the coupling π is the expectation, over source sequences x_0, of the KL divergence from the joint distribution $\pi(X_1 \mid x_0)$ of the data sequence given that source to the product over the N positions of its per-position marginals."

Total correlation is zero exactly when the positions of X_1 are conditionally independent given X_0, making the single-step factorized sampler exact. It depends on the coupling: different pairings of the same source and data distributions can have total correlation zero or several nats. Since its units are nats, we can compare it with data entropy to quantify the structure lost in one factorized step.

5.39 Total correlation on two bases

Let the data be the four complementary pairs AT, TA, CG and GC, each with probability 1/4, and let the source be uniform over all sixteen ordered pairs of bases. Under independent pairing the data given any source is uniform over the four valid pairs, so $\pi(x_1 \mid x_0) = q(x_1)$ with q that uniform distribution.

Each position of X_1 alone is uniform over A, C, G, T: position one takes A, T, C, G across the four valid pairs, once each. The product of the two marginals therefore spreads 1/16 over all sixteen ordered pairs, while the joint is concentrated on four of them with 1/4 apiece,

$$TC_\pi(X_1 \mid X_0) = \sum_{x_1} q(x_1)\,\log\frac{q(x_1)}{q(x_1^1)\,q(x_1^2)} = 4\cdot\tfrac{1}{4}\log\frac{1/4}{1/16} = \log 4 = 1.386.$$

"The total correlation sums, over the data sequences x_1, the probability $q(x_1)$ times the logarithm of $q(x_1)$ divided by the product of its two per-position marginals $q(x_1^1)q(x_1^2)$, which here is four terms of one quarter times the logarithm of one quarter over one sixteenth, or 1.386 nats."

Independent draws give probability 1/16 to each ordered pair. Only four of the sixteen pairs are complementary, so one parallel step succeeds with probability 1/4. Each source position matches its paired data token with probability 1/4. The expected Hamming distance between source and data is therefore $2 \times 3/4 = 1.5$.

5.40 ReDi

We can improve single-step sampling by reducing the coupling's total correlation. Train on the current pairs, generate new samples, and pair each output with its source,

$$\pi_{k+1}(x_0, x_1) = p(x_0)\, p_\theta(x_1 \mid x_0).$$

"The next coupling pairs a source sample x_0, drawn from the unchanged source distribution $p(x_0)$, with the sequence x_1 that the trained model generates when started from that source."

Here $p_\theta(x_1 \mid x_0)$ is the output distribution of the complete multi-step sampler from $t = 0$ to $t = 1$. In ReDi [Yoo et al., 2025], we use these source-output pairs as the next training set, paralleling the source-endpoint pairing in continuous rectification.

Under the stated assumptions, rectification cannot increase conditional total correlation.

Theorem 5.8 (Rectification does not raise the factorization error)**.** *Assume that the end-to-end sampler kernel lies in a log-convex family of multi-step decoders, and that training reaches the projection of the current coupling onto that family. Then*

$$\boxed{TC_{\pi_{k+1}}(X_1 \mid X_0) \;\le\; TC_{\pi_k}(X_1 \mid X_0).}$$

"The conditional total correlation $TC_{\pi_{k+1}}(X_1 \mid X_0)$ after rectification round $k+1$ is no greater than its value under the previous coupling π_k."

To prove the bound, first apply the Pythagorean inequality for KL projection onto a log-convex family. It bounds the divergence from the new coupling to any family member by the corresponding divergence from the old coupling. Next, apply the KL chain rule to separate divergence to a product distribution into total correlation and per-position terms. Dropping the nonnegative per-position terms gives the result. The sequence TC_{π_k} is non-increasing and bounded below by zero, so it converges, although its limit need not be zero.

5.40.1 Rectifying the two-base coupling

Apply the exact sampler to the coupling from Section 5.39. We compute the true marginal velocity from the posterior $p_{1|t}$ of the current coupling, then solve the master equation without discretization error. The only remaining approximation is the factorization over positions. Across rectification rounds, total correlation decreases as follows:

$$TC: \quad 1.386 \;\to\; 1.101 \;\to\; 0.843 \;\to\; 0.696 \;\to\; \cdots \;\to\; 0.520,$$

and the expected Hamming distance between paired source and data sequences decreases from 1.5 through 1.022, 0.859, and 0.797 to 0.750. We can verify that this limit is minimal. Four of the sixteen sources are valid pairs and can match at zero cost. Each remaining source is one substitution from a valid pair. Any coupling must therefore have expected cost at least $12/16 = 0.75$.

The success rate of a single parallel step increases from 25% to 40.3% after one round, 52.5% after two, and 62.5% in the limit.

Let's look at one source to see where the first round's gain comes from. Under the independent coupling the source AA is paired uniformly with all four valid sequences. After one round it is paired with

$$\pi_1(\cdot \mid \mathrm{AA}) = (\mathrm{AT}\ 0.400,\ \ \mathrm{TA}\ 0.400,\ \mathrm{CG}\ 0.100,\ \ \mathrm{GC}\ 0.100),$$

because AT and TA require one substitution from AA, while CG and GC require two. Both position marginals are $(0.400, 0.100, 0.100, 0.400)$ in A, C, G, T order. A product draw therefore gives probability 0.400^2 to each of AT and TA and 0.100^2 to each of CG and GC. Summing gives success probability 0.340. Starting from a valid source gives a larger success probability. For AT, the endpoint probabilities are $(0.755, 0.082, 0.082, 0.082)$ over AT, TA, CG, GC. The corresponding product draw succeeds with probability 0.590. Averaging 0.590 across the four valid sources and 0.340 across the other twelve recovers 40.3%. The revised coupling concentrates on nearby data sequences, reducing total correlation.

The limiting coupling retains uncertainty between equally close valid pairs. Each of the twelve invalid sources is one substitution from two valid pairs. AA is one substitution from AT or TA, and AC is one substitution from AT or GC. To preserve the marginals, the coupling splits each source equally between its two neighbors. For AA, both position marginals assign probability 1/2 to A and T. Their product gives AT, AA, TT, and TA with probability 1/4 each, so half the draws are valid. Four sources always succeed and twelve succeed half the time,

$$\frac{4 \cdot 1 + 12 \cdot \frac{1}{2}}{16} = \frac{10}{16} = 0.625,$$

and the residual total correlation is the $\log 2$ contributed by each of those twelve splits, $\frac{12}{16} \log 2 = 0.520$ nats, which is where the sequence converged.

The ReDi authors also evaluated single-step ImageNet generation. Their unrectified masked model had FID 95 with one network evaluation per sample. After two rectification rounds, the same architecture had FID 21.8 at the same sampling cost [Yoo et al., 2025].

5.41 AReUReDi

In AReUReDi [Chen et al., 2026b], we combine rectification with guided refinement. We first rectify a peptide flow three times to reduce its step count while preserving the sampled distribution. Then we refine each generated sequence one position at a time toward a multi-objective target, using the rectified per-position marginals as the proposal prior.

Here we score a sequence by its weakest weighted objective,

$$S_\omega(x) = \min_{1 \le n \le N} \omega_n \, \tilde{s}_n(x).$$

"The Tchebycheff score $S_\omega(x)$ of a sequence is the smallest value of the weight ω_n times the normalized score $\tilde{s}_n(x)$ of objective n, over the N objectives."

This score uses the minimum term from pCoMole's utility. For a peptide with strong binding and poor solubility, the score depends on the solubility term, limiting compensation by the stronger property. Increasing ω_n raises $\omega_n \tilde{s}_n$, making objective n less likely to be the minimum and reducing the pressure to improve it.

Next, we can tilt the base model by the exponentiated score,

$$W_{\eta,\omega}(x) = \exp\left(\eta\, S_\omega(x)\right), \qquad \pi_{\eta,\omega}(x) \;\propto\; p_1(x)\, W_{\eta,\omega}(x).$$

"The sequence weight $W_{\eta,\omega}(x)$ exponentiates the Tchebycheff score $S_\omega(x)$ multiplied by strength η. Multiplying the base probability $p_1(x)$ by this weight and normalizing gives the target $\pi_{\eta,\omega}(x)$."

Here η controls how strongly we weight high-scoring sequences. At $\eta = 0$, the target equals the base distribution. Increasing η concentrates probability on sequences with the highest weakest-objective score. We increase η linearly over refinement iterations,

$$\eta_t = \eta_{\min} + (\eta_{\max} - \eta_{\min})\, \frac{t}{T-1},$$

which becomes $\eta_t = 1 + t$ for $\eta_{\min} = 1$, $\eta_{\max} = 20$, and $T = 20$ iterations. Early iterations sample a weakly weighted target; later iterations use a more concentrated target.

5.41.1 The proposal and its acceptance step

The target $\pi_{\eta,\omega}$ has an unknown normalizer over the whole sequence space, so we sample it with a Markov chain. In one iteration we select a position i, propose a new letter there, and accept or reject. The proposal starts from the model's own per-position distribution and reweights it by how much the tilt would change,

$$q_i(y \mid x) = \frac{p_t^i(y \mid x)\; g(r_i(y;x))}{Z(x)}, \qquad r_i(y;x) = \frac{W_{\eta,\omega}(x^{(i\leftarrow y)})}{W_{\eta,\omega}(x)}.$$

"The proposal $q_i(y \mid x)$ of letter y at position i is the model's own distribution $p_t^i(y \mid x)$ over letters there, times a balancing function g of the ratio $r_i(y;x)$ between the tilt at the sequence with y placed at position i and the tilt at the current sequence, divided by the normalizer $Z(x)$ that sums that product over the candidate letters."

The normalizer $Z(x) = \sum_y p_t^i(y \mid x) g(r_i(y;x))$ runs over the alphabet at one position, so it costs $|\mathcal{V}|$ terms and we compute it exactly. The balancing function must satisfy

$$g(u) = u\, g(1/u),$$

and both Barker's function $g(u) = u/(1+u)$ and the square root $g(u) = \sqrt{u}$ satisfy it. Using this identity, we can simplify acceptance to a ratio of one-position normalizers.

We can correct the proposal in the usual way,

$$\alpha_i(x, x') = \min\Big\{1,\; \frac{\pi_{\eta,\omega}(x')\, q_i(x^i \mid x')}{\pi_{\eta,\omega}(x)\, q_i(y \mid x)}\Big\}, \qquad x' = x^{(i\leftarrow y)}.$$

"The acceptance probability $\alpha_i(x, x')$ for the move from x to the sequence x' that carries letter y at position i is the minimum of one and the target at x' times the reverse proposal $q_i(x^i \mid x')$, divided by the target at x times the forward proposal $q_i(y \mid x)$."

The acceptance step gives detailed balance with respect to $\pi_{\eta,\omega}$, making it a stationary distribution of refinement. To simplify the acceptance ratio, write $r = r_i(y; x)$, so that $r_i(x^i; x') = 1/r$. Substitute the proposal and target,

$$\frac{\pi_{\eta,\omega}(x')\, q_i(x^i \mid x')}{\pi_{\eta,\omega}(x)\, q_i(y \mid x)} = \underbrace{\frac{p_1(x')}{p_1(x)} \frac{p_t^i(x^i \mid x')}{p_t^i(y \mid x)}}_{\text{model terms}} \cdot \underbrace{r \frac{g(1/r)}{g(r)}}_{\text{balancing}} \cdot \frac{Z(x)}{Z(x')}.$$

The balancing identity makes the middle factor $r \cdot g(1/r)/(r\, g(1/r)) = 1$. The model terms cancel whenever the base model factorizes at position i given the rest of the sequence, since then $p_1(x')/p_1(x) = p_t^i(y)/p_t^i(x^i)$, which is the reciprocal of the second factor. What remains is a ratio of two one-position normalizers,

$$\alpha_i(x, x') = \min\Big\{1,\ \frac{Z(x)}{Z(x')}\Big\}.$$

"The acceptance probability $\alpha_i(x, x')$ is the smaller of one and the ratio $Z(x)/Z(x')$, where $Z(x)$ normalizes proposals at the current sequence and $Z(x')$ normalizes them at the proposed sequence."

Acceptance is one in both directions exactly when the two normalizers agree, and the normalizers depend on the whole profile of tilt ratios at the position, which changes when the letter there changes.

5.41.2 Acceptance-ratio example

We take two objectives with $\omega = (0.5, 0.5)$, a position currently holding A, and the twelfth refinement iteration, where the annealing schedule above sets $\eta = 12$. The normalized scores of the four sequences that differ only at this position are

$$\tilde{s}(\mathrm{A}) = (0.60, 0.80), \quad \tilde{s}(\mathrm{C}) = (0.70, 0.76), \quad \tilde{s}(\mathrm{G}) = (0.50, 0.90), \quad \tilde{s}(\mathrm{T}) = (0.64, 0.62),$$

so the weighted pairs are $(0.300, 0.400)$, $(0.350, 0.380)$, $(0.250, 0.450)$ and $(0.320, 0.310)$, and the Tchebycheff scores are the smaller entry of each,

$$S_\omega = (0.300,\ 0.350,\ 0.250,\ 0.310).$$

At $\eta = 12$ the tilts are $W = e^{3.6}, e^{4.2}, e^{3.0}, e^{3.72}$, which is $(36.60, 66.69, 20.09, 41.26)$. Let the model's per-position distribution be $p_t^i = (0.40, 0.25, 0.20, 0.15)$ over A, C, G, T.

From the current sequence the ratios are $r = (1.000, 1.822, 0.549, 1.127)$, and Barker's balancing function maps them to $g(r) = (0.500, 0.646, 0.354, 0.530)$. Weighting by the model,

$$Z(x) = 0.40(0.500) + 0.25(0.646) + 0.20(0.354) + 0.15(0.530) = 0.5118,$$

and the proposal is $q_i(\cdot \mid x) = (0.391, 0.315, 0.138, 0.155)$. Suppose the draw returns C, so $x' = x^{(i \leftarrow \mathrm{C})}$.

From x' the ratios are measured against $W(\mathrm{C}) = 66.69$, so $r = (0.549, 1.000, 0.301, 0.619)$ and $g(r) = (0.354, 0.500, 0.231, 0.382)$. The same model weights produce

$$Z(x') = 0.40(0.354) + 0.25(0.500) + 0.20(0.231) + 0.15(0.382) = 0.3704.$$

The ratio is $Z(x)/Z(x') = 0.5118/0.3704 = 1.382$, so we accept A to C with probability one and C to A with probability $1/1.382 = 0.724$. At A, three candidate letters have higher tilts and Z is large. At C, every other letter has a lower tilt and Z is smaller, reducing the probability of accepting a reverse move.

5.41.3 Stationarity and concentration guarantees

Each single-position kernel satisfies detailed balance with respect to $\pi_{\eta,\omega}$. Uniformly selecting a position at each iteration gives a mixture of these kernels, so the random-scan refinement also preserves $\pi_{\eta,\omega}$.

We can also bound concentration on the highest-scoring sequences. Let $\mathcal{F}_\omega$ contain the maximizers of S_ω, and let Δ_ω be the gap between the best and next-best scores. For any sequence outside this set,

$$\pi_{\eta,\omega}(x) \;\le\; e^{-\eta\Delta_\omega}\,\frac{p_1(x)}{\sum_{z\in\mathcal{F}_\omega} p_1(z)} \quad \text{for } x \notin \mathcal{F}_\omega.$$

"For a sequence x outside the maximizing set $\mathcal{F}_\omega$, its target probability $\pi_{\eta,\omega}(x)$ is bounded by $e^{-\eta\Delta_\omega}$ times its base probability $p_1(x)$, divided by the base probability of all maximizers. Here η is the weighting strength and Δ_ω is the score gap."

Write $\pi_{\eta,\omega}(x) = p_1(x)e^{\eta S_\omega(x)}/\sum_z p_1(z)e^{\eta S_\omega(z)}$. Keep only maximizing sequences in the denominator and use $S_\omega(x) \le \max S_\omega - \Delta_\omega$ in the numerator to obtain the bound. As η increases, the right side approaches zero geometrically, concentrating probability on the weighted worst-objective maximizers. Every maximizer of S_ω is weakly Pareto optimal; a unique maximizer is Pareto optimal.

We take four sequences whose base probabilities are $(0.50, 0.25, 0.15, 0.10)$ and whose Tchebycheff scores are $(0.20, 0.30, 0.45, 0.45)$, so the maximizers are the last two, with base mass 0.25, and the gap is $\Delta_\omega = 0.15$. At $\eta = 20$ the tilted distribution is $(0.0127, 0.0468, 0.5643, 0.3762)$, putting 94.1% of its mass on the maximizers, and the bound for the second sequence is $e^{-3} \times 0.25/0.25 = 0.0498$ against its actual 0.0468. At $\eta = 60$ the mass on the maximizers is 99.99%, the second sequence has probability 1.234×10^{-4}, and the bound reads $e^{-9} \times 1 = 1.234 \times 10^{-4}$. The bound is loose for the sequences far from the gap and tight for the ones just below it, which follows from the way it was derived.

5.42 State spaces, paths, and velocities

To construct each method, we choose the state space, conditional path and velocity.

1. A state space: tokens, the simplex, a sphere of square roots, or sequences of varying length.

2. A conditional path from a source sample to a data sample on that state space.

3. A velocity that generates the path, followed by the posterior average over the data sequence that the denoiser predicts.

We train with cross-entropy on data tokens or squared error against geometric velocities. During sampling, we can supplement Euler steps with a corrector, guidance adjustment or Metropolis acceptance step. The table compares these choices across the five paths.

The four sampling modifications change distinct quantities and can be selected separately.

Table 5.5: The five constructions of this chapter, by state space, conditional path and velocity. The rate $\dot\kappa_t/(1-\kappa_t)$ is the hazard of the mixture schedule, $\mathbf{e}_k$ is the corner of the simplex at letter k, the sphere is the sphere of square roots of probability vectors, $\log_{x_t}$ is its logarithm map at the current point, and x_t is the current state.

Construction	State space	Conditional path	Velocity
Jump rates	tokens	mixture path	$\frac{\dot\kappa_t}{1-\kappa_t}$ times the denoiser, one jump per position
Dirichlet	the simplex	$\mathrm{Dir}(1+t\,\mathbf{e}_k)$	a Beta-function speed toward the data corner
Fisher-Rao	the sphere	great-circle arc	$\log_{x_t}(x_1)$ over the time remaining
Gumbel-softmax	the simplex	a cooling temperature	a replicator velocity
Edit Flows	any length	an alignment	insertion, deletion and substitution rates

Table 5.6: The four additions of this chapter, with the quantity each one changes and the quantity each one leaves alone. The corrector strength is η, the guidance strength γ, and TC is the conditional total correlation of the coupling.

Addition	What it changes	What it leaves alone
Correctors, strength η	more jumps per position, more chances to fix a token	the probability path
Metric and kinetic-optimal paths	corruption that respects token similarity, fewer steps on images	the loss and the sampler
Guidance, strength γ	the sampled distribution, exact at $\gamma = 1$	the trained model, the step count
Rectification	the coupling, so one parallel step can work	the generated distribution

5.43 Summary

In discrete flow matching, we choose a conditional path from source to data and derive a velocity that generates it. On tokens, we multiply the difference between the data-token posterior and the current-token point mass by the schedule hazard,

$$u_t^i(\cdot, x) = \frac{\dot\kappa_t}{1-\kappa_t}\big[p_{1|t}(\cdot \mid x) - \delta_{x^i}\big].$$

"The per-position velocity equals the schedule hazard $\dot\kappa_t/(1-\kappa_t)$ times the difference between the data-token posterior $p_{1|t}(\cdot \mid x)$ and the point mass δ_{x^i} on the current token at position i."

With continuous token relaxations, we integrate an ODE whose velocity depends on the chosen geometry,

$$u_t(x \mid \mathbf{e}_i) = C(x_i, t)\,(\mathbf{e}_i - x), \qquad u_t(x_t \mid x_1) = \frac{\log_{x_t}(x_1)}{1-t}.$$

"On the simplex, the conditional velocity $u_t(x \mid \mathbf{e}_i)$ is the displacement $\mathbf{e}_i - x$ toward the data corner multiplied by the Beta-function speed $C(x_i, t)$. On the sphere, the conditional velocity is the tangent displacement $\log_{x_t}(x_1)$ toward data point x_1, divided by remaining time $1-t$."

Guidance multiplies every rate by a ratio of expected terminal rewards,

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x),$$

which is Doob's h-transform of the trained velocity and samples the conditional law exactly at strength one. Rectification changes the pairing without increasing the error of a one-step factorized sampler,

$$TC_{\pi_{k+1}}(X_1 \mid X_0) \le TC_{\pi_k}(X_1 \mid X_0).$$

Each sampler uses a local velocity rule over short intervals where the first-order expansion applies and simultaneous position changes are negligible. We generate a sample by accumulating these updates. Rectification reduces the number of intervals needed by changing the coupling, while preserving the step-by-step sampling procedure.

To cross a large interval accurately in a continuous state space, we need the displacement accumulated along the trajectory. A single starting velocity generally gives only a local approximation. In Chapter 6, we learn a flow map that predicts the destination for a chosen start and end time. We constrain its predictions so that two shorter moves agree with one longer move. We can then generate samples in fewer steps by learning these longer moves directly.

Chapter 6

Flow Maps

6.1 Sampling cost

In Chapter 5, we learned local changes along a path from noise to data: jump rates on tokens and velocities on the simplex or sphere. To generate a sample, we apply those changes repeatedly, with one network evaluation per step. The integrator needs enough steps to approximate the trajectory accurately, so the step count sets the cost per sample. Learning the map between two times directly allows us to generate a sample with only a few evaluations.

Let's start from the continuous case. A generative model there is an ordinary differential equation,

$$\frac{dx}{dt} = b(x, t), \qquad x(0) = x_0$$

"The state x changes at the instantaneous velocity $b(x, t)$, evaluated at the current state and time t, starting from the source sample x_0 at time zero."

We use $b(x, t)$, or b_t when the state is clear, for the instantaneous velocity. This is the field denoted by $v_t(x)$ in Chapter 2 and trained by flow matching; in Chapter 5, we used jump rates on a discrete space. The notation b agrees with the flow map papers. We reserve $v_{s,t}$ for the average velocity over an interval. The subscripts s and t give the start and end times, respectively, and the solution at $t = 1$ is the generated sample.

We can examine the sampling error in the scalar problem

$$\frac{dx}{dt} = x, \qquad x(0) = 1$$

by separating the variables and integrating,

$$\int_1^x \frac{dz}{z} = \int_0^t ds \quad \Longrightarrow \quad \log x = t \quad \Longrightarrow \quad x(t) = e^t,$$

so the exact destination at $t = 1$ is $e = 2.71828$. Applying Euler's method as in Chapter 2, we obtain 2.00 with one step of size one, 2.25 with two steps of size one half, and 2.441 with four steps of size one quarter. Each approximation falls below the exact curve. Euler's method uses the

velocity at the start of each step, which is the smallest velocity attained during that step for this problem. Even four steps fall short by about ten percent (Figure 2.2).

If we take N equal Euler steps on this problem, the state is multiplied by $1 + 1/N$ each time, so the endpoint has a closed form,

$$\left(1 + \tfrac{1}{N}\right)^N \longrightarrow e \quad \text{as } N \to \infty$$

"After N equal Euler steps, the endpoint is one plus the step size $1/N$, raised to the number of steps N. As N increases without bound, this endpoint converges to the exact value e."

At $N = 10$, the endpoint is 2.5937, short of e by 0.1245. At $N = 100$, it is 2.7048, short by 0.01347. Increasing the work tenfold reduces the error by one digit, as expected for a first-order method. The gap decreases like $1/N$: the products $N \times$ gap are 0.718, 0.937, 1.107, 1.245, and 1.347 for $N = 1, 2, 4, 10, 100$, approaching $e/2 = 1.359$. In a learned model, each step requires a network evaluation. A hundred-step sampler therefore requires a hundred forward passes per batch.

An integrator reconstructs the trajectory from pointwise velocity evaluations each time we sample. If we learn the destination over a finite interval, we can evaluate it in one call.

6.2 Endpoint prediction

We want a function of the starting state x, the start time s, and the end time t that returns the trajectory's position at t. The network now receives two times; the networks in Chapters 2 through 5 received one time and one state. The differential equation determines this function and the conditions we can use to train it.

The destination is well defined when exactly one trajectory passes through each state at each time. As in Chapter 2, we assume that the velocity satisfies a Lipschitz condition,

$$\|b(x,t) - b(y,t)\| \le L \, \|x - y\| \quad \text{for all } x,\, y,\, t$$

"The distance between the velocities $b(x,t)$ and $b(y,t)$ at states x and y is at most the Lipschitz constant L times the distance between those states, at every time t."

- $b(x,t)$ is the velocity field, a vector in $\mathbb{R}^d$ for a state $x \in \mathbb{R}^d$ and a time $t \in [0,1]$.
- $L \ge 0$ is the Lipschitz constant of b in its state argument, uniform in time, with units of inverse time.
- $\|\cdot\|$ is the Euclidean norm on $\mathbb{R}^d$.

Under this condition, the Picard-Lindelöf theorem applies at every starting point. Exactly one solution passes through each state at each time, and it depends continuously on that state. The destination is therefore a function of the starting state and the two times. The constant L also bounds how quickly nearby trajectories separate, including the separation caused by an error in the learned velocity b.

6.3 The flow map

We call this unique destination the flow map.

Definition 6.1 (Flow map)**.** Let b be Lipschitz in the state, uniformly in time. The flow map $X_{s,t}(x)$ is the state at time t of the unique trajectory that passes through x at time s. It solves

$$\frac{d}{dt}\, X_{s,t}(x) = b(X_{s,t}(x),\, t), \qquad X_{s,s}(x) = x$$

in the end time t, with the start time s and the state x held fixed.

"The flow map $X_{s,t}(x)$ changes with its end time t at the velocity $b(X_{s,t}(x), t)$, evaluated at the transported state. At the start time $t = s$, the map returns the starting state x."

- $X_{s,t}(x) \in \mathbb{R}^d$ is the state reached at time t by the trajectory passing through x at time s. The subscripts read from the start time to the end time.
- s is the start time and t the end time, both in $[0, 1]$. The map is defined for $t < s$ as well as $t > s$, by integrating the same equation backward.
- x is the state at the start time, the point the map is applied to.

We solve the same ordinary differential equation for each starting state and time. For the scalar problem in Section 6.1, the trajectory through x at time s is $x\, e^{t-s}$, so

$$X_{s,t}(x) = x\, e^{t-s},$$

for every pair of times. To generate a sample, we evaluate

$$x_1 = X_{0,1}(x_0), \qquad x_0 \sim p_0,$$

in one call. Starting from $x_0 = 1$, we obtain $e^1 = 2.71828$ exactly. Because this map is the exact solution, it has none of the discretization error from the N Euler steps in Section 6.1.

A network with one state input and two time inputs does not automatically satisfy the conditions of a flow map. We can constrain its predictions using the three properties of the true map.

6.4 The three axioms

Proposition 6.2 (Flow axioms)**.** *Let b be Lipschitz in the state, uniformly in time, and let $X_{s,t}$ be the flow map of Definition 6.1. Then*

Identity: $X_{t,t} = \mathrm{id}$ *for every t;*

Cocycle: $X_{t,u} \circ X_{s,t} = X_{s,u}$ *for every s, t, u;*

Tangent: $\partial_t X_{s,t}(x)\big|_{t=s} = b(x, s)$ *for every x and s.*

Conversely, a two-time family satisfying all three with b Lipschitz is the flow map of b.

We can use each axiom in a training objective. The choice of axioms and the source of their targets determine how we train the map.

6.4.1 Identity

The first axiom is a boundary condition on the diagonal of the two-time square,

$$X_{t,t}(x) = x \quad \text{for every } x \text{ and every } t$$

"The zero-duration flow map $X_{t,t}(x)$ returns the starting state x at every time t."

The trajectory starts at x, so its position after zero elapsed time is still x. We can satisfy this boundary condition exactly by adding the starting state to the elapsed time multiplied by a network output,

$$\hat{X}_{s,t}(x) = x + (t - s)\,\hat{v}_{s,t}(x),$$

where the hat denotes a model prediction and we suppress its parameters θ. At $t = s$, the second term vanishes for every network output. We therefore need no loss term to enforce the identity.

6.4.2 Cocycle

The second axiom relates one long move to two short ones. Fix a trajectory at three times $s < t < u$. We can follow it from s to t, then continue to u, or evaluate its position at u directly. Both routes follow the same curve and reach the same point (Figure 6.1).

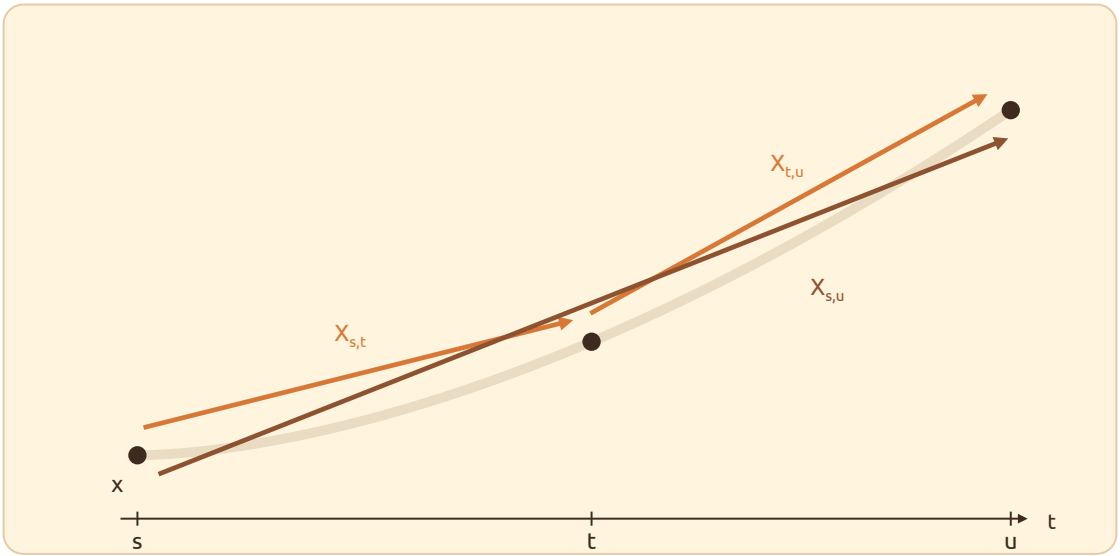


Figure 6.1: One trajectory of $dx/dt = b(x, t)$, drawn in pale gray with the horizontal axis as time and the vertical position as the state. The three black dots sit on the trajectory at times s, t and u, with the state at s labeled x. The two light arrows are the two legs of the composed route: $X_{s,t}$ carries x from s to t, and $X_{t,u}$ carries the result from t to u. The dark arrow is the direct map $X_{s,u}$ across the whole interval. Both routes end at the same point of the same trajectory, which is the cocycle property.

We can write the agreement between the two routes as

$$\boxed{X_{t,u}(X_{s,t}(x)) = X_{s,u}(x) \quad \text{for all } s,\ t,\ u}$$

"Applying the flow map $X_{s,t}$ to the starting state x, then applying $X_{t,u}$ to that result, gives the direct-map destination $X_{s,u}(x)$, for any three times s, t, and u."

Uniqueness establishes this identity. The trajectory through x at time s reaches one state at time t. Only that same trajectory passes through the reached state at that time. Continuing to u therefore gives the same point as evaluating the original trajectory at u.

On the scalar flow $X_{s,t}(x) = x\,e^{t-s}$ the identity is visible in the arithmetic. We can take $x = 1$, $s = 0$, $t = 0.5$, $u = 1$. The first leg lands at $X_{0,0.5}(1) = e^{0.5} = 1.64872$. Continuing from there, the second leg lands at $X_{0.5,1}(1.64872) = 1.64872 \times e^{0.5} = 2.71828$, and the direct map lands at $X_{0,1}(1) = e^1 = 2.71828$. The two routes agree to every digit, because $e^{0.5}e^{0.5} = e$ is the cocycle for this particular flow.

We can sample a model that satisfies the cocycle with any number of steps, including one. Every partition of the interval gives the same endpoint, which allows few-step sampling.

6.4.3 Tangent

The third axiom ties the map to the velocity we already know how to train,

$$\frac{\partial}{\partial t} X_{s,t}(x)\Big|_{t=s} = b(x, s)$$

"The end-time derivative $\partial_t X_{s,t}(x)$, evaluated when the end time t equals the start time s, equals the instantaneous velocity $b(x, s)$ at the starting state x."

Near the diagonal the map is a short Euler step, $X_{s,s+h}(x) = x + h\,b(x, s) + o(h)$, so the map agrees there with the velocity field trained by flow matching in Chapter 2. A flow map model therefore contains a flow matching model along its diagonal, and we can train that diagonal with the flow matching loss.

The identity and cocycle axioms alone do not determine the desired map. For example, $X_{s,t} = \mathrm{id}$ for all s and t satisfies both: composing two identity maps gives the identity map. Every sample then stays at its initial state. The simplest combination of objectives permits this solution.

6.5 Time-independent velocity

When the velocity has no explicit time dependence, only the elapsed time matters,

$$b(x, t) = b(x) \quad \Longrightarrow \quad X_{s,t} = \varphi_{t-s}, \qquad \varphi_t \circ \varphi_s = \varphi_{t+s}$$

"For a time-independent velocity $b(x)$, the flow map $X_{s,t}$ depends only on elapsed time and equals φ_{t-s}. Composing the duration-s map φ_s with the duration-t map φ_t gives the duration-$s + t$ map φ_{t+s}."

The family $\{\varphi_t\}$ is a one-parameter group: φ_0 is the identity and φ_{-t} undoes φ_t. The flow map literature takes its language from this case. The scalar flow of Section 6.1 is of this kind, with $\varphi_t(x) = xe^t$ and $\varphi_t \circ \varphi_s = \varphi_{t+s}$ following from $e^t e^s = e^{t+s}$.

The generative interpolants in Chapters 2 and 5 weight the source and data by functions of t. Their velocities therefore depend explicitly on time, and the flow map needs both s and t. The corresponding composition law is the two-time cocycle. For discrete states and stochastic paths with only forward-time evolution, the family is a semigroup without an inverse.

6.6 The inverse map

Setting $u = s$ in the cocycle turns the composition law into an inverse,

$$X_{t,s} \circ X_{s,t} = X_{s,s} = \mathrm{id} \quad \Longrightarrow \quad X_{s,t}^{-1} = X_{t,s}$$

"Composing the forward map $X_{s,t}$ with the backward map $X_{t,s}$ returns the identity map $X_{s,s}$. Thus the inverse $X_{s,t}^{-1}$ is the same flow map with its start and end times exchanged."

We invert the map by integrating the same equation backward in time and recover every point along the trajectory. Thus each flow map is a bijection of the state space onto itself.

If b is continuously differentiable in the state, the solution is continuously differentiable in its starting point. Both $X_{s,t}$ and its inverse $X_{t,s}$ are then continuously differentiable, so the map is a C^1 diffeomorphism.

A diffeomorphism maps each starting point to one destination. All mass at a state at time s therefore remains together at time t. The map cannot divide an atom. This creates no restriction for smooth densities on a continuous space. On a discrete space, it prevents some transports, including a deterministic map from a two-atom source to a three-atom target.

We can also compute the volume change under a diffeomorphism from the determinant of its derivative matrix. This volume factor gives the exact likelihood.

6.7 The Jacobian of the map

Differentiating the flow map in its starting point measures how the destination responds when the departure point is moved,

$$J_{s,t}(x) = \frac{\partial X_{s,t}(x)}{\partial x}$$

"The Jacobian $J_{s,t}(x)$ contains the derivatives of each coordinate of the destination $X_{s,t}(x)$ with respect to each coordinate of the starting state x."

- $J_{s,t}(x)$ is a $d \times d$ matrix for a state space $\mathbb{R}^d$, with entry (i, j) equal to $\partial [X_{s,t}(x)]_i / \partial x_j$.
- $\nabla b(y, t)$ is the $d \times d$ matrix of derivatives of the velocity in its state argument, evaluated at the state y and the time t.

A displacement δ applied to the starting point reaches the destination as $J_{s,t}(x)\delta + o(\|\delta\|)$, so the Jacobian is the linearization of the map around one trajectory. We can track it in time with no second solve.

6.7.1 Jacobian evolution

Let's differentiate the defining equation of the flow map with respect to the starting point x. Because b is continuously differentiable, we can exchange the t and x derivatives. Applying the chain rule to $b(X_{s,t}(x), t)$ gives ∇b evaluated along the trajectory, multiplied by the derivative of $X_{s,t}(x)$ with respect to x. Thus the Jacobian satisfies

$$\frac{d}{dt} J_{s,t}(x) = \nabla b(X_{s,t}(x),\, t)\, J_{s,t}(x), \qquad J_{s,s}(x) = I$$

"The Jacobian $J_{s,t}(x)$ changes with end time t at the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ multiplied by $J_{s,t}(x)$. At the start time $t = s$, the Jacobian is the identity matrix I."

Differentiating the identity axiom in x gives the initial condition. The evolution equation is linear in J, even when b is nonlinear in the state. We evaluate ∇b along the trajectory, so it becomes a time-dependent coefficient matrix. We can then solve one extra linear equation alongside the trajectory with the same integrator.

6.7.2 Volume along the flow

Let's take a small box of volume 1 at time s and carry every one of its points along the flow. At time t it is stretched and sheared into a parallelogram whose volume is $\det J_{s,t}$ (Figure 6.2).

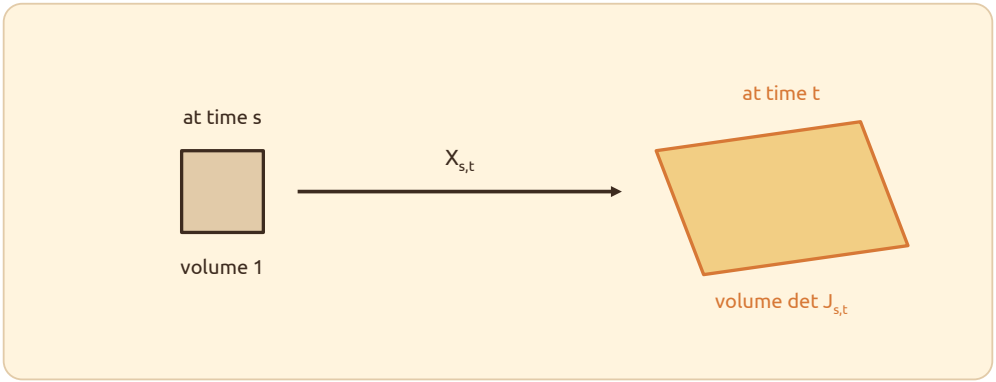


Figure 6.2: A small box of volume 1 at time s, on the left, and its image under the flow map $X_{s,t}$ at time t, on the right. The image is a parallelogram because the map is linearized by its Jacobian over a small box, and its volume is $\det J_{s,t}$ evaluated at the center of the box. The shearing of the right-hand shape is the off-diagonal part of the Jacobian, and the change in area is its determinant.

6.8 Liouville's formula

If we follow that determinant in time, the matrix equation becomes a scalar one.

Proposition 6.3 (Liouville's formula)**.** *Let b be continuously differentiable in the state. Then along any trajectory,*

$$\frac{d}{dt} \log\det J_{s,t}(x) = \nabla \cdot b(X_{s,t}(x),\, t)$$

"The log volume factor $\log\det J_{s,t}(x)$ changes with end time t at the divergence $\nabla \cdot b(X_{s,t}(x), t)$ of the velocity, evaluated along the trajectory."

For an invertible matrix $A(t)$,

$$\frac{d}{dt} \log \det A = \operatorname{tr}(A^{-1} \dot{A}),$$

so substituting the Jacobian equation of Section 6.7 for $\dot{J}$,

$$\frac{d}{dt} \log \det J_{s,t} = \operatorname{tr}(J_{s,t}^{-1} \, \nabla b \, J_{s,t}) = \operatorname{tr}\,(\nabla b),$$

where the second equality uses invariance of the trace under conjugation. The trace of ∇b sums the diagonal entries $\partial b_i / \partial x_i$, giving the divergence $\nabla \cdot b$. After the Jacobian cancels, the instantaneous rate of volume change depends only on the velocity at the current point.

Zero divergence preserves volume, so the flow map has unit Jacobian determinant for every pair of times. The radial field from Chapter 1 has positive divergence and expands each small box. The rotational field has zero divergence and moves the box without changing its volume.

The scalar flow of Section 6.1 is the smallest case in which we can check both sides by hand. Its map is $X_{s,t}(x) = xe^{t-s}$, so the Jacobian in one dimension is $J_{s,t} = e^{t-s}$ and its logarithm is $t - s$. The velocity $b(x, t) = x$ has derivative $\partial b / \partial x = 1$, so the divergence is 1 everywhere and its integral from s to t is $t - s$. The two agree for every pair of times, and the growth rate of the log Jacobian is the constant 1, matching $d(t - s)/dt = 1$. An interval of length ℓ at time s becomes, at time t, an interval of length $\ell \, e^{t-s}$, and the density over it falls by the same factor.

6.9 Gaussian flow example

We can calculate both quantities explicitly for a Gaussian flow. Take the source $N(0, 1)$ and data $N(2, 9)$, pair them by $x_1 = 2+3x_0$, and interpolate linearly between paired points, $x_t = (1-t)x_0+tx_1$. Substituting the pairing gives

$$x_t = (1 - t)\, x_0 + t\,(2 + 3x_0) = 2t + (1 + 2t)\, x_0,$$

which is the flow map from time zero. Differentiating the trajectory gives $\dot{x}_t = x_1 - x_0 = 2 + 2x_0$. To express the velocity in terms of the current state, we solve $x_t = 2t + (1 + 2t)x_0$ for x_0 and substitute,

$$X_{0,t}(x) = 2t + (1 + 2t)\, x, \qquad b(x, t) = 2 + \frac{2\,(x - 2t)}{1 + 2t}$$

"The transported state $X_{0,t}(x)$ equals the translation $2t$ plus the starting state x scaled by $1 + 2t$. The instantaneous velocity $b(x, t)$ equals 2 plus twice the current state's displacement $x - 2t$ from that translation, divided by $1 + 2t$."

The translation $2t$ changes the mean from 0 to 2 over the unit interval. The scale factor $1 + 2t$ changes the standard deviation from 1 to 3. To obtain the map between arbitrary times, we first apply the inverse map from s to zero, then the map from zero to t,

$$X_{s,t}(x) = 2t + (1 + 2t)\, \frac{x - 2s}{1 + 2s},$$

which at $s = 0$ returns the formula above and at $t = s$ returns x.

6.9.1 The Jacobian and the divergence

In one dimension the Jacobian is a single number and the divergence is a single derivative,

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

"The scalar Jacobian $J_{0,t}$ is the expansion factor $1 + 2t$. The velocity divergence $\nabla \cdot b$ is $2/(1 + 2t)$, and integrating it from time zero to one gives the log volume change $\log 3$."

If we differentiate $X_{0,t}(x) = 2t + (1 + 2t)x$ in x, we get $1 + 2t$. Differentiating $b(x, t) = 2 + 2(x - 2t)/(1 + 2t)$ in x produces $2/(1 + 2t)$, since the numerator is linear in x and the denominator does not involve x. The integral is $\big[\log(1 + 2t)\big]_0^1 = \log 3 - \log 1 = \log 3 = 1.0986$.

Direct evaluation at $t = 1$ yields $J_{0,1} = 1 + 2 = 3$. Integrating Liouville's formula over the unit interval accumulates $\log \det J_{0,1} = \log 3$, and exponentiating that logarithm returns 3. The two routes to the volume factor therefore agree. The flow stretches every length by a factor of 3, so it divides every density by 3.

6.10 Transporting a density along the flow

To transport a probability distribution, we apply the change-of-variables formula from Chapter 1 to the diffeomorphism $X_{s,t}$. Conservation of probability requires

$$p_t\big(X_{s,t}(x)\big)\;\big|\det J_{s,t}(x)\big| \;=\; p_s(x)$$

"The transported density $p_t(X_{s,t}(x))$, multiplied by the local volume factor $|\det J_{s,t}(x)|$, equals the initial density $p_s(x)$ at starting state x and time s."

The mass inside a small box is the same before and after transport, since no trajectory crosses the moving boundary of the box in either direction. Where the flow stretches the box, the same mass is spread over a larger volume, so the density falls by exactly the factor by which the volume grew.

For the Gaussian flow in Section 6.9, take $x = 0.7$ at time zero. Its image is $X_{0,1}(0.7) = 2 + 3(0.7) = 4.1$. The source density is

$$p_0(0.7) = \frac{1}{\sqrt{2\pi}}\, e^{-0.7^2/2} = 0.3989 \times 0.7827 = 0.3123,$$

and the data density at the image point is

$$p_1(4.1) = \frac{1}{3\sqrt{2\pi}}\, e^{-(4.1-2)^2/18} = 0.1330 \times 0.7827 = 0.1041.$$

The ratio $0.3123/0.1041 = 3.000$ is the Jacobian determinant, in agreement with the transport formula.

6.10.1 Continuity equation derivation

If we take logarithms, the product becomes a sum,

$$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

"The log density $\log p_t(X_{s,t}(x))$ at the transported point, plus the log volume factor $\log\det J_{s,t}(x)$, equals the starting log density $\log p_s(x)$."

The right-hand side is constant in t, so the derivative of the left-hand side is zero. Applying the chain rule to the first term gives $\partial_t \log p_t + b \cdot \nabla \log p_t$, where b accounts for the moving evaluation point. Liouville's formula gives $\nabla \cdot b$ for the derivative of the second term. Adding these derivatives yields

$$\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0$$

"The time derivative $\partial_t \log p_t$ of the log density, its directional derivative $b \cdot \nabla \log p_t$ along the velocity, and the velocity divergence $\nabla \cdot b$ sum to zero. Multiplying by the density p_t gives the continuity equation: its time derivative $\partial_t p_t$ plus the divergence $\nabla \cdot (p_t b)$ of probability flux is zero."

We recover the continuity equation from Chapter 2. There, we derived it from a flux balance on an interval; here, we derived it from the flow map and a change of variables. Both derivations express conservation of probability under the velocity field.

6.10.2 The likelihood of a generated sample

Integrating Liouville's formula from 0 to 1 and substituting into the transport statement isolates the log density of the sample,

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\, t\big)\, dt$$

"The generated sample's log density $\log p_1(X_{0,1}(x_0))$ equals the source log density $\log p_0(x_0)$ minus the velocity divergence $\nabla \cdot b$ integrated along the trajectory $X_{0,t}(x_0)$ from time zero to one."

This identity makes a continuous normalizing flow an exact likelihood model. In the diffusion models of Chapter 3, we calculated a likelihood bound. Evaluating the divergence integral requires the trajectory and divergence at many times, so an exact likelihood can require many evaluations even when sampling takes one.

For the Gaussian flow, the source log density at $x_0 = 0.7$ is $\log 0.3123 = -1.1639$, the divergence integral is $\log 3 = 1.0986$, and their difference is -2.2625. Evaluating the data log density directly at 4.1 returns $\log 0.1041 = -2.2625$, the same number to four decimals.

6.11 The generator of the flow

We can also describe the change in a function evaluated along the flow. The corresponding operator has a discrete-state version as well.

For a smooth test function $f : \mathbb{R}^d \to \mathbb{R}$, define

$$(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)$$

"The generator $\mathcal{L}_t$, applied to the test function f at state x, equals the velocity $b(x,t)$ dotted with the spatial gradient $\nabla f(x)$."

The time subscript in $\mathcal{L}_t$ identifies the generator, an operator from functions to functions. A name subscript identifies a scalar training loss: $\mathcal{L}_b$ fits the velocity b, and $\mathcal{L}_{\text{LSD}}$ measures the Lagrangian residual.

The generator is the rate of change of f along a trajectory. Differentiating $f(X_{s,t}(x))$ in t by the chain rule, and substituting the defining equation of the map for the inner derivative,

$$\frac{d}{dt} f\big(X_{s,t}(x)\big) = \big(\mathcal{L}_t f\big)\big(X_{s,t}(x)\big)$$

"The value $f(X_{s,t}(x))$ changes along the trajectory at the rate $(\mathcal{L}_t f)(X_{s,t}(x))$, the generator applied to f at the transported state."

6.11.1 Composition as an operator

We can restate the cocycle on functions. Define the operator that composes a function with the flow map,

$$(T_{s,t} f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\, T_{t,u}$$

"The operator $T_{s,t}$ maps a function f to its value at the transported state $X_{s,t}(x)$. The full-interval operator $T_{s,u}$ equals $T_{s,t}T_{t,u}$, with the later-interval operator acting on the function first."

Composition of maps becomes multiplication of operators, with the order of the two factors reversed because each operator acts on the argument of the function. The adjoint of $T_{s,t}$ acts on densities, and the infinitesimal form of that adjoint statement is the continuity equation of Section 6.10. The same cocycle therefore appears three times: on points, on functions, and on densities.

6.11.2 Comparison with jump-rate matrices

If we write the generator abstractly, the discrete case becomes a substitution,

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

"On a continuous space, the generator $\mathcal{L}_t$ applies the velocity $b(x,t)$ to the gradient $\nabla f(x)$. On a discrete space, it sums the rate-matrix entries $u_t(y,x)$ from the current state x over states y, each multiplied by the function value $f(y)$."

Both generators give the rate of change of f along the process. Their adjoints describe density evolution through the continuity equation on a continuous space and the master equation on a discrete space. The continuous generator uses a derivative; the discrete generator uses weighted

differences over neighboring states. Adding diffusion introduces a second-derivative term in the continuous generator.

The velocity field b, flow map $X_{s,t}$, and generator describe the same dynamics. We train b by flow matching, constrain $X_{s,t}$ through identity, cocycle, and tangent, and evolve the density with the generator's adjoint.

6.12 Flow-map computation and verification

We can check the axioms numerically using the Gaussian flow. Integrating its velocity with a high-order solver reproduces the exact map with error far below the quantities being checked. A larger discrepancy would therefore indicate an error in the claimed identity.

```
import torch
from torch.func import jacrev

def b(x, t):                                        # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)
```

The function `b` implements the velocity from Section 6.9: translation at speed 2 plus expansion with strength $2/(1+2t)$, which decreases as the distribution widens. We import `jacrev` from `torch.func` to differentiate the integration with respect to its starting point.

```
def flow(x, s, t, n=200):                           # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x
```

The function `flow` uses fourth-order Runge-Kutta to integrate from `s` to `t` in `n` substeps of size `h`. For each substep, we evaluate the velocity at its start, twice at its midpoint using different trial states, and at its end. We combine these four evaluations with weights $1, 2, 2, 1$, divided by six. The variable `u` records the substep's starting time, and each update overwrites `x`. The returned value is therefore the state at `t`. When `t` is less than `s`, `h` is negative and the same function computes the inverse map by integrating backward. With $n = 200$, the local error is of order h^5 and accumulated error is of order $h^4 = (1/200)^4$, well below the quantities being checked.

```
x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))                            # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))            # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))            # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))       # det J = 3
```

We evaluate all four expressions at the starting point $x = 0.7$.

The first line crosses the whole interval in one call and returns 4.100, which matches the closed form $2 + 3x = 2 + 2.1 = 4.1$. The integrator reproduces the analytic map, so the velocity and the map of

Section 6.9 are consistent with each other.

The second expression composes two legs. We first obtain $X_{0,0.5}(0.7) = 2(0.5) + 2(0.7) = 2.400$, then $2 + 3(2.4 - 1)/2 = 4.100$. This agrees with the direct evaluation and verifies the cocycle at $s = 0$, $t = 0.5$, and $u = 1$.

The third line runs the map forward and then backward. Starting from 0.7, the forward call lands at 4.1 and the backward call returns 0.700, the original point. The inverse of Section 6.6 is the same function with its two times exchanged, and no separate implementation is needed.

In the fourth expression, `jacrev` differentiates the composition of all 200 substeps with respect to the starting state. It returns 3.000, matching $J_{0,1} = 1 + 2(1) = 3$. The logarithm $\log 3 = 1.0986$ also matches the integrated divergence from Liouville's formula. Differentiating through the solver and integrating divergence therefore give the same volume factor.

These calculations verify the map by integrating a known velocity. To produce the destination without integration, we need to learn the map. We can satisfy the identity axiom through its parameterization and enforce the remaining axioms through losses.

6.13 The average velocity

We can write the map as a step of the right length, with the network producing the velocity to use over that length,

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

"The destination $X_{s,t}(x)$ equals the starting state x plus the elapsed time $t - s$ multiplied by the interval's average velocity $v_{s,t}(x)$."

- $v_{s,t}(x) \in \mathbb{R}^d$ is the average velocity over the piece of trajectory running from time s at the state x to time t, equal to the displacement divided by the elapsed time.
- $t - s$ is the elapsed time, which may be negative, in which case the map runs backward.
- $\hat{v}_{s,t}$ with a hat denotes the model's prediction of this quantity, with parameters θ suppressed.

At $t = s$, the second term vanishes for every network output, so the identity axiom holds without a loss term. Rearranging gives $v_{s,t}(x) = (X_{s,t}(x) - x)/(t - s)$, the displacement divided by elapsed time. Thus $v_{s,t}$ is the time average of the instantaneous velocity along this trajectory segment,

$$v_{s,t}(x) = \frac{1}{t - s} \int_s^t b(X_{s,r}(x),\, r)\, dr.$$

Different losses can train the same average-velocity parameterization.

If we let the two times meet, we recover the velocity of Chapter 2,

$$v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)$$

"As the start time s approaches the end time t, the displacement $X_{s,t}(x) - x$ divided by elapsed time $t - s$ converges to the instantaneous velocity $b(x, t)$. This defines the diagonal average velocity $v_{t,t}(x)$."

This limit expresses the tangent axiom in average-velocity notation. We can train it with the flow matching loss, using the same architecture and a target that is independent of the model.

On the scalar flow $X_{s,t}(x) = xe^{t-s}$, the average velocity is

$$v_{s,t}(x) = \frac{x\,(e^{t-s} - 1)}{t - s},$$

which at $s = t$ reduces to x by the limit $\lim_{h\to 0}(e^h - 1)/h = 1$, matching $b(x, t) = x$. Over a long interval the two differ substantially. At $s = 0$ and $t = 1$ the average velocity is $x(e - 1) = 1.718x$, against an instantaneous velocity of x at the start.

6.14 The start-time and end-time domain

The pair (s, t) lies in a square (Figure 6.3). Flow matching trains the velocity on its diagonal, $s = t$. A generative flow map covers the triangle $s \le t$. Fixing the end time at 1 restricts the map to the top edge, leaving only the start time free. This is the domain of a consistency model.

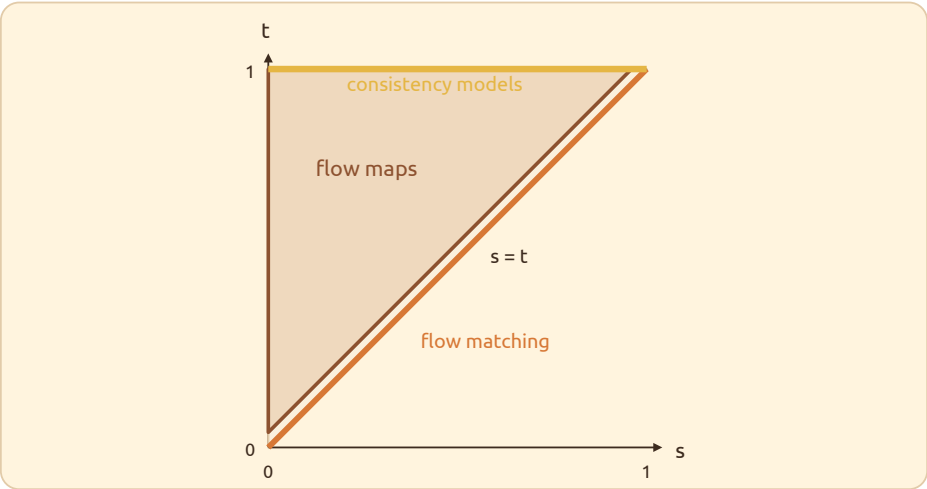


Figure 6.3: The square of start time s, on the horizontal axis, and end time t, on the vertical axis, both running from 0 to 1. The shaded triangle is the region $s \le t$, which a flow map model covers. Its lower boundary is the diagonal $s = t$, where the map reduces to the instantaneous velocity and flow matching applies. Its upper boundary is the edge $t = 1$, a one-dimensional family of maps into the data end, which is the domain of a consistency model.

Training over the triangle includes both the flow matching diagonal and the consistency-model edge. A function of two times also requires constraints between different time pairs. We obtain those constraints from the cocycle.

6.15 Differentiating the cocycle

The cocycle is an equation in three times. Differentiating it in each of its times produces two differential identities, and keeping it finite produces a third, algebraic identity. All three characterize the same map.

6.15.1 The Lagrangian identity

Hold the start time fixed and differentiate with respect to the end time. By Definition 6.1,

$$\boxed{\frac{\partial}{\partial t} X_{s,t}(x) = b(X_{s,t}(x),\, t)}$$

"The flow map's end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the transported state and end time t."

Increasing the end time moves the destination along the same trajectory at its current velocity. We differentiate in t and evaluate b at the output, with no spatial derivative. This is the Lagrangian form, named for a reference frame that moves with the particle.

6.15.2 The Eulerian identity

Now hold the end time t fixed and follow the trajectory's starting state x_s as s increases. The state x_s changes, but its destination at the fixed time t stays the same. The total start-time derivative therefore vanishes,

$$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

"Following a trajectory with current state x_s, the destination $X_{s,t}(x_s)$ at fixed end time t stays constant as the start time s changes. Its total derivative in s is therefore zero."

Expanding that total derivative by the chain rule separates the explicit dependence on s from the dependence through x_s, whose rate of change is $b(x_s, s)$,

$$\boxed{\partial_s X_{s,t}(x) + b(x, s) \cdot \nabla_x X_{s,t}(x) = 0}$$

"The start-time derivative $\partial_s X_{s,t}(x)$ plus the directional derivative $b(x, s) \cdot \nabla_x X_{s,t}(x)$, taken along the starting velocity $b(x, s)$, equals zero."

Here we differentiate in s and evaluate the velocity at the input. This Eulerian form uses a frame fixed in space, so it includes a spatial gradient. The directional derivative contracts the map's $d \times d$ Jacobian with a d-dimensional velocity vector.

6.15.3 The composition identity

The third condition is the cocycle itself, left undifferentiated. For $s < u < t$,

$$X_{s,t}(x) = X_{u,t}(X_{s,u}(x)),$$

with no derivative of any kind. Evaluating it costs three forward passes through the network, one for the long map and two for the legs, and the intermediate time u is ours to choose.

Written in the average-velocity parameterization, the composition identity becomes a statement about weighted averages. Substituting $X_{s,t}(x) = x + (t-s)v_{s,t}(x)$ into each of the three maps and cancelling the state x,

$$(t-s)\, v_{s,t}(x) = (u-s)\, v_{s,u}(x) + (t-u)\, v_{u,t}\big(X_{s,u}(x)\big),$$

so the long displacement equals the sum of the two short displacements. Dividing by the total elapsed time gives a weighted average of the two short-interval velocities, with weights set by their durations.

On the scalar flow we can check this with numbers. Over $[0,1]$ from $x=1$, the average velocity is $v_{0,1}(1) = e - 1 = 1.71828$. Splitting at $u = 0.5$, the first leg has average velocity

$$v_{0,0.5}(1) = \frac{e^{0.5}-1}{0.5} = \frac{0.64872}{0.5} = 1.29744,$$

and the second leg starts from $X_{0,0.5}(1) = e^{0.5} = 1.64872$ and has average velocity

$$v_{0.5,1}(1.64872) = \frac{e - e^{0.5}}{0.5} = \frac{1.06956}{0.5} = 2.13912.$$

The intervals have equal length, so both weights are one half: $\frac{1}{2}(1.29744) + \frac{1}{2}(2.13912) = 1.71828$, matching $v_{0,1}(1)$ to five decimals. The second leg has a higher average velocity because the state has grown by $t = 0.5$. The long-interval average lies between the two. Shortcut models enforce this weighted relation for step sizes that differ by a factor of two.

The cocycle therefore has three trainable forms.

- *Lagrangian*: differentiate in the end time, evaluate b at the output.
- *Eulerian*: differentiate in the start time, evaluate b at the input, and carry a spatial gradient.
- *Composition*: require that two shorter maps compose into the longer one.

The two differential forms require one forward-mode derivative of the network without constructing a full Jacobian. The composition form requires three forward passes and no derivative. When v is continuous in both time arguments, all three have the true flow map as their unique solution. Their residuals behave differently for an imperfect network during training.

6.16 Three residuals and the diagonal anchor

Each identity becomes a loss by squaring the amount by which the model violates it. Throughout, I_s is the interpolant of Chapter 2, $I_s = (1-s)X_0 + sX_1$ with $X_0 \sim p_0$ and $X_1 \sim p_1$ drawn independently [Albergo and Vanden-Eijnden, 2023], and $\dot{I}_s$ is its time derivative. Because we sample the evaluation points from the interpolant, we never have to simulate a trajectory during training. The expectation runs over the pair (X_0, X_1) and over the times, drawn from the triangle $s \le t$.

The Lagrangian residual compares the end-time derivative of the model map against the model's own diagonal velocity at the transported point,

$$\boxed{\mathcal{L}_{\mathrm{LSD}} = \mathbb{E}\Big[\big|\, \partial_t \hat{X}_{s,t}(I_s) - \hat{v}_{t,t}(\hat{X}_{s,t}(I_s))\big|^2\Big]}$$

"The Lagrangian loss $\mathcal{L}_{\mathrm{LSD}}$ averages the squared distance between the predicted map's end-time derivative $\partial_t \hat{X}_{s,t}(I_s)$ and its diagonal velocity $\hat{v}_{t,t}$ at the transported interpolant $\hat{X}_{s,t}(I_s)$, over source-data pairs and sampled start and end times s and t."

We evaluate the target with the model's own diagonal velocity, so we need no separately trained teacher. This is self-distillation [Boffi et al., 2025b]. The diagonal supplies the target and is trained by the anchor loss.

The Eulerian residual squares the left-hand side of the start-time identity,

$$\mathcal{L}_{\mathrm{ESD}} = \mathbb{E}\Big[\big|\, \partial_s \hat{X}_{s,t}(I_s) + \nabla \hat{X}_{s,t}(I_s)\, \hat{v}_{s,s}(I_s)\big|^2\Big]$$

"The Eulerian loss $\mathcal{L}_{\mathrm{ESD}}$ averages the squared norm of the start-time derivative $\partial_s \hat{X}_{s,t}(I_s)$ plus the spatial Jacobian $\nabla \hat{X}_{s,t}(I_s)$ applied to the predicted starting velocity $\hat{v}_{s,s}(I_s)$, over interpolant samples I_s and time pairs (s, t)."

We compute the start-time derivative and directional derivative along $\hat{v}_{s,s}(I_s)$ in one forward-mode pass. The evaluation count matches the Lagrangian residual. Its tangent lies along the time axis; the Eulerian tangent also spans the state coordinates.

The composition residual squares the gap between the long map and the two-leg route,

$$\mathcal{L}_{\mathrm{PSD}} = \mathbb{E}\Big[\big|\, \hat{X}_{s,t}(I_s) - \hat{X}_{u,t}(\hat{X}_{s,u}(I_s))\big|^2\Big], \qquad s < u < t$$

"The composition loss $\mathcal{L}_{\mathrm{PSD}}$ averages the squared distance between the long prediction $\hat{X}_{s,t}(I_s)$ and the two-step prediction $\hat{X}_{u,t}(\hat{X}_{s,u}(I_s))$, over interpolant samples I_s and ordered times $s < u < t$."

We can implement this residual with three forward passes and no network derivative. It gives no bound on the generated distribution's error: the errors of the two short maps propagate through their composed arguments.

6.16.1 The diagonal anchor

Each residual compares the model with itself, so a self-consistent model can satisfy it and still generate the wrong distribution. We connect the model to the data through a flow matching loss on the diagonal,

$$\mathcal{L}_b = \mathbb{E}[\,|\, \hat{v}_{t,t}(I_t) - \dot{I}_t\,|^2]$$

"The anchor loss $\mathcal{L}_b$ averages the squared distance between the diagonal velocity prediction $\hat{v}_{t,t}(I_t)$ and the interpolant's time derivative $\dot{I}_t$, over times t, source samples X_0, and data samples X_1 that define I_t."

By the posterior-average argument in Chapter 2, the conditional mean $\mathbb{E}[\dot{I}_t \mid I_t]$ minimizes this loss and equals the true velocity. We train with the anchor plus one residual, constraining the diagonal to that velocity and the interior of the triangle to the chosen identity.

6.17 The zero-velocity solution

Consider a model that preserves every starting state. We can check which combinations of losses exclude it.

We take the average velocity to equal the true velocity when the two times coincide and zero everywhere else,

$$v_{s,t}(x) = \begin{cases} b(x,t) & s = t \\ 0 & s \neq t \end{cases} \quad \Longrightarrow \quad X_{s,t}(x) = x \ \text{ for all } s, t$$

"The average velocity $v_{s,t}(x)$ equals the true velocity $b(x,t)$ on the diagonal $s = t$ and zero elsewhere. In the map $X_{s,t}(x) = x + (t-s)v_{s,t}(x)$, the displacement is therefore zero for every time pair, so every destination equals its starting state x."

Off the diagonal the second term of the parameterization is $(t-s) \times 0 = 0$, and on the diagonal it is $0 \times b = 0$, so the map returns x in every case. A sampler built on it moves no source point at all, so it generates the source distribution p_0 and nothing else.

The identity axiom holds by construction. The cocycle also holds because two identity maps compose to the identity. On $s = t$, the model equals the true velocity, so the anchor is at its minimum. The anchor and composition loss can therefore both be zero even though the model only generates the source distribution. This is also the simplest pair of losses to implement.

The two differential residuals exclude it. Evaluate them on $dx/dt = x$, where $b(x,t) = x$, at $s = 0$, $t = 1$ and $x = 1$. For the Lagrangian residual,

$$\partial_t X_{s,t}(x) = 0, \qquad b(X_{s,t}(x), t) = b(x,t) \neq 0$$

"The identity map has zero end-time derivative $\partial_t X_{s,t}(x)$. Its destination is x, where the velocity $b(x,t)$ is nonzero in this example, so the Lagrangian equality fails."

At $x = 1$, the Lagrangian terms are 0 and $b(1,1) = 1$, giving $|0-1|^2 = 1$. For the Eulerian residual, the start-time derivative is zero and the spatial gradient is the identity. Its second term is $\hat{v}_{s,s}(x) = b(x,s) = 1$, giving $|0+1|^2 = 1$. Both residuals are zero for the correct map, so either detects this stay-still solution.

6.17.1 The role of continuity

The stay-still velocity jumps from 0 to b at $s = t$. This discontinuity permits it to satisfy the composition condition and the anchor simultaneously. Requiring continuity excludes this model.

Proposition 6.4 (Composition under continuity)**.** *Let $v_{s,t}(x)$ be continuous in both time arguments and let the induced map satisfy the composition identity for all $s < u < t$. Then the map satisfies the Lagrangian identity, and therefore equals the flow map of $b = v_{t,t}$.*

The argument is a Taylor expansion of the composition over a short second leg. We can apply the composition identity with an intermediate time t and an end time $t+h$, and expand the short map $X_{t,t+h}$ through the parameterization,

$$X_{s,t+h}(x) = X_{t,t+h}\big(X_{s,t}(x)\big) = X_{s,t}(x) + h\, v_{t,t}\big(X_{s,t}(x)\big) + o(h)$$

"Extending the destination $X_{s,t}(x)$ by a short duration h adds h times the diagonal velocity $v_{t,t}$ at that destination, plus a remainder $o(h)$ that vanishes faster than h. The cocycle equates this two-leg route with $X_{s,t+h}(x)$."

Subtract $X_{s,t}(x)$, divide by h, and take h to zero. We obtain $\partial_t X_{s,t}(x) = v_{t,t}(X_{s,t}(x))$, the Lagrangian identity with the model's diagonal velocity. The anchor sets that velocity to b, whose Lagrangian equation has the unique solution given by the true flow map. Continuity justifies replacing the short-step velocity by $v_{t,t}$. The stay-still construction fails this step because $v_{s,t}$ does not approach $v_{t,t}$ as $s \to t$.

The composition loss depends on continuity in both time arguments to determine the map. A smooth architecture satisfies that assumption. The loss alone does not impose it.

6.18 MeanFlow

To train MeanFlow, we express the Eulerian identity in average-velocity coordinates [Geng et al., 2025]. Substitution gives a target in terms of the model and the instantaneous velocity.

We now start from the Eulerian identity $\partial_s X + b \cdot \nabla_x X = 0$ of Section 6.15.2, and substitute $X_{s,t}(x) = x + (t-s)\, v_{s,t}(x)$. The start-time derivative acts on the product $(t-s)v_{s,t}$, producing $-v_{s,t}$ from the factor and $(t-s)\partial_s v_{s,t}$ from the velocity. The spatial gradient of X is $I + (t-s)\nabla_x v_{s,t}$, and contracting it with $b(x,s)$ produces $b(x,s)$ from the identity matrix and $(t-s)\, b(x,s) \cdot \nabla_x v_{s,t}$ from the second term,

$$-\, v_{s,t}(x) + (t-s)\, \partial_s v_{s,t}(x) \;+\; b(x,s) + (t-s)\, b(x,s) \cdot \nabla_x v_{s,t}(x) \;=\; 0$$

"The start-time derivative contributes minus the average velocity $v_{s,t}$ plus elapsed time $t-s$ times its derivative $\partial_s v_{s,t}$. The spatial derivative contributes the instantaneous velocity $b(x,s)$ plus $t-s$ times the directional derivative $b(x,s) \cdot \nabla_x v_{s,t}$. These four terms sum to zero."

The second and fourth terms share the factor $t-s$ and combine into a single total derivative along the trajectory. Collecting them and moving $v_{s,t}$ to the left,

$$v_{s,t}(x) = b(x,s) + (t-s)\, \frac{d}{ds} v_{s,t}(x), \qquad \frac{d}{ds} = \partial_s + b(x,s) \cdot \nabla_x$$

"The average velocity $v_{s,t}(x)$ equals the starting velocity $b(x,s)$ plus the elapsed time $t-s$ multiplied by its total start-time derivative along the trajectory. That total derivative adds the partial derivative $\partial_s v_{s,t}$ and the directional derivative $b(x,s) \cdot \nabla_x v_{s,t}$."

On the diagonal $t = s$, the correction vanishes and we recover the tangent axiom, $v_{t,t}(x) = b(x,t)$. Away from the diagonal, the correction is the difference between the interval's average velocity and its initial velocity. It grows with the interval length.

6.18.1 MeanFlow time conventions

MeanFlow states the same identity with the time convention of the diffusion literature,

$$u(z, r, t) = v(z, t) - (t - r)\,\frac{d}{dt}u(z, r, t)$$

"The average velocity $u(z, r, t)$ at state z equals the instantaneous velocity $v(z, t)$ minus the interval length $t - r$ times the total derivative of u in its starting time t, with the endpoint r fixed."

In MeanFlow the time runs the other way, with $t = 1$ at noise and $t = 0$ at data, so their $u(z, r, t)$ is our $v_{1-t,\,1-r}(z)$. Their t is the start of their map, so their d/dt is a derivative in the start time, matching our d/ds. The minus sign in front of the correction is the Jacobian of the time reversal, $d(1 - t) = -dt$. With those three changes the two equations are the same equation. Their sampling of $r < t$ becomes our $s < t$ after the mirror, which is the half of the square that generation uses.

6.18.2 Computing the target

To train the identity we need the total derivative on its right-hand side. Expanding it by the chain rule, with r held fixed,

$$\frac{d}{dt}u(z, r, t) = \partial_z u\; v(z, t) + \partial_t u$$

"The total time derivative of the average velocity u equals its spatial Jacobian $\nabla_z u$ applied to the instantaneous velocity $v(z, t)$, plus its explicit time derivative $\partial_t u$. The other time argument r stays fixed."

We can compute this total derivative by applying the full Jacobian of u to the tangent $(v, 0, 1)$. The state component is velocity v, the fixed r component is zero, and the t component is one. Forward-mode differentiation evaluates this product in one pass without forming the Jacobian. The number of passes is independent of state dimension.

Before computing the squared difference, we detach the right-hand side from the computation graph. Gradients then pass only through the left-hand side, with a fixed target for that step. We also sample a fraction of each batch at $r = t$. There the correction vanishes and training reduces to flow matching, the diagonal anchor from Section 6.16.1.

6.18.3 Scalar-flow verification

The exact map of $dx/dt = x$ over $[s, t]$ multiplies the state by e^{t-s}, so the average velocity is $v_{s,t}(x) = x(e^{t-s} - 1)/(t - s)$, as in Section 6.13. Fixing $s = 0$ and $t = 1$, we evaluate each term of the identity.

The two velocities are

$$v_{0,1}(x) = x\,(e - 1) = 1.718\,x, \qquad b(x, 0) = x$$

"Over the unit interval, the average velocity $v_{0,1}(x)$ equals the starting state x multiplied by $e - 1$, or approximately $1.718x$. The initial instantaneous velocity $b(x, 0)$ equals x."

The correction term is the total derivative in the start time. Its two pieces are a partial derivative and a transport term. Differentiating $v_{s,t}(x) = x(e^{t-s} - 1)/(t - s)$ in s by the quotient rule produces

$$\partial_s v_{s,t}(x) = x \, \frac{-e^{t-s}(t-s) + (e^{t-s} - 1)}{(t-s)^2},$$

which at $s = 0$, $t = 1$ is $x(-e + e - 1) = -x$. The spatial gradient is $\partial_x v_{0,1} = e - 1 = 1.718$, and the velocity at the input is $b(x, 0) = x$, so the transport term is $1.718x$. Adding the two pieces,

$$\frac{d}{ds} v_{s,t}(x)\Big|_{s=0,\, t=1} = -x + (e-1)\, x = x\,(e-2) = 0.718\, x$$

"At start time zero and end time one, the total start-time derivative of the average velocity $v_{s,t}(x)$ equals the starting state x multiplied by $e - 2$, or approximately $0.718x$."

Substituting the three quantities into the identity, with elapsed time $t - s = 1$,

$$\underbrace{1.718\, x}_{v_{0,1}(x)} \;=\; \underbrace{x}_{b(x,0)} \;+\; \underbrace{0.718\, x}_{(t-s)\, dv/ds}$$

"The average velocity $1.718x$ equals the instantaneous starting velocity x plus its finite-interval correction $0.718x$. Thus the identity recovers the exact average velocity."

Starting from $x = 1$, one flow-map evaluation gives $1 + 1 \times 1.718 = 2.718$, the exact endpoint e to four figures. One Euler step gives $1 + 1 \times 1 = 2.00$, short by 0.718 or twenty-six percent. The learned average velocity includes the correction $0.718x$ that accounts for this difference.

For the Gaussian flow in Section 6.9, the average velocity from zero is $v_{0,t}(x) = \big(2t + (1+2t)x - x\big)/t = 2 + 2x$, independent of end time. Its total start-time derivative and correction term are zero. At $x = 0.7$, both sides equal $b(0.7, 0) = 2 + 1.4 = 3.4$. Linear interpolation between paired Gaussians gives straight trajectories, where the average and initial velocities agree. A nonzero correction accounts for velocity changes along a curved trajectory and separates a one-step map evaluation from one Euler step.

6.19 Identity-based losses

Each cocycle identity gives a squared residual: $\mathcal{L}_{\text{LSD}}$ for the end-time Lagrangian identity, $\mathcal{L}_{\text{ESD}}$ for the start-time Eulerian identity, and $\mathcal{L}_{\text{PSD}}$ for finite composition. The Lagrangian residual evaluates the velocity at the map's output. The Eulerian residual evaluates it at the input and includes a spatial gradient. The composition residual compares two short maps with one long map. We pair each residual with $\mathcal{L}_b$, the diagonal anchor and the only data-dependent loss term. This excludes a self-consistent map to the wrong distribution. MeanFlow uses the Eulerian identity with the model's own total derivative in its target.

Each training step begins with a pair (X_0, X_1) drawn from the coupling. Draw start and end times $s \le t$, then construct $I_s = (1 - s)X_0 + sX_1$. For a fixed fraction of the batch, set $t = s$ and apply the anchor. Its target $\dot{I}_s = X_1 - X_0$ is available without a network call. For the remaining samples, evaluate the chosen residual at (s, t). Detach its target so that gradients pass only through the

fitted prediction. A differential residual requires one forward pass and one forward-mode derivative. Composition requires three forward passes.

To sample, draw $x_0 \sim p_0$ and evaluate $\hat{X}_{0,1}(x_0) = x_0 + \hat{v}_{0,1}(x_0)$ in one network call. For two steps, compose $\hat{X}_{0,1/2}$ with $\hat{X}_{1/2,1}$; for four, divide each interval again. The cocycle makes these endpoints agree. We choose the number of evaluations using the trained map's accuracy.

Two choices determine the training method: which identity we enforce and where its target comes from. The identity specifies the trained time pairs and the penalty for a violation. Targets can come from the model's own diagonal, a pretrained teacher, or a composition of shorter model steps.

6.20 Shortcut and consistency models

Shortcut models, consistency models, and consistency trajectory models all use two-time maps. They differ in the identity used for training and the source of its target.

6.20.1 Shortcut models

The composition identity compares a long map with two shorter maps. In shortcut models [Frans et al., 2025], the network receives the start time and step length. We enforce the identity between a step and one twice as long,

$$s_\theta(x, t, 2d) = \tfrac{1}{2}\, s_\theta(x, t, d) + \tfrac{1}{2}\, s_\theta(x', t + d, d), \qquad x' = x + d\, s_\theta(x, t, d)$$

"The shortcut prediction $s_\theta(x, t, 2d)$ for a step of length $2d$ equals the average of two predictions of duration d: $s_\theta(x, t, d)$ from starting state x, and $s_\theta(x', t + d, d)$ from the first step's destination x'."

- $s_\theta(x, t, d)$ is the network, taking a state x, a start time t and a step length d, and producing the average velocity over the interval $[t, t + d]$. The letter s here is the published name of the network and has no relation to the start time s used everywhere else in this chapter.
- d is the step length, so the end time of the step is $t + d$.
- x' is the state after the first step of length d.

Substitute $s_\theta(x, t, d) = v_{t,t+d}(x)$ to express the shortcut identity in two-time notation. It becomes the composition identity with the midpoint as intermediate time. Starting from $X_{t,t+2d} = X_{t+d,t+2d} \circ X_{t,t+d}$, substitute $X_{s,t}(x) = x + (t - s)v_{s,t}(x)$. The left-hand side is $x + 2d\, v_{t,t+2d}(x)$, and the right-hand side is

$$x' + d\, v_{t+d,\, t+2d}(x'), \qquad x' = x + d\, v_{t,t+d}(x),$$

so the two sides agree when

$$x + 2d\, v_{t,t+2d}(x) = x + d\, v_{t,t+d}(x) + d\, v_{t+d,\, t+2d}(x').$$

If we cancel x and divide by $2d$, the shortcut equation comes out exactly. Written as an average of two velocities, the composition identity has both sides on the same scale, so a single squared error weights every step length equally.

We sample three quarters of each batch at step size zero, using the flow matching target to train the diagonal anchor. The remaining quarter uses doubled steps [Frans et al., 2025], connecting a sequence of dyadic step sizes.

Repeated doubling connects the finest step to the whole interval. For $d_0 = 1/128$, take $d_k = 2^k d_0$ for $k = 0, \ldots, 7$, so $d_7 = 1$. Each training pair connects levels k and $k+1$; seven constraints span the interval because $\log_2 128 = 7$. The anchor trains level zero, and seven composition relations connect it to the one-step map at level seven. The network represents eight distinct step sizes, which we can encode with a discrete embedding. Errors at level zero propagate through each successive doubling.

Let's put numbers on the identity using the scalar flow $dx/dt = x$, whose exact map multiplies the state by e^{t-s} and whose average velocity is therefore $v_{s,t}(x) = x\,(e^{t-s} - 1)/(t-s)$. Take $x = 1$, $t = 0$ and $d = 0.25$. The first step uses

$$s_\theta(1, 0, 0.25) = v_{0,\,0.25}(1) = \frac{e^{0.25} - 1}{0.25} = \frac{0.284025}{0.25} = 1.136102,$$

and it lands at $x' = 1 + 0.25 \times 1.136102 = 1.284025$, which is $e^{0.25}$ to six figures. The second step starts there,

$$s_\theta(x', 0.25, 0.25) = x'\,\frac{e^{0.25} - 1}{0.25} = 1.284025 \times 1.136102 = 1.458783,$$

and the average of the two velocities is $\frac{1}{2}(1.136102 + 1.458783) = 1.297443$. The doubled step computed directly is

$$s_\theta(1, 0, 0.5) = \frac{e^{0.5} - 1}{0.5} = \frac{0.648721}{0.5} = 1.297443,$$

matching the average of the two short-step velocities. Its endpoint is $1 + 0.5 \times 1.297443 = 1.648721 = e^{0.5}$. The squared difference between the two sides is therefore zero for the true map at every step length.

6.20.2 Consistency models

Fixing the end time leaves a map from any starting time to the data. This is the top edge $t = 1$ of the triangle in Section 6.14. We train consistency models on this edge [Song et al., 2023] and impose identity at the endpoint through the parameterization,

$$f_\theta(x, t) = c_{\text{skip}}(t)\,x + c_{\text{out}}(t)\,F_\theta(x, t), \qquad c_{\text{skip}}(\epsilon) = 1,\ c_{\text{out}}(\epsilon) = 0$$

"The consistency prediction $f_\theta(x, t)$ adds the input state x weighted by the skip coefficient $c_{\text{skip}}(t)$ to the raw network output $F_\theta(x, t)$ weighted by $c_{\text{out}}(t)$. At the data time ϵ, these coefficients are one and zero, so the prediction equals the input."

- F_θ is the raw network output, with no constraint at the boundary.
- c_{skip} and c_{out} are fixed scalar functions of time chosen before training, not learned.
- ϵ is the time at which the path reaches the data. Consistency models use the diffusion time convention, with data at a small positive ϵ and noise at the other end. Our data endpoint is $t = 1$. The published symbol ϵ therefore corresponds to our $t = 1$.

At the data end, the parameterization returns the input for any learned weights, so no loss term is needed there. In our direction of time, the simplest coefficients with this property are $c_{\text{skip}}(s) = 1$ and $c_{\text{out}}(s) = 1 - s$. Substituting them restricts the flow map to $t = 1$,

$$f_\theta(x, s) = x + (1 - s)\, F_\theta(x, s), \qquad F_\theta = v_{s,1}.$$

On the scalar flow $dx/dt = x$ at $s = 0.5$ and $x = 1$, the velocity head is $v_{0.5,1}(1) = (e^{0.5} - 1)/0.5 = 1.297443$, the output coefficient is 0.5, and $f = 1 + 0.5 \times 1.297443 = 1.648721 = e^{0.5}$, the exact destination. At $s = 0.9$ the output coefficient has shrunk to 0.1 and the model returns $1 + 0.1 \times 1.051709 = 1.105171 = e^{0.1}$. At $s = 1$ the coefficient is zero and the model returns 1.

On the rest of the edge, a consistency model trains the composition identity with a short second leg. Let Φ_h be one step of a teacher solver, moving a state at time s to a state at time $s + h$ along the teacher's velocity. The distillation loss is

$$\mathcal{L}_{\text{CD}} = \mathbb{E}\Big[\big|\, f_\theta(x_s, s) - f_{\theta^-}\big(\Phi_h(x_s),\, s + h\big)\,\big|^2\Big],$$

where θ^- is a frozen or slowly updated weight copy. Gradients pass only through the student prediction. Setting $u = s + h$ in the composition residual and identifying $f_\theta(\cdot, \sigma)$ with $\hat{X}_{\sigma,1}(\cdot)$ gives the same expression, with the teacher performing the leg of length h.

The size of that residual at the true map is the teacher's own discretization error. Let's take the scalar flow again, $x = 1$ at $s = 0.5$, and one teacher Euler step of size $h = 0.01$. The teacher moves the state to $1 + 0.01 \times 1 = 1.01$, and the exact trajectory reaches $e^{0.01} = 1.010050$. Evaluating the exact map at both ends of the step,

$$f(1,\ 0.5) = e^{0.5} = 1.648721, \qquad f(1.01,\ 0.51) = 1.01\, e^{0.49} = 1.648639,$$

giving a residual of 8.189×10^{-5}. The teacher's local Euler error, $e^{0.01} - 1 - 0.01 = 5.0167 \times 10^{-5}$, grows by the factor $e^{0.49} = 1.632316$ on the remaining interval. Halving h reduces this error floor by four. The distilled consistency model therefore inherits the teacher solver's integration error at the data endpoint.

6.20.3 Consistency trajectory models

A consistency model has a fixed endpoint, so it cannot stop at an intermediate time to trade more evaluations for accuracy. Consistency trajectory models [Kim et al., 2024] restore the second time argument and learn over the full triangle. Their diffusion-time convention moves from t to $s < t$, closer to data. Reversing time by $t \mapsto 1 - t$ gives our start and end times in their usual order.

With both times free, the identity axiom has to hold along the whole diagonal, and there are two ways to arrange it. One multiplies a velocity head by a factor that vanishes on the diagonal, which is the parameterization $X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)$ already in use here. The other writes the map as a weighted average of the input and a head that predicts a destination,

$$X_\theta(x, s, t) = a_{s,t}\, x + (1 - a_{s,t})\, g_\theta(x, s, t), \qquad a_{s,s} = 1,$$

and consistency trajectory models are built on this second form. The weight $a_{s,t}$ is fixed in advance, and at $t = s$ it equals one, so the map returns its input for any g_θ.

Choosing $a_{s,t} = (1-t)/(1-s)$ makes g_θ a data prediction. On the scalar flow, with $s = 0$, $t = 0.5$, and $x = 1$, the weight is $a = 0.5/1 = 0.5$, and the exact map gives $X_{0,0.5}(1) = e^{0.5} = 1.648721$. Solving for the prediction head,

$$g = \frac{X_{0,0.5}(1) - 0.5 \times 1}{0.5} = \frac{1.148721}{0.5} = 2.297443.$$

At $s = 0$, $t = 1$, the weight is zero, so the head is the endpoint itself, $g = e = 2.718282$. The remaining interval determines the weight of the predicted destination. The discrete denoiser parameterization uses this same pair of weights.

A consistency trajectory sampler combines two routes to an intermediate time. One route goes to the data endpoint first, then returns to the intermediate time. The other takes the short step there directly. A parameter divides noise between the two routes, which coincide for an exact map by the cocycle. For $x = 1$, $s = 0$, and $u = 0.5$, the direct step gives $X_{0,0.5}(1) = 1.648721$. The route through the endpoint gives

$$X_{1,0.5}\big(X_{0,1}(1)\big) = 2.718282 \times e^{-0.5} = 1.648721,$$

the same state. The splitting parameter affects results only when the learned map has error. Increasing it places more dependence on the long prediction. The same network evaluates both routes because both consist of time pairs in its training triangle.

6.21 Endpoint error under each residual

All three residuals vanish at the true map, but a trained network generally has a nonzero residual. We need to relate its size to the endpoint error. For velocity models and flow maps, we can derive this relation by following the error along a trajectory.

6.21.1 Velocity-error propagation

Assume the learned velocity differs from the true velocity by at most ε everywhere. Also assume the true velocity is Lipschitz with constant L, as in Section 6.2. Two trajectories starting together then separate at a rate bounded by the Lipschitz term plus the velocity error,

$$\frac{d}{dt}\,\|x_t - \hat{x}_t\| \le L\,\|x_t - \hat{x}_t\| + \varepsilon \quad \Longrightarrow \quad \|x_1 - \hat{x}_1\| \le \frac{\varepsilon}{L}(e^L - 1)$$

"For trajectories x_t and $\hat{x}_t$ starting together, their separation grows at a rate bounded by the Lipschitz constant L times the current separation plus the uniform velocity error ε. At time one, their separation is at most ε/L multiplied by $e^L - 1$."

We can apply Grönwall's inequality using an integrating factor. Write $\rho(t) = \|x_t - \hat{x}_t\|$, with $\rho(0) = 0$. Multiply the differential inequality by e^{-Lt} and collect the two terms on the left,

$$\frac{d}{dt}\Big(e^{-Lt}\rho(t)\Big) = e^{-Lt}\Big(\frac{d\rho}{dt} - L\rho\Big) \le \varepsilon\, e^{-Lt}.$$

Integrating from 0 to t and using $\rho(0) = 0$,

$$e^{-Lt}\rho(t) \le \frac{\varepsilon}{L}(1 - e^{-Lt}),$$

and multiplying by e^{Lt} gives $\rho(t) \le (\varepsilon/L)(e^{Lt} - 1)$. At $t = 1$ this is the bound displayed above. The exponential comes from feeding an error back into the dynamics, since each instant starts from a state already displaced by the accumulated error.

For $dx/dt = x$, the Lipschitz constant is $L = 1$, and a constant velocity error attains the bound. Take $\hat{b}(x) = x + 0.01$, with uniform error $\varepsilon = 0.01$. Starting from $\hat{x}_0 = 1$, the solution is $\hat{x}_t = 1.01\, e^t - 0.01$. At $t = 1$,

$$\hat{x}_1 = 1.01 \times 2.718282 - 0.01 = 2.735465, \qquad \hat{x}_1 - x_1 = 2.735465 - 2.718282 = 0.017183,$$

matching the bound $(0.01/1)(e - 1) = 0.017183$ to six figures. This constant velocity error attains the worst case permitted by the inequality.

We can compare this with Euler's discretization error on the same flow. With N equal steps, the endpoint is $(1 + 1/N)^N$. Its gap to e is 0.718282 at $N = 1$, 0.124539 at $N = 10$, and 0.013468 at $N = 100$. About eighty steps are needed to match the 0.017183 error from a velocity model with a 1% error. Beyond roughly eighty evaluations, the velocity error limits the accuracy. The exponential factor also amplifies errors over long intervals, so we need an endpoint-error estimate for a learned flow map.

A flow map trained to a Lagrangian residual of size ε obeys a bound of the same shape. Let $\hat{X}$ be the model map, X the true one, and

$$r_{s,t}(x) = \partial_t \hat{X}_{s,t}(x) - b(\hat{X}_{s,t}(x),\, t)$$

the Lagrangian residual, with $\|r_{s,t}\| \le \varepsilon$. Write $e_t = \hat{X}_{s,t}(x) - X_{s,t}(x)$, which starts at $e_s = 0$ because both maps satisfy the identity axiom. Differentiating in t,

$$\frac{de_t}{dt} = r_{s,t}(x) + b(\hat{X}_{s,t}(x), t) - b(X_{s,t}(x), t), \qquad \Big\|\frac{de_t}{dt}\Big\| \le L\|e_t\| + \varepsilon,$$

which is the same differential inequality, so $\|e_t\| \le (\varepsilon/L)(e^{L(t-s)} - 1)$. There is one exponential factor for the whole interval, however many evaluations the sampler makes inside it.

For the Eulerian residual, we can derive an estimate without an exponential factor. Define

$$q_{s,t}(x) = \partial_s \hat{X}_{s,t}(x) + b(x, s) \cdot \nabla_x \hat{X}_{s,t}(x),$$

and follow a true trajectory x_σ, evaluating the model map from the moving start time σ to the fixed end time t. Along that trajectory $\dot{x}_\sigma = b(x_\sigma, \sigma)$, so the chain rule turns the total derivative into the residual itself,

$$\frac{d}{d\sigma}\, \hat{X}_{\sigma,t}(x_\sigma) = \partial_\sigma \hat{X}_{\sigma,t}(x_\sigma) + b(x_\sigma, \sigma) \cdot \nabla_x \hat{X}_{\sigma,t}(x_\sigma) = q_{\sigma,t}(x_\sigma).$$

If we integrate from s to t and use $\hat{X}_{t,t} = \mathrm{id}$, which holds exactly under the parameterization, the left side telescopes to $x_t - \hat{X}_{s,t}(x_s)$, and x_t is $X_{s,t}(x_s)$. Therefore

$$\|\hat{X}_{s,t}(x_s) - X_{s,t}(x_s)\| \le \int_s^t \|q_{\sigma,t}(x_\sigma)\|\, d\sigma \le (t - s) \sup_\sigma \|q_{\sigma,t}\|.$$

The integral of the residual along the trajectory bounds the endpoint error without amplification. Flow Map Matching gives corresponding bounds on the Wasserstein-2 distance between the generated

and true distributions. The Eulerian bound there is tighter than the Lagrangian bound by at most a factor of two [Boffi et al., 2025a].

The composition residual gives neither bound because it compares the model only with itself. For example, consider the family

$$X^{\lambda}_{s,t}(x) = x\, e^{\lambda(t-s)}, \qquad \lambda \in \mathbb{R},$$

whose members satisfy identity and cocycle because $e^{\lambda(u-t)}e^{\lambda(t-s)} = e^{\lambda(u-s)}$. Their composition residuals are zero, although their endpoints span all positive values. For example, $X^{0.5}_{0,1}(1) = 1.648721$ differs from $e = 2.718282$ by 1.069561. The diagonal velocity is λx, so the anchor selects $\lambda = 1$ from this family. It still permits the discontinuous stay-still map, whose diagonal velocity is correct but whose off-diagonal map is identity. That model has zero anchor and composition losses while generating only the source distribution.

6.21.2 Residual comparison on an approximate map

We take the second-order truncation of the exact map of $dx/dt = x$,

$$\hat{X}_{s,t}(x) = x\Big[1 + h + \tfrac{1}{2}h^2\Big], \qquad h = t - s,$$

whose average velocity is $\hat{v}_{s,t}(x) = x\,(1 + h/2)$. On the diagonal $h = 0$ and $\hat{v}_{t,t}(x) = x = b(x,t)$, so the anchor is exactly zero for this model, and every error it has is off the diagonal. At $s = 0$, $t = 1$, $x = 1$ it lands at $\hat{X}_{0,1}(1) = 2.5$ against $e = 2.718282$, an endpoint error of 0.218282.

The Lagrangian residual is the derivative of the map in t against the velocity at the output. The derivative is $\partial_t \hat{X}_{0,t}(1) = 1 + t$, the velocity at the output is $b(\hat{X}_{0,t}(1)) = 1 + t + t^2/2$, and the residual is their difference,

$$r_{0,t}(1) = (1 + t) - \Big(1 + t + \tfrac{1}{2}t^2\Big) = -\tfrac{1}{2}t^2,$$

which ranges from -0.031250 at $t = 0.25$ to -0.500000 at $t = 1$. Using the supremum $\varepsilon = 0.5$, Grönwall's bound gives $0.5\,(e - 1) = 0.859141$, about four times the actual error 0.218282. The estimate uses worst-case exponential growth even though the residual is concentrated near the end of the interval.

The Eulerian residual on the same map is

$$q_{s,t}(x) = \partial_s \hat{X}_{s,t}(x) + b(x,s)\,\partial_x \hat{X}_{s,t}(x) = x\big[-1 - h\big] + x\big[1 + h + \tfrac{1}{2}h^2\big] = \tfrac{1}{2}x\,h^2,$$

which at $x = 1$, $s = 0$, $t = 1$ equals 0.5, the same size as the Lagrangian residual there. To build the estimate from it we integrate the residual along the true trajectory $x_\sigma = e^\sigma$ with $h = 1 - \sigma$,

$$\int_0^1 \tfrac{1}{2}\, e^{\sigma}(1 - \sigma)^2\, d\sigma = \tfrac{1}{2}\, e \int_0^1 u^2 e^{-u}\, du = \tfrac{1}{2}\, e\, \Big(2 - \tfrac{5}{e}\Big) = 0.218282,$$

using the substitution $u = 1 - \sigma$ and $\int_0^1 u^2 e^{-u} du = 2 - 5/e = 0.160603$. The estimate reproduces the endpoint error exactly, to six figures, because the residual has one sign along the path and the inequality is then an equality.

For the composition residual, choose $u = 0.5$. The two short maps give $\hat{X}_{0,0.5}(1) = 1.625$, followed by $\hat{X}_{0.5,1}(1.625) = 1.625 \times 1.625 = 2.640625$. Comparing with $\hat{X}_{0,1}(1) = 2.5$ gives a residual of -0.140625, the smallest of the three. It still cannot bound the endpoint error: the λ-family in

Section 6.21.1 has zero composition residual for every positive endpoint. Table 6.1 lists the three residuals and their endpoint-error estimates.

This model has zero anchor loss and an endpoint error of 8.0%, so accuracy on the diagonal alone is insufficient. Composing two steps reduces the endpoint error by nearly a factor of three: 2.640625 is closer to the true endpoint than 2.5. Additional legs improve this model even though its training residual compares only its own predictions.

Table 6.1: The three residuals evaluated on the truncated map $\hat{X}_{s,t}(x) = x[1 + h + h^2/2]$, $h = t - s$, for the flow $dx/dt = x$ at $s = 0$, $t = 1$, $x = 1$, where the exact endpoint is $e = 2.718282$ and the model lands at 2.500000. The residual column is the value of each residual at that point; the estimate column is the bound on the endpoint error that the residual carries. The model has zero anchor loss, so all three residuals are measuring the same off-diagonal error.

residual	value at $(0, 1, 1)$	endpoint estimate
Lagrangian	-0.500000	0.859141 by Grönwall
Eulerian	$+0.500000$	0.218282 by the path integral
composition	-0.140625	none
actual endpoint error		0.218282

6.22 Flow Map Matching and self-distillation

Flow Map Matching [Boffi et al., 2025a] trains a two-time map using the three flow axioms, their associated identities, and a squared residual for each identity. The paper proves the bounds in Section 6.21.1. We evaluate the map once to move between two times and train it by enforcing one of these identities.

For distillation, we begin with a velocity model pretrained by flow matching. We evaluate this teacher at the map's output for the Lagrangian target and at its input for the Eulerian transport velocity. Training requires forward passes through the frozen teacher without additional data. The teacher's accuracy limits the student's accuracy at the data endpoint.

For self-distillation, we use the model's own diagonal velocity $\hat{v}_{t,t}$ as the teacher [Boffi et al., 2025b]. We train that diagonal with $\mathcal{L}_b$ on directly sampled interpolant points. The off-diagonal residual constrains the rest of the triangle relative to this evolving diagonal. Two choices stabilize training: we detach the off-diagonal target so that the diagonal receives gradients only from the anchor, and we add each off-diagonal loss to the same anchor. This shared term fixes the model's scale while the residuals constrain its predictions away from the diagonal.

Each residual relates predictions at different time pairs. Those relations alone do not fix a global rescaling, as the λ-family in Section 6.21.1 demonstrates. Only the anchor depends on the data through $\dot{I}_t = X_1 - X_0$ for the linear interpolant. The residuals extend that constraint away from the diagonal.

The Lagrangian residual performed best on the image datasets in the self-distillation experiments [Boffi et al., 2025b]. On tokens, the ranking reverses for the same three losses, reflecting the dependence on the state space and target form. All three identities agree at their common minimum.

6.23 The three residuals in code

We can implement all three residuals using the same model, sampled times, and interpolant points. Each requires either a forward-mode derivative or an additional forward pass. We first define the average-velocity network and its induced map.

```
class Map(torch.nn.Module):
    def v(self, x, s, t):                          # the network
        return self.net(torch.cat([x, s, t], -1))

    def X(self, x, s, t):                          # identity axiom, free
        return x + (t - s) * self.v(x, s, t)
```

We concatenate the state and both times as the network input and predict the average velocity $v_{s,t}(x)$. Substituting into $X_{s,t}(x) = x + (t-s)v_{s,t}(x)$ enforces identity for any weights: at $t = s$, the second term vanishes and `X` returns its input. No identity loss is needed.

```
def anchor(m, x0, x1, t):                          # tangent axiom
    xt = (1 - t) * x0 + t * x1
    return ((m.v(xt, t, t) - (x1 - x0)) ** 2).mean()

def lagrangian(m, xs, s, t):                       # d/dt X = b at the output
    f = lambda tt: m.X(xs, s, tt)
    X, dXdt = torch.func.jvp(f, (t,), (torch.ones_like(t),))
    return ((dXdt - m.v(X, t, t).detach()) ** 2).mean()
```

In the anchor we build the linear interpolant $I_t = (1-t)x_0 + tx_1$ from a source sample and a data sample, evaluate the model on the diagonal, and regress it onto $\dot{I}_t = x_1 - x_0$. This is the flow matching loss of Chapter 2 with both time arguments set equal, and every one of the three residuals is added to it.

To compute the Lagrangian residual, we fix the state and start time in the closure `f` and differentiate in the end time t. Calling `torch.func.jvp` with tangent one in t returns the map and its time derivative in a single forward-mode pass. We evaluate the diagonal velocity at the transported point X for the target and apply `.detach()` there. The anchor alone then trains the diagonal.

```
def eulerian(m, xs, s, t):                         # d/ds X + b . grad X = 0
    vs = m.v(xs, s, s).detach()
    f = lambda xx, ss: m.X(xx, ss, t)
    _, dX = torch.func.jvp(f, (xs, s), (vs, torch.ones_like(s)))
    return (dX ** 2).mean()

def composition(m, xs, s, u, t):                   # two short maps, one long one
    tgt = m.X(m.X(xs, s, u), u, t).detach()
    return ((m.X(xs, s, t) - tgt) ** 2).mean()
```

For the Eulerian residual, we differentiate jointly in the state and start time. The tangent $(v, 1)$ in the arguments (x, s) gives $\partial_s X + v \cdot \nabla_x X$ in one forward-mode pass. This is the full left-hand side of the Eulerian identity, whose target is zero. The number of passes matches the Lagrangian residual, but the tangent now includes every state coordinate.

For composition, we evaluate and compose two short maps, detach the result, and compare it with the long map. This requires three forward passes, two detached, with no derivative. It is the simplest

implementation, although its residual gives no endpoint-error bound, as shown in Section 6.21.1.

We place the stop-gradient differently for the two targets. In `lagrangian`, we detach the diagonal velocity so that errors in the map cannot alter the diagonal. In `composition`, we detach the composed target and update only the long prediction. We sample $s \le t$ over the triangle and u between them. A fixed fraction of the batch uses $s = t$, since a continuous draw has zero probability of selecting the diagonal. None of the four functions simulates a trajectory: we construct I_s directly from a source sample and a data sample. Each update therefore requires two or three network evaluations per residual term, plus one for the anchor.

6.23.1 Four models on four Gaussian blobs

We train four models on a mixture of four Gaussian blobs in two dimensions. Each uses the same network, anchor, and training budget; only the added residual changes. We compare the one-step output with the endpoint obtained by accurately integrating that model's velocity field. We also measure the fraction of samples that reach one of the four modes.

Table 6.2: Four models on a mixture of four Gaussian blobs in two dimensions, with identical network, anchor and training budget. The first column is the loss each model was trained on. The second is the distance between the one-step output and the endpoint of the model's own ordinary differential equation integrated accurately, so a value near zero means the one-step map agrees with the velocity field it carries on its diagonal. The third is the fraction of generated samples landing on one of the four modes.

trained on	distance to its own ODE	fraction on a mode
the diagonal alone	0.896	0.12
+ Lagrangian	0.292	0.84
+ Eulerian	0.242	0.82
+ composition	0.325	0.84

With only diagonal training, one-step flow matching ends 0.896 from its accurately integrated velocity endpoint, and 12% of generated samples reach a mode (Table 6.2). Adding any residual reduces that distance to between 0.242 and 0.325 and increases the fraction on a mode to between 0.82 and 0.84.

The three residuals perform within each other's spread in this experiment, so the published ranking does not appear at this scale. With two dimensions, four modes, and a four-layer network, the Eulerian spatial gradient is small and composition error has little room to accumulate. The differences become apparent for images and language, with millions of state coordinates. At this small scale, adding any residual substantially improves the map trained with the anchor alone.

6.23.2 Choosing a residual

We can compare derivative requirements, evaluation counts, and endpoint-error estimates in Table 6.3.

The Lagrangian residual requires an end-time derivative and no spatial gradient. The Eulerian residual requires a start-time derivative and a directional derivative through every state coordinate, computed in one forward-mode pass; it also gives the tighter error estimate. The composition residual requires three forward passes and no derivative. It gives no endpoint-error estimate and, even with the anchor, permits the discontinuous stay-still map.

Table 6.3: The three identities compared. The derivative column names which time argument is differentiated; the gradient column marks whether a spatial gradient of the map appears; the passes column counts network evaluations per residual term; the estimate column names the endpoint bound of Section 6.21.1; the last column marks whether the residual, added to the diagonal anchor, excludes the stay-still map.

identity	derivative in	spatial gradient	passes	endpoint estimate	excludes stay-still
Lagrangian	end time t	no	2	Grönwall	yes
Eulerian	start time s	yes	2	path integral	yes
composition	none	no	3	none	no

6.24 Deterministic maps and atoms

The Jacobian, change-of-variables formula, and inverse map above rely on a continuous state space. Language, molecules, and proteins are sequences over finite alphabets. In Chapter 5, we generated these sequences with jump rates and probability velocities. A deterministic map on the sequences themselves cannot realize every target distribution.

Let's take a source with two atoms, each of mass $1/2$, and a target with three atoms, each of mass $1/3$. A deterministic map sends each source point somewhere whole, so each of the two masses arrives undivided at one target point. At most two of the three target atoms receive anything, and whichever they are, they receive $1/2$, $1/2$ or 1. No arrangement produces three masses of $1/3$ (Figure 6.4).

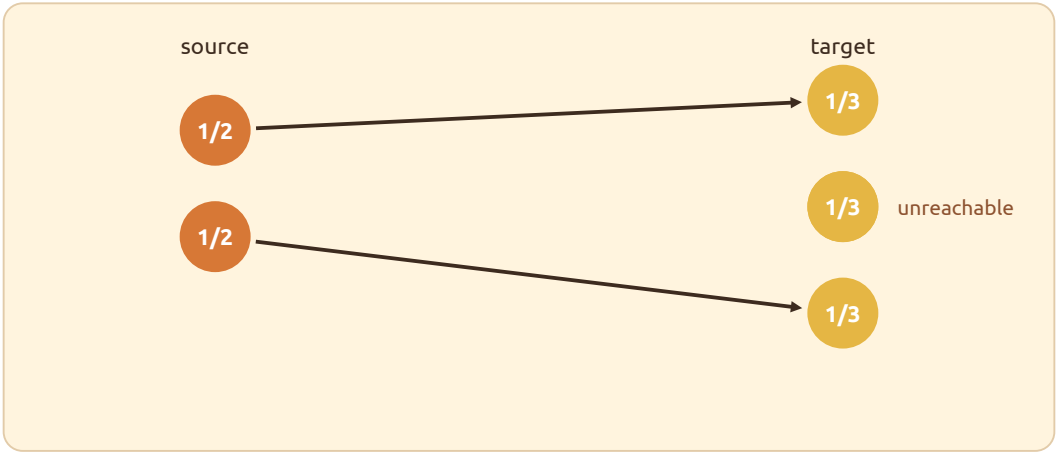


Figure 6.4: A source distribution with two atoms of mass $1/2$ each and a target with three atoms of mass $1/3$ each. A deterministic map sends each source atom whole to one target atom, drawn as the two arrows, so the masses that arrive are sums of the numbers $1/2$ and $1/2$ and lie in $\{0, 1/2, 1\}$. The third target atom is marked unreachable: no deterministic map, smooth or not, produces a mass of $1/3$ anywhere.

For a general finite source distribution, the smallest positive mass imposes a lower bound.

Proposition 6.5 (Reachable masses under a deterministic map)**.** *Let μ be a probability distribution on a finite set with smallest positive mass $\mu_{\min} = \min\{\mu(x) : \mu(x) > 0\}$, and let f be any map from*

that set to itself. Then the pushforward $\nu = f_{\#}\mu$ satisfies

$$\nu(y) = \sum_{x \in f^{-1}(y)} \mu(x) \quad \Longrightarrow \quad \nu(y) \in \{0\} \cup [\mu_{\min},\, 1]$$

for every y. Consequently no map carries μ to a target that places mass strictly between zero and $\mu_{\min}$ at any point.

"The transported mass $\nu(y)$ sums the source masses $\mu(x)$ over all inputs in the preimage $f^{-1}(y)$. It is zero when none has positive mass and otherwise at least the smallest positive source mass $\mu_{\min}$."

To prove the bound, sum the source masses over the preimage $f^{-1}(y)$. An empty preimage has zero mass. A preimage containing positive source mass contributes at least $\mu_{\min}$, so its total cannot lie strictly between zero and $\mu_{\min}$. The argument requires no smoothness, invertibility, or continuity.

For the two-atom source, $\mu_{\min} = 1/2$, so the bound restricts reachable masses to $\{0\} \cup [0.5, 1]$. Each target atom has mass $1/3 = 0.3333$, outside that range. A target with mass $\mu_{\min}/2 = 0.25$ at any point is also unreachable, regardless of how we refine the map. This restriction applies to all deterministic maps, extending the earlier obstruction for diffeomorphisms. Proposition 3.5 of Flow Map Language Models uses it to rule out a sample-level flow map on tokens [Lee et al., 2026].

The mask and uniform sources from Chapters 4 and 5 give two extremes. A fully masked sequence is one point in $\mathcal{V}^L$, with $\mu_{\min} = 1$. Any deterministic map therefore generates the same sequence on every call. For a uniform source over $\mathcal{V}^L$, $\mu_{\min} = |\mathcal{V}|^{-L} = 10^{-574}$ at the vocabulary and sequence sizes considered here. The forbidden masses are then too small to matter numerically. To permit arbitrary division of source mass, we need a map whose state contains more than a single sequence.

6.25 Sequence-distribution states

We can apply a deterministic map to a distribution over sequences. The rate matrix from Chapter 4 already defines such an evolution,

$$\dot{p}_t = Q_t\, p_t, \qquad p_t \in \Delta^{|\mathcal{V}|^L - 1},$$

"The sequence distribution p_t changes at the rate $Q_t p_t$, where Q_t is the transition-rate matrix. This deterministic evolution takes place in the simplex over the $|\mathcal{V}|^L$ sequences of length L from vocabulary $\mathcal{V}$."

The later distribution is a deterministic function of the earlier distribution. Its cocycle is the Chapman-Kolmogorov equation from Chapter 4. Because the state is a point in a convex set, we can divide probability mass continuously; the atomic obstruction does not apply.

The state has $|\mathcal{V}|^L$ coordinates. With the vocabulary of a standard tokenizer, $|\mathcal{V}| = 30{,}522$, and a context of $L = 128$ positions, that is

$$30{,}522^{128} = 10^{\,128 \log_{10} 30{,}522} = 10^{574.0},$$

coordinates, too many to store or evaluate. Per-position factorization reduces this to L simplices of size $|\mathcal{V}|$, or $128 \times 30{,}522 = 3{,}906{,}816$ numbers. As calculated in Chapter 5, long steps with a factorized sampler discard correlations between positions.

Another construction embeds each token as a one-hot vector in $\mathbb{R}^{|\mathcal{V}|}$, samples a Gaussian source in the same space, and evolves a state in $\mathbb{R}^{L\times|\mathcal{V}|}$. The source is continuous, so there are no source atoms to split. We can again use a diffeomorphism, its Jacobian, the density-transport formula, and its inverse. An argmax at each position decodes the final continuous state into tokens.

Consider one position with DNA vocabulary $|\mathcal{V}| = 4$, ordered as $(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T})$. Choose data token G, represented by $\mathbf{e}_{\mathrm{G}} = (0, 0, 1, 0)$, and Gaussian source draw $x_0 = (0.5, -0.3, 0.2, 1.1)$. Evaluating $I_t = (1-t)x_0 + t\,\mathbf{e}_{\mathrm{G}}$ at three times gives

$$\begin{aligned} I_{0.3} &= (0.350,\ -0.210,\ 0.440,\ 0.770),\\ I_{0.45} &= (0.275,\ -0.165,\ 0.560,\ 0.605),\\ I_{0.5} &= (0.250,\ -0.150,\ 0.600,\ 0.550). \end{aligned}$$

The argmax decodes to T at $t = 0.3$ and at $t = 0.45$, and to G from $t = 0.5$ onward. The G coordinate overtakes the T coordinate when

$$(1-t)(0.2) + t > (1-t)(1.1) \quad\Longleftrightarrow\quad 1.9\,t > 0.9 \quad\Longleftrightarrow\quad t > 0.473684,$$

and it overtakes the A coordinate earlier, at $t = 0.3/1.3 = 0.230769$. The continuous state remains in $\mathbb{R}^4$, while its decoded token is a step function of t. After $t = 0.4737$, the decoded token stays fixed.

6.26 The two-time denoiser

At an intermediate time, the map returns a point in $\mathbb{R}^{L\times|\mathcal{V}|}$ with no probabilistic interpretation, so we compare such outputs through squared error. To use a categorical divergence, we need a prediction that is a distribution at every pair of times. We obtain one by multiplying the average velocity by the full remaining time $1 - s$,

$$\boxed{\delta_{s,t}(x) \;=\; x + (1-s)\,v_{s,t}(x)}$$

"The two-time denoiser $\delta_{s,t}(x)$ equals the current state x plus the remaining time $1 - s$ multiplied by the average velocity $v_{s,t}(x)$ over the interval from s to t."

- $\delta_{s,t}(\cdot)$ is the two-time denoiser, a prediction of the data made at the state x from the information the model has about the interval $[s, t]$. In the parameterized model it is written $d_\theta(\cdot \mid x, s, t)$.
- $1 - s$ is the remaining time measured from the start of the interval, not from its end.
- $v_{s,t}(x)$ is the average velocity over $[s, t]$, the quantity the network already produces.

On the diagonal, $v_{t,t}(x) = b(x, t)$, so $\delta_{t,t}(x) = x + (1-t)b(x, t)$ is the denoiser from Chapter 2. For the linear interpolant, it equals the posterior mean $\mathbb{E}[X_1 \mid X_t = x]$. We denote this single-time denoiser by $D_t(x) = \delta_{t,t}(x)$. Away from the diagonal, $\delta_{s,t}$ predicts the data over a finite interval. Three groups introduced it independently under different names. Flow Map Language Models uses the two-time denoiser δ [Lee et al., 2026]. Categorical Flow Maps uses the partial denoiser π [Roos et al., 2026], and Discrete Flow Maps uses the mean denoiser ψ [Potaptchik et al., 2026b]. Each establishes a different result and trains a different identity.

6.26.1 The convex form and the one-step output

If we invert the definition, $v_{s,t}(x) = (\delta_{s,t}(x) - x)/(1-s)$, and substitute it into the parameterization $X_{s,t}(x) = x + (t-s)v_{s,t}(x)$, the map becomes a weighted average,

$$X_{s,t}(x) = x + \frac{t-s}{1-s}(\delta_{s,t}(x) - x) = \Big(1 - \frac{t-s}{1-s}\Big)x + \frac{t-s}{1-s}\,\delta_{s,t}(x),$$

and simplifying $(1-s) - (t-s) = 1-t$ gives the denoiser parameterization,

$$\boxed{X_{s,t}(x) \;=\; \frac{1-t}{1-s}\,x \;+\; \frac{t-s}{1-s}\,\delta_{s,t}(x)}$$

"The destination $X_{s,t}(x)$ is a weighted average of the starting state x, with weight $(1-t)/(1-s)$, and the two-time denoiser prediction $\delta_{s,t}(x)$, with weight $(t-s)/(1-s)$."

For $s \le t \le 1$, both weights are nonnegative and sum to $((1-t) + (t-s))/(1-s) = 1$. Each step is therefore a convex combination of the current state and predicted endpoint. At $s = 0.2$, $t = 0.8$, the weights are $(0.2/0.8, 0.6/0.8) = (0.25, 0.75)$, moving three quarters of the way toward the prediction. At $t = s$, they are $(1, 0)$, enforcing identity for every δ without training. This agrees with the consistency-trajectory parameterization using $a_{s,t} = (1-t)/(1-s)$, whose head g_θ is the two-time denoiser.

Setting the end time to one collapses the first weight:

$$X_{s,1}(x) = \delta_{s,1}(x) \in \big(\Delta^{|\mathcal{V}|-1}\big)^L$$

"The endpoint map $X_{s,1}(x)$ equals the denoiser prediction $\delta_{s,1}(x)$, a vocabulary probability vector at each of the L sequence positions."

One-step generation returns a vocabulary distribution at every position, which we decode with argmax. Earlier states blend a Gaussian source with a one-hot vector and lie off the simplex; the terminal state lies on it. To use distributional losses at $t < 1$, we also need to establish that $\delta_{s,t}$ is a probability vector at those times.

6.26.2 The two-time denoiser on the simplex

We can prove the simplex property by integrating along a trajectory [Lee et al., 2026]. For the linear interpolant, solve $D_u(x) = x + (1-u)b(x,u)$ for the velocity to obtain $b(x,u) = (D_u(x) - x)/(1-u)$. Let $y_u = X_{s,u}(x)$ be the state at intermediate time $u \in [s,t]$, and rescale it as $w(u) = y_u/(1-u)$. Differentiating gives

$$\frac{dw}{du} = \frac{\dot{y}_u}{1-u} + \frac{y_u}{(1-u)^2} = \frac{D_u(y_u) - y_u}{(1-u)^2} + \frac{y_u}{(1-u)^2} = \frac{D_u(y_u)}{(1-u)^2},$$

where the two terms in y_u cancel. Integrating from s to t,

$$\frac{X_{s,t}(x)}{1-t} - \frac{x}{1-s} = \int_s^t \frac{D_u(X_{s,u}(x))}{(1-u)^2}\,du,$$

and if we divide the convex form of Section 6.26.1 by $1-t$, its left side becomes exactly that difference, with the coefficient $(t-s)/((1-s)(1-t))$ left over on $\delta_{s,t}$. Solving for $\delta_{s,t}$,

$$\delta_{s,t}(x) = \frac{(1-s)(1-t)}{t-s} \int_s^t \frac{D_u(X_{s,u}(x))}{(1-u)^2}\, du$$

"The two-time denoiser $\delta_{s,t}(x)$ averages the single-time prediction $D_u(X_{s,u}(x))$ along the trajectory, with weight $1/(1-u)^2$ at intermediate time u. The prefactor $(1-s)(1-t)/(t-s)$ normalizes the integral over $[s,t]$."

The weight in that integral is a probability density on $[s,t]$. Its total mass is

$$\frac{(1-s)(1-t)}{t-s} \int_s^t \frac{du}{(1-u)^2} = \frac{(1-s)(1-t)}{t-s} \left(\frac{1}{1-t} - \frac{1}{1-s} \right) = 1,$$

since the bracket equals $((1-s)-(1-t))/((1-t)(1-s)) = (t-s)/((1-t)(1-s))$, which cancels the prefactor exactly. Each D_u is a posterior over the data token at that position, so it is on the simplex, and a probability-weighted average of simplex points is a simplex point. Therefore $\delta_{s,t}$ lies on the simplex at every pair of times.

The weight is far from uniform, which matters for how the prediction is formed. At $s = 0.2$ and $t = 0.8$ the prefactor is $(0.8)(0.2)/0.6 = 0.266667$, and the density $0.266667/(1-u)^2$ has the values

$$0.416667 \text{ at } u = 0.2, \quad 1.066667 \text{ at } u = 0.5, \quad 2.176871 \text{ at } u = 0.65, \quad 6.666667 \text{ at } u = 0.8,$$

rising by a factor of sixteen across the interval. If we solve $0.266667\,(1/(1-m) - 1/0.8) = 0.5$ for the median we get $m = 0.68$, so half of the weight is in $[0.68,\ 0.8]$, the last fifth of the interval. A long-range prediction is dominated by the denoiser evaluated near its own end time, where the single-time denoiser is sharpest.

6.26.3 The cocycle in denoiser coordinates

We can derive a denoiser training identity by substituting its parameterization into the cocycle. Let $s < u < t$ and $y = X_{s,u}(x)$. Expand both sides of $X_{s,t}(x) = X_{u,t}(y)$ using the convex form,

$$\frac{1-t}{1-s}x + \frac{t-s}{1-s}\delta_{s,t}(x) = \frac{1-t}{1-u}\left[\frac{1-u}{1-s}x + \frac{u-s}{1-s}\delta_{s,u}(x)\right] + \frac{t-u}{1-u}\delta_{u,t}(y).$$

The terms in x cancel, because $\frac{1-t}{1-u} \cdot \frac{1-u}{1-s} = \frac{1-t}{1-s}$. Multiplying what remains by $(1-s)/(t-s)$,

$$\boxed{\delta_{s,t}(x) = \gamma\, \delta_{s,u}(x) + (1-\gamma)\, \delta_{u,t}(X_{s,u}(x)), \qquad \gamma = \frac{(1-t)(u-s)}{(1-u)(t-s)}}$$

"The long-interval denoiser $\delta_{s,t}(x)$ combines the first-leg prediction $\delta_{s,u}(x)$ with weight γ and the second-leg prediction $\delta_{u,t}(X_{s,u}(x))$ with weight $1-\gamma$. The intermediate time u sets the weight $\gamma = (1-t)(u-s)/((1-u)(t-s))$."

The second coefficient comes out as $(t-u)(1-s)/((1-u)(t-s))$, and the two sum to one because

$$(1-t)(u-s) + (t-u)(1-s) = \big(u - s - tu + ts\big) + \big(t - ts - u + us\big) = (t-s)(1-u),$$

which is the denominator. Both weights are nonnegative for $s < u < t \le 1$, so the prediction is a convex combination of two simplex points. It therefore stays on the simplex. We can use it as the composition target without projection or clipping.

Take two legs of the same length, with start time 0.2, midpoint 0.5 and end time 0.8:

$$(s, u, t) = (0.2,\ 0.5,\ 0.8) \implies \gamma = \frac{0.2 \times 0.3}{0.5 \times 0.6} = \frac{0.06}{0.30} = 0.2.$$

The first leg has weight 0.2 and the second 0.8, even though the legs have equal length. The asymmetry comes from the remaining time. The prediction over $[0.5, 0.8]$ is formed from a state with half the remaining time of the one over $[0.2, 0.5]$, and the convex weights scale with $1/(1-u)$. Moving the midpoint earlier to $u = 0.35$ drops γ to 0.076923, and moving it later to $u = 0.65$ raises it to 0.428571.

We can check the denoiser identity on $dx/dt = x$, where $\delta_{s,t}(x) = x\big[1 + (1-s)(e^{t-s} - 1)/(t-s)\big]$. For $x = 1$ and $(s, u, t) = (0.2, 0.5, 0.8)$, the predictions are

$$\delta_{0.2,0.8}(1) = 2.096158, \qquad \delta_{0.2,0.5}(1) = 1.932957, \qquad \delta_{0.5,0.8}\big(X_{0.2,0.5}(1)\big) = 2.136959,$$

where the intermediate state is $X_{0.2,0.5}(1) = e^{0.3} = 1.349859$. Combining the last two with $\gamma = 0.2$,

$$0.2 \times 1.932957 + 0.8 \times 2.136959 = 2.096158,$$

matching the long prediction to six figures. Substituting into the convex form also recovers the map: $0.25 \times 1 + 0.75 \times 2.096158 = 1.822119 = e^{0.6} = X_{0.2,0.8}(1)$.

At the end time one the weight vanishes:

$$t = 1 \implies \gamma = 0, \qquad \delta_{s,1}(x) = \delta_{u,1}\big(X_{s,u}(x)\big).$$

The endpoint prediction is constant along a trajectory. On the scalar flow, both sides equal $e^{0.8} = 2.225541$, the destination from $x = 1$ at $s = 0.2$. We obtain the same value directly or after moving to $u = 0.5$. This is the prediction used for one-step sampling. Training must deliberately include $t = 1$, since a continuous distribution over triples (s, u, t) assigns zero probability to this slice.

6.26.4 The identity on an exact categorical model

Let's also calculate a categorical example on the simplex. Use two tokens, G and T, embedded as $\mathbf{e}_{\mathrm{G}} = (1, 0)$ and $\mathbf{e}_{\mathrm{T}} = (0, 1)$ in $\mathbb{R}^2$, with $P(\mathrm{G}) = 0.7$ and $P(\mathrm{T}) = 0.3$. Under a Gaussian source and linear interpolant, the token posterior given $X_t = x$ is proportional to $P(w) \exp\big(-\|x - t\,\mathbf{e}_w\|^2/2(1-t)^2\big)$. Expanding the squared distance gives $\|x\|^2 - 2t\,x_w + t^2$. Only the middle term depends on token w, so the denoiser is

$$D_t(x) = \mathrm{softmax}\Big(\log P(w) + \frac{t\,x_w}{(1-t)^2}\Big)_w, \qquad b(x,t) = \frac{D_t(x) - x}{1-t},$$

with the velocity read off from $D_t(x) = x + (1-t)b(x,t)$.

We can take the state $x = (0.40,\ 0.90)$ at $s = 0.2$, which comes from the source draw $x_0 = (0.25,\ 1.125)$ with the data token G. Its argmax decodes to T, the wrong token. The single-time

denoiser there already puts more mass on G, with $D_{0.2}(x) = (0.6662,\ 0.3338)$. Integrating the velocity with a fourth-order Runge-Kutta scheme,

$$X_{0.2,\,0.5}(x) = (0.486410,\ 0.701090), \qquad X_{0.2,\,0.8}(x) = (0.603892,\ 0.471108),$$

so the decoded token flips from T to G somewhere between $t = 0.5$ and $t = 0.8$. The three two-time denoisers are

$$\begin{aligned} \delta_{0.2,\,0.5}(x) &= (0.630428,\ 0.369572), \\ \delta_{0.5,\,0.8}(X_{0.2,0.5}(x)) &= (0.682213,\ 0.317787), \\ \delta_{0.2,\,0.8}(x) &= (0.671856,\ 0.328144), \end{aligned}$$

each with non-negative entries summing to one, as required by the argument of Section 6.26.2. Combining the first two with $\gamma = 0.2$,

$$0.2 \times 0.630428 + 0.8 \times 0.682213 = 0.671856, \qquad 0.2 \times 0.369572 + 0.8 \times 0.317787 = 0.328144,$$

which is the long prediction in both coordinates. Pushing it back through the convex form with the weights $(0.25,\ 0.75)$,

$$0.25\,(0.40,\ 0.90) + 0.75\,(0.671856,\ 0.328144) = (0.603892,\ 0.471108) = X_{0.2,0.8}(x).$$

At the data endpoint, $\delta_{0.2,1}(x) = (0.999988, 0.000012)$ assigns almost all mass to G. One map evaluation from $(s, t) = (0.2, 1)$ therefore changes the decoded token from T at the input to the data token G at the output.

6.26.5 The training target under each identity

The identities have different target types in denoiser coordinates. The composition target is a convex combination of distributions, so we can use KL divergence between normalized vectors. The Lagrangian and Eulerian targets contain derivatives of δ, whose entries have no sign constraint. Their derivative counts remain those in Section 6.23.2.

This sign issue follows from the simplex constraint. The coordinates of a curve on the simplex sum to one, so the coordinates of its derivative sum to zero. Any nonzero tangent vector must therefore have at least one negative entry. Such a tangent is not a distribution.

Differentiating the integral representation of Section 6.26.2 in t, with $C(t) = (1 - s)(1 - t)/(t - s)$ and $A(t) = \int_s^t D_u(y_u)(1 - u)^{-2}du$, so that $\delta_{s,t} = C(t)A(t)$,

$$C'(t) = -\frac{(1-s)^2}{(t-s)^2}, \qquad \frac{C'(t)}{C(t)} = -\frac{1-s}{(t-s)(1-t)}, \qquad \frac{C(t)}{(1-t)^2} = \frac{1-s}{(t-s)(1-t)},$$

and the product rule $\partial_t\delta = C'A + C\,D_t(y_t)(1 - t)^{-2}$ collapses to

$$\partial_t\,\delta_{s,t}(x) = \frac{1-s}{(t-s)(1-t)}\Big[\,D_t(X_{s,t}(x)) - \delta_{s,t}(x)\,\Big].$$

"The end-time derivative $\partial_t\delta_{s,t}(x)$ equals the single-time denoiser $D_t(X_{s,t}(x))$ at the destination minus the two-time prediction $\delta_{s,t}(x)$, multiplied by the time-dependent factor $(1-s)/((t-s)(1-t))$."

The bracket is a difference of two simplex points, so its coordinates sum to zero. Now take $s = 0.2$, $t = 0.8$ and the DNA alphabet, with $\delta_{0.2,0.8} = (0.10,\ 0.20,\ 0.60,\ 0.10)$ and $D_{0.8}$ at the transported

state equal to (0.05, 0.10, 0.80, 0.05). The prefactor is $0.8/(0.6 \times 0.2) = 6.6667$, the bracket is $(-0.05, -0.10, 0.20, -0.05)$, and the target is

$$\partial_t \, \delta_{0.2,0.8} = (-0.3333,\ -0.6667,\ 1.3333,\ -0.3333),$$

three of whose four entries are negative and whose sum is zero. The Eulerian target has the same character, since it too is built from derivatives of δ.

The exact model of Section 6.26.4 confirms the identity and the sign pattern together. There the two-time denoiser is $\delta_{0.2,0.8}(x) = (0.671856,\ 0.328144)$, and the single-time denoiser at the transported state is

$$D_{0.8}(X_{0.2,0.8}(x)) = (0.970773,\ 0.029227),$$

so the bracket is $(0.298917,\ -0.298917)$ and the target is

$$\partial_t \, \delta_{0.2,0.8}(x) = 6.6667 \times (0.298917,\ -0.298917) = (1.992776,\ -1.992776).$$

A central difference of $\delta_{0.2,t}(x)$ in t, with step 10^{-4}, gives $(1.992773,\ -1.992773)$, matching to six figures. This target is a direction with zero-sum coordinates. Changing the alphabet or prior does not make it a distribution.

KL divergence requires normalized distributions for both arguments, so it is available for the composition target. Cross-entropy requires a normalized student, which we obtain with a softmax head, and can be evaluated with the derivative targets.

6.27 Flow Map Language Models

Flow Map Language Models [Lee et al., 2026] uses the one-hot lift, predicts a two-time denoiser, and reconstructs the map with the convex parameterization. We train its composition identity in denoiser coordinates, using KL divergence for both loss terms.

6.27.1 Diagonal and off-diagonal KL losses

The loss has one term for the off-diagonal identity and one anchor on the diagonal:

$$\mathcal{L} = \mathbb{E}\Big[\textstyle\sum_l D_{\mathrm{KL}}\Big(\bar{\delta}^l_{s,t} \,\Big\|\, \hat{\delta}^l_{s,t}(I_s)\Big)\Big] + \mathbb{E}\Big[\textstyle\sum_l D_{\mathrm{KL}}\Big(D^l_t(I_t) \,\Big\|\, \hat{\delta}^l_{t,t}(I_t)\Big)\Big]$$

"The loss $\mathcal{L}$ averages two sums over positions l: the KL divergence from the composed target $\bar{\delta}^l_{s,t}$ to the student's long prediction $\hat{\delta}^l_{s,t}(I_s)$, and the KL divergence from the teacher denoiser $D^l_t(I_t)$ to the student's diagonal prediction $\hat{\delta}^l_{t,t}(I_t)$. The averages use sampled times and interpolant states I_s and I_t."

- $\hat{\delta}^l_{s,t}$ is the student's two-time denoiser at position l, the softmax head of the network, written $d_\theta(\cdot \mid x, s, t)$ when the parameters matter.
- $\bar{\delta}^l_{s,t}$ is the composed target.
- I_s and I_t are interpolant points at the two times, drawn without simulating any trajectory.
- D^l_t is a single-time denoiser used as the teacher on the diagonal.

We construct the target by substituting the student into both predictions of the denoiser cocycle and detaching the result,

$$\bar{\delta}_{s,t} = \mathrm{sg}\Big(\gamma\,\hat{\delta}_{s,u}(I_s) + (1-\gamma)\,\hat{\delta}_{u,t}\big(\hat{X}_{s,u}(I_s)\big)\Big),$$

so the two short predictions and intermediate state are all detached model outputs. Only the long prediction $\hat{\delta}_{s,t}$ receives a gradient from this term. For diagonal training, we can replace D_t^l with the one-hot data token. The KL term then equals cross-entropy between that token and the model prediction, allowing training from data without a teacher.

Let's compute one term of that loss on the DNA alphabet, at the triple $(s,u,t) = (0.2,\, 0.5,\, 0.8)$ where $\gamma = 0.2$. Suppose the student predicts

$$\hat{\delta}_{0.2,0.5}(I_s) = (0.10,\ 0.20,\ 0.60,\ 0.10), \qquad \hat{\delta}_{0.5,0.8}\big(\hat{X}_{0.2,0.5}(I_s)\big) = (0.05,\ 0.10,\ 0.80,\ 0.05)$$

over $(\mathrm{A},\mathrm{C},\mathrm{G},\mathrm{T})$. The composed target is

$$\bar{\delta}_{0.2,0.8} = 0.2\,(0.10,\ 0.20,\ 0.60,\ 0.10) + 0.8\,(0.05,\ 0.10,\ 0.80,\ 0.05) = (0.06,\ 0.12,\ 0.76,\ 0.06),$$

which sums to one, as the weights guarantee. If the student's long prediction is $\hat{\delta}_{0.2,0.8}(I_s) = (0.10,\ 0.15,\ 0.65,\ 0.10)$, the four terms of the KL divergence are

$$0.06 \log \frac{0.06}{0.10} = -0.030650, \qquad 0.12 \log \frac{0.12}{0.15} = -0.026777,$$
$$0.76 \log \frac{0.76}{0.65} = +0.118823, \qquad 0.06 \log \frac{0.06}{0.10} = -0.030650,$$

for a total of 0.030747 nats. Gradient descent transfers mass from A, C, and T to G, whose target probability exceeds the long prediction. The divergence is zero when student and target agree, satisfying consistency with the two-leg prediction. We sum this loss over L positions, average over training triples (s,u,t), and add the diagonal term.

6.27.2 The decoding clock

Uniform sampling in t allocates few evaluations to the times when decoding changes. A position decodes to the data token only when its coordinate exceeds the other $|\mathcal{V}| - 1$ coordinates. For $|\mathcal{V}| = 30{,}522$, most argmax changes occur late in the path, leaving many uniform grid points in regions with little change.

We can allocate evaluations according to the decoding error. Define the error rate and a reparameterized time by

$$P_e(t) = \tfrac{1}{L} \sum_l \mathbb{P}(g^l(x_t) \neq g^l(x_1)), \qquad \tau(t) = \frac{P_e(0) - P_e(t)}{P_e(0)}$$

"The decoding error $P_e(t)$ is the fraction of L positions where the current argmax $g^l(x_t)$ differs from the data token $g^l(x_1)$. The new time $\tau(t)$ equals one minus the remaining error $P_e(t)$ divided by the initial error $P_e(0)$."

- $g^l(x)$ is the argmax over the vocabulary at position l, the decoding rule of Section 6.25.

- $P_e(0)$ is the error rate at the source, close to one for a large vocabulary since a Gaussian draw agrees with the data token only by chance.

- τ runs from 0 at the source to 1 at the data, by construction.

Under the Gaussian interpolant this quantity is computable in closed form for one position. We can write the interpolant as $I_t = (1-t)x_0 + t\,\mathbf{e}_w$ with $x_0 \sim \mathcal{N}(0, \mathrm{Id})$ and w the data token at that position. The coordinate of w is $(1-t)z_w + t$ and every other coordinate is $(1-t)z_j$, so the position decodes correctly when

$$z_w + \frac{t}{1-t} > \max_{j \neq w} z_j,$$

a comparison of one standard normal shifted by $a = t/(1-t)$ against the maximum of $|\mathcal{V}|-1 = 30{,}521$ others. Integrating over z_w,

$$1 - P_e(t) = \int \phi(z)\, \Phi\big(z + a\big)^{\,|\mathcal{V}|-1} dz,$$

with ϕ and Φ the standard normal density and distribution function. At $t = 0$ the shift is zero and the integral is $1/|\mathcal{V}| = 3.2763 \times 10^{-5}$, as symmetry requires, so $P_e(0) = 0.999967$.

Evaluating the integral at four times gives

$$\tau(0.60) = 0.006, \quad \tau(0.75) = 0.141, \quad \tau(0.80) = 0.456, \quad \tau(0.85) = 0.931.$$

Only 0.6% of the decoding error is removed over the first six tenths of the path. Between $t = 0.75$ and $t = 0.85$, the error falls from 0.859 to 0.069, a decrease of 0.790 in one tenth of the interval. At $t = 0.80$, the error is still 0.544, so more than half of the positions decode incorrectly. Most decoding changes occur within this window of width about 0.1.

Correct decoding requires the shift $a = t/(1-t)$ to be comparable to the largest of 30,521 standard normal draws. That maximum concentrates near $\sqrt{2 \log 30{,}521} = 4.544$, with a correction that places its mean near 4.01. Solving $t/(1-t) = 4.01$ gives the transition time $t = 0.800$. Fluctuations of the maximum set the transition width. Their scale is $1/\sqrt{2 \log 30{,}521} = 0.22$ in a. Using $dt/da = (1-t)^2 = 0.04$ gives a width of order 0.01 in t. Increasing the vocabulary moves the transition later and narrows it because the maximum grows as $\sqrt{2 \log |\mathcal{V}|}$ and its spread shrinks as the reciprocal.

6.27.3 Four-step decoding example

To space four steps evenly in τ we solve $\tau(t) = 0.25,\ 0.5,\ 0.75$ and add the two endpoints. The grid that comes out is

$$t = 0,\ 0.773,\ 0.804,\ 0.828,\ 1,$$

and the solutions to six figures are 0.773490, 0.804481 and 0.828095. Three of the five times are inside an interval of width $0.828 - 0.773 = 0.055$. The first step crosses most of the path and settles almost no position, the next three do the decoding, and the last crosses the short remainder to the data.

The same four steps spaced evenly in t land at $t = 0,\ 0.25,\ 0.5,\ 0.75,\ 1$, whose clock values are

$$\tau = 0, \quad 0.00009, \quad 0.00124, \quad 0.14109, \quad 1,$$

so the first two steps reduce the decoding error by about one part in a thousand. The last step must remove the remaining 0.859, concentrating almost all decoding work in one evaluation.

A flow map can use this uneven grid because it is trained to jump between arbitrary time pairs. Euler integration loses accuracy on long steps, including the initial jump from $t = 0$ to $t = 0.773$. With a map trained over the full triangle, every adjacent grid pair is within the training domain, so we can choose the schedule directly.

We can read what each of the four steps does to the state from the convex form of Section 6.26.1. The step from s to t mixes the current state with the prediction in the proportion $((1-t)/(1-s),\ (t-s)/(1-s))$, and on this grid the four pairs are

$$(0.2270,\ 0.7730), \qquad (0.8634,\ 0.1366), \qquad (0.8776,\ 0.1224), \qquad (0,\ 1).$$

The first step moves the state 77.3% of the way to its own prediction, and a quarter of the decoding error is gone with it. The two middle steps move it 13.7% and 12.2% of the way, and between them they account for half of the decoding error. The last step discards the state entirely and returns the prediction, and the output then becomes a point of the simplex. A small displacement in $\mathbb{R}^{L\times|\mathcal{V}|}$ flips a large number of positions during the middle steps, because the state lies against the decision boundaries of the argmax at those times. The clock τ measures the flipping, and the convex weights measure the displacement.

The training triples are on the same grid. For $(s, u, t) = (0.773,\ 0.804,\ 0.828)$ the composition weight is

$$\gamma = \frac{(1-0.828)(0.804-0.773)}{(1-0.804)(0.828-0.773)} = \frac{0.172\times 0.031}{0.196\times 0.055} = 0.494620,$$

nearly an even split, because the two legs are short and both predictions are made from states with similar amounts of remaining time. Over the long first leg, $(s, u, t) = (0,\ 0.773,\ 0.804)$ gives $\gamma = 0.830144$, and the target is dominated by the prediction made at the source.

With the same model, teacher, and budget, the one-step generative perplexities are 119.34 for composition, 149.13 for Eulerian, and 193.08 for Lagrangian training. Their sample entropies are 4.16, 4.16, and 4.22 nats, respectively [Lee et al., 2026]. We report entropy alongside perplexity because repetitive generation can achieve low perplexity with little diversity. The entropies here differ by at most 0.06 nats, making the perplexity comparison meaningful.

This ranking reverses the image-model ranking in Section 6.22 because the target has a different form.

6.27.4 Dependence of the ranking on the target

For images, each target is a vector in $\mathbb{R}^d$, and squared error is defined for all three identities. They differ in how errors propagate. For tokens, the composition target is a distribution, permitting KL divergence on the simplex. The derivative targets lie off the simplex, as shown in Section 6.26.5, so we use cross-entropy with an unnormalized target.

With a softmax head producing the student $q = \mathrm{softmax}(z)$, the KL divergence from a target p differs from the cross-entropy by a constant, so

$$\frac{\partial}{\partial z_i} D_{\mathrm{KL}}(p \,\|\, q) = q_i - p_i,$$

a gradient whose size in coordinate i is the error in coordinate i. The squared error $\frac{1}{2}\sum_j (q_j - p_j)^2$ passes through the softmax Jacobian $\partial q_j / \partial z_i = q_j(\delta_{ij} - q_i)$ and becomes

$$\frac{\partial}{\partial z_i} \tfrac{1}{2} \sum_j (q_j - p_j)^2 = q_i \, (q_i - p_i) - q_i \sum_j q_j (q_j - p_j),$$

which has a factor q_i in every term. Now take the target and the student of Section 6.27.1,

$$p = (0.06,\ 0.12,\ 0.76,\ 0.06), \qquad q = (0.10,\ 0.15,\ 0.65,\ 0.10).$$

The KL gradient in the logits is $(0.04, 0.03, -0.11, 0.04)$.

The squared-error gradient is $(0.0099, 0.01335, -0.03315, 0.0099)$, between one quarter and one half as large in each coordinate. The difference increases as the student saturates. Set $q_1 = 10^{-6}$ and renormalize the remaining coordinates. The KL gradient on that coordinate is -0.0600, and the squared-error gradient is -4.62×10^{-8}, six orders of magnitude smaller. Squared error therefore gives almost no correction to a token assigned very low probability. KL divergence retains the full correction.

The Eulerian derivative also spans the entire length-by-vocabulary state. At $L = 128$ and $|\mathcal{V}| =$ 30,522, its tangent contains 3,906,816 coordinates. The corresponding full Jacobian would contain 1.526×10^{13} entries.

The composition target stays on the simplex, so its KL loss is well defined. The derivative targets lie off the simplex by the tangent argument in Section 6.26.5, and reweighting does not change their signs. Using distributional losses for those identities requires changing the targets or the loss.

6.28 Categorical Flow Maps

For Categorical Flow Maps [Roos et al., 2026], we retain the Lagrangian residual and train an upper bound with two terms that are each well defined on the simplex.

We use the continuous one-hot representation from Section 6.25 and apply softmax to the network output at each position. Denote the prediction by $\pi_{s,t}(x)$, the same two-time denoiser as $d_\theta(\cdot \mid x, s, t)$, with the position index suppressed. Softmax guarantees a simplex point for every state and time pair. Training constrains how these predictions relate across time pairs.

6.28.1 The Lagrangian identity in denoiser coordinates

The flow map and the two-time denoiser are two coordinates for the same prediction, tied by the convex parameterization of Section 6.26.1,

$$X_{s,t}(x) = \frac{1-t}{1-s}\, x + \frac{t-s}{1-s}\, \pi_{s,t}(x).$$

The Lagrangian identity is $\partial_t X_{s,t}(x) = b(X_{s,t}(x), t)$. For a linear interpolant, the velocity is the affine function $b(y,t) = (\pi_{t,t}(y) - y)/(1-t)$ of the diagonal denoiser. It points from the current state toward the posterior data mean, scaled by the remaining time. Substituting the denoiser forms for both map and velocity gives an identity among predictions.

Differentiate the parameterization in t, holding s and x fixed,

$$\partial_t X_{s,t}(x) = \frac{-x + \pi_{s,t}(x) + (t-s)\,\partial_t \pi_{s,t}(x)}{1-s}.$$

Substituting the velocity in denoiser form on the right of the Lagrangian identity and multiplying both sides by $(1-s)(1-t)$,

$$(1-t)\Big(-x + \pi_{s,t}(x) + (t-s)\,\partial_t \pi_{s,t}(x)\Big) = (1-s)\,\pi_{t,t}\big(X_{s,t}(x)\big) - (1-t)\,x - (t-s)\,\pi_{s,t}(x),$$

where the last two terms come from expanding $(1-s)X_{s,t}(x)$ with the parameterization. The terms in x cancel, the two terms in $\pi_{s,t}$ combine as $(1-t)+(t-s) = 1-s$, and dividing by $1-s$ produces

$$\boxed{\pi_{s,t}(x) = \pi_{t,t}\big(X_{s,t}(x)\big) - \frac{(t-s)(1-t)}{1-s}\,\partial_t \pi_{s,t}(x)}$$

"The two-time prediction $\pi_{s,t}(x)$ equals the diagonal prediction $\pi_{t,t}(X_{s,t}(x))$ at the transported state minus the end-time derivative $\partial_t \pi_{s,t}(x)$ multiplied by the coefficient $(t-s)(1-t)/(1-s)$."

- $\pi_{s,t}(x)$ is the network's two-time prediction at the state x, a point of the simplex at every position, produced by a softmax head.
- $\pi_{t,t}(y)$ is the same network read on the diagonal, which is the ordinary denoiser at the state y and time t.
- $\partial_t \pi_{s,t}(x)$ is the derivative of the prediction in its end time, a vector whose coordinates sum to zero and whose signs are unconstrained.
- $\gamma_{s,t} = (t-s)(1-t)/(1-s)$ is the coefficient in front of that derivative. It vanishes on the diagonal and at $t = 1$, and over the whole square it never exceeds $1/4$.

The right-hand side subtracts a derivative from a simplex point, so it can lie outside the simplex. Cross-entropy remains evaluable with a normalized student, but a negative target coordinate changes the sign of a loss term and can change the minimizer. KL divergence is undefined when any target coordinate is negative.

6.28.2 Teacher-gap and drift decomposition

We average the square of $r_{s,t}(x) = \partial_t X_{s,t}(x) - b(X_{s,t}(x), t)$ in the Lagrangian self-distillation loss. Applying the preceding algebra without assuming equality splits this residual into two terms,

$$(1-t)\,r_{s,t}(x) = \underbrace{\pi_{s,t}(x) - \pi_{t,t}\big(X_{s,t}(x)\big)}_{a_{s,t}(x)} + \underbrace{\tfrac{(1-t)(t-s)}{1-s}\,\partial_t \pi_{s,t}(x)}_{b_{s,t}(x)}$$

"The remaining time $1-t$ multiplied by the Lagrangian residual $r_{s,t}(x)$ equals the teacher gap $a_{s,t}(x)$ plus the drift $b_{s,t}(x)$. The gap compares predictions at the starting and transported states; the drift is the scaled end-time derivative of the prediction."

The gap $a_{s,t}$ is the difference of two simplex points. Its coordinates sum to zero and lie in $[-1, 1]$. The drift $b_{s,t}$ is the derivative scaled by $\gamma_{s,t}$; its coordinates also sum to zero. Splitting the residual allows us to measure each term with a separate loss.

For the gap, the teacher $\pi_{t,t}(X_{s,t}(x))$ and the student $\pi_{s,t}(x)$ are both distributions, so a divergence between them is available. For the drift, there is no distribution to compare against, and the natural measurement is its squared norm. We define

$$\mathcal{L}_{\mathrm{EC}} = \mathbb{E}\Big[D_{\mathrm{KL}}\Big(\pi_{t,t}(\hat{X}_{s,t}(I_s)) \,\Big\|\, \pi_{s,t}(I_s)\Big)\Big], \qquad \mathcal{L}_{\mathrm{TD}} = \mathbb{E}\Big[\big\|b_{s,t}(I_s)\big\|^2\Big],$$

where the first term measures the gap to the detached teacher and the second measures the drift. Both averages use interpolant states I_s and the time pairs used for sampling.

6.28.3 Residual bound

We want to minimize $\mathcal{L}_{\mathrm{CSD}} = \mathbb{E}\|a_{s,t}(I_s) + b_{s,t}(I_s)\|^2$, the Lagrangian residual in denoiser coordinates with the factor $1 - t$ absorbed. The parallelogram bound gives $\|a + b\|^2 \le 2\|a\|^2 + 2\|b\|^2$. For distributions p and q, Pinsker's inequality gives $\|p - q\|_1^2 \le 2D_{\mathrm{KL}}(p \,\|\, q)$. We can apply it to the gap because both predictions are distributions. Combining the inequalities with $\|a\|_2 \le \|a\|_1$ gives

$$\|a_{s,t}\|^2 \;\le\; \|a_{s,t}\|_1^2 \;\le\; 2\, D_{\mathrm{KL}}\big(\pi_{t,t}(X_{s,t}(x)) \,\|\, \pi_{s,t}(x)\big)\,,$$

so that $2\|a\|^2 \le 4\,\mathrm{KL}$, and taking expectations,

$$\mathcal{L}_{\mathrm{CSD}} \;\le\; 4\,\mathcal{L}_{\mathrm{EC}} + 2\,\mathcal{L}_{\mathrm{TD}}$$

"The expected squared Lagrangian residual $\mathcal{L}_{\mathrm{CSD}}$ is bounded by four times the expected teacher-gap divergence $\mathcal{L}_{\mathrm{EC}}$ plus twice the expected squared drift norm $\mathcal{L}_{\mathrm{TD}}$, averaged over interpolant states and time pairs."

The gap term is written as a cross-entropy against the detached teacher,

$$H(q,\, \pi_{s,t}(x)) = -\sum_k q_k \log \pi_{s,t}(x)_k = D_{\mathrm{KL}}(q \,\|\, \pi_{s,t}(x)) + H(q), \qquad q = \mathrm{sg}\, \pi_{t,t}(X_{s,t}(x)),$$

which differs from KL divergence by the teacher entropy $H(q)$. Because the teacher is detached, that entropy contributes no gradient, so both losses update the student identically. For example, a divergence of 0.0391 and teacher entropy of 0.8043 nats give cross-entropy 0.8434. The values differ by a constant but have the same derivative in θ. Computing the drift requires one forward-mode end-time derivative, as the Lagrangian residual did. Both terms avoid logarithms of negative quantities.

6.28.4 The bound at the true flow map

We can evaluate the upper bound at the exact map to compare its minimizer with the residual's minimizer. At the true map, $r_{s,t}(x) = 0$ for every state and time pair, so

$$r_{s,t} = 0 \implies a_{s,t} = -\,b_{s,t} \implies \|a_{s,t}\| = \|b_{s,t}\| > 0$$

"When the Lagrangian residual $r_{s,t}$ is zero, the teacher gap $a_{s,t}$ equals minus the drift $b_{s,t}$. Their norms are equal and nonzero wherever the prediction varies with end time."

Proposition 6.6 (The bound is not minimized at the flow map)**.** *Let $\pi_{s,t}$ be the two-time denoiser of the exact flow map, and suppose that at some state x and some pair $s < t < 1$ the prediction is not constant in its end time, so that $\partial_t \pi_{s,t}(x) \neq 0$. Then $\mathcal{L}_{\mathrm{CSD}}$ is zero at $\pi_{s,t}$ while $4\mathcal{L}_{\mathrm{EC}} + 2\mathcal{L}_{\mathrm{TD}}$ is strictly positive there, and $\pi_{s,t}$ is not a stationary point of the bound.*

Proof. The residual vanishes at the exact map, so $a_{s,t}(x) = -b_{s,t}(x)$ by the split. Since $\gamma_{s,t} > 0$ for $s < t < 1$ and $\partial_t \pi_{s,t}(x) \neq 0$, the drift $b_{s,t}(x)$ is non-zero, so $\mathcal{L}_{\mathrm{TD}} > 0$. The gap has the same non-zero norm, so the two predictions it compares differ and their divergence is strictly positive, giving $\mathcal{L}_{\mathrm{EC}} > 0$. A sum of two strictly positive terms is strictly positive, and a point where a non-negative quantity takes a strictly positive value with no constraint binding is not a minimizer of it. □

At the exact solution, both bound terms are strictly positive, although their unsquared residual components cancel. The true map minimizes the residual without minimizing the bound. Gradient descent on $4\mathcal{L}_{\mathrm{EC}} + 2\mathcal{L}_{\mathrm{TD}}$ reduces both $\|b_{s,t}\|$ and $\|a_{s,t}\|$. Reducing the drift makes the prediction constant in end time; reducing the gap makes it agree with the diagonal prediction at the transported state. Satisfying both requires a prediction constant in t. With the diagonal anchor as well, this is the stay-still solution.

6.28.5 A worked cancellation

Take one position with three token probabilities $(0.5, 0.3, 0.2)$, represented by the standard basis vectors in $\mathbb{R}^3$. Use source $\mathcal{N}(0, I_3)$ and linear interpolant $I_t = (1-t)I_0 + tI_1$. We can calculate every part of the solution explicitly except one integral. The token posterior is

$$\pi_{t,t}(x)_k \;=\; \frac{p_k \exp\!\big(-\|x - t\,\mathbf{e}_k\|^2/2(1-t)^2\big)}{\sum_j p_j \exp\!\big(-\|x - t\,\mathbf{e}_j\|^2/2(1-t)^2\big)},$$

with velocity $b(x,t) = (\pi_{t,t}(x) - x)/(1-t)$. We integrate $\dot{x} = b(x,t)$ to twelve digits to obtain the exact flow map and recover its two-time denoiser from $\pi_{s,t}(x) = \big[(1-s)X_{s,t}(x) - (1-t)x\big]/(t-s)$.

Fix $s = 0.2$ and $t = 0.6$, so that the coefficient is

$$\gamma_{s,t} = \frac{(0.6 - 0.2)(1 - 0.6)}{1 - 0.2} = \frac{0.4 \times 0.4}{0.8} = 0.2.$$

The data token is the first and the source draw is $I_0 = (-0.5739, -0.8344, -0.0584)$, so the state at time s is

$$x = 0.8\, I_0 + 0.2\, \mathbf{e}_1 = (-0.2591,\ -0.6675,\ -0.0467).$$

If we integrate the flow from 0.2 to 0.6 we reach $X_{s,t}(x) = (0.1611, -0.2420, 0.0943)$, and the two predictions at the two ends of that leg are

$$\pi_{s,t}(x) = (0.5813,\ 0.1835,\ 0.2352), \qquad \pi_{t,t}\big(X_{s,t}(x)\big) = (0.6927,\ 0.0917,\ 0.2157).$$

Both predictions have nonnegative coordinates summing to one. Their difference and the scaled derivative are

$$a = (-0.111,\ 0.092,\ 0.020), \qquad b = (0.111,\ -0.092,\ -0.020),$$

with the derivative itself equal to $\partial_t \pi_{s,t}(x) = (0.5570, -0.4592, -0.0979)$ and $b = 0.2 \times \partial_t \pi_{s,t}(x)$. The teacher gap is the exact negative of the drift, to every digit we carry, which is the cancellation derived in Section 6.28.4.

Next, we can evaluate the three losses at this state. The residual is

$$\|a + b\|^2 = 3 \times 10^{-27},$$

which is zero to the integrator's accuracy. Evaluating the two bound terms gives

$$4\, D_{\mathrm{KL}}(\pi_{t,t}(X_{s,t}(x)) \,\|\, \pi_{s,t}(x)) = 4 \times 0.0391 = 0.156, \qquad 2\,\|b\|^2 = 2 \times 0.02123 = 0.042,$$

for a total of 0.199. The zero residual satisfies this bound. However, descending on the bound moves the model away from that zero-residual solution. The same behavior holds after averaging over three thousand interpolant states at the same time pair. The residual remains 10^{-25}, the gap term averages 0.393, and the drift term averages 0.095. Together they give a bound of 0.489 on a quantity that is zero.

For this small gap, the divergence is approximately one quarter of the squared ℓ_1 distance, half the value allowed by Pinsker's inequality. The parallelogram estimate is also loose because a and b point in opposite directions. Their sum has norm zero, while $2\|a\|^2 + 2\|b\|^2 = 4\|a\|^2$. Thus the positive bound differs from the zero residual at the exact map.

In the published ablation, increasing the weight on the exact Lagrangian residual gives 0.864 valid and unique molecules in one step. Training with the bounded loss gives 0.819 [Roos et al., 2026].

This loss bound preserves the Lagrangian identity and makes every logarithm well defined, even when the derivative target leaves the simplex. We can also change coordinates to construct a normalized target directly.

6.29 Discrete Flow Maps

Adding a zero-sum vector to a simplex point preserves its total mass but can make a coordinate negative. In Discrete Flow Maps [Potaptchik et al., 2026b], we express the identity in logit coordinates and apply a softmax after the derivative adjustment. The resulting probabilities remain nonnegative.

Write the mean denoiser as $\psi_{s,t}(x) = \mathrm{softmax}(z_{s,t}(x))$, with logits $z_{s,t}(x) \in \mathbb{R}^{|\mathcal{V}|}$ at each position. This is the function denoted by π in Categorical Flow Maps. As a weighted average of token posteriors, it remains on the simplex.

6.29.1 Differentiating a softmax

The derivative of the prediction in the end time follows from the derivative of the softmax. Let's write $\psi_k = e^{z_k} / \sum_j e^{z_j}$ and differentiate in one logit,

$$\frac{\partial \psi_k}{\partial z_m} = \frac{\mathbf{1}[k = m]\, e^{z_k}}{\sum_j e^{z_j}} - \frac{e^{z_k} e^{z_m}}{\left(\sum_j e^{z_j}\right)^2} = \psi_k\big(\mathbf{1}[k = m] - \psi_m\big),$$

so the Jacobian of the softmax is $\mathrm{diag}(\psi) - \psi\psi^\top$. Contracting it with the logit derivative $\partial_t z_{s,t}$ through the chain rule gives

$$\partial_t \psi_{s,t} = \psi_{s,t} \odot \big(\partial_t z_{s,t} - \langle \psi_{s,t},\, \partial_t z_{s,t} \rangle \mathbf{1}\big)$$

"The end-time derivative $\partial_t \psi_{s,t}$ equals the probability prediction $\psi_{s,t}$ multiplied coordinatewise by the centered logit derivative. We obtain that centered derivative by subtracting its probability-weighted mean $\langle \psi_{s,t}, \partial_t z_{s,t} \rangle$ from each coordinate of $\partial_t z_{s,t}$, using the all-ones vector $\mathbf{1}$."

- $z_{s,t}(x)$ are the logits the network produces for the pair of times (s,t) at the state x, one vector of length $|\mathcal{V}|$ per position.
- $\odot$ is the coordinatewise product, and $\mathbf{1}$ is the vector of ones in $\mathbb{R}^{|\mathcal{V}|}$.
- $c_{s,t} = \partial_t z_{s,t} - \langle \psi_{s,t}, \partial_t z_{s,t} \rangle \mathbf{1}$ is the centered logit derivative, the quantity the whole identity is built on.

Because of the centring, $\langle \psi_{s,t}, c_{s,t} \rangle = 0$, so the coordinates of $\partial_t \psi_{s,t} = \psi_{s,t} \odot c_{s,t}$ sum to zero and the step stays tangent to the simplex. The centring also makes the identity insensitive to the shift invariance of the softmax, since adding a constant to every logit changes neither ψ nor c.

6.29.2 The identity as a multiplicative relation

We now substitute this derivative into the Lagrangian identity of Section 6.28.1. Moving the derivative term to the other side and factoring out the prediction,

$$\psi_{t,t}(X_{s,t}(x)) = \psi_{s,t}(x) \odot (\mathbf{1} + \gamma_{s,t}\, c_{s,t}(x)), \qquad \gamma_{s,t} = \tfrac{(t-s)(1-t)}{1-s}, \quad c_{s,t} = \partial_t z_{s,t} - \langle \psi_{s,t}, \partial_t z_{s,t} \rangle \mathbf{1}$$

"The diagonal prediction $\psi_{t,t}(X_{s,t}(x))$ at the destination equals the starting prediction $\psi_{s,t}(x)$ multiplied coordinatewise by $\mathbf{1} + \gamma_{s,t} c_{s,t}(x)$. Here $c_{s,t}$ is the centered logit derivative and $\gamma_{s,t} = (t-s)(1-t)/(1-s)$ is its time-dependent scale."

At the true map, $\mathbf{1} + \gamma_{s,t} c_{s,t}$ is the coordinatewise ratio of two strictly positive predictions. Every factor is therefore positive.

6.29.3 The repaired teacher

Solving the multiplicative identity for the student gives a target that is a distribution whenever that factor is positive. Dividing both sides coordinatewise and taking logarithms, with $\psi_{t,t} = \mathrm{softmax}(z_{t,t})$,

$$T^{\mathrm{LSD}}_{s,t}(\psi)(x) = \mathrm{Softmax}\Big(z_{t,t}(X_{s,t}(x)) - \log\left(\mathbf{1} + \gamma_{s,t}\, c_{s,t}(x)\right) \Big)$$

"The Lagrangian teacher $T^{\mathrm{LSD}}_{s,t}(\psi)(x)$ applies softmax to the diagonal logits $z_{t,t}(X_{s,t}(x))$ minus the coordinatewise logarithm of $\mathbf{1} + \gamma_{s,t} c_{s,t}(x)$. This factor uses the centered logit derivative $c_{s,t}$ scaled by $\gamma_{s,t} = (t-s)(1-t)/(1-s)$."

The softmax makes the target a distribution, so we can enforce the identity with KL divergence directly. The logarithm still requires a positive multiplicative factor. This holds near the solution; during training, we clip the factor away from zero. For the Eulerian identity, we apply the same algebra using the total start-time derivative $d/ds = \partial_s + b(x,s) \cdot \nabla_x$. Substitution into $(d/ds) X_{s,t}(x) = 0$ gives

$$\psi_{s,s}(x) = \psi_{s,t}(x) - \gamma^{\mathrm{E}}_{s,t} \frac{d}{ds} \psi_{s,t}(x), \qquad \gamma^{\mathrm{E}}_{s,t} = \frac{(1-s)(t-s)}{1-t},$$

"The input's diagonal prediction $\psi_{s,s}(x)$ equals the two-time prediction $\psi_{s,t}(x)$ minus its total start-time derivative along the trajectory, multiplied by the Eulerian coefficient $\gamma^{\mathrm{E}}_{s,t} = (1-s)(t-s)/(1-t)$."

We evaluate the Eulerian teacher at the input, so it requires no flow-map evaluation. Its coefficient exceeds the Lagrangian coefficient and diverges as $t \to 1$. At $s = 0.2$, $t = 0.6$, the coefficients are 0.8 and 0.2, respectively, a fourfold difference. Expressing the derivative as a centered multiplicative factor and dividing gives

$$c^{\mathrm{E}}_{s,t} = \frac{dz_{s,t}}{ds} - \Big\langle \psi_{s,t},\, \frac{dz_{s,t}}{ds} \Big\rangle \mathbf{1}, \qquad T^{\mathrm{ESD}}_{s,t}(\psi)(x) = \mathrm{Softmax}\Big(z_{s,s}(x) - \log\big(\mathbf{1} - \gamma^{\mathrm{E}}_{s,t}\, c^{\mathrm{E}}_{s,t}(x)\big)\Big),$$

which is again a softmax of network predictions. In the exact three-token example at the same state and time pair, the factor is $\mathbf{1} - \gamma^{\mathrm{E}}_{s,t} c^{\mathrm{E}}_{s,t} = (0.8798, 1.4717, 0.9292)$. The diagonal prediction is $\psi_{s,s}(x) = (0.5114, 0.2701, 0.2186)$. Dividing coordinatewise gives $(0.5813, 0.1835, 0.2352)$, matching $\psi_{s,t}(x)$ to four decimals without normalization. Computing this teacher requires the spatial gradient inside d/ds, whose forward-mode tangent spans the full length-by-vocabulary state.

The training loss is then the divergence from the detached teacher to the student, summed over positions,

$$\mathcal{L}^{\mathrm{LSD}} = \mathbb{E}\Big[\textstyle\sum_l D_{\mathrm{KL}}\Big(\mathrm{sg}(T^{\mathrm{LSD}}_{s,t}(\psi)(I_s)^l) \,\Big\|\, \psi^l_{s,t}(I_s) \Big) \Big] \;+\; \mathbb{E}\Big[\textstyle\sum_l D_{\mathrm{KL}}\Big(\mathbf{e}_{x_1^l} \,\Big\|\, \psi^l_{t,t}(I_t) \Big) \Big],$$

with the second term the diagonal anchor, which is the cross-entropy of Chapter 5 against the one-hot data token and needs no teacher network.

6.29.4 Teacher correction on three tokens

Let's take a single position over three tokens, at $s = 0.2$ and $t = 0.6$, so that $\gamma_{s,t} = 0.2$ as before. Suppose the student's prediction and logit derivative at the state x are

$$\psi_{s,t}(x) = (0.2,\ 0.5,\ 0.3), \qquad \partial_t z_{s,t}(x) = (9.60,\ -2.28,\ -2.60).$$

The weighted average of the logit derivative under the prediction is

$$0.2(9.60) + 0.5(-2.28) + 0.3(-2.60) = 1.92 - 1.14 - 0.78 = 0,$$

so the centered derivative is the derivative itself, $c_{s,t}(x) = (9.60, -2.28, -2.60)$, and the derivative of the prediction is

$$\partial_t \psi_{s,t}(x) = \psi_{s,t}(x) \odot c_{s,t}(x) = (1.92,\ -1.14,\ -0.78),$$

whose coordinates sum to zero, as the centring guarantees.

Suppose the teacher prediction at the transported state is $\psi_{t,t}(X_{s,t}(x)) = (0.10, 0.60, 0.30)$, as can occur for an imperfect model during training. The additive target from the identity is $\psi_{t,t}(X_{s,t}(x)) - \gamma_{s,t}\, \partial_t \psi_{s,t}(x)$, giving

$$(0.10,\ 0.60,\ 0.30) - 0.2\,(1.92,\ -1.14,\ -0.78) = (-0.284,\ 0.828,\ 0.456).$$

The additive target sums to one but has a negative first coordinate, so KL divergence is undefined. Its cross-entropy contains $-(-0.284)\log \psi_1$, reversing that coordinate's sign and favoring mass on the first token where the target requires a decrease.

The repaired target starts from the same two quantities. The multiplicative factor is

$$\mathbf{1} + \gamma_{s,t}\, c_{s,t}(x) = (1 + 1.920,\ 1 - 0.456,\ 1 - 0.520) = (2.920,\ 0.544,\ 0.480),$$

positive in every coordinate. Dividing the teacher by it coordinatewise,

$$\left(\frac{0.10}{2.920},\ \frac{0.60}{0.544},\ \frac{0.30}{0.480}\right) = (0.0342,\ 1.1029,\ 0.6250),$$

with total mass 1.7622, and the softmax normalization divides by that total,

$$T^{\mathrm{LSD}}_{s,t}(\psi)(x) = (0.0194,\ 0.6259,\ 0.3547).$$

Every coordinate is positive and the total is one, so KL divergence is defined. Computing this target in logit coordinates subtracts $\log(\mathbf{1} + \gamma_{s,t} c_{s,t}) = (1.0716, -0.6088, -0.7340)$ from the teacher logits before applying softmax.

Both targets reduce the first token's mass, with different correction sizes. The additive target crosses the simplex boundary; the multiplicative target stays inside it. At the exact solution, the factor equals the coordinatewise ratio of the two predictions. Dividing then recovers a vector that already sums to one, so normalization leaves it unchanged.

With repaired targets, all three identities can be trained on text. The repaired Eulerian identity reaches one-step perplexity 68.11 at entropy 3.79 nats [Potaptchik et al., 2026b]. Reporting both matters because a model that repeats one phrase can achieve low perplexity with entropy far below the data.

6.30 Discrete flow-map comparison

All three discrete methods use a continuous state in $\mathbb{R}^{L\times|\mathcal{V}|}$, predict a two-time denoiser on the simplex at each position, and decode by argmax. They differ in the trained identity and the associated treatment of its target.

Table 6.4: The three discrete flow maps, by the identity each one trains. The two-time denoiser is the same function in all three, named δ, π and ψ by the three methods. The last column names the obstruction each one has to handle: the composition target is already a distribution, while the two derivative targets are not.

Method	Identity trained	Residual	Target on the simplex
Flow Map Language Models	composition	KL to a convex target	yes, by convexity
Categorical Flow Maps	Lagrangian	KL plus squared drift	bounded, not enforced
Discrete Flow Maps	all three	KL to a softmax target	yes, by the logit repair

For Flow Map Language Models, we train the composition identity with a target that is a convex combination of two simplex points. The decoding clock concentrates training and sampling times where argmax changes. For Categorical Flow Maps, we train the Lagrangian identity through two well-defined terms that upper-bound its residual. For Discrete Flow Maps, we express the derivative targets in logit coordinates, so each target is a softmax distribution and each residual can use a divergence.

All three targets describe one function, and on the exact solution of the three-token example they agree to every digit. We can keep the state $x = (-0.2591, -0.6675, -0.0467)$ of Section 6.28.5 and the pair $s = 0.2$, $t = 0.6$, where the exact two-time denoiser is $\pi_{s,t}(x) = (0.5813, 0.1835, 0.2352)$.

The composition target splits the interval at $u = 0.4$, where the convexity weight is

$$\gamma = \frac{(1-t)(u-s)}{(1-u)(t-s)} = \frac{0.4 \times 0.2}{0.6 \times 0.4} = \frac{1}{3}.$$

The first leg moves x to $X_{s,u}(x) = (-0.0627, -0.4399, 0.0225)$, and the two predictions are $\pi_{s,u}(x) = (0.5265, 0.2432, 0.2303)$ over the first leg and $\pi_{u,t}(X_{s,u}(x)) = (0.6086, 0.1537, 0.2377)$ over the second. Combining them,

$$\tfrac{1}{3}(0.5265,\ 0.2432,\ 0.2303) + \tfrac{2}{3}(0.6086,\ 0.1537,\ 0.2377) = (0.5813,\ 0.1835,\ 0.2352),$$

which is $\pi_{s,t}(x)$. Every quantity in that line is a point of the simplex, including the two inputs and the output, which is the convexity the first method relies on.

The Lagrangian target subtracts the scaled derivative from the prediction at the transported state,

$$(0.6927,\ 0.0917,\ 0.2157) - 0.2\,(0.5570,\ -0.4592,\ -0.0979) = (0.5813,\ 0.1835,\ 0.2352),$$

the same vector. It lands inside the simplex here because the derivative is small at this state and $\gamma_{s,t} = 0.2$ is modest. The Eulerian target, built by dividing the diagonal prediction $\psi_{s,s}(x) = (0.5114, 0.2701, 0.2186)$ by the factor $(0.8798, 1.4717, 0.9292)$, returns the same vector again.

All three identities agree at the exact solution. During training, the composition target stays on the simplex because it is a convex combination. An additive derivative target can leave the simplex when its scaled derivative exceeds one prediction coordinate.

The observed ranking differs between text and images. For tokens, composition gives a valid distributional target directly; the derivative identities require a target or loss adjustment.

6.31 Flow maps of a stochastic differential equation

For generation with a stochastic differential equation, each trajectory also depends on a Brownian path, as in Chapter 3. We can define a solution map for these trajectories. Its cocycle holds when we use the same noise path across the composed intervals.

6.31.1 The solution map

Consider a stochastic differential equation with drift f and diffusion coefficient g. Its solution map takes a starting state and a driving path and returns the state at the end of the interval,

$$dX_t = f(t, X_t)\,dt + g(t)\,dW_t, \qquad \Psi_{s,t} : (X_s,\, W_{[s,t]}) \mapsto X_t$$

"For drift f and diffusion coefficient g, the stochastic solution map $\Psi_{s,t}$ takes the starting state X_s and the complete Brownian path $W_{[s,t]}$ on the interval. Its output is the state X_t at end time t."

- $W_{[s,t]}$ is the entire Brownian path restricted to the interval, not a single increment and not a single number.
- $f(t, x)$ is the drift and $g(t)$ the diffusion coefficient, in the notation of Chapter 3.
- $\Psi_{s,t}$ is the Itô map of the equation, the pathwise solution operator.

This solution map depends on both the starting state and the driving path. The path is an infinite-dimensional input, in addition to the state used by a deterministic flow map.

With the path held fixed, the cocycle holds unchanged, with each leg driven by the restriction of the same path to its own interval,

$$\Psi_{s,t}(X_s,\, W_{[s,t]}) = \Psi_{u,t}\Big(\Psi_{s,u}(X_s,\, W_{[s,u]}),\, W_{[u,t]}\Big)$$

"The full-interval solution $\Psi_{s,t}$ equals the first-leg map $\Psi_{s,u}$ followed by the second-leg map $\Psi_{u,t}$. Both legs start from the appropriate intermediate state and use restrictions of the same Brownian path W to their own intervals."

For a fixed noise realization, the trajectory is a single curve through the state space, determined by an ordinary differential equation with a rough driving term. Following two successive segments gives the same endpoint as following the whole curve. These properties also determine the solution map.

Proposition 6.7 (The pathwise cocycle characterizes the Itô map)**.** *Let $\widehat{\Psi}_{s,t}$ be measurable in its two arguments, satisfy the cocycle above for every triple $s \le u \le t$ with the path restricted to each leg, and match the Euler-Maruyama step of the equation on the diagonal to order $o(h)$. Then $\widehat{\Psi}_{s,t}$ is the Itô map of that equation, for every realization of the driving path.*

Proof sketch. Fix a partition $s = u_0 < u_1 < \cdots < u_N = t$ of mesh h. Applying the cocycle $N-1$ times writes $\widehat{\Psi}_{s,t}$ as the composition of the N short maps over the partition cells, each driven by the restriction of the one path to its own cell. On each cell the diagonal condition replaces the short map by one Euler-Maruyama step plus a term of size $o(h)$. The composition of those steps is the Euler-Maruyama scheme for the equation on the given path, whose limit as $h \to 0$ is the Itô map, and the accumulated remainder is $N \cdot o(h) \to 0$ [McCallum et al., 2026]. □

The cocycle and diagonal condition determine the pathwise solution. Matching transition kernels on every interval determines only the endpoint distributions, leaving the trajectories unspecified.

6.31.2 Limitations of weak maps

Suppose a map reproduces the transition kernel, the law of X_t given $X_s = x$. It takes the state and a fresh noise draw and produces the correct endpoint distribution on each interval. We call this a weak map, $\bar{\Psi}_{s,t}$. For a full step, we draw one noise variable; for two half steps, we draw one per leg. These three independent draws do not describe a common driving path (Figure 6.5).

Let's take the linear equation

$$dX = X\,dt + \sigma\,dW \quad \text{on } [0,1],$$

whose transition kernel is Gaussian in closed form. Starting at x at time s, the state at time t is $e^{t-s}x$ plus a centered Gaussian of variance $\sigma^2 \int_s^t e^{2(t-u)}\,du$, so the weak map is

$$\bar{\Psi}_{s,t}(x,\varepsilon) = e^{t-s}x + \sqrt{K_{t-s}}\,\varepsilon, \qquad K_d = \tfrac{1}{2}\sigma^2(e^{2d}-1)$$

"The weak map $\bar{\Psi}_{s,t}$ returns the starting state x multiplied by e^{t-s}, plus a standard normal draw ε scaled by $\sqrt{K_{t-s}}$. The interval variance K_d is $\sigma^2(e^{2d}-1)/2$ for duration d and noise level σ."

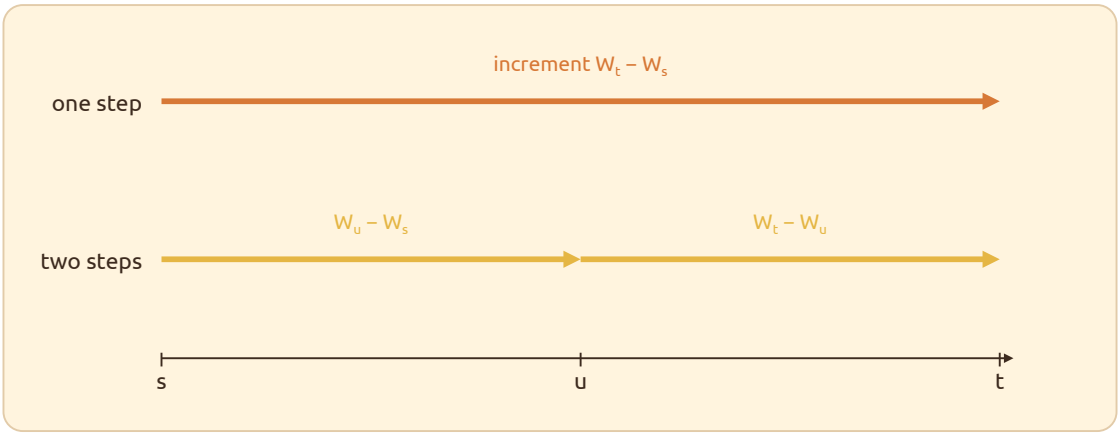


Figure 6.5: The noise a weak map draws on one interval against the noise it draws on two. The upper arrow is a single step from s to t, driven by one increment $W_t - W_s$. The lower pair is the same interval crossed in two legs through the midpoint u, driven by the increments $W_u - W_s$ and $W_t - W_u$. For the two crossings to land on the same state, the two lower increments must add to the upper one. A map that samples a fresh increment for each leg has no mechanism that enforces that sum.

This map has the right kernel on every interval, and its variances compose correctly. If we carry the variance of the first leg through the second and add the variance of the second,

$$e^{2(t-u)}K_{u-s} + K_{t-u} = \tfrac{\sigma^2}{2}\left(e^{2(t-s)} - e^{2(t-u)}\right) + \tfrac{\sigma^2}{2}\left(e^{2(t-u)} - 1\right) = \tfrac{\sigma^2}{2}\left(e^{2(t-s)} - 1\right) = K_{t-s}.$$

The endpoint distributions therefore agree. To compare individual endpoints, use three independent draws $\varepsilon_1, \varepsilon_2, \varepsilon_3$. The full step and two half steps differ by

$$\bar{\Psi}_{s,t}(x, \varepsilon_3) - \bar{\Psi}_{u,t}(\bar{\Psi}_{s,u}(x, \varepsilon_1), \varepsilon_2) = \sqrt{K_{t-s}}\,\varepsilon_3 - e^{t-u}\sqrt{K_{u-s}}\,\varepsilon_1 - \sqrt{K_{t-u}}\,\varepsilon_2,$$

since the deterministic parts $e^{t-s}x$ agree. The three draws are independent, so the variances add and the composition in the previous display turns the sum into twice the full-interval variance,

$$\mathbb{E}\left\|\bar{\Psi}_{s,t}(x, \varepsilon_3) - \bar{\Psi}_{u,t}(\bar{\Psi}_{s,u}(x, \varepsilon_1), \varepsilon_2)\right\|^2 = 2\,K_{t-s}$$

"Averaging over independent Gaussian draws, the squared difference between a full-step weak-map output and a two-half-step output equals twice the full-interval noise variance K_{t-s}."

With $\sigma = 1$ on $[s, t] = [0, 1]$ this is

$$2K_1 = e^2 - 1 = 6.389$$

and a Monte Carlo average over 400,000 draws returns 6.39, with a standard deviation of about 0.01 across repetitions of that average.

Take $x = 0.3$, $s = 0$, $u = 0.5$, $t = 1$, so that $\sqrt{K_1} = 1.7873$ and $\sqrt{K_{0.5}} = 0.9269$. One full step with $\varepsilon_3 = 0.8$ lands at

$$e\,(0.3) + 1.7873\,(0.8) = 0.8155 + 1.4299 = 2.2453.$$

Two half steps with $\varepsilon_1 = 0.5$ and $\varepsilon_2 = -1.2$ land at

$$e^{0.5}(0.4946 + 0.9269 \times 0.5) + 0.9269\,(-1.2) = 0.8155 + 0.7640 - 1.1123 = 0.4673,$$

a gap of 1.778, whose square is 3.161. If we average that square over the three draws we get $K_1 + e\,K_{0.5} + K_{0.5} = 3.1945 + 2.3354 + 0.8591 = 6.389$, which is the number above.

The gap vanishes only when the third draw matches the first two. Here, that requires $\varepsilon_3 = (1.5282 \times 0.5 - 0.9269 \times 1.2)/1.7873 = -0.195$. The weak map's inputs do not specify this coupling. Its composition residual therefore has a floor of $2K_{t-s}$, regardless of model capacity or training accuracy.

6.31.3 The path as an input

To give a finite network information about an infinite-dimensional path, we project the noise onto a basis and pass a finite set of coefficients. Strong Stochastic Flow Maps uses shifted Legendre polynomials on the interval [McCallum et al., 2026],

$$I_{s,t}^{(n)} = \int_s^t \widetilde{P}_n\left(\frac{u-s}{t-s}\right) dW_u, \qquad I_{s,t}^{(0)} = W_t - W_s$$

"The Brownian-path coefficient $I_{s,t}^{(n)}$ integrates the shifted Legendre polynomial $\widetilde{P}_n$, evaluated at normalized time $(u-s)/(t-s)$, against the Brownian increment dW_u. For order $n = 0$, it equals the interval increment $W_t - W_s$."

- $\widetilde{P}_n(r) = P_n(2r-1)$ is the n-th Legendre polynomial shifted to $[0,1]$, with $\widetilde{P}_0 = 1$, $\widetilde{P}_1(r) = 2r-1$ and $\widetilde{P}_2(r) = 6r^2 - 6r + 1$.
- $I_{s,t}^{(n)}$ is a vector of the same dimension as the state, one coefficient per coordinate of the driving path.

In practice, we use three coefficients: the increment and two terms describing its timing within the interval. Paths with the same increment can differ in whether their displacement occurred early or late. Integration by parts gives the first coefficient on $[0,1]$,

$$I_{0,1}^{(1)} = \int_0^1 (2u-1)\, dW_u = W_1 - 2\int_0^1 W_u\, du,$$

so with the increment held at $I_{0,1}^{(0)} = W_1 = 0$, the first coefficient is minus twice the time average of the path. A path with $I_{0,1}^{(1)} = +0.5$ spent the interval at an average height of -0.25 and returned to zero, and a path with $I_{0,1}^{(1)} = -0.5$ spent it at $+0.25$. Both have the same increment, and a map whose only noise argument is that increment assigns them the same endpoint.

The distribution of these coefficients is available in closed form. By the Itô isometry the variance is the integral of the squared polynomial, $\int_s^t \widetilde{P}_n((u-s)/(t-s))^2 du = (t-s)\int_0^1 \widetilde{P}_n(r)^2 dr$, and the normalization of the Legendre family gives $\int_0^1 \widetilde{P}_n(r)^2 dr = 1/(2n+1)$. The coefficients are also jointly Gaussian with vanishing covariances, because distinct Legendre polynomials are orthogonal on the interval. Together,

$$I_{s,t}^{(n)} \sim \mathcal{N}\left(0,\ \frac{t-s}{2n+1}\, I\right), \qquad n = 0, 1, 2, \ldots$$

"Each path coefficient $I^{(n)}_{s,t}$ is Gaussian with mean zero and covariance equal to the interval length $t-s$ divided by $2n+1$, multiplied by the identity matrix. Coefficients of different polynomial orders n are independent."

We can sample these Gaussian coefficients directly during training without simulating a path. On the unit interval, their first three standard deviations are 1, $1/\sqrt{3} = 0.5774$, and $1/\sqrt{5} = 0.4472$. Fine-grid simulation gives variances 1, 0.333, and 0.200, with covariances below 0.005.

Keeping the first three coefficients preserves the increment exactly. Every higher polynomial integrates to zero over the interval, so truncation discards only information about the path inside the interval.

6.31.4 Chen relations

The cocycle compares one long map against two short ones, so we have to build the coefficients of the long interval from the coefficients of the two halves. If we write a Legendre polynomial on $[s,t]$ in the dilated basis of each half, we get a fixed linear rule,

$$I^{(n)}_{s,t} = \sum_{m=0}^{n} c_{n,m}\Big(I^{(m)}_{s,u} + (-1)^{n+m}\, I^{(m)}_{u,t}\Big), \qquad u = \tfrac{s+t}{2}$$

"The full-interval coefficient $I^{(n)}_{s,t}$ sums fixed multiples of the half-interval coefficients $I^{(m)}_{s,u}$ and $I^{(m)}_{u,t}$. The basis-expansion constants $c_{n,m}$ set their magnitudes, with signs determined by the parity of $n+m$."

These are the Chen relations, and the constants come from expanding a shifted Legendre polynomial of the long interval in the shifted Legendre basis of a half. For $n=0$ the polynomial is constant and the integrals add,

$$I^{(0)}_{s,t} = I^{(0)}_{s,u} + I^{(0)}_{u,t},$$

which is the statement that the increments add. For $n=1$, write $r_1 = (v-s)/(u-s)$ on the first half, so that $\widetilde{P}_1$ of the long interval becomes $r_1 - 1 = \frac{1}{2}\widetilde{P}_1(r_1) - \frac{1}{2}$, and $r_2 = (v-u)/(t-u)$ on the second, where it becomes $r_2 = \frac{1}{2}\widetilde{P}_1(r_2) + \frac{1}{2}$. Integrating each half against its own expansion,

$$I^{(1)}_{s,t} = -\tfrac{1}{2}(I^{(0)}_{s,u} - I^{(0)}_{u,t}) + \tfrac{1}{2}(I^{(1)}_{s,u} + I^{(1)}_{u,t}).$$

"The coefficient $I^{(1)}_{s,t}$ of order one equals minus one half the difference of the half-interval increments $I^{(0)}_{s,u}$ and $I^{(0)}_{u,t}$, plus one half the sum of their order-one coefficients $I^{(1)}_{s,u}$ and $I^{(1)}_{u,t}$."

The minus sign follows from the odd symmetry of $\widetilde{P}_1$ about the midpoint. The expansion for $n=2$ gives $I^{(2)}_{s,t} = \frac{1}{4}(I^{(2)}_{s,u} + I^{(2)}_{u,t}) - \frac{3}{4}(I^{(1)}_{s,u} - I^{(1)}_{u,t})$. The zeroth coefficient is absent because $c_{2,0} = 0$.

We can check the variance of each relation. For $n=1$ on an interval of length h, each half-interval increment has variance $h/2$. Each first coefficient has variance $h/6$. Adding their weighted variances gives

$$\tfrac{1}{4}\Big(\tfrac{h}{2}\Big) + \tfrac{1}{4}\Big(\tfrac{h}{2}\Big) + \tfrac{1}{4}\Big(\tfrac{h}{6}\Big) + \tfrac{1}{4}\Big(\tfrac{h}{6}\Big) = \tfrac{h}{4} + \tfrac{h}{12} = \tfrac{h}{3},$$

which is $h/(2n+1)$ at $n=1$. The same accounting at $n=2$ gives $\frac{1}{16}\cdot\frac{h}{10}\cdot 2 + \frac{9}{16}\cdot\frac{h}{6}\cdot 2 = \frac{h}{80} + \frac{3h}{16} = \frac{h}{5}$.

We then take $s = 0$, $u = 0.5$, $t = 1$, and draw the four half-interval coefficients

$$I^{(0)}_{s,u} = 0.80, \quad I^{(0)}_{u,t} = -0.30, \quad I^{(1)}_{s,u} = 0.20, \quad I^{(1)}_{u,t} = 0.50,$$

each from its own Gaussian, with standard deviation $\sqrt{0.5} = 0.7071$ for the increments and $\sqrt{1/6} = 0.4082$ for the first coefficients. The long interval then has

$$I^{(0)}_{s,t} = 0.80 - 0.30 = 0.50, \qquad I^{(1)}_{s,t} = -\tfrac{1}{2}(0.80 + 0.30) + \tfrac{1}{2}(0.20 + 0.50) = -0.55 + 0.35 = -0.20,$$

with marginal variances 1 and 1/3. During training, we sample the two half-interval coefficient sets and derive the full-interval coefficients from them. All three map evaluations then use the same driving path, avoiding the residual floor in Section 6.31.2.

6.31.5 The three identities for a stochastic map

Brownian motion is nowhere differentiable in time. Differentiating the map in s or t also differentiates its path argument, so the Lagrangian and Eulerian derivative losses have no stochastic counterpart. We retain the tangent and composition conditions.

The tangent condition matches a small Euler-Maruyama step of the true equation on the diagonal,

$$\Psi_{t,t+h}(x, W_{[t,t+h]}) = x + f(t,x)\,h + g(t)(W_{t+h} - W_t) + o(h),$$

"Over a short duration h, the solution map $\Psi_{t,t+h}$ adds to the current state x the drift displacement $hf(t,x)$ and the Brownian increment $W_{t+h} - W_t$ scaled by diffusion coefficient $g(t)$, with remainder $o(h)$ smaller than h."

Matching the coefficient of h determines the drift. Matching the Brownian-increment coefficient determines the diffusion. This is the stochastic counterpart of the diagonal flow matching anchor.

To enforce the cocycle, we compare the long map with the composition of two short maps and square their difference. We construct the full-interval coefficients from the two halves using the Chen relations. All three evaluations then share one noise realization. The parameterization enforces identity.

Composition compares finite map evaluations and requires no time derivative, so it remains valid for stochastic maps. Each map application requires two network evaluations, one for drift and one for diffusion.

Fix one driving path and generate with two, four, eight, and thirty-two steps. A strong map gives the same terminal state for each partition because all evaluations follow the same realization. Drawing a fresh path at each step count gives different endpoints. A deterministic map has only one trajectory per starting state and therefore only the first behavior. A weak map redraws noise when the step count changes and therefore has only the second.

On a scalar test problem, the one-step pathwise error decreases from about 0.6 using only the increment to about 0.02 using four coefficients [McCallum et al., 2026]. This measures agreement on individual noise realizations. Agreement in distribution alone permits the two maps to differ on each realized path.

6.32 Expanding flow maps

For fixed-dimensional maps, both the input and output of $X_{s,t}$ lie in $\mathbb{R}^d$. The Jacobian determinant, density formula, and diffeomorphism property depend on this. Generating sentences, molecules, or proteins also requires generating their lengths. Chapter 5 used insertion and deletion rates; an expanding flow map changes the number of coordinates during generation.

6.32.1 Growing state spaces

Let $d(t)$ be the number of coordinates at time t, non-decreasing, and let $\Phi_{s,t}$ be the map between the two state spaces. Its Jacobian is rectangular,

$$\nabla_x \Phi_{s,t}(x) \in \mathbb{R}^{d(t)\times d(s)}, \qquad d(s) \le d(t)$$

"The Jacobian $\nabla_x \Phi_{s,t}(x)$ has one row per destination coordinate, $d(t)$, and one column per starting coordinate, $d(s)$. Expansion requires $d(t)$ to be at least $d(s)$."

A rectangular Jacobian has neither a determinant nor an inverse. We therefore cannot directly use Liouville's log-density formula or the exact likelihood obtained by integrating divergence. An inverse flow $X_{t,s}$ also requires a bijection. Spaces of different dimensions admit no diffeomorphism, so a fixed-dimensional velocity field is insufficient.

We can split the move into a step that changes the length and a step that moves the state,

$$\Phi_{s,t} = X_{s,t} \circ E_{s,t}, \qquad E_{s,t}(x_s, \epsilon) : \mathbb{R}^{d(s)} \times \mathbb{R}^{d(t)-d(s)} \to \mathbb{R}^{d(t)}$$

"The expanding map $\Phi_{s,t}$ first applies the expansion operator $E_{s,t}$ to the current state x_s and fresh noise ϵ, producing $d(t)$ coordinates. It then applies the ordinary flow map $X_{s,t}$ in that expanded space."

- $E_{s,t}$ inserts $d(t) - d(s)$ new coordinates at chosen positions and fills them with independent source noise.
- $\epsilon \in \mathbb{R}^{d(t)-d(s)}$ is that noise, drawn from the source distribution of the model, one draw per inserted coordinate.
- $X_{s,t}$ is the ordinary flow map of this chapter, now acting on $\mathbb{R}^{d(t)}$, where it is square again.

With the expansion done first, the transport that follows has a square Jacobian, so Liouville's formula applies in the expanded space. The density of the expanded state is the density of the current state times the source density of each inserted coordinate, and the transport then contributes the integrated divergence,

$$\log p_t\big(\Phi_{s,t}(x_s)\big) = \log p_s(x_s) + \sum_{i\,:\,t_i^{\text{ins}} \in (s,t]} \log p_0(\epsilon_i) \; - \; \int_s^t \sum_{i\,:\,t_i^{\text{ins}} \le u} \partial_{x_i} b^i(x_u, u)\, du.$$

"The destination log density $\log p_t(\Phi_{s,t}(x_s))$ equals the starting log density $\log p_s(x_s)$ plus source log densities $\log p_0(\epsilon_i)$ for newly inserted coordinates. We subtract the integrated divergence, summing $\partial_{x_i} b^i$ only over coordinates whose insertion times t_i^{ins} are at or before the integration time u."

Only existing coordinates contribute to the inner sum. The velocity is zero on absent coordinates, so the corresponding diagonal derivative entries vanish. Insertions are jumps; the full generator adds a transport term between jumps to an insertion kernel, using the jump-process construction from Chapter 4.

6.32.2 Local clocks

A coordinate inserted late has less time to evolve from noise to data. We assign each coordinate a local interpolant time that runs from 0 at insertion to 1 at the data endpoint,

$$t_i = \frac{t - t_i^{\text{ins}}}{1 - t_i^{\text{ins}}} \in [0, 1]$$

"The local time t_i of coordinate i equals the elapsed time since insertion, $t - t_i^{\text{ins}}$, divided by the total time available to that coordinate, $1 - t_i^{\text{ins}}$."

The local clock advances at $dt_i/dt = 1/(1 - t_i^{\text{ins}})$. A coordinate inserted at 0.9 has one tenth of the interval remaining, so its clock runs ten times faster than global time. A sequence growing from two coordinates to four therefore has several simultaneous local clocks (Figure 6.6).

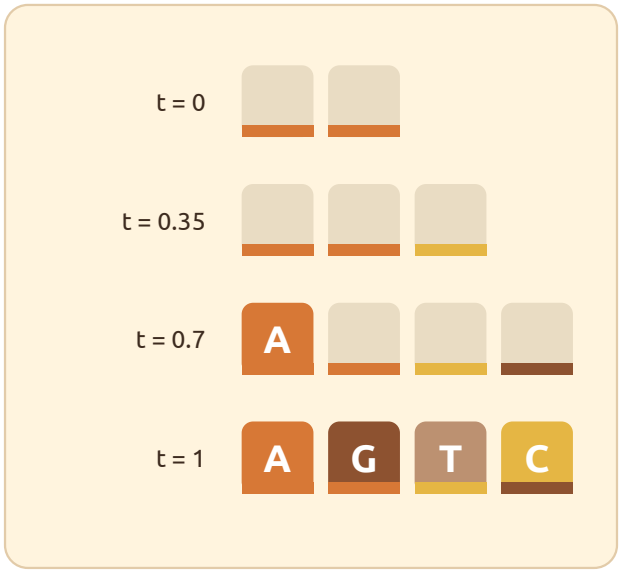


Figure 6.6: A sequence growing from two coordinates at $t = 0$ to four at $t = 1$, read downward. Each tile is one coordinate and the bar beneath it is colored by the time that coordinate was inserted: the first two at $t = 0$, the third at $t = 0.35$ and the fourth at $t = 0.7$. A tile is drawn blank while the coordinate is still noise and carries its base once its argmax has settled, so the first coordinate reads A from $t = 0.7$ onward while the last settles only at $t = 1$. A late birth leaves less global time, so that coordinate's local clock runs faster.

Let's work the clocks of that figure at a single global time. The insertion times are 0, 0, 0.35 and 0.7, and at $t = 0.8$,

$$t_1 = t_2 = \frac{0.8 - 0}{1 - 0} = 0.800, \qquad t_3 = \frac{0.8 - 0.35}{1 - 0.35} = \frac{0.45}{0.65} = 0.692, \qquad t_4 = \frac{0.8 - 0.7}{1 - 0.7} = \frac{0.1}{0.3} = 0.333,$$

with clock speeds 1, 1, $1/0.65 = 1.538$, and $1/0.3 = 3.333$. At the same global time, one position is 80% through its schedule and another is 33% through. A single forward pass predicts velocities at both local times.

Each interpolant uses its coordinate's local time. At $t = 0.8$, the first coordinate has weights 0.2 on source and 0.8 on data. The third has weights 0.308 and 0.692, and the fourth has weights 0.667 and 0.333. The fourth coordinate still has two thirds of its local schedule to complete in one fifth of the global interval. Its clock speed accounts for the difference: $0.667 = 3.333 \times 0.2$. Late coordinates therefore require faster denoising with the same available global steps. We set the velocity to zero for coordinates that do not yet exist. Encoding this mask in the parameterization removes the need to train it.

6.32.3 Flow-map identities with insertions

Between two insertions the dimension is constant, the map is square, and the Lagrangian and Eulerian identities hold exactly as they did at fixed length. Across an insertion the state changes length, so neither derivative in time is defined, because the two sides of the derivative belong to spaces of different dimension. The composition identity is a finite difference and needs neither derivative, and it holds for every triple of times,

$$\psi_{s,t}(x_s^\epsilon) = \omega_{s,u,t}\, \psi_{s,u}^{(t)}(x_s^\epsilon) + (1 - \omega_{s,u,t})\, \psi_{u,t}(E_{u,t}(\Phi_{s,u}(x_s)))$$

"The long-interval denoiser $\psi_{s,t}(x_s^\epsilon)$ combines the first-leg prediction $\psi_{s,u}^{(t)}$, expanded to the final length, with weight $\omega_{s,u,t}$, and the second-leg prediction $\psi_{u,t}$ with weight $1 - \omega_{s,u,t}$. We evaluate the second prediction at the first leg's output after applying the expansion $E_{u,t}$."

- x_s^ϵ is the state at s after it has been expanded to the final length $d(t)$, with the new coordinates filled from the source.
- $\psi_{s,u}^{(t)}$ is the prediction of the first leg, lifted to length $d(t)$ by leaving the coordinates inserted after u at their source values.
- $\omega_{s,u,t} = (u - s)(1 - t)/\big((t - s)(1 - u)\big)$ is the remaining-time weight of the earlier section, unchanged.

The weight ω depends only on remaining-time ratios in the parameterization. After we expand each prediction to length $d(t)$, all vectors share a space and the fixed-length derivation applies unchanged. At $(s, u, t) = (0.3, 0.6, 0.9)$,

$$\omega = \frac{(0.6 - 0.3)(1 - 0.9)}{(0.9 - 0.3)(1 - 0.6)} = \frac{0.3 \times 0.1}{0.6 \times 0.4} = \frac{0.03}{0.24} = 0.125,$$

so the first leg contributes one eighth and the second contributes seven eighths, despite their equal durations. Predictions closer to the data are sharper. Their weights also depend on the remaining time at the start of each leg, giving the later prediction more weight.

6.32.4 Shared noise across the three maps

To compare the three maps, we must insert the same coordinates with the same source noise in each. Independent source draws would produce different sequences, so their prediction difference would include a noise mismatch unrelated to map accuracy.

We generate the randomness once by expanding to the final length $d(t)$ before evaluating any map. We draw source values for every inserted position, then leave positions inserted after u at those values during the first leg. Each map uses the same draw, restricted by a mask.

Both stochastic and expanding maps require shared randomness across composed intervals. For a stochastic map, we sample the two half-interval coefficient sets and combine them with the Chen relations. For an expanding map, we sample the full insertion set and restrict it to each leg. The three map evaluations then describe one realization.

Time derivatives fail for both constructions: a Brownian path is nowhere differentiable, and an expanding state's dimension jumps at insertion. Composition remains defined because it compares finite maps without taking either derivative.

For molecular graphs with generated atom counts, the expanding map achieves one-step Fréchet ChemNet distance 0.44 and uniqueness 97.7. The fixed-length map has distance 2.14 at the same budget [Tang and Chatterjee, 2026]. The distance gap closes at two steps.

6.33 Guidance and the posterior over endpoints

In Chapter 5, we guided a jump process by multiplying rates by ratios of expected terminal rewards, using Doob's h-transform [Doob, 1957]. A continuous flow admits the same tilt, which requires an expectation over endpoints that a deterministic map cannot estimate.

Let $r(z)$ be a reward on terminal states and define the tilted target and the value function,

$$p^r(z) \;\propto\; p_{\text{data}}(z)\, e^{r(z)}, \qquad V_t(x) = \log \mathbb{E}\big[e^{r(X_1)} \,\big|\, X_t = x\big]$$

"The tilted target p^r weights the data distribution by the exponential reward $e^{r(z)}$. The value function $V_t(x)$ is the logarithm of that exponential reward averaged over terminal states conditioned on the current state x at time t."

The exponentiated value function is the ratio between the tilted and original marginal densities at time t. Its gradient determines the correction to the original velocity,

$$e^{V_t(x)} \;\propto\; \frac{p_t^r(x)}{p_t(x)}, \qquad b_t^r(x) = b_t(x) + \tfrac{1}{2} g_t^2\, \nabla V_t(x)$$

"The exponentiated value $e^{V_t(x)}$ is proportional to the tilted-to-original density ratio $p_t^r(x)/p_t(x)$. The controlled velocity $b_t^r(x)$ adds to the base velocity $b_t(x)$ one half the squared diffusion coefficient g_t^2 times the value gradient $\nabla V_t(x)$."

With e^V as the h-function, this is the transform from Chapter 5 in velocity notation. Sampling the tilted target requires estimating $\nabla V_t(x)$.

We take the data distribution on two points, $z \in \{0, 2\}$ with equal mass, and the reward $r(z) = z$. The tilted target weights the two by 1 and $e^2 = 7.389$,

$$p^r(0) = \frac{1}{1 + e^2} = 0.1192, \qquad p^r(2) = \frac{e^2}{1 + e^2} = 0.8808,$$

so a reward gap of two nats changes equal probabilities to roughly seven to one. At an intermediate state, we apply the same weighting to its endpoint posterior. For $x_t = 0.7$ at $t = 0.8$ under the linear interpolant, the posterior is $(0.9820, 0.0180)$. Multiplying the second entry by e^2 and normalizing gives $(0.8808, 0.1192)$. The value function is $V_t(0.7) = \log(0.9820 + 0.0180 \times 7.389) = \log 1.1149 = 0.1088$. Both calculations require the endpoint posterior, which the deterministic map does not represent.

6.33.1 The limit of a deterministic map

Let's rewrite the value function as an average over the posterior on terminal states,

$$V_t(x) = \log \mathbb{E}_{z \sim p_{1|t}(\cdot|x)}\big[e^{r(z)}\big]$$

"The value $V_t(x)$ is the logarithm of the exponential reward $e^{r(z)}$ averaged over terminal states z drawn from the posterior $p_{1|t}(\cdot \mid x)$, conditional on state x at time t."

A deterministic flow map returns one terminal state for each input, namely $X_{t,1}(x)$, so the only estimate it supports is the reward at that single state,

$$V_t(x) \;\approx\; r\big(X_{t,1}(x)\big)$$

"We approximate the value $V_t(x)$ by evaluating the reward r at the single terminal state $X_{t,1}(x)$ produced by the deterministic map from state x and time t."

This approximation discards variation across possible endpoints. A deterministic function of x returns one value regardless of training accuracy. We can quantify the resulting Jensen gap with $z \in \{0, 2\}$, equal endpoint probabilities, and reward $r(z) = z$. The exact value is

$$V = \log\left(0.5\, e^0 + 0.5\, e^2\right) = \log(0.5 + 3.6945) = \log 4.1945 = 1.4338.$$

Using the posterior mean $z = 1$ as the deterministic endpoint gives reward 1, so the single-endpoint estimate is 1.000, short by 0.434. Convexity of the exponential makes this estimate lower than the exact average. The gap increases with posterior spread, which is largest at early times when guidance has the greatest effect.

6.33.2 An exogenous noise argument

We can sample multiple endpoints by giving the map an additional noise input and conditioning it on the current state. Two methods developed this construction independently [Holderrieth et al., 2026, Potaptchik et al., 2026a],

$$\Phi(\epsilon; t, x) := X_{0,1}(\epsilon \mid x, t), \qquad \epsilon \sim \mathcal{N}(0, I) \implies \Phi(\epsilon; t, x) \sim p_{1|t}(\cdot \mid x)$$

"The conditional map $\Phi(\epsilon; t, x)$ transforms a fresh Gaussian draw ϵ into a terminal sample distributed according to $p_{1|t}(\cdot \mid x)$, the endpoint posterior at the fixed conditioning time t and state x."

For each fixed conditioning pair this is an ordinary two-time flow map on an inner time axis. Write $Y_{a_1,a_2}(\cdot \mid x, t)$ for the inner map from inner time a_1 to inner time a_2 at the conditioning pair (t, x), so that $\Phi(\epsilon; t, x) = Y_{0,1}(\epsilon \mid x, t)$. All three axioms hold on that axis,

$$Y_{a_1,a_1} = \mathrm{id}, \qquad Y_{a_2,a_3} \circ Y_{a_1,a_2} = Y_{a_1,a_3}, \qquad \partial_{a_2} Y_{a_1,a_2}(\epsilon \mid x, t)\big|_{a_2=a_1} = b^{\mathrm{in}}(\epsilon,\, a_1 \mid x, t),$$

"At fixed conditioning pair (t, x), the inner map Y_{a_1,a_2} equals the identity when its inner times agree, and two inner intervals compose into one. Its derivative in inner end time a_2, evaluated at $a_2 = a_1$, equals the inner velocity b^{in}."

For each fixed conditioning pair, the inner map satisfies the same flow-map results. Varying the conditioning pair gives a family indexed by the sampler's current state and time.

The outer time t indexes the sampler's trajectory from source to data, with state x_t advanced by Euler steps of the controlled velocity. The inner time runs from 0 to 1 while x_t remains fixed and ends at a posterior sample. Independent inner noise draws produce independent posterior samples. Holding that draw fixed while varying nearby conditioning states makes the output vary continuously, allowing differentiation in x.

In both methods the inner velocity comes from a pretrained model by a change of coordinates, so no new denoiser is trained for the inner axis.

6.33.3 The value gradient from K samples

With posterior samples available, we estimate the value function by its sample average,

$$V_t(x) \approx \log \frac{1}{K} \sum_{k=1}^{K} e^{r(z^k)}, \qquad z^k = \Phi(\epsilon^k; t, x)$$

"We estimate the value $V_t(x)$ by taking the logarithm of the average exponential reward $e^{r(z^k)}$ over K posterior samples. Each terminal sample $z^k = \Phi(\epsilon^k; t, x)$ uses an independent inner Gaussian draw ϵ^k at the same conditioning pair (t, x)."

To differentiate the estimate with respect to the conditioning state, write $S = \sum_j e^{r(z^j)}$ and apply the chain rule,

$$\nabla_x \log \frac{S}{K} = \frac{\nabla_x S}{S} = \frac{\sum_k e^{r(z^k)} \nabla_x r(z^k)}{\sum_j e^{r(z^j)}},$$

where the constant $1/K$ drops out of the logarithmic derivative and each term picks up the chain rule through its own sample. Rearranging the quotient into a weighted sum,

$$\nabla_x V_t(x) \approx \sum_{k=1}^{K} \frac{e^{r(z^k)}}{\sum_j e^{r(z^j)}} \nabla_x r(z^k)$$

"The value-gradient estimate $\nabla_x V_t(x)$ sums the sample reward gradients $\nabla_x r(z^k)$, weighted by the softmax of the K rewards $r(z^k)$. We differentiate each reward through its posterior sample and the map's conditioning state x."

The softmax weights emphasize gradients from higher-reward samples; equal rewards give a uniform average. Reparameterizing the inner noise permits differentiation through the conditioning argument, with $\nabla_x r(z^k) = (\partial z^k / \partial x)^\top \nabla r(z^k)$ computed by backpropagation.

Suppose the rewards at the three posterior samples are $r = (0.2,\ 1.5,\ 0.9)$, so the exponentials are $(1.2214,\ 4.4817,\ 2.4596)$ with total 8.1627 and the softmax weights are

$$\left(\frac{1.2214}{8.1627},\ \frac{4.4817}{8.1627},\ \frac{2.4596}{8.1627}\right) = (0.1496,\ 0.5491,\ 0.3013).$$

If the reward gradients at the three samples are $(0.4,\ -1.2,\ 0.5)$ in some coordinate, the estimated value gradient in that coordinate is

$$0.1496\,(0.4) + 0.5491\,(-1.2) + 0.3013\,(0.5) = 0.0598 - 0.6589 + 0.1507 = -0.4483,$$

against a plain average of -0.1000. The sample with the highest reward has over half the weight, so the guided velocity points along its gradient, and the two lower-reward samples move it only a little. With equal rewards the weights would be $1/3$ each and the two estimates would coincide.

The estimate is consistent and biased, since a logarithm of an average is not an average of logarithms. Continuing the two-point example, with K independent posterior draws the expectation of the estimate is

$$\mathbb{E}\big[\widehat{V}_K\big] = \sum_{j=0}^{K} \binom{K}{j} 2^{-K} \log\!\left(\frac{(K-j) + je^2}{K}\right),$$

summing over the number j of draws that landed on $z = 2$. Evaluating it,

$$\mathbb{E}\big[\widehat{V}_1\big] = 1.000, \qquad \mathbb{E}\big[\widehat{V}_2\big] = 1.217, \qquad \mathbb{E}\big[\widehat{V}_4\big] = 1.340, \qquad \mathbb{E}\big[\widehat{V}_8\big] = 1.393,$$

compared with the exact value 1.4338. The biases are 0.434, 0.217, 0.093, and 0.041. At $K = 1$, the bias equals the single-endpoint Jensen gap because affine rewards have the same expectation as their value at the posterior mean. Doubling K roughly halves the bias. In the published particle-count ablation, alignment improves from 0.778 with one particle to 0.818 with two [Holderrieth et al., 2026].

Each outer Euler step requires K inner-map calls, K reward evaluations, and a backward pass through each sample for its gradient. The cost scales as K, while the bias decreases roughly as $1/K$. We choose K according to the acceptable bias in the reward estimate.

6.33.4 Posterior sampling with a trained map

Training a conditional family is one way to get posterior samples. A second way uses a deterministic flow map that already exists. We can renoise the current state back to an earlier time with fresh noise, then push the renoised state through the existing map,

$$x_{t'}(x_t, \epsilon) = \frac{\alpha_{t'}}{\alpha_t} x_t + \sqrt{\sigma_{t'}^2 - \frac{\alpha_{t'}^2}{\alpha_t^2}\sigma_t^2}\ \epsilon, \qquad z = X_{t',1}(x_{t'})$$

"The renoised state $x_{t'}$ equals the current state x_t times the signal ratio $\alpha_{t'}/\alpha_t$, plus a Gaussian draw ϵ scaled by the restored noise's standard deviation. Its variance is the earlier noise variance $\sigma_{t'}^2$ minus the current variance σ_t^2 times the squared signal ratio. Applying the existing map to $x_{t'}$ gives the terminal sample z."

- α_t and σ_t are the signal and noise coefficients of the interpolant at time t, so that $X_t = \alpha_t X_1 + \sigma_t X_0$.
- $t' < t$ is the earlier time we renoise to, and the variance under the square root is non-negative exactly when the schedule has more noise at t' than the rescaling preserves.

Nonnegative variance requires $\sigma_{t'}/\alpha_{t'} \geq \sigma_t/\alpha_t$, so the schedule's noise-to-signal ratio must decrease with time. The interpolants used here satisfy this condition, allowing any $t' < t$. Choosing t' just below t adds little variance and gives almost the deterministic endpoint. Choosing t' near zero adds nearly the full source variance and produces almost a prior sample. An intermediate value can cover several modes while avoiding weights concentrated on a single sample.

Renoising discards information that was in x_t, so the law of z is wider than the posterior. Take the two-point data distribution again, $z \in \{0, 2\}$ with equal mass, the linear schedule $\alpha_t = t$ and $\sigma_t = 1 - t$, and the current state $x_t = 0.7$ at $t = 0.8$. The exact posterior there is

$$p(z = 2 \mid x_t = 0.7, t = 0.8) = \frac{e^{-(0.7-1.6)^2/0.08}}{e^{-(0.7-1.6)^2/0.08} + e^{-0.7^2/0.08}} = 0.0180,$$

so the far mode has under two percent. Renoising to $t' = 0.6$ rescales by $\alpha_{t'}/\alpha_t = 0.75$ and adds variance $0.4^2 - 0.75^2 \times 0.2^2 = 0.16 - 0.0225 = 0.1375$, a standard deviation of 0.3708,

$$x_{0.6} = 0.75 \times 0.7 + 0.3708\,\epsilon = 0.525 + 0.3708\,\epsilon.$$

The exact map from time 0.6 sends states above 0.6 to $z = 2$ and states below it to $z = 0$. The renoised proposal therefore assigns $\mathbb{P}(\epsilon > 0.2023) = 0.420$ to the far mode, twenty-three times its posterior weight. For $\epsilon = +1$, we get $x_{0.6} = 0.896$ and endpoint $z = 2$. For $\epsilon = -1$, we get $x_{0.6} = 0.154$ and $z = 0$. The proposal reaches both modes even though the original state almost certainly came from $z = 0$.

The correction is an importance weight, written as a shift of the reward,

$$r_{\text{recov}}(x_t, \epsilon) = r\big(z(x_t, \epsilon)\big) - \frac{\|x_t - \alpha_t z(x_t, \epsilon)\|^2}{2\sigma_t^2}$$

"The recovery reward $r_{\text{recov}}(x_t, \epsilon)$ equals the terminal reward $r(z(x_t, \epsilon))$ minus the squared distance from current state x_t to the signal-scaled endpoint $\alpha_t z(x_t, \epsilon)$, divided by twice the interpolant noise variance σ_t^2."

The second term is the log-likelihood of observing x_t from the terminal sample. Endpoints that are unlikely to have produced the current state receive small weights. In the two-point example, the penalties are

$$\frac{(0.7 - 1.6)^2}{0.08} = 10.125 \quad (z = 2), \qquad \frac{0.7^2}{0.08} = 6.125 \quad (z = 0),$$

so the far mode is penalized by four nats, a factor $e^{-4} = 0.0183$. Applying the weights to the proposal, the far mode's share falls from 0.420 to

$$\frac{0.420 \times e^{-10.125}}{0.420 \times e^{-10.125} + 0.580 \times e^{-6.125}} = 0.0131,$$

close to the true posterior weight 0.0180. The full estimator also includes the difference between scores at the renoised and current states. With that term, we can estimate the posterior from a pretrained flow map without additional training.

6.34 Meta flow maps

Training the conditional family requires posterior samples, which are also its desired output. For Meta Flow Maps [Potaptchik et al., 2026a], we construct a coupling that gives both a conditioning state and an exact posterior sample from one data point.

6.34.1 Interpolants with a shared data endpoint

We draw a data point and use it as the endpoint of two interpolants at once, with independent source draws,

$$I_t = (1-t)\, I_0 + t\, I_1, \qquad \bar{I}_s = (1-s)\, \bar{I}_0 + s\, I_1, \qquad I_0,\, \bar{I}_0 \text{ independent}$$

"The outer interpolant I_t combines source draw I_0 and data point I_1 with weights $1-t$ and t. The inner interpolant $\bar{I}_s$ combines an independent source draw $\bar{I}_0$ with that same data point using weights $1-s$ and s."

The inner source draw $\bar{I}_0$ is independent of the variables used to construct the outer state. Conditioning on that state therefore leaves $\bar{I}_0$ at its prior. The shared data point has the corresponding posterior law,

$$\mathrm{Law}(\bar{I}_0, I_1 \mid I_t = x) = p_0 \times p_{1|t}(\cdot \mid x)$$

"Conditioning on the outer state $I_t = x$, the joint law of inner source $\bar{I}_0$ and data point I_1 factors into the source law p_0 and the endpoint posterior $p_{1|t}(\cdot \mid x)$. Thus these two variables remain independent under the conditioning."

The outer state $x = I_t$ is the conditioning context, and the shared data point is an exact posterior sample for it. We can therefore train the inner map by regression without simulation, including (t, x) as an additional input.

6.34.2 Fine-tuning to the controlled velocity

With posterior samples available, we can fit the controlled velocity directly. Writing the gradient of the value function through the family, the target is

$$b_t^\star(x) = b_t(x) + \frac{g_t^2}{2}\, \frac{\mathbb{E}_\epsilon\big[\nabla_x e^{r(\Phi(\epsilon;t,x))}\big]}{\mathbb{E}_\epsilon\big[e^{r(\Phi(\epsilon;t,x))}\big]}$$

"The controlled velocity $b_t^\star(x)$ equals the base velocity $b_t(x)$ plus one half g_t^2, the squared diffusion coefficient, multiplied by a ratio of expectations. The numerator averages the conditioning-state gradient of the exponential reward $e^{r(\Phi(\epsilon;t,x))}$; the denominator averages that exponential reward itself, both over inner noise ϵ."

Regressing onto a sample estimate of this ratio introduces finite-sample bias because $\mathbb{E}[G(\epsilon)/w(\epsilon)]$ differs from $\mathbb{E}[G]/\mathbb{E}[w]$. Multiplying by the denominator gives an equivalent condition at the optimum. We can enforce it with a squared loss and one stop-gradient,

$$\mathcal{L} = \mathbb{E}\Big\| \Delta \;-\; \tfrac{1}{2} g_t^2\, G(\epsilon) \;+\; \big(w(\epsilon) - 1\big)\, \mathrm{sg}(\Delta) \Big\|^2, \quad \Delta = \hat{b}_t(x) - b_t(x)$$

"The loss $\mathcal{L}$ averages a squared norm over inner noise ϵ. Inside it, we subtract one half g_t^2 times the exponential-reward gradient $G(\epsilon)$ from the velocity correction $\Delta = \hat{b}_t(x) - b_t(x)$, then add the exponential reward $w(\epsilon)$ minus one, multiplied by the detached correction $\mathrm{sg}(\Delta)$."

- $w(\epsilon) = e^{r(\Phi(\epsilon;t,x))}$ is the exponential reward at one posterior sample.
- $G(\epsilon) = \nabla_x e^{r(\Phi(\epsilon;t,x))}$ is its gradient in the conditioning state, taken through the map.
- $\mathrm{sg}(\cdot)$ is the stop-gradient, so the detached copy contributes a value and no gradient.

Differentiating in Δ with the detached copy held constant and setting the result to zero,

$$\mathbb{E}_\epsilon[\Delta - \tfrac{1}{2} g_t^2\, G(\epsilon) + (w(\epsilon) - 1)\,\mathrm{sg}(\Delta)] = 0, \quad \mathrm{sg}(\Delta) = \Delta \implies \Delta = \tfrac{1}{2} g_t^2\, \frac{\mathbb{E}[G]}{\mathbb{E}[w]}$$

"At the optimum, the expected correction Δ minus the scaled sample gradient $g_t^2 G(\epsilon)/2$, plus $(w(\epsilon) - 1)\mathrm{sg}(\Delta)$, equals zero. At a fixed point, the detached value equals Δ, giving the correction as $g_t^2/2$ times the expected gradient $\mathbb{E}[G]$ divided by the expected reward weight $\mathbb{E}[w]$."

The integrand of that gradient is linear in the sample, so a single draw of ϵ gives an unbiased estimate of the full gradient. The ratio appears only in the solution of the equation and never in an estimate, so one sample per step does not bias it.

Let's take a scalar state with $g_t^2 = 2$ and three posterior samples whose weights and reward gradients are

$$w = (1.0,\ 2.0,\ 5.0), \qquad G = (0.4,\ 1.0,\ 3.6),$$

so that $\mathbb{E}[w] = 8/3 = 2.6667$ and $\mathbb{E}[G] = 5/3 = 1.6667$. The correct correction is

$$\Delta^\star = \tfrac{1}{2} g_t^2\, \frac{\mathbb{E}[G]}{\mathbb{E}[w]} = \frac{1.6667}{2.6667} = 0.6250.$$

Regressing onto per-sample ratios averages $0.4/1.0 = 0.400$, $1.0/2.0 = 0.500$, and $3.6/5.0 = 0.720$, giving 0.5400, short by 0.085. For the stop-gradient loss, the expected gradient is $2[\Delta - \mathbb{E}[G] + (\mathbb{E}[w] - 1)\Delta]$. Setting it to zero gives $\Delta\mathbb{E}[w] = \mathbb{E}[G]$, so its stationary point is exactly 0.625. We can express the difference using a covariance,

$$\mathbb{E}\left[\frac{G}{w}\right] - \frac{\mathbb{E}[G]}{\mathbb{E}[w]} = -\frac{\mathrm{Cov}\,(G/w,\, w)}{\mathbb{E}[w]},$$

by expanding $\mathrm{Cov}(G/w, w) = \mathbb{E}[G] - \mathbb{E}[G/w]\mathbb{E}[w]$ and dividing by $\mathbb{E}[w]$.

In this example, $\mathrm{Cov}(G/w, w) = 1.6667 - 0.5400 \times 2.6667 = 0.2267$. Dividing by $\mathbb{E}[w] = 2.6667$ gives the 0.085 gap with the sign reversed. Its value depends on the joint distribution of weight and gradient, so additional samples of the per-sample ratio do not remove it. The stop-gradient form avoids that bias even with one sample per update because its gradient is linear in the sampled quantities.

The loss imposes a pointwise condition in x, so we can sample states from any distribution with full support. Using the interpolant avoids sampler rollouts and permits simulation-free regression for reward fine-tuning.

6.34.3 Objective comparison

Table 6.5 compares the four consistency objectives by their derivatives and targets.

Table 6.5: The four consistency objectives, by what each differentiates and where each evaluates its target. The first two are the Eulerian and the Lagrangian identities of this chapter; the third is the Eulerian row with one realized velocity in place of the model's own diagonal, which is the MeanFlow loss; the fourth is the composition identity of this chapter, which the taxonomy names the semigroup objective. Every row carries a stop-gradient on its target, and the first two come in a self-distillation variant, where the target is the model's own diagonal, and a teacher-distillation variant, where it is a separate pretrained model.

Objective	Differentiates in	Target evaluated at	Extra cost
Eulerian	start time s	the input x	spatial gradient of the map
Lagrangian	end time t	the output $X_{s,t}(x)$	one time derivative
MeanFlow	start time s	the input, one realized $\dot{I}_t$	one time derivative
Semigroup	neither	the composed state	third forward pass

The Eulerian objective differentiates in the start time and through all state coordinates, making it the least stable of the four on high-dimensional problems. The Lagrangian objective differentiates in the end time and evaluates its target at the output. MeanFlow replaces the Eulerian diagonal prediction with one realized velocity, saving one of two network evaluations. The semigroup objective is the finite composition identity and requires no derivative.

For the first two objectives, we can obtain targets from either self-distillation or teacher distillation. Self-distillation uses a detached diagonal prediction from the same model, allowing training from scratch. The diagonal target changes during training, and the anchor constrains it. Teacher distillation uses a fixed pretrained velocity model and trains the off-diagonal relation. Each step then includes a forward pass through that teacher. Both variants have the same minimizer and use stop-gradients so that only the student receives gradients, but their training dynamics differ.

Fine-tuning the deterministic map into a conditional family increases one-step Fréchet inception distance from 2.16 to 3.72. At four steps, the family improves the distance from 1.97 to 1.51, using 1.3% more parameters [Potaptchik et al., 2026a]. Conditioning permits posterior endpoint sampling at each visited state, which the deterministic map cannot provide at any sampling budget.

6.35 Map inputs and training identities

The five cases in Table 6.6 differ in the map's additional input. That input determines which training identities are available.

For a continuous state alone, all three identities are available. For token distributions, the composition target stays on the simplex, while the derivative targets require adjustment. A Brownian path has no time derivative, leaving composition available. A growing state also lacks a time derivative at insertions, so composition remains the available identity across them. Conditioning on an additional state permits all three identities because that argument is differentiable, and the resulting family can sample endpoint posteriors.

We can summarize these requirements in two conditions.

Table 6.6: The five cases, by what the map takes as an argument besides the state. The third column names the identities available in that case. A derivative identity needs every argument of the map to be differentiable in time; the composition identity needs only that the extra argument can be generated once and restricted to each leg.

Extra argument	Obstruction	Identities left	Methods
none, a state alone	none	all three	Flow Map Matching, self-distillation, MeanFlow, shortcut models, consistency models
a distribution over tokens	derivative targets leave the simplex	all three, after repair	Flow Map Language Models, Categorical Flow Maps, Discrete Flow Maps
a Brownian path	nowhere differentiable in time	composition only	Strong Stochastic Flow Maps
a dimension that grows	no time derivative across an insertion	composition only	Expanding Flow Maps
a conditioning state	none	all three	Diamond Maps, Meta Flow Maps

- A derivative identity is available only when every argument of the map is differentiable in time.
- A composition identity is available whenever the extra argument can be generated once and restricted to each leg.

Composition remains available in each case, and two methods train it alone. The state space and additional inputs therefore constrain the choice of identity before we run experiments.

Training with a derivative identity requires two network evaluations, for the student at (s, t) and the diagonal target, plus one forward-mode derivative. The Eulerian tangent spans the state; the Lagrangian tangent has the output's size. Composition requires three forward evaluations, no derivative, and shared randomness across all three. These evaluation counts are independent of dimension, from scalar examples to length-128 sequences with thirty-thousand-token vocabularies. The state space determines which identities are defined.

6.36 Summary

Three axioms determine a flow map,

$$\boxed{X_{t,t} = \mathrm{id}, \qquad X_{t,u} \circ X_{s,t} = X_{s,u}, \qquad \partial_t X_{s,t}(x)\big|_{t=s} = b(x, s)}$$

"The zero-duration map $X_{t,t}$ is the identity, and composing $X_{s,t}$ with $X_{t,u}$ gives $X_{s,u}$. The end-time derivative of $X_{s,t}(x)$ on the diagonal $t = s$ equals the instantaneous velocity $b(x, s)$."

We parameterize the map with its average velocity,

$$X_{s,t}(x) = x + (t - s)\, v_{s,t}(x),$$

which enforces identity for every $v_{s,t}$. We train the tangent and composition conditions. In denoiser coordinates, the step is a convex combination of the current state and a predicted endpoint, with predictions constrained to the simplex.

MeanFlow, shortcut models, consistency models, and deterministic self-distillation use the state alone, permitting all three identities. Flow Map Language Models uses a convex target, Categorical Flow Maps uses a loss bound, and Discrete Flow Maps uses logit coordinates to accommodate token distributions. For Strong Stochastic Flow Maps, we share one Brownian realization by combining half-interval coefficients. For Expanding Flow Maps, we share inserted noise by drawing it once and restricting it to each interval. Diamond Maps and Meta Flow Maps condition on the current state and time, allowing all three identities and posterior endpoint sampling for guidance.

The corresponding training residuals are

$$\mathcal{L}_{\mathrm{LSD}}, \quad \mathcal{L}_{\mathrm{ESD}}, \quad \mathcal{L}_{\mathrm{PSD}},$$

for the Lagrangian, Eulerian, and composition identities. Each is paired with the diagonal anchor. When the average velocity is continuous in both times, the combined objectives have the same unique minimizer and exclude the stay-still solution. For imperfect networks, their residuals and error bounds differ.

The coupling used to draw training pairs determines the trajectory compressed by a flow map. To evaluate these losses, we draw I_0 and I_1 independently from source and data, then interpolate linearly. Different couplings between the same distributions produce different trajectories and maps, some easier to learn than others. Crossing trajectories can make the average velocity vary sharply in space; noncrossing trajectories keep it smooth. The discrete rectification procedure in Chapter 5 changes the coupling by retraining on pairs produced by a trained model.

Choosing a source-to-data pairing is a transport problem. Its computational cost depends on the coupling constraints, including whether the pairing must respect reference dynamics as well as distance.

To choose a coupling, we need a criterion for comparing pairings that preserve the source and data distributions. In Chapter 7, we assign a cost to moving probability between endpoints and minimize the total cost. We then connect this endpoint problem to the paths used during generation. For Schrödinger bridges, we compare complete path distributions with a specified reference process and find the closest distribution that satisfies the endpoint constraints.

Chapter 7

Optimal Transport and Schrödinger Bridges

7.1 Choosing the pairing

We train the models of Chapters 2 through 6 on pairs of samples. In flow matching, we draw a source sample x_0 and a data sample x_1, then regress a velocity onto the straight path between them. In diffusion, we corrupt a data sample toward noise and learn the reverse process. Discrete flow matching interpolates between a source token and a data token. With a flow map, we learn the two-time map along that interpolation. In almost all of these constructions, we drew x_0 and x_1 independently, so each training pair consisted of two independently drawn endpoints.

The two distributions specify how often each source sample and each data sample appears. We can optimize the pairing subject to preserving those frequencies. For a generative model, we also need a path between each pair of endpoints.

7.1.1 Transport cost

To compare pairings, assign a cost $c(x, y)$ to moving one unit of mass from source point x to target point y. We use squared Euclidean distance as the default cost,

$$c(x, y) = \|x - y\|^2.$$

"The transport cost $c(x, y)$ equals the squared Euclidean distance from source point x to target point y per unit of mass, so doubling the distance multiplies the cost by four."

- x is a source point, a location in $\mathbb{R}^d$ that currently holds mass.
- y is a target point, a location in $\mathbb{R}^d$ where mass is wanted.
- $c(x, y) \geq 0$ is the cost of one unit of mass making that move, in arbitrary units.

Squaring the distance penalizes long moves more heavily. Two moves of length 1 cost 2 together, whereas one move of length 2 costs 4. Under this cost, many short moves can cost less than a few

long ones. The squared cost also connects optimal transport to the straight-line interpolations of Chapter 2.

7.1.2 Four points and two pairings

Let's take two source points on the real line, one at 0 and one at 2, and two target points, one at 1 and one at 3, each of the four with mass 1/2. Two pairings move all of the mass.

The ordered pairing sends 0 to 1 and 2 to 3. Its two moves have length 1 each, and with the squared cost the total is $1 + 1 = 2$. The crossed pairing sends 0 to 3 and 2 to 1. The first move has length 3 and costs 9, the second has length 1 and costs 1, and the total is $9 + 1 = 10$. The crossed pairing costs five times as much for the same transfer of probability (Figure 7.1).

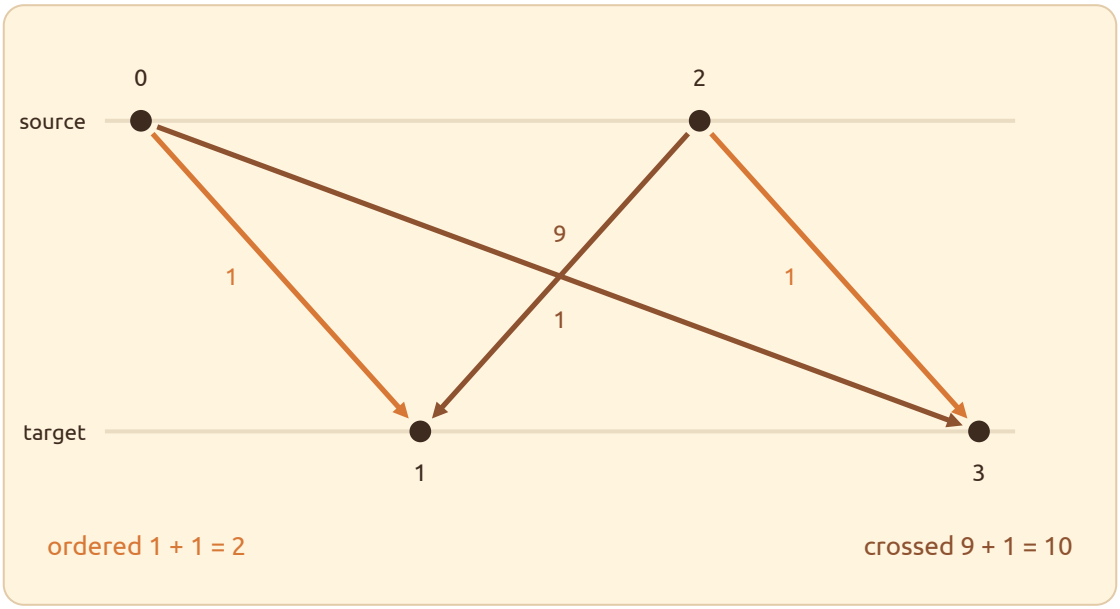


Figure 7.1: Two source points at 0 and 2 on the upper line, two target points at 1 and 3 on the lower line, and the arrows of the two pairings, each labeled with the squared distance it costs. The light arrows are the ordered pairing, $0 \to 1$ and $2 \to 3$, at total cost $1 + 1 = 2$. The dark arrows are the crossed pairing, $0 \to 3$ and $2 \to 1$, at total cost $9 + 1 = 10$.

Both pairings move the same source distribution to the same target distribution, so their marginals are identical. We distinguish them by cost: the ordered pairing has the lower squared cost.

7.1.3 Transport matrices

Let a be a probability vector on n source points $x_1, \dots, x_n$, so a_i is the mass at x_i and $\sum_i a_i = 1$. Let b be a probability vector on m target points $y_1, \dots, y_m$, with b_j the mass required at y_j. A pairing is then an $n \times m$ matrix π whose entry π_{ij} is the mass moved from x_i to y_j. All of the mass at x_i must be sent somewhere, and all of the mass required at y_j must be received, which fixes the row sums and the column sums,

$$\Pi(a, b) = \left\{ \pi \in \mathbb{R}_+^{n \times m} \;:\; \sum_j \pi_{ij} = a_i, \;\; \sum_i \pi_{ij} = b_j \right\}.$$

"The coupling set $\Pi(a, b)$ consists of nonnegative n-by-m transport matrices π whose row sums $\sum_j \pi_{ij}$ equal the source masses a_i and whose column sums $\sum_i \pi_{ij}$ equal the target masses b_j."

Definition 7.1 (Coupling)**.** A matrix $\pi \in \Pi(a, b)$ is a *coupling* of a and b, also called a *transport plan*. Equivalently, π is the joint law of a pair of random indices whose first marginal is a and whose second marginal is b.

We can write both constraints using the all-ones vectors $\mathbf{1}_m$ and $\mathbf{1}_n$, of lengths m and n,

$$\pi \mathbf{1}_m = a, \qquad \pi^\top \mathbf{1}_n = b.$$

"Multiplying the transport matrix π by the all-ones vector $\mathbf{1}_m$ sums each row to give the source mass vector a. Multiplying its transpose $\pi^\top$ by the all-ones vector $\mathbf{1}_n$ sums each column to give the target mass vector b."

In Chapter 5 we already wrote π for a coupling of a source sample with a data sample, and the symbol means the same here, with the index pair (i, j) naming a route between two points.

The set $\Pi(a, b)$ is never empty. The independent coupling

$$\pi_{ij} = a_i \, b_j$$

has row sums $a_i \sum_j b_j = a_i$ and column sums $b_j \sum_i a_i = b_j$, so it always belongs to $\Pi(a, b)$. Drawing x_0 and x_1 independently in Chapters 2 through 6 uses this coupling. It is one candidate in $\Pi(a, b)$.

The two pairings of Section 7.1.2 become two matrices. With the sources at 0 and 2 and the targets at 1 and 3, each point with mass 1/2, the two pairings are

$$\pi^{\text{ord}} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}, \qquad \pi^{\text{cross}} = \begin{pmatrix} 0 & 1/2 \\ 1/2 & 0 \end{pmatrix},$$

and the squared costs of the four routes, laid out the same way, are 1, 9, 1 and 1. If we weight each route cost by the mass that moves along it, the ordered pairing costs $\frac{1}{2}(1) + \frac{1}{2}(1) = 1$ and the crossed pairing costs $\frac{1}{2}(9) + \frac{1}{2}(1) = 5$. The arrow count of Section 7.1.2 added the two route costs without weighting them by mass, so both numbers there were twice these, and the ratio of five between the two pairings is the same either way. The independent coupling of these marginals is the matrix with 1/4 in every entry, at cost $\frac{1}{4}(1 + 9 + 1 + 1) = 3$, between the two.

The linear constraints and the bound of 1 on each entry make $\Pi(a, b)$ a compact convex set. Every continuous function on this set attains its minimum. A row of π can also contain several positive entries, so we can split the mass at one source point across several destinations.

7.2 Monge's formulation

Monge originally asked for an assignment in which each source point has exactly one destination, with no splitting [Monge, 1781].

We write μ for the source law and ν for the target law, now allowed to be continuous distributions on $\mathbb{R}^d$ as well as finite collections of point masses. A map $T : \mathbb{R}^d \to \mathbb{R}^d$ sends each source point x to

the single destination $T(x)$. The map transports μ to ν exactly when the law of $T(X)$, for X drawn from μ, is ν,

$$T_{\#}\mu = \nu.$$

"Applying the map T to a sample X from the source law μ gives a sample with law $T_{\#}\mu$, the pushforward, which must equal the target law ν."

- μ is the source law, a probability distribution on $\mathbb{R}^d$.
- ν is the target law, a probability distribution on $\mathbb{R}^d$.
- T is a measurable map from $\mathbb{R}^d$ to $\mathbb{R}^d$, the destination of each source point.
- $T_{\#}\mu$ is the pushforward of μ under T, defined in Chapter 1 by $T_{\#}\mu(B) = \mu\big(T^{-1}(B)\big)$ for every measurable set B.

Many maps satisfy the constraint, and we want the cheapest one,

$$\min_{T\,:\,T_{\#}\mu=\nu} \int c(x, T(x))\, d\mu(x).$$

"We choose a map T that transports the source law μ to the target law ν and minimizes the average cost $c(x, T(x))$ of moving each source point x to its destination $T(x)$."

For n equal point masses on each side, a mass-preserving map permutes the n indices. We therefore minimize over $n!$ permutations. That special case is the assignment problem. For $n = 2$ the two permutations are the ordered and crossed pairings of Section 7.1.2, for $n = 3$ there are six, and for a batch of $n = 256$ there are 256!, a number with 507 digits. We can enumerate only small cases. Linear structure allows us to solve larger problems.

7.2.1 Splitting source mass

A transport map may fail to exist even in an example with two target points. Take the source and target laws

$$\mu = \delta_0, \qquad \nu = \tfrac{1}{2}\delta_1 + \tfrac{1}{2}\delta_2,$$

where δ_z is the point mass at z. All of the source mass is at 0, and the target has mass $1/2$ at 1 and mass $1/2$ at 2.

A map sends the point 0 to one destination $T(0)$, and every unit of source mass follows it there. The pushforward of a point mass under a map is therefore again a point mass,

$$T_{\#}\delta_0 = \delta_{T(0)} \neq \nu \quad \text{for every } T.$$

"A map T sends all the mass of δ_0, the point mass at zero, to the single destination $T(0)$. Its pushforward $\delta_{T(0)}$ therefore cannot equal the target law ν, which has mass at two distinct points."

No map satisfies the constraint, so the minimization in Monge's formulation is over an empty set and has no solution. To split the mass at 0 we have to send half of it to 1 and half to 2, and a function of x alone cannot record two destinations for the same x. Any formulation that is always solvable has to allow a source point to divide its mass.

7.3 Kantorovich's relaxation

We can split the source mass by replacing the map with the coupling matrix π from Section 7.1.3 [Kantorovich, 1942]. We can write C for the cost matrix, with $C_{ij} = c(x_i, y_j)$ the cost of one unit on the route from x_i to y_j. The total cost of a plan π is the mass on each route times the price of that route, summed over routes, and we minimize it over the couplings,

$$\boxed{\min_{\pi \in \Pi(a,b)} \langle C, \pi \rangle \;=\; \min_{\pi \in \Pi(a,b)} \sum_{i,j} C_{ij}\, \pi_{ij}}$$

"We minimize the total transport cost $\langle C, \pi \rangle$ over couplings π of the source masses a and target masses b. This cost sums the unit cost C_{ij} times the transported mass π_{ij} over every route (i, j)."

- $C \in \mathbb{R}^{n \times m}$ is the cost matrix, with $C_{ij} = c(x_i, y_j)$ fixed by the points and the cost function.
- $\pi \in \Pi(a, b)$ is the decision variable, the plan.
- $\langle C, \pi \rangle = \sum_{i,j} C_{ij} \pi_{ij}$ is the Frobenius inner product, the total cost of the plan.

The objective is linear in π and the constraints are linear equalities and nonnegativity constraints, so this is a linear program in nm variables. A linear program over a nonempty compact set attains its minimum, and $\Pi(a, b)$ is nonempty because it contains the independent coupling, so the Kantorovich problem always has a solution.

A map T corresponds to the plan that puts the whole mass a_i in the single entry of row i for the destination $T(x_i)$, and that plan is a coupling with the same total cost. Every Monge map therefore defines a Kantorovich plan. The Kantorovich minimum is at most the Monge minimum. It also exists in cases such as Section 7.2.1, where no Monge map exists.

7.3.1 Two-by-two transport example

We can solve a transport problem with two points on each side using one variable. Take the source and target points of Section 7.1.2 with unequal masses,

$$x = (0,\ 2), \quad a = (0.6,\ 0.4) \qquad\qquad y = (1,\ 3), \quad b = (0.3,\ 0.7).$$

"The source locations $x = (0, 2)$ carry masses $a = (0.6, 0.4)$, and the target locations $y = (1, 3)$ require masses $b = (0.3, 0.7)$."

The squared distances between the two source points and the two target points fill the cost matrix. The move from 0 to 1 has length 1, the move from 0 to 3 has length 3, the move from 2 to 1 has length 1, and the move from 2 to 3 has length 1, so

$$C = \begin{pmatrix} 1 & 9 \\ 1 & 1 \end{pmatrix}.$$

Three of the four routes cost 1 and the fourth costs 9, so we want as little mass as possible on the route from 0 to 3.

The constraints allow one degree of freedom. We can set $\pi_{11} = r$, the mass sent from 0 to 1. The first row must sum to 0.6, so $\pi_{12} = 0.6 - r$. The first column must sum to 0.3, so $\pi_{21} = 0.3 - r$. The second row must sum to 0.4, which fixes $\pi_{22} = 0.4 - (0.3 - r) = 0.1 + r$, and that entry also makes the second column sum to $(0.6 - r) + (0.1 + r) = 0.7$ as required. The whole feasible set is therefore

$$\pi = \begin{pmatrix} r & 0.6 - r \\ 0.3 - r & 0.1 + r \end{pmatrix}.$$

"Choosing the mass r on the route from 0 to 1 determines every coupling of source masses a and target masses b. The other routes carry $0.6 - r$ from 0 to 3, $0.3 - r$ from 2 to 1, and $0.1 + r$ from 2 to 3."

Nonnegativity of the four entries bounds r. The entry $0.3 - r$ is nonnegative only for $r \leq 0.3$, the entry $0.6 - r$ only for $r \leq 0.6$, and r and $0.1 + r$ are nonnegative for $r \geq 0$. The binding pair is

$$0 \leq r \leq 0.3,$$

so $\Pi(a, b)$ here is a line segment with two endpoints, and the whole linear program is a minimization over one interval.

7.3.2 The cost along the segment

If we multiply each mass by the price of its route and add the four terms, the objective becomes a function of r alone,

$$\langle C, \pi \rangle = r + 9\,(0.6 - r) + (0.3 - r) + (0.1 + r) = 5.8 - 8r.$$

"The total transport cost $\langle C, \pi \rangle$ sums four contributions: mass r at unit cost 1, mass $0.6 - r$ at unit cost 9, and masses $0.3 - r$ and $0.1 + r$ at unit cost 1. Collecting them gives $5.8 - 8r$."

The coefficient of r is $1 - 9 - 1 + 1 = -8$. Moving an extra unit of mass onto the cheap route from 0 to 1 removes one unit from the expensive route from 0 to 3. The two compensating changes in the second row cancel, so we save 8 per unit. The cost decreases strictly in r, so the minimum is at the largest feasible value, $r = 0.3$. Substituting it,

$$\pi^\star = \begin{pmatrix} 0.3 & 0.3 \\ 0 & 0.4 \end{pmatrix} \qquad \langle C, \pi^\star \rangle = 5.8 - 8\,(0.3) = 3.4.$$

"The optimal transport plan $\pi^\star$ sends mass 0.3 from 0 to 1, mass 0.3 from 0 to 3, and mass 0.4 from 2 to 3. It sends no mass from 2 to 1, and its total cost is 3.4."

The target at 3 must receive 0.7 and the source at 2 holds only 0.4, so at least 0.3 has to move from 0 to 3 at price 9, whatever else the plan does. Under the optimal plan, exactly that minimum mass moves along the expensive route. Its first row splits the mass at 0 between two destinations, which is impossible under Monge's formulation.

Our feasible plan has cost 3.4, so the minimum cannot exceed 3.4. A matching lower bound would certify optimality. We need a way to compute such a bound for any transport problem.

7.4 Prices and the dual problem

We can check optimality by scanning a line segment, but larger feasible sets make that search impractical. A lower bound on the cost gives us another way to certify optimality. We can compute such a bound from the problem data using Lagrange multipliers.

We attach a price f_i to each source point and a price g_j to each target point, both real numbers of either sign, in the same units as the cost. Then we add each constraint to the objective, weighted by its own price,

$$L(\pi, f, g) = \langle C, \pi \rangle + \sum_i f_i \Big(a_i - \sum_j \pi_{ij} \Big) + \sum_j g_j \Big(b_j - \sum_i \pi_{ij} \Big).$$

"The Lagrangian $L(\pi, f, g)$ adds two constraint penalties to the transport cost $\langle C, \pi \rangle$: each source price f_i multiplies its row residual $a_i - \sum_j \pi_{ij}$, and each target price g_j multiplies its column residual $b_j - \sum_i \pi_{ij}$."

- $f \in \mathbb{R}^n$ collects the source prices, one per source point.
- $g \in \mathbb{R}^m$ collects the target prices, one per target point.
- the residual $a_i - \sum_j \pi_{ij}$ is zero exactly when row i of π carries the right mass, and likewise for the column residual.

For a feasible π every residual vanishes and $L(\pi, f, g) = \langle C, \pi \rangle$, so the Lagrangian equals the cost on the feasible set for any choice of prices.

7.4.1 Lagrangian coefficients

We expand the two bracketed sums and sort the result by which entry of π each term multiplies. The terms without π are $\sum_i f_i a_i = \langle a, f \rangle$ and $\sum_j g_j b_j = \langle b, g \rangle$. The entry π_{ij} appears once in the cost with coefficient C_{ij}, once in the row sum of row i with coefficient $-f_i$, and once in the column sum of column j with coefficient $-g_j$, so

$$L(\pi, f, g) = \langle a, f \rangle + \langle b, g \rangle + \sum_{i,j} \pi_{ij} (C_{ij} - f_i - g_j).$$

"The Lagrangian L equals the mass-weighted source prices $\langle a, f \rangle = \sum_i a_i f_i$, plus the mass-weighted target prices $\langle b, g \rangle = \sum_j b_j g_j$, plus the transported mass π_{ij} times the remaining route cost $C_{ij} - f_i - g_j$, summed over routes."

A carrier pays f_i to collect one unit at x_i and receives $-g_j$ for delivering it at y_j. The carrier's charge for the route is $f_i + g_j$. The difference $C_{ij} - f_i - g_j$ is the amount by which shipping the unit ourselves undercuts the carrier.

We can now minimize the Lagrangian over all nonnegative π, with the row and column constraints dropped. If one bracket $C_{ij} - f_i - g_j$ is negative, raising that single entry π_{ij} without bound drives L to $-\infty$, so these prices give no finite lower bound. If every bracket is nonnegative, each term in

the sum is minimized at $\pi_{ij} = 0$, and the minimum of L is $\langle a, f\rangle + \langle b, g\rangle$. Keeping only the prices for which the minimum is finite, we are left with

$$\boxed{\max_{f,g} \ \langle a, f\rangle + \langle b, g\rangle \quad \text{subject to} \quad f_i + g_j \le C_{ij} \ \text{ for all } i, j}$$

"We choose source prices f_i and target prices g_j to maximize their mass-weighted total $\sum_i a_i f_i + \sum_j b_j g_j$, subject to each endpoint-price sum $f_i + g_j$ being at most the route cost C_{ij}."

We call $\langle a, f\rangle + \langle b, g\rangle$ the dual value of the prices (f, g), and a pair (f, g) satisfying every one of the nm inequalities is called feasible. The dual has $n + m$ variables against the nm variables of the primal, which is a large reduction when n and m are both large.

7.4.2 Feasible prices and the lower bound

We can take any feasible plan π and any feasible prices (f, g), chosen with no relation to each other. The row sums of π are a and its column sums are b, so we can rewrite the dual value as a sum over routes, and the nm inequalities then bound it by the cost,

$$\langle a, f\rangle + \langle b, g\rangle = \sum_{i,j} \pi_{ij}\,(f_i + g_j) \ \le \ \sum_{i,j} \pi_{ij}\, C_{ij} = \langle C, \pi\rangle.$$

"The dual value equals the sum of each transported mass π_{ij} times its endpoint-price sum $f_i + g_j$. Since that price sum is at most the route cost C_{ij} and the mass is nonnegative, the dual value is at most the total transport cost $\langle C, \pi\rangle$."

The first equality comes from the constraints, applied twice. Summing $\pi_{ij} f_i$ over j, we get $f_i \sum_j \pi_{ij} = f_i a_i$, and summing $\pi_{ij} g_j$ over i, we get $g_j b_j$. The inequality then replaces each $f_i + g_j$ by the larger number C_{ij}, which is valid term by term because the weights π_{ij} are nonnegative.

Theorem 7.2 (Kantorovich duality)**.** *For a finite cost matrix C and probability vectors a and b, the dual value of any feasible pair of prices is at most the total cost of any feasible plan. The maximum of the dual equals the minimum of the primal, and both are attained.*

The inequality we derived is called weak duality. Equality of the primal minimum and dual maximum is called strong duality. It holds for every feasible, bounded linear program, including this one. If a plan and a pair of feasible prices have the same value, both are optimal. We can certify their optimality without searching further over $\Pi(a, b)$.

7.4.3 Prices for the two-by-two example

We can find the prices for the example of Section 7.3.1 in three substitutions. Under $\pi^\star$, mass moves on the three routes $(1,1)$, $(1,2)$ and $(2,2)$, and the route $(2,1)$ is empty. We look for prices that make the inequality tight on the three loaded routes, which is three equations in four unknowns. The prices are determined only up to adding a constant to all of f and subtracting it from all of g, since every sum $f_i + g_j$ is then unchanged, so we may fix one of them. Setting $f_1 = 0$, we solve one route at a time,

$$f_1 = 0, \qquad g_1 = C_{11} - f_1 = 1, \qquad g_2 = C_{12} - f_1 = 9, \qquad f_2 = C_{22} - g_2 = -8.$$

"Fixing source price $f_1 = 0$, we match the price sum to the cost on each loaded route. Route $(1, 1)$ gives target price $g_1 = C_{11} - f_1 = 1$, route $(1, 2)$ gives target price $g_2 = C_{12} - f_1 = 9$, and route $(2, 2)$ gives source price $f_2 = C_{22} - g_2 = 1 - 9 = -8$."

Prices here are accounting quantities and only their differences matter, so a negative price is admissible, and shifting all of f up by 8 and all of g down by 8 produces an equally valid pair.

We now check feasibility on the one route that was not used. Subtracting $f_i + g_j$ from each cost entry, we get the slack matrix

$$(C_{ij} - f_i - g_j)_{ij} = \begin{pmatrix} 0 & 0 \\ 8 & 0 \end{pmatrix} \geq 0.$$

The three loaded routes have zero slack by construction, and the route $(2, 1)$ has slack $1-(-8)-1 = 8$. No entry is negative, so these prices are feasible and Theorem 7.2 applies to them. Their dual value is

$$\langle a, f \rangle + \langle b, g \rangle = 0.6\,(0) + 0.4\,(-8) + 0.3\,(1) + 0.7\,(9) = 3.4.$$

"The dual value weights the source prices 0 and -8 by their masses 0.6 and 0.4, and the target prices 1 and 9 by their masses 0.3 and 0.7. The source contribution -3.2 plus the target contribution 6.6 equals 3.4."

The dual value equals the plan cost 3.4. Weak duality bounds every feasible plan below by this value and every feasible dual value above by it, so both are optimal. This confirms the result of Section 7.3.2 without searching the feasible segment.

7.4.4 Complementary slackness

If we subtract the dual value from the cost of a plan, we recover the sum over routes that appeared in the Lagrangian,

$$\langle C, \pi \rangle - \langle a, f \rangle - \langle b, g \rangle = \sum_{i,j} \pi_{ij} (C_{ij} - f_i - g_j).$$

"The gap between the plan cost $\langle C, \pi \rangle$ and the dual value $\langle a, f \rangle + \langle b, g \rangle$ equals the sum of transported mass π_{ij} times price slack $C_{ij} - f_i - g_j$ over routes (i, j)."

Each term of the sum is a nonnegative mass times a nonnegative slack, so each term is nonnegative, and the gap is zero only when every term is zero. At an optimal pair the gap is zero by Theorem 7.2, so every term must vanish,

$$\pi_{ij} > 0 \implies f_i + g_j = C_{ij}.$$

"If an optimal plan places positive mass π_{ij} on route (i, j), its optimal source and target prices f_i and g_j sum to the route cost C_{ij}, so the slack on that route is zero."

We call this condition complementary slackness. Mass flows only on routes of zero slack. A route with strictly positive slack carries no mass under any optimal plan. In the example the route $(2, 1)$ has slack 8, and $\pi^\star_{21} = 0$ accordingly. If we run the implication in reverse, a candidate set of loaded

routes becomes a set of linear equations for the prices, which is how the simplex method and the auction algorithms for transport proceed.

We can also optimize the prices over functions. On continuous spaces, we replace the vectors f and g with functions on $\mathbb{R}^d$. The constraint becomes $f(x) + g(y) \le c(x, y)$ for all x and y. We then maximize the dual over pairs of functions, which we can parameterize with neural networks [Peyré and Cuturi, 2019, Villani, 2009].

7.5 Distance-based costs

We can use transport to define a distance between distributions, then use that distance as a loss or to describe the geometry of distributions. Fix a distance d on the sample space and an exponent $p \ge 1$, and define

$$W_p(\mu, \nu) = \Big(\min_{\pi \in \Pi(\mu,\nu)} \int d(x, y)^p \, d\pi(x, y) \Big)^{1/p}.$$

"The Wasserstein distance $W_p(\mu, \nu)$ is the p-th root of the minimum average powered distance $d(x, y)^p$, where d measures distance between source point x and target point y, and we minimize over all couplings π of the laws μ and ν."

- $d(x, y)$ is a distance on the sample space, the Euclidean distance in all of our examples.
- $p \ge 1$ is the exponent; $p = 2$ with d Euclidean is the squared cost of Section 7.1.1.
- $\Pi(\mu, \nu)$ is the set of couplings of the two laws, the continuous version of Definition 7.1.

The outer root is there to make W_p a distance. It is symmetric because d is, it vanishes only when $\mu = \nu$, and it satisfies the triangle inequality on the set of laws with finite p-th moments. With the squared Euclidean cost the transport cost itself is W_2^2, so the example of Section 7.3.1 has $W_2^2 = 3.4$ and $W_2 = 1.844$.

7.5.1 Non-crossing on the line

The ordered pairing in Section 7.1.2 was cheaper than the crossed pairing. We can prove the same inequality for any two ordered source points and two ordered target points.

Proposition 7.3 (No crossing under the squared cost)**.** *Let $x_1 < x_2$ be two source points and $y_1 < y_2$ two target points on the real line, each pair carrying equal mass, and take the squared cost. Pairing x_1 with y_1 and x_2 with y_2 costs strictly less than pairing x_1 with y_2 and x_2 with y_1.*

Let Δ be the cost of the ordered pairing minus the cost of the crossed pairing,

$$\Delta = [c(x_1, y_1) + c(x_2, y_2)] - [c(x_1, y_2) + c(x_2, y_1)].$$

Expanding the four squares, every squared term x_1^2, x_2^2, y_1^2, y_2^2 appears once with a plus sign and once with a minus sign, and only the cross terms remain,

$$\begin{aligned} \Delta &= (x_1 - y_1)^2 + (x_2 - y_2)^2 - (x_1 - y_2)^2 - (x_2 - y_1)^2 \\ &= -2\,(x_2 - x_1)(y_2 - y_1) \end{aligned}$$

"The cost difference Δ subtracts the crossed-pair distances squared, $(x_1 - y_2)^2$ and $(x_2 - y_1)^2$, from the ordered-pair distances squared, $(x_1 - y_1)^2$ and $(x_2 - y_2)^2$. Cancelling the squared terms leaves minus twice the source gap $x_2 - x_1$ times the target gap $y_2 - y_1$."

The cross terms are $-2x_1y_1 - 2x_2y_2 + 2x_1y_2 + 2x_2y_1 = -2(x_2 - x_1)(y_2 - y_1)$. Both gaps are positive by assumption, so $\Delta < 0$ and the ordered pairing is strictly cheaper.

An optimal plan on the line therefore contains no crossing pair. If two routes (x_1, y_2) and (x_2, y_1) both carried positive mass with $x_1 < x_2$ and $y_1 < y_2$, moving the smaller of the two masses onto the ordered routes would lower the total cost while preserving every row sum and every column sum, contradicting optimality. The same argument applies to any cost of the form $c(x, y) = h(x - y)$ with h strictly convex, which includes every power $p > 1$ of the distance.

The unique non-crossing coupling on the line pairs equal quantiles of the two laws. We can therefore calculate its transport cost by integrating over quantile levels,

$$W_p(\mu, \nu)^p = \int_0^1 \left|F_\mu^{-1}(q) - F_\nu^{-1}(q)\right|^p \, dq.$$

"The powered Wasserstein distance $W_p(\mu, \nu)^p$ integrates the distance between the source quantile $F_\mu^{-1}(q)$ and target quantile $F_\nu^{-1}(q)$, raised to power p, over quantile levels q from zero to one."

- $F_\mu(x) = \mu\big((-\infty, x]\big)$ is the cumulative distribution function of μ.
- $F_\mu^{-1}(q) = \inf\{x : F_\mu(x) \geq q\}$ is its quantile function, the value below which a fraction q of the mass lies.
- q runs over levels in $[0, 1]$, and the pairing matches the level q of μ with the level q of ν.

With n equally weighted points on each side the quantile function is a step function whose values are the sorted points, so the quantile pairing is exactly the sorted pairing, and computing W_p costs one sort on each side.

7.5.2 Three points on each side

Now take source points at 4, 1 and 6 and target points at 9, 2 and 3, each of the six with mass 1/3. Sorting both sides and pairing in order,

$$(1,\ 4,\ 6) \ \text{ and } \ (2,\ 3,\ 9) \qquad W_2^2 = \tfrac{1}{3}\big[(1-2)^2 + (4-3)^2 + (6-9)^2\big] = \tfrac{11}{3}.$$

"Pairing sorted source points $1, 4, 6$ with sorted target points $2, 3, 9$ gives squared distances $1, 1, 9$. Each pair carries mass $1/3$, so the squared Wasserstein distance is $W_2^2 = 11/3 \approx 3.667$."

We can enumerate the other five permutations directly, and they cost 17/3, 35/3, 45/3, 77/3 and 81/3. The sorted pairing is cheapest by a clear margin, and the worst pairing, which sends $1 \to 9$, $4 \to 3$ and $6 \to 2$, costs more than seven times as much. Even the cheapest pairing contains the single longest move, from 6 to 9 at squared distance 9, and in exchange for it the other two moves have squared distance 1. Sending each source point to its nearest target generally fails to preserve the target masses. Here that rule would send both 4 and 6 to 3, leaving no mass at 9, so it would not define a coupling.

We can certify the sorted pairing with the duality of Section 7.4, without enumerating anything. The cost matrix of the sorted points, with rows indexed by 1, 4, 6 and columns by 2, 3, 9, is

$$C = \begin{pmatrix} 1 & 4 & 64 \\ 4 & 1 & 25 \\ 16 & 9 & 9 \end{pmatrix},$$

and the sorted plan $\pi^\star = \frac{1}{3}I$ loads the three diagonal routes. Complementary slackness requires $f_i + g_j = C_{ij}$ on those three routes. We have three equations in six unknowns, with one feasible solution

$$f = (0,\ 0,\ 8), \qquad g = (1,\ 1,\ 1).$$

The slack matrix $C_{ij} - f_i - g_j$ has rows $(0, 3, 63)$, $(3, 0, 24)$, and $(7, 0, 0)$. Every entry is nonnegative, so the prices are feasible. Their dual value is $\frac{1}{3}(0+0+8) + \frac{1}{3}(1+1+1) = \frac{8}{3} + 1 = \frac{11}{3}$, which equals the cost of $\pi^\star$, and the sorted pairing is optimal. The route $(3, 2)$, which sends 6 to 3, also has zero slack while carrying no mass. Complementary slackness forbids positive mass on a route of positive slack and permits zero mass on a route of zero slack, so the implication runs in one direction only. Here the three row sums and the three column sums still require $\pi_{32} = 0$ on any plan supported by the zero-slack routes, so the optimal plan is unique despite the extra zero.

7.6 Gaussian transport maps

We can compute optimal transport between two one-dimensional Gaussians in closed form. The corresponding formula in d dimensions is used in the Fréchet Inception Distance.

Let $\mu = \mathcal{N}(m_0, \sigma_0^2)$ and $\nu = \mathcal{N}(m_1, \sigma_1^2)$, with m_0, m_1 the means and $\sigma_0, \sigma_1 > 0$ the standard deviations, so that σ_0^2 and σ_1^2 are the variances. Then

$$W_2^2(\mathcal{N}(m_0, \sigma_0^2),\ \mathcal{N}(m_1, \sigma_1^2)) = (m_0 - m_1)^2 + (\sigma_0 - \sigma_1)^2.$$

"The squared Wasserstein distance W_2^2 between the two Gaussian laws equals the squared difference of their means m_0 and m_1, plus the squared difference of their standard deviations σ_0 and σ_1."

The optimal coupling here is carried by a map, and the map is the increasing affine function that matches quantiles,

$$T(x) = m_1 + \frac{\sigma_1}{\sigma_0}\,(x - m_0).$$

"The optimal map T subtracts the source mean m_0 from point x, rescales that displacement by the target-to-source standard-deviation ratio σ_1/σ_0, then adds the target mean m_1."

We obtain this map by standardizing x to $(x - m_0)/\sigma_0$, then applying the target scale and mean. The map T therefore sends the level-q quantile of μ to the same quantile of ν. By Section 7.5.1, this is the optimal pairing on the line.

7.6.1 Transporting $\mathcal{N}(0, 1)$ to $\mathcal{N}(3, 4)$

Here the source is $\mathcal{N}(0, 1)$ and the target is $\mathcal{N}(3, 4)$. The means are 0 and 3, and the standard deviations are 1 and 2, since the target variance is 4. Substituting into the closed form,

$$W_2^2 = (0 - 3)^2 + (1 - 2)^2 = 9 + 1 = 10.$$

The squared difference of the means contributes 9, and the squared difference of the standard deviations contributes 1. Thus $W_2^2 = 10$ and $W_2 = \sqrt{10} = 3.162$. Substituting these means and standard deviations into the map gives

$$T(x) = 3 + 2x,$$

since $m_1 = 3$, $\sigma_1/\sigma_0 = 2$ and $m_0 = 0$. A sample at $x = 1$ lands at 5, a sample at $x = -1$ lands at 1, and a sample at the source mean 0 lands at the target mean 3.

We can also compute the transport cost directly from the map, as a check. The displacement of a sample is $T(X) - X = 3 + 2X - X = 3 + X$, and its mean square is

$$\mathbb{E}\big[(T(X) - X)^2\big] = \mathbb{E}\big[(3 + X)^2\big] = 9 + 1 = 10,$$

"The mean squared displacement under the map T is the expected value of $(3 + X)^2$, where the source sample X is standard normal. The squared mean displacement $3^2 = 9$ plus its variance 1 gives a total cost of 10."

This agrees with the closed form. We obtain the expansion from $\mathbb{E}[(3 + X)^2] = 9 + 6\,\mathbb{E}[X] + \mathbb{E}[X^2]$ with $\mathbb{E}[X] = 0$ and $\mathbb{E}[X^2] = 1$.

In d dimensions, the Gaussian formula uses covariance matrices in place of standard deviations. The Fréchet Inception Distance scores generative image models by computing this d-dimensional W_2^2 between Gaussians fitted to image features.

7.6.2 Brenier's theorem

The Gaussian optimum turned out to be a map, and the example of Section 7.2.1 had no map at all. The difference between the two cases is whether the source law has a density.

Theorem 7.4 (Brenier)**.** *Take the squared cost on $\mathbb{R}^d$. If the source law μ has a density with respect to Lebesgue measure and both μ and ν have finite second moments, then the optimal Kantorovich plan is carried by a map,*

$$T = \nabla\varphi, \quad \varphi \text{ convex}, \qquad T_{\#}\mu = \nu,$$

and T is unique up to μ-null sets.

"The optimal map $T = \nabla\varphi$ sends each source point x to one destination $T(x)$, where φ is convex. Applying this map to the source law μ gives the target law ν, so $T_{\#}\mu = \nu$."

The density hypothesis rules out the counterexample of Section 7.2.1. The law δ_0 puts an atom at a single point and has no density, so the theorem does not apply to it, and no map exists there.

For the Gaussian example, the map $T(x) = 3 + 2x$ is the derivative of $\varphi(x) = 3x + x^2$, which is convex because $\varphi''(x) = 2 > 0$. In one dimension the derivative of a convex function is nondecreasing, so a Brenier map on the line is an increasing map, which is the quantile pairing of Section 7.5.1. The gradient of a convex function is the d-dimensional analogue of sorting, and in that sense Theorem 7.4 extends the no-crossing result to every dimension.

A plan specifies the mass transferred between each source point and target point. For a generative model, we also need trajectories between those endpoints, since we sample by integrating a velocity from the source to the data.

7.7 Dynamic optimal transport

In the Benamou-Brenier formulation we replace the plan by a density ρ_t and a velocity field v_t, for t running from 0 to 1, and measure the transport by the energy the particles expend [Benamou and Brenier, 2000]. We also used a density and a velocity field in Chapter 2, so we can connect this formulation directly to flow matching.

The density must satisfy the continuity equation from Chapter 2 under this velocity field. It must also start at μ and end at ν,

$$\partial_t \rho_t + \nabla \cdot (\rho_t v_t) = 0 \qquad \rho_0 = \mu, \quad \rho_1 = \nu.$$

"The density change $\partial_t \rho_t$ plus the divergence $\nabla \cdot (\rho_t v_t)$ of probability flux is zero at every state and time, where ρ_t is the density and v_t the velocity field. The initial density is the source law μ, and the final density is the target law ν."

- $\rho_t(x)$ is the density at the point x at time t, with $\int \rho_t(x)\, dx = 1$ for every t.
- $v_t(x) \in \mathbb{R}^d$ is the velocity of the particles passing through x at time t.
- $\rho_t v_t$ is the probability flux, the mass per unit time crossing a unit area at x.

Among all pairs (ρ, v) obeying those conditions we minimize the kinetic action,

$$\min_{\rho, v} \int_0^1 \int \rho_t(x)\, \|v_t(x)\|^2 \, dx\, dt.$$

"The kinetic action integrates the density $\rho_t(x)$ times the squared particle speed $\|v_t(x)\|^2$ over positions x and times t from zero to one."

The integrand is twice the kinetic energy density of a unit-mass fluid, so the action is the total energy of the transport. Many pairs (ρ, v) move μ to ν, and they differ in the energy their particles expend.

7.7.1 One particle

Let's follow a single particle along its trajectory $x(t)$, with velocity $\dot{x}(t)$. Its net displacement over the unit interval is the integral of its velocity,

$$x(1) - x(0) = \int_0^1 \dot{x}(t)\, dt.$$

Taking the squared norm of both sides and applying the Cauchy-Schwarz inequality to the integral against the constant function 1 on an interval of length 1,

$$\|x(1) - x(0)\|^2 = \Big\| \int_0^1 \dot{x}(t)\, dt \Big\|^2 \;\le\; \int_0^1 \|\dot{x}(t)\|^2 \, dt.$$

"The squared displacement from initial position $x(0)$ to final position $x(1)$ equals the squared norm of the integrated velocity $\dot{x}(t)$. Cauchy-Schwarz bounds it by the integral of the squared speed over the same unit interval."

A particle's energy is therefore at least its squared displacement. Equality holds exactly when the velocity is constant in time, by the equality condition of Cauchy-Schwarz. Remaining still and then moving faster costs more than moving at a constant velocity.

A particle that moves from 0 to 2 at the constant speed 2 has action $\int_0^1 2^2\,dt = 4$, which equals the squared displacement $2^2 = 4$. A particle that stays at 0 until $t = 1/2$ and then covers the same distance at speed 4 has action $\int_{1/2}^1 4^2\,dt = 16 \times 0.5 = 8$, twice as much, for the same two endpoints.

7.7.2 Averaging over the particles

We now average the one-particle inequality over the whole population. Each particle starts at x_0 and ends at x_1. The starting points have law $\rho_0 = \mu$, and the endpoints have law $\rho_1 = \nu$. Their joint law is therefore a coupling π of μ and ν. Averaging the inequality and then minimizing over couplings,

$$\int_0^1 \int \rho_t \, \|v_t\|^2 \, dx\, dt \;\geq\; \int \|x_1 - x_0\|^2 \, d\pi(x_0, x_1) \;\geq\; W_2^2(\mu, \nu).$$

"The action of a density ρ and velocity field v is at least the average squared displacement from starting point x_0 to endpoint x_1 under their coupling π. That average is at least the minimum transport cost $W_2^2(\mu, \nu)$."

The first inequality is the one-particle bound averaged under π. The second holds because π is one coupling of μ and ν and W_2^2 is the minimum over all of them. No pair (ρ, v) can have an action below $W_2^2(\mu, \nu)$.

7.7.3 Straight lines and the bound

We can draw the endpoint pair (x_0, x_1) from the optimal coupling and move each particle along the straight segment between them,

$$x(t) = (1 - t)\, x_0 + t\, x_1 \qquad \dot{x}(t) = x_1 - x_0.$$

"The particle position $x(t)$ interpolates linearly from starting point x_0 to endpoint x_1, with constant velocity $\dot{x}(t) = x_1 - x_0$ over the unit interval."

The velocity is constant in time, so Cauchy-Schwarz is tight and the first inequality is an equality for every particle. The coupling is optimal, so the second inequality is an equality as well. The density ρ_t of the point $x(t)$ then satisfies the continuity equation with the velocity field obtained by assigning to each location the common velocity of the particles passing through it, and its action equals $W_2^2(\mu, \nu)$. Combining this with Section 7.7.2,

Proposition 7.5 (Benamou-Brenier)**.** *For laws μ and ν with finite second moments,*

$$\boxed{\min_{\rho, v} \int_0^1 \int \rho_t \, \|v_t\|^2 \, dx\, dt \;=\; W_2^2(\mu, \nu)}$$

where the minimum runs over pairs (ρ, v) satisfying the continuity equation with $\rho_0 = \mu$ and $\rho_1 = \nu$.

"Minimizing kinetic action over density and velocity pairs (ρ, v) that transport source law μ to target law ν gives the squared Wasserstein distance $W_2^2(\mu, \nu)$."

The static and dynamic problems have the same minimum value. At the dynamic minimum, each particle moves in a straight line at constant speed between optimally paired endpoints. The least-energy velocity field therefore comes from straight-line interpolations under the optimal coupling. These are the conditional paths used in flow matching, now with an optimized pairing.

7.7.4 Displacement interpolation

At time t, each particle is at $(1-t)x_0 + tx_1$. The law ρ_t of these positions is the displacement interpolation between μ and ν [McCann, 1997].

We can carry the Gaussian example through in closed form. With $\mu = \mathcal{N}(0, 1)$, $\nu = \mathcal{N}(3, 4)$ and the optimal map $X_1 = 3 + 2X_0$ of Section 7.6.1,

$$X_t = (1-t)\,X_0 + t\,(3 + 2X_0) = 3t + (1+t)\,X_0 \qquad \rho_t = \mathcal{N}\big(3t,\ (1+t)^2\big).$$

"The interpolated sample X_t weights the source sample X_0 by $1-t$ and its mapped endpoint $3+2X_0$ by t. This gives $3t + (1+t)X_0$, so its law ρ_t is Gaussian with mean $3t$ and standard deviation $1+t$, since X_0 is standard normal."

The coefficient of X_0 is $(1-t) + 2t = 1 + t$ and the constant is $3t$. Every ρ_t is a single Gaussian, with the mean sliding from 0 to 3 and the standard deviation from 1 to 2, both at constant rate. At $t = 1/2$ the law is $\mathcal{N}(1.5, 2.25)$, a Gaussian of mean 1.5 and standard deviation 1.5.

We can compare this displacement interpolation with a mixture of the two endpoint densities, each weighted by $1/2$. The mixture $\frac{1}{2}\mathcal{N}(0, 1) + \frac{1}{2}\mathcal{N}(3, 4)$ peaks at $x = 0.13$, near the source mean, and has a long shoulder toward 3. It is non-Gaussian. Displacement interpolation moves the mass between the endpoints. Density mixing decreases the source component and increases the target component, so the two paths differ at every interior time (Figure 7.2).

The displacement path also moves at constant speed in the W_2 geometry,

$$W_2(\rho_s, \rho_t) = |t - s| \cdot W_2(\mu, \nu),$$

"The Wasserstein distance $W_2(\rho_s, \rho_t)$ between the intermediate laws at times s and t equals their time separation $|t - s|$ times the full source-to-target distance $W_2(\mu, \nu)$."

The path is therefore a geodesic. To check this on the Gaussian example, take $s = 0$ and $t = 1/2$. The mean moves from 0 to 1.5 and the standard deviation from 1 to 1.5, and substituting into the closed form of Section 7.6 we obtain $W_2(\rho_0, \rho_{1/2}) = \sqrt{1.5^2 + 0.5^2} = \sqrt{2.5} = 1.581$, half of $W_2(\mu, \nu) = 3.162$ as the identity requires.

7.7.5 Displacement velocity field

In the dynamic formulation we need a velocity field, and in the Gaussian example we have one in closed form. Each particle obeys $X_t = 3t + (1+t)X_0$, so its velocity is $\dot{X}_t = 3 + X_0$, constant in

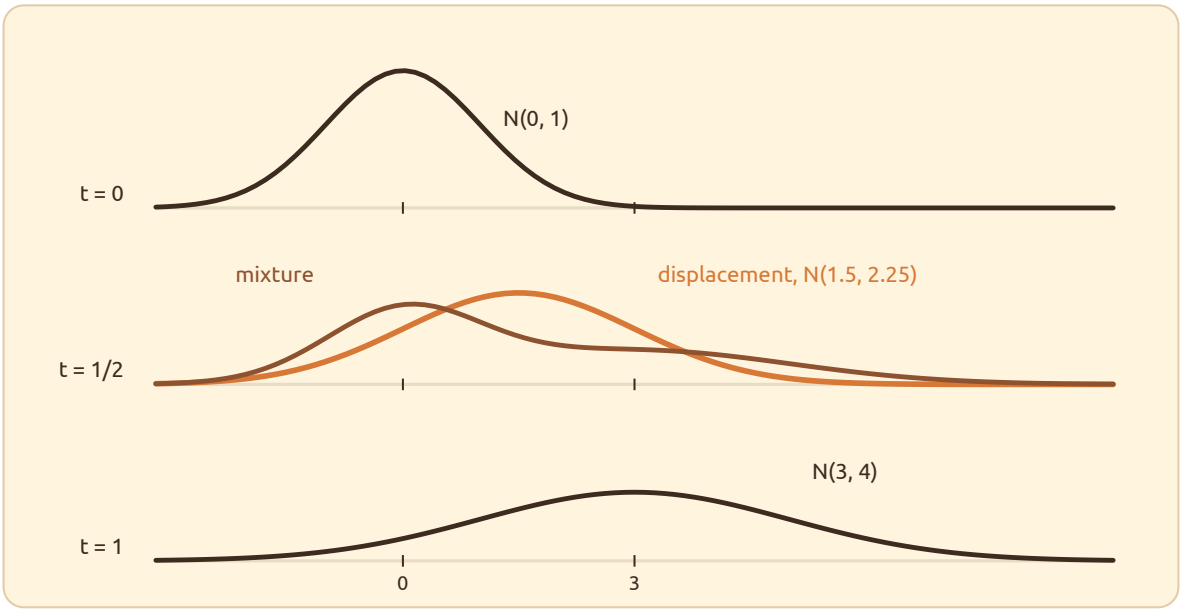


Figure 7.2: The source $\mathcal{N}(0,1)$ at $t=0$ in the top row, the target $\mathcal{N}(3,4)$ at $t=1$ in the bottom row, and two candidate densities at $t=1/2$ in the middle row, all plotted against the state on the horizontal axis with tick marks at 0 and 3. The light curve is the displacement interpolation, the single Gaussian $\mathcal{N}(1.5, 2.25)$ of mean 1.5 and standard deviation 1.5. The dark curve is the mixture of the two end densities with weights $1/2$ and $1/2$, which retains a peak near the source mean and a long shoulder toward the target mean.

time, as in Section 7.7.3. If we solve the trajectory for $X_0 = (X_t - 3t)/(1+t)$ and substitute, that velocity becomes a function of the current position alone,

$$v_t(x) = 3 + \frac{x - 3t}{1+t}.$$

"The velocity $v_t(x)$ at position x and time t equals the mean velocity 3, plus the displacement $x - 3t$ from the current mean divided by the current standard deviation $1 + t$."

The constant 3 translates the whole density at the speed of its mean, which covers the distance from 0 to 3 over the unit interval. The second term is proportional to the signed distance from the mean, so it stretches the density, and the standard deviation moves from 1 to 2 at unit rate. A particle at the mean, where $x = 3t$, moves at speed exactly 3, and a particle one standard deviation above the mean, at $x = 3t + (1+t)$, moves at speed 4.

If we evaluate the action density at a fixed time, we can confirm Proposition 7.5 on this example. At any t,

$$\int \rho_t(x)\, v_t(x)^2 \, dx = \mathbb{E}\big[(3 + X_0)^2\big] = 10,$$

the same number at every time because no particle changes speed, and integrating the constant 10 over t from 0 to 1 returns $W_2^2(\mu, \nu) = 10$.

This velocity field is also the regression target of flow matching on this pair of laws. Training on optimal pairs (X_0, X_1) and their straight interpolations gives v_t. A single Euler step of size 1

from source sample x_0 gives $x_0 + v_0(x_0) = x_0 + 3 + x_0 = 3 + 2x_0$, which is the Brenier map T of Section 7.6.1. One step of the sampler reproduces the exact transport, because the trajectories are straight.

7.8 Optimal transport inside flow matching

In Chapter 2 we regressed a network velocity onto the conditional velocity $x_1 - x_0$ of a straight path between a source sample and a data sample [Lipman et al., 2023]. Nothing in that derivation required the pair to be drawn independently. The loss is defined for any joint law whose two marginals are p_0 and p_1, and such a joint law is a coupling. If we replace the independent coupling by the optimal one, the training pairs change and the loss, the network and the sampler stay as they are.

Written out, the training loss is

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\ (x_0,x_1)\sim\pi} \left\| v_t^\theta\big((1-t)\,x_0 + t\,x_1\big) - (x_1 - x_0) \right\|^2,$$

"The flow-matching loss $\mathcal{L}(\theta)$ averages the squared error between predicted velocity $v_t^\theta((1-t)x_0+tx_1)$ at the interpolated point and target velocity $x_1 - x_0$, over uniform times $t \in [0,1]$ and endpoint pairs (x_0, x_1) from coupling π."

Only π changes in this loss. The network predicts a velocity, and the regression target is the straight-line displacement. The minimizer is the posterior average of the conditional velocities. We sample by integrating v_t^θ from $t = 0$ to $t = 1$.

Under the independent coupling two straight paths can meet at a point x at a time t with different velocities, and the marginal velocity there is the posterior average of the two, which points in neither direction. Under an optimal coupling for the squared cost no two paths cross at the same time, by the argument of Section 7.5.1, so the marginal velocity at each point is the conditional velocity of the single path passing through it. The learned trajectories are then straight lines, and a straight trajectory is integrated exactly by one Euler step. In Chapter 5 we reached straight trajectories by a different route, through rectification.

7.8.1 The cost of a batch coupling

We cannot compute the optimal coupling between p_0 and p_1, since neither law is known in closed form. Each training step has a batch of n source samples and m data samples, and we solve the transport problem on that batch instead, with uniform masses $1/n$ and $1/m$ [Tong et al., 2024]. Counting the pieces of the batch problem,

$$\pi \in \mathbb{R}_+^{n\times m}, \qquad nm \text{ unknowns}, \qquad n + m \text{ equality constraints}.$$

For a batch with $n = m = 256$, the plan has 65,536 unknown entries and 512 equality constraints. Solving the linear program exactly costs on the order of n^3 operations, about 1.7×10^7 for $n = 256$, and we repeat the solve for every batch of training.

The solution can also change discontinuously when the samples move. A linear program attains its minimum at a vertex of the feasible set. Each vertex of $\Pi(a, b)$ has at most $n + m - 1$ nonzero

entries. The optimal plan for batches of 256 source and 256 target samples therefore uses at most 511 of its 65,536 routes. The identity of those 511 routes can change abruptly when one sample moves slightly, because the vertex attaining the minimum can switch. The pairing is a piecewise constant function of the batch, with jumps.

Let's take a batch of two source samples at 0 and 2 and two data samples at 1 and 3, all with mass $1/2$. Under the independent coupling each of the four pairs has weight $1/4$, and the average squared conditional velocity is

$$\tfrac{1}{4}\big[(0-1)^2+(0-3)^2+(2-1)^2+(2-3)^2\big]=\tfrac{12}{4}=3.$$

Under the optimal coupling, which pairs 0 with 1 and 2 with 3 with weight $1/2$ each, the same average is $\frac{1}{2}[1+1]=1$. The regression targets are three times smaller in mean square, and by Proposition 7.5 that average squared displacement is the kinetic action the sampler has to expend.

Solving the linear program exactly incurs cubic cost and can produce discontinuous pairings. We can reduce these difficulties by smoothing the objective.

7.9 Entropic transport

Spreading mass over many routes makes a plan more stable under perturbations than concentrating it on a few routes. We can encourage this spread using the entropy of the plan,

$$H(\pi)=-\sum_{i,j}\pi_{ij}\log\pi_{ij}.$$

"The plan entropy $H(\pi)$ is the negative sum of each transported mass π_{ij} times its logarithm over routes (i,j), with empty routes contributing zero by the convention $0\log 0=0$."

Entropy is larger when the mass is more evenly spread across routes and smaller when it concentrates. In the running example the optimal plan $\pi^\star$ has entropy

$$H(\pi^\star)=-\big[0.3\log 0.3+0.3\log 0.3+0.4\log 0.4\big]=1.089,$$

with the empty route contributing nothing, and the independent coupling a_ib_j, whose entries are 0.18, 0.42, 0.12 and 0.28, has entropy 1.284. The independent coupling has the largest entropy of any coupling of a and b, equal to $H(a)+H(b)$.

Subtracting ε times the entropy from the cost defines the entropic transport problem,

$$\min_{\pi\in\Pi(a,b)}\ \langle C,\pi\rangle-\varepsilon\,H(\pi).$$

"We choose a coupling π of source masses a and target masses b to minimize transport cost $\langle C,\pi\rangle$ minus plan entropy $H(\pi)$ weighted by regularization strength $\varepsilon>0$."

- $\varepsilon>0$ is the regularization strength, in the units of the cost, and it fixes how much spreading is worth.
- $H(\pi)$ is the entropy of the plan, in nats.

The function $\pi \mapsto -H(\pi)$ is strictly convex on the nonnegative orthant, and the cost is linear, so the objective is strictly convex and the minimizer is unique for every $\varepsilon > 0$. The vertex structure disappears along with the ties, and the minimizer varies smoothly with the problem data. A larger ε spreads the minimizer further.

7.9.1 Relative entropy from a positive matrix

We can derive an algorithm by rewriting the entropy term as a divergence from a fixed matrix. For a matrix ξ with strictly positive entries, define

$$D_{\mathrm{KL}}(\pi \,\|\, \xi) = \sum_{i,j} \pi_{ij} \log \frac{\pi_{ij}}{\xi_{ij}}.$$

"The relative entropy $D_{\mathrm{KL}}(\pi \,\|\, \xi)$ sums the plan mass π_{ij} times the logarithm of its ratio to the positive reference entry ξ_{ij}, over routes (i, j)."

When ξ is itself a probability matrix this is the Kullback-Leibler divergence of Chapter 1, nonnegative and zero only at $\pi = \xi$. The same definition applies to any positive matrix ξ.

Take ξ to be the independent coupling $a \otimes b$, whose entries are $a_i b_j$. Splitting the logarithm of the ratio into three pieces,

$$\begin{aligned} D_{\mathrm{KL}}(\pi \,\|\, a \otimes b) &= \sum_{i,j} \pi_{ij} \log \pi_{ij} - \sum_{i,j} \pi_{ij} \log a_i - \sum_{i,j} \pi_{ij} \log b_j \\ &= -H(\pi) + H(a) + H(b) \end{aligned}$$

"Splitting the logarithm in $D_{\mathrm{KL}}(\pi \,\|\, a \otimes b)$ gives the sum of mass-weighted log plan entries $\pi_{ij} \log \pi_{ij}$, minus the sums of mass-weighted log source and target probabilities $\pi_{ij} \log a_i$ and $\pi_{ij} \log b_j$. The result is minus plan entropy $H(\pi)$, plus source entropy $H(a)$ and target entropy $H(b)$."

The first sum is $-H(\pi)$ by definition. In the second, summing π_{ij} over j produces the row sum a_i, so the sum is $\sum_i a_i \log a_i = -H(a)$, and the minus sign in front turns it into $+H(a)$. The third behaves the same way with the columns. Both $H(a)$ and $H(b)$ depend only on the marginals and are constant over $\Pi(a, b)$, so subtracting $\varepsilon H(\pi)$ from the cost and adding $\varepsilon D_{\mathrm{KL}}(\pi \,\|\, a \otimes b)$ to it differ by a constant and have the same minimizer.

In the running example $H(a) = 0.673$, $H(b) = 0.611$, and their sum 1.284 is the entropy of the independent coupling as claimed. The relative entropy of the exact optimal plan from the independent coupling is then $1.284 - 1.089 = 0.195$ nats, so $\pi^\star$ is a modest distance from independence in this geometry.

7.9.2 The Lagrangian and the scaling form

The entropic objective is smooth and strictly convex, so we can find its minimizer by setting a derivative to zero. We then attach a multiplier f_i to each row constraint and g_j to each column constraint,

$$\mathcal{E}(\pi, f, g) = \langle C, \pi \rangle - \varepsilon H(\pi) - \sum_i f_i \Big(\sum_j \pi_{ij} - a_i \Big) - \sum_j g_j \Big(\sum_i \pi_{ij} - b_j \Big).$$

"The entropic Lagrangian $\mathcal{E}(\pi, f, g)$ subtracts constraint terms from transport cost minus weighted entropy. Each row multiplier f_i weights the row excess $\sum_j \pi_{ij} - a_i$, and each column multiplier g_j weights the column excess $\sum_i \pi_{ij} - b_j$."

The multipliers play the role the prices played in Section 7.4, with the sign convention flipped so that each residual measures excess mass. Differentiating with respect to one entry π_{ij} gives contributions from one cost term, one entropy term, and two constraint terms,

$$\frac{\partial \mathcal{E}}{\partial \pi_{ij}} = C_{ij} + \varepsilon(\log \pi_{ij} + 1) - f_i - g_j = 0.$$

"Differentiating the Lagrangian $\mathcal{E}$ with respect to route mass π_{ij} gives route cost C_{ij}, plus the entropy derivative $\varepsilon(\log \pi_{ij} + 1)$, minus row and column multipliers f_i and g_j. This derivative is zero at the minimum."

The $\log \pi_{ij} + 1$ comes from differentiating $-H(\pi) = \sum \pi_{ij} \log \pi_{ij}$ in the single variable π_{ij}. Every entry of the minimizer is therefore strictly positive, since the derivative tends to $-\infty$ as $\pi_{ij} \to 0$ and no entry can lie at the boundary.

Solving for the logarithm,

$$\varepsilon \log \pi_{ij} = f_i + g_j - C_{ij} - \varepsilon,$$

and dividing by ε and exponentiating splits the right-hand side into three factors, one depending only on i, one only on j, and one on the cost,

$$\pi_{ij} = \underbrace{e^{\frac{f_i}{\varepsilon} - \frac{1}{2}}}_{u_i} \; \underbrace{e^{-C_{ij}/\varepsilon}}_{K_{ij}} \; \underbrace{e^{\frac{g_j}{\varepsilon} - \frac{1}{2}}}_{v_j}.$$

"The route mass π_{ij} equals source scaling u_i times kernel entry K_{ij} times target scaling v_j. These factors are the exponentials of the rescaled source price $f_i/\varepsilon - 1/2$, negative rescaled cost $-C_{ij}/\varepsilon$, and rescaled target price $g_j/\varepsilon - 1/2$, respectively."

The constant $-\varepsilon$ in the exponent has been split evenly between the two scalings, which is a choice of normalization and has no effect on the product. In matrix notation,

$$\boxed{\pi^\varepsilon = \operatorname{diag}(u)\, K\, \operatorname{diag}(v) \qquad K = \exp(-C/\varepsilon)}$$

"We obtain the entropic plan π^ε by multiplying row i of the Gibbs kernel K by source scaling u_i and column j by target scaling v_j. The kernel is the entrywise exponential of the negative cost matrix $-C$ divided by regularization strength ε."

- $K = \exp(-C/\varepsilon)$ is the Gibbs kernel, taken entrywise, with every entry strictly positive and larger entries on the cheaper routes.
- $u \in \mathbb{R}^n_+$ is the vector of source scalings, one per row.
- $v \in \mathbb{R}^m_+$ is the vector of target scalings, one per column.

The vectors u and v are scaling factors with no time index. We distinguish them from the velocity field v_t of Section 7.7 by its subscript t.

We now solve for the $n + m$ entries of u and v. The cost already determines the kernel K, so we no longer need nm separate unknowns for π. For the batch of Section 7.8.1, that is 512 numbers in place of 65,536. The entropic problem came into machine learning through Sinkhorn Distances [Cuturi, 2013], as a transport cost that is differentiable in the problem data and therefore usable inside a learning objective.

7.10 Sinkhorn's algorithm

We can solve for u and v by imposing the marginal constraints. Writing out the row and column sums of $\operatorname{diag}(u) K \operatorname{diag}(v)$ gives

$$u \odot (Kv) = a \qquad v \odot (K^\top u) = b.$$

"The source constraint requires the entrywise product $u \odot (Kv)$ of the row scalings and scaled kernel row sums to equal source masses a. The target constraint requires the corresponding column product $v \odot (K^\top u)$ to equal target masses b."

Row i sums to $u_i \sum_j K_{ij} v_j = u_i (Kv)_i$. Similarly, column j sums to $v_j \sum_i u_i K_{ij} = v_j (K^\top u)_j$. We can solve each equation for its own scaling with the other one held fixed,

$$u \;\leftarrow\; a \oslash (Kv) \qquad v \;\leftarrow\; b \oslash (K^\top u),$$

"Holding target scalings v fixed, we update source scalings u by dividing source masses a entrywise by Kv. Holding the updated u fixed, we update v by dividing target masses b entrywise by $K^\top u$."

Here $\oslash$ denotes entrywise division. After the first update the row sums are exactly a, and after the second the column sums are exactly b and the row sums have moved away from a again. We apply the two updates alternately, and the pair of them is one sweep. This is Sinkhorn's algorithm [Sinkhorn, 1964], and each sweep costs two matrix-vector products, on the order of nm operations, against the n^3 of an exact solve.

7.10.1 Sinkhorn updates as projections

We can explain convergence using projections onto convex sets. If we substitute $\log K_{ij} = -C_{ij}/\varepsilon$ into the relative entropy of a plan from the kernel, we get

$$\begin{aligned} \varepsilon\, D_{\mathrm{KL}}(\pi \,\|\, K) &= \varepsilon \sum_{i,j} \pi_{ij} \log \pi_{ij} \;-\; \varepsilon \sum_{i,j} \pi_{ij} \log K_{ij} \\ &= \langle C, \pi \rangle - \varepsilon\, H(\pi) \end{aligned}$$

"Multiplying relative entropy $D_{\mathrm{KL}}(\pi \,\|\, K)$ by regularization strength ε gives two terms: minus weighted plan entropy $\varepsilon H(\pi)$, and minus the mass-weighted kernel log entries $\varepsilon \sum_{i,j} \pi_{ij} \log K_{ij}$. The latter equals transport cost $\langle C, \pi \rangle$, giving the entropic objective."

For the second term we have $-\varepsilon \log K_{ij} = C_{ij}$, so $-\varepsilon \sum \pi_{ij} \log K_{ij} = \sum \pi_{ij} C_{ij} = \langle C, \pi \rangle$. The entropic plan is therefore the coupling of a and b closest to K in relative entropy,

$$\pi^{\varepsilon} = \arg \min_{\pi \in \Pi(a,b)} D_{\mathrm{KL}}(\pi \,\|\, K)\,.$$

Each Sinkhorn update is itself a projection of the same kind, onto one of the two constraint sets alone. If we minimize $D_{\mathrm{KL}}(\pi \,\|\, \xi)$ over the matrices with row sums a, with no condition imposed on the columns, the Lagrangian is $\sum_{i,j} \pi_{ij} \log(\pi_{ij}/\xi_{ij}) - \sum_i \lambda_i (\sum_j \pi_{ij} - a_i)$, whose derivative in the entry π_{ij} is $\log(\pi_{ij}/\xi_{ij}) + 1 - \lambda_i$. Setting that derivative to zero,

$$\pi_{ij} = \xi_{ij}\, e^{\lambda_i - 1},$$

so the minimizer is ξ with each row multiplied by a factor depending on the row alone, and once we impose the row sums, that factor is fixed at $a_i / \sum_j \xi_{ij}$. The update $u \leftarrow a \oslash (Kv)$ performs exactly this rescaling, and the update of v performs it on the columns. Sinkhorn's algorithm is therefore alternating relative-entropy projection onto two convex sets, started at K, and it converges to the unique point of their intersection (Figure 7.3).

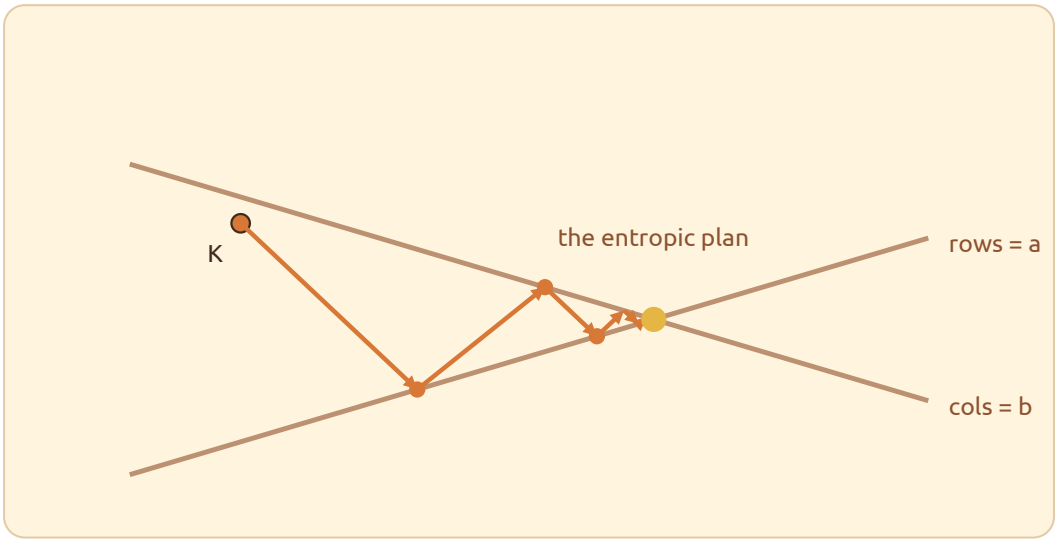


Figure 7.3: The two constraint sets of the entropic problem, drawn as lines: the matrices whose row sums equal a, and the matrices whose column sums equal b. The iteration starts at the Gibbs kernel K, and each update is a relative-entropy projection onto one of the two sets, alternating between them. The steps shorten and converge to the entropic plan at the intersection, which is the unique coupling of a and b closest to K.

7.10.2 Sinkhorn example at $\varepsilon = 2$

Let's run the algorithm on the two-by-two example of Section 7.3.1, with $a = (0.6, 0.4)$, $b = (0.3, 0.7)$ and the cost matrix C whose entries are 1, 9, 1, 1. At $\varepsilon = 2$ the kernel is

$$K = \exp(-C/2) = \begin{pmatrix} 0.606531 & 0.011109 \\ 0.606531 & 0.606531 \end{pmatrix}.$$

Three of the routes cost 1, with kernel entry $e^{-1/2} = 0.606531$, and the expensive route costs 9, with kernel entry $e^{-9/2} = 0.011109$. The kernel entries are about 55 times larger on the cheap routes

than on the expensive one, so the bias toward cheap routes is already built into K before we apply any scaling.

We can start from $v = (1, 1)$ and apply the first update. The vector Kv holds the row sums of K, and u is a divided by it entrywise,

$$Kv = \big(0.606531 + 0.011109,\ \ 0.606531 + 0.606531\big) = (0.617640,\ 1.213061)$$

$$u = a \oslash (Kv) = \Big(\frac{0.6}{0.617640},\ \frac{0.4}{1.213061}\Big) = (0.971440,\ 0.329744)$$

"With target scalings $v = (1, 1)$, the product Kv gives kernel row sums 0.617640 and 1.213061. Dividing source masses 0.6 and 0.4 by these sums gives source scalings $u = (0.971440, 0.329744)$."

The second row of K sums to more than the first, because both of its routes are cheap, so the second scaling comes out smaller. Next, we can update v with this u,

$$K^\top u = \big(0.606531\,(0.971440 + 0.329744),\ \ 0.011109\,(0.971440) + 0.606531\,(0.329744)\big)$$

$$= (0.789208,\ 0.210792)$$

$$v = b \oslash (K^\top u) = \Big(\frac{0.3}{0.789208},\ \frac{0.7}{0.210792}\Big) = (0.38013,\ 3.32081)$$

"The first scaled column sum in $K^\top u$ is 0.606531 times the sum of the two source scalings, giving 0.789208. The second is $0.011109 \times 0.971440 + 0.606531 \times 0.329744 = 0.210792$. Dividing target masses 0.3 and 0.7 by these sums gives target scalings $v = (0.38013, 3.32081)$."

The second target requires more mass, 0.7, and its kernel column contains a small entry. It therefore needs a larger scaling factor. Multiplying out $\operatorname{diag}(u)K\operatorname{diag}(v)$ after this first sweep,

$$\operatorname{diag}(u)\,K\,\operatorname{diag}(v) = \begin{pmatrix} 0.2240 & 0.0358 \\ 0.0760 & 0.6642 \end{pmatrix}.$$

Its column sums are $0.2240 + 0.0760 = 0.3$ and $0.0358 + 0.6642 = 0.7$, exactly b, because the column update was the last one applied. Its row sums are 0.2598 and 0.7402, which are far from the required 0.6 and 0.4. If we apply the next update of u, the row sums are restored to 0.6 and 0.4 and the column sums move to 0.558 and 0.442, and the alternation continues.

7.10.3 Convergence

We track the total marginal error after each sweep, defined as the sum of the absolute differences between the row sums and a plus the sum of the absolute differences between the column sums and b. Over the first five sweeps it falls as

$$0.680,\quad 0.382,\quad 0.155,\quad 0.047,\quad 0.012.$$

The ratio between successive errors shrinks from about 0.56 to about 0.26, which is the linear convergence that alternating projections exhibit on two sets meeting at a positive angle. The error drops below 10^{-9} after 17 sweeps, at a cost of order nm per sweep.

At convergence the plan is

$$\pi^{\varepsilon} = \begin{pmatrix} 0.293122 & 0.306878 \\ 0.006878 & 0.393122 \end{pmatrix}.$$

"The entropic plan π^{ε} sends mass 0.293122 from 0 to 1, mass 0.306878 from 0 to 3, mass 0.006878 from 2 to 1, and mass 0.393122 from 2 to 3."

The route $(2, 1)$, which has no mass under the exact plan $\pi^{\star}$, now holds 0.006878. Every entry is strictly positive, as it must be by the derivative argument of Section 7.9.2, and the four entries sum to 1. Comparing the two total costs,

$$\langle C, \pi^{\varepsilon} \rangle = 3.45502 \qquad \langle C, \pi^{\star} \rangle = 3.4.$$

Entropy regularization increases the transport cost by 0.055, or 1.6 percent. The resulting plan is unique, strictly positive, and smooth in the problem data. Computing it costs order nm operations per sweep.

7.10.4 The position of the entropic plan

Let's compare the three couplings by cost and entropy. The entropic plan falls between the exact optimum and the independent coupling on both quantities. The exact optimum $\pi^{\star}$ has cost 3.4 and entropy 1.089. The independent coupling $a \otimes b$ has cost 4.36 and entropy 1.284. The entropic plan at $\varepsilon = 2$ has cost 3.455 and entropy 1.124, higher than $\pi^{\star}$ on both counts and lower than $a \otimes b$ on both. Evaluating the regularized objective $\langle C, \pi \rangle - 2H(\pi)$ at the three,

$$1.208 \;\; (\pi^{\varepsilon}), \qquad 1.222 \;\; (\pi^{\star}), \qquad 1.792 \;\; (a \otimes b),$$

The entropic plan π^{ε} has the smallest objective value, as required for the unique minimizer of this strictly convex problem. Measured from independence, $D_{\mathrm{KL}}(\pi^{\varepsilon} \,\|\, a \otimes b) = 0.160$ against $D_{\mathrm{KL}}(\pi^{\star} \,\|\, a \otimes b) = 0.195$, so the regularized plan is closer than the exact one to the coupling that Chapters 2 through 6 used.

We can recover the multipliers from the converged scalings. Inverting $u_i = e^{f_i/\varepsilon - 1/2}$ for $f_i = \varepsilon(\log u_i + \frac{1}{2})$, and v_j for g_j in the same way, the converged $u = (3.986,\ 0.0935)$ and $v = (0.1212,\ 6.930)$ correspond to

$$f = (3.766,\ -3.739), \qquad g = (-3.220,\ 4.872).$$

These are the entropic counterparts of the prices of Section 7.4.3, and the hard constraint $f_i + g_j \le C_{ij}$ of the exact dual no longer holds. If we rearrange $\varepsilon \log \pi_{ij} = f_i + g_j - C_{ij} - \varepsilon$, each slack is expressed through the mass on its route,

$$C_{ij} - f_i - g_j = -\varepsilon\big(\log \pi^{\varepsilon}_{ij} + 1\big),$$

which is negative on any route carrying more than $e^{-1} = 0.368$ of the mass. Here the four sums $f_i + g_j$ are 0.546, 8.637, -6.959 and 1.133 against the costs 1, 9, 1 and 1, and the route $(2, 2)$, which holds 0.393, overshoots its cost by 0.133. Complementary slackness has softened into a smooth relation, in which a route of small slack has a large mass, and the empty routes of Section 7.4.4 become routes of large positive slack and exponentially small mass.

ε	$\langle C, \pi^\varepsilon \rangle$	π^ε_{21}	sweeps to 10^{-9}
10	4.0540165	8.18×10^{-2}	7
2	3.4550204	6.88×10^{-3}	17
0.5	3.4000004	4.5×10^{-8}	28
0.1	3.4000000	7×10^{-36}	80

Table 7.1: The entropic plan for the two-by-two example of Section 7.3.1 at four values of ε. The columns are the regularization strength, the total transport cost of the resulting plan, the mass it places on the route $(2, 1)$ that the exact optimum leaves empty, and the number of Sinkhorn sweeps needed to bring the total marginal error below 10^{-9}. The exact optimum has cost 3.4 and places no mass on that route.

7.10.5 The effect of ε

If we solve the same problem at four values of the strength, we can trace the trade-off (Table 7.1).

As ε decreases the cost approaches 3.4 and the mass on the route $(2, 1)$ collapses toward zero, so the entropic plan converges to an optimal plan of the exact problem. As ε grows the plan spreads toward the independent coupling $a_i b_j$, whose cost in this example is 4.36; at $\varepsilon = 10$ the cost 4.054 has already covered roughly two thirds of the gap between the two.

The last column records what a small ε costs in computation. The sweep count grows as ε falls, and the kernel entries $e^{-C_{ij}/\varepsilon}$ underflow in floating point once C_{ij}/ε exceeds a few hundred. We can therefore run both updates on logarithms, replacing each division by a log-sum-exp over a row or a column, which costs the same nm operations per sweep and holds at any ε.

7.11 Endpoint couplings and reference paths

In optimal transport, we minimize the total cost over couplings of two laws. This linear program has a dual in prices and a complementary-slackness condition. For Gaussians, we have a closed-form solution. In the dynamic formulation, the minimizer moves every particle in a straight line. In entropic transport, we find the coupling closest in relative entropy to the Gibbs kernel $K = \exp(-C/\varepsilon)$. We compute it with Sinkhorn's algorithm by alternately rescaling rows and columns, at nm operations per sweep.

The derivation in Section 7.9.2 requires only that K have strictly positive entries. We can differentiate the Lagrangian, split the exponential into three factors, and impose the marginals without assuming that K comes from a cost. We can therefore use positive endpoint probabilities from a stochastic process as the matrix entries. Each entry then gives the probability of starting at one state and ending at the other.

The entropic plan specifies endpoint pairs. To sample a generative model by integration, we also need paths between those endpoints. We can obtain them from reference dynamics with a known trajectory law. Its endpoint weights replace K, and we can sample its trajectories conditional on a fixed endpoint pair. The Schrödinger bridge is the closest path law to this reference among those with the required endpoint marginals.

7.12 The reference process and the bridge problem

Fix a reference process on the time interval $[0, 1]$, and write R for its path law. To replace K with its endpoint weights, we record the joint law of its starting and ending states,

$$R_{01}(x, y) = R_0(x)\ \Pr\left(X_1 = y \mid X_0 = x\right).$$

"The reference endpoint probability $R_{01}(x, y)$ equals the starting probability $R_0(x)$ times the conditional probability of final state $X_1 = y$ given initial state $X_0 = x$. It is the probability of a reference path starting at x at time zero and ending at y at time one."

- R is the law of the reference process on paths, the dynamics we are given.
- R_0 is its law at time 0, a probability vector over states or a density on $\mathbb{R}^d$.
- R_{01} is its endpoint coupling, a joint law on pairs (x, y) of start and end.

The letter R has a different meaning in this chapter from the one it had in Chapter 4, where $R_t(x, y)$ was a jump rate. Here R is a path law throughout.

Suppose we observe a cloud of diffusing particles with law π_0 at time 0 and law π_1 at time 1. The reference process started from π_0 does not end at π_1. Schrödinger asked for the process that departs least from the reference while matching both observations [Schrödinger, 1931]. The departure is measured in relative entropy on paths:

$$\boxed{\min_P\ D_{\mathrm{KL}}(P \,\|\, R) \quad \text{subject to} \quad P_0 = \pi_0,\quad P_1 = \pi_1.}$$

"The Schrödinger bridge minimizes relative entropy $D_{\mathrm{KL}}(P \,\|\, R)$ from reference path law R over candidate path laws P, subject to initial marginal $P_0 = \pi_0$ and final marginal $P_1 = \pi_1$, where π_0 and π_1 are the required source and target laws."

To define path laws and their relative entropy, we can start in discrete time on a finite state space. A path is then a finite list of states, and its law is a probability vector over the finite set of possible paths [Léonard, 2014].

7.12.1 Path probabilities

We can put N steps on the interval, so the times are $0, 1/N, 2/N, \ldots, 1$, and write x_k for the state at step k. A path is the list $x_{0:N} = (x_0, x_1, \ldots, x_N)$. For a Markov process, the probability of that whole list is a product:

$$R(x_{0:N}) = R_0(x_0)\ \prod_{k=1}^{N} R_k(x_{k-1}, x_k).$$

"The reference probability $R(x_{0:N})$ of the state sequence $x_0, \ldots, x_N$ equals its initial-state probability $R_0(x_0)$ times the product of transition probabilities $R_k(x_{k-1}, x_k)$ from step $k-1$ to step k."

- $x_{0:N}$ is one complete path, a list of $N + 1$ states.

- $R_k(x, y)$ is the reference probability of moving from x to y at step k.
- $R(x_{0:N})$ is a probability over the set of all paths, which has K^{N+1} elements when there are K states.

Let's calculate path probabilities for a chain with two states, labeled 1 and 2. Take the transition matrix A and starting law π_0,

$$A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix} \qquad \pi_0 = (0.5,\ 0.5).$$

Each row of A belongs to one current state. From state 1 the chain stays with probability 0.7 and moves to state 2 with probability 0.3. From state 2 it moves to state 1 with probability 0.4 and stays with probability 0.6. Under π_0 each state has mass 0.5, and we run the chain for $N = 2$ steps, so a path is a triple (x_0, x_1, x_2) and there are eight of them. For the reference we use the same matrix at both steps, $R_1 = R_2 = A$.

The probability of the path $1, 1, 2$ is the starting mass times the two transition probabilities:

$$R(1, 1, 2) = 0.5 \times 0.7 \times 0.3 = 0.105.$$

The chain starts in state 1 with probability 0.5, stays in state 1 with probability 0.7, and then moves to state 2 with probability 0.3. If we multiply the three numbers we get 0.105. The same multiplication over the other seven triples fills Table 7.2, and the eight numbers sum to one, which is the statement that R is a probability law on paths.

Table 7.2: The eight paths of the two-state chain over $N = 2$ steps, with their probabilities under the reference law R. A path is written as the triple of states it visits at steps 0, 1 and 2. Each probability is the starting mass 0.5 times the two transition probabilities read off the rows of A. The eight probabilities sum to one.

path	factors	$R(x_{0:2})$	path	factors	$R(x_{0:2})$
$1, 1, 1$	$0.5 \times 0.7 \times 0.7$	0.245	$2, 1, 1$	$0.5 \times 0.4 \times 0.7$	0.140
$1, 1, 2$	$0.5 \times 0.7 \times 0.3$	0.105	$2, 1, 2$	$0.5 \times 0.4 \times 0.3$	0.060
$1, 2, 1$	$0.5 \times 0.3 \times 0.4$	0.060	$2, 2, 1$	$0.5 \times 0.6 \times 0.4$	0.120
$1, 2, 2$	$0.5 \times 0.3 \times 0.6$	0.090	$2, 2, 2$	$0.5 \times 0.6 \times 0.6$	0.180

7.12.2 Relative entropy between path laws

With R a probability law on a finite set, relative entropy has the definition of Chapter 1, and the only change is that the summation index runs over paths:

$$D_{\mathrm{KL}}(P \,\|\, R) = \sum_{x_{0:N}} P(x_{0:N}) \log \frac{P(x_{0:N})}{R(x_{0:N})} \ \geq\ 0.$$

"The path-space relative entropy $D_{\mathrm{KL}}(P \,\|\, R)$ averages the log probability ratio $\log(P(x_{0:N})/R(x_{0:N}))$ over paths $x_{0:N}$ drawn from candidate law P. It is nonnegative and equals zero only when P and reference law R agree on every path."

For the chain above, a candidate P is a list of eight numbers summing to one, and $D_{\mathrm{KL}}(P \,\|\, R)$ is one sum over those eight terms. The Schrödinger problem minimizes that sum under two linear constraints, one at each end. Written this way it is a convex problem over an eight-dimensional simplex, and it would be solvable by any general-purpose method. To solve problems at the scale of images or molecules, we can factor $D_{\mathrm{KL}}(P \,\|\, R)$ into endpoint and interior contributions, then apply the constraints to the endpoint term.

7.13 The chain rule for relative entropy

The Schrödinger constraints specify only the first and last states of a path. We can therefore split $D_{\mathrm{KL}}(P \,\|\, R)$ into an endpoint term and an unconstrained interior term. The chain rule for relative entropy gives this decomposition whenever we factor a joint law into a marginal and a conditional.

We can take two joint laws π and π' on a pair (x, y), and factor each one:

$$\pi(x, y) = \pi_X(x)\, \pi_{Y|X}(y \mid x) \qquad \pi'(x, y) = \pi'_X(x)\, \pi'_{Y|X}(y \mid x).$$

"The joint probability $\pi(x, y)$ equals the first-coordinate marginal $\pi_X(x)$ times the second-coordinate conditional probability $\pi_{Y|X}(y \mid x)$. The second joint law π' factors in the same way into marginal $\pi'_X(x)$ and conditional $\pi'_{Y|X}(y \mid x)$."

Next, we take the logarithm of the ratio of the two joint laws. A logarithm turns each product into a sum, and the two sums line up term by term:

$$\log \frac{\pi(x, y)}{\pi'(x, y)} = \log \frac{\pi_X(x)}{\pi'_X(x)} + \log \frac{\pi_{Y|X}(y \mid x)}{\pi'_{Y|X}(y \mid x)}.$$

We average both sides under π. The first term on the right depends on x alone, so averaging it under π is the same as averaging it under π_X, which produces $D_{\mathrm{KL}}(\pi_X \,\|\, \pi'_X)$. The second term depends on both coordinates, and we average it in two stages, first over y given x, which produces a relative entropy between the two conditionals at that x, and then over x drawn from π_X:

$$\boxed{D_{\mathrm{KL}}(\pi \,\|\, \pi') = D_{\mathrm{KL}}(\pi_X \,\|\, \pi'_X) + \mathbb{E}_{x \sim \pi_X}\Big[D_{\mathrm{KL}}\Big(\pi_{Y|X}(\cdot \mid x) \,\Big\|\, \pi'_{Y|X}(\cdot \mid x) \Big) \Big].}$$

"The joint relative entropy $D_{\mathrm{KL}}(\pi \,\|\, \pi')$ equals the marginal relative entropy $D_{\mathrm{KL}}(\pi_X \,\|\, \pi'_X)$, plus the conditional relative entropy between $\pi_{Y|X}(\cdot \mid x)$ and $\pi'_{Y|X}(\cdot \mid x)$, averaged over first-coordinate values x drawn from π_X."

Both terms on the right are nonnegative. The first measures the difference between the marginal laws of the first coordinate. The second measures the difference between the conditional laws of the second coordinate, averaged over the first coordinate under π.

7.13.1 Relative-entropy decomposition example

Let's compute both terms for two small tables. Let the rows be x and the columns be y, and take

$$\pi = \begin{pmatrix} 0.5 & 0.1 \\ 0.1 & 0.3 \end{pmatrix} \qquad \pi' = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}.$$

Every entry of π' is 0.25, the uniform law on the four cells. For the relative entropy of π from π', we multiply each entry of π by the logarithm of its ratio to 0.25 and add the four terms:

$$0.5 \log 2 + 0.1 \log 0.4 + 0.1 \log 0.4 + 0.3 \log 1.2 = 0.34657 - 0.09163 - 0.09163 + 0.05470 = 0.2180.$$

Two of the four terms are negative, because the cells $(1, 2)$ and $(2, 1)$ have less mass under π than under π'. The total is positive, as it must be.

We now compute the two parts separately. The row sums of π are $0.5 + 0.1 = 0.6$ and $0.1 + 0.3 = 0.4$, which is the marginal π_X, and the row sums of π' are 0.5 and 0.5:

$$D_{\mathrm{KL}}(\pi_X \,\|\, \pi'_X) = 0.6 \log \tfrac{0.6}{0.5} + 0.4 \log \tfrac{0.4}{0.5} = 0.10939 - 0.08926 = 0.0201.$$

The marginal term is small because the row probabilities 0.6 and 0.5 are close. For the conditional term, we normalize each row of π. The first row becomes $(0.8333, 0.1667)$ and the second becomes $(0.25, 0.75)$, while both rows of π' normalize to $(0.5, 0.5)$. Their relative entropies from $(0.5, 0.5)$ are

$$0.8333 \log \tfrac{0.8333}{0.5} + 0.1667 \log \tfrac{0.1667}{0.5} = 0.2426, \qquad 0.25 \log \tfrac{0.25}{0.5} + 0.75 \log \tfrac{0.75}{0.5} = 0.1308.$$

Weighting these by the masses 0.6 and 0.4 that π_X puts on the two rows,

$$\mathbb{E}_x\big[D_{\mathrm{KL}}(\text{row of } \pi \,\|\, \text{row of } \pi')\,\big] = 0.6\,(0.2426) + 0.4\,(0.1308) = 0.1979,$$

and the two parts add to $0.0201 + 0.1979 = 0.2180$, the number computed directly from the four cells. Almost all of the disagreement between these two tables is in the conditionals, because the rows of π are far from uniform even after their totals have been matched.

7.13.2 Splitting a path law at its two endpoints

We can apply the chain rule to a path law with the pair of endpoints (x_0, x_N) playing the part of x, and the interior states $x_{1:N-1}$ playing the part of y:

$$\boxed{D_{\mathrm{KL}}(P \,\|\, R) = D_{\mathrm{KL}}(P_{01} \,\|\, R_{01}) + \mathbb{E}_{(x_0,x_N)\sim P_{01}}\Big[D_{\mathrm{KL}}\Big(P_{|01}\,\Big\|\, R_{|01}\Big)\Big].}$$

"The path relative entropy $D_{\mathrm{KL}}(P \,\|\, R)$ equals the endpoint relative entropy $D_{\mathrm{KL}}(P_{01} \,\|\, R_{01})$, plus the relative entropy between conditional interior laws $P_{|01}$ and $R_{|01}$, averaged over endpoint pairs (x_0, x_N) from the candidate endpoint law P_{01}."

- P_{01} is the law of the pair (x_0, x_N) under P, a coupling of its two marginals.
- $P_{|01}(x_{1:N-1} \mid x_0, x_N)$ is the bridge of P, the law of the interior states given the two ends.
- $R_{|01}$ is the reference bridge, the same conditional law computed under R.

The two constraints specify the row and column sums of P_{01}. We can set the nonnegative second term to zero by using the reference bridge between every endpoint pair. This choice preserves the endpoint law and both constraints. The optimal path law therefore uses the reference bridges, and we only need to optimize the endpoint coupling,

$$\boxed{P^\star(x_{0:N}) = \pi^\star(x_0, x_N)\; R_{|01}(x_{1:N-1} \mid x_0, x_N).}$$

"The bridge path probability $P^\star(x_{0:N})$ equals endpoint-pair probability $\pi^\star(x_0, x_N)$ times reference conditional probability $R_{|01}(x_{1:N-1} \mid x_0, x_N)$ for the interior states $x_{1:N-1}$ between those endpoints."

We have reduced the optimization over path laws to an optimization over couplings. To sample the bridge, draw an endpoint pair from $\pi^\star$, then run the reference process conditioned on those endpoints.

The set of all path laws is a simplex. The linear marginal constraints define a convex subset, and the bridge is its closest point to R in relative entropy (Figure 7.4). This is the same projection geometry as in the entropic problem, now applied to path laws.

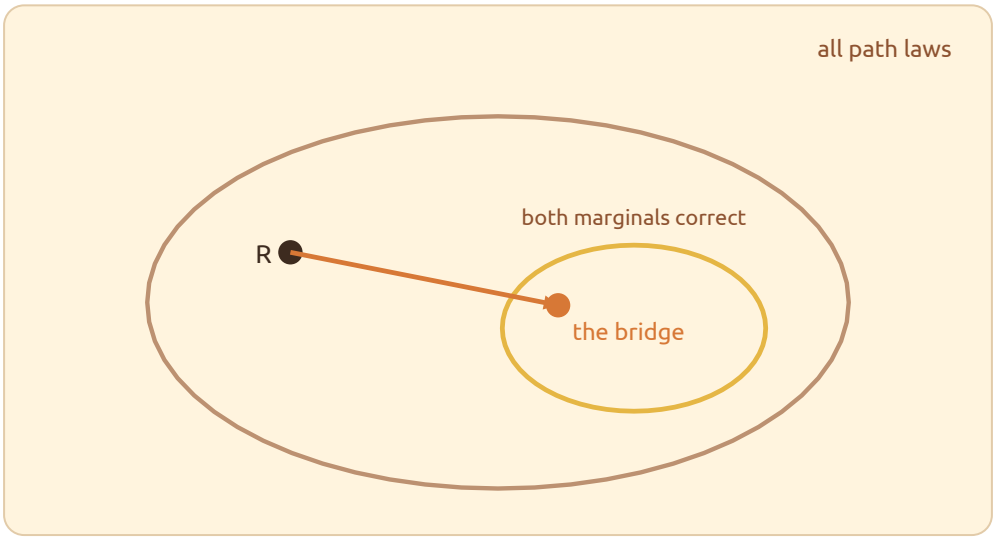


Figure 7.4: The Schrödinger bridge as a projection. The outer region is the set of all path laws on the time grid. The reference path law R, the black point, lies outside the convex subset of path laws whose marginal at time 0 is π_0 and whose marginal at time 1 is π_1, drawn as the inner region. The bridge $P^\star$, the orange point, is the member of that subset with the smallest relative entropy $D_{\mathrm{KL}}(P \,\|\, R)$, and the segment marks that relative entropy. The subset is convex because both constraints are linear in P.

7.14 The static problem and the two potentials

If we substitute $P^\star = \pi^\star R_{|01}$ back into the chain rule, only the first term is left, so the bridge coupling minimizes the endpoint relative entropy under the two marginal constraints:

$$\pi^\star = \operatorname*{arg\,min}_{\pi \in \Pi(\pi_0, \pi_1)} D_{\mathrm{KL}}(\pi \,\|\, R_{01})\,.$$

"The optimal endpoint coupling $\pi^\star$ minimizes relative entropy from reference endpoint law R_{01} over the coupling set $\Pi(\pi_0, \pi_1)$, whose first and second marginals are the required laws π_0 and π_1."

This is the entropic transport problem that opened the chapter, with R_{01} written in place of the Gibbs kernel K. The Lagrangian computation is unchanged, so the minimizer has the same scaling form:

$$\boxed{\pi^\star(x, y) = f(x)\; R_{01}(x, y)\; g(y).}$$

"The bridge endpoint probability $\pi^\star(x, y)$ equals the reference endpoint probability $R_{01}(x, y)$ multiplied by positive source potential $f(x)$ at starting state x and positive target potential $g(y)$ at final state y."

- f and g are the Schrödinger potentials, positive functions on the state space.
- They are in the position that the Sinkhorn scalings u and v had in the entropic problem, and they are determined only up to a constant, since $f \mapsto cf$ and $g \mapsto g/c$ do not change $\pi^\star$.

We can relate these multiplicative potentials to the additive prices from Section 7.4. The exact dual constrains the prices by $f_i + g_j \le C_{ij}$. In Section 7.9.2, we exponentiated each price to obtain a row scaling $u_i = e^{f_i/\varepsilon - 1/2}$. Here f and g denote the multiplicative scalings, following the Schrödinger literature. We can recover the additive prices by taking logarithms, as in Section 7.10.4.

The two marginal constraints determine f and g by the same alternation:

$$f \;\leftarrow\; \pi_0 \oslash (R_{01}\, g) \qquad g \;\leftarrow\; \pi_1 \oslash (R_{01}^\top f).$$

"We update source potential f by dividing source masses π_0 entrywise by $R_{01}g$, enforcing the row sums of the scaled endpoint law. We then update target potential g by dividing target masses π_1 entrywise by $R_{01}^\top f$, enforcing its column sums."

Here $\oslash$ is entrywise division of one vector by another. Each update enforces one margin and changes the other, and iterating them converges to the fixed point at which both hold. The procedure is Sinkhorn's algorithm with R_{01} in place of K; the name for it in statistics is iterative proportional fitting [Peyré and Cuturi, 2019].

7.14.1 The squared cost from a Brownian reference

The entropic transport problem is a special case with a Brownian reference. Let the reference be a Brownian motion on $\mathbb{R}^d$ with variance ε per unit time, started from π_0. Its transition density over one unit of time is Gaussian:

$$R_{1|0}(y \mid x) = (2\pi\varepsilon)^{-d/2} \exp\Big(-\frac{\|y - x\|^2}{2\varepsilon}\Big).$$

"The Brownian transition density $R_{1|0}(y \mid x)$ equals normalization $(2\pi\varepsilon)^{-d/2}$ times the exponential of minus the squared displacement from starting point x to endpoint y, divided by twice the noise variance ε, where d is the state dimension."

Take minus its logarithm:

$$-\log R_{1|0}(y \mid x) = \frac{\|y - x\|^2}{2\varepsilon} + \frac{d}{2}\log(2\pi\varepsilon).$$

The second term is a constant in x and y, so the transition density is a Gibbs kernel for the squared distance at strength 2ε. Substituting it into $D_{\mathrm{KL}}(\pi \,\|\, R_{01})$ and dropping every term that does not depend on π,

$$\operatorname*{arg\,min}_{\pi \in \Pi(\pi_0, \pi_1)} D_{\mathrm{KL}}(\pi \,\|\, R_{01}) \;=\; \operatorname*{arg\,min}_{\pi \in \Pi(\pi_0, \pi_1)} \Big\{ \int \tfrac{1}{2}\|y - x\|^2 \, d\pi \;-\; \varepsilon\, H(\pi) \Big\},$$

in which $H(\pi)$ is the entropy of the plan.

"Minimizing relative entropy from the Brownian endpoint law over couplings π of source law π_0 and target law π_1 also minimizes the average half-squared endpoint distance $\frac{1}{2}\|y-x\|^2$, minus coupling entropy $H(\pi)$ weighted by noise variance ε."

The entropic transport problem and the static Schrödinger problem are therefore the same problem for a Brownian reference, with the noise variance playing the part of the regularization strength. The four-point example scaled by hand earlier in this chapter transfers unchanged. Take source points 0 and 2 with masses 0.6 and 0.4, and target points 1 and 3 with masses 0.3 and 0.7. At noise variance $\varepsilon = 1$, the transition density is proportional to $e^{-C/2}$, where C is the squared-distance cost matrix. This is the kernel we scaled at regularization strength 2. The bridge coupling is the plan found there, with entries

$$\pi^\star = \begin{pmatrix} 0.2931 & 0.3069 \\ 0.0069 & 0.3931 \end{pmatrix},$$

and the paths that fill it in are Brownian paths pinned at their two ends.

7.14.2 Two-state terminal reweighting

On a finite state space we can carry out the same scaling in full, by hand. Let's return to the chain with matrix A and starting law $\pi_0 = (0.5, 0.5)$, and request a terminal law after two steps:

$$\pi_1 = (0.2,\ 0.8) \qquad A^2 = \begin{pmatrix} 0.61 & 0.39 \\ 0.52 & 0.48 \end{pmatrix} \qquad R_{01} = \operatorname{diag}(\pi_0)\, A^2 = \begin{pmatrix} 0.305 & 0.195 \\ 0.26 & 0.24 \end{pmatrix}.$$

The two-step matrix A^2 has rows $(0.61, 0.39)$ and $(0.52, 0.48)$, computed by squaring A. If we multiply each row by its starting mass 0.5 we get the reference endpoint law R_{01}, whose rows are $(0.305, 0.195)$ and $(0.26, 0.24)$. Its row sums are 0.5 and 0.5, which already match π_0, and its column sums are 0.565 and 0.435, far from the requested 0.2 and 0.8. Running the reference forward from π_0 therefore gives a total absolute error of 0.73 at the terminal time.

We can start the alternation from $f = (1, 1)$. With that f, the vector $R_{01}^\top f$ is the pair of column sums, so

$$g = \pi_1 \oslash (R_{01}^\top f) = \Big(\frac{0.2}{0.565},\ \frac{0.8}{0.435}\Big) = (0.3540,\ 1.8391).$$

We multiply the mass of the first state by about one third and the mass of the second by about two, changing the terminal law from $(0.565, 0.435)$ to $(0.2, 0.8)$. Applying R_{01} to this g gives $(0.4666, 0.5334)$, and the second update is

$$f = \pi_0 \oslash (R_{01}\, g) = \Big(\frac{0.5}{0.4666},\ \frac{0.5}{0.5334}\Big) = (1.0716,\ 0.9374).$$

The correction to f is much smaller than the correction to g, because the row sums were already correct and the g update disturbed them only a little. As we repeat the two updates, the potentials settle after three rounds to six figures, and the error at the terminal marginal falls by a factor of about 190 per round (Table 7.3).

At the fixed point, $f = (1.072002, 0.937019)$ and $g = (0.350517, 1.843637)$, and the coupling is

$$\pi^\star = \begin{pmatrix} 0.114605 & 0.385395 \\ 0.085395 & 0.414605 \end{pmatrix}.$$

Table 7.3: Iterative proportional fitting of the reference endpoint law R_{01} to the marginals $\pi_0 = (0.5, 0.5)$ and $\pi_1 = (0.2, 0.8)$. One round is a g update followed by an f update, started from $f = (1, 1)$. The last column is the total absolute error of the column sums of $fR_{01}g$ against π_1, measured at the end of the round, when the row sums are exact. The error at round 0, for the reference itself, is 0.73.

round	f	g	terminal error	ratio
1	(1.071615, 0.937357)	(0.353982, 1.839080)	3.933×10^{-3}	185.6
2	(1.072000, 0.937021)	(0.350536, 1.843612)	2.098×10^{-5}	187.5
3	(1.072002, 0.937019)	(0.350517, 1.843637)	1.119×10^{-7}	187.5
4	(1.072002, 0.937019)	(0.350517, 1.843637)	5.966×10^{-10}	187.5

Under this coupling, 0.114605 of the mass moves from state 1 to state 1, 0.385395 from 1 to 2, 0.085395 from 2 to 1, and 0.414605 from 2 to 2. The row sums are 0.5 and 0.5 and the column sums are 0.2 and 0.8, so both constraints hold. Its relative entropy from R_{01} is 0.2820, against 0.2847 for the independent coupling $\pi_0\pi_1^\top$, which also satisfies both constraints. Scaling the reference therefore improves on the simplest feasible choice by 0.0027 nats.

We can now sample an endpoint pair and a reference bridge between those endpoints. To sample forward one step at a time without drawing the final endpoint first, we need transition probabilities.

7.15 Markov bridge construction

A sampler starts at x_0 and steps forward with no access to its final state. To run the bridge in that way we need transition matrices under which π_0 becomes π_1 after N steps, with the reference bridges between every pair of ends. We can compute a potential backward from g, then use it to reweight each reference transition.

Define a backward potential h on states, one vector per step, starting from g at the last step:

$$h_N = g \qquad h_k(x) = \sum_y A(x, y)\, h_{k+1}(y).$$

"The final backward potential h_N equals terminal potential g. At earlier step k, the potential $h_k(x)$ averages next-step potentials $h_{k+1}(y)$ with reference transition probabilities $A(x, y)$ over destination states y."

- $h_k(x)$ is a positive number attached to the state x at step k, the reference average of g at the final state, started from x at step k.
- The recursion runs from $k = N$ down to $k = 0$, one matrix-vector product per step.

For the two-state chain, $g = (0.350517, 1.843637)$ at convergence, so $h_2 = g$. One step back,

$$h_1(1) = 0.7 \times 0.350517 + 0.3 \times 1.843637 = 0.798453,$$

and in the same way $h_1(2) = 0.4 \times 0.350517 + 0.6 \times 1.843637 = 1.246389$. One more step back, $h_0 = (0.932834, 1.067215)$. Collecting the three vectors,

$$h_2 = (0.350517,\ 1.843637) \quad h_1 = (0.798453,\ 1.246389) \quad h_0 = (0.932834,\ 1.067215).$$

The potential is 5.26 times larger on state 2 than on state 1 at the last step. The target requires four times as much mass on state 2 as on state 1. The reference ends with masses 0.435 and 0.565 on those states, respectively. The terminal potential corrects this difference in relative mass. Two steps earlier the two states are nearly level, at a ratio of 1.14. Mixing reduces the difference between the potential values at earlier steps. The first steps of the bridge therefore stay close to the reference.

7.15.1 The tilted matrices

We can reweight each reference transition by the ratio of the potential at the destination to the potential at the current state:

$$\boxed{A_k^h(x,y) = A(x,y)\,\frac{h_{k+1}(y)}{h_k(x)}.}$$

"The tilted transition probability $A_k^h(x,y)$ from state x at step k to state y at step $k+1$ equals reference probability $A(x,y)$ multiplied by destination potential $h_{k+1}(y)$ divided by current-state potential $h_k(x)$."

This reweighting is Doob's h-transform [Doob, 1957]. The potential ratio increases transition probabilities toward states with a larger expected terminal value of g and decreases the other transition probabilities.

Proposition 7.6 (The tilted matrix is stochastic)**.** *Let h satisfy the backward recursion $h_k(x) = \sum_y A(x,y)h_{k+1}(y)$ with $h_k(x) > 0$ for every k and x. Then each row of A_k^h sums to one.*

Proof. Sum the tilted entries over the destination and pull out the factor that does not depend on y:

$$\sum_y A_k^h(x,y) = \frac{1}{h_k(x)} \sum_y A(x,y)\,h_{k+1}(y) = \frac{h_k(x)}{h_k(x)} = 1.$$

The inner sum is exactly the backward recursion, which defines $h_k(x)$. □

The backward recursion therefore normalizes every row of the tilted transition matrix. For the two-state chain the two tilted matrices are

$$A_0^h = \begin{pmatrix} 0.599160 & 0.400840 \\ 0.299266 & 0.700734 \end{pmatrix} \qquad A_1^h = \begin{pmatrix} 0.307297 & 0.692703 \\ 0.112490 & 0.887510 \end{pmatrix}.$$

For example, $A_0^h(1,1) = 0.7 \times 0.798453/0.932834 = 0.599160$. The first step moves from state 1 to state 2 with probability 0.400840, compared with 0.3 under the reference. The second step moves from state 1 to state 2 with probability 0.692703, more than twice the reference value, and from state 2 it stays with probability 0.887510, against 0.6 under the reference. Most of the correction therefore falls on the last step, which is the behavior the nearly level h_0 and the strongly tilted h_2 already imply.

Applying the two matrices in order to the source law,

$$(0.5,\ 0.5) \longrightarrow (0.449213,\ 0.550787) \longrightarrow (0.2,\ 0.8),$$

the chain lands exactly on the requested terminal law. The intermediate law $(0.449213, 0.550787)$ is the marginal of the bridge at the middle step; the reference at that step has $(0.55, 0.45)$, so the bridge has already moved mass toward state 2 by then.

7.15.2 The product along a path

The tilted chain has the required marginals. We can verify that its conditional bridges also match the reference using a telescoping product. Multiply the tilted probabilities along a path and cancel the potentials at the interior steps, where each h_k appears once in a numerator and once in a denominator:

$$\prod_{k=0}^{N-1} A_k^h(x_k, x_{k+1}) = \Big[\prod_{k=0}^{N-1} A(x_k, x_{k+1}) \Big] \frac{h_N(x_N)}{h_0(x_0)}.$$

"Multiplying tilted transition probabilities $A_k^h(x_k, x_{k+1})$ along a path gives the product of reference transition probabilities $A(x_k, x_{k+1})$, multiplied by terminal potential $h_N(x_N)$ divided by initial potential $h_0(x_0)$."

That ratio depends only on the two ends. Conditioned on the ends, the tilted law and the reference law therefore assign probabilities to interior paths that differ by a common factor, which cancels in the normalization. The two conditional bridges are therefore identical.

Next, we multiply by $\pi_0(x_0)$ and sum over the interior states to read off the endpoint law of the tilted chain:

$$\pi_0(x_0)\, A^N(x_0, x_N)\, \frac{h_N(x_N)}{h_0(x_0)} = \frac{1}{h_0(x_0)}\; R_{01}(x_0, x_N)\; g(x_N).$$

"The tilted endpoint law equals reference endpoint probability $R_{01}(x_0, x_N) = \pi_0(x_0)A^N(x_0, x_N)$, multiplied by terminal potential $g(x_N)$ and divided by initial backward potential $h_0(x_0)$, where π_0 is the starting law and A^N the N-step reference transition matrix."

The right-hand side has the form $fR_{01}g$ with $f = 1/h_0$, and by construction it has both requested marginals. It is therefore the bridge coupling $\pi^\star$. The two-state numbers confirm the identification: $1/h_0 = (1.072002, 0.937019)$, which is the f that the scaling produced, and $\mathrm{diag}(\pi_0)A_0^h A_1^h$ reproduces $\pi^\star$ entry by entry.

Theorem 7.7 (The bridge is the h-transform of the reference)**.** *Let R be a Markov chain with positive transition matrix A and initial law π_0, let $\pi^\star = fR_{01}g$ solve the static problem, and let h satisfy $h_N = g$ and $h_k = Ah_{k+1}$. Then the chain with initial law π_0 and transition matrices $A_k^h(x, y) = A(x, y)h_{k+1}(y)/h_k(x)$ is Markov, has endpoint coupling $\pi^\star$, has the reference bridges between every pair of ends, and is therefore the Schrödinger bridge $P^\star$.*

Proposition 7.6 establishes stochastic transition matrices. The telescoping product establishes the reference conditional bridges, and the endpoint calculation establishes the required coupling. The resulting Schrödinger bridge is Markov, with reference transitions reweighted by a potential satisfying a linear backward recursion.

7.15.3 Eight-path bridge example

Let's multiply the bridge coupling $\pi^\star$ by the reference bridge between each pair of ends, which for this chain is

$$R_{|01}(x_1 \mid x_0, x_2) = \frac{A(x_0, x_1)\, A(x_1, x_2)}{A^2(x_0, x_2)},$$

the probability that a reference path visiting x_0 and x_2 passed through x_1 in between. The product is the bridge path law $P^\star$, listed against the reference in Table 7.4.

Table 7.4: The bridge path law $P^\star$ on the eight paths of the two-state chain, beside the reference law R of Table 7.2 and their ratio. Each entry of $P^\star$ is $\pi^\star(x_0, x_2)$ times the reference bridge probability $R_{|01}(x_1 \mid x_0, x_2)$. The ratio column takes only four values, one for each pair of ends, and each value is $f(x_0)\, g(x_2)$ for the potentials $f = (1.072002, 0.937019)$ and $g = (0.350517, 1.843637)$.

path	R	$P^\star$	$P^\star/R$	path	R	$P^\star$	$P^\star/R$
$1,1,1$	0.245000	0.092060	0.375755	$2,1,1$	0.140000	0.045982	0.328441
$1,1,2$	0.105000	0.207520	1.976383	$2,1,2$	0.060000	0.103651	1.727522
$1,2,1$	0.060000	0.022545	0.375755	$2,2,1$	0.120000	0.039413	0.328441
$1,2,2$	0.090000	0.177874	1.976383	$2,2,2$	0.180000	0.310954	1.727522

The ratio $P^\star/R$ takes four values in the table. Paths with the same endpoints have the same ratio, as we derived from the telescoping product in Section 7.15.2. Each of those four values factorizes, since $1.072002 \times 0.350517 = 0.375755$ for the pair $(1,1)$ and $0.937019 \times 1.843637 = 1.727522$ for the pair $(2,2)$, which is the scaling form $\pi^\star = f R_{01} g$. If we sum $P^\star \log(P^\star/R)$ over the eight paths, we get

$$D_{\mathrm{KL}}(P^\star \,\|\, R) = 0.281961,$$

exactly the value of $D_{\mathrm{KL}}(\pi^\star \,\|\, R_{01})$ computed in Section 7.14.2. The conditional term of the chain rule is zero, because $P^\star$ and R assign the same law to the interior given the ends.

If we group the same eight numbers by the middle state, we recover the marginal of the bridge at step 1. The four paths with $x_1 = 1$ are $1,1,1$, $1,1,2$, $2,1,1$ and $2,1,2$, and their probabilities add to

$$0.092060 + 0.207520 + 0.045982 + 0.103651 = 0.449213,$$

which with the complementary sum 0.550787 is the intermediate law that the two tilted matrices produced in Section 7.15.1. The reference puts 0.55 on state 1 at that step, so the bridge has already moved 0.1008 of mass toward state 2 halfway through.

For a diffusion on $\mathbb{R}^d$, we replace sums over states with integrals and the backward recursion with a partial differential equation. Reweighting adds a drift to the reference. We also need a continuous-path definition of relative entropy, since we cannot sum over continuous curves as we did over the finite set of paths in Section 7.12.2.

7.16 A diffusion reference

Let the reference be a diffusion on $\mathbb{R}^d$,

$$dX_t = b(X_t, t)\, dt + \sqrt{\varepsilon}\, dB_t,$$

where b is the reference drift and ε is the noise variance per unit time. Over a short interval dt, the state changes by $b(X_t, t)\, dt$ plus $\sqrt{\varepsilon}$ times Brownian increment dB_t. The candidates P in the Schrödinger problem are the laws obtained by adding a drift correction and keeping the noise:

$$\boxed{dX_t^u = \big(b(X_t^u, t) + u(X_t^u, t)\big)\, dt + \sqrt{\varepsilon}\, dB_t.}$$

"The controlled state X_t^u changes by reference drift $b(X_t^u, t)$ plus control drift $u(X_t^u, t)$, multiplied by the interval dt, and a Brownian increment dB_t scaled by the square root of reference noise variance ε."

- $u_t(x)$, written $u(x,t)$ when both arguments are needed, is the control, a drift correction in $\mathbb{R}^d$ at every state and time. Chapter 5 used $u_t(y,x)$ for a probability velocity between two states of a finite set; the symbol here has the control meaning throughout this chapter.
- R is the path law of the reference, P^u the path law of the controlled process, both started from the same initial law.
- B_t is a standard Brownian motion in $\mathbb{R}^d$.

Changing the drift and keeping the noise is the only change under which P^u stays absolutely continuous with respect to R. A different noise level changes the quadratic variation of the paths. The two laws would then be mutually singular, with infinite $D_{\mathrm{KL}}(P^u \,\|\, R)$. Every path law with finite relative entropy from R is of this form, so searching over u is the same as searching over P.

7.16.1 One short step

Relative entropy on paths is a limit of relative entropies on finite grids, so we compare the two laws over one step of length Δt. Over such a step both increments are Gaussian with the same covariance:

$$\begin{aligned} \text{under } R: &\quad \Delta X \sim \mathcal{N}(b\,\Delta t,\ \ \varepsilon\,\Delta t\, I) \\ \text{under } P^u: &\quad \Delta X \sim \mathcal{N}\big((b+u)\,\Delta t,\ \ \varepsilon\,\Delta t\, I\big) \end{aligned}$$

The two differ only in the mean, by $u\,\Delta t$. We can take the ratio of the two densities at the same increment ΔX. The normalizing constants are equal and cancel, leaving the two exponents:

$$\log \frac{p^u(\Delta X)}{p^R(\Delta X)} = \frac{\|\Delta X - b\,\Delta t\|^2 - \|\Delta X - (b+u)\,\Delta t\|^2}{2\,\varepsilon\,\Delta t}.$$

"The log density ratio $\log\Big(p^u(\Delta X)/p^R(\Delta X)\Big)$ compares the same increment ΔX under the controlled and reference laws. It equals the squared distance from reference mean $b\Delta t$, minus the squared distance from controlled mean $(b+u)\Delta t$, divided by twice the increment variance $\varepsilon\Delta t$."

We write $w = \Delta X - b\,\Delta t$ for the part of the increment beyond the reference drift, and expand the second square as $\|w - u\,\Delta t\|^2 = \|w\|^2 - 2w^\top u\,\Delta t + \|u\|^2\Delta t^2$. The $\|w\|^2$ terms cancel:

$$\log \frac{p^u(\Delta X)}{p^R(\Delta X)} = \frac{2\,w^\top u\,\Delta t - \|u\|^2\Delta t^2}{2\,\varepsilon\,\Delta t} = \frac{w^\top u}{\varepsilon} - \frac{\|u\|^2\,\Delta t}{2\,\varepsilon}.$$

"The one-step log density ratio equals the inner product of residual increment $w = \Delta X - b\Delta t$ and control drift u, divided by noise variance ε, minus the squared control norm $\|u\|^2$ times step duration Δt, divided by 2ε."

The first term is of order $\sqrt{\Delta t}$, because w is a noise increment of that size, and the second is of order Δt. When we sum many steps, both terms remain, the first as a stochastic integral and the second as an ordinary one.

7.16.2 Summing over the steps

Under the reference, w is the noise $\sqrt{\varepsilon}\,\Delta B$. Adding the per-step contributions along a whole path and letting Δt go to zero,

$$\log \frac{dP^u}{dR}(X) = \frac{1}{\sqrt{\varepsilon}} \int_0^1 u(X_t, t)^\top dB_t \;-\; \frac{1}{2\varepsilon} \int_0^1 \|u(X_t, t)\|^2 \, dt.$$

"The path log density ratio $\log(dP^u/dR)(X)$ equals the stochastic integral of control $u(X_t, t)$ against reference Brownian motion B_t, scaled by $1/\sqrt{\varepsilon}$, minus the integrated squared control scaled by $1/(2\varepsilon)$, over times zero to one along path X."

This is Girsanov's formula, and it is the continuous counterpart of the ratio of path probabilities written for the chain in Section 7.12.1. The two terms match the one-step expansion: a martingale term built from the noise, and a drift term built from the size of the control.

7.16.3 The average under the controlled law

To calculate relative entropy, we average this log density under P^u. The first term has a nonzero mean under that law because B is Brownian under R and has a drift under P^u. Comparing the two descriptions of the same increment dX_t,

$$\sqrt{\varepsilon}\, dB_t = u\, dt + \sqrt{\varepsilon}\, dB_t^u.$$

Under R the increment is $b\, dt + \sqrt{\varepsilon}\, dB_t$ and under P^u it is $(b + u)\, dt + \sqrt{\varepsilon}\, dB_t^u$, in which B^u is a Brownian motion under P^u. If we set the two expressions equal and cancel $b\, dt$, we obtain the display above. Substituting it into the stochastic integral,

$$\frac{1}{\sqrt{\varepsilon}} \int_0^1 u^\top dB_t = \frac{1}{\sqrt{\varepsilon}} \int_0^1 u^\top dB_t^u + \frac{1}{\varepsilon} \int_0^1 \|u\|^2 dt.$$

The integral against B^u has mean zero under P^u, so averaging the log density produces the two energy terms, with coefficients $1/\varepsilon$ and $-1/(2\varepsilon)$:

$$\boxed{D_{\mathrm{KL}}(P^u \,\|\, R) = \Big(\frac{1}{\varepsilon} - \frac{1}{2\varepsilon}\Big)\, \mathbb{E}_{P^u} \int_0^1 \|u\|^2 \, dt = \frac{1}{2\varepsilon}\, \mathbb{E}_{P^u} \int_0^1 \|u(X_t, t)\|^2 \, dt.}$$

"The path relative entropy $D_{\mathrm{KL}}(P^u \,\|\, R)$ equals the squared control norm $\|u(X_t, t)\|^2$ integrated from time zero to one and averaged over controlled paths from P^u, divided by twice the reference noise variance ε."

- The quantity $\mathbb{E}_{P^u} \int_0^1 \|u\|^2 dt$ is the control energy, the total squared drift correction accumulated along a path and averaged over paths.
- The factor $1/(2\varepsilon)$ makes a given correction cheap when the reference is noisy and expensive when it is nearly deterministic.

We can therefore minimize path-space relative entropy by minimizing an integral of the squared drift correction. This gives a stochastic control objective that we can use to train a neural network.

7.16.4 Constant-control example

We take no reference drift, one dimension, $\varepsilon = 1$, and the constant control $u = 2$. The squared control is 4 at every time, so its integral over the unit interval is 4, and

$$D_{\mathrm{KL}}(P^u \,\|\, R) = \frac{1}{2 \cdot 1} \int_0^1 2^2 \, dt = 2.$$

We can compute the same quantity a second way, from the endpoint laws. Starting from a fixed point x, both laws give a Gaussian endpoint with variance 1. Its mean is $x + 2$ under P^u and x under R. For Gaussians with equal variance, relative entropy is the squared difference of their means divided by twice the variance,

$$D_{\mathrm{KL}}(\mathcal{N}(x+2,\ 1) \,\|\, \mathcal{N}(x,\ 1)) = \frac{2^2}{2 \cdot 1} = 2.$$

The two values agree because a constant drift changes the Brownian endpoint law and preserves the conditional bridges between fixed endpoints. All of the relative entropy is therefore in the endpoint term of the chain rule of Section 7.13.2, and the two computations are the two sides of that identity for this particular control.

7.17 The bridge as a stochastic control problem

With relative entropy written as control energy, the Schrödinger problem becomes a minimization over drift corrections:

$$\min_u \ \frac{1}{2\varepsilon} \mathbb{E} \int_0^1 \|u(X_t^u, t)\|^2 \, dt \quad \text{subject to} \quad X_0^u \sim \pi_0, \quad X_1^u \sim \pi_1.$$

"We minimize expected integrated squared control $\|u(X_t^u, t)\|^2$, divided by twice the noise variance ε, over controls u whose controlled process X_t^u starts with source law π_0 and ends with target law π_1."

Among all controlled processes that connect the two laws, the bridge is the one of least control energy. With no reference drift, as ε goes to zero the smallest value of the expected integral tends to $W_2^2(\pi_0, \pi_1)$, the value of the Benamou-Brenier problem of Section 7.7 [Benamou and Brenier, 2000]. Entropic transport and least-action transport are at the two ends of one family, indexed by the noise of the reference.

7.17.1 The value function and Bellman's principle

Dynamic programming gives a local condition for the optimal control. Replace the terminal constraint with a terminal cost Φ, then define the minimum expected cost remaining from state x at time t,

$$V_t(x) = \min_u \ \mathbb{E}\Big[\int_t^1 \tfrac{1}{2} \|u(X_s^u, s)\|^2 \, ds + \Phi(X_1^u) \ \Big|\ X_t^u = x \Big].$$

"The value function $V_t(x)$ is the minimum expected remaining cost from state x at time t, optimized over controls u. That cost adds half the squared control $\|u(X_s^u, s)\|^2$ integrated over remaining times $s \in [t, 1]$ to terminal cost $\Phi(X_1^u)$."

- $V_t(x)$ is a scalar attached to every state and time, the cost-to-go.
- Φ is the terminal cost, chosen so that the terminal constraint $X_1 \sim \pi_1$ holds; the constant factor $1/\varepsilon$ has been dropped, since it scales the whole objective.

By Bellman's principle, the best cost from x equals the cost of one short step plus the best cost from wherever that step ends:

$$V_t(x) = \min_u \ \mathbb{E}\Big[\tfrac{1}{2}\|u\|^2 \Delta t + V_{t+\Delta t}(X^u_{t+\Delta t}) \ \Big|\ X^u_t = x\Big].$$

We can now expand the second term with Itô's formula. For a diffusion with drift $b+u$ and noise variance ε, the expected change in a smooth function over a short step is the time derivative, plus the drift against the gradient, plus one half the variance against the Laplacian:

$$\mathbb{E}\big[V_{t+\Delta t}(X^u_{t+\Delta t})\big] = V_t(x) + \Big(\partial_t V_t + \langle b+u, \nabla V_t\rangle + \tfrac{\varepsilon}{2}\nabla^2 V_t\Big)\Delta t + o(\Delta t),$$

in which $\nabla^2 V_t$ is the Laplacian of V_t and $o(\Delta t)$ is a remainder small compared with Δt.

7.17.2 The Hamilton-Jacobi-Bellman equation

If we substitute the expansion into Bellman's principle, subtract $V_t(x)$ from both sides, divide by Δt and let Δt go to zero, we get

$$0 = \min_u \Big\{\tfrac{1}{2}\|u\|^2 + \partial_t V_t + \langle b+u, \nabla V_t\rangle + \tfrac{\varepsilon}{2}\nabla^2 V_t\Big\}.$$

"Minimizing over control u makes the sum of four terms equal zero: instantaneous control cost $\frac{1}{2}\|u\|^2$, time derivative $\partial_t V_t$ of the value function, total drift $b+u$ dotted with its gradient ∇V_t, and half the noise variance ε times its Laplacian $\nabla^2 V_t$."

Only two terms contain u, and if we complete the square in those two we can isolate it:

$$\tfrac{1}{2}\|u\|^2 + \langle u, \nabla V_t\rangle = \tfrac{1}{2}\big\|u + \nabla V_t\big\|^2 - \tfrac{1}{2}\|\nabla V_t\|^2.$$

The square is smallest when it is zero, so the optimal control is

$$u^\star(x,t) = -\nabla V_t(x).$$

"The optimal control drift $u^\star(x,t)$ equals the negative gradient $-\nabla V_t(x)$ of the remaining-cost function V_t at state x and time t."

The control points toward lower cost-to-go, with magnitude equal to the gradient norm. Substituting it back and retaining $-\frac{1}{2}\|\nabla V_t\|^2$ gives a closed equation for V,

$$\partial_t V_t + \langle b, \nabla V_t\rangle + \frac{\varepsilon}{2}\nabla^2 V_t - \frac{1}{2}\|\nabla V_t\|^2 = 0, \qquad V_1 = \Phi.$$

"The value function V_t has zero sum of its time derivative, reference drift b dotted with its gradient, half the noise variance ε times its Laplacian, and minus half its squared gradient norm. At time one, V_1 equals terminal cost Φ."

This is the Hamilton-Jacobi-Bellman equation, which we solve backward from the terminal condition. Three of its terms are linear in V. The squared-gradient term is nonlinear, so we cannot directly apply the linear backward recursion used for the chain.

7.17.3 Removing the squared gradient

With one change of variables we can remove that term exactly. We write the value function as minus ε times a logarithm:

$$V_t(x) = -\,\varepsilon \log \varphi_t(x),$$

so that $\varphi_t = e^{-V_t/\varepsilon}$ is positive wherever V_t is finite. This is the Hopf-Cole transform. Differentiating,

$$\partial_t V = -\,\varepsilon\,\frac{\partial_t \varphi}{\varphi} \quad \nabla V = -\,\varepsilon\,\frac{\nabla \varphi}{\varphi} \quad \nabla^2 V = -\,\varepsilon\,\frac{\nabla^2 \varphi}{\varphi} + \varepsilon\,\frac{\|\nabla \varphi\|^2}{\varphi^2}.$$

The Laplacian has an extra term because differentiating $\nabla\varphi/\varphi$ a second time produces the quotient-rule term $\|\nabla\varphi\|^2/\varphi^2$. Substituting all three into the Hamilton-Jacobi-Bellman equation,

$$-\varepsilon\,\frac{\partial_t \varphi}{\varphi} - \varepsilon\,\frac{\langle b, \nabla \varphi\rangle}{\varphi} - \frac{\varepsilon^2}{2}\frac{\nabla^2 \varphi}{\varphi} + \frac{\varepsilon^2}{2}\frac{\|\nabla\varphi\|^2}{\varphi^2} - \frac{\varepsilon^2}{2}\frac{\|\nabla\varphi\|^2}{\varphi^2} = 0.$$

The fourth term comes from $\frac{\varepsilon}{2}\nabla^2 V$ and the fifth from $-\frac{1}{2}\|\nabla V\|^2$, and they are equal with opposite signs, so they cancel. Multiplying what is left by $-\varphi/\varepsilon$,

$$\boxed{\partial_t \varphi_t + \langle b, \nabla \varphi_t\rangle + \frac{\varepsilon}{2}\nabla^2 \varphi_t = 0, \qquad \varphi_1 = e^{-\Phi/\varepsilon}.}$$

"The transformed potential φ_t satisfies a linear backward equation: its time derivative plus reference drift b dotted with its gradient, plus half the noise variance ε times its Laplacian, equals zero. Its terminal value φ_1 is the exponential of negative terminal cost Φ divided by ε."

This is the Kolmogorov backward equation of the reference diffusion, and it is linear. Its solution is an average of the terminal data over reference paths, $\varphi_t(x) = \mathbb{E}_R[\varphi_1(X_1) \mid X_t = x]$, which is the continuous form of the backward recursion $h_k = A h_{k+1}$ of Section 7.15. In the new variable the optimal control reads

$$u^\star(x,t) = -\,\nabla V_t(x) = \varepsilon\,\nabla \log \varphi_t(x).$$

"The optimal control $u^\star(x,t)$ equals the negative gradient of value function $V_t(x)$, which is also noise variance ε times the log-gradient $\nabla \log \varphi_t(x)$ of the transformed potential."

7.17.4 Continuous-space h-transform

If we reweight each transition of a diffusion by a ratio $h(y,s)/h(x,t)$, where h solves the backward equation, we add a drift of $\varepsilon\nabla \log h$:

$$dX_t = \big[b(X_t,t) + \varepsilon\,\nabla \log h(X_t,t)\big]\,dt + \sqrt{\varepsilon}\,dB_t.$$

"The tilted state X_t has drift equal to reference drift $b(X_t,t)$ plus noise variance ε times the log-gradient $\nabla \log h(X_t,t)$ of backward potential h. Its noise remains the Brownian increment dB_t scaled by $\sqrt{\varepsilon}$."

With $h = \varphi$ the added drift is exactly $u^\star$. The bridge is the h-transform of the reference diffusion, which is the statement of Theorem 7.7 with sums replaced by integrals. The ratio $h_{k+1}(y)/h_k(x)$ of

the discrete tilt becomes the logarithmic derivative $\nabla \log h$, and the backward recursion becomes the backward equation.

The marginal density of the bridge at time t factors into the backward potential and a forward one:

$$p_t^\star(x) = \varphi_t(x)\,\psi_t(x) \qquad \varphi_0\,\psi_0 = \pi_0, \quad \varphi_1\,\psi_1 = \pi_1.$$

"The bridge density $p_t^\star(x)$ equals backward potential $\varphi_t(x)$ times forward potential $\psi_t(x)$. Their product must equal source law π_0 at time zero and target law π_1 at time one."

The forward potential ψ_t satisfies the Fokker-Planck equation of the reference, so the pair (φ, ψ) solves one backward and one forward linear equation, coupled only through the two boundary conditions. These are the Schrödinger system, the continuous counterpart of the two alternating updates for f and g. At the two ends $\varphi_1 = g$ and $\psi_0 = \pi_0 f$, so the terminal cost is $\Phi = -\varepsilon \log g$, and the control problem closes back onto the static problem we started from.

7.18 The Brownian bridge

We can calculate the bridge in closed form for a driftless Brownian reference with fixed endpoints. Start at x at time 0 and condition on $X_1 = y$. The potential h is then the reference density of arriving at y, seen from the state z at time t:

$$h(z,t) = R_{1|t}(y \mid z) \;\propto\; \exp\Big(-\frac{\|y-z\|^2}{2\,\varepsilon\,(1-t)}\Big).$$

"The backward potential $h(z,t)$ equals reference transition density $R_{1|t}(y \mid z)$ at fixed endpoint y, starting from state z at time t. It is proportional to the exponential of minus squared remaining displacement $\|y-z\|^2$, divided by twice the remaining noise variance $\varepsilon(1-t)$."

That is the Gaussian density with mean z and variance $\varepsilon(1-t)$ in each coordinate, evaluated at y, since a driftless diffusion run for the remaining time $1-t$ accumulates that much variance.

Let's verify that h satisfies the backward equation required by the transform (Section 7.17.3). We work in one dimension, write $s = 1-t$ for the remaining time, and take the full density $h = (2\pi\varepsilon s)^{-1/2}\exp(-(y-z)^2/(2\varepsilon s))$. Differentiating the normalization and the exponent in turn,

$$\partial_s h = h\Big[-\frac{1}{2s} + \frac{(y-z)^2}{2\varepsilon s^2}\Big], \qquad \partial_z h = h\,\frac{y-z}{\varepsilon s}, \qquad \partial_z^2 h = h\Big[\frac{(y-z)^2}{\varepsilon^2 s^2} - \frac{1}{\varepsilon s}\Big].$$

Since $\partial_t = -\partial_s$, the two terms of the backward equation with $b = 0$ are

$$\partial_t h = h\Big[\frac{1}{2s} - \frac{(y-z)^2}{2\varepsilon s^2}\Big], \qquad \frac{\varepsilon}{2}\,\partial_z^2 h = h\Big[\frac{(y-z)^2}{2\varepsilon s^2} - \frac{1}{2s}\Big],$$

which cancel term by term, so $\partial_t h + \frac{\varepsilon}{2}\nabla^2 h = 0$. The reference transition density, read as a function of the earlier state and the earlier time, always solves the backward equation, which is why conditioning on a fixed endpoint is a special case of the h-transform.

Taking ε times the gradient of the logarithm of h with respect to z, the normalizing constant drops out and the quadratic differentiates to

$$\varepsilon\,\nabla_z \log h(z,t) = \frac{y-z}{1-t}.$$

"Multiplying the log-gradient $\nabla_z \log h(z,t)$ of the backward potential by noise variance ε gives the remaining displacement $y-z$ to endpoint y, divided by remaining time $1-t$."

The noise level cancels, leaving the velocity that would reach y exactly on time if nothing further perturbed the state. Added to the zero reference drift, it produces the bridge, together with its marginal law at every time:

$$dX_t = \frac{y-X_t}{1-t}\,dt + \sqrt{\varepsilon}\,dB_t \qquad X_t \sim \mathcal{N}\big((1-t)\,x + t\,y,\quad \varepsilon\,t(1-t)\big).$$

"The Brownian bridge state X_t has drift $(y-X_t)/(1-t)$, the remaining displacement to endpoint y divided by remaining time, and noise $\sqrt{\varepsilon}\,dB_t$. Its time-t law is Gaussian with mean $(1-t)x+ty$, interpolating from initial point x to endpoint y, and variance $\varepsilon t(1-t)$."

The variance $\varepsilon t(1-t)$ vanishes at both ends and peaks at $t=1/2$, where it equals $\varepsilon/4$. The paths fan out from the start, reach their widest spread halfway, and close back onto y, and the width of the fan scales as $\sqrt{\varepsilon}$ (Figure 7.5). At $t=1/2$ the standard deviations for $\varepsilon = 0.25$, 1 and 4 are 0.25, 0.5 and 1.

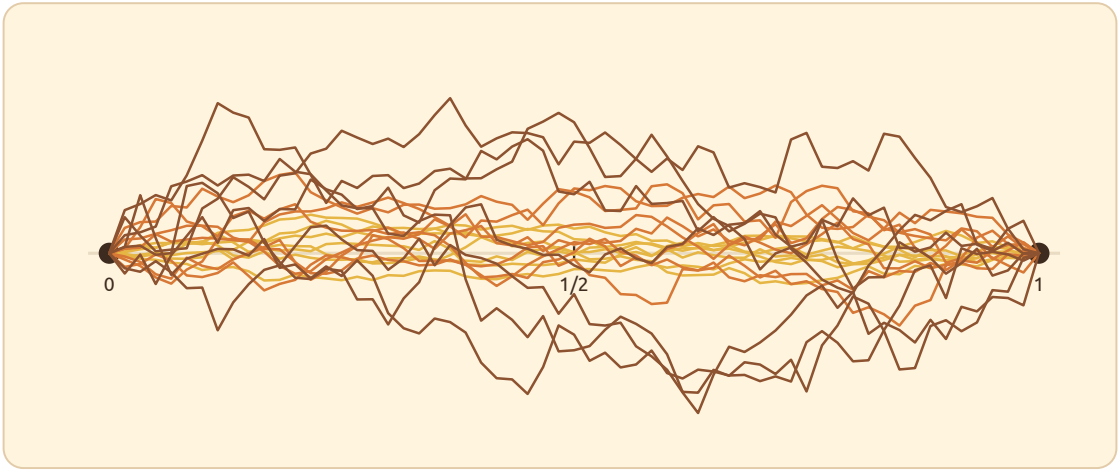


Figure 7.5: Sample paths of the Brownian bridge pinned at the two black points, at time 0 on the left and time 1 on the right, drawn at three reference noise levels. The pale paths have the smallest ε and the dark paths the largest, and at every noise level the spread of the paths is widest at $t=1/2$, where the variance $\varepsilon t(1-t)$ reaches $\varepsilon/4$, and closes to zero at both ends. The horizontal line is the straight interpolation $(1-t)x+ty$, which is the mean of every bridge.

With $x=0$, $y=2$, $\varepsilon=1$ and $t=0.25$, the mean is $0.75\times 0 + 0.25\times 2 = 0.5$ and the variance is $1\times 0.25\times 0.75 = 0.1875$, so the standard deviation is 0.4330. At the particular state $X_{0.25}=0.6732$, the drift is

$$\frac{2-0.6732}{1-0.25} = \frac{1.3268}{0.75} = 1.7691,$$

smaller than the straight-line velocity $y - x = 2$, the value the drift would have at the mean 0.5. The state is 0.1732 above the interpolating line, so less displacement is left to cover in the remaining three quarters of the interval, and the drift needed to arrive on time is correspondingly lower.

7.19 Learning the bridge from samples

Our explicit calculations required reference transition probabilities on a state grid. We used the matrix R_{01} for scaling and A for the backward recursion. Computing the continuous potentials φ and ψ requires solving two partial differential equations. For images, molecules, or protein sequences, we cannot construct such a grid, and we only have samples from π_0 and π_1. We can learn the bridge with neural networks by choosing which paths to simulate and which quantities to regress.

7.19.1 Diffusion Schrödinger Bridge

In Diffusion Schrödinger Bridge we run iterative proportional fitting directly on path laws [De Bortoli et al., 2021]. The two updates of Section 7.14 fixed one margin at a time; lifted to path space, each half-step is itself a relative-entropy projection onto the set of path laws with one marginal correct. Starting from $P^0 = R$,

$$P^{2n+1} = \arg\min_P \left\{ D_{\mathrm{KL}}\Big(P \,\Big\|\, P^{2n}\Big) \;:\; P_1 = \pi_1 \right\}$$

$$P^{2n+2} = \arg\min_P \left\{ D_{\mathrm{KL}}\Big(P \,\Big\|\, P^{2n+1}\Big) \;:\; P_0 = \pi_0 \right\}$$

"The odd path-law iterate P^{2n+1} minimizes relative entropy from P^{2n} while matching final law π_1. The even iterate P^{2n+2} minimizes relative entropy from P^{2n+1} while matching initial law π_0, where n indexes fitting rounds."

We can solve each half-step in closed form using the chain rule. We apply it with the constrained end in the place of x and everything else in the place of y. The constraint binds only the marginal term, the conditional term is nonnegative and free, and when we set it to zero the conditional law of the previous iterate is unchanged:

$$P^{2n+1}(x_{0:N}) = \pi_1(x_N)\; P^{2n}(x_{0:N-1} \mid x_N).$$

"The updated path probability $P^{2n+1}(x_{0:N})$ equals the required final-state probability $\pi_1(x_N)$ times the previous iterate's conditional probability $P^{2n}(x_{0:N-1} \mid x_N)$ for all earlier states given endpoint x_N."

The conditional law on the right is the current process run backward in time from its final state. Each half-step therefore amounts to reversing the current process and restarting it from the requested law at the other end.

The half-steps on the two-state chain. Let's run two half-steps on the chain of Section 7.14.2 and compare with the answer already computed. The reference started from π_0 has marginals $(0.5, 0.5)$, $(0.55, 0.45)$ and $(0.565, 0.435)$ at the three time points. To reverse the last step we use Bayes' rule:

$$B_2(x_2, x_1) = \frac{p_1(x_1)\, A(x_1, x_2)}{p_2(x_2)}, \qquad B_2 = \begin{pmatrix} 0.681416 & 0.318584 \\ 0.379310 & 0.620690 \end{pmatrix}.$$

"The backward transition probability $B_2(x_2, x_1)$ equals the step-one marginal probability $p_1(x_1)$ times forward transition probability $A(x_1, x_2)$, divided by step-two marginal probability $p_2(x_2)$. It is the probability of earlier state x_1 given later state x_2."

As one entry, $B_2(1,1) = 0.55 \times 0.7/0.565 = 0.681416$, so $B_2(1,2) = 0.318584$. From state 2 the two backward probabilities are 0.379310 and 0.620690. The earlier step reverses the same way, with $B_1(1,1) = 0.5 \times 0.7/0.55 = 0.636364$.

We now start the backward chain from the requested terminal law, in place of the reference marginal it would otherwise begin at:

$$(0.2,\ 0.8) \ \longrightarrow\ (0.439731,\ 0.560269) \ \longrightarrow\ (0.466585,\ 0.533415).$$

After one backward step the law is $(0.439731, 0.560269)$ and after two it is $(0.466585, 0.533415)$. The marginal at the last step is now exactly $(0.2, 0.8)$, as the half-step requires, and the law at step 0 has drifted away from $(0.5, 0.5)$ by 0.0668 in total absolute error. The second half-step reverses this backward chain and starts the result from π_0, which produces the forward matrices

$$F_0 = \begin{pmatrix} 0.599738 & 0.400262 \\ 0.299771 & 0.700229 \end{pmatrix} \qquad F_1 = \begin{pmatrix} 0.309924 & 0.690076 \\ 0.113725 & 0.886275 \end{pmatrix},$$

and the terminal law $(0.201967, 0.798033)$, off by 0.0039. One full round has brought the free end to within four parts in a thousand, and the two matrices are already close to the tilted matrices A_0^h and A_1^h of Section 7.15.1, entry by entry within 0.003.

The error 0.0039 is the entry already recorded at round 1 of Table 7.3, and the agreement is exact at every later round as well. Each half-step on path laws keeps the conditional law of the previous iterate and resets one marginal, which, read on the endpoint pair alone, is one of the two updates for f and g. Proportional fitting on path laws and proportional fitting on the endpoint coupling are therefore the same iteration, run on two different representations of the same sequence of path laws. The error at the free end falls by the same factor of 187.5 per round, and three rounds reach 10^{-7}.

Learning the reverse process. On a finite state space the reversal was Bayes' rule on a matrix. On $\mathbb{R}^d$ it is the time reversal of a diffusion, recalled from Chapter 3: for a diffusion with drift b, noise variance ε and marginal density p_t, the process run backward in time is a diffusion with drift

$$\beta(x,t) = -\,b(x,t) + \varepsilon\,\nabla \log p_t(x).$$

"The reverse drift $\beta(x,t)$ equals negative forward drift $-b(x,t)$, plus reference noise variance ε times score $\nabla \log p_t(x)$, the log-gradient of the current marginal density p_t."

The score of the current iterate has no closed form, because the current iterate is itself the output of the previous half-step. We can therefore simulate paths of the current process on a grid of N steps and fit the mean of each reversed step by least squares, training one network per direction of time and alternating between them. Each half-step requires a full simulation of the current process and a full training run. When the reference is a noising process started from the data, the first half-step is the training of an ordinary score-based diffusion model, and the later rounds correct it toward the bridge.

7.19.2 Diffusion Schrödinger Bridge Matching

Proportional fitting enforces one marginal at a time, and the two constraints hold together at convergence. We can preserve both marginals throughout by alternating between two properties of the path law [Shi et al., 2023].

The bridge is an endpoint coupling filled in with reference bridges, and we give a name to every path law of that shape:

$$\Pi = \Pi_{0,1}\ R_{\cdot|0,1}.$$

"The mixture path law Π samples an endpoint pair (x_0, x_1) from coupling $\Pi_{0,1}$, then samples the reference bridge $R_{\cdot|0,1}$ conditional on those endpoints."

The set of all such laws is the reciprocal class, written $\mathcal{R}$, and its members differ only in the coupling. The symbol Π denotes one member of it and $\Pi_{0,1}$ that member's endpoint coupling; the set of couplings with two prescribed marginals keeps its arguments, $\Pi(\pi_0, \pi_1)$, as in Section 7.14.

For a Brownian reference, each bridge is the Brownian bridge of Section 7.18 ending at the drawn point x_1:

$$dX_t = \frac{x_1 - X_t}{1 - t}\, dt + \sqrt{\varepsilon}\, dB_t.$$

The drift depends on x_1, which is not a function of the current state, so a mixture of bridges is in general not a Markov process. Two members of the reciprocal class with the same marginals at every time can still assign different probabilities to the future given the present, because the endpoint contains information that the current state does not.

The Markov projection. To obtain a drift that depends only on the current state and time, we average over the endpoints of all bridges passing through that state,

$$v^\star(x, t) = \mathbb{E}_\Pi\Big[\frac{X_1 - x}{1 - t} \;\Big|\; X_t = x\Big].$$

"The Markov drift $v^\star(x, t)$ averages conditional bridge drift $(X_1 - x)/(1 - t)$ over final endpoints X_1, given current state $X_t = x$ under mixture law Π."

The Markov projection of Π is the diffusion with that drift:

$$\mathrm{proj}_{\mathcal{M}}(\Pi): \quad dX_t = v^\star(X_t, t)\, dt + \sqrt{\varepsilon}\, dB_t.$$

Its drift depends only on the current state and time, so it is Markov. It has the same marginal as Π at every t, since the average drift reproduces the same flux of probability, and among all Markov diffusions M with the reference noise it minimizes $D_{\mathrm{KL}}(\Pi \,\|\, M)$. Its endpoint coupling is in general different from $\Pi_{0,1}$, because averaging the drifts mixes the pairings.

The Markov drift at one point. Suppose two bridges pass through $x = 1$ at $t = 0.5$ with equal probability, one ending at $x_1 = 3$ and the other at $x_1 = 0$. Their drifts at that point and the average are

$$\frac{3 - 1}{1 - 0.5} = 4 \qquad \frac{0 - 1}{1 - 0.5} = -2 \qquad v^\star(1,\ 0.5) = \tfrac{1}{2}(4) + \tfrac{1}{2}(-2) = 1.$$

A least-squares fit gives the same value. The average squared error of a single prediction a against the two targets,

$$\tfrac{1}{2}(a-4)^2 + \tfrac{1}{2}(a+2)^2,$$

has derivative $(a-4)+(a+2) = 2a-2$, which vanishes at $a=1$. This is the crossing-path computation of Chapter 2 with bridge drifts in place of conditional velocities [Lipman et al., 2023], so a drift network trained by regression on bridge drifts has the Markov drift as its minimizer. The projection is then implementable without a solver: draw pairs from the coupling, sample a time, sample the bridge at that time, and regress.

The reciprocal projection. We can also take a Markov law M and replace its bridges by reference bridges, keeping its endpoint coupling:

$$\mathrm{proj}_{\mathcal{R}}(M) = M_{0,1}\, R_{\cdot|0,1}.$$

"The reciprocal projection $\mathrm{proj}_{\mathcal{R}}(M)$ preserves the endpoint coupling $M_{0,1}$ of Markov law M and uses reference conditional law $R_{\cdot|0,1}$ for the interior path between each endpoint pair."

By the chain rule of Section 7.13.2, this is the member of the reciprocal class with the smallest $D_{\mathrm{KL}}(M \,\|\, \Pi)$, by the same argument that put reference bridges inside the Schrödinger bridge. In practice we simulate M once, store the endpoint pairs its paths produce, and draw reference bridges between them.

Alternating between the two sets. The Markov laws and the reciprocal class are two sets of path laws, and the bridge is the unique law in their intersection with both marginals correct (Figure 7.6). By alternating the two projections we can drive an arbitrary starting law to it:

$$P^{2n+1} = \mathrm{proj}_{\mathcal{M}}(P^{2n}) \qquad P^{2n+2} = \mathrm{proj}_{\mathcal{R}}(P^{2n+1}).$$

"At round n, the odd iterate P^{2n+1} is the Markov projection $\mathrm{proj}_{\mathcal{M}}$ of current law P^{2n}. The next even iterate P^{2n+2} is the reciprocal projection $\mathrm{proj}_{\mathcal{R}}$ of that odd iterate."

We call this alternation iterative Markovian fitting. A Markov projection preserves every time marginal, and a reciprocal projection preserves the endpoint coupling. Both π_0 and π_1 therefore remain exact at every iterate if the starting law has them. In proportional fitting only one of them is exact at a time. When we start the alternation from the independent coupling with reference bridges, every intermediate model is therefore already transporting π_0 to π_1, and the iterations improve only the coupling.

One round on the two-state chain. We start from the independent coupling of $\pi_0 = (0.5, 0.5)$ and $\pi_1 = (0.2, 0.8)$:

$$\Pi^0_{0,1} = \begin{pmatrix} 0.1 & 0.4 \\ 0.1 & 0.4 \end{pmatrix},$$

whose entries are $0.5 \times 0.2 = 0.1$ in the first column and $0.5 \times 0.8 = 0.4$ in the second. Between each pair of ends we draw the middle state from the reference bridge $R_{|01}$ of Section 7.15.3. From $x_0 = 1$ to $x_2 = 1$ the middle state is 1 with probability $0.7 \times 0.7/0.61 = 0.803279$, and from $x_0 = 1$ to $x_2 = 2$ it is 1 with probability $0.7 \times 0.3/0.39 = 0.538462$.

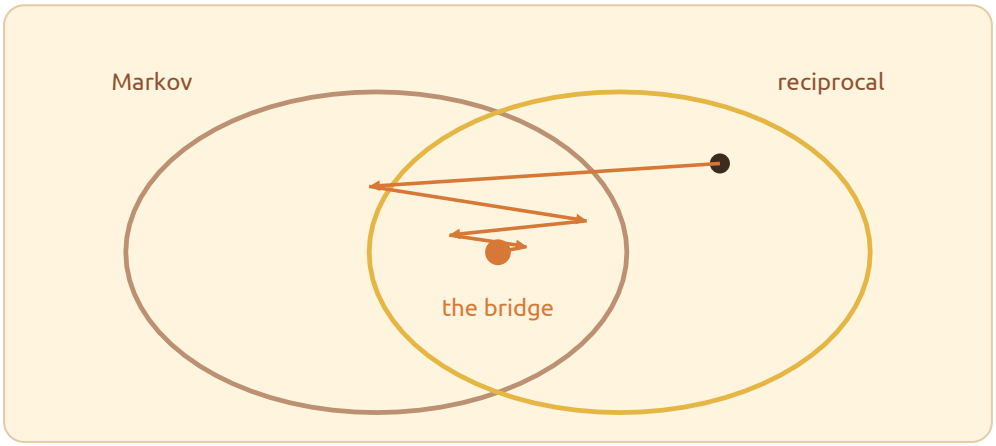


Figure 7.6: Iterative Markovian fitting as an alternating projection. The left region is the set of Markov path laws with the reference noise, the right region is the reciprocal class of mixtures of bridges, and the Schrödinger bridge, the orange point, lies in their intersection. Starting from the black point, each arrow is one projection, onto the Markov set or onto the reciprocal class in turn, and the iterates converge to the bridge.

For the Markov projection we compute the two one-step conditionals of the resulting three-state law:

$$M_0 = \begin{pmatrix} 0.591425 & 0.408575 \\ 0.307692 & 0.692308 \end{pmatrix} \qquad M_1 = \begin{pmatrix} 0.298457 & 0.701543 \\ 0.119588 & 0.880412 \end{pmatrix}.$$

Here M_0 is the conditional law of the middle state given the first, and M_1 the conditional law of the last state given the middle. From the first state 1, the independent coupling has the last state at 1 with probability 0.2 and at 2 with probability 0.8, and the two bridge probabilities above then apply, so

$$M_0(1,1) = 0.2 \times 0.803279 + 0.8 \times 0.538462 = 0.591425.$$

The Markov projection is the chain with these two matrices, and the averaging in that line is the discrete form of the drift average $v^\star$.

If we multiply them out with π_0 on the diagonal, we get the endpoint coupling of the projected chain, the coupling that the next reciprocal projection holds fixed:

$$\operatorname{diag}(\pi_0)\, M_0\, M_1 = \begin{pmatrix} 0.112688 & 0.387312 \\ 0.087312 & 0.412688 \end{pmatrix}.$$

Its row sums are 0.5 and 0.5 and its column sums are 0.2 and 0.8, exactly, as the preservation property requires. We can compare this coupling with $\pi^\star$ from Section 7.14.2, whose entries are 0.114605, 0.385395, 0.085395, and 0.414605. One round reduces the total absolute gap from 0.0584 for the independent coupling to 0.0077. Each subsequent round reduces it by a factor of 7.37. Nine rounds reach 10^{-9}. The convergence is slower than the 187.5 per round of proportional fitting on this problem, and each round is far cheaper, since the simulation produces endpoint pairs alone.

In Diffusion Schrödinger Bridge Matching we fit the Markov projection alternately backward from π_1 and forward from π_0, and the two initializations have the names DSBM-IPF and DSBM-IMF. Started from the reference coupling R_{01}, the iterates coincide with the proportional fitting iterates of Section 7.19.1. Started from the independent coupling, they are the iterative Markovian fitting

iterates worked above. In both cases the simulation happens once per round, to produce the endpoint pairs, and every other step of the round is a regression on sampled bridges.

7.19.3 Simulation-Free Score and Flow Matching

DSB and DSBM compute the bridge coupling iteratively. If we can compute that coupling directly, a single regression stage suffices. Combining the chain-rule decomposition with the Brownian case of Section 7.14.1, the bridge of a Brownian reference of variance ε is

$$P^\star = \int R_{\cdot|0,1}(\cdot \mid x_0, x_1)\; d\pi^{2\varepsilon}(x_0, x_1),$$

"The bridge law $P^\star$ averages Brownian reference bridges $R_{\cdot|0,1}(\cdot \mid x_0, x_1)$ over endpoint pairs (x_0, x_1) from entropic coupling $\pi^{2\varepsilon}$, computed with squared-distance cost and regularization strength 2ε."

In Simulation-Free Score and Flow Matching we compute that plan directly, with an entropic transport solver run on each minibatch, and then train two networks by regression on quantities available in closed form [Tong et al., 2024].

One sample from the bridge. The Brownian bridge at time t is Gaussian, so a sample costs one normal draw:

$$m_t = (1-t)\, x_0 + t\, x_1 \qquad \sigma_t = \sqrt{\varepsilon\, t(1-t)} \qquad x = m_t + \sigma_t\, z, \quad z \sim \mathcal{N}(0, I).$$

"The bridge mean m_t interpolates linearly between endpoints x_0 and x_1, and its standard deviation σ_t equals $\sqrt{\varepsilon t(1-t)}$, where ε is the reference noise variance. We draw bridge sample x by adding σ_t times standard normal vector $z \sim \mathcal{N}(0, I)$ to the mean, with I the identity covariance."

We sample directly at time t, so training is simulation-free. At that sample, both the bridge drift and the score of the bridge marginal are available in closed form:

$$u_t(x \mid x_0, x_1) = \frac{x_1 - x}{1-t} \qquad \nabla \log p_t(x \mid x_0, x_1) = \frac{m_t - x}{\varepsilon\, t(1-t)}.$$

"The conditional bridge drift $u_t(x \mid x_0, x_1)$ equals remaining displacement $x_1 - x$ divided by remaining time $1-t$. The conditional score $\nabla \log p_t(x \mid x_0, x_1)$ equals displacement $m_t - x$ toward bridge mean m_t, divided by bridge variance $\varepsilon t(1-t)$."

Both are conditional on the drawn pair, as the bar in the notation marks, and the first is the control of Section 7.16 for the particular bridge that ends at x_1. Neither depends on any quantity a network has to estimate first.

Bridge probability-flow velocity. The drift above generates the bridge as a stochastic process. Chapter 3 established that subtracting half the noise variance times the score converts a diffusion drift into the velocity of the probability flow, which has the same marginals with no noise at all. Applying that here,

$$u_t^\circ = u_t - \frac{\varepsilon}{2} \nabla \log p_t = (x_1 - x_0) + \frac{1-2t}{2\, t(1-t)}\, (x - m_t).$$

"The probability-flow velocity u_t° subtracts half the reference noise variance ε times conditional score $\nabla \log p_t$ from bridge drift u_t. This equals straight-line velocity $x_1 - x_0$, plus coefficient $(1-2t)/(2t(1-t))$ times sample displacement $x - m_t$ from the bridge mean."

The second form follows by writing $\delta = x - m_t$ and substituting. Since $x_1 - x = (1-t)(x_1 - x_0) - \delta$, the drift is $(x_1 - x_0) - \delta/(1-t)$, and minus half the score adds $+\delta/(2t(1-t))$; the two coefficients of δ sum to $(1-2t)/(2t(1-t))$. The coefficient is positive before $t = 1/2$ and negative afterward. Samples therefore move away from the interpolating line early in the trajectory and return toward it later. This deterministic motion reproduces a distribution that first spreads and then contracts. The dependence on ε has cancelled, so the flow target is the straight-line velocity plus a correction that depends only on where the sample landed.

The two targets for one pair. Let's take $x_0 = 0$, $x_1 = 2$, $\varepsilon = 1$, $t = 0.25$ and the draw $z = 0.4$:

$$m_t = 0.5 \qquad \sigma_t = \sqrt{0.25 \times 0.75} = 0.4330 \qquad x = 0.5 + 0.4330 \times 0.4 = 0.6732.$$

The mean is one quarter of the way from 0 to 2. The variance is $0.25 \times 0.75 = 0.1875$, whose square root is 0.4330, and the sample is 0.1732 above the mean. At that sample,

$$u_t = \frac{2 - 0.6732}{0.75} = 1.7691 \qquad \nabla \log p_t = \frac{0.5 - 0.6732}{0.1875} = -0.9238,$$

the drift being the remaining displacement 1.3268 over the remaining time 0.75, and the score being the displacement -0.1732 back toward the mean divided by the variance 0.1875. The two regression targets follow:

$$u_t^\circ = 1.7691 - \tfrac{1}{2}(-0.9238) = 2.2309 \qquad \sigma_t \nabla \log p_t = 0.4330 \times (-0.9238) = -0.4.$$

The flow target is the straight-line velocity 2 plus 0.2309, the correction produced by the displacement 0.1732 at $t = 0.25$, where the coefficient $(1-2t)/(2t(1-t))$ equals $4/3$. The scaled score target is exactly $-z$, the negative of the normal draw, which holds at every time because $\sigma_t \nabla \log p_t = (m_t - x)/\sigma_t = -z$.

Repeating the computation at $t = 0.75$ with the same pair and normal draw changes the sign of the velocity correction. The mean is 1.5, the variance is $0.75 \times 0.25 = 0.1875$ again, and the sample is $x = 1.6732$, the same distance 0.1732 above the mean. The drift is $(2 - 1.6732)/0.25 = 1.3072$ and the score is again -0.9238, so

$$u_{0.75}^\circ = 1.3072 + \tfrac{1}{2}(0.9238) = 1.7691 = 2 - 1.3333 \times 0.1732.$$

The coefficient has changed sign, from $+4/3$ to $-4/3$, and the two flow targets 2.2309 and 1.7691 lie symmetrically about the straight-line velocity 2. Averaged over the normal draw at a fixed time, the correction has mean zero, since $x - m_t = \sigma_t z$ and z is centered, so the regression target is the straight-line velocity plus a correction proportional to the sample displacement from the mean.

The loss and the sampler. We train two networks on these two targets:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big\|v_\theta(t,x) - u_t^\circ(x \mid x_0, x_1)\big\|^2 + \lambda(t)^2 \big\|s_\theta(t,x) - \nabla \log p_t(x \mid x_0, x_1)\big\|^2\Big].$$

"The loss $\mathcal{L}(\theta)$ averages squared velocity error between prediction $v_\theta(t,x)$ and conditional flow target $u_t^\circ(x \mid x_0, x_1)$, plus squared score error between prediction $s_\theta(t,x)$ and conditional score

$\nabla \log p_t(x \mid x_0, x_1)$, weighted by $\lambda(t)^2$, over uniform times t, coupled endpoints (x_0, x_1), and bridge samples x."

With $\lambda(t) = \sigma_t$, the score target is $-z$, which has unit scale at every time, so no part of the schedule dominates the gradient. Both regressions are conditional, and by the argument of Chapter 2 they have the same gradient as regression on the marginal velocity and the marginal score [Lipman et al., 2023]. At sampling time we fix a noise level $g(t)$ and integrate

$$dX_t = \Big[v_\theta(t, X_t) + \tfrac{1}{2}\, g(t)^2\, s_\theta(t, X_t)\Big]\, dt + g(t)\, dB_t.$$

"The sampled state X_t changes by predicted flow velocity $v_\theta(t, X_t)$ plus half the sampling variance $g(t)^2$ times predicted score $s_\theta(t, X_t)$, multiplied by dt, and a Brownian increment dB_t scaled by sampling noise $g(t)$."

The noise level $g(t)$ is the noise scale of Chapter 3, written with its time argument so that it is not confused with the Schrödinger potential g of Section 7.14.

If we set $g^2 = \varepsilon$ we recover a diffusion with the reference noise, and if we set $g = 0$ we are left with the probability flow ordinary differential equation, with the same marginals at every time. One training run therefore supports the whole family of samplers between the two.

The role of the coupling. The endpoint coupling determines the learned process. If we draw the pairs from the entropic plan at strength 2ε, the learned stochastic differential equation at $g^2 = \varepsilon$ is the Schrödinger bridge. With pairs from any other coupling of π_0 and π_1, the learned equation is the Markov projection of that mixture of bridges, which still starts at π_0 and ends at π_1 without minimizing relative entropy from the reference. We compute the plan on a minibatch, so it approximates the full plan, and exact transport on each minibatch is the usual choice. On a batch with source points 0 and 2 and target points 1 and 3, exact transport pairs 0 with 1 and 2 with 3 at cost $1 + 1 = 2$, and puts no mass on the crossed pairing, whose cost is $9 + 1 = 10$. The bridges drawn between those pairs are short, and the velocity field regressed on them is close to constant.

7.20 Bridge-learning methods

All three methods target the h-transform of the reference with the two requested marginals. They differ in what we compute before training and what we learn with a network (Table 7.5). In Diffusion Schrödinger Bridge, we learn the coupling through iteration, simulating a process to reverse at each half-step. In Diffusion Schrödinger Bridge Matching, both marginals stay fixed while we improve the pairing, so each round requires only endpoint pairs from the current model. In Simulation-Free Score and Flow Matching, we compute the coupling with a transport solver before training. Its inner loop requires no forward simulation.

Proportional fitting on path laws assumes only that the current process can be reversed. Markovian fitting assumes in addition that reference bridges can be sampled in closed form, which holds for the Brownian bridge of Section 7.18. Simulation-free training assumes further that the entropic plan can be solved on a minibatch, which Sinkhorn's algorithm from the first half of this chapter does at a cost quadratic in the batch size. With each additional assumption, one more simulation drops out of the inner loop.

Table 7.5: The three methods for learning a Schrödinger bridge between two distributions on $\mathbb{R}^d$. The coupling column names where the endpoint pairs come from, the simulation column what has to be run forward before a gradient step can be taken, and the regression column what the network is fitted to. All three use the same reference noise ε and reach the same bridge, the first two in the limit of many rounds and the third when the entropic plan is solved on the whole of the two laws.

method	endpoint coupling	simulated per round	regression target
DSB	one margin per half-step	full paths	mean of the reversed step
DSBM	kept by $\mathrm{proj}_{\mathcal{R}}$	endpoint pairs	bridge drift
SF2M	entropic plan on a batch	nothing	flow velocity and score

All three methods learn a bridge between distributions over $\mathbb{R}^d$, using vector-field controls, Brownian noise, Gaussian bridges, and gradient scores. On a finite set, we cannot use ∇, $\mathcal{N}$, or dB_t directly. We can still apply the chain rule, projections, and the h-transform to jump rates.

7.21 Finite-state bridges

The three continuous methods learn a drift, which specifies a small displacement in a chosen direction. DNA sequences, peptides, molecular graphs, and token strings have finite state spaces. We change these states through jumps, so we need corresponding bridge constructions for jump processes.

For sequences, the state space is $\mathcal{S} = \mathcal{V}^L$, with $K = |\mathcal{V}|$ tokens at each of L positions, so a peptide of 30 residues over the 20 amino acids already has $20^{30} \approx 1.1 \times 10^{39}$ states. The formulas use sums over states that we cannot evaluate explicitly at this scale. Factorization over positions replaces each sum over $\mathcal{S}$ with L sums of K terms.

Over a finite set, we use a continuous-time Markov chain as the reference, as in Chapter 4. Its paths are piecewise constant, staying on one state for a random wait and then moving to another state, and the matrix of jump rates controls them. Three properties fix that matrix,

$$Q_t(x, y) \geq 0 \quad (x \neq y), \qquad \sum_y Q_t(x, y) = 0, \qquad \partial_t\, p_t(x) = \sum_y Q_t(y, x)\, p_t(y).$$

"The generator Q_t has nonnegative jump rates $Q_t(x, y)$ for distinct states x and y, and zero row sums. The probability change $\partial_t p_t(x)$ sums generator entries $Q_t(y, x)$ weighted by state probabilities $p_t(y)$ over states y."

Chapter 4 wrote this matrix R_t. Here R denotes the reference path law and R_{01} its endpoint coupling, so we use Q for the rate matrix. The entry $Q_t(x, y)$ is the quantity called $R_t(x, y)$ in Chapter 4, measured in probability per unit time. The third property is the master equation, and it is the finite-state counterpart of the Fokker-Planck equation that governed every density in Chapter 3.

7.21.1 Relative entropy between two jump processes

We compare a candidate path law with the reference through relative entropy. For diffusions, Girsanov's formula gave the integrated squared drift difference. For a finite state space, we can derive the corresponding expression from the jump rates, one step at a time.

Fix a small step h and a current state x. Under the controlled law P^θ the chain moves to a different state y with probability $h\,Q_t^\theta(x,y) + O(h^2)$ and stays at x with the remaining probability. We write $a = \sum_{y\neq x} Q_t^\theta(x,y)$ and $b = \sum_{y\neq x} Q_t(x,y)$ for the two total jump rates out of x. The relative entropy of the two one-step laws is then

$$\sum_{y\neq x} h\,Q_t^\theta(x,y)\log\frac{h\,Q_t^\theta(x,y)}{h\,Q_t(x,y)} \;+\; (1-ha)\log\frac{1-ha}{1-hb}$$

$$=\; h\sum_{y\neq x} Q_t^\theta(x,y)\log\frac{Q_t^\theta(x,y)}{Q_t(x,y)} \;+\; h\,(b-a) \;+\; O(h^2).$$

The step size h cancels inside the first logarithm, which is why a ratio of rates remains in the limit. In the second term, $\log(1-ha) - \log(1-hb) = h(b-a) + O(h^2)$, and the prefactor $1-ha$ differs from 1 at order h, so its effect is only at order h^2. If we divide by h and add up the steps along a path, the sum becomes a time integral,

$$D_{\mathrm{KL}}\Big(P^\theta \,\Big\|\, R\Big) = \mathbb{E}_{P^\theta}\int_0^1 \sum_{y\neq X_t}\Big(Q_t^\theta\log\frac{Q_t^\theta}{Q_t} - Q_t^\theta + Q_t\Big)(X_t,y)\,dt.$$

"The path relative entropy $D_{\mathrm{KL}}\Big(P^\theta \,\Big\|\, R\Big)$ averages an integrated rate divergence over controlled paths from P^θ. At each current state X_t, we sum controlled rate $Q_t^\theta(X_t,y)$ times the log ratio to reference rate $Q_t(X_t,y)$, minus that controlled rate and plus the reference rate, over destinations $y \neq X_t$."

The expression in parentheses is the Bregman divergence of the scalar $u \mapsto u\log u$ between the two rates, which is the rate error Chapter 4 used to train a reverse generator. It is nonnegative for every pair of positive rates, and it vanishes exactly when $Q_t^\theta(x,y) = Q_t(x,y)$. If we set $Q^\theta = c\,Q$ for a constant c, the integrand is $Q(c\log c - c + 1)$ per edge, which is positive for every $c \neq 1$. The function is not symmetric in c. Halving a rate costs $0.5\log 0.5 - 0.5 + 1 = 0.1534$ per unit of reference rate, and doubling it costs $2\log 2 - 2 + 1 = 0.3863$, so under this cost a control that slows the reference down is cheaper than one that speeds it up by the same factor. We can therefore compute path-space relative entropy by integrating a pointwise rate divergence. This gives a regression objective for a network, analogous to the control-energy objective obtained from Girsanov's formula.

7.21.2 The Schrödinger problem for a chain

We minimize relative entropy from the reference chain over path laws with the required endpoint marginals,

$$P^\star = \arg\min_P \big\{D_{\mathrm{KL}}(P\,\|\,R) \;:\; P_0 = \pi_0,\;\; P_1 = \pi_1\big\}.$$

"The bridge path law $P^\star$ minimizes relative entropy $D_{\mathrm{KL}}(P\,\|\,R)$ from reference chain law R, subject to candidate initial marginal P_0 equaling source law π_0 and final marginal P_1 equaling target law π_1."

The chain-rule decomposition requires only that we factor the path law into an endpoint law and an interior law conditional on the endpoints. On a finite set, we replace the integral with a sum over

paths. We can again set the interior relative entropy to zero by using the reference bridges. We then minimize over endpoint couplings,

$$P^\star(\cdot) = \int R_{\cdot|0,1}(\cdot \mid x_0, x_1)\, d\pi^\star(x_0, x_1), \qquad \pi^\star(x_0, x_1) = f(x_0)\, R_{01}(x_0, x_1)\, g(x_1).$$

"The bridge law $P^\star$ averages reference conditional paths $R_{\cdot|0,1}(\cdot \mid x_0, x_1)$ over endpoint pairs from coupling $\pi^\star$. That coupling multiplies reference endpoint law $R_{01}(x_0, x_1)$ by source potential $f(x_0)$ and target potential $g(x_1)$."

The two potentials f and g are the Sinkhorn scalings of the matrix R_{01} to the marginals π_0 and π_1. For a chain on S states, R_{01} is the $S \times S$ matrix $\mathrm{diag}(\pi_0^{\mathrm{ref}})\, e^Q$ when the rates do not depend on time, where e^Q is the matrix exponential of the generator over one unit of time and π_0^{ref} is the reference initial law. Scaling cancels that initial law, so only the transition matrix matters.

To sample the bridge forward in time, we need jump rates computed from the potentials (f, g).

7.22 Rates reweighted by an h-transform

We can obtain these rates using Doob's h-transform, derived for jump processes in Chapter 4 and used for guidance in Chapter 5. First, propagate the terminal potential backward under the reference,

$$\varphi_t(x) = \sum_y R_{1|t}(y \mid x)\, g(y).$$

"The backward potential $\varphi_t(x)$ averages terminal potential $g(y)$ over final states y, weighted by reference transition probabilities $R_{1|t}(y \mid x)$ from current state x at time t to time one."

We used h_t for the corresponding diffusion potential. Here we write φ_t, following the discrete literature. Its two ends are $\varphi_1 = g$ and $\varphi_0 = e^Q g$, and in between it satisfies the backward equation $\partial_t \varphi_t + Q_t \varphi_t = 0$, since averaging a fixed terminal quantity over the reference makes φ space-time harmonic for the reference.

To see what the reweighting does to the rates, we write the bridge transition kernel first. The bridge is the reference conditioned to end with the law that g encodes, so its kernel from time s to time t is

$$P^\star_{st}(x, y) = R_{st}(x, y)\, \frac{\varphi_t(y)}{\varphi_s(x)}.$$

Each row of this matrix sums to one, because summing $R_{st}(x, y)\varphi_t(y)$ over y is exactly the tower property that defines $\varphi_s(x)$. Let's take $t = s + h$ and expand both factors to first order. The reference kernel is $R_{s,s+h}(x, y) = \mathbf{1}_{x=y} + h\, Q_s(x, y) + O(h^2)$, and $\varphi_{s+h}(y) = \varphi_s(y) + O(h)$. For a destination $y \neq x$ the indicator is zero, so the leading term is $h\, Q_s(x, y)\, \varphi_s(y)/\varphi_s(x)$. Dividing by h and letting h go to zero,

$$\boxed{Q^\star_t(x, y) = Q_t(x, y)\, \frac{\varphi_t(y)}{\varphi_t(x)}, \qquad x \neq y.}$$

"The bridge jump rate $Q^\star_t(x, y)$ from current state x to a distinct destination y multiplies reference rate $Q_t(x, y)$ by destination potential $\varphi_t(y)$ divided by current-state potential $\varphi_t(x)$."

The off-diagonal rates determine the diagonal entry through the zero-row-sum condition. If the reference reaches the target more often from destination y than from current state x, then $\varphi_t(y) > \varphi_t(x)$. The ratio exceeds one, increasing the jump rate and shortening the waiting time for that move. A smaller destination potential decreases the transition rate and delays that move. States that the reference can never reach have rate zero, since the tilt is multiplicative.

The h-transform has a corresponding expression in each of the three settings. In discrete time, Chapter 4 multiplied a transition probability from x to y by $h_{k+1}(y)/h_k(x)$. For a diffusion, the bridge added $\varepsilon\nabla \log \varphi_t$ to the drift, and the ratio $\varphi_t(x+\delta)/\varphi_t(x)$ becomes exactly that when the displacement δ is small and the state space is continuous. For a chain the displacement is never small, so the ratio stays a ratio and multiplies the rate directly.

In all three the reference bridges are untouched and only the endpoint pairs that occur change, which is the statement that $P^\star$ lies in the reciprocal class of R.

Proposition 7.8 (The reweighted chain has the requested marginals)**.** *Let R be a continuous-time Markov chain on a finite set, with initial law ν, transition matrix $R_{1|0}$ over the unit interval and endpoint law $R_{01}(x_0, x_1) = \nu(x_0)R_{1|0}(x_1 \mid x_0)$. Let f and g be positive functions with $\pi^\star = \operatorname{diag}(f)\, R_{01} \operatorname{diag}(g)$ having row sums π_0 and column sums π_1. Then the chain with rates $Q_t^\star$ started from π_0 has endpoint law $\pi^\star$, and in particular its law at time 1 is π_1.*

Proof. Write $\varphi_0(x_0) = \sum_{x_1} R_{1|0}(x_1 \mid x_0)\, g(x_1)$ for the potential at time 0, and recall that $\varphi_1 = g$. Over the unit interval the reweighted chain has the kernel

$$P^\star_{1|0}(x_1 \mid x_0) = R_{1|0}(x_1 \mid x_0)\, \frac{\varphi_1(x_1)}{\varphi_0(x_0)}.$$

The row sum of $\pi^\star$ at x_0 is $\pi_0(x_0) = f(x_0)\, \nu(x_0)\, \varphi_0(x_0)$, by the definition of φ_0. Multiplying it by the kernel,

$$\pi_0(x_0)\, P^\star_{1|0}(x_1 \mid x_0) = f(x_0)\, \nu(x_0)\, \varphi_0(x_0)\, \frac{R_{1|0}(x_1 \mid x_0)\, g(x_1)}{\varphi_0(x_0)} = f(x_0)\, R_{01}(x_0, x_1)\, g(x_1) = \pi^\star(x_0, x_1),$$

and summing over x_0 leaves the column sums of $\pi^\star$, which are π_1. □

The factor $\varphi_0(x_0)$ cancels because f corrects the change in starting mass introduced by the backward potential.

7.23 Two-state backward propagation from $t = 1/2$

Let's compute the bridge on two states, labeled 1 and 2, with the reference generator, source law, and target law

$$Q = \begin{pmatrix} -1 & 1 \\ 1.5 & -1.5 \end{pmatrix}, \qquad \pi_0 = (0.5,\ 0.5), \qquad \pi_1 = (0.2,\ 0.8).$$

"The reference generator Q specifies jump rate 1 from state 1 to state 2, and rate 1.5 in the reverse direction. Source law π_0 places mass 0.5 on each state, and target law π_1 requires masses 0.2 and 0.8 at time one."

The reference is in state 1 more of the time, since its stationary law is proportional to $(1.5,\ 1)$, which is $(0.6,\ 0.4)$. The target law puts more mass on state 2, so the bridge rates must correct the reference bias toward state 1.

We can start with the endpoint matrix. The reference transition matrix over one unit of time is the matrix exponential of Q, and scaling its rows by π_0 produces the reference endpoint law,

$$e^{Q} = \begin{pmatrix} 0.6328 & 0.3672 \\ 0.5507 & 0.4493 \end{pmatrix}, \qquad R_{01} = \operatorname{diag}(\pi_0)\, e^{Q} = \begin{pmatrix} 0.3164 & 0.1836 \\ 0.2754 & 0.2246 \end{pmatrix}.$$

Started at state 1, the reference is at state 1 at time 1 with probability 0.6328 and at state 2 with probability 0.3672; started at state 2, the two probabilities are 0.5507 and 0.4493. The column sums of R_{01} are 0.5918 and 0.4082, so the reference ends with far too much mass on state 1, against the 0.2 we asked for.

Running Sinkhorn on R_{01} against the two marginals converges to the scalings

$$f = (1.0716,\ 0.9373), \qquad g = (0.3349,\ 1.9643), \qquad \pi^{\star} = \begin{pmatrix} 0.1136 & 0.3864 \\ 0.0864 & 0.4136 \end{pmatrix}.$$

The row sums of $\pi^{\star}$ are 0.5 and 0.5 and the column sums are 0.2 and 0.8, as required. The terminal potential g divides the weight of paths ending at state 1 by about three and multiplies the weight of paths ending at state 2 by about two. Its ratio $1.9643/0.3349 = 5.865$ gives the relative reweighting of the second column against the first.

Next, we can carry g backwards. At $t = 1/2$ the reference transition matrix over the remaining half unit of time is $e^{Q/2}$, and applying it to g produces the potential there,

$$e^{Q/2} = \begin{pmatrix} 0.7146 & 0.2854 \\ 0.4281 & 0.5719 \end{pmatrix}, \qquad \varphi_{1/2} = e^{Q/2} g = (0.7999,\ 1.2667).$$

From state 1, the reference reaches state 2 at time 1 with probability 0.2854, so $\varphi_{1/2}(1) = 0.7146 \times 0.3349 + 0.2854 \times 1.9643 = 0.2393 + 0.5606 = 0.7999$. From state 2 the same average is $0.4281 \times 0.3349 + 0.5719 \times 1.9643 = 0.1434 + 1.1233 = 1.2667$. The expected terminal weight from state 2 at halfway is therefore about 1.58 times that from state 1.

Reweighting the two off-diagonal rates by that ratio produces the bridge generator at $t = 1/2$,

$$Q^{\star}_{1/2} = \begin{pmatrix} -1.5836 & 1.5836 \\ 0.9472 & -0.9472 \end{pmatrix},$$

since $1 \times 1.2667/0.7999 = 1.5836$ and $1.5 \times 0.7999/1.2667 = 0.9472$. The bridge jumps out of state 1 at more than one and a half times the reference rate and out of state 2 at about two thirds of it. Both changes push mass the same way, toward state 2.

The rates are not constant in time, because φ_t is not. At $t = 0$ the potential is $\varphi_0 = e^{Q} g = (0.9331,\ 1.0669)$, giving rates 1.1433 and 1.3120, which are close to the reference. At $t = 1$ the potential is g itself, giving rates 5.8652 and 0.2557. The bridge therefore starts out behaving almost

like the reference and becomes sharply biased near the end, which is the finite-state version of the picture in which bridge paths fan out in the middle and are pulled together at the terminal time.

Integrating the master equation with these rates, started from the source law, produces the marginal $(0.4668,\ 0.5332)$ at $t = 1/2$ and $(0.2000,\ 0.8000)$ at $t = 1$. The terminal law matches the requested law to the reported precision, confirming the backward potential calculation.

7.23.1 Sampling the bridge with Euler steps

We can sample the bridge with the Euler rule from Chapter 4, using its time-dependent generator. From state x at time t, draw a move to each distinct state y with probability $\Delta t\, Q_t^\star(x, y)$. The remaining probability corresponds to staying at x. On this two-state problem with $\Delta t = 0.1$, a chain at state 1 jumps out with probability 0.1143 in the first step, 0.1187 in the second and 0.1245 in the third, rising to 0.3649 in the last. A chain at state 2 jumps out with probability 0.1312 in the first step and 0.0411 in the last. The exit probability increases for state 1 and decreases for state 2, correcting the reference bias more strongly as the terminal time approaches.

If we propagate the marginal with the same steps we can measure the discretization error. Starting at $(0.5,\ 0.5)$ and applying $p \mapsto p\,(I + \Delta t\, Q_t^\star)$ ten times with $\Delta t = 0.1$ lands at $(0.2223,\ 0.7777)$, against the exact $(0.2,\ 0.8)$. Halving the step to 0.05 lands at $(0.2106,\ 0.7894)$, and halving once more lands at $(0.2051,\ 0.7949)$. The errors 0.0223, 0.0106 and 0.0051 fall by about a factor of two per halving, which is the first-order accuracy of the Euler scheme, and a hundred steps bring the terminal marginal within 0.0021 of the request. Bridge samplers incur this discretization cost. Shorter bridge paths can reduce the number of steps needed for a given accuracy.

7.23.2 Reverse bridge dynamics

For training in both time directions, we need reverse rates as well as forward rates. In Chapter 4, we derived the reverse rates by matching probability flux across each edge,

$$\hat{Q}_t(y, x) = Q_t(x, y)\, \frac{p_t(x)}{p_t(y)}.$$

"The reverse jump rate $\hat{Q}_t(y, x)$ equals forward rate $Q_t(x, y)$ multiplied by source-state probability $p_t(x)$ divided by destination-state probability $p_t(y)$, with all quantities evaluated at the same time t."

At $t = 1/2$ in this example the marginal is $(0.4668,\ 0.5332)$ and the forward rates are 1.5836 out of state 1 and 0.9472 out of state 2. The reverse rates are therefore $1.5836 \times 0.4668/0.5332 = 1.3862$ from state 2 back to state 1 and $0.9472 \times 0.5332/0.4668 = 1.0821$ from state 1 back to state 2. We can confirm the formula by checking the flux, since $0.4668 \times 1.5836 = 0.7392$ and $0.5332 \times 1.3862 = 0.7391$ are the same number to four figures. Run from π_1 at time 1 down to time 0, the reverse chain produces the bridge path law read backwards, so it ends at π_0.

The reverse rates require the marginal p_t, which we cannot evaluate from a forward model by summing over a large sequence or graph state space. In a method that needs both directions we therefore train a second network on the reverse rates directly, with the same loss and with pinned targets taken from the other end of the path.

7.23.3 Bridge cost comparison

We can check the path-space relative entropy from Section 7.21.1 against the static calculation on the same example. Computed on the endpoints, the cost is the relative entropy of $\pi^\star$ from R_{01},

$$D_{\mathrm{KL}}(\pi^\star \,\|\, R_{01}) = \sum_{i,j} \pi^\star_{ij} \log \frac{\pi^\star_{ij}}{(R_{01})_{ij}} = 0.3235.$$

Computed on the paths, it is the time integral of the rate error between $Q^\star_t$ and Q. At $t = 1/2$, the term at state 1 is $1.5836 \log(1.5836/1) - 1.5836 + 1 = 0.7280 - 0.5836 = 0.1444$, and the term at state 2 is $0.9472 \log(0.9472/1.5) - 0.9472 + 1.5 = -0.4354 + 0.5528 = 0.1174$. Weighting these by the marginal $(0.4668,\ 0.5332)$ at that time, the integrand is $0.4668 \times 0.1444 + 0.5332 \times 0.1174 = 0.1300$. Integrating the same quantity over $[0, 1]$ produces 0.3235, the same number as the static calculation.

Scaling the reference jump rates changes the cost. Scaling the generator by a factor λ and recomputing, the cost is 0.3148 at $\lambda = 1/2$, 0.3235 at $\lambda = 1$, 0.3337 at $\lambda = 2$ and 0.3348 at $\lambda = 4$, close to the limiting value. As λ grows, every row of $e^{\lambda Q}$ approaches stationary law $(0.6,\ 0.4)$. The endpoint matrix R_{01} therefore approaches a rank-one matrix. Its Sinkhorn scaling is the independent coupling $\pi_0 \otimes \pi_1$, and the cost reduces to

$$D_{\mathrm{KL}}(\pi_1 \,\|\, (0.6,\ 0.4)) = 0.2 \log \frac{0.2}{0.6} + 0.8 \log \frac{0.8}{0.4} = -0.2197 + 0.5545 = 0.3348.$$

When the reference endpoint is independent of its starting state, the entire cost comes from changing the terminal marginal. A slower reference has a lower total cost here and produces a coupling further from independence. The reference noise level therefore controls both the cost and the endpoint dependence.

The two computations are the chain rule read in both directions. The static one prices the change of endpoint coupling; the dynamic one prices the change of rates, instant by instant, along the paths that coupling induces. Any method that fits a rate matrix to bridge samples is minimizing the second expression, and the first is the value it reaches at the optimum.

For two states and a known generator, we can compute φ_t with a matrix exponential. For sequences or graphs, its defining sum spans the full state space, making that matrix exponential impractical. We can approximate the required quantities with a network.

7.24 Discrete Diffusion Schrödinger Bridge Matching

Discrete Diffusion Schrödinger Bridge Matching [Kim et al., 2025] transforms molecular graphs. Each node has a categorical atom type, and each pair of nodes has a categorical bond type, including "no bond.

The two laws are two sets of molecules. In one such pair of sets, π_0 is the set of ZINC molecules with $\log P$ near 2 and π_1 the set of ZINC molecules with $\log P$ near 4 [Kim et al., 2025]. A model trained on independent pairs from these two sets learns to produce a molecule of the second kind from a molecule of the first, with no reason for the two to resemble each other. Under the reference, changing an atom type or bond type incurs an edit cost. The bridge coupling is the entropic transport plan for that cost, so it pairs molecules from the two sets using few edits.

7.24.1 The reference process on one feature

Every node feature and every bond feature evolves independently under the reference, so the whole construction is specified by what happens to one categorical feature with S possible values. The reference transition kernel between two times s and t is

$$K_{st}(x, y) = r\,\mathbf{1}_{x=y} + (1 - r)\,m(y), \qquad r = \frac{\bar{\alpha}(t)}{\bar{\alpha}(s)}.$$

"The reference transition probability $K_{st}(x, y)$ retains category x with survival probability r, and redraws category y from prior $m(y)$ with probability $1-r$. The survival probability is noise-schedule ratio $r = \bar{\alpha}(t)/\bar{\alpha}(s)$ between later time t and earlier time s."

- m is a fixed prior distribution over the S categories, the law a feature is redrawn from when it is resampled. For bond types this is often uniform; for atom types it can be the marginal frequency in the training set.
- $\bar{\alpha}(t)$ is the noise schedule, a decreasing function of time with $\bar{\alpha}(0) = 1$. It is the probability that a feature survives from time 0 to time t without ever being redrawn.
- $r \in (0, 1]$ is the survival probability over the interval $[s, t]$, so with probability r the category is kept and with probability $1 - r$ it is redrawn from m.

This kernel is the uniform-replacement kernel of Chapter 4, written with a general prior m in place of the uniform one. Its generator follows from the kernel over a short step. We put $s = t$ and $t \to t + h$, so that $r = \bar{\alpha}(t + h)/\bar{\alpha}(t) = 1 - h\,\beta(t) + O(h^2)$ with $\beta(t) = -\frac{d}{dt}\log\bar{\alpha}(t) \geq 0$. Substituting and keeping the terms of order h,

$$K_{t,t+h}(x, y) = \mathbf{1}_{x=y} + h\,\beta(t)\big(m(y) - \mathbf{1}_{x=y}\big) + O(h^2), \qquad \text{so} \qquad Q_t(x, y) = \beta(t)\,m(y) \quad (x \neq y).$$

The rate out of x toward a different category y is the resampling rate $\beta(t)$ times the chance $m(y)$ that the resampling lands on y, and the diagonal entry $Q_t(x, x) = -\beta(t)(1 - m(x))$ makes the row sum zero.

We can compute the coupling for a single feature by scaling the resulting small endpoint matrix.

7.24.2 A bridge for a single bond

Let's take one bond with three types, no bond, single and double, and work the coupling out by hand. We can set $r = 0.4$ over the whole unit interval and let m be uniform, so under the reference a type stays the same with probability $0.4 + 0.6/3 = 0.6$ and becomes each of the other two with probability $0.6/3 = 0.2$,

$$K_{01} = \begin{pmatrix} 0.6 & 0.2 & 0.2 \\ 0.2 & 0.6 & 0.2 \\ 0.2 & 0.2 & 0.6 \end{pmatrix}, \qquad \pi_0 = (0.7,\ 0.2,\ 0.1), \qquad \pi_1 = (0.1,\ 0.2,\ 0.7).$$

Most source mass is on "no bond," and most target mass is on "double." Transport therefore requires changing bond types. These changes cost more under a reference that retains a type more

often than it changes one. In the continuous-time picture, $\bar{\alpha}(t) = 0.4^t$ realizes this r with the constant resampling rate $\beta = -\log 0.4 = 0.9163$, and the off-diagonal rates of the reference are $0.9163/3 = 0.3054$.

If we scale $\mathrm{diag}(\pi_0)\, K_{01}$ to the two marginals with Sinkhorn, we get the bridge coupling. First, divide the target masses by the column sums $(0.48,\ 0.28,\ 0.24)$ of $\mathrm{diag}(\pi_0)K_{01}$. We obtain $g = (0.2083,\ 0.7143,\ 2.9167)$.

After a few more half-steps, the scalings converge to $f = (1.1779,\ 0.9506,\ 0.5062)$ and $g = (0.1842,\ 0.6918,\ 3.0004)$. The resulting plan is

$$\pi^\star = \begin{pmatrix} 0.091 & 0.114 & 0.495 \\ 0.007 & 0.079 & 0.114 \\ 0.002 & 0.007 & 0.091 \end{pmatrix}.$$

Rows are the starting type and columns are the final type. The entry 0.495 is the mass that moves from no bond to double, which is most of the transport that has to happen here. The diagonal entries add to $0.091 + 0.079 + 0.091 = 0.261$, and that is the fraction of bonds the bridge does not change.

We can compare this unchanged fraction of 0.261 with the independent and edit-minimizing couplings. Under the independent coupling, the mass that happens to stay is $\sum_i \pi_0(i)\pi_1(i) = 0.07 + 0.04 + 0.07 = 0.18$. Under any coupling of these two laws, the mass that stays is at most $\sum_i \min(\pi_0(i), \pi_1(i)) = 0.1 + 0.2 + 0.1 = 0.4$, and the edit-minimizing plan achieves it. The bridge keeps 0.261 unchanged, between the two values, because entropy regularization spreads mass beyond the cheapest plan.

The survival probability r determines the position between those limits, and it plays the part that ε played in the entropic problem. The diagonal kernel entry 0.6 is three times the off-diagonal entry 0.2. In a Gibbs kernel, this ratio is $e^{c/\varepsilon}$, where c is the cost of one edit. Increasing r increases the preference for retaining a type, corresponding to a smaller effective ε. Recomputed at a few values of r, the plan has a diagonal mass of 0.2250 at $r = 0.2$, 0.2612 at $r = 0.4$, 0.2926 at $r = 0.6$, 0.3248 at $r = 0.8$ and 0.3812 at $r = 0.99$. The two ends of that range are the two limits named above, 0.18 as r goes to zero and the reference retains nothing of the starting type, and 0.4 as r goes to one and every edit costs an unbounded amount. When we choose the schedule $\bar{\alpha}$ we are also choosing how far along this line the transformation lies.

7.24.3 Training with the terminal graph fixed

To fit the Markov projection we need a regression target at each state, and the target is the rate of the reference bridge that ends at a known terminal graph. Pinning the endpoint is an h-transform with $h_t = K_{t1}(\cdot, x_1)$, the reference probability of arriving at x_1,

$$Q_t(x, y;\, x_1) = Q_t(x, y)\, \frac{K_{t1}(y, x_1)}{K_{t1}(x, x_1)}, \qquad x \neq y.$$

"The pinned jump rate $Q_t(x, y; x_1)$, conditional on terminal state x_1, equals reference rate $Q_t(x, y)$ times endpoint probability $K_{t1}(y, x_1)$ from destination y, divided by endpoint probability $K_{t1}(x, x_1)$ from current state x."

Under $\bar{\alpha}(t) = 0.4^t$, the survival ratio from t to 1 equals 0.5 at $t = \log 0.8 / \log 0.4 = 0.2435$, where K_{t1} is $0.5 + 0.5/3 = 0.6667$ on the diagonal and $0.5/3 = 0.1667$ off it. We write x_1 for the terminal type of the single bond worked out above, and the pinned rate takes three values. A move toward x_1 from any other type has its rate multiplied by $0.6667/0.1667 = 4$, so the reference rate 0.3054 becomes 1.2217. A move away from x_1 has its rate multiplied by $0.1667/0.6667 = 0.25$, so 0.3054 becomes 0.0764. A move between two types that are both different from x_1 has numerator and denominator equal, so the rate stays 0.3054. The pinned process is the reference with a factor of sixteen between the rate onto the terminal type and the rate off it.

We train the network against this pinned rate with the rate error of Chapter 4,

$$\mathcal{L}(\theta) = \mathbb{E} \int_0^1 \sum_{y \neq X_t} \Big(Q_t^{\text{pin}} \log \frac{Q_t^{\text{pin}}}{Q_t^\theta} - Q_t^{\text{pin}} + Q_t^\theta \Big)(X_t, y)\, dt.$$

"The rate loss $\mathcal{L}(\theta)$ averages a time-integrated sum over destinations $y \neq X_t$: pinned target rate $Q_t^{\text{pin}}(X_t, y)$ times its log ratio to network rate $Q_t^\theta(X_t, y)$, minus the pinned rate and plus the network rate. We average over endpoint pairs from the coupling and reference-bridge states X_t."

Here Q_t^{pin} abbreviates $Q_t(\cdot, \cdot;\, x_1)$. We can build a training batch by drawing an endpoint pair (x_0, x_1) from the current coupling, drawing the state X_t from the reference bridge between them at a uniformly drawn time, and evaluating the two rates there. For example, take pinned rate 1.2217 toward the terminal type and predicted rate 1.0000. This edge contributes $1.2217 \log 1.2217 - 1.2217 + 1.0000 = 0.2446 - 0.2217 = 0.0229$ to the loss. The contribution is zero only when the prediction matches 1.2217.

Suppose the state X_t is on a bond of type "no bond" and the current coupling allows two possible terminal types for that bond, "double" with posterior probability 0.6 and "single" with probability 0.4. The pinned rate toward "double" is 1.2217 when the terminal type is "double", since that move lands on the terminal type, and 0.3054 when the terminal type is "single", since that move is then between two non-terminal types. Minimizing the expected Bregman term over the single number Q_t^θ and setting the derivative to zero,

$$\mathbb{E}\Big[1 - \frac{Q_t^{\text{pin}}}{Q_t^\theta}\Big] = 0 \qquad \Longrightarrow \qquad Q_t^\theta = \mathbb{E}[Q_t^{\text{pin}}] = 0.6 \times 1.2217 + 0.4 \times 0.3054 = 0.8552.$$

The optimal prediction is therefore 0.8552, the posterior average of the two conditional rates. The minimum loss is $\mathbb{E}[Q^{\text{pin}} \log Q^{\text{pin}}] - 0.8552 \log 0.8552 = 0.1357$. This residual measures the variation between the conditional rates for the two terminal graphs. The loss at 1.0000 would be 0.1467 instead, and at 0.5000 it would be 0.2395.

We used the same loss to fit a reverse generator in Chapter 4. Here we change the endpoint used to define the conditional target. There the pinning was on the clean data point; here it is on the terminal graph of the current coupling. In both cases the minimizer over a state is the posterior average of the conditional rates, so the network learns the rate of the marginal process, which is the Markov projection of the mixture of pinned bridges.

The reciprocal projection costs a simulation. We run the learned chain from one marginal to the other, store the endpoint pairs it produces, and use them as the coupling for the next regression. The two projections alternate as they did for diffusions, and we train one network for each direction of

time, so the forward model maps π_0 to π_1 and the backward model maps π_1 to π_0. The alternation converges to the law that is both Markov and reciprocal, which is the bridge.

Written out as half-steps, one round of the alternation is four operations. First, simulate the current backward model from π_1 to collect endpoint pairs (x_0, x_1). Fit the forward rates using the loss above on reference-bridge states between those pairs. Then simulate the new forward model from π_0 to collect fresh pairs, and fit the backward rates in the same way. Compared with the continuous alternation, we regress a rate matrix and measure its error with the Bregman divergence of $u \log u$. The continuous version regresses a drift with squared error. The remaining steps and the fixed-point argument from Section 7.21.2 are unchanged.

Sampling costs one network evaluation per Euler step and one categorical draw per feature that moves. The learned rates are large near $t = 1$, as the two-state example showed, so the step size near the terminal time limits the accuracy.

7.24.4 The factorization over a graph

For a molecule, we can compute the required quantities feature by feature because the reference factorizes over node and bond features.

The whole graph has the product kernel, K_{st} applied independently at every node and every pair of nodes. For a molecule with nine heavy atoms there are nine node features and $9 \times 8/2 = 36$ bond features, forty-five categorical variables in all, and with five atom types and four bond types the state space holds $5^9 \times 4^{36} \approx 9.2 \times 10^{27}$ graphs. We cannot sum explicitly over this state space. We therefore calculate each quantity one feature at a time.

The product form gives single-feature jumps because two resamplings in the same short interval have probability of order h^2. Simulating each jump requires one categorical draw. The total exit rate is the sum of the feature rates. For thirty-six bonds with a uniform prior over four types, it is $36 \times 0.9163 \times (1 - 1/4) = 24.74$ per unit time. This rate determines the sampler step size.

Factorization also simplifies the training target. The pinned rate of Section 7.24.3 contains the ratio $K_{t1}(y, x_1)/K_{t1}(x, x_1)$, and with K_{t1} a product over features that ratio is

$$\frac{K_{t1}(y, x_1)}{K_{t1}(x, x_1)} = \prod_j \frac{K_{t1}(y_j, x_{1,j})}{K_{t1}(x_j, x_{1,j})} = \frac{K_{t1}(y_{j^*}, x_{1,j^*})}{K_{t1}(x_{j^*}, x_{1,j^*})},$$

where the second equality holds when x and y differ at the single feature j^*, because every other factor has the same numerator and denominator and cancels. The pinned process is therefore a product of independent single-feature pinned processes, and the whole of Section 7.24.2 describes one coordinate of it. A network that outputs, for each of the forty-five features, a distribution over its categories then determines all of the rates, and the loss splits into forty-five independent terms.

The continuous-time chain can jump at any time. Euler simulation requires a step size, and training involves rates that can be large. Using discrete time gives us transition probabilities to fit and sample directly.

7.25 Categorical Schrödinger Bridge Matching

In Categorical Schrödinger Bridge Matching [Ksenofontov and Korotin, 2025] we put N intermediate times $t_1, \dots, t_N$ between 0 and 1 and work with the sequence of states at the times $0, t_1, \dots, t_N, 1$. A path is then $N+2$ states joined by $N+1$ transitions, and the reference is a product of $N+1$ transition matrices. The bridge construction also applies in discrete time. For small N, we can store and check the whole path law as a finite table.

The reference over S categories can be the uniform one,

$$[A_n]_{x\,y} = \begin{cases} 1-\alpha, & y = x \\ \dfrac{\alpha}{S-1}, & y \neq x \end{cases}$$

"The reference transition entry $[A_n]_{xy}$ gives probability $1-\alpha$ of retaining category x at step n, and probability $\alpha/(S-1)$ of moving to each different category y, where α is the total change probability and S the number of categories."

The single parameter $\alpha \in (0, 1)$ is the probability of changing category in one step, spread evenly over the alternatives. For ordered categories, such as the 256 levels of a pixel, we can also use a discretized Gaussian centered on the current category. Nearby categories then have higher transition probabilities than distant ones. With five ordered categories and a width of one category, the row out of the middle category is proportional to $e^{-2}, e^{-1/2}, 1, e^{-1/2}, e^{-2}$, which normalizes to

$$(0.0545,\ 0.2442,\ 0.4026,\ 0.2442,\ 0.0545),$$

against $(\alpha/4,\ \alpha/4,\ 1-\alpha,\ \alpha/4,\ \alpha/4)$ for the uniform reference. Both put their largest share on staying put, and the remainder is spread evenly over the alternatives in one case and concentrated on the two neighbors in the other. The reference determines which edits have low cost. The ordered reference produces bridge paths with small category changes. The uniform reference also permits occasional large changes.

7.25.1 The two projections in discrete time

We can write both projections from Section 7.24 directly in terms of discrete-time conditional probabilities. Let $x_{\text{in}} = (x_{t_1}, \dots, x_{t_N})$ denote the interior states. The reciprocal projection holds the endpoint coupling fixed and redraws the interior from the reference,

$$\text{proj}_{\mathcal{R}}(P) = P(x_0, x_1)\ R(x_{\text{in}} \mid x_0, x_1).$$

"The reciprocal projection $\text{proj}_{\mathcal{R}}(P)$ multiplies the current endpoint coupling $P(x_0, x_1)$ by reference conditional probability $R(x_{\text{in}} \mid x_0, x_1)$ for the N interior states x_{in} between those endpoints."

The Markov projection retains the one-step conditionals and multiplies them into a chain,

$$\text{proj}_{\mathcal{M}}(P) = P(x_0) \prod_{n=1}^{N+1} P(x_{t_n} \mid x_{t_{n-1}}).$$

"The Markov projection $\mathrm{proj}_{\mathcal{M}}(P)$ multiplies the current initial probability $P(x_0)$ by all $N+1$ one-step conditional probabilities $P(x_{t_n} \mid x_{t_{n-1}})$, each giving the next state at time t_n conditional on the previous state at time t_{n-1}."

Each projection minimizes relative entropy from the current law over its constraint set. A fixed point of the alternation belongs to both sets.

Theorem 7.9 (The bridge is the only law in both sets)**.** *Among path laws on $0, t_1, \ldots, t_N, 1$ with marginals π_0 at time 0 and π_1 at time 1, the discrete-time Schrödinger bridge of the reference R is the only law that is both Markov and a member of the reciprocal class of R [Ksenofontov and Korotin, 2025].*

Alternating the two projections is iterative Markovian fitting with discrete time, named D-IMF [Ksenofontov and Korotin, 2025]. By Theorem 7.9, the alternation has a unique fixed point: the bridge, which belongs to both sets and is unchanged by either projection.

To fit the Markov projection we fit $N+1$ conditional distributions, one per transition, which is a classification problem over S categories. The loss is a relative entropy on each transition,

$$\mathcal{L}(\theta) = \sum_{n=1}^{N+1} \mathbb{E}\, D_{\mathrm{KL}}\big(P(x_{t_n} \mid x_{t_{n-1}}) \,\big\|\, m_\theta(x_{t_n} \mid x_{t_{n-1}})\big)\,.$$

"The transition loss $\mathcal{L}(\theta)$ sums expected relative entropies between target conditional $P(x_{t_n} \mid x_{t_{n-1}})$ and predicted conditional $m_\theta(x_{t_n} \mid x_{t_{n-1}})$, over the $N+1$ transitions, averaging over previous states from the current path law P."

In practice we take the expectation by drawing an endpoint pair from the current coupling, filling in the interior from the reference bridge, and reading off consecutive states. The network takes $(x_{t_{n-1}}, n)$ as input and produces a distribution over the S categories at t_n, and minimizing the relative entropy over a batch is the cross-entropy of the next state against that distribution. Sampling is then $N+1$ categorical draws, with no step size to choose and no rate to integrate.

Suppose the true conditional out of state 1 at some transition is $(0.3160,\ 0.6840)$ and the network produces $(0.25,\ 0.75)$. The term is

$$0.3160 \log \frac{0.3160}{0.25} + 0.6840 \log \frac{0.6840}{0.75} = 0.3160 \times 0.2343 - 0.6840 \times 0.0921 = 0.0740 - 0.0630 = 0.0110,$$

about one hundredth of a nat, and the two parts of it nearly cancel because the error is a small reallocation between the two categories. The probability errors have opposite signs across the two categories. This loss measures the redistribution of mass and is less sensitive to a bias that preserves the category ranking.

As in Chapter 4, we fit a network that outputs a distribution over S categories from a noisy input. In Chapter 4, the network predicted the clean token given a corrupted token, and we used that conditional distribution to construct reverse rates. Here the head produces the next state given the previous state, and we can sample that transition probability directly without assembling a rate matrix or its diagonal.

7.25.2 D-IMF on sixteen paths

Two states and three transitions make the whole path law small enough to write out, so we can run the alternation exactly and compare it against the bridge computed in closed form. We take $S = 2$, four time points, the uniform reference with $\alpha = 0.35$ at every step, the source law $p_0 = (0.5,\ 0.5)$ and the requested law $p_1 = (0.2,\ 0.8)$. With four time points there are $2^4 = 16$ paths, so a path law is a list of sixteen numbers.

The reference matrices for one step and for the whole interval are

$$A = \begin{pmatrix} 0.65 & 0.35 \\ 0.35 & 0.65 \end{pmatrix}, \qquad A^3 = \begin{pmatrix} 0.5135 & 0.4865 \\ 0.4865 & 0.5135 \end{pmatrix}.$$

Over three steps the chain retains little of where it started, so the reference is a reasonable noising process and the bridge a correction of modest size.

Scaling $\mathrm{diag}(p_0)A^3$ to the two marginals produces the exact bridge coupling, and the independent coupling is the starting point of the alternation,

$$\pi^\star = \begin{pmatrix} 0.10432 & 0.39568 \\ 0.09568 & 0.40432 \end{pmatrix}, \qquad p_0 \otimes p_1 = \begin{pmatrix} 0.1 & 0.4 \\ 0.1 & 0.4 \end{pmatrix}.$$

The starting law P^0 fills the independent coupling in with reference bridges, so it already has the two requested marginals exactly. Both laws assign a probability to each of the sixteen paths through the same rule, the endpoint mass times the reference bridge weight. The path 1 1 1 1 has reference bridge weight $0.65^3/0.5135 = 0.5348$, so its probability is $0.1 \times 0.5348 = 0.0535$ under P^0 and $0.10432 \times 0.5348 = 0.0558$ under the bridge. The path 2 2 2 2 has the same weight and endpoint masses 0.4 and 0.40432, giving 0.2139 and 0.2162. Adding the sixteen absolute differences,

$$p_0 = (0.5,\ 0.5), \qquad p_1 = (0.2,\ 0.8), \qquad \sum |P^0 - P^\star| = 0.0173.$$

If we checked the marginals we would not find this gap. The two laws agree at time 0 and at time 1 by construction, and in this symmetric example they also agree at the two interior times, where both put $(0.4754,\ 0.5246)$ at t_1 and $(0.4107,\ 0.5893)$ at t_2. The difference is in the joint law over paths, which is the quantity the bridge is defined by.

Now let's run the alternation. The Markov projection of P^0 is built from its three one-step conditionals,

$$\begin{pmatrix} 0.6243 & 0.3757 \\ 0.3265 & 0.6735 \end{pmatrix}, \quad \begin{pmatrix} 0.5626 & 0.4374 \\ 0.2729 & 0.7271 \end{pmatrix}, \quad \begin{pmatrix} 0.3160 & 0.6840 \\ 0.1192 & 0.8808 \end{pmatrix},$$

which the network m_θ would learn from an infinite batch at this round. The three matrices are far from each other, and read from left to right they are the schedule of the bridge. The first transition stays close to the reference. The last transition must reach the target law in one step. Almost nine tenths of the mass at state 2 stays there, and two thirds of the mass at state 1 moves to state 2.

If we multiply these three conditionals into a chain, we get a Markov law whose endpoint coupling is $\left(\begin{smallmatrix} 0.10425 & 0.39575 \\ 0.09575 & 0.40425 \end{smallmatrix}\right)$, already within 10^{-4} of $\pi^\star$ in every entry. The reciprocal projection then puts the

reference bridges back in the interior and completes the round. Measuring the same sum of sixteen absolute differences after each of the first five rounds,

$$2.9 \times 10^{-4}, \quad 5.0 \times 10^{-6}, \quad 8.4 \times 10^{-8}, \quad 1.4 \times 10^{-9}, \quad 2.4 \times 10^{-11}.$$

One round brings the gap from 0.0173 down to 2.9×10^{-4}, and every round after that divides it by about 59. Five rounds reach 2.4×10^{-11} and eight rounds reach machine precision. The convergence factor of 59 depends on the reference and marginals. A reference closer to the identity converges more slowly: at $\alpha = 0.2$, the gap decreases by a factor of about 5 per round. At $\alpha = 0.4$, that factor is about 284. Faster reference mixing therefore gives faster convergence in this example.

The number of intermediate times changes the problem in the same direction. If we hold $\alpha = 0.35$ per step and run the same alternation with one interior time in place of two, so that eight paths replace sixteen, the starting gap is 0.0574 and the rounds divide it by about 7. With four interior times and sixty-four paths, the starting gap is 0.00156 and the rounds divide it by about 4670. Both effects have the same cause, which is that more steps at a fixed per-step flip probability mix the reference more thoroughly, and a reference that retains less of its starting state needs a smaller correction to meet the terminal law. Choosing N therefore changes the size of the required bridge correction and the sampling cost of $N + 1$ network evaluations.

We can run this exactly because 16 paths fit in a list. For sequences of L tokens over K categories, we approximate the conditionals with m_θ and estimate endpoint couplings by simulation. We can run the same alternation, but we can no longer evaluate the full path-law error after each round. The finite example gives a concrete case of convergence within a few rounds.

7.26 Branched Schrödinger Bridge Matching

We can extend the single-source, single-target bridge to several target laws with specified mass fractions. We can also use a target density to guide a system of interacting particles.

Branched Schrödinger Bridge Matching [Tang et al., 2026b] connects one source population to several weighted targets. A population of cells treated with a drug does not end in one state. Some die, some enter a resistant state, some continue dividing, and an experiment measures the fraction of cells in each outcome. We have one source law π_0 and $K + 1$ target laws $\pi_1^0, \ldots, \pi_1^K$, each with a measured weight. A single-terminal-law bridge produces their mixture and can place cells between the observed outcomes, in regions with no measured cells.

We can split the population into $K + 1$ branches, each with its own drift and its own share of the mass (Figure 7.7). Branch k is a controlled diffusion,

$$dX_t^k = u_t^k(X_t^k)\, dt + \sqrt{\varepsilon}\, dB_t, \qquad X_0 \sim \pi_0, \quad X_1^k \sim \pi_1^k.$$

"The state X_t^k of branch k changes by its drift $u_t^k(X_t^k)$ times dt, plus Brownian increment dB_t scaled by $\sqrt{\varepsilon}$, with noise variance ε. Every branch starts from source law π_0, and branch k ends with its target law π_1^k."

A branch differs from an independent bridge in holding a share of the mass. The weight $w_t^k \in [0, 1]$ is the fraction of the population on branch k at time t, with $\sum_k w_t^k = 1$, and it changes at a learned

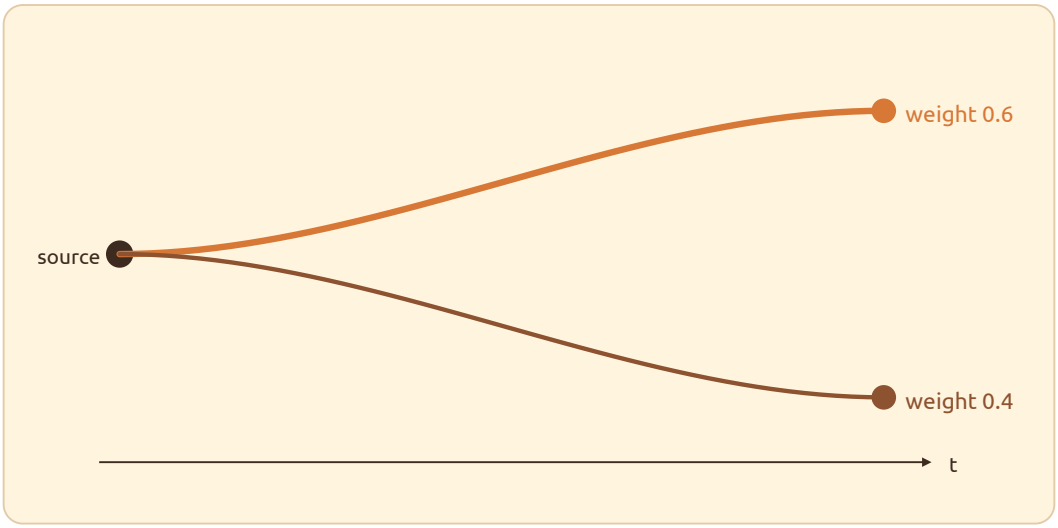


Figure 7.7: A population leaving a single source and splitting into two branches that end in two separate terminal populations. Time runs from left to right. Each curve is the trajectory of one branch, and the labels at the right are the weights the two branches carry at time 1, drawn here as 0.6 and 0.4. The weights sum to one at every time, so mass that one branch gains is mass the other loses, and both branches start from the same source. The weights are illustrative, and the worked example of Section 7.26.1 uses 0.7 and 0.3 instead.

growth rate g_t^k,

$$w_t^k = w_0^k + \int_0^t g_s^k(X_s^k)\, ds, \qquad \sum_{k=0}^{K} w_t^k = 1.$$

"The population fraction w_t^k on branch k equals its initial fraction w_0^k, plus growth rate $g_s^k(X_s^k)$ integrated along that branch's states from time zero to t. The fractions sum to one at every time."

The letter g stood for the terminal Schrödinger potential earlier in this chapter; the growth rate g_t^k has a branch index and a time subscript, and the two never appear in the same expression. All of the mass starts on branch 0, so $w_0^0 = 1$ and $w_0^k = 0$ for $k \geq 1$, and at time 1 each weight has to equal its measured target. Total mass is conserved only if $\sum_k g_t^k = 0$ at every time, so a branch grows only by taking mass from another branch.

We write p_t^k for the density of branch k, which satisfies the Fokker-Planck equation of its own controlled diffusion, and write the population density as the weighted mixture $p_t = \sum_k w_t^k\, p_t^k$. Differentiating the mixture in time and substituting the weight equation and the Fokker-Planck equation,

$$\partial_t p_t = \sum_{k=0}^{K} \Big[\underbrace{\dot{w}_t^k\, p_t^k}_{\text{mass transferred}} \quad \underbrace{-\nabla \cdot (w_t^k\, p_t^k\, u_t^k) + \tfrac{\varepsilon}{2} \nabla^2 (w_t^k\, p_t^k)}_{\text{motion within the branch}} \Big].$$

"The population-density change $\partial_t p_t$ sums three contributions over branches k: weight change $\dot{w}_t^k$ times branch density p_t^k, minus the divergence of weighted drift flux $w_t^k p_t^k u_t^k$, plus half the noise variance ε times the Laplacian of weighted density $w_t^k p_t^k$. The first transfers mass between branches; the other two move mass within each branch."

Here $\dot{w}_t^k$ is the average of the growth rate g_t^k over the particles of branch k, as the weight equation above states once the integral is read as a population fraction. The first group integrates to $\sum_k \dot{w}_t^k = 0$ over the state space, so the total mass of the population stays one, and the second group integrates to zero because a divergence and a Laplacian do. A single Schrödinger bridge is the case $K = 0$, where the first group disappears and the equation is the Fokker-Planck equation of Chapter 3. The branched problem is therefore the same transport problem with one extra channel, a source term that moves mass between branches without moving it in space.

In the objective, both the motion and the mass that undergoes it have a price,

$$\min_{\{u_t^k,\, g_t^k\}} \sum_{k=0}^{K} \int_0^1 \mathbb{E}\Big[\Big(\tfrac{1}{2}\|u_t^k(X_t^k)\|^2 + V_t(X_t^k)\Big)\, w_t^k\Big]\, dt.$$

"We optimize branch drifts u_t^k and growth rates g_t^k to minimize the expected sum of control energy density $\frac{1}{2}\|u_t^k(X_t^k)\|^2$ and state cost $V_t(X_t^k)$, weighted by branch fraction w_t^k, integrated over time and summed over branches $k = 0, \dots, K$."

Here V_t is a state cost, a price for occupying a position at a time, which on single-cell data is built from the density of observed cells so that paths through empty regions cost more. It is a different quantity from the value function of the control problem earlier in this chapter, which was also written V. Without the weights and with $V = 0$, each term is the energy of a single Schrödinger bridge. With the weights in place, energy spent by a branch counts in proportion to how much of the population is on that branch when it is spent.

7.26.1 Two-branch example

Let's work out the smallest branched problem. The source is a point at 0. Branch 0 ends at $+2$ with target weight 0.7, and branch 1 ends at -2 with target weight 0.3. We use constant growth rates,

$$g_t^1 = 0.3, \quad g_t^0 = -0.3, \qquad \text{so} \qquad w_t^0 = 1 - 0.3\,t, \quad w_t^1 = 0.3\,t.$$

Branch 1 gains mass at rate 0.3 and branch 0 loses it at the same rate, so the weights sum to one at every time. At $t = 1/2$ they are 0.85 and 0.15, and at $t = 1$ they are 0.7 and 0.3, the target weights.

We can ignore the noise and take the straight-line drifts $+2$ and -2, which move each branch from 0 to its endpoint in unit time. The control energy density is $\frac{1}{2}(2)^2 = 2$ on both branches, so each weighted energy is 2 times the average weight of that branch,

$$\int_0^1 \tfrac{1}{2}\,(2)^2\,(1 - 0.3\,t)\,dt = 2 \times 0.85 = 1.7, \qquad \int_0^1 \tfrac{1}{2}\,(-2)^2\,(0.3\,t)\,dt = 2 \times 0.15 = 0.3.$$

Branch 0 has average weight 0.85 over the unit interval and weighted energy 1.7; branch 1 has average weight 0.15 and weighted energy 0.3. The total is 2.0, the energy of a single particle moving a distance 2 in unit time. The split costs nothing extra here, because the two branches have the same energy density and the weights sum to one.

When the branches cost different amounts, the schedule of the split starts to matter. Let's move the second target to -4, so that branch 1 needs drift -4 and energy density 8. With the linear

weights above, the total is $2 \times 0.85 + 8 \times 0.15 = 1.7 + 1.2 = 2.9$. With a growth rate $g_t^1 = 0.6\,t$ that starts slowly, the weights are $w_t^1 = 0.3\,t^2$ and $w_t^0 = 1 - 0.3\,t^2$, they still reach 0.3 and 0.7 at time 1, and the total is $2 \times 0.9 + 8 \times 0.1 = 1.8 + 0.8 = 2.6$. Holding mass on the cheap branch and releasing it late is worth 0.3 of energy here, and the growth rates are learned for exactly this reason.

7.26.2 The interpolant and the training stages

We parameterize the paths so that training can move them away from regions with high state cost. We add a network correction to the straight line with a factor that vanishes at both ends,

$$x_{t,\eta}^k = (1-t)\,x_0 + t\,x_1^k + t(1-t)\,\varphi_{t,\eta}(x_0, x_1^k).$$

"The branch interpolant $x_{t,\eta}^k$ adds a learned correction $\varphi_{t,\eta}(x_0, x_1^k)$, scaled by $t(1-t)$, to the straight path from source point x_0 to branch endpoint x_1^k. The parameters η control the correction at time t."

The factor $t(1-t)$ is zero at $t = 0$ and at $t = 1$, so the path starts at x_0 and ends at x_1^k for every value of η, and the endpoint constraints hold during training without a penalty term. With $x_0 = 0$, $x_1 = 2$, and correction 1, the path is $x_t = 3t - t^2$. At times 0, 1/2, and 1, its positions are 0, 1.25, and 2. Its velocity $3 - 2t$ is 3, 2, and 1 at those times. The path starts fast and ends slowly. Its energy is $\int_0^1 \frac{1}{2}(3-2t)^2 dt = 2.1667$, against 2 for the straight line, so with $V = 0$ the training would drive this correction to zero, and only a state cost that is higher along the straight line makes a nonzero correction worthwhile.

We train the interpolant, drifts, and growth rates in four stages.

First, hold the endpoints fixed and minimize the weighted energy and state cost over the interpolant parameters η. We obtain its velocity by differentiating the interpolant with respect to t. For the straight line plus a constant correction c, that derivative is $\dot{x}_t = (x_1^k - x_0) + (1-2t)\,c$, which at $x_0 = 0$, $x_1^k = 2$ and $c = 1$ is the $3 - 2t$ computed above.

Second, fit the drifts by flow matching against the fitted interpolant velocity,

$$\mathcal{L}(\theta) = \sum_{k=0}^{K} \mathbb{E}_{t,\,x_0,\,x_1^k}\Big[w_t^k\,\big\|u_\theta^k(t,\,x_{t,\eta}^k) - \dot{x}_{t,\eta}^k\big\|^2\Big].$$

"The drift loss $\mathcal{L}(\theta)$ averages branch weight w_t^k times squared error between predicted drift $u_\theta^k(t, x_{t,\eta}^k)$ and target interpolant velocity $\dot{x}_{t,\eta}^k$, over uniform times t, source points x_0, and branch endpoints x_1^k, then sums over branches k."

This is the conditional velocity regression of Chapter 2, with the branch endpoint in place of the data point and the branch weight as a sample weight. At $t = 1/2$ on the worked path, the target is $\dot{x}_{1/2} = 2$ and the weight is 0.85, so a point on branch 0 halfway through contributes 0.85 times the squared error against 2.

Third, fit the growth rates. Integrating g_θ^k along sampled paths gives w_1^k, which we match to the measured terminal fraction. On the worked example the constraint is one equation, $\int_0^1 g_t^1\,dt = 0.3$,

since branch 1 starts at weight zero. Several schedules satisfy the constraint. We choose among them by their energy cost, as in the comparison of 2.9 with 2.6 above.

In the last stage we train the drifts and the growth rates together, so each can adjust to the other. Joint training can reduce the objective by delaying mass transfer onto a branch with an expensive drift.

Sampling is $K + 1$ runs of the same Euler-Maruyama loop used for any bridge, one per branch, with the weights integrated alongside. To sample the population at time t, choose branch k with probability w_t^k, then draw a state from that branch. One sample costs one bridge simulation. Simulating the full branched population costs $K + 1$ times as much.

Each branch is a controlled diffusion with its own endpoint law. Holding the weights constant reduces the objective to a sum of ordinary bridge energies. Learned growth rates add the freedom to choose when mass transfers between outcomes, beyond what a single bridge can represent.

On a surface benchmark with two targets of weight 0.5 each, the 1-Wasserstein distance to the targets was 0.239 with two branches and 0.975 with a single branch, over five runs with standard deviations of at most 0.009 [Tang et al., 2026b]. The two-branch model was closer to the measured targets.

7.27 Entangled Schrödinger Bridge Matching

For interacting particles, we can choose the system's physical dynamics as the reference. Take n particles with positions $r_t^1, \dots, r_t^n$ coupled by a potential energy U. Our earlier references were Brownian motions or simple noising chains. In Entangled Schrödinger Bridge Matching [Tang et al., 2026a] the reference is underdamped Langevin dynamics,

$$dr_t^i = v_t^i \, dt, \qquad dv_t^i = \frac{-\nabla_{r_t^i} U(\boldsymbol{R}_t)}{m_i} \, dt \; - \; \gamma \, v_t^i \, dt \; + \; \sqrt{\frac{2\gamma k_B \tau}{m_i}} \, dW_t^i.$$

"Particle position r_t^i changes by velocity v_t^i times dt. Its velocity changes by force $-\nabla_{r_t^i} U(\boldsymbol{R}_t)$ from the joint potential energy U, divided by mass m_i, minus friction γv_t^i, both times dt, plus Brownian increment dW_t^i scaled by $\sqrt{2\gamma k_B \tau / m_i}$, where $\boldsymbol{R}_t$ contains all positions, k_B is Boltzmann's constant, and τ the temperature."

- $\boldsymbol{R}_t = (r_t^1, \dots, r_t^n)$ is the configuration of all n positions and $\boldsymbol{V}_t$ the configuration of all n velocities. The bold $\boldsymbol{R}$ is a configuration; the plain R remains the reference path law.
- m_i is the mass of particle i and γ the friction coefficient, in inverse time.
- k_B is Boltzmann's constant and τ the temperature, so $k_B \tau$ is an energy and the noise term has the size that keeps the system at that temperature.
- $X_t = (\boldsymbol{R}_t, \boldsymbol{V}_t)$ is the full state, the variable the path law is a law over.

Using molecular dynamics as the reference, we seek the closest path law among those ending in a chosen region. This is a transition path sampling problem. The resulting paths are possible trajectories of the physical system.

7.27.1 The target as a reweighting of the reference

Here the target is a density $\pi_{\mathcal{B}}$ over final configurations, large on the region $\mathcal{B}$ we want to reach and small elsewhere, and there is no sample set to match. The target path law tilts the reference by that density at the final time,

$$P^\star(X_{0:1}) = \frac{1}{Z}\, R(X_{0:1})\, \pi_{\mathcal{B}}(\boldsymbol{R}_1), \qquad Z = \mathbb{E}_R[\pi_{\mathcal{B}}(\boldsymbol{R}_1)].$$

"The target path probability $P^\star(X_{0:1})$ equals reference path probability $R(X_{0:1})$ multiplied by target density $\pi_{\mathcal{B}}(\boldsymbol{R}_1)$ at the final particle positions $\boldsymbol{R}_1$, divided by normalization Z, the reference expectation of that terminal density."

This is the same tilt that produced the bridge from the reference in the earlier sections, with the terminal potential g replaced by $\pi_{\mathcal{B}}$. The difference is in where the terminal potential comes from. The potential g had to be found by Sinkhorn so that the terminal marginal came out exactly π_1. Here we specify $\pi_{\mathcal{B}}$ directly, and the terminal marginal is whatever the tilt produces.

Suppose the reference ends in the target state $\mathcal{B}$ with probability 0.1, where $\pi_{\mathcal{B}} = 1$, and elsewhere $\pi_{\mathcal{B}} = 0.01$. Then $Z = 0.1 \times 1 + 0.9 \times 0.01 = 0.109$, and under $P^\star$ a path ends in $\mathcal{B}$ with probability $0.1/0.109 = 0.917$. Reweighting increases the probability of reaching $\mathcal{B}$ by a factor of $0.917/0.1 = 9.17$.

That reweighting has a price in relative entropy, and a control that reproduces it has exactly that price. Define the terminal reward $r = \log \pi_{\mathcal{B}}$. It equals 0 inside $\mathcal{B}$ and $\log 0.01 = -4.605$ outside. Averaging under the target law gives $\mathbb{E}_{P^\star}[r] = 0.917 \times 0 + 0.0826 \times (-4.605) = -0.380$, so

$$D_{\mathrm{KL}}(P^\star \,\|\, R) = \mathbb{E}_{P^\star}[r] - \log Z = -0.380 + 2.216 = 1.836.$$

Raising the chance of reaching $\mathcal{B}$ from one time in ten to nine times out of ten costs 1.836 nats of control, and that is also the energy of the optimal bias force.

7.27.2 Particle bias forces

Sampling from $P^\star$ means running the Langevin dynamics with an added force. We can insert one term into the velocity equation,

$$dv_t^i = \frac{-\nabla_{r_t^i} U(\boldsymbol{R}_t) + b^i(\boldsymbol{R}_t, \boldsymbol{V}_t)}{m_i}\, dt \;-\; \gamma\, v_t^i\, dt \;+\; \sqrt{\frac{2\gamma k_B \tau}{m_i}}\, dW_t^i.$$

"The particle velocity v_t^i has acceleration equal to physical force $-\nabla_{r_t^i} U(\boldsymbol{R}_t)$ plus bias force $b^i(\boldsymbol{R}_t, \boldsymbol{V}_t)$, divided by particle mass m_i, with the same friction and thermal noise as the reference. The bias depends on all positions $\boldsymbol{R}_t$ and velocities $\boldsymbol{V}_t$."

Each bias force depends on all n particle positions and velocities, coupling controls even for distant particles. This coupling gives EntangledSBM its name. Velocity conditioning allows the force to depend on momentum, so the correction at a configuration accounts for the system's current direction of motion.

We take one Euler-Maruyama step on a single coordinate with $m = 1$, $\gamma = 1$, $k_B\tau = 1$ and a step $\Delta t = 0.01$, so the noise scale is $\sqrt{2} = 1.4142$ and one increment has standard deviation

$1.4142\sqrt{0.01} = 0.1414$. With the physical force $-\nabla U = -0.3$, a bias force $b = 0.5$, a current velocity $v_t = 0.8$ and a normal draw $z = 0.4$, the three terms of the velocity update are

$$\Delta v = \underbrace{(-0.3 + 0.5) \times 0.01}_{0.0020} \; \underbrace{- \, 1 \times 0.8 \times 0.01}_{-0.0080} \; + \; \underbrace{0.1414 \times 0.4}_{0.0566} \; = \; 0.0506,$$

and the position moves by $v_t \Delta t = 0.0080$. At this step size, the noise increment exceeds both drift terms, so estimating the effect of the control requires averaging over paths. Measured in units of the noise, the control is $0.5/\sqrt{2} = 0.3536$, so this step contributes $\frac{1}{2}(0.3536)^2 \times 0.01 = 0.000625$ to the control energy, and a hundred steps at the same bias contribute 0.0625 nats.

By Girsanov's formula, the price of a bias force, measured in units of the noise, is the time integral of half its squared size. Subtracting the reward at the final state leaves the control problem,

$$\boxed{J(b) = \mathbb{E}_{P^b}\Big[\int_0^1 \tfrac{1}{2} \|b(X_t)\|^2 \, dt - r(X_1) \Big] = D_{\mathrm{KL}}\Big(P^b \,\Big\|\, P^\star\Big) - \log Z.}$$

"The control objective $J(b)$ averages half the time-integrated squared control $\|b(X_t)\|^2$ minus terminal reward $r(X_1)$ over controlled paths from P^b. It equals relative entropy $D_{\mathrm{KL}}\Big(P^b \,\Big\|\, P^\star\Big)$ from the target path law, minus the fixed log normalization $\log Z$."

The term $\log Z$ is constant with respect to b. Relative entropy reaches its minimum of zero when the controlled and target path laws agree. Provided every path starts from the same state, the optimal bias force therefore produces path law $P^\star$. The minimum value is $-\log Z$.

We can check this on the two-outcome example. There $-\log Z = -\log 0.109 = 2.216$. At the optimum the expected control energy equals $D_{\mathrm{KL}}(P^\star \,\|\, R) = 1.836$, by Girsanov's formula applied to the optimal control, and the expected reward is -0.380, so $J(b^\star) = 1.836 - (-0.380) = 2.216$, which is $-\log Z$. The control energy and the reward add up to a quantity fixed by the reference and the target alone.

7.27.3 Parameterizing and training the force

We constrain the bias force to have a nonnegative component toward the target, avoiding energy spent pushing directly away from it. We split the force into a component along the direction of steepest increase of the target density and a component orthogonal to it,

$$b_\theta^i = \alpha_\theta^i \, \hat{s}_i + \big(I - \hat{s}_i \hat{s}_i^\top\big) \, h_\theta^i, \qquad \hat{s}_i = \frac{\nabla_{r_t^i} \log \pi_{\mathcal{B}}}{\|\nabla_{r_t^i} \log \pi_{\mathcal{B}}\|}.$$

"The unit direction $\hat{s}_i$ normalizes the gradient of log target density $\log \pi_{\mathcal{B}}$ with respect to particle position r_t^i. The bias force b_θ^i combines nonnegative predicted magnitude α_θ^i along $\hat{s}_i$ with predicted vector h_θ^i projected orthogonally by $I - \hat{s}_i \hat{s}_i^\top$, where I is the identity matrix."

The scalar α_θ^i is produced through a nonnegative activation, so the component of the force along $\hat{s}_i$ is never negative, whatever the second output does. For a planar example, take $\hat{s} = (1, 0)$, $\alpha = 0.5$, and $h = (0.3, -0.4)$. Projecting h orthogonally to $\hat{s}$ removes its first coordinate, leaving $(0, -0.4)$. The total force is

$$b = 0.5\,(1,\ 0) + (0,\ -0.4) = (0.5,\ -0.4).$$

Its inner product with $\hat{s}$ is 0.5, the scalar α, and its length is 0.640. The orthogonal force component is unrestricted, and α gives the component toward the target.

In training we minimize the relative entropy in the other order from J, which is the cross-entropy objective of Entangled Schrödinger Bridge Matching [Tang et al., 2026a],

$$\mathcal{L}_{\mathrm{CE}}(\theta) = D_{\mathrm{KL}}\Big(P^\star \,\Big\|\, P^{b_\theta}\Big) = \mathbb{E}_{P^{\bar{b}}}\Big[\frac{dP^\star}{dP^{\bar{b}}}\ \log\frac{dP^\star}{dP^{b_\theta}}\Big].$$

"The cross-entropy loss $\mathcal{L}_{\mathrm{CE}}(\theta)$ is relative entropy from target path law $P^\star$ to learned path law P^{b_θ}. We estimate it over paths simulated under fixed sampling control $\bar{b}$, weighting each log ratio $\log\big(dP^\star/dP^{b_\theta}\big)$ by importance ratio $dP^\star/dP^{\bar{b}}$."

That sampling control has the letter v in the source paper; here v_t^i is already a particle velocity, so it is written $\bar{b}$, a copy of the current bias force held fixed with no gradient. We cannot sample $P^\star$ directly, so we estimate its expectations with importance weights.

We can calculate the path weight from its terminal reward and change-of-measure factor. Substituting the definition of $P^\star$ splits the ratio into a reward and a change of measure,

$$\frac{dP^\star}{dP^{\bar{b}}}(X_{0:1}) = \frac{1}{Z}\,e^{r(X_1)}\,\frac{dR}{dP^{\bar{b}}}(X_{0:1}), \qquad \log\frac{dR}{dP^{\bar{b}}} = -\int_0^1 \bar{b}(X_t)\cdot dW_t - \frac{1}{2}\int_0^1 \big\|\bar{b}(X_t)\big\|^2\,dt.$$

"The importance ratio $dP^\star/dP^{\bar{b}}$ equals exponential terminal reward $e^{r(X_1)}$ times the ratio of reference law to sampling law $dR/dP^{\bar{b}}$, divided by normalization Z. The log of the latter ratio is minus the stochastic integral of sampling control $\bar{b}(X_t)$ against simulated Brownian increments dW_t, minus half its time-integrated squared norm."

We accumulate both terms on the right while the path is simulated, since $\bar{b}$ is evaluated at each step anyway and the Brownian increments are the draws we made. The constant Z is unknown, so we normalize the weights over the batch with a softmax, which cancels it. Three paths with log-weights -1, 0 and 1 receive the weights

$$\frac{e^{-1}}{e^{-1}+e^{0}+e^{1}} = 0.090, \qquad 0.245, \qquad 0.665,$$

so one path in three accounts for two thirds of the gradient. When one log-weight runs far ahead of the others, the remaining paths contribute almost nothing, which is the usual cost of importance weighting and the reason $\bar{b}$ is kept close to b_θ.

This control problem uses a prescribed terminal density. It requires no second sample set, Sinkhorn scaling, endpoint coupling, or alternating projection. We learn a local drift correction that produces a terminal reweighting of the reference. For the bridge in Section 7.21, we choose the reweighting to match π_1 exactly and compute it with Sinkhorn.

On a cell perturbation task, the 1-Wasserstein distance to the perturbed population at 50 principal components was 0.342 with velocity conditioning and 1.741 without it, from five simulations of batches of 16 cells with 100 simulation steps each [Tang et al., 2026a].

7.28 Given quantities and learned quantities

In optimal transport, we look for the coupling of two laws with the smallest transport cost. In entropic transport, an entropy term of strength ε is added, and the objective becomes a relative entropy from a reference kernel. A Schrödinger bridge is the path law with the two required marginals and the smallest relative entropy from a reference process. In bridge matching, that path law is learned by regression on reference bridges drawn between endpoint pairs, where the regression target is a drift in continuous space and a rate matrix or a transition matrix over a finite set.

Table 7.6 compares the eight constructions by their inputs and learned quantities. Each starts from a reference and information about the endpoints.

Table 7.6: The eight constructions of this chapter, by what is in hand before training starts and what has to be computed or trained. The first four work between two distributions on a continuous state space. Of the last four, the first two work over a finite set, and the last two change the shape of the problem, to several weighted targets and to a system of interacting particles. Sinkhorn alone involves no learning; the other seven fit one or more networks.

method	given	learned
Sinkhorn	two marginals, a cost matrix, ε	two scaling vectors u, v
DSB	a noising reference on a fixed grid	a forward and a backward mean
DSBM	a reference with a tractable bridge	a forward and a backward drift
$[\mathrm{SF}]^2\mathrm{M}$	a Brownian reference, a transport solver	the flow velocity and the score
DDSBM	a factorized graph reference, two marginals	a rate matrix in each direction
CSBM	a categorical reference in discrete time	one-step conditionals
BranchSBM	one source, $K{+}1$ weighted targets, V_t	$K{+}1$ drifts and $K{+}1$ growth rates
EntangledSBM	a Langevin reference, $\pi_{\mathcal{B}}$	one bias force per particle

We can specify a reference through a cost matrix and temperature, a noising schedule, Brownian motion, a factorized graph kernel, a categorical chain, or molecular dynamics. Sampling its conditional bridges requires either a closed form or a computationally practical simulation.

Seven methods learn a local rule for moving probability: a mean, drift, rate matrix, one-step conditional, or force. In the eighth, we compute scaling factors with the corresponding role in closed form. The training signal is always a regression or a relative entropy against a quantity computed on a reference bridge between two endpoints, and the endpoints come either from a transport solver or from the previous round of an alternation.

7.29 Generative model summary

A generative Markov process connects a law we can sample with the data law. We specify its learned quantities and sampling rule through its dynamics. The generator describes the instantaneous rate of change in the expected value of a test function,

$$\mathcal{A}_t\phi(x) \;=\; \lim_{\Delta t\to 0} \frac{\mathbb{E}\big[\phi(X_{t+\Delta t}) \mid X_t = x\big] - \phi(x)}{\Delta t}.$$

"The generator value $\mathcal{A}_t\phi(x)$ is the expected change in test function ϕ over a step of duration Δt, conditional on current state $X_t = x$, divided by Δt, in the limit as the step duration approaches zero."

We can derive the density equation by averaging the generator definition over the law of X_t, which gives $\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}[\mathcal{A}_t\phi(X_t)]$. We then transfer the operator from ϕ to p_t, integrating by parts in continuous space or exchanging summation order in discrete space. The resulting identity holds for every test function ϕ, so it determines the density equation. Table 7.7 gives the generator for each process.

Table 7.7: The kinds of process used in this book, with the generator that defines each one and the equation its density satisfies. Here ϕ is a test function, v_t a velocity field, b_t a drift, ε a noise variance, $Q_t(x, y)$ a jump rate, and $X_{s,t}$ a flow map from time s to time t. The fourth row is the integrated form of the first.

process	generator $\mathcal{A}_t\phi$	equation for p_t
flow (ODE)	$v_t \cdot \nabla\phi$	$\partial_t p_t + \nabla \cdot (p_t v_t) = 0$
diffusion (SDE)	$b_t \cdot \nabla\phi + \frac{\varepsilon}{2}\nabla^2\phi$	$\partial_t p_t = -\nabla \cdot (p_t b_t) + \frac{\varepsilon}{2}\nabla^2 p_t$
jump (CTMC)	$\sum_{y \neq x} Q_t(x, y)\,[\phi(y) - \phi(x)]$	$\partial_t p_t(x) = \sum_y Q_t(y, x)\, p_t(y)$
flow map	$\phi \circ X_{s,t}$, with no limit taken	$p_t = (X_{s,t})_{\#}\, p_s$

The first row is the continuity equation of Chapter 2, which states that probability is conserved and carried by the velocity field. The second is the Fokker-Planck equation of Chapter 3, which adds the Laplacian term that noise contributes. The third is the master equation of Chapter 4 and Chapter 5, where a sum over edges of the state space takes the place of a divergence. In the fourth row there is no limit at all. A flow map covers the whole interval at once, so its action on a test function is composition and its action on a density is the pushforward of Chapter 1. Differentiating $\phi \circ X_{s,t}$ at $t = s$ returns $v_s \cdot \nabla\phi$, which is why the flow map is the integrated form of the first row and why the two agree on marginals.

Table 7.8 compares the learned quantities and sampling rules.

We learn local functions of the current state and time to generate whole sequences. Each is an average of a conditional quantity with a closed form: an interpolation velocity, a Gaussian score, a pinned-bridge rate, or a clean-token distribution given a corrupted token. We can therefore train by regression on these conditional quantities without simulating the process during that regression.

Squared-error, Bregman-divergence, and cross-entropy regression objectives share a conditional-average minimizer. The value c that minimizes $\mathbb{E}[(a - c)^2]$ over the randomness in a is $\mathbb{E}[a]$. For the divergence $a \log(a/c) - a + c$ built from $u \log u$, differentiating in c produces $1 - a/c$, so setting the derivative of the expectation to zero leaves $c = \mathbb{E}[a]$ again. A cross-entropy over categories is the same statement with a distribution in place of a number, where the minimizer is the conditional distribution. This identity gives the marginal rate when we regress conditional rates, as in Chapter 4 and in the Markov projections. Choosing the conditional target determines the corresponding regression loss.

The sampling rules have different computational costs. An autoregressive sampler costs one network evaluation per position and cannot revise. A diffusion or flow sampler costs one evaluation per step

Table 7.8: What each model class in this book learns and how a sample is drawn from it once the learned quantity is in hand. The clean conditional is the distribution of the data point given the current noisy state. The Monge map $T = \nabla\varphi$ with φ convex exists for the squared cost whenever the source law has a density, by Theorem 7.4.

model class	learned quantity	sampling rule
autoregression	next-token conditionals	draw the positions in a fixed order
continuous diffusion and flow	a velocity field or a score	integrate an ODE or an SDE
discrete diffusion	a rate matrix, or the clean conditional	Euler steps with the learned rates
discrete flow matching	the clean conditional, or a simplex velocity	jumps, or an ODE on the simplex
flow maps	a two-time map $X_s \mapsto X_t$	apply the map once, or a few times
optimal transport	a coupling, or a map $T = \nabla\varphi$	push the source through the map
Schrödinger bridges	a drift, a rate, or a pair of potentials	run the controlled process from 0 to 1

and can take as many steps as the budget allows. A flow map compresses those steps into a few. A bridge sampler has the same per-step cost as a flow sampler, and its saving is in the number of steps needed for a given accuracy, because the path it follows is shorter.

See *Foundations of Schrödinger Bridges for Generative Modeling* [Tang, 2026] for proofs and a fuller treatment of the static and dynamic Schrödinger formulations, proportional and Markovian fitting, and discrete-state bridges.

Its notation differs from ours in two places. There the control is written inside the noise, $dX = (f + \sigma_t u)\,dt + \sigma_t\,dB$, with reference drift f and noise scale σ_t, so the path relative entropy is one half the expected squared control. Here the convention has been $dX = (b + u)\,dt + \sqrt{\varepsilon}\,dB$, with reference drift b, so the factor in the path relative entropy is $1/(2\varepsilon)$. The two agree when our u equals σ_t times theirs and $\varepsilon = \sigma_t^2$. And there the letter Q names the reference path law, where this chapter has used R for the reference and Q for a generator.

These models combine a path, a conditional training target, and a sampling rule. The path connects a sampleable law to the data law. Given its endpoints, we can compute a conditional quantity in closed form and learn its conditional average by regression. We then integrate the learned quantity to draw samples. Flow matching uses a straight path and velocity targets. Score matching uses a noising process and score targets. The discrete constructions use jumps and rate targets. Flow maps cover an interval in one application. Optimal transport adds the choice of endpoint pairing, and Schrödinger bridges derive that pairing and its intervening paths from a reference process. Many discrete generative methods combine these same choices of path, conditional target, and sampling rule.

I hope you leave this course comfortable deriving a training objective, following how a sampler changes a distribution, and checking an implementation against its equations. Those skills will help you work through an unfamiliar model and decide how to adapt it to your own problem. The numerical examples also give you a way to check an idea before committing to a large training run.

Generative modeling is moving quickly. The methods in this book give you a starting point for reading new work and developing your own. There is still plenty to understand about how we choose paths, use conditioning, and generate useful samples efficiently.

Thank you for working through this material with me. Good luck with your projects and whatever you take on next. I look forward to seeing what you build!

Bibliography

Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*, 2023.

Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.

Marianne Arriola, Aaron Kerem Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993, 2021.

Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the Monge–Kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, 2000. doi: 10.1007/s002110050002.

Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models. *Transactions on Machine Learning Research*, 2025a. URL https://openreview.net/forum?id=cqDH0e6ak2.

Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. How to build a consistency model: Learning flow maps via self-distillation. In *Advances in Neural Information Processing Systems*, volume 38, 2025b.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems*, volume 35, pages 28266–28279, 2022.

Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5453–5512. PMLR, 2024.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31, pages 6571–6583, 2018.

Tianlai Chen, Pranay Vure, Rishab Pulugurta, and Pranam Chatterjee. AMP-Diffusion: Integrating latent diffusion with protein language models for antimicrobial peptide generation. In *Generative AI and Biology (GenBio) Workshop at NeurIPS 2023*, 2023. URL `https://openreview.net/forum?id=145TM9VQhx`.

Tong Chen, Yinuo Zhang, Sophia Tang, and Pranam Chatterjee. Multi-objective-guided discrete flow matching for controllable biological sequence design. In *2nd Generative AI for Biology Workshop at ICML 2025*, 2025.

Tong Chen, Maximilian Holsman, Lin Zhao, and Pranam Chatterjee. pCoMole: Pareto-constrained molecule editing with discrete flows. In *Advances in Neural Information Processing Systems*, 2026a. To appear.

Tong Chen, Yinuo Zhang, and Pranam Chatterjee. AReUReDi: Annealed rectified updates for refining discrete flows with multi-objective guidance. In *Learning Meaningful Representations of Life (LMRL) Workshop at ICLR 2026*, 2026b.

Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013.

Oscar Davis, Samuel Kessler, Mircea Petrache, İsmail İlkan Ceylan, Michael Bronstein, and Avishek Joey Bose. Fisher flow matching for generative modeling over discrete data. In *Advances in Neural Information Processing Systems*, volume 37, pages 139054–139084, 2024.

Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion Schrödinger bridge with applications to score-based generative modeling. In *Advances in Neural Information Processing Systems*, volume 34, pages 17695–17709, 2021.

Justin Deschenaux, Alexandre Galashov, Andrew Campbell, Li Kevin Wenliang, James Thornton, Arnaud Doucet, and Valentin De Bortoli. Simplex diffusion models. arXiv:2609.35553 [cs.LG], 2026.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: `10.18653/v1/N19-1423`.

Prafulla Dhariwal and Alex Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794, 2021.

J. L. Doob. Conditional Brownian motion and the boundary limits of harmonic functions. *Bulletin de la Société Mathématique de France*, 85:431–458, 1957. doi: 10.24033/bsmf.1494.

Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *International Conference on Learning Representations*, 2025.

Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Advances in Neural Information Processing Systems*, volume 37, pages 133345–133385, 2024.

Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean Flows for one-step generative modeling. In *Advances in Neural Information Processing Systems*, volume 38, pages 75460–75482, 2025.

Daniel T. Gillespie. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of Computational Physics*, 22(4):403–434, 1976. doi: 10.1016/0021-9991(76)90041-3.

Marton Havasi, Brian Karrer, Itai Gat, and Ricky T. Q. Chen. Edit Flows: Variable length discrete flow matching with sequence-level edit operations. In *Advances in Neural Information Processing Systems*, volume 38, pages 139037–139070, 2025.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Peter Holderrieth, Douglas Chen, Luca Eyring, Ishin Shah, Giri Anantharaman, Yutong He, Zeynep Akata, Tommi Jaakkola, Nicholas Matthew Boffi, and Max Simchowitz. Diamond Maps: Efficient reward alignment via stochastic flow maps. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 43401–43435. PMLR, 2026.

Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

E. T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106(4):620–630, 1957. doi: 10.1103/PhysRev.106.620.

Leonid V. Kantorovich. On the translocation of masses. *Doklady Akademii Nauk SSSR*, 37(7–8): 227–229, 1942. English translations: Management Science 5(1):1–4, 1958 (doi:10.1287/mnsc.5.1.1); Journal of Mathematical Sciences 133(4):1381–1382, 2006 (doi:10.1007/s10958-006-0049-2).

Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency Trajectory Models: Learning probability flow ODE trajectory of diffusion. In *International Conference on Learning Representations*, 2024.

Jun Hyeong Kim, Seonghwan Kim, Seokhyun Moon, Hyeongwoo Kim, Jeheon Woo, and Woo Youn

Kim. Discrete diffusion Schrödinger bridge matching for graph transformation. In *International Conference on Learning Representations*, 2025.

Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

Grigoriy Ksenofontov and Alexander Korotin. Categorical Schrödinger bridge matching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 31727–31751. PMLR, 2025.

S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. doi: 10.1214/aoms/1177729694.

Chanhyuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M. Boffi, and Jinwoo Kim. Flow Map Language Models: One-step language modeling via continuous denoising. In *Advances in Neural Information Processing Systems*, 2026. To appear.

Christian Léonard. A survey of the Schrödinger problem and some of its connections with optimal transport. *Discrete and Continuous Dynamical Systems*, 34(4):1533–1574, 2014. doi: 10.3934/dcds.2014.34.1533.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32819–32848. PMLR, 2024.

Sam McCallum, Zander W. Blasingame, Timothy Herschell, Niklas Rindtorff, Alexander Tong, and James Foster. Strong Stochastic Flow Maps. In *Advances in Neural Information Processing Systems*, 2026. To appear.

Robert J. McCann. A convexity principle for interacting gases. *Advances in Mathematics*, 128(1): 153–179, 1997. doi: 10.1006/aima.1997.1634.

Gaspard Monge. Mémoire sur la théorie des déblais et des remblais. *Histoire de l'Académie Royale des Sciences de Paris, avec les Mémoires de Mathématique et de Physique pour la même année*, pages 666–704, 1781.

Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance

for discrete state-space diffusion and flow models. In *International Conference on Learning Representations*, 2025.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.

Fred Zhangzhi Peng, Zachary Bezemek, Sawan Patel, Jarrid Rector-Brooks, Sherwood Yao, Avishek Joey Bose, Alexander Tong, and Pranam Chatterjee. Path planning for diffusion language model sampling. *Transactions on Machine Learning Research*, 2026a.

Fred Zhangzhi Peng, Zachary Bezemek, Jarrid Rector-Brooks, Shuibai Zhang, Anru R. Zhang, Michael Bronstein, Alexander Tong, and Avishek Joey Bose. Planner aware path learning in diffusion language models training. In *International Conference on Learning Representations*, 2026b.

Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073.

Peter Potaptchik, Adhi Saravanan, Abbas Mammadov, Alvaro Prat, Michael Albergo, and Yee Whye Teh. Meta Flow Maps enable scalable reward alignment. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 99255–99295. PMLR, 2026a.

Peter Potaptchik, Jason Yim, Adhi Saravanan, Peter Holderrieth, Eric Vanden-Eijnden, and Michael S. Albergo. Discrete Flow Maps. In *Advances in Neural Information Processing Systems*, 2026b. To appear.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.

Daan Roos, Oscar Davis, Floor Eijkelboom, Michael Bronstein, Max Welling, İsmail İlkan Ceylan, Luca Ambrogioni, and Jan-Willem van de Meent. Categorical Flow Maps. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 105618–105642. PMLR, 2026.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 130136–130184, 2024.

Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dalla-Torre, Bernardo P. de Almeida, Alexander Rush, Thomas Pierrot, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In *International Conference on Learning Representations*, 2025.

Erwin Schrödinger. Über die Umkehrung der Naturgesetze. *Sitzungsberichte der Preussischen Akademie der Wissenschaften, Physikalisch-Mathematische Klasse*, 8(9):144–153, 1931.

Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x. Continued in vol. 27, no. 4, pp. 623–656.

Neta Shaul, Itai Gat, Marton Havasi, Daniel Severo, Anuroop Sriram, Peter Holderrieth, Brian Karrer, Yaron Lipman, and Ricky T. Q. Chen. Flow matching with general discrete paths: A kinetic-optimal perspective. In *International Conference on Learning Representations*, 2025.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems*, volume 37, pages 103131–103167, 2024.

Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion Schrödinger bridge matching. In *Advances in Neural Information Processing Systems*, volume 36, pages 62183–62223, 2023.

Richard Sinkhorn. A relationship between arbitrary positive matrices and doubly stochastic matrices. *The Annals of Mathematical Statistics*, 35(2):876–879, 1964. doi: 10.1214/aoms/1177703591.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2256–2265. PMLR, 2015.

Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, volume 32, pages 11895–11907, 2019.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency Models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32211–32252. PMLR, 2023.

Hannes Stark, Bowen Jing, Chenyu Wang, Gabriele Corso, Bonnie Berger, Regina Barzilay, and Tommi Jaakkola. Dirichlet flow matching with applications to DNA sequence design. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 46495–46513. PMLR, 2024.

Sophia Tang. Foundations of Schrödinger bridges for generative modeling. arXiv:2603.18992 [cs.LG], 2026.

Sophia Tang and Pranam Chatterjee. Expanding Flow Maps. In *Advances in Neural Information Processing Systems*, 2026. To appear.

Sophia Tang, Yinuo Zhang, and Pranam Chatterjee. PepTune: De novo generation of therapeutic

peptides with multi-objective-guided discrete diffusion. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 59017–59065. PMLR, 2025a.

Sophia Tang, Yinuo Zhang, Alexander Tong, and Pranam Chatterjee. Gumbel-Softmax flow matching with straight-through guidance for controllable biological sequence generation. In *ICLR 2025 Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficacy*, 2025b.

Sophia Tang, Yuchen Zhu, Molei Tao, and Pranam Chatterjee. TR2-D2: Tree search guided trajectory-aware fine-tuning for discrete diffusion. arXiv:2509.25171 [cs.LG], 2025c.

Sophia Tang, Yinuo Zhang, and Pranam Chatterjee. Entangled Schrödinger bridge matching. In *Advances in Neural Information Processing Systems*, 2026a. To appear.

Sophia Tang, Yinuo Zhang, Alexander Tong, and Pranam Chatterjee. Branched Schrödinger bridge matching. In *International Conference on Learning Representations*, 2026b.

Sophia Tang, Yuchen Zhu, Molei Tao, and Pranam Chatterjee. A2D2: Fine-tuning any-length discrete diffusion for adaptive decoding. arXiv:2606.13565 [cs.LG], 2026c.

Alexander Tong, Nikolay Malkin, Kilian Fatras, Lazar Atanackovic, Yanlei Zhang, Guillaume Huguet, Guy Wolf, and Yoshua Bengio. Simulation-free Schrödinger bridges via score and flow matching. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 1279–1287. PMLR, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

Cédric Villani. *Optimal Transport: Old and New*, volume 338 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 2009. doi: 10.1007/978-3-540-71050-9.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/neco_a_00142.

Jaehoon Yoo, Wonjung Kim, and Seunghoon Hong. ReDi: Rectified discrete flow. In *Advances in Neural Information Processing Systems*, volume 38, pages 81651–81683, 2025.